# Programming Languages From the Ground Up

Memory, Types, Ownership, and Concurrency — From Java 6 and Modern C++ Through Rust, OCaml, Haskell, TypeScript, Kotlin, and Go to a Production Financial Backend

A Systems and Type-Theory Path into Modern Software Engineering

June 2026

# Preface

Most books about programming languages fall into one of two camps, and neither one quite helps a working engineer make the jump from an older object-oriented language to modern systems and type-driven software.

The first camp is the syntax tour: "here is how you write a loop in Rust, here is how you declare a class in Kotlin, here is how you spawn a goroutine in Go." After fifty pages you can transcribe small programs, yet the first time the Rust borrow checker rejects a perfectly ordinary listener callback, or TypeScript narrows a union type in a way Java generics cannot express, or a Go interface holding a `nil` pointer evaluates `err != nil` to `true`, you hit a wall. Syntax was never the hard part. The mental model underneath the syntax was.

The second camp is pure programming-language theory: operational semantics, lambda calculus reductions, and inference rules stacked like fractions before you ever inspect a single byte in memory or send a packet over a socket. Theory matters — algebraic data types, parametric polymorphism, variance, and linear ownership are clean mathematical ideas — yet studying those ideas detached from stack frames, CPU cache lines, vtable pointers, and real production failure modes strips away the engineering pressure that forced language designers to invent them.

This book works in both layers at once. It keeps one eye on the hardware — exact stack frames, 64-byte cache lines, object headers, fat pointers, state-machine layouts — and the other on the type system and mathematical structure that let a compiler rule out whole categories of bugs before an executable exists. The running example throughout is a concrete production system: a high-assurance financial ledger backend in Rust with Axum, SQLx, and PostgreSQL.

## Who this book is for, and where we start

We assume a very specific, realistic starting point:

- **You know Java at roughly the Java 6 level.** You are comfortable with classes, interfaces, inheritance (`extends`, `implements`), `ArrayList` and `HashMap` generics, `null`, checked and unchecked `try`/`catch`/`finally` exceptions, `synchronized` blocks, OS threads, and garbage collection. You do *not* need Java 8 streams, records, or sealed classes — when modern JVM ideas appear, we build them cleanly from Java 6 upward.
- **You know basic C and basic C++.** You have written `for` loops, `if`/`else` branches, functions, `struct` declarations, arrays, `malloc`/`free`, and raw pointers (`T*` and `&x`). You do *not* need modern C++11–C++20 move semantics, smart pointers, templates, or RAII — Chapter 2 builds all of those from scratch.

Because you already know how loops, conditionals, and functions work, we spend zero pages re-explaining beginner control flow. Every page goes toward the questions that actually rewire how you read and write software:

- When you write a variable declaration in Java 6, C++, Rust, Go, Kotlin, or TypeScript, *what exact bytes sit on the stack, what sits on the heap, how many pointer dereferences stand between the CPU and your integer, and who frees those bytes when?*
- Why did Rust reject class inheritance, `null`, exceptions, and garbage collection — and how do *ownership*, *borrowing* (`&T` versus `&mut T`), *lifetimes* (`'a`), *algebraic data types* (`struct` and `enum`), and *traits* fit together into a single coherent lens?
- How do functional programmers in **OCaml** and **Haskell** actually think? Once you strip away academic jargon, how do immutability, algebraic variants, pattern matching, higher-order pipelines, typeclasses, functors, and monads show up directly inside everyday idiomatic Rust?
- How do **TypeScript** (structural set-theoretic types erased at runtime), **Kotlin** (null-safe algebraic hierarchies and compiler-rewritten CPS coroutines on the JVM), and **Go** (flat value structs, escape analysis, implicit fat-pointer interfaces, and an M:N goroutine scheduler) each solve the exact pain points of Java 6 with radically different engineering tradeoffs?
- How do all of these low-level and type-level tools pay off when real money is on the line — building an asynchronous **Rust + Axum + SQLx + PostgreSQL** double-entry financial ledger where negative transfers, currency mismatches, lost concurrent updates, unbalanced journal entries, and unrolled-back partial failures are prevented by construction?

## Why a financial ledger is our north star

Toy examples flatter weak abstractions. Almost any language looks pleasant when you are printing `Animal.speak()` for a `Dog` and a `Cat`. Financial systems are unforgiving in a much more instructive way:

1. **Representation precision matters.** Using IEEE-754 binary floating-point (`double` / `f64`) for money produces $0.10 + 0.20 = 0.30000000000000004$. Silent integer overflow turns a large credit into a large negative balance. Mixing USD cents with EUR cents or swapping an `AccountId` with a `JournalId` corrupts books silently unless types keep them distinct at zero runtime cost.
2. **Illegal states must be unrepresentable.** In a Java 6 `PaymentOrder` class with eight nullable fields and three boolean flags (`isVerified`, `isSettled`, `isFailed`), $2^3 = 8$ flag combinations exist even when only three make business sense, and every method starts with defensive `if (order.getSettledAt() != null)` checks. Sum types (`enum`s) and typestate collapse invalid combinations at compile time.
3. **Concurrency and partial failure are adversarial.** If two HTTP requests simultaneously withdraw $80 from an account holding $100, or if a client drops its TCP connection halfway through an `async` transfer handler, what happens to the lock, the memory, and the uncommitted PostgreSQL transaction? Answering that requires understanding stackless `Future` state machines, `Drop`/RAII guards, and database isolation together.

Even in Parts I through IV, before we reach the full service in Part V, you will meet orders, accounts, minor-unit money amounts, and ledger postings alongside systems classics like buffers, parsers, and schedulers, so that by Chapter 12 the capstone uses only ideas you have already met.

## How to read this book, and what the boxes mean

Every concept in this book follows a steady four-step rhythm: *the concrete bug or hardware cost first*, *the physical byte-level picture second*, *the language or type-system rule third*, and *idiomatic code side by side across languages fourth*.

Throughout the chapters, recurring colored boxes tell you what kind of thinking a passage asks for:

**Intuition**

The core mental picture — the physical or geometric way of seeing a rule so you never have to memorize it by rote. If you revisit a chapter months later, reading the intuition boxes reconstructs the whole arc.

**Focus**

A central invariant or guiding principle that organizes an entire section or language design choice — for instance, Rust's *Aliasing XOR Mutability* rule or TypeScript's *types as sets of values*.

**Formal**

The exact rule, memory layout, or type relation stated without hand-waving — byte offsets, alignment rules, subtyping judgments, or compiler desugarings. When intuition and edge cases collide, this box gives the ground truth.

**Why it works**

The engineering and mathematical reason *why* a rule is designed the way it is — why C++ move constructors cannot simply skip the moved-from destructor, or why mutable references in Rust and mutable generic containers in Java/TypeScript must be invariant.

**Common trap**

A classic footgun that catches smart engineers moving from Java 6 or C/C++ into modern languages — shown with the exact code that triggers it and the mechanism behind the failure.

**Work it yourself**

A concrete trace or micro-experiment to work through on paper or at a terminal: counting cache misses, writing down a compiler-generated closure struct, or hand-stepping an `async` state machine across `.await` points.

**In production**

How the mechanism behaves in real production systems: tail-latency spikes in trading and payment engines, database connection pools, serialization boundaries, and network servers.

You will also occasionally see *History and connection* boxes (tracing how ideas traveled from ML, Haskell, and C++ into Rust, Kotlin, TypeScript, and Go), *Under the hood* boxes (peeking at assembly, JVM bytecode, or desugared compiler output), *Walking question* boxes (a puzzle to turn over in your head), and *Pause* boxes (checkpoints summarizing what you can now do).

## A ten-minute preview: one transfer, three mental models

To taste the difference between reading syntax and reading through the machine-and-types lens, look at the same miniature operation — debiting an account balance — written in Java 6, basic C, and Rust.

First, a familiar Java 6 method:

```
// Java 6: heap reference, implicit null, exception control flow
public class Account {
    private Long balanceCents; // boxed heap object!

    public void debit(Long amountCents) throws InsufficientFundsException {
        if (amountCents <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
        if (this.balanceCents < amountCents) {
            throw new InsufficientFundsException(this.balanceCents, amountCents);
        }
        this.balanceCents = this.balanceCents - amountCents;
    }
}
```

Look past the clean surface and ask what the JVM actually executes, and what the type signature hides:

1. **Hidden `null`s.** Both `balanceCents` and `amountCents` are boxed `java.lang.Long` references. If a caller passes `null`, the check `amountCents <= 0` calls `amountCents.longValue()` under the hood and throws `NullPointerException` — an exit path mentioned nowhere in the signature.

2. **Pointer chasing and allocation.** Reading `this.balanceCents` follows a pointer to the `Account` object (with its 16-byte JVM object header), then follows *another* pointer to a separate heap-allocated `Long` object (another 16-byte header plus 8 payload bytes = 24 bytes just to hold an 8-byte integer). Subtracting `amountCents` allocates a brand-new `Long` object on the heap unless the result happens to fall in `Long.valueOf`'s tiny $[-128, 127]$ cache.

3. **Unrestricted aliasing.** Any number of threads, lists, and caches may hold references to this same `Account` simultaneously while `debit` mutates it in place. Nothing in the type tells you whether you hold the only handle or one of fifty.

Now swing to the opposite extreme in C:

```c
/* C: flat bytes in memory, zero GC, zero compile-time guardrails */
#include <stdint.h>

typedef struct {
    uint64_t id;
    int64_t  balance_cents;
} Account;

int account_debit(Account *acct, int64_t amount_cents) {
    if (acct == NULL || amount_cents <= 0) return -1;
    if (acct->balance_cents < amount_cents) return -2;
    acct->balance_cents -= amount_cents;
    return 0;
}
```

The hardware picture is now plain: `Account` is a flat 16-byte value (`id` at offset 0, `balance_cents` at offset 8), with zero object headers and zero heap allocations during subtraction. Yet notice what we gave up to get that flatness:

- `acct` might still be `NULL`, or worse, a *dangling pointer* to a stack frame that already returned or a heap block already passed to `free()`.
- The error codes `-1` and `-2` sit in an ordinary `int`. A caller who writes `account_debit(&a, 5000);` and forgets to check the return value continues executing as if the debit succeeded.
- Nothing stops two pointers from aliasing the same `Account` across threads or across function arguments while mutating it.

Now look at the idiomatic Rust version we will be able to read down to the register level by the end of Part II:

```rust
// Rust: flat C-like memory layout + compile-time ownership & algebraic types
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct PositiveCents(i64); // invariant: always > 0

#[derive(Debug, PartialEq, Eq)]
pub enum DebitError {
    InsufficientFunds { available: i64, requested: i64 },
    ArithmeticOverflow,
}

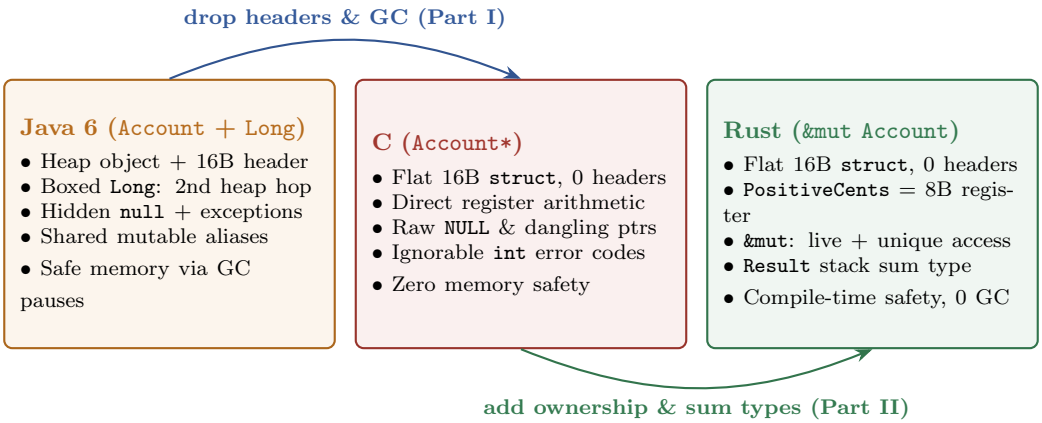
pub struct Account {
    pub id: u64,
    balance_cents: i64,
}

impl Account {
    pub fn debit(&mut self, amount: PositiveCents) -> Result<(), DebitError> {
        if self.balance_cents < amount.0 {
            return Err(DebitError::InsufficientFunds {
                available: self.balance_cents,
                requested: amount.0,
            });
        }
```

```
        self.balance_cents = self.balance_cents
            .checked_sub(amount.0)
            .ok_or(DebitError::ArithmeticOverflow)?;
        Ok(())
    }
}
```

Pause and count how many runtime checks and heap allocations disappeared *at the same time*:

- **Exact C memory layout.** `Account` occupies the exact same 16 flat bytes as the C `struct`, and `PositiveCents(i64)` compiles down to a bare 64-bit register value with zero wrapper header.
- **No `null`, no dangling pointer, no aliasing race.** The parameter `&mut self` is a compile-time proof that `self` points to a live, properly aligned `Account` *and* that no other active reference anywhere in the program can read or write this `Account` for the duration of `debit`.
- **Invariants baked into types.** Because `PositiveCents`'s inner `i64` is private to its module and can only be constructed through a smart constructor that checks `val > 0` once at the boundary, `debit` never even needs an `amount <= 0` check.
- **Errors as flat values.** `Result<(), DebitError>` is a stack-allocated tagged union, not a heap-allocated exception unwinding stack frames. And if a caller ignores a `Result`, the Rust compiler immediately warns that a `#[must_use]` failure value was dropped.




Seeing *both* layers at once — the flat 16 bytes on the machine and the mathematical guarantees in the type signature — is what we mean by "from the ground up."

## Roadmap of the five parts

The fourteen chapters of this book are grouped into five progressive parts:

**Part I — Leaving Java 6: Memory, Values, and the Machine (Chapters 1–2).** Chapter 1 strips away the JVM illusion and inspects what Java 6 actually puts in RAM: stack frames versus heap objects, 16-byte object headers, pointer chasing across 64-byte CPU cache lines, stop-the-world GC pauses, aliasing bugs, `null`, and exception control flow, contrasted directly against

flat value arrays in C, C++, and Rust. Chapter 2 builds **Modern C++ (C++11–C++20)** from basic C/C++: deterministic RAII destructors, lvalue/rvalue references, copy versus move semantics (`std::move` and why C++ moves are *non-destructive*), `unique_ptr`/`shared_ptr`/`string_view`/`span`, monomorphized templates with C++20 Concepts, lambdas, and the exact four C++ footguns (dangling views, iterator invalidation on `vector` reallocation, use-after-move, and data races) that Rust was invented to eliminate at compile time.

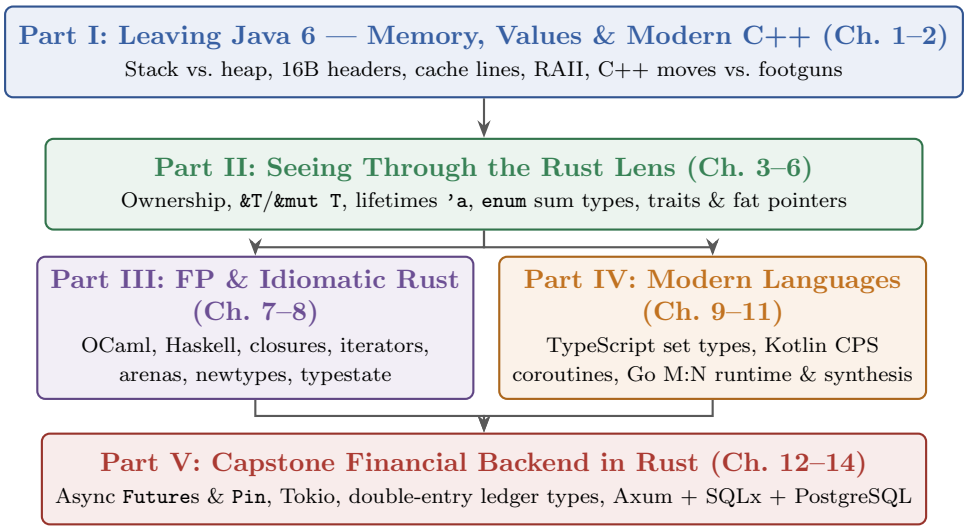
**Part II — Seeing Through the Rust Lens (Chapters 3–6).** Chapter 3 derives Rust's single foundational invariant — *Aliasing XOR Mutability* — and maps out the exact stack and heap byte layouts of `String`, `&str`, `Vec<T>`, `&[T]`, `Box<T>`, `Rc<T>`, `Arc<T>`, `Cell<T>`, `RefCell<T>`, and `Mutex<T>`. Chapter 4 removes all mystery from lifetimes (`'a`), Non-Lexical Lifetimes, elision, `'static`, variance, and the `unsafe` proof boundary. Chapter 5 develops algebraic data types — product types (`struct`) and sum types (`enum`), niche layout optimizations where `Option<&T>` occupies just 8 bytes, `Option`/`Result`/`?`, and exhaustive pattern matching. Chapter 6 opens up traits, generics, static dispatch via monomorphization versus dynamic dispatch via 16-byte `(data_ptr, vtable_ptr)` fat pointers, coherence rules, and standard library traits.

**Part III — Thinking Functionally and Writing Idiomatic Rust (Chapters 7–8).** Chapter 7 rewires imperative loop-and-flag habits by visiting Rust's direct functional ancestors: **OCaml** (immutability by default, expression-oriented control flow, Hindley–Milner type inference, algebraic variants, persistent structural sharing, higher-order pipelines) and **Haskell** (purity, laziness without thunks in Rust iterators, Typeclasses as the direct parent of traits, Functors as `.map()`, and Monads as `.and_then()` and `?`), ending with how Rust compiles closures (`Fn`, `FnMut`, `FnOnce`) and iterator chains into single zero-allocation machine loops. Chapter 8 shows how experienced engineers actually architect Rust systems: curing "Java-in-Rust" cyclic OOP graphs with flat `Vec` arenas and typed index handles, *Parse, Don't Validate*, newtypes, builders, zero-byte compile-time typestate (`PhantomData`), borrowing views at API boundaries, and `thiserror`/`anyhow` error design.

**Part IV — Modern Type Systems and Concurrency: TypeScript, Kotlin, and Go (Chapters 9–11).** Once you hold both the hardware lens and the Rust/FP lens, other modern languages snap into focus rapidly. Chapter 9 builds **TypeScript** from set theory and structural subtyping: `never` ($\emptyset$), `unknown` ($U$), unions ($A \cup B$), intersections ($A \cap B$), discriminated unions, control-flow narrowing, mapped and conditional types (`keyof`, `infer`), branded types, and runtime validation (`Zod`) across the erase-to-JS boundary. Chapter 10 bridges Java 6 directly to **Kotlin** down to JVM bytecode: nullability in the type system, `@JvmInline value class`, `sealed interface`, `inline`/`reified` generics defeating JVM type erasure, and how `suspend` functions compile via Continuation-Passing Style (CPS) into state machines under structured concurrency. Chapter 11 dissects **Go** (value structs, escape analysis, fat-pointer interfaces and the typed-`nil` trap, and the M:N `G-M-P` goroutine scheduler with channels) and closes with a seven-axis architectural synthesis across Java 6, Modern C++, Rust, OCaml, Haskell, TypeScript, Kotlin, and Go.

**Part V — Capstone: A Financial Backend in Rust with Axum, SQLx, and PostgreSQL (Chapters 12–14).** Chapter 12 goes under the hood of **Async Rust and Tokio**: how `async fn` compiles into a stackless `enum` state machine, why self-referential borrows across `.await` require `Pin<&mut Self>`, work-stealing scheduling, `Send`/`'static` bounds, backpressure, and cancellation safety when clients disconnect mid-request. Chapter 13 models **money and double-entry ledgers** in types: minor-unit integers and fixed-point decimals, checked arithmetic, banker's rounding with remainder allocation, currency safety, conservation of debits and credits ($\sum \text{debits} = \sum \text{credits}$), and typestate proof tokens (`BalancedEntry`). Chapter 14 assembles the complete **Axum + Tower + SQLx + PostgreSQL** financial service: production DDL, compile-time SQL type-checking

(`sqlx::query!`), RAII transaction rollback on `Drop`, defeating concurrent overdraft races via canonically ordered `SELECT ... FOR UPDATE` and Stripe-style idempotency keys, transactional outbox delivery, graceful shutdown, and integration tests.




**Connection**

When Chapter 14 locks rows in PostgreSQL, evaluates isolation levels, and stores idempotency keys and double-entry postings, it meets the companion volume *Databases and PostgreSQL From the Ground Up* head-on. Here we examine database interactions from the application language and type-system side; the database reader opens up PostgreSQL's 8 KB heap pages, MVCC tuple headers, write-ahead log, and query planner underneath those exact SQL statements.

## How to get the most out of every chapter

Do not read passively. Whenever a diagram shows a 24-byte `Vec<T>` header on the stack pointing to a heap buffer, grab a scrap of paper and trace what happens to every word when the vector reallocates or moves into another function. Whenever a chapter contrasts Java 6, C++, and Rust, ask yourself *which bugs failed at runtime in the first program, which bugs became undefined behavior in the second, and which exact rule rejected them at compile time in the third.*

Every chapter closes with exercises that ask you to calculate byte sizes and alignments by hand, spot subtle aliasing or concurrency bugs, predict compiler errors before running a compiler, and refactor brittle imperative designs into clean type-checked state machines. Working through them is where fluency sticks. To set that habit right away, try these four short warm-ups using only what we just previewed.

## Exercises

**Exercise 0.1** (Counting hidden exit paths in Java 6)**.** Go back to the ten-line Java 6 `Account.debit` method in this preface. List every distinct way a call `acct.debit(amt)` can terminate *without*

returning normally — including what happens if `acct` itself is `null`, if `amt` is `null`, or if `balanceCents` was left `null` by an incomplete constructor. Which of those exit paths are visible in the method's type signature?

**Exercise 0.2** (Silent overflow with signed 64-bit cents)**.** Suppose the Java 6 `Account` class used primitive `long` instead of `Long`, and a separate `credit(long amount)` method updated `balanceCents += amount` without a range check. In Java (and two's-complement hardware), what happens if `balanceCents` equals `Long.MAX_VALUE` (9,223,372,036,854,775,807) and `credit(100)` executes? Contrast this silent wraparound with `checked_add` returning `Option<i64>` in Rust.

**Exercise 0.3** (The self-transfer aliasing bug)**.** Consider this short C transfer function that reads both balances before writing:

```
void transfer(Account *from, Account *to, int64_t cents) {
    int64_t f = from->balance_cents;
    int64_t t = to->balance_cents;
    if (f >= cents) {
        from->balance_cents = f - cents;
        to->balance_cents   = t + cents;
    }
}
```

Trace line by line what happens when a caller invokes `transfer(&alice, &alice, 5000)` with `alice`'s starting balance equal to `10000`. What is `alice`'s final balance, and where did the extra $50.00 come from? Why does a Rust signature taking `from: &mut Account` and `to: &mut Account` reject `transfer(&mut alice, &mut alice, ...)` at compile time?

**Exercise 0.4** (Floating-point money on a napkin)**.** In IEEE-754 double-precision binary floating point (`double` in Java/C, `f64` in Rust), the decimal fraction $0.1 = \frac{1}{10}$ has the infinite repeating binary expansion $0.0001100110011\ldots_2$, just as $\frac{1}{3} = 0.3333\ldots_{10}$ repeats forever in base 10. Suppose a payment service adds ten thousand micro-fees of $0.01 stored as `double` and then compares `total == 100.00` with `==`. Explain in two sentences why binary floating point rounds $\frac{1}{100}$, and why storing integer minor units (`1` cent = `1_i64`) makes addition exact.

# Contents

**Preface** iii

**I Leaving Java 6: Memory, Values, and the Machine** 1

**1 Leaving Java 6: Memory, Values, and the Machine** 3

1.1 The Two Arenas of a Running Process: Stack Frames vs. the Heap . . . 3

1.1.1 How the hardware stack works in two CPU instructions . . . 4

1.1.2 Why the heap exists, and what you pay for it . . . 5

1.2 Inside a Java 6 Object: Headers, Alignment, and Pointer Chasing . . . 7

1.2.1 The 16-byte HotSpot object header . . . 7

1.3 The Memory Hierarchy: Cache Lines, Prefetchers, and Stop-the-World GC . . . 10

1.3.1 64-byte cache lines and the hardware prefetcher . . . 11

1.3.2 How Java 6 Garbage Collection actually works — and why Old Gen hurts tail latency . . . 12

1.4 Value Semantics vs. Reference Semantics . . . 13

1.4.1 Clearing up "Java passes objects by reference" . . . 14

1.5 Aliasing + Mutation: Why Shared Mutable State Breaks Local Reasoning . . . 15

1.5.1 Three single-threaded aliasing bugs . . . 15

1.5.2 Multi-threaded shared mutability and Java 6's detached locks . . . 17

1.6 Dishonest Signatures: `null`, Exceptions, and Hidden Control Flow . . . 18

1.6.1 Crack 1: Tony Hoare's billion-dollar `null` . . . 18

1.6.2 Crack 2: Exceptions as invisible `goto`s across stack frames . . . 19

1.6.3 Preview: Honest signatures with flat value types . . . 19

1.7 Exercises . . . 20

**2 Modern C++: RAII, Moves, Smart Pointers, and Templates** 25

2.1 From `malloc`/`free` and `finalize()` to Deterministic RAII . . . 25
2.1.1 Why manual C cleanup and Java 6 `finalize()` both fall short . . . 26
2.1.2 Bjarne Stroustrup's insight: tie every resource to a stack object's lifetime . . . 27
2.2 References, Value Categories, Copies, and Move Semantics . . . 29
2.2.1 Lvalue references: `T&` and `const T&` . . . 29
2.2.2 Lvalues vs. rvalues, and why C++98 wasted gigabytes copying temporaries . . . 30
2.2.3 C++11 Move Semantics: `T&&` and `std::move` . . . 31
2.2.4 The Central Contrast: C++ Non-Destructive Moves vs. Rust Destructive Moves 33
2.2.5 The Rule of Five vs. the Rule of Zero . . . 34
2.3 Smart Pointers and Non-Owning Views . . . 34
2.3.1 `std::unique_ptr<T>`: Zero-overhead exclusive heap ownership . . . 35
2.3.2 `std::shared_ptr<T>` and `std::weak_ptr<T>`: Reference-counted shared ownership . . . 35
2.3.3 Non-owning fat-pointer views: `std::string_view` and `std::span<T>` . . . 36
2.4 Zero-Cost Generics, `constexpr`, C++20 Concepts, and Lambdas . . . 37
2.4.1 Templates via monomorphization vs. Java 6 type erasure . . . 37
2.4.2 `constexpr` and C++20 Concepts: taming template chaos . . . 38
2.4.3 Lambdas under the hood: zero-allocation closure structs . . . 39
2.5 The Four C++ Footguns That Forced the Invention of Rust . . . 40
2.5.1 Footgun 1: Dangling `string_view` and `span` into destroyed temporaries . . . 40
2.5.2 Footgun 2: Single-threaded reference/iterator invalidation on `std::vector` growth . . . 41
2.5.3 Footgun 3: Silent use-after-move . . . 42
2.5.4 Footgun 4: Dangling `[&]` lambda captures and unchecked data races . . . 42
2.6 Exercises . . . 43

**II Seeing Through the Rust Lens 47**

**3 Ownership, Moves, and Borrowing: Seeing Through the Rust Lens 49**

3.1 The Single Root Bug Behind Two Different Worlds . . . 49
3.2 Ownership and Exact Byte Layouts on Stack and Heap . . . 52
3.2.1 `String` (24 Bytes Owned) versus `&str` (16-Byte Fat Slice) . . . 53
3.2.2 `Vec<T>` versus `&[T]`, and `Box<T>` . . . 54

3.3 Bitwise Copy, Destructive Move, and Explicit Clone . . . 55
3.3.1 When Is a Type Allowed to Be `Copy`? And When Do We `.clone()`? . . . 57
3.4 Borrowing, Reborrowing, and Splitting Slices . . . 58
3.4.1 Implicit Reborrowing: Why Doesn't Passing `&mut T` Move It? . . . 59
3.4.2 Disjoint Struct Fields and Splitting Slices (`split_at_mut`) . . . 60
3.5 Shared Ownership: `Rc<T>` versus `Arc<T>` . . . 62
3.6 Controlled Interior Mutability: `Cell`, `RefCell`, and `Mutex` . . . 63
3.6.1 1. `Cell<T>`: Zero-Byte-Overhead Value Replacement (Single-Threaded) . . . 64
3.6.2 2. `RefCell<T>`: Single-Threaded Runtime Borrow Checking . . . 64
3.6.3 3. `Mutex<T>` and `RwLock<T>`: Putting the Data *Inside* the Lock . . . 65
3.6.4 Putting the Entire Pointer & Ownership Toolkit Together . . . 67
3.7 Exercises . . . 68

**4 Lifetimes Without Mysticism: Regions, Elision, and the Unsafe Boundary 71**
4.1 Demystifying `'a` From the Machine Up . . . 71
4.1.1 Lexical Scopes vs. Non-Lexical Lifetimes (NLL) . . . 73
4.2 Functions, Borrow Signatures, and the Three Elision Rules . . . 74
4.2.1 Distinct Lifetimes, Outlives Bounds, and Callee Stack Frames . . . 75
4.2.2 Deriving the Three Lifetime Elision Rules . . . 76
4.3 Structs Holding References: Zero-Copy Views vs. Owned Data . . . 76
4.3.1 Architectural Guidance: Avoiding "Lifetime Infection" and Self-References . . . 78
4.4 Busting the #1 Beginner Trap: `&'static str` vs. `T: 'static` . . . 79
4.5 Subtyping and Variance: Why `&'a mut T` Is Invariant in `T` . . . 80
4.6 The Unsafe Boundary: Raw Pointers, `UnsafeCell`, and Sound Abstractions . . . 82
4.6.1 Case Study: Implementing `split_two_mut` Behind a Safe Signature . . . 82
4.6.2 `UnsafeCell<T>`, Aliasing UB, `MaybeUninit<T>`, and Miri . . . 83
4.7 Exercises . . . 85

**5 Algebraic Data Types: Making Illegal States Unrepresentable 87**
5.1 Why Rust Dropped `extends` and `null`: The Algebra of Cardinality . . . 87
5.1.1 Product Types: `struct` Multiplies Possibilities . . . 88
5.1.2 Sum Types: `enum` Adds Possibilities . . . 89
5.1.3 Why Java 6 Subclasses and C `unions` Fall Short . . . 90

5.2 Exact Byte Layout: Struct Reordering, Tagged Unions, and Niche Optimization . . . 91
5.2.1 Struct Field Reordering: `repr(Rust)` vs. `#[repr(C)]` . . . 91
5.2.2 How a General `enum` Sits in Memory: The Tagged Union . . . 92
5.2.3 Niche Optimization: Zero-Overhead `Option<T>` . . . 93
5.3 `Option<T>`, `Result<T, E>`, and `?` vs. Java 6 `null` and Checked Exceptions . . . 95
5.3.1 `Option<T>` and Combinator Pipelines . . . 95
5.3.2 `Result<T, E>` vs. C `errno` and Java 6 Checked Exceptions . . . 96
5.3.3 Desugaring the `?` Operator Step by Step . . . 96
5.4 Exhaustive Pattern Matching as Compiler-Enforced Case Analysis . . . 98
5.4.1 `match` Expressions, Destructuring, `@` Bindings, and Guards . . . 98
5.4.2 Match Ergonomics: Borrowing Without Moving . . . 99
5.4.3 Exhaustiveness Across Codebase Evolution and Control-Flow Shortcuts . . . 99
5.5 Case Study: Refactoring a Legacy Java 6 `PaymentOrder` into a Rust State Machine . . . 100
5.5.1 Four Silent Bugs in the Legacy Java 6 `PaymentOrder` . . . 100
5.5.2 Step-by-Step Refactoring into an Algebraic State Machine . . . 101
5.6 Exercises . . . 105

**6 Traits, Generics, Monomorphization, and Fat Pointers 107**
6.1 Decoupling Data from Behavior: `struct` vs `impl Trait` . . . 108
6.1.1 Reading `self` vs `Self` Through the Ownership Lens . . . 110
6.2 Generics Across Three Worlds: Erasure, Templates, and Bounds . . . 111
6.2.1 World 1: Java 6 Type Erasure (Safe Definition Check, Boxed Runtime) . . . 111
6.2.2 World 2: C++ Templates (Zero-Cost Machine Code, Late Duck-Typed Checks) 112
6.2.3 World 3: Rust's Synthesis (Definition-Checked Bounds + Specialization) . . . 113
6.2.4 Syntax Ergonomics: `where` Clauses and `impl Trait` . . . 114
6.3 Static Dispatch and Monomorphization Down to Machine Code . . . 115
6.3.1 The Cost of Monomorphization—and the Inner-Function Cure . . . 116
6.4 Dynamic Dispatch and the 16-Byte Fat Pointer (`&dyn Trait`) . . . 117
6.4.1 Why C++ and Java Hide Vtable Pointers Inside Every Object . . . 117
6.4.2 Rust's Inversion: Keep Objects Headerless, Make the Pointer 16 Bytes . . . 118
6.4.3 Tracing a `dyn` Call and a `Box<dyn Trait>` Drop by Hand . . . 119
6.5 `dyn`-Compatibility (Object Safety) Derived from the Vtable Layout . . . 121
6.5.1 Two Practical Patterns When You Hit `dyn`-Compatibility . . . 121

6.6 Associated Types, Coherence, Supertraits, and Blanket Impls . . . 122
6.6.1 Associated Types (`type Item`) vs Generic Type Parameters (`From<T>`) . . . 122
6.6.2 Coherence and the Orphan Rule: Saving Your `HashMap` from Silent Corruption 123
6.6.3 Supertraits and Blanket Implementations . . . 124
6.6.4 Choosing Your Polymorphism Tool: Generics vs `dyn Trait` vs `enum` . . . 125
6.7 The Everyday Standard Library Trait Vocabulary . . . 126
6.7.1 `Debug` vs `Display`, and `Default` . . . 126
6.7.2 `PartialEq`/`Eq` and `PartialOrd`/`Ord`: Why `f64` Cannot Corrupt Your `BTreeMap` 127
6.7.3 Conversions: `From`/`Into` and `TryFrom`/`TryInto` . . . 127
6.7.4 Pointer-Like Views and Cleanup: `AsRef`, `Borrow`, `Deref`, and `Drop` . . . 128
6.8 Exercises . . . 129

**III Thinking Functionally and Writing Idiomatic Rust 131**

**7 Thinking Functionally: From Imperative Loops through OCaml and Haskell to Rust 133**

7.1 Rewiring the `for`-Loop Brain: Statements vs. Expressions . . . 133
7.1.1 Everything is an Expression: Java 6 vs. OCaml vs. Rust . . . 134
7.1.2 Why `;` Yields `()` and Why C/Java `void` Breaks Generics . . . 136
7.2 OCaml's Direct DNA in Rust . . . 137
7.2.1 Immutability by Default, Shadowing, and `ref` Cells vs. `let mut` . . . 137
7.2.2 Algebraic Variants and Structural Pattern Matching . . . 138
7.2.3 Persistent Data Structures and Structural Sharing (`x :: xs`) . . . 139
7.2.4 Currying, Partial Application, Pipes (`|>`), and Tail Recursion . . . 139
7.2.5 Hindley–Milner Inference, `fn` Boundaries, and Modules . . . 140
7.3 Haskell's Big Ideas Without the Ivory-Tower Fog . . . 141
7.3.1 Referential Transparency vs. Rust's Controlled Local Mutation . . . 141
7.3.2 Lazy Evaluation vs. Zero-Allocation Demand-Driven Iterators . . . 142
7.3.3 Typeclasses (`Eq`, `Ord`, `Show`, `FeePolicy`) → Rust Traits . . . 143
7.3.4 Functors (`.map()`) and Monads (`.and_then()` and `?`) . . . 144
7.4 Zero-Cost Closures in Rust: Anonymous Structs and `Fn`, `FnMut`, `FnOnce` . . . 145
7.4.1 Exact Compiler Desugaring of Three Ledger Closures . . . 146
7.4.2 Why the Hierarchy Runs `Fn: FnMut: FnOnce`, and the `move` Trap . . . 148

7.5 Zero-Cost Iterator Pipelines, `Result` Transposition, and Loop Fusion . . . 149
7.5.1 Ownership Entry Points, Combinators, and `Result` Transposition . . . 150
7.5.2 Engineering Judgment: When a Plain `for` Loop Wins . . . 151
7.6 Exercises . . . 152

**8 How Idiomatic Rust Is Actually Written: Architecture, Patterns, and Anti-Patterns 155**

8.1 Curing "Java-in-Rust Syndrome": Data-Oriented Arenas and Typed Index Handles . . . 156
8.1.1 Why Direct Translation Produces `Rc<RefCell<T>>` Soup . . . 156
8.1.2 The Rust Architectural Shift: Separate Identity from Ownership . . . 158
8.1.3 Generational Indices: Solving the Stale-Handle (ABA) Problem . . . 159
8.1.4 Mutating Two Nodes Safely with `split_at_mut` . . . 160
8.2 Parse, Don't Validate: Zero-Cost Newtypes and Smart Constructors . . . 164
8.2.1 Validation Throws Proof Away; Parsing Preserves It in a Type . . . 164
8.2.2 Zero-Cost Newtypes, `#[repr(transparent)]`, and Module Privacy . . . 165
8.3 Compile-Time State Machines: The Typestate Pattern and Builders . . . 169
8.3.1 Encoding Protocol States in Type Parameters and `PhantomData` . . . 170
8.3.2 Why Rust's Destructive Move Makes Typestate Airtight . . . 170
8.3.3 Compile-Time Typestate vs Runtime `enum` State Machines . . . 173
8.3.4 Idiomatic Builders: Owned Chaining and Typestate Builders . . . 174
8.4 Ergonomic, Zero-Copy API Boundaries and Borrow-Splitting Idioms . . . 177
8.4.1 Why `&String`, `&Vec<T>`, and `&Box<T>` Are Anti-Patterns . . . 177
8.4.2 Caller-Controlled Allocation: `impl Into<String>` and `Cow<'a, str>` . . . 178
8.4.3 Defeating the Whole-Struct `&mut self` Trap (Split Borrows) . . . 180
8.5 Idiomatic Error Handling: `thiserror` vs `anyhow`, and `Result` vs `panic!` . . . 183
8.5.1 Zone 1: Domain and Library Layers Use Structured `enums` (`thiserror`) . . . 183
8.5.2 Zone 2: Application Entrypoints, Workers, and CLIs Use `anyhow` . . . 186
8.5.3 The Hard Line Between `Result` and `panic!` . . . 186
8.6 Modules, Visibility Capsules, `#[must_use]`, and Rust Testing Culture . . . 187
8.6.1 Privacy Lives at the Module Boundary, Not the Struct Boundary . . . 187
8.6.2 Compiler Guardrails with `#[must_use]` . . . 188
8.6.3 Crate Layering for Production Services . . . 189
8.6.4 Rust's Four-Tier Testing Culture and Conservation Laws . . . 189

8.6.5 Summary: Translating Instincts into Idiomatic Rust . . . 191
8.7 Exercises . . . 191

**IV Modern Type Systems and Concurrency: TypeScript, Kotlin, and Go 195**

**9 TypeScript From the Ground Up: Structural Subtyping, Set Theory, and Erase-to-JS 197**

9.1 Two Mental Shifts from Java 6: Shape Typing and Total Erasure . . . 198
9.1.1 Mental Shift 2: Total Erasure and What V8 Actually Sees . . . 199
9.2 Types as Mathematical Sets: From `never` to `unknown` . . . 201
9.2.1 Cardinality 0: `never` Is the Empty Set $\emptyset$ . . . 202
9.2.2 Cardinality 1 and Finite Sets: Literal Types and Unions ($A \cup B$) . . . 202
9.2.3 Open Object Sets: Why More Properties Mean a *Smaller* Set . . . 203
9.2.4 Intersections ($A \cap B$) and the Dual Algebra of `keyof` . . . 205
9.2.5 Top of the Lattice: `unknown` ($\mathcal{U}$) vs. the Escape Hatch `any` . . . 206
9.3 Discriminated Unions and Control-Flow Narrowing . . . 207
9.3.1 Tagged Shapes: TypeScript's Counterpart to Rust's `enum` . . . 208
9.3.2 How Control-Flow Narrowing Works—and Three Classic JavaScript Traps . . . 210
9.3.3 User-Defined Type Guards (`x is T`) and Assertion Functions (`asserts x is T`) . . . 211
9.4 Type-Level Programming: Transforming Types Like Data . . . 212
9.4.1 `typeof`, `keyof`, and Indexed Access `T[K]` . . . 212
9.4.2 Mapped Types: Looping Over Keys at Compile Time . . . 213
9.4.3 Conditional Types, Union Distributivity, and Pattern Matching with `infer` . . . 214
9.4.4 Template Literal Types: Compile-Time String Algebra . . . 216
9.5 Variance, Unsoundness Holes, Branding, and Runtime Boundaries . . . 217
9.5.1 Variance From Set Inclusion—and the Mutable Array Trap . . . 217
9.5.2 Nominal Branding: Teaching a Structural Type Checker About Units and IDs 219
9.5.3 The Runtime Trust Boundary: Parse, Don't Cast . . . 220
9.6 Exercises . . . 222

**10 Kotlin From Java 6: Null-Safety, Algebraic Hierarchies, and Coroutines on the JVM 225**

10.1 Bridging Java 6 to Kotlin: Same Bytecode Machine, Sharper Lens . . . 226

10.2 Nullability in the Type System: Splitting Every Reference Set in Two . . . . . 227
10.2.1 Zero-Wrapper Reference Nullability vs. Boxed Nullable Primitives . . . . . 228
10.2.2 Safe Calls, Elvis with `Nothing`, Smart Casts, and Platform Types . . . . . 229
10.3 Data, Value, and Sealed Types: Algebraic Modeling on the JVM . . . . . 230
10.3.1 `data class`: Value Equality and Copy Without Boilerplate . . . . . 230
10.3.2 `@JvmInline value class`: Zero-Header Domain Types on the JVM . . . . . 231
10.3.3 `sealed interface` and Exhaustive `when`: Sum Types on the JVM . . . . . 232
10.4 Functions, Receivers, Inline Reification, and Sequences . . . . . 234
10.4.1 Lambdas with Receivers (`T.() -> R`): How Type-Safe Builders Work . . . . . 234
10.4.2 `inline` Functions and `reified T`: Defeating Closure Allocation and JVM Erasure . . . . . 235
10.4.3 Eager `Iterable` Extensions vs. Lazy `Sequence<T>` . . . . . 237
10.5 Kotlin Coroutines Under the Hood: CPS State Machines and Structured Concurrency 238
10.5.1 Inside `kotlinc`: Continuation-Passing Style (CPS) and the `label` Switch . . . 238
10.5.2 Dispatchers and Structured Concurrency: Taming Background Work . . . . . 241
10.5.3 Cold `Flow<T>` vs. Hot `Channel<T>` and `StateFlow<T>` . . . . . 243
10.6 Exercises . . . . . 244

**11 Go and the Grand Language Synthesis: Memory, Types, and Concurrency Compared 247**

11.1 Go's Design Trade-off: Values, Slices, Errors, and Fast Builds . . . . . 248
11.1.1 Why Go Compiles So Fast . . . . . 248
11.1.2 Flat Value Structs and Escape Analysis . . . . . 248
11.1.3 Slices: A 24-Byte Header Over a Shared Backing Array . . . . . 250
11.1.4 Errors Are Values, and `defer` Runs Cleanup . . . . . 251
11.2 Implicit Interfaces: 16-Byte Fat Pointers and the Typed-Nil Trap . . . . . 252
11.2.1 What an Interface Value Looks Like in Memory . . . . . 252
11.2.2 The Typed-Nil Interface Trap . . . . . 253
11.2.3 No Sum Types: Interfaces Plus Type Switches . . . . . 254
11.2.4 Generics Since Go 1.18: GC-Shape Stenciling . . . . . 255
11.3 Go's M:N Runtime Scheduler: Goroutines, Channels, and Context . . . . . 256
11.3.1 Goroutines: 2 KB Stacks That Grow by Copying . . . . . 256
11.3.2 The G-M-P Model . . . . . 256

11.3.3 Channels and `select`: Communicating Sequential Processes . . . 257
11.3.4 `context.Context`: Cancellation Down a Tree of Goroutines . . . 259
11.4 The Grand Synthesis: Eight Languages on Seven Axes . . . 260
11.4.1 One Task, Five Languages . . . 260
11.4.2 Axes 1 and 2: Memory and Ownership, Dispatch and Polymorphism . . . 262
11.4.3 Axes 3 and 4: Absence, Alternatives, and Failure . . . 264
11.4.4 Axes 5 and 6: Generics Implementation and Concurrency Runtime . . . 264
11.4.5 Axis 7: Where Each Language Is Used, and Why . . . 265
11.4.6 Putting It Together: Choosing Languages for the Financial Backend . . . 266
11.5 Exercises . . . 266

**V Capstone: A Financial Backend in Rust with Axum, SQLx, and PostgreSQL 269**

**12 Async Rust From Silicon to Tokio: State Machines, Pinning, and Cancellation 271**
12.1 Why Thread-per-Request Hits a Hardware Wall . . . 272
12.1.1 The Memory Cost of Preallocated Call Stacks . . . 273
12.1.2 The Hidden Tax of Kernel Context Switches . . . 274
12.1.3 Non-Blocking Sockets and Kernel Readiness (`epoll` and `io_uring`) . . . 274
12.2 Rust's Stackless Model: The `Future` Trait . . . 275
12.2.1 Dissecting `std::future::Future` . . . 276
12.2.2 Writing a Leaf `Future` by Hand . . . 277
12.3 How `async/await` Compiles Into an Enum State Machine . . . 278
12.4 Self-Referential Futures and Why `Pin` Exists . . . 282
12.4.1 Decoding the Double Negative: What `Unpin` Means . . . 285
12.4.2 The Two Ways You Pin a Future in Real Code . . . 285
12.5 Wakers, Readiness, and Tokio's Work-Stealing Scheduler . . . 286
12.5.1 The `Waker` Contract and Lost-Wakeup Freedom . . . 286
12.5.2 Inside Tokio's Multi-Threaded Work-Stealing Runtime . . . 287
12.5.3 Why `tokio::spawn` Demands `Future + Send + 'static` . . . 288
12.5.4 How the Compiler Saves You From a `MutexGuard` Worker Deadlock . . . 288
12.5.5 Cooperative Scheduling and `tokio::task::spawn_blocking` . . . 290
12.6 Backpressure, Bounded Channels, and Cancellation Safety . . . 291

12.6.1 Explicit Backpressure with Bounded `mpsc` and `Semaphore` . . . 291
12.6.2 Async Cancellation in Rust: Dropping a `Future` Mid-Flight . . . 291
12.6.3 The `tokio::select!` Cancellation-Safety Trap . . . 293
12.7 Exercises . . . 294

**13 Modeling Money and Ledgers in Types: Fixed-Point Arithmetic, Double-Entry Accounting, and Typestate 297**
13.1 Rule 1 of Money: Why IEEE-754 Floats Fail and Fixed-Point Works . . . 297
13.1.1 The Three Financial Failures of Binary Floating-Point . . . 298
13.1.2 Minor-Unit Signed Integers (`i64` and `i128`) Versus Scaled Decimals . . . 299
13.1.3 Checked Arithmetic Versus Silent Release-Mode Wraparound . . . 300
13.1.4 Banker's Rounding and Penny-Exact Fee Allocation . . . 300
13.2 Type-Safe Identifiers and Multi-Currency Money . . . 302
13.2.1 Zero-Cost Domain Identifiers and Time-Ordered UUIDv7 . . . 303
13.2.2 Phantom Compile-Time Currencies Versus Runtime Multi-Currency `Money` . . . 304
13.3 Double-Entry Accounting From First Principles . . . 307
13.3.1 Deriving Debits, Credits, and the Five Account Kinds Without Rote Memorization . . . 307
13.4 Parse, Don't Validate: Typestate and Proof-Carrying Entries . . . 310
13.4.1 Step 1: Strictly Positive Amounts (`PositiveMoney`) . . . 311
13.4.2 Step 2: Typestate Sealing of `JournalDraft` Into `BalancedEntry` . . . 312
13.4.3 Putting It Together: Domain Transfer Constructor With Optional Fee Split . . . 315
13.4.4 Lifecycle Typestate: Authorization Holds, Captures, and Voids . . . 316
13.5 Exercises . . . 318

**14 Building the Financial Service End-to-End: Axum, Tower, SQLx, and PostgreSQL 321**
14.1 The PostgreSQL Schema: Constraints as a Second Line of Defense . . . 322
14.1.1 A Database-Level Balance Check . . . 324
14.2 SQLx: SQL Checked at Compile Time . . . 325
14.2.1 What `query!` Does During `cargo build` . . . 325
14.2.2 Offline Mode: Building Without a Database . . . 326
14.2.3 Connection Pools and the Settings That Protect You . . . 326
14.2.4 `Transaction` as an RAII Guard . . . 327

14.3 Two Withdrawals, One Balance: Concurrency Done Right . . . 328
14.3.1 The Naive Version and Its Race . . . 328
14.3.2 Fix 1: A Single Conditional Statement . . . 329
14.3.3 Fix 2: Lock Every Touched Account, in Sorted Order . . . 330
14.3.4 The Repository Function: `commit_entry` . . . 331
14.3.5 Fix 3: `SERIALIZABLE` and the Retry Loop . . . 334
14.4 Idempotency Keys: Safe Retries Over an Unreliable Network . . . 336
14.4.1 The Protocol, Step by Step . . . 336
14.5 Axum and Tower: From Bytes to a Typed Handler . . . 338
14.5.1 Tower Middleware: Services Wrapping Services . . . 338
14.5.2 How Axum Calls a Plain `async fn` . . . 339
14.5.3 Writing Our Own Extractor . . . 341
14.5.4 One Error Type, Mapped Once . . . 342
14.5.5 Tracing Across Threads . . . 343
14.5.6 Assembling the Application and Shutting Down Gracefully . . . 343
14.6 The Transactional Outbox: Telling Other Systems, Reliably . . . 345
14.7 Testing Against a Real Database With `#[sqlx::test]` . . . 347
14.8 Exercises . . . 349

# Part I

# Leaving Java 6: Memory, Values, and the Machine

# Chapter 1

# Leaving Java 6: Memory, Values, and the Machine

If you spent years writing Java 6, you learned a remarkably productive mental simplification of a computer: there are *primitives* (`int`, `long`, `double`, `boolean`) that hold numbers directly, and there are *objects* created with `new` that float somewhere in a managed cloud called "the heap" until a garbage collector sweeps them away when nobody needs them anymore. You almost never had to ask how many bytes an `Order` occupied, which 64-byte slice of RAM it landed in, or how many pointer hops stood between the CPU's arithmetic units and the price field you wanted to add.

That simplification helped an entire generation ship large business systems without corrupting memory via stray C pointers. Yet it also hid three steep costs — one in **hardware performance**, one in **aliasing and concurrency bugs**, and one in **type-system honesty**. Every major systems and application language designed over the past fifteen years — Modern C++, Rust, Go, Kotlin, and TypeScript — is a direct response to one or more of those hidden costs.

To see why Rust's ownership model, C++'s deterministic destructors, Kotlin's null-safe value classes, and Go's flat structs look the way they do, we first have to take the lid off the JVM and put Java 6 side by side with the physical machine. In this chapter we go all the way down to virtual address spaces, stack pointers, 16-byte JVM object headers, 64-byte hardware cache lines, and stop-the-world garbage collection safepoints — and then climb back up to value semantics, aliasing bugs, `null`, and exceptions.

---

## 1.1 The Two Arenas of a Running Process: Stack Frames vs. the Heap

Whether a program was compiled from C, C++, Rust, Go, or JIT-compiled by the HotSpot JVM from Java 6 bytecode, the operating system hands the running process the same kind of workspace: a flat, numbered array of bytes called its **virtual address space**. On a 64-bit Linux or macOS machine, every pointer is an unsigned 64-bit integer address (typically using the low 48 bits in user space, from `0x0000_0000_0000_0000` up to `0x0000_7FFF_FFFF_FFFF`).

Inside that address space, your program's live data does not float around at random. Aside from

read-only machine code (`.text`), constant literals (`.rodata`), and static globals (`.data`/`.bss`) near the bottom of memory, nearly every byte your code creates at runtime lives in one of two places: **a thread stack** or **the heap**.

### 1.1.1 How the hardware stack works in two CPU instructions

Every OS thread gets its own contiguous region of virtual memory for its **call stack** — typically 1 MB on 64-bit Linux JVMs and 2 MB to 8 MB for native C/C++/Rust threads. A dedicated CPU register (`RSP` on x86-64, `SP` on ARM64) always holds the address of the current top of that thread's stack. By convention on modern architectures, the stack starts at a high address and *grows downward* toward lower addresses as functions call deeper functions.

Suppose function `settleBatch()` calls `computeFee(10000, 25)` in C:

```c
#include <stdint.h>

int64_t compute_fee(int64_t amount_cents, int64_t bps) {
    int64_t gross = amount_cents * bps;
    int64_t rounded = (gross + 5000) / 10000;
    return rounded;
}
```

Even before optimization (where all four numbers simply stay in CPU registers like `RDI`, `RSI`, and `RAX`), watch how cheap it is when a function needs 32 bytes of local workspace on the stack:

1. **Allocate on entry:** The CPU subtracts 32 from the stack pointer register: `sub rsp, 32`. That single instruction costs about one cycle — a fraction of a nanosecond — and a superscalar core often overlaps it with neighboring work. Those 32 bytes at `[rsp .. rsp+31]` now belong exclusively to this call's **stack frame**.

2. **Read and write at fixed offsets:** Local variable `gross` lives at `[rsp + 8]`; `rounded` lives at `[rsp + 16]`. No lookup table, no pointer dereference to a separate block, no lock. And because the top of the stack was touched moments ago by the caller, those exact bytes are almost guaranteed to sit in the CPU core's fastest **L1 data cache**.

3. **Deallocate on return:** Just before `ret`, the CPU adds 32 back: `add rsp, 32`. Every local variable in that frame vanishes in a single instruction. There is nothing to scan, nothing to mark, and zero fragmentation left behind.

**Intuition**

Think of the thread stack as a stack of cafeteria trays where you are only ever allowed to add a tray on top (`call`) and lift that exact top tray back off (`return`). Because lifetimes follow strict **Last-In, First-Out (LIFO)** order — a callee always finishes before its caller resumes — there are never "holes" in the middle of the stack. One integer register (`RSP`) knows the exact boundary between live bytes and free space at every instant.

### 1.1.2 Why the heap exists, and what you pay for it

If the stack is that fast and clean, why do we ever use a heap at all? Because the cafeteria-tray rule imposes two hard constraints. To place a value directly inside a stack frame, the compiler must know:

1. **Its exact byte size at compile time.** An `int64_t` is always 8 bytes; a C `struct Order` of four fixed fields might always be 32 bytes. By contrast, an array of orders read from a network socket might contain 3 elements on one request and 50,000 on the next. You cannot emit a fixed `sub rsp, K` instruction when $K$ is unknown until runtime (and blowing a 1 MB thread stack on a large runtime buffer crashes the process with a stack overflow).

2. **That its lifetime ends before the function returns.** As soon as `compute_fee` executes `ret`, its stack frame lies *below* `RSP` and will be overwritten by the very next function call. Any data that must outlive the function that created it — unless copied or moved directly into the caller's own stack frame — needs storage independent of the LIFO call stack.

That independent arena is **the heap**. Unlike the stack, heap allocations can be requested in any size and freed in any order. That freedom immediately destroys the single-pointer trick: when a program allocates blocks $A$, $B$, $C$, and $D$, and then frees $B$, there is now a gap between $A$ and $C$. Consequently, every heap allocation pays up to four distinct taxes:

- **Allocation bookkeeping.** Native allocators (`glibc malloc`, `jemalloc`, `mimalloc`) maintain size-segregated free lists, thread caches, and block metadata headers to find an open slot without fragmenting RAM. Managed runtimes like the JVM use fast bump-pointer allocation inside per-thread Eden buffers (*TLABs*), deferring the bookkeeping bill until garbage collection runs.

- **Pointer indirection.** Because a heap block can land anywhere in virtual memory, your stack frame still has to hold an 8-byte pointer storing the heap block's address. Reaching the data requires reading the pointer first and then fetching the target address.

- **Cache misses.** Two heap blocks allocated five milliseconds apart — or rearranged differently over hours of churn — frequently sit megabytes apart in RAM, defeating the CPU cache hierarchy we will measure in §1.3.

- **Deallocation tracking.** Someone must eventually reclaim dead heap blocks: either the programmer manually (`free` in C), deterministic compiler-inserted destructors (RAII/`Drop` in C++ and Rust), reference counting (`shared_ptr`, `Rc`/`Arc`), or a tracing garbage collector (Java, Go, Kotlin, TypeScript/V8).

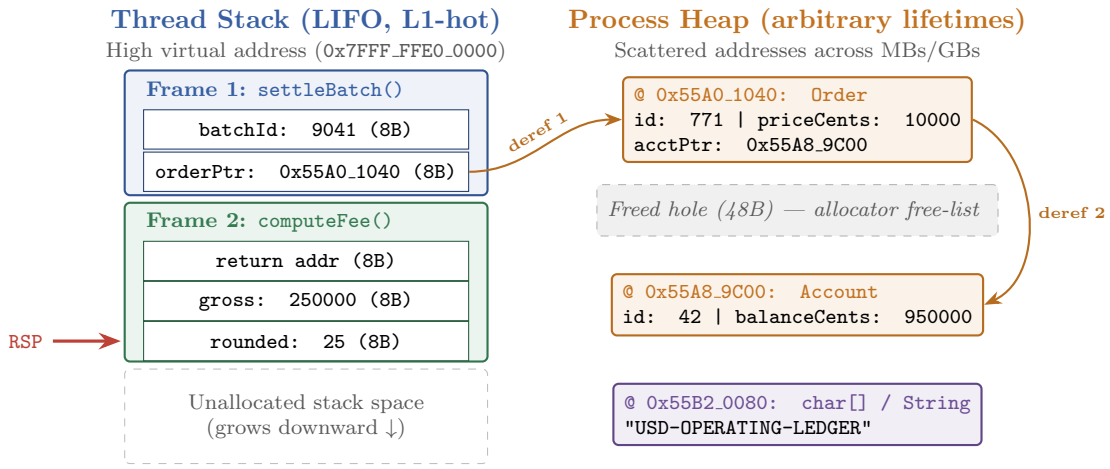



Figure 1.1: Left: Thread stack frames allocated and reclaimed in a single instruction by moving `RSP`, packed contiguously in L1 cache. Right: Heap allocations scattered across virtual memory with freed holes and multi-hop pointer indirection.

Now comes the design fork that separates Java 6 from C, C++, Rust, and Go:

- In **C, C++, and Rust**, *you* choose the arena per variable. Every `struct` (and in Rust, every `enum` and fixed-size array `[T; N]`) is a plain **value** placed directly on the stack by default, or embedded inline inside a parent `struct` or contiguous heap array (`Vec<T>`) with zero per-element headers. You pay for a heap pointer (`malloc`, `std::unique_ptr<T>`, `Box<T>`) only when dynamic size, recursive types, or cross-scope ownership genuinely demand it.

- In **Java 6**, the language makes that choice *by category of type*. Only the eight built-in primitives (`boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, `double`) and raw object references can sit directly in stack frames or inline inside arrays. *Every user-defined `class` instance — even a tiny two-integer `Money(long cents, int currencyCode)` pair — is conceptually a heap-allocated object accessed through a pointer.*

**Reading note**

Modern HotSpot JVMs include a JIT optimization called **escape analysis** (introduced experimentally in late Java 6 updates and enabled by default in Java 7+). When the JIT compiler can prove that a newly allocated object never "escapes" a compiled method (is never stored into a heap field, returned, or passed into a non-inlined method), it can *scalar-replace* the object's fields into CPU registers or stack slots. That helps short-lived local iterators, but it cannot flatten an `Order[]`, an `ArrayList<Order>`, a `HashMap` entry, or any object shared across method boundaries. Whenever you build data structures in Java 6, you get full heap objects.

## 1.2 Inside a Java 6 Object: Headers, Alignment, and Pointer Chasing

Let us open up a Java 6 object and count every byte. Suppose we are building an order-matching or ledger service and define a clean domain object representing a limit order:

```
// Java 6 domain class for an order in our financial service
public enum Side { BUY, SELL }

public final class Order {
    private final long orderId;      // 8 bytes
    private final long accountId;    // 8 bytes
    private final long priceCents;   // 8 bytes
    private final int  quantity;     // 4 bytes
    private final Side side;         // object reference!

    public Order(long orderId, long accountId, long priceCents,
                 int quantity, Side side) {
        this.orderId = orderId;
        this.accountId = accountId;
        this.priceCents = priceCents;
        this.quantity = quantity;
        this.side = side;
    }
}
```

How much payload information is actually in one `Order`? Three 64-bit integers ($3 \times 8 = 24$ bytes), one 32-bit integer (4 bytes), and one bit of direction (`BUY` or `SELL`, stored nicely in 1 byte in C or Rust): 29 **bytes of business information**, padded to 32 **bytes** for 8-byte CPU alignment.

Now let us count what a 64-bit HotSpot JVM actually allocates on the heap for `new Order(...)`.

### 1.2.1 The 16-byte HotSpot object header

In Java 6, *every single object instance on the heap* carries two hidden machine words at offset 0 before any of your fields begin, because every `Object` in Java promises three capabilities whether you use them or not:

1. You can call `obj.getClass()` or perform `instanceof` / virtual method dispatch on any object.
2. You can call `synchronized (obj) { ... }` and `obj.wait()`/`obj.notify()` on *any* object — even an `Integer` or an immutable `Order` you never intend to lock.
3. You can call `System.identityHashCode(obj)`, and the garbage collector needs mark and age bits right inside the object to track survivors across GC cycles.

To support those promises, a standard 64-bit HotSpot JVM prepends a **16-byte object header** (two 64-bit words):

Formal

**64-bit HotSpot JVM Object Layout (Uncompressed 64-bit OOPs vs. Compressed OOPs).**

- **Bytes 0..7 — The Mark Word (8 bytes):** A bit-packed runtime header holding the object's lock state (2–3 tag bits distinguishing unlocked, biased lock thread ID, stack-locked lightweight lock pointer, or inflated heavyweight `ObjectMonitor*`), the cached 31-bit `identityHashCode` (computed lazily on first call), and 4 bits of generational GC survivor age (0..15).
- **Bytes 8..15 — The Klass Word (8 bytes raw; 4 bytes with `-XX:+UseCompressedOops`):** A pointer to the JVM's internal `Klass` metadata structure describing the class's fields, type hierarchy, and virtual method table (*vtable*). When *Compressed Ordinary Object Pointers* (`UseCompressedOops`) are enabled (applicable for heaps up to 32 GB, since 8-byte object alignment lets a 32-bit unsigned offset address $2^{32} \times 8 = 32$ GB via addr = base + (oop $\ll 3$)), the Klass word shrinks from 8 bytes to 4 bytes, yielding a **12-byte header**.
- **Array length (arrays only, +4 bytes):** Every Java array (`int[]`, `Object[]`) adds a 4-byte signed 32-bit `length` field right after the Klass word.
- **8-byte external alignment:** HotSpot rounds the total size of *every* heap object up to a multiple of **8 bytes**.

Let us run the exact byte arithmetic on two everyday Java 6 types so the overhead stops being abstract.

**Example 1.1** (Why `java.lang.Long` costs 3× its payload)**.** Suppose you put ledger balances into a Java 6 generic map or list: `List<Long> balances = new ArrayList<Long>();`. Java 6 generics cannot specialize over primitive `long`, so every balance is boxed into a `java.lang.Long` heap object containing a single field `private final long value;` (8 bytes).

- **Without Compressed OOPs (raw 64-bit):** 8B (Mark) + 8B (Klass) + 8B (`long value`) = **24** bytes. Plus the 8-byte reference slot inside `ArrayList`'s backing `Object[]` array = **32 bytes per 8-byte integer** (4× total RAM).
- **With Compressed OOPs (heaps $<$ 32 GB):** 8B (Mark) + 4B (Klass) + 4B (alignment gap before 8B `long`) + 8B (`long value`) = **24** bytes still. Plus the 4-byte compressed reference slot inside `Object[]` = **28 bytes per 8-byte integer** (3.5× total RAM).

Even with compressed pointers turned on, storing one million `Long` values in an `ArrayList<Long>` consumes roughly **28 MB** scattered across a million tiny heap allocations, whereas `long[1_000_000]` in Java or `Vec<i64>` in Rust consumes exactly **8 MB** in one contiguous block.

Now let us size our `ArrayList<Order>` containing $N$ orders and compare it byte for byte against the identical data in C or Rust:

```
// Rust (and equivalent C struct): flat 32-byte value, zero headers
#[repr(u8)]
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Side {
    Buy  = 0,
    Sell = 1,
```

```
}

#[repr(C)]
#[derive(Clone, Copy, Debug)]
pub struct Order {
    pub order_id:    u64,  // offset  0..8   (8 bytes)
    pub account_id:  u64,  // offset  8..16  (8 bytes)
    pub price_cents: i64,  // offset 16..24  (8 bytes)
    pub quantity:    u32,  // offset 24..28  (4 bytes)
    pub side:        Side, // offset 28..29  (1 byte)
    // 3 bytes of compiler-inserted tail padding at 29..32 so
    // the next Order in [Order; N] / Vec<Order> starts 8-byte aligned!
}
// std::mem::size_of::<Order>() == 32, align_of::<Order>() == 8
```

Compare what sits in memory for `ArrayList<Order>` in Java 6 versus `Vec<Order>` in Rust (or `std::vector<Order>` in C++):

1. **In Java 6 (`ArrayList<Order>`):**

   - A local reference variable `orders` on the stack (4 or 8 bytes) points to Heap Object #1: the `ArrayList` wrapper (24 bytes), which holds `int size` and a reference `Object[] elementData`.
   - Following `elementData` lands at Heap Object #2: an `Object[]` array (16- or 24-byte array header plus $N \times 4$B compressed or $N \times 8$B raw pointers). Note that `elementData` does *not* hold `Order` fields — it holds an array of **addresses**.
   - Each slot `elementData[i]` points to a separate Heap Object #$(3 + i)$: an individual `Order` instance. With raw 64-bit OOPs: 16B header + 24B ($3 \times$ `long`) + 4B (`int`) + 4B padding + 8B (`Side` ref) = **56** bytes (or with compressed OOPs: 12B header + 4B (`int quantity`) + 24B ($3 \times$ `long`) + 4B (`Side` ref) + 4B padding = **48** bytes after HotSpot field reordering).
   - Reading `order.side` follows *one more pointer* to one of the two global `Side.BUY` / `Side.SELL` singleton enum objects on the heap.

2. **In C, C++, or Rust (`Vec<Order>`):**

   - On the stack sits a 24-byte header `(ptr, len, cap)` — just three 64-bit machine words, with zero heap allocation for the wrapper itself.
   - Following `ptr` once lands on a **single contiguous slab of** $32 \times N$ **bytes** on the heap: `Order[0]` occupies bytes `0..31`, `Order[1]` occupies bytes `32..63`, `Order[2]` occupies bytes `64..95`, and so on. Zero per-element object headers, zero array of pointers, zero pointer hop to read `side`.

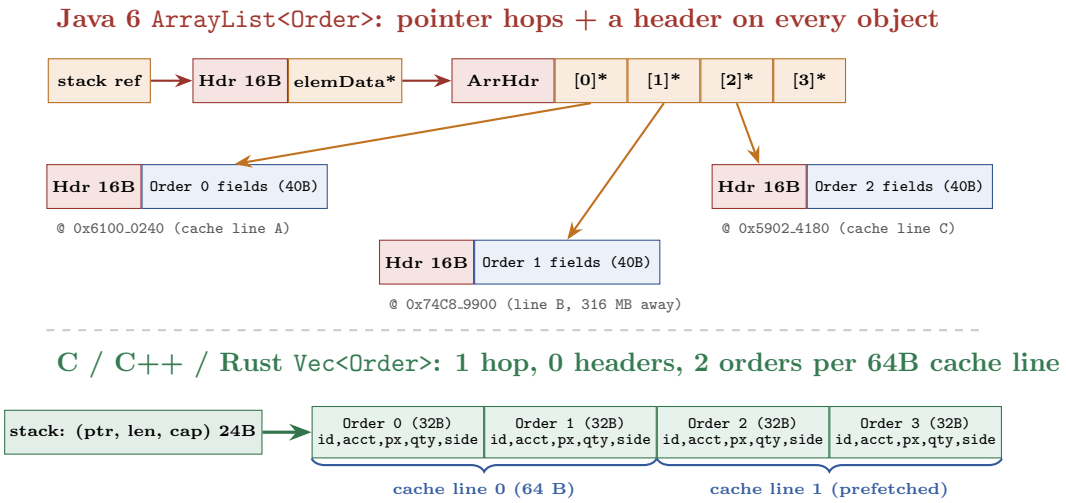



Figure 1.2: Memory layout of `ArrayList<Order>` in Java 6 (top) versus `Vec<Order>` in Rust or `std::vector<Order>` in C++ (bottom). Flat value arrays eliminate per-object headers and pack two 32-byte orders into each 64-byte CPU cache line.

**Work it yourself**

**Check struct padding rules by hand.** Why did our C/Rust `struct Order` occupy 32 bytes instead of $8 + 8 + 8 + 4 + 1 = 29$ bytes? Because in C's layout ABI (`#[repr(C)]`), a struct's alignment equals the largest alignment of any of its fields — here 8 bytes (for `u64`/`i64`) — and a struct's total `sizeof` must be a multiple of its alignment so that in an array `Order[N]`, `Order[1]` also starts at an address divisible by 8.
Now test yourself: what if a careless author declared the C struct fields in alternating order: `uint8_t side; uint64_t order_id; uint32_t quantity; uint64_t account_id; int64_t price_cents;`? Count the internal alignment gaps before each 8-byte field: byte 0 holds `side`, bytes 1..7 are **7 wasted padding bytes**, bytes 8..15 hold `order_id`, bytes 16..19 hold `quantity`, bytes 20..23 are **4 wasted padding bytes**, bytes 24..31 hold `account_id`, and bytes 32..39 hold `price_cents` — bloating the struct from **32 bytes to 40 bytes**. (A preview of Part II: Rust's default `struct` layout — without `#[repr(C)]` — leaves field order unspecified, so the compiler is free to reorder fields, and `rustc` does; you typically get the compact 32-byte layout whatever order you write the fields in.)

## 1.3 The Memory Hierarchy: Cache Lines, Prefetchers, and Stop-the-World GC

Seeing that `ArrayList<Order>` uses roughly 2× the RAM of `Vec<Order>` is only the mild half of the story. RAM is cheap; **memory latency** is not. Even if your server has 256 GB of DDR5 memory, a 4 GHz CPU core executes an arithmetic instruction (`add`, `cmp`) in **0.25 nanoseconds**, while fetching a single uncached address from main DRAM takes roughly **70 to 100 nanoseconds** — nearly **300 to 400 CPU cycles** during which the execution units waiting on that value sit stalled.

To bridge that 300× speed gap between silicon registers and off-chip DRAM, every modern x86-64

and ARM64 processor places three levels of SRAM hardware caches between the core and main memory:

| Storage Tier | Typical Size | Latency | Human Scale (1 cycle = 1 s) |
|---|---|---|---|
| CPU Registers (`RAX`..`R15`) | 16 × 8 B | 0.25 ns (1 cyc) | 1 sec (on your desk) |
| L1 Data Cache (per core) | 32–48 KB | ∼ 1 ns (4 cyc) | 4 sec (across desk) |
| L2 Cache (per core) | 512 KB–2 MB | ∼ 3.5 ns (14 cyc) | 14 sec (across room) |
| L3 Cache (shared) | 16–64 MB | ∼ 12 ns (45 cyc) | 45 sec (down the hall) |
| Main Memory (DRAM) | 16–512 GB | ∼ 80 ns (320 cyc) | **5** min **20** sec |

Look at that last column. Every time your code misses L1, L2, and L3 and goes out to main RAM, the CPU core waits the human equivalent of **five minutes** for one number to come back.

### 1.3.1 64-byte cache lines and the hardware prefetcher

Two hardware rules turn Figure 1.2 from a tidy memory diagram into a 20× to 50× throughput gap:

1. **Memory always moves in 64-byte cache lines.** When you read a 4-byte `quantity` field at address `0x55A0_1058`, the memory controller never fetches 4 bytes. It fetches the entire aligned **64-byte cache line** (`0x55A0_1040` . `0x55A0_107F`) into L3, L2, and L1. In a contiguous `Vec<Order>` of 32-byte structs whose buffer is 64-byte aligned, fetching `orders[0]` brings *all of `orders[0]` AND all of `orders[1]`* into L1 cache in one transfer.
2. **The hardware stride prefetcher predicts linear scans, not pointer hops.** Inside every core sits a dedicated silicon unit watching cache-miss addresses. The moment it sees misses at cache line $L$, $L{+}1$, $L{+}2$, it runs ahead of your loop and streams $L{+}3$, $L{+}4$, ... from DRAM into L2/L1 *before your loop counter even reaches those indices.* By contrast, in `ArrayList<Order>`, `elementData[i]` holds an arbitrary 64-bit address on the heap. Until `elementData[i]` arrives from memory, even the hardware does not know what address `Order i` lives at — defeating the prefetcher whenever objects were allocated out of order or shuffled by mutations.

Let us put real numbers on summing open exposure across $N$ = 1,000,000 resting orders in an in-memory risk check:

```
// Java 6: clean-looking loop, pointer-chasing every iteration
long totalExposureCents = 0L;
for (int i = 0; i < orders.size(); i++) {
    Order o = orders.get(i); // hop 1: read ref; hop 2: chase heap Order
    if (o.getAccountId() == targetAccount) {
        totalExposureCents += o.getPriceCents() * o.getQuantity();
    }
}
```

Cost summary

**Back-of-the-envelope scan time ($N$ = 1,000,000 orders).**

- **Contiguous `Vec<Order>` (32 MB total):** 500,000 sequential 64-byte cache lines. Streaming sequentially at a modest single-core memory bandwidth of 32 GB/s takes

$\frac{32\text{ MB}}{32\text{ GB/s}} \approx$ **1.0** millisecond.

- **Scattered `ArrayList<Order>` ($\sim$ 52 MB total after churn):** Scanning `Object[]` is sequential, but dereferencing 1,000,000 scattered `Order` pointers misses L3 cache once the working set exceeds L3 size. Even with out-of-order execution overlapping $\sim$ 8 concurrent misses (*Memory-Level Parallelism*), $1{,}000{,}000 \times \frac{80\text{ ns}}{8} \approx$ **10** to **35** milliseconds — more than an order of magnitude slower for identical arithmetic.

### 1.3.2 How Java 6 Garbage Collection actually works — and why Old Gen hurts tail latency

If Java 6 allocates nearly everything on the heap, why doesn't `new Order(...)` feel slow in microbenchmarks? Because HotSpot's allocator is well engineered. Instead of grabbing a global heap lock on every `new`, HotSpot hands each Java thread a private chunk of the Young Generation's **Eden** space called a **Thread-Local Allocation Buffer (TLAB)**. Inside a TLAB, `new Order(...)` is just a fast bump-pointer addition (`top += 48`) — almost as cheap as bumping a stack pointer.

The bill arrives later, when Eden fills up. HotSpot's garbage collectors rely on the **weak generational hypothesis**: *most objects die young.* Temporary strings, request DTOs, and boxed numbers created while handling an HTTP request usually become unreachable within milliseconds. HotSpot exploits this by dividing the heap into generations:

1. **Minor GC (Eden + Survivor spaces `S0`/`S1`):** When Eden fills, the JVM pauses application threads at **safepoints**, traces live roots (thread stacks, registers, globals, plus Old-to-Young pointers recorded in a **card table**, one byte per 512-byte region of the old generation, kept up to date by a *write barrier* on every reference assignment `a.f = b`), and copies *only the live objects* into a clean Survivor space. If 98% of Eden died, copying the 2% that survived is fast; dead objects cost zero work per object.
2. **Promotion to Old (Tenured) Generation:** Objects that survive several Minor GC cycles (tracked via the 4-bit age counter in their 16-byte Mark Word) get *promoted* into the much larger Old Generation.
3. **Major / Full GC and Stop-the-World (STW) Pauses:** This is where latency-sensitive backend services suffer. In Java 6's standard collectors (`ParallelGC` and `ConcurrentMarkSweep` / CMS), when Old Generation fills up or becomes too fragmented to fit a promoted array, the JVM must pause *every application thread simultaneously* (**Stop-the-World**) to mark live graphs across millions of heap pointers and slide surviving objects together to eliminate fragmentation, rewriting every pointer in the process.

In production

**Why financial order books and caches break the generational hypothesis.** Suppose your Java 6 payment gateway or matching engine keeps 5,000,000 active limit orders and 2,000,000 recent idempotency records in memory. Those objects do *not* die in ten milliseconds — they live for two minutes or two hours, easily surviving Minor GC and promoting into Old Generation, and *then* die as orders fill or expire. That medium-lifetime churn is the worst-case workload for generational GC: Old Generation continually accumulates millions of dead

48-byte holes, forcing periodic Stop-the-World compaction pauses of **50 ms to 500+ ms** (or seconds on large heaps).
Even modern low-pause collectors introduced years after Java 6 (G1, ZGC, Shenandoah, Go's concurrent tri-color collector) do not make garbage collection free: to avoid long pauses, they trade **CPU throughput** (running concurrent read/write barriers on pointer loads and stores plus background marker/relocator threads) and **headroom RAM** (1.5× to 2.5× live data size so allocation never outpaces concurrent sweeping). Languages without a tracing GC — C, C++, and Rust — pay **no GC barriers, no GC pauses, and no GC headroom multiplier** by tying deallocation deterministically to ownership scopes.

## 1.4 Value Semantics vs. Reference Semantics

Once you can picture stack slots and heap blocks as numbered byte ranges, one of the oldest confusions in programming — *value semantics* versus *reference semantics* — turns into simple arithmetic on bytes.

Look at two pairs of lines in Java 6 that have identical surface syntax (`b = a;`) and completely opposite behavior:

```
// Pair 1: primitive values
long a = 10000L;
long b = a;       // copies the 8-byte integer 10000
b = 25000L;       // mutates b's stack slot; a is STILL 10000!

// Pair 2: object references
Account acctA = new Account(42L, 10000L);
Account acctB = acctA;   // copies the 8-byte ADDRESS, not the Account!
acctB.debit(4000L);      // mutates the shared heap object; acctA sees 6000!
```

**Definition 1.1** (Value semantics vs. reference semantics)**.** A type exhibits **value semantics** when variables hold independent values directly: assigning `b = a` produces two separate copies of the state, so mutating `b` can never alter `a`. What matters about a value is *only its contents* (two $100 bills with the same value are interchangeable).

A type exhibits **reference semantics** when variables hold *handles* (addresses/pointers) to an identity-bearing storage cell elsewhere: assigning `b = a` copies only the handle, creating a second name (**an alias**) for the *exact same* underlying object, so mutating through `b` immediately changes what `a` observes.

In Java 6, you never get to choose between value semantics and reference semantics for your own types: the eight primitives always have value semantics, and *every class or array always has reference semantics*. Even when Java programmers want value-like behavior for a class — as with `java.lang.String` or `java.math.BigDecimal` — they have to fake it by making every field `final` and heap-allocating a brand-new object on every single addition or substring.

In C, C++, and Rust, value versus reference is **orthogonal to whether a type is primitive or user-defined**. Look at the same `Account` struct in C:

```
Account a = { .id = 42, .balance_cents = 10000 };
```

```
2
3  /* 1. Value copy: copies all 16 bytes onto b's stack slot */
4  Account b = a;
5  b.balance_cents -= 4000;
6  /* Here a.balance_cents is STILL 10000; b.balance_cents is 6000! */
7
8  /* 2. Explicit pointer (reference semantics when you ask for it): */
9  Account *p = &a;          /* p holds the 8-byte address of a */
10 p->balance_cents -= 4000; /* mutates a directly -> a.balance_cents is 6000 */
```

Nothing is hidden behind the type declaration: `Account` means "the 16 bytes of the account itself," and `Account*` means "an 8-byte address pointing to an account somewhere else."

### 1.4.1 Clearing up "Java passes objects by reference"

Almost every Java developer has heard someone say "Java passes primitives by value and objects by reference," and then watched a beginner get burned by a method like this:

```
1  public static void resetAndDebit(Account acct) {
2      acct.debit(2000L);              // Effect 1: caller SEES this!
3      acct = new Account(99L, 0L);    // Effect 2: caller NEVER sees this!
4  }
5
6  // Caller:
7  Account mine = new Account(42L, 10000L);
8  resetAndDebit(mine);
9  // What are mine.id and mine.balanceCents here?
10 // Answer: mine.id == 42, mine.balanceCents == 8000!
```

Why did Effect 1 reach the caller while Effect 2 vanished? Because **Java 6 never passes objects at all — it passes object references by value.** Trace the stack bytes:

1. In the caller's stack frame, local variable `mine` is an 8-byte slot holding address `0x55A8_9C00`.
2. Calling `resetAndDebit(mine)` copies those 8 bytes (`0x55A8_9C00`) into a brand-new 8-byte parameter slot named `acct` in `resetAndDebit`'s stack frame.
3. Line 1 (`acct.debit(2000L)`) follows address `0x55A8_9C00` to the heap object and subtracts `2000` from its `balanceCents` field. Because `mine` still points to `0x55A8_9C00`, `mine` sees `8000`.
4. Line 2 (`acct = new Account(99L, 0L)`) allocates a second object at `0x55B0_0400` and overwrites *only the callee's local parameter slot* `acct` with `0x55B0_0400`. The caller's stack slot `mine` was never touched.

**Common trap**

**Confusing "mutating the pointee" with "rebinding the caller's variable."** In C, passing `Account *acct` does the exact same thing as Java's `Account acct`: `acct->balance_cents -= 2000` mutates the pointee, while `acct = NULL` only zeroes the local pointer copy. To rebind the caller's pointer in C, you must pass a pointer-to-pointer (`Account **acct`); in C++ (Chapter 2) or Rust (Chapter 3), you pass a true reference to the variable (`Account*&` or `&mut Account`). Keeping "the 8-byte slot on the stack" distinct from "the bytes on the heap at that

address" makes every parameter-passing rule across C, C++, Java, Go, and Rust completely transparent.

## 1.5 Aliasing + Mutation: Why Shared Mutable State Breaks Local Reasoning

Now we reach the idea that the rest of this book keeps returning to. Ask yourself: why did unrestricted reference semantics in Java 6 and raw pointers in C/C++ produce decades of subtle production bugs, even in code written by careful, experienced teams?

It is *not* aliasing by itself. Ten variables pointing to the same immutable `java.lang.String` `"USD"` never cause a bug; nobody can change the characters underneath anyone else, so sharing is 100% safe and saves memory.

And it is *not* mutation by itself. Mutating a local primitive counter `long sum = 0; sum += x;` inside a function never surprises anyone, because no other part of the program holds a pointer to `sum`.

Focus

**The Fundamental Law of Stateful Bugs.** Memory corruption, iterator invalidation, representation exposure, lost updates, and multithreaded data races all require **two ingredients at the exact same instant**:

**Aliasing** (two or more active paths to the same memory)
**AND** **Mutation** (at least one path writes to it).

Eliminate *either* half of that conjunction at any given point in time — allow many readers with zero writers, *or* allow one unique writer with zero other readers — and the entire class of bugs becomes structurally impossible.

Before we see how Rust turns that law into a compile-time type checker in Chapter 3, you need to recognize how **Aliasing + Mutation** bites you in ordinary Java 6 and C code — first on a *single thread* with zero concurrency involved, and then across multiple threads.

### 1.5.1 Three single-threaded aliasing bugs

Newcomers often assume aliasing only matters when multiple threads run in parallel. Not true. Here are three classic single-threaded bugs caused purely by mutating state through one alias while reading or holding invariants through another.

**Bug 1: Mutating a collection while iterating over it.** Every Java 6 developer has triggered `java.util.ConcurrentModificationException` on a single thread with no threads in sight:

```
// Single-threaded Java 6: crashes at runtime!
public void cancelZeroQuantityOrders(List<Order> orders) {
    for (Order o : orders) {        // Alias 1: hidden Iterator reading orders
        if (o.getQuantity() == 0) {
```

```
            orders.remove(o);       // Alias 2: mutating orders directly!
        }
    }
}
```

Look at the two aliases: the `for`-each loop creates an `ArrayList$Itr` object holding an internal index `cursor` into `orders`, while the loop body reaches around the iterator and mutates `orders` directly via `orders.remove(o)`, shifting all subsequent elements left. In Java 6, `ArrayList` defends itself at runtime by incrementing an integer `modCount` on every structural change and checking `modCount == expectedModCount` on every `next()` call, crashing your request in production. In C or C++ (as we will dissect in Chapter 2), the equivalent mistake — holding a pointer or iterator into a dynamic array while mutating/growing it — has no `modCount` net: it reads freed heap memory silently.

**Bug 2: Hidden parameter aliasing (`from == to`).** Look at an innocent-looking ledger transfer method where two parameters have the same type:

```
// Buggy if caller ever passes transfer(acct, acct, 5000L)!
public void transfer(Account from, Account to, long cents) {
    long fromStart = from.getBalanceCents();
    long toStart   = to.getBalanceCents();
    if (fromStart < cents) {
        throw new IllegalStateException("insufficient funds");
    }
    from.setBalanceCents(fromStart - cents);
    to.setBalanceCents(toStart + cents); // overwrites debit if from == to!
}
```

When you read those eight lines locally, you picture two boxes on a whiteboard labeled `from` and `to`. Your eyes assume distinct variable names mean distinct objects. The moment a user submits a self-transfer from checking account `#42` to checking account `#42`, `from` and `to` alias the *exact same heap address*: `from.setBalanceCents(5000)` writes `5000`, and one nanosecond later `to.setBalanceCents(15000)` overwrites that exact memory word with `15000`, so the account ends up $50.00 richer than it started.

**Bug 3: Representation exposure (leaking internal mutable references).** Suppose you try to build an "immutable" Java 6 `SettlementBatch` class by marking its fields `private final`:

```
public final class SettlementBatch {
    private final Date cutoffTime;        // java.util.Date is MUTABLE!
    private final List<Order> orders;     // java.util.List is MUTABLE!

    public SettlementBatch(Date cutoffTime, List<Order> orders) {
        this.cutoffTime = cutoffTime;
        this.orders = orders;
    }
    public Date getCutoffTime()   { return cutoffTime; }
    public List<Order> getOrders(){ return orders; }
}
```

In Java 6, `final` on a reference field only freezes *the 8-byte pointer inside `SettlementBatch`* — it does zero to freeze the object at the other end of that pointer. Any caller who invokes `batch.getCutoffTime().setTime(0L)` or `batch.getOrders().clear()` mutates the private guts of your

"immutable" batch from across the codebase. Defending against this in Java 6 forces defensive heap cloning (`new Date(cutoffTime.getTime())` and wrapping a copied `ArrayList` inside `Collections.unmodifiableList(...)`) at every constructor and getter boundary.

### 1.5.2 Multi-threaded shared mutability and Java 6's detached locks

Now add a second OS thread. In Java 6, any heap object reachable from two threads can be read and mutated by both simultaneously unless the programmer manually coordinates access using `synchronized` or `java.util.concurrent.locks`.

When two threads mutate a shared `Account` without synchronization, two hardware realities collide with your source code:

1. **Non-atomic read-modify-write.** `balanceCents -= cents` is three machine operations: load `[rdi+16]` into `rax`, subtract `rsi`, store `rax` back to `[rdi+16]`. Two threads interleaving those three instructions against a $100 balance both load $100, both subtract $80, and both store $20 — losing an $80 withdrawal completely (and on 32-bit Java 6 JVMs, an unsynchronized 64-bit `long` or `double` store wasn't even guaranteed atomic, allowing one thread to read the high 32 bits of one write and the low 32 bits of another: *word tearing*).
2. **Store buffers and visibility reordering.** Each CPU core buffers outgoing writes in a private hardware **store buffer** before flushing them to L1/L3 cache, and both the JIT compiler and the CPU reorder independent memory loads/stores unless constrained by memory barriers (`volatile` or lock acquire/release). Without a *happens-before* edge in the Java Memory Model, Thread B can read stale cached values indefinitely or observe writes from Thread A in an order Thread A never executed them.

Java 6 provides `synchronized` to establish mutual exclusion and a happens-before memory barrier (acquiring the lock inside the target object's 8-byte Mark Word). Now notice *where* Java 6 attaches the lock versus the data:

```
public class LedgerAccount {
    private final Object lock = new Object();
    private long balanceCents;

    public void credit(long cents) {
        synchronized (lock) { balanceCents += cents; }
    }

    // Six months later, a teammate adds a fast reporting getter:
    public long getBalanceCents() {
        return balanceCents; // OOPS: forgot synchronized(lock)! Compiles fine!
    }
}
```

Look at what compiled without a single warning: **in Java 6 (and C++), a lock is physically and type-theoretically detached from the memory it protects.** The relationship "`lock` guards `balanceCents`" lives solely in a comment or inside the original author's head. Worse still, if a transfer method acquires two locks dynamically — `synchronized (from) { synchronized (to) { ... } }` — and Thread 1 calls `transfer(alice, bob, 100)` at the same instant Thread 2 calls `transfer(bob, alice, 50)`, Thread 1 holds `alice` waiting for `bob` while Thread 2 holds `bob` waiting

for `alice`: an instant, permanent **deadlock** (unless locks are always acquired in a strict canonical order such as `accountId`, a pattern we will use both in memory and in PostgreSQL `SELECT ... FOR UPDATE` in Chapter 14).

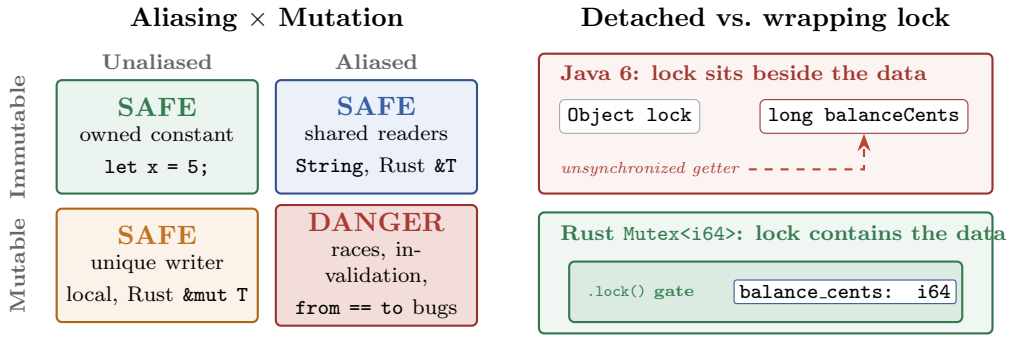



Figure 1.3: Left: Every aliasing/concurrency bug lives exclusively in the bottom-right quadrant (*Aliased + Mutable*). Right: In Java 6 and C++, locks sit alongside fields by convention and can be accidentally bypassed; in Rust, `Mutex<T>` physically wraps `T` so the bytes cannot even be named without acquiring the lock guard.

## 1.6 Dishonest Signatures: `null`, Exceptions, and Hidden Control Flow

We have inspected the hardware cost of heap objects and the reasoning cost of shared mutability. Now look at the third foundational crack in Java 6's design: **you cannot trust a method's type signature to tell you what the method returns or how it exits.**

Here is a one-line Java 6 repository interface from a payment system:

```
public interface OrderRepository {
    Order findById(long orderId) throws SQLException;
}
```

Ask a deceptively simple question: *if I call `Order o = repo.findById(771L);`, what can happen?*

The return type says `Order`. In mathematical reality, `Order` is a lie in two independent directions:

### 1.6.1 Crack 1: Tony Hoare's billion-dollar `null`

In 1965, while designing ALGOL W, Tony Hoare introduced the null reference because it was easy to implement in hardware as the zero address `0x0`. Forty-four years later he famously apologized for it as his "billion-dollar mistake."

Why is Java 6's `null` so destructive to type systems? Not because "absence of a value" is a bad concept — looking up order `771` when no such order exists *genuinely produces an absent result.* The mistake in Java 6 (and C pointers) is **making `null` a silent legal value of *every* reference type `T` simultaneously**.

Look at what that conflation breaks:

- **You cannot express "definitely present `Order`" in Java 6's type system.** Whether `findById` returns `null` when an order is missing, or throws `OrderNotFoundException` and *never* returns `null`, has identical type `Order`. Every caller either litters the codebase with paranoid `if (o != null)` checks or forgets one and throws a `NullPointerException` at 3 a.m.
- **The crash happens far from the bug.** If a method accidentally stores `null` into `order.side` during construction at 14:00:01, nothing fails at 14:00:01. Ten minutes later, an end-of-day settlement job reads `order.getSide()` and blows up with `NullPointerException` at a line that has no idea who put `null` there.

### 1.6.2 Crack 2: Exceptions as invisible gotos across stack frames

Now look at the error side of `findById`. Java 6 actually tried something ambitious with **checked exceptions** (`throws SQLException`): forcing the signature to advertise recoverable failures. Every experienced Java 6 engineer knows what happened next in real codebases:

1. **Unchecked exceptions bypass the signature.** Alongside checked `SQLException`, a call to `findById` can throw `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, Spring/Hibernate's unchecked `DataAccessException`, or `OutOfMemoryError` — none of which appear anywhere in the signature.
2. **Interface composition breaks down.** When you pass a callback or implement a generic interface (like `Runnable.run()` or `Callable<T>`) whose signature did not declare `throws SQLException`, checked exceptions become illegal to propagate cleanly. Developers retaliate with two anti-patterns: wrapping everything into unchecked `throw new RuntimeException(e)` (erasing the type information) or, far worse, swallowing errors via `catch (Exception e) { log.error(e); }` and continuing with half-mutated state.
3. **Non-local stack unwinding fractures invariants.** Every single method call in a 20-line Java 6 function is an invisible conditional `goto` that may abort execution mid-way and jump five stack frames up, leaving partially mutated heap objects behind unless wrapped carefully in rollback logic (and paying microsecond-scale `fillInStackTrace()` costs whenever exceptions are misused for expected business outcomes like "insufficient funds").

### 1.6.3 Preview: Honest signatures with flat value types

How do modern type systems heal both cracks at once without returning to C's ignorable `-1` integer error codes? By separating "a present `Order`" from "maybe an `Order`" and "either a result or an error," and representing both as **ordinary, flat values returned through the single normal return path**:

```
// Rust: 100% of outcomes live right inside the return type!
pub fn find_by_id(order_id: OrderId) -> Result<Option<Order>, DbError>
```

Read that return type from outside in like plain English:

- Reaching the database can fail with `DbError` (`Err(e)`) or succeed (`Ok(...)`): that outer layer is `Result<T, DbError>`.

- When the query succeeds, row `order_id` might be absent (`None`) or present (`Some(order)`): that is `Option<Order>`.
- And once you unwrap `Some(order)`, `order` is a guaranteed-valid 32-byte `Order` value — *never* `null`. Even better, you cannot accidentally read `order.price_cents` without first handling both `Err` and `None`, because `Result` and `Option` are distinct sum types (`enums`), not `Order` itself.

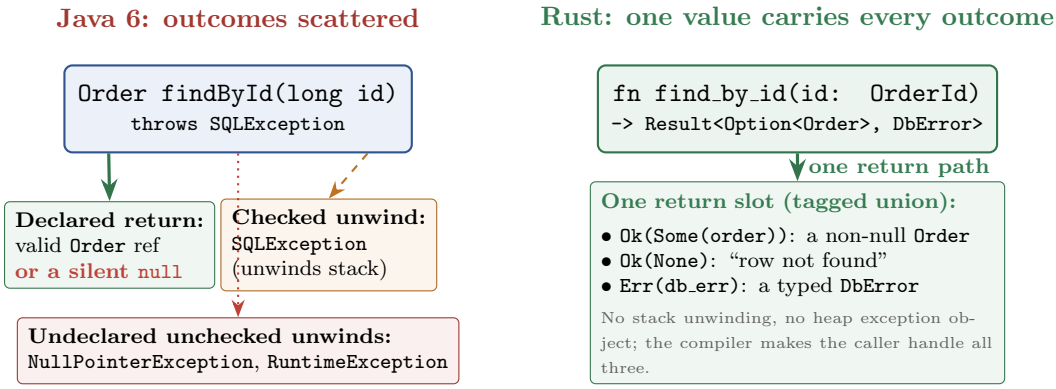



Figure 1.4: Left: A Java 6 method splits its actual outcomes across a nullable reference return, declared checked exceptions, and undeclared runtime exceptions that unwind stack frames. Right: Algebraic types (`Result` and `Option`, covered deeply in Chapter 5) fold every outcome into one explicit, compiler-checked value on the stack.

**Pause**

**Take stock of the three lenses you just gained.** Whenever you read code in *any* language from here on, ask three questions automatically:

1. **Where are the bytes?** Are values inline on the stack / contiguous in one array, or scattered across 16-byte-header heap islands connected by pointers across 64-byte cache lines?
2. **Who else can touch these bytes right now?** Is this value unaliased, shared read-only, or sitting in the *Aliased + Mutable* danger zone? And if a lock exists, does the type system enforce acquiring it?
3. **Does the type signature tell the whole truth?** Can `null` sneak inside a "present" type, or can hidden exceptions jump past cleanup logic?

In Chapter 2, we step into **Modern C++ (C++11–C++20)** — the language that mastered deterministic stack-driven cleanup (RAII), contiguous zero-overhead containers, and move semantics, while leaving just enough compile-time blind spots to show you *why* Rust's borrow checker had to be invented next.

## 1.7 Exercises

**Exercise 1.1** (Exact JVM vs. flat struct memory budget)**.** A pricing service holds 2,000,000 `Quote` records in memory. Each quote consists of `long instrumentId`, `long bidCents`, `long askCents`,

`int bidSize`, and `int askSize`.

(a) Calculate the exact byte size of a single `struct Quote` in C or Rust (`#[repr(C)]`), including any padding bytes, and the total memory consumed by a contiguous `Vec<Quote>` of 2,000,000 elements (assume capacity equals length).

(b) Calculate the exact memory consumed by `ArrayList<Quote>` of 2,000,000 elements on a 64-bit HotSpot JVM *without* Compressed OOPs (16-byte object header, 8-byte references, 8-byte external object alignment), counting both the backing `Object[]` array and the 2,000,000 `Quote` heap instances.

(c) Repeat part (b) *with* Compressed OOPs enabled (12-byte object header, 4-byte references, 8-byte external alignment). How many `Quote` structs fit inside a single 64-byte CPU cache line in Rust/C versus Java 6?

**Exercise 1.2** (Struct field ordering and alignment holes)**.** Using C's alignment rules (`uint8_t` aligns to 1B, `uint16_t` to 2B, `uint32_t` to 4B, `uint64_t` to 8B, and total `sizeof` rounds up to a multiple of the struct's maximum field alignment), compute `offsetof` for every field and total `sizeof` for:

```
struct BadPosting {
    uint8_t  is_credit;
    uint64_t journal_id;
    uint16_t currency_num;
    int64_t  amount_cents;
    uint32_t sequence_no;
};
```

Then rearrange the exact same five fields so `sizeof(struct GoodPosting)` is as small as possible. How many megabytes do you save across 10,000,000 postings in a flat array?

**Exercise 1.3** (Cache-line arithmetic on an order book scan)**.** Suppose L1/L2/L3 caches are cold, sequential DRAM streaming bandwidth on one core is 40 GB/s, and each random uncached DRAM pointer dereference costs an effective 15 ns (accounting for 80 ns raw DRAM latency divided by $\sim 5.3$ overlapping out-of-order loads). Estimate the wall-clock time to scan 4,000,000 32-byte `Order` records stored (a) contiguously in a C++/Rust array versus (b) as randomly scattered 48-byte heap objects referenced from a Java 6 `ArrayList<Order>`.

**Exercise 1.4** (Tracing Java 6 pass-by-value of object references)**.** Without running a compiler, determine the exact four numbers printed at the end of `run()` and draw the stack frame slots and heap objects after each statement:

```
static void swapAndCredit(Account x, Account y) {
    x.credit(500L);
    Account tmp = x;
    x = y;
    y = tmp;
    x.credit(1000L);
}

static void run() {
    Account a = new Account(1L, 2000L);
    Account b = new Account(2L, 3000L);
    swapAndCredit(a, b);
    System.out.println(a.getId() + ":" + a.getBalanceCents()
```

```
                        + " " + b.getId() + ":" + b.getBalanceCents());
}
```

**Exercise 1.5** (Fixing representation exposure in Java 6)**.** Look back at `SettlementBatch` in §1.5. Even if the constructor copies the list into a fresh `ArrayList<Order>(orders)` and `getOrders()` wraps it with `Collections.unmodifiableList(...)`, under what condition on the `Order` class itself could an external caller *still* mutate the financial contents of a `SettlementBatch`? Write down the complete set of conditions required for deep immutability in Java 6, and explain why value semantics makes defensive copying unnecessary when passing a flat `struct` by value in C or Rust.

**Exercise 1.6** (Two bugs in naive `synchronized` transfer)**.** Consider this Java 6 thread-safe transfer attempt:

```
public static void transfer(Account from, Account to, long cents) {
    synchronized (from) {
        synchronized (to) {
            if (from.getBalanceCents() < cents) {
                throw new IllegalArgumentException("low funds");
            }
            from.setBalanceCents(from.getBalanceCents() - cents);
            to.setBalanceCents(to.getBalanceCents() + cents);
        }
    }
}
```

(a) Construct a two-thread execution schedule using accounts `acct1` (`id = 1`) and `acct2` (`id = 2`) that deadlocks permanently, naming which lock each thread holds and waits for.

(b) What happens in Java 6 when a single thread calls `transfer(acct1, acct1, 500L)` — does `synchronized (from)` followed by `synchronized (to)` deadlock on a single thread, or are Java intrinsic monitors *reentrant*? And does the balance end up corrupted or preserved in *this* version compared to the cached-local version in §1.5?

(c) Rewrite `transfer` in Java 6 so it neither deadlocks when two threads transfer in opposite directions nor allows self-transfers to misbehave, using canonical lock ordering on `getId()`.

**Exercise 1.7** (Why medium-lived caches hurt generational GC)**.** A Java 6 service processes 20,000 payment requests per second. Each request allocates 10 KB of short-lived JSON parsing objects that die within 2 ms, *plus* inserts a 512-byte `IdempotencyEntry` into an in-memory LRU cache with a 10-minute TTL. Calculate the allocation rate in MB/s for both categories, trace which generation (Eden vs. Old Generation) each category dies in if Minor GC runs every 2 seconds and promotes objects after 4 survivals, and explain why the smaller allocation stream ($20{,}000 \times 512$ B/s $\approx 10$ MB/s) causes virtually all of the Stop-the-World Full GC pauses.

**Exercise 1.8** (Designing an honest lookup-and-debit contract)**.** In a Java 6 codebase you find:

```
// Returns updated balance, or null if account is frozen!
Long debitAccount(String accountNumber, long cents) throws SQLException;
```

List at least five distinct outcomes (success, domain rejections, missing account, infrastructure failures, invalid arguments) currently crammed into `Long`, `null`, checked `SQLException`, and undocumented runtime exceptions. Sketch a clean algebraic return type in Rust (or a Java 6 tagged result

class with zero `null` fields) where a caller can distinguish every outcome explicitly by inspecting the returned value.

---

# Chapter 2

# Modern C++: RAII, Moves, Smart Pointers, and Templates

Chapter 1 left us facing an uncomfortable engineering tension. On one side stood Java 6: memory-safe against stray pointer corruption, yet paying a steep runtime tax in 16-byte object headers, pointer-chasing cache misses across scattered heap allocations, Stop-the-World garbage collection pauses, unrestricted aliasing, and dishonest `null`/exception signatures. On the other side stood classic C: flat, predictable byte layouts and zero garbage-collection pauses, yet relying entirely on human discipline to pair every `malloc` with exactly one `free` and never dereference a dead pointer.

To see where Rust got its best ideas and where it broke new ground, you need to meet the language that sits directly between classic C and Rust: **Modern C++** (specifically C++11 through C++20).

If your mental image of C++ stops at "C with classes, `new`, and `delete`," Modern C++ looks quite different. Well-written C++20 code almost never contains a raw `new` or `delete`. Instead, it manages memory, files, locks, and database transactions deterministically with zero garbage collector via **RAII (Resource Acquisition Is Initialization)**; transfers ownership of megabyte buffers in three register instructions via **move semantics**; expresses single versus shared heap ownership in types via **smart pointers**; passes zero-copy slices via `std::span` and `std::string_view`; and compiles generic algorithms down to flat, specialized machine loops via **monomorphized templates** and **C++20 Concepts**.

Every single one of those mechanisms carried straight over into Rust. Yet Modern C++ also left four specific, sharp compile-time blind spots — non-destructive moves leaving hollowed-out shells, dangling views, single-threaded `std::vector` reference invalidation, and unchecked shared mutability — where clean-looking C++20 code without a single raw pointer still triggers undefined behavior. In this chapter we first learn what Modern C++ does well, and then watch those four blind spots motivate Part II of this book.

---

## 2.1 From `malloc`/`free` and `finalize()` to Deterministic RAII

To see why RAII matters so much in systems programming, start by watching how resource cleanup fails in both classic C and Java 6 when a function manages more than one resource across multiple exit paths.

### 2.1.1 Why manual C cleanup and Java 6 `finalize()` both fall short

Suppose a C function opens a temporary audit file, allocates a scratch buffer, and begins a PostgreSQL transaction to record a settlement batch:

```c
/* Classic C: every early return must remember exact reverse cleanup! */
int settle_batch_c(PGconn *db, const char *audit_path, size_t n_orders) {
    FILE *fp = fopen(audit_path, "a");
    if (fp == NULL) return -1;

    uint8_t *buf = malloc(n_orders * 64);
    if (buf == NULL) {
        fclose(fp);          /* forget this -> file descriptor leak! */
        return -2;
    }

    if (pg_begin(db) != 0) {
        free(buf);
        fclose(fp);
        return -3;
    }

    if (validate_and_post(db, buf, n_orders) != 0) {
        pg_rollback(db);     /* forget this -> connection stuck in txn! */
        free(buf);
        fclose(fp);
        return -4;
    }

    pg_commit(db);
    free(buf);
    fclose(fp);
    return 0;
}
```

Count the traps in twenty-five lines of C. With three resources ($R_1, R_2, R_3$) and four exit paths, cleanup logic is duplicated across every branch (seasoned C kernel programmers resort to `goto cleanup_txn;` labels at the bottom of the function just to keep cleanup in one place). Worse, six months later a maintainer adds `if (batch_expired()) return -5;` in the middle of the function and leaks a file descriptor, a heap buffer, *and* an open PostgreSQL transaction holding row locks in production. And if an error branch accidentally calls `free(buf)` twice (**double free**) or reads `buf[0]` after `free(buf)` (**use-after-free**), the allocator's internal free-list metadata is corrupted.

Now ask: did Java 6's garbage collector actually solve this problem? **Only for RAM — and it actively made non-memory resources harder to reason about.**

A running server manages many scarce resources besides heap bytes: OS file descriptors (often capped at 1,024 or 65,536 per process), TCP sockets, mutex locks, off-heap direct byte buffers, and pooled PostgreSQL connections. Java 6 originally offered `protected void finalize()` on `Object`, tempting developers to close sockets or database connections inside `finalize()`. That failed badly in production:

1. **Garbage collection triggers on heap pressure, not file-descriptor exhaustion.** You

can open 10,000 tiny 48-byte `FileInputStream` wrappers consuming less than 500 KB of a 16 GB Eden space. The GC sees plenty of free RAM and never runs, while the Linux kernel refuses the 1,025th `open()` syscall with `EMFILE (Too many open files)`, taking down your service.

2. **No timeliness, ordering, or even execution guarantee.** The JVM specification makes zero promise about *when* — or even *whether* — a finalizer runs, and objects with finalizers survive extra GC cycles and slow allocation down by orders of magnitude.

So Java 6 programmers had to fall back on manual `try/finally` blocks (Java 7 later added `try`-with-resources to automate this exact boilerplate, though still only at explicit `try(...)` sites rather than automatically across arbitrary fields and containers):

```
// Java 6 manual cleanup: verbose, and close() can mask the real exception!
Connection conn = pool.getConnection();
try {
    conn.setAutoCommit(false);
    postJournalEntries(conn, batch); // suppose this throws SQLException A!
    conn.commit();
} catch (SQLException e) {
    conn.rollback(); // what if the socket dropped and rollback() throws B?!
    throw e;
} finally {
    conn.close();    // if close() throws C, exception A is LOST in Java 6!
}
```

### 2.1.2 Bjarne Stroustrup's insight: tie every resource to a stack object's lifetime

C++ solved memory, files, sockets, locks, and database transactions with one unified mechanism that requires zero garbage collector and zero `finally` blocks. Bjarne Stroustrup gave it an admittedly clunky name — **Resource Acquisition Is Initialization (RAII)** (sometimes more clearly called *Scope-Bound Resource Management*) — around a simple physical rule:

Formal

**The RAII Contract in C++.**

1. **Acquire in the constructor:** Encapsulate each raw resource handle (heap pointer, file descriptor, mutex, database transaction) inside a small stack-allocated `class` or `struct` whose constructor acquires the resource or fails immediately before the object ever exists.
2. **Release unconditionally in the destructor (`~T()`):** For every local variable of type `T` in a block { ... }, the C++ compiler *automatically* emits a call to `T`'s destructor `~T()` at the exact closing brace }, in **strict reverse order of construction**, across *every possible exit path*: falling off the end, early `return`, `break`/`continue`, or stack unwinding when an exception is thrown.
3. **Composition is automatic:** When a `struct SettlementJob` contains three RAII fields (`FileHandle`, `ByteBuffer`, `PgTransaction`), the compiler automatically generates `~SettlementJob()` to destroy all three fields in reverse declaration order.

Watch what happens to our brittle C settlement function when we wrap a PostgreSQL transaction in a twenty-line C++ RAII guard:

```
// Modern C++ RAII guard for a PostgreSQL transaction
class PgTransaction {
    PGconn* db_;
    bool    committed_;

public:
    explicit PgTransaction(PGconn* db) : db_(db), committed_(false) {
        if (pg_begin(db_) != 0) {
            throw DbError("BEGIN failed");
        }
    }

    // Forbid copying a live transaction handle! (Explained in Section 2.2)
    PgTransaction(const PgTransaction&)            = delete;
    PgTransaction& operator=(const PgTransaction&) = delete;

    void commit() {
        if (pg_commit(db_) != 0) {
            throw DbError("COMMIT failed");
        }
        committed_ = true;
    }

    // Destructor: ALWAYS runs at '}' unless commit() already succeeded!
    ~PgTransaction() noexcept {
        if (!committed_) {
            pg_rollback_quiet(db_); // never throw out of a destructor!
        }
    }
};
```

Now rewrite `settle_batch` using RAII types (`std::ofstream` for the file, `std::vector<uint8_t>` for the heap buffer, and `PgTransaction` for the database transaction):

```
void settleBatchCpp(PGconn* db, const char* auditPath, size_t nOrders) {
    std::ofstream        auditFile(auditPath, std::ios::app);
    std::vector<uint8_t> buf(nOrders * 64);
    PgTransaction        txn(db);

    if (nOrders == 0) {
        return; // ~txn rolls back, ~buf frees RAM, ~auditFile closes fd!
    }
    validateAndPost(db, buf.data(), nOrders); // if this throws, same 3 run!
    txn.commit(); // sets committed_ = true
} // ~txn sees committed_ == true (no-op); ~buf frees RAM; ~auditFile closes!
```

Notice what is missing from those ten lines: there is not a single `free`, `fclose`, `rollback`, or `finally` anywhere at the call site, yet every exit path — early `return`, thrown exception, or normal completion — releases the transaction, the heap buffer, and the file descriptor in exact reverse order (`txn`, then `buf`, then `auditFile`) at the moment control leaves the scope. (Remember this exact `PgTransaction` pattern: when we reach `sqlx::Transaction<'_, Postgres>` in Chapter 14, its Rust

`Drop` implementation uses this exact uncommitted-rollback design to keep money safe even when an HTTP client disconnects mid-request.)

**Common trap**

**Never allow an exception to escape a C++ destructor (`noexcept`).** Notice the keyword `noexcept` on `~PgTransaction() noexcept`. Why is that mandatory in C++ (in fact, C++11 implicitly marks all destructors `noexcept` by default)? Suppose `validateAndPost` throws an exception `E1`, and while the runtime is *already* unwinding the stack, `~PgTransaction()` throws a *second* exception `E2`. C++ cannot propagate two simultaneous active exceptions on one thread, so the runtime immediately aborts the entire process via `std::terminate()`. Destructors must log or swallow cleanup transport failures quietly; explicit callers who care about commit errors call `txn.commit()` before the scope ends.

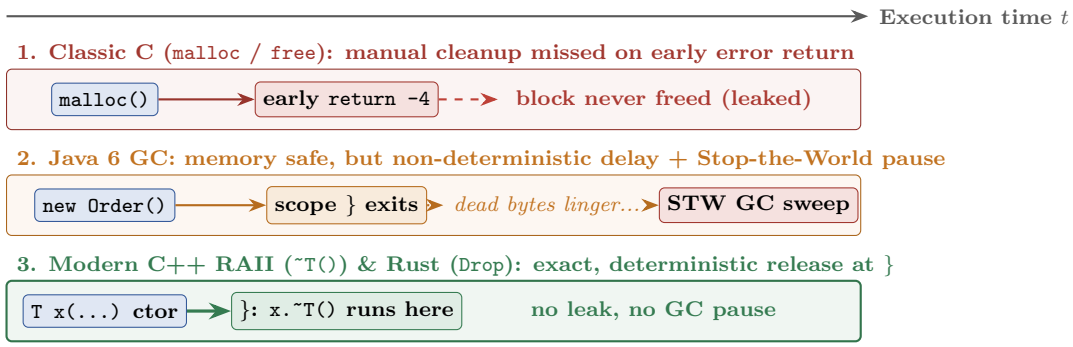



Figure 2.1: Resource lifetime timelines across C, Java 6, and Modern C++/Rust. Deterministic RAII binds arbitrary resources (heap memory, locks, sockets, database transactions) to lexical scope exit.

## 2.2 References, Value Categories, Copies, and Move Semantics

RAII immediately raises a sharp physical question. Suppose a stack object `ByteBuffer` owns a 4 MB heap array and its destructor `~ByteBuffer()` calls `delete[] data_`. What happens when you assign `ByteBuffer b = a;`, pass `a` into a function, or return a `ByteBuffer` from a factory function?

If `b = a` blindly copied the 8-byte pointer `data_` (a *shallow bitwise copy*, the default in C structs), both `a` and `b` would point to the *exact same* 4 MB heap block. At the end of the scope, `~b()` would free that block, and one nanosecond later `~a()` would free the *exact same address a second time* — instant heap corruption.

How C++ solved that problem — first with **copy constructors** in C++98, and then with **rvalue references and move semantics** in C++11 — is the key that unlocks both Modern C++ and Rust. Let us build it step by step, starting with C++ references.

### 2.2.1 Lvalue references: `T&` and `const T&`

In C, whenever you want a function to read a large struct without copying its bytes, or mutate the caller's struct in place, you pass a raw pointer (`const Order *o` or `Account *acct`), which can be

`NULL`, can be reassigned mid-function to point elsewhere, and supports error-prone pointer arithmetic (`acct + 5`).

C++ introduced **lvalue references**, written `T&` (mutable reference) and `const T&` (read-only reference):

```
// 1. Pass by value: copies all 32 bytes onto the callee's frame
void inspectByVal(Order o);

// 2. Pass by const reference: passes an 8B address under the hood;
//    zero copy, cannot be rebound, promises not to mutate o!
int64_t notionalCents(const Order& o) {
    return o.price_cents * static_cast<int64_t>(o.quantity);
}

// 3. Pass by mutable reference: mutates caller's Account directly!
void applyCredit(Account& acct, int64_t cents) {
    acct.balance_cents += cents;
}
```

At the machine-code level, `const Order&` and `Account&` compile to the *exact same 8-byte register address* as `const Order*` and `Account*`. At the language level, references give you two clean guarantees raw pointers lack: a reference must be bound to an existing object the instant it is created (there is no "null reference" in well-formed C++), and it can *never be reseated* to refer to a different object later (assigning `r = other` overwrites the target object, not the binding).

### 2.2.2 Lvalues vs. rvalues, and why C++98 wasted gigabytes copying temporaries

Why are `T&` and `const T&` called *lvalue* references? The word goes back to C's mental picture of an assignment `lhs = rhs`:

- An **lvalue** (historically "left-hand-side value," better remembered as a **locator value**) is an expression that designates a persistent storage location with an identity whose address you can take with `&`: a named variable like `acct`, an array element `orders[3]`, or a dereferenced pointer `*p`. Because other code later in the scope can still read a named variable, *you must not silently destroy its contents behind its back.*
- An **rvalue** (an ephemeral temporary or literal without persistent named identity) is an expression about to evaporate at the end of the full-expression (typically the semicolon `;`): an integer literal `42`, the return value of a function call `serializeBatch(orders)`, or an arithmetic temporary `s1 + s2`. *Nobody else has a name for that temporary, and it is milliseconds away from destruction.*

Before C++11, C++ could not reliably distinguish "copying from a persistent named variable `a`" from "initializing from a dying temporary `serializeBatch(orders)`" in user constructors (aside from compiler Return Value Optimization when elision applied). Every heap-owning RAII class had to define a **deep copy constructor** `ByteBuffer(const ByteBuffer& other)` that allocated a brand-new heap array and ran `std::memcpy`. If you inserted a temporary `ByteBuffer` into a `std::map` or reassigned `buf = buildPayload();`, C++98 faithfully allocated a second multi-megabyte buffer, copied every byte out of the dying temporary, and immediately freed the temporary's perfectly good buffer.

### 2.2.3 C++11 Move Semantics: `T&&` and `std::move`

C++11 fixed that waste by adding **rvalue references**, written with two ampersands: `T&&`. Overload resolution obeys a simple, intuitive rule:

- `const T&` binds to anything, and is chosen when the argument is a persistent **lvalue** (`a`) that must be preserved via a deep copy.
- `T&&` binds *only* to modifiable **rvalues** (dying temporaries, or variables explicitly cast with `std::move`), giving the constructor permission to **pilfer (steal) the heap pointer in $O(1)$ time** instead of copying the heap payload.

Let us write a complete `ByteBuffer` class in Modern C++ showing both the deep copy constructor and the $O(1)$ move constructor side by side:

```
#include <cstddef>
#include <cstdint>
#include <cstring>
#include <utility>

class ByteBuffer {
    uint8_t* data_; // 8 bytes (heap pointer, or nullptr when empty/moved-from)
    size_t   len_;  // 8 bytes
    size_t   cap_;  // 8 bytes

public:
    explicit ByteBuffer(size_t n)
        : data_(n > 0 ? new uint8_t[n]() : nullptr), len_(n), cap_(n) {}

    // 1. Destructor (RAII): delete[] nullptr is a guaranteed safe no-op in C++!
    ~ByteBuffer() noexcept {
        delete[] data_;
    }

    // 2. COPY Constructor (lvalue source -> deep O(N) allocation + memcpy)
    ByteBuffer(const ByteBuffer& other)
        : data_(other.cap_ > 0 ? new uint8_t[other.cap_] : nullptr),
          len_(other.len_),
          cap_(other.cap_) {
        if (len_ > 0) {
            std::memcpy(data_, other.data_, len_);
        }
    }

    // 3. MOVE Constructor (rvalue source -> O(1) pointer theft, 0 allocs!)
    ByteBuffer(ByteBuffer&& other) noexcept
        : data_(other.data_), len_(other.len_), cap_(other.cap_) {
        // CRITICAL: hollow out 'other' so ~other() does NOT double-free data_!
        other.data_ = nullptr;
        other.len_  = 0;
        other.cap_  = 0;
    }

    // Copy and Move assignment operators (Rule of Five)
```

```
    ByteBuffer& operator=(ByteBuffer&& other) noexcept {
        if (this != &other) {
            delete[] data_; // free our old buffer first!
            data_ = other.data_; len_ = other.len_; cap_ = other.cap_;
            other.data_ = nullptr; other.len_ = 0; other.cap_ = 0;
        }
        return *this;
    }
    ByteBuffer& operator=(const ByteBuffer& other) {
        if (this != &other) {
            ByteBuffer tmp(other);    // copy-and-swap idiom
            *this = std::move(tmp);
        }
        return *this;
    }

    size_t         size() const noexcept { return len_; }
    const uint8_t* data() const noexcept { return data_; }
};
```

Focus on the move constructor, and then consider what happens when you hold a named local variable `a` that you *know* you are done using and want to hand off to `b` without copying 4 MB:

```
ByteBuffer a(4 * 1024 * 1024);      // allocates 4 MB on heap
ByteBuffer b = a;                   // lvalue -> calls COPY ctor (another 4 MB!)
ByteBuffer c = std::move(a);        // rvalue cast -> calls MOVE ctor (0 allocs!)
```

**Intuition**

`std::move(x)` **does not move anything.** The name is misleading. At runtime, `std::move(a)` executes **zero CPU instructions** and copies **zero bytes**. Its entire standard-library implementation is essentially a compile-time type cast:

`static_cast<typename std::remove_reference<T>::type&&>(x).`

Calling `std::move(a)` merely sticks a yellow "FREE TO PILFER" sticky note onto the lvalue `a` so overload resolution picks `ByteBuffer(ByteBuffer&&)` instead of `ByteBuffer(const ByteBuffer&)`. All actual pointer stealing happens inside the move constructor or move assignment operator that receives that rvalue reference. (In fact, if you call `std::move(x)` on a `const ByteBuffer x`, the sticky note yields `const ByteBuffer&&`, which *cannot* bind to `ByteBuffer&&` and silently falls back to a full deep copy)

**Why it works**

**Why `noexcept` on move constructors is critical for `std::vector` speed.** Suppose a `std::vector<ByteBuffer>` grows from capacity 4 to capacity 8. To preserve its *strong exception guarantee* (if growing fails mid-way, the original 4-element vector must remain intact), `std::vector` asks at compile time: *can `ByteBuffer`'s move constructor throw an exception?* If a move threw on element #2 after elements #0 and #1 had already had their guts stolen out of the old array, rolling back could itself throw and corrupt the vector. Therefore, `std::vector` **only moves elements during reallocation if the move constructor is**

**marked `noexcept`** (or if the type cannot be copied at all) — otherwise it silently falls back to deep-copying every element. Always mark C++ move constructors and move assignment operators `noexcept`.

### 2.2.4 The Central Contrast: C++ Non-Destructive Moves vs. Rust Destructive Moves

Now compare C++'s move model with the Rust move model you will meet in Chapter 3. Many experienced C++ programmers initially assume Rust moves work just like `std::move`. **They do not — and the difference explains half of Rust's ergonomic and safety edge over C++.**

Consider what happens to `a` in C++ after `ByteBuffer c = std::move(a);`:

1. **In C++ (Non-Destructive Move):** Variable `a` is *still a live variable in lexical scope.* When the enclosing curly brace } is reached twenty lines later, the C++ compiler **unconditionally calls `a.~ByteBuffer()`.**

   - Because `~a()` will still run, the move constructor *had* to write `nullptr` and zeros back into `a` so `~a()` would not double-free `c`'s buffer.
   - Every movable C++ class wishing to uphold a "never-empty / never-null" invariant (such as `ValidDbConn` or `NonEmptyPath`) is **forced to invent a fake "hollowed-out zombie" state** solely to survive the gap between `std::move(a)` and `a.~T()`.
   - Worst of all, **the C++ compiler does not stop you from continuing to use `a` after `std::move(a).`** Writing `uint8_t first = a.data()[0];` five lines after `std::move(a)` compiles cleanly without error and segfaults in production by dereferencing `nullptr`.

2. **In Rust (Destructive Move, default on assignment `let c = a;`):**

   - Assigning `let c = a;` for a heap-owning type (like `Vec<u8>` or `String`) is a **move by default** (no `std::move` cast required; deep copies require an explicit `a.clone()` call so $O(N)$ heap allocations are never accidental).
   - At runtime, moving `a` into `c` is just a blind 24-byte `memcpy` of `(ptr, len, cap)` — Rust does *not* even need to zero out `a`'s bytes in the normal static case.
   - Why doesn't `~a()` double-free? Because the Rust compiler marks `a` **statically uninitialized / moved-out** in its control-flow graph: `Drop` **is never called on** `a`, and any attempt to read `a` on any subsequent line is rejected at compile time with `error[E0382]: borrow of moved value`. Consequently, a Rust type that guarantees "my pointer/fd is never null" *never needs a hollowed-out zombie state.*

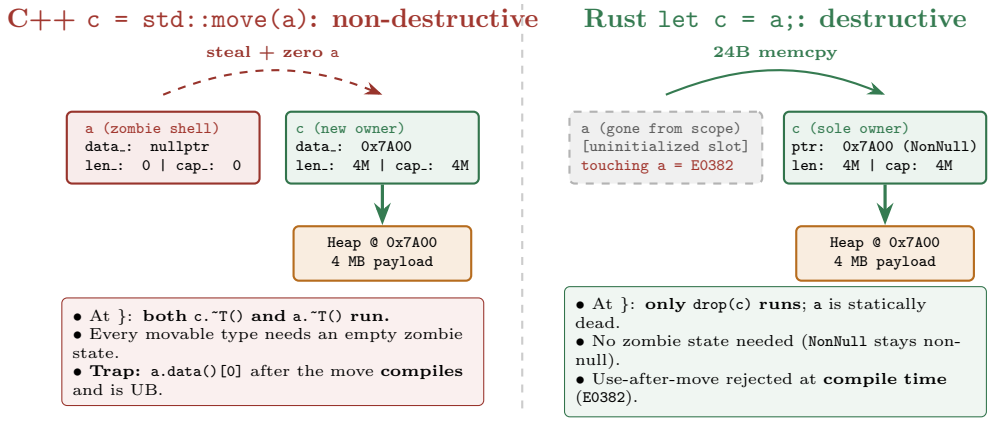



Figure 2.2: Why C++ non-destructive moves and Rust destructive moves differ fundamentally. In C++, moved-from variables stay live and still run destructors; in Rust, moving statically ends the source variable's life.

### 2.2.5 The Rule of Five vs. the Rule of Zero

Writing `ByteBuffer` required defining five special member functions together — destructor, copy constructor, copy assignment, move constructor, and move assignment — known in C++ as the **Rule of Five**: *if a class manages a raw resource directly and needs a custom destructor, you almost certainly must explicitly define or* `= delete` *all five copy/move operations.*

Even better is Modern C++'s everyday guideline, the **Rule of Zero**: *application classes (like* `Order`*,* `Account`*, or* `LedgerService`*) should manage zero raw resources themselves.* Instead, compose your structs out of standard RAII building blocks (`std::vector`, `std::string`, `std::unique_ptr`), and write **zero** custom destructors, copy constructors, or move constructors — the C++ compiler synthesizes exact memberwise RAII destruction, copying, and $O(1)$ moving for you automatically.

## 2.3 Smart Pointers and Non-Owning Views

Look at a raw C pointer `Order* p;` in an older C++ header file and ask: *what does* `p` *mean?*

- Does the caller own `p` and have a duty to call `delete p` (a single object) or `delete[] p` (an array)?
- Is ownership shared among three subsystems so the last one alive must free it?
- Or is `p` merely *borrowing* a temporary look at an `Order` living on someone else's stack frame or inside a `std::vector`, so calling `delete p` would blow up the process?

A bare `Order*` gives you zero clue; five completely different ownership contracts collapse into the same two characters. Modern C++ splits those roles into explicit, self-documenting types: **smart pointers** for heap ownership (`unique_ptr`, `shared_ptr`, `weak_ptr`) and **non-owning views** for borrowed slices (`string_view`, `span`).

### 2.3.1 `std::unique_ptr<T>`: Zero-overhead exclusive heap ownership

Whenever a single owner needs a heap-allocated object (for instance, a polymorphic handler node, a recursive tree child, or a large struct kept at a stable address), `std::unique_ptr<T>` is the Modern C++ default:

```cpp
#include <memory>

void routeOrder() {
    // Allocates Order on the heap; returns move-only unique_ptr<Order>
    std::unique_ptr<Order> ptr = std::make_unique<Order>(
        Order{771, 42, 10000, 5, Side::Buy}
    );

    // std::unique_ptr<Order> copy = ptr; // COMPILE ERROR: copy is = delete!
    std::unique_ptr<Order> nextOwner = std::move(ptr); // O(1) ownership handoff
    // Here ptr == nullptr; nextOwner owns the heap Order!
} // ~nextOwner() calls delete automatically at '}'!
```

Open `std::unique_ptr<Order>` in a debugger or inspect `sizeof(std::unique_ptr<Order>)`:

- **Byte size:** Exactly **8 bytes** (with the default deleter) — just the single raw `Order*` pointer wrapped in a zero-overhead RAII struct.
- **Machine code:** Dereferencing `nextOwner->price_cents` compiles to the *exact same single machine load instruction* as a raw C pointer, and copying is deleted at compile time (`= delete`), making accidental double-frees impossible without touching raw `new`/`delete`. (In Chapter 3 you will meet its direct Rust twin: `Box<T>`.)

### 2.3.2 `std::shared_ptr<T>` and `std::weak_ptr<T>`: Reference-counted shared ownership

What if an immutable `InstrumentConfig` or an open session truly has multiple independent owners whose lifetimes end in unpredictable order (say, three concurrent worker tasks, where the last task to finish must free the object)? That is `std::shared_ptr<T>` (paired with Rust's `Arc<T>` and `Rc<T>` in Chapter 3):

```cpp
auto cfg1 = std::make_shared<InstrumentConfig>("BTC-USD", 2); // strong = 1
{
    std::shared_ptr<InstrumentConfig> cfg2 = cfg1;            // strong = 2
    // Both cfg1 and cfg2 read the same heap InstrumentConfig
} // ~cfg2() atomically decrements strong count: 2 -> 1 (stays alive!)
// When cfg1 later leaves scope: strong count 1 -> 0 -> destroys object!
```

Unlike 8-byte `unique_ptr`, `sizeof(std::shared_ptr<T>)` is **16 bytes** on 64-bit hardware and carries real runtime mechanics you should picture down to the cache line:

1. **Two pointers on the stack (16 bytes):** Word 0 points directly to the `T` payload (so reading fields needs only 1 hop); Word 1 points to a heap-allocated **Control Block**.
2. **Inside the Control Block:** An *atomic strong reference count* (how many `shared_ptr`s keep `T` alive), an *atomic weak reference count* (how many `weak_ptr`s keep the control block header

itself alive so they can safely ask "did `T` already die?"), plus deleter/allocator metadata.

3. **Why `std::make_shared<T>(...)` beats `std::shared_ptr<T>(new T(...))`:** Calling `new T(...)` first allocates `T` in one heap block and *then* allocates the Control Block in a *second* separate heap block. `std::make_shared<T>(...)` performs a **single fused heap allocation** placing the Control Block and `T` next to each other in memory.

4. **The atomic refcount tax *and* reference cycles:** Because C++ cannot statically tell whether a `shared_ptr` stays on one thread or crosses threads, *every* copy and destruction of `shared_ptr` executes a hardware atomic read-modify-write instruction (`lock xadd` on x86-64), bouncing the control block's cache line across CPU cores (an uncontended `lock xadd` costs on the order of 10–20 cycles versus about one for a plain pointer copy, and far more when cores contend for the line; contrast Rust splitting single-threaded non-atomic `Rc<T>` from thread-safe atomic `Arc<T>`). And if object $A$ holds `shared_ptr<B>` while $B$ holds `shared_ptr<A>`, neither strong count ever drops to 0 — leaking both forever unless the back-edge is broken with a non-owning `std::weak_ptr<A>` (upgraded temporarily via `if (auto sp = weakParent.lock()) { ... }`).

### 2.3.3 Non-owning fat-pointer views: `std::string_view` and `std::span<T>`

Smart pointers express *ownership*. Just as often, a function does *not* want ownership at all — it merely wants to inspect a string or a contiguous array of `Order`s regardless of where the caller stored them.

Consider a subtly wasteful pre-C++17 habit:

```
// Subtle allocation trap in C++98/11 when called with a literal or slice!
bool isUsdPair(const std::string& symbol);

// Calling isUsdPair("BTC-USD") constructs a temporary std::string on the
// stack (and on the heap if longer than ~15-22 bytes of Small String Opt!)
// just to read 7 constant characters already sitting in .rodata!
```

C++17 introduced `std::string_view` and C++20 added `std::span<T>` to make read-only (or mutable slice) borrowing uniform and 100% allocation-free:

```
#include <span>
#include <string_view>

// 16-byte value (const char* ptr, size_t len) passed directly in 2 registers!
bool isUsdPair(std::string_view symbol) noexcept {
    return symbol.ends_with("-USD"); // zero heap allocation for any input!
}

// 16-byte value (const Order* ptr, size_t len) over vector, C array, or slice!
int64_t sumNotional(std::span<const Order> window) noexcept {
    int64_t total = 0;
    for (const Order& o : window) {
        total += o.price_cents * static_cast<int64_t>(o.quantity);
    }
    return total;
}
```

Look at the exact bytes of `std::string_view` and `std::span<const Order>`: both are flat **16-byte fat pointers** `(ptr, len)` passed by value in two CPU registers. A single `sumNotional` function accepts a `std::vector<Order>`, a stack-allocated `std::array<Order, 64>`, a C array `Order[10]`, or a sub-window `std::span(vec).subspan(10, 5)` with zero copies and zero virtual dispatch. (Hold onto that 16-byte `(ptr, len)` mental picture: Rust's `&str` and `&[T]` have the *exact same 16-byte layout*)

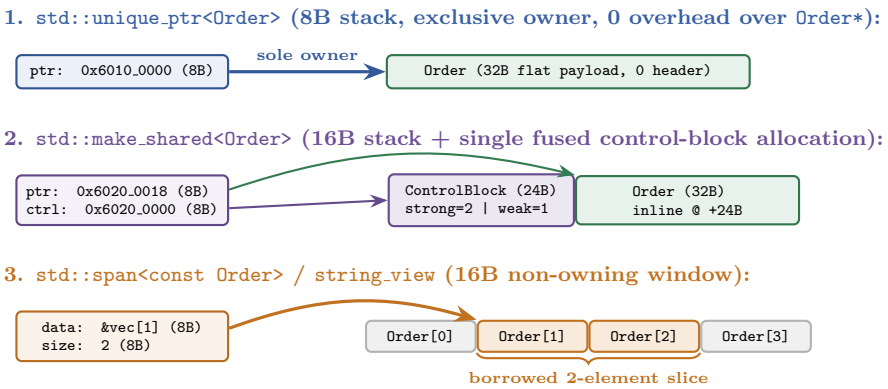



Figure 2.3: Exact memory layouts of Modern C++ smart pointers and non-owning views. Compare single-pointer `unique_ptr` (8B), dual-pointer `shared_ptr` (16B), and fat-pointer slice `span` (16B).

## 2.4 Zero-Cost Generics, `constexpr`, C++20 Concepts, and Lambdas

Now let us see how Modern C++ achieves generic abstraction and functional-style callbacks *without* paying Java 6's heap-boxing and virtual-call penalties.

### 2.4.1 Templates via monomorphization vs. Java 6 type erasure

In Java 6, generics (`ArrayList<T>`) were retrofitted onto Java 1.4's JVM without changing bytecode array layouts, using **type erasure**: at compile time, `ArrayList<Order>` and `ArrayList<Long>` both erase to raw `ArrayList` holding `Object[]`, inserting hidden heap boxing (`Long.valueOf`) and runtime `checkcast` instructions on reads.

C++ templates (and Rust generics, as we will explore deeply in Chapter 6) take the exact opposite route: **monomorphization** ("turning into single concrete shapes"). When you write:

```
template <typename T>
T clampValue(T val, T lo, T hi) noexcept {
    return (val < lo) ? lo : ((hi < val) ? hi : val);
}
```

the template itself emits zero machine code until called. When your program calls `clampValue` once with `int64_t` arguments and once with `double` arguments, the compiler stamps out two completely separate, concrete functions — one using integer `cmp`/`cmovg` instructions on 64-bit general registers, and one using scalar floating-point `maxsd`/`minsd` instructions on `XMM` registers — and then inlines

both straight into the caller. Zero heap boxing, zero vtable lookup, full CPU auto-vectorization, at the engineering cost of larger compiled binaries and slower builds if huge templates are instantiated across dozens of types.

### 2.4.2 `constexpr` and C++20 Concepts: taming template chaos

Historically, C++ templates had an infamous dark side: *unconstrained duck typing*. In C++98–C++17, `template <typename T>` accepted *any* `T` at the front door and only discovered deep inside five nested standard-library headers that `T` lacked an `operator<`, dumping a 180-line substitution failure backtrace onto your terminal.

Modern C++ added two clean compile-time tools:

1. **`constexpr` (and C++20 `consteval`):** Functions and constants marked `constexpr` can be evaluated *by the compiler itself during compilation*, baking precomputed fee multipliers, CRC lookup tables, or currency exponent powers directly into read-only instruction immediates with zero startup cost.
2. **C++20 `concept` and `requires`:** Named compile-time predicates that specify *up front* what operations a type `T` must support:

```cpp
#include <concepts>
#include <cstdint>

// C++20 Concept: states the exact structural contract on Priceable types
template <typename T>
concept HasNotional = requires(const T& item) {
    { item.priceCents() } -> std::convertible_to<int64_t>;
    { item.quantity()   } -> std::convertible_to<int64_t>;
};

constexpr int64_t bpsDivisor() noexcept { return 10000; }

// Checked right at the call boundary with a 1-line error if T fails!
template <HasNotional T>
int64_t totalFeeCents(std::span<const T> items, int64_t feeBps) {
    int64_t sum = 0;
    for (const T& x : items) {
        sum += (x.priceCents() * x.quantity() * feeBps) / bpsDivisor();
    }
    return sum;
}
```

**Reading note**

Notice one subtle difference you will appreciate in Chapter 6 when we meet Rust `traits`: C++20 Concepts constrain *call sites* cleanly, yet the compiler still waits until instantiation with a concrete `T` to fully type-check the body of `totalFeeCents` (so `totalFeeCents` could accidentally call an undeclared method `x.foo()` and still compile until someone instantiates it with a type lacking `foo()`). Rust traits check the generic function body *once, against the trait definition itself*, guaranteeing that if a generic library compiles, *no* downstream caller

type satisfying the trait can ever break inside its body.

### 2.4.3 Lambdas under the hood: zero-allocation closure structs

In Java 6, passing a custom predicate or comparator into a sort or filter meant writing a verbose anonymous inner class (`new Comparator<Order>() { ... }`), allocating a wrapper object on the heap and dispatching through a virtual interface call on every comparison.

C++11 introduced **lambda expressions** with an explicit **capture list** inside square brackets `[...]`:

```cpp
#include <algorithm>

size_t countLargeOrdersForAccount(std::span<const Order> orders,
                                  uint64_t targetAcct,
                                  int64_t  minCents,
                                  int64_t& runningNotional) {
    // Capture targetAcct & minCents BY VALUE, runningNotional BY REFERENCE!
    return std::count_if(orders.begin(), orders.end(),
        [targetAcct, minCents, &runningNotional](const Order& o) {
            if (o.account_id == targetAcct && o.price_cents >= minCents) {
                runningNotional += o.price_cents * o.quantity;
                return true;
            }
            return false;
        });
}
```

A C++ lambda (like a Rust closure in Chapter 7) is ordinary data. The C++ compiler rewrites that lambda expression into an **anonymous stack-allocated** `struct` whose fields are the captured variables and whose call operator `operator()` holds the body:

```cpp
// Exact mental desugaring generated by the C++ compiler:
struct __Lambda_Line9 {
    uint64_t targetAcct;       // copied by value (8 bytes)
    int64_t  minCents;         // copied by value (8 bytes)
    int64_t& runningNotional;  // captured by reference (8-byte pointer!)

    bool operator()(const Order& o) const {
        if (o.account_id == targetAcct && o.price_cents >= minCents) {
            runningNotional += o.price_cents * o.quantity;
            return true;
        }
        return false;
    }
};
```

Because `__Lambda_Line9` is a unique, concrete struct type known at compile time and `std::count_if` is a monomorphized template, the compiler places the 24-byte closure struct right in registers/stack slots and inlines `operator()` straight into the loop body — zero heap allocation, zero indirect function-pointer call.

Now look at the capture choices inside `[...]`:

- `[x]` or `[=]` captures local variables **by value** (copying them into the closure struct).
- `[&x]` or `[&]` captures local variables **by reference** (storing raw 8-byte pointers to the surrounding stack frame inside the closure struct).
- `[buf = std::move(buf)]` (C++14 init-capture) **moves** an owning RAII value like `unique_ptr` or `vector` straight into the closure struct.

And that `[&]` capture is exactly where Modern C++ runs out of compile-time guardrails.

## 2.5 The Four C++ Footguns That Forced the Invention of Rust

Take stock of what we just built. Using RAII, vectors, `unique_ptr`, moves, spans, string views, templates, and lambdas, we eliminated every performance tax of Java 6 *and* eliminated manual C `malloc/free` boilerplate.

That raises a fair question: **if Modern C++20 already gives us flat contiguous memory, deterministic RAII destructors, smart pointers, and zero-cost generics, why did engineers still spend a decade building Rust?**

Because the bugs kept coming. Security teams at Microsoft (across its C and C++ products) and Google (in Chromium) each reported that **roughly 70% of their high-severity security bugs were memory-safety bugs** — and that share stayed high even as those codebases adopted RAII and smart pointers.

Let us inspect the exact four C++17/C++20 footguns that compile cleanly without a single raw pointer and fail at runtime. Every single one is an instance of **untracked lifetimes** or **Aliasing + Mutation** from Chapter 1.

### 2.5.1 Footgun 1: Dangling `string_view` and `span` into destroyed temporaries

Look at two lines of clean, modern C++17 that contain zero raw pointers, zero `new`, and zero casts:

```
1 std::string buildLedgerTag(uint64_t acctId) {
2     return "ACCT-" + std::to_string(acctId);
3 }
4
5 void badView() {
6     // COMPILES CLEANLY -- UNDEFINED BEHAVIOR (USE-AFTER-FREE)!
7     std::string_view tag = buildLedgerTag(42) + "-USD";
8     logAudit(tag); // reads freed heap memory!
9 }
```

Trace the RAII timeline on line 7 step by step:

1. `buildLedgerTag(42) + "-USD"` constructs a temporary `std::string` owning a heap buffer containing `"ACCT-42-USD"`.
2. `std::string_view tag` borrows a 16-byte `(ptr, len)` pointing directly into that temporary's heap buffer.

3. **At the semicolon ; at the end of line 7**, the temporary `std::string`'s lifetime ends, and **RAII faithfully runs `~basic_string()`, freeing the heap buffer immediately**.

4. On line 8, `tag` is a **dangling view** pointing into freed heap RAM. RAII worked exactly as specified — and killed the referent while a non-owning view still pointed to it, because C++'s type `std::string_view` carries **zero compile-time lifetime tie** back to the `std::string` it borrowed from.

### 2.5.2 Footgun 2: Single-threaded reference/iterator invalidation on `std::vector` growth

Here is the #1 crash that bites seasoned C++ backend engineers on a single thread with zero multithreading anywhere:

```
void appendMirrorOrder(std::vector<Order>& book) {
    // Alias 1: take a clean C++ const reference to the best resting order
    const Order& best = book[0];

    // Alias 2: mutate 'book' by appending a derived hedge order!
    book.push_back(Order{
        best.order_id + 1, best.account_id, best.price_cents, best.quantity, Side::Sell
    });

    // BUG: if push_back reallocated 'book', 'best' points to FREED RAM!
    recordSpread(best.price_cents);
}
```

This bug hides in testing and appears in production:

- During unit tests the vector usually has spare capacity (say size 3, capacity 4), so `push_back` writes into the spare slot at `book[3]` without moving anything, and your test passes 100% of the time.

- In production, on the exact call where `book.size() == book.capacity()` (say `4 == 4`), `push_back` allocates a brand-new 8-element array on the heap, moves `Order[0..3]` to the new address, **calls `delete[]` on the old 4-element buffer**, and updates `book`'s internal pointer. Meanwhile, `const Order& best` on your stack still holds the old address — and `best.price_cents` reads freed memory.

Look back at Chapter 1's $2 \times 2$ matrix (Figure 1.3): `best` held a read alias into `book` at the exact moment `book.push_back(...)` mutated `book`.

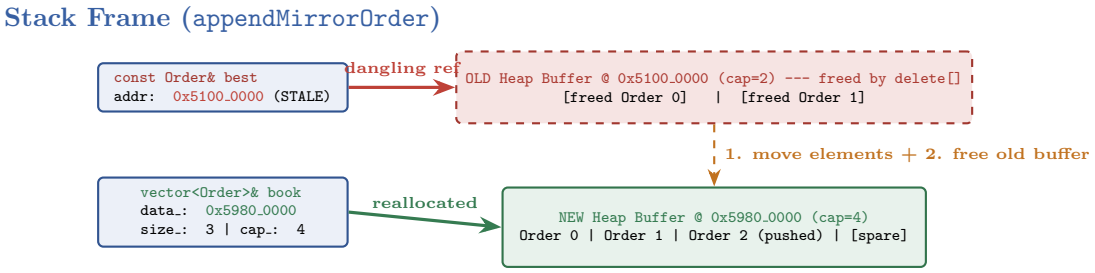



Figure 2.4: Anatomy of `std::vector` reference invalidation in C++. Holding `const Order& best = book[0]` while calling `book.push_back(...)` leaves `best` pointing into the freed pre-reallocation heap block.

### 2.5.3 Footgun 3: Silent use-after-move

Because C++ moves are non-destructive (§2.2), calling `std::move` on a local object leaves the variable in scope and completely accessible to typos or later edits:

```
void dispatchSettlement(std::unique_ptr<SettlementBatch> batch) {
    auditLogger.enqueue(std::move(batch)); // batch is now nullptr!
    // Twenty lines later, someone adds metrics logging:
    metrics.recordCount(batch->orders.size()); // COMPILES! Derefs nullptr!
}
```

Static linters (`clang-tidy`'s `bugprone-use-after-move`) catch simple straight-line cases, but cannot enforce soundness across helper methods, conditional loops, or struct fields because moved-from state is not tracked in C++'s type system.

### 2.5.4 Footgun 4: Dangling [&] lambda captures and unchecked data races

Finally, combine C++11 lambdas with asynchronous execution or threads:

```
void scheduleAsyncReceipt(ThreadPool& pool, Order localOrder) {
    // [&] captures a pointer to 'localOrder' on THIS stack frame!
    pool.submit([&]() {
        sendReceiptEmail(localOrder); // runs 5 ms AFTER localOrder's frame died!
    });
} // <-- localOrder stack slot is popped right here!
```

Two characters — `[&]` instead of `[=]` — store a pointer to `scheduleAsyncReceipt`'s stack frame inside a heap-queued task that executes milliseconds after `scheduleAsyncReceipt` has returned and its stack bytes have been overwritten by another HTTP request. And even if `localOrder` lived on the heap, passing references to the same `Account` across two `std::thread` workers without locking a detached `std::mutex` is immediate undefined behavior under the C++11 memory model.

**Focus**

**The Bridge from Modern C++ to Part II (Seeing Through the Rust Lens).** Look across all four C++ footguns at once. Notice that *not one of them requires a garbage collector*

*to fix.* Every single bug could have been caught *before generating machine code* if the compiler knew just three things in its type system:

1. **True destructive ownership:** Moving a value out of `x` marks `x` statically dead at compile time (eliminating Footgun 3).
2. **Aliasing XOR Mutability:** While `best` borrows an immutable reference `&book[0]`, calling `book.push_back(...)` (`&mut book`) is forbidden at compile time (eliminating Footgun 2).
3. **Compile-time lifetimes (`'a`) and thread-safety traits (`Send`/`Sync`):** No view or closure may outlive the scope of the value it borrows, and no non-synchronized reference may cross threads (eliminating Footguns 1 and 4).

Modern C++ invented the runtime physical model (flat values + deterministic RAII + $O(1)$ moves + fat-pointer slices + monomorphized generics). **Rust is what happens when you take that exact physical model and promote its safety invariants from programmer discipline into a sound static type system.**

| Dimension | Java 6 | Classic C | Modern C++20 | Rust (Part II) |
|---|---|---|---|---|
| Type storage | Heap + 16B hdr | Flat stack/inline | Flat stack/inline | Flat stack/inline |
| Deallocation | Tracing GC (STW) | Manual `free()` | Scope RAII `~T()` | Scope `Drop` |
| Assign `b = a` | Alias heap ref | Bitwise copy | Copy / `std::move` | Move / `Copy` |
| Moved-from `a` | N/A | N/A | Live zombie shell | Statically dead |
| Slice view | Wrapper object | `(T*, size_t)` | `span`/`string_view` | Checked `&[T]`/`&str` |
| Generics | Type erasure | Macros / `void*` | Monomorph + Concepts | Monomorph + Traits |
| Safety check | Runtime GC | None (UB) | Discipline (UB) | Compile-time |

Pause

**Pause before entering Part II.** If Part I did its job, nothing in Chapters 3 through 6 will feel arbitrary when the Rust borrow checker speaks up. Every error message you meet in Part II will trace directly back to a physical byte diagram you have already seen: either a heap block that would have been double-freed, a `Vec` reallocation that would have left a reference dangling, a temporary string whose buffer died at a semicolon, or a mutable alias racing across threads. Try the exercises below to cement Modern C++'s mechanics before we step through the Rust lens in Chapter 3.

## 2.6 Exercises

**Exercise 2.1** (Exact destructor ordering in RAII)**.** Trace the exact sequence of lines printed when `runJob(true)` and `runJob(false)` execute:

```
struct Tracer {
    const char* name;
    explicit Tracer(const char* n) : name(n) { std::printf("+%s ", name); }
    ~Tracer() noexcept                       { std::printf("-%s ", name); }
};

```

```
void runJob(bool failEarly) {
    Tracer a("AuditLog");
    {
        Tracer b("Buffer");
        Tracer c("PgTxn");
        if (failEarly) return;
        std::printf("COMMIT ");
    }
    std::printf("DONE ");
}
```

Explain in one sentence why C++ guarantees reverse construction order (`-c`, then `-b`, then `-a`) rather than forward order when cleaning up local variables or struct fields.

**Exercise 2.2** (Writing a move-only `PosixFd` RAII wrapper)**.** In POSIX C on Linux, an open socket or file is an `int fd` where valid descriptors satisfy `fd >= 0`, `-1` represents "no open descriptor," and `close(fd)` releases an open descriptor. Write a complete Modern C++ class `PosixFd` satisfying the Rule of Five:

(a) Forbid copy construction and copy assignment with `= delete` (why would copying a raw `fd` without `dup()` break RAII?).

(b) Implement the destructor `~PosixFd() noexcept`, the move constructor taking `PosixFd&&`, and move assignment `operator=(PosixFd&&) noexcept` (watch out: if `this` already holds an open `fd != -1` before move-assigning `fd1 = std::move(fd2)`, what must happen to `fd1`'s old descriptor first, and why must you guard against self-move `this == &other`?).

(c) Explain why `-1` had to serve as `PosixFd`'s moved-from zombie state in C++, whereas a Rust `OwnedFd` requires no `-1` state when moved.

**Exercise 2.3** (When `std::move` silently copies instead of moving)**.** A teammate writes the following optimization to avoid copying a 10 MB payload buffer:

```
void archivePayload(const ByteBuffer buf) {
    storageQueue.push_back(std::move(buf)); // Surprise: deep copies 10 MB!
}
```

Using the exact C++ types and overload resolution, explain why calling `std::move` on a `const ByteBuffer` selects the $O(N)$ *copy constructor* (`const ByteBuffer&`) rather than the $O(1)$ *move constructor* (`ByteBuffer&&`), and fix `archivePayload`'s parameter declaration so zero deep copies occur when passed an rvalue.

**Exercise 2.4** (`make_shared` vs. `weak_ptr` memory retention subtlety)**.** In §2.3 we saw that calling `std::make_shared<BigOrderBook>()` fuses the 24-byte Control Block and a 100 MB `BigOrderBook` into a *single* heap allocation. Suppose all `std::shared_ptr<BigOrderBook>` handles go out of scope (so the strong count drops to 0 and `~BigOrderBook()` runs), while one monitoring widget still holds a live `std::weak_ptr<BigOrderBook>` (so the weak count is 1). Can the C++ runtime call `free()` on the 100 MB fused heap block while that `weak_ptr` still exists? Explain why or why not, naming the one tradeoff where separate allocation `std::shared_ptr<T>(new T())` actually reclaims RAM earlier than `std::make_shared<T>()`.

**Exercise 2.5** (Hand-desugaring a C++ lambda capture bug)**.** Write down the compiler-generated closure `struct` (with exact field types) for both lambdas below, calculate `sizeof` each closure

struct on 64-bit hardware, and state which one exhibits undefined behavior when the returned `std::function` is invoked later by the caller:

```
auto makeFeeCalcBad(int64_t bps) {
    int64_t minFeeCents = 50;
    return [&](int64_t amt) { return std::max(minFeeCents, (amt * bps) / 10000); };
}

auto makeFeeCalcGood(int64_t bps) {
    int64_t minFeeCents = 50;
    return [=](int64_t amt) { return std::max(minFeeCents, (amt * bps) / 10000); };
}
```

**Exercise 2.6** (Spotting hidden `std::vector` reference invalidation)**.** A subtle bug hides in this six-line C++20 function that duplicates the highest-value order at the end of a vector:

```
void duplicateFirstIfNonEmpty(std::vector<Order>& orders) {
    if (!orders.empty()) {
        orders.push_back(orders.front()); // look at push_back(const T&)!
    }
}
```

Wait: `std::vector::push_back(const Order& val)` takes its argument *by `const` reference*, and `orders.front()` is a reference into `orders`'s own backing buffer.

(a) Explain how a naive `push_back(const T& val)` implementation that freed the old buffer *before* constructing the new element from `val` would crash when called as `v.push_back(v.front())`.

(b) Now suppose instead someone wrote:

```
const Order& first = orders.front();
orders.reserve(orders.size() + 10);
processOrder(first);
```

Does the C++ standard library save you here, or is `processOrder(first)` reading freed memory? How could you rewrite those three lines safely in C++ using an integer index or a value copy?

**Exercise 2.7** (Dangling `std::string_view` return value)**.** Explain step by step why this helper function compiles without error in C++17 and corrupts strings at runtime, and fix its return type or signature cleanly:

```
std::string_view formatMoneyDisplay(int64_t cents) {
    std::string formatted = "$" + std::to_string(cents / 100) + "."
                          + std::to_string(std::abs(cents % 100));
    return formatted; // implicit conversion std::string -> std::string_view!
}
```

**Exercise 2.8** (Monomorphization code size vs. Java 6 type erasure)**.** Suppose a trading library defines a generic binary search template:

```
template <typename T>
size_t lowerBound(std::span<const T> xs, const T& key);
```

Assume its compiled machine body is 512 bytes, instantiated for 12 distinct flat value types (`int32_t`, `int64_t`, `Order`, `Quote`, etc.).

(a) How many bytes of `.text` instruction memory do all 12 monomorphized C++ instantiations occupy in total, and how many heap allocations or virtual method dispatches occur during a binary search over 1,000,000 `Order`s?

(b) Contrast this with `Collections.binarySearch(List<Order>, key, comparator)` in Java 6 over 1,000,000 elements: how many copies of `binarySearch` bytecode exist, and how many pointer dereferences and interface calls (`Comparator.compare`) happen across $\log_2(1{,}000{,}000) \approx 20$ probe steps?

---

# Part II

# Seeing Through the Rust Lens

# Chapter 3

# Ownership, Moves, and Borrowing: Seeing Through the Rust Lens

Chapter 1 covered what Java 6 gets from a tracing garbage collector and what it costs: 16-byte object headers, pointer chasing across cache lines, and stop-the-world pauses. Chapter 2 covered modern C++, where destructors free memory at the closing brace of a scope and values sit contiguously in arrays, but where a dangling reference or an invalidated iterator can read freed memory without any warning.

Before Rust, a systems language picked one of those two designs. Either a garbage collector manages object lifetimes and you pay its latency and memory cost, or you manage lifetimes yourself in C or C++ and some use-after-free bugs and data races reach production.

Rust frees memory deterministically, with no garbage collector, and still rejects use-after-free and data races at compile time. It does this by enforcing one rule about how pointers may coexist, checked by the part of the compiler called the *borrow checker*. This chapter states that rule, shows the bytes each pointer type occupies, and then derives moves, borrows, slice splitting, reference counting, and mutexes from the rule.

---

## 3.1 The Single Root Bug Behind Two Different Worlds

Before writing a line of Rust, let us put two short, single-threaded bugs side by side — one in modern C++, one in Java 6. Neither program spawns a thread, neither uses raw `malloc` or `delete`, and yet both fail for the exact same structural reason.

First, look at seven lines of C++ processing a list of ledger postings (in minor currency units, cents):

```
#include <cstdint>
#include <iostream>
#include <vector>

void append_bonus_if_positive(std::vector<int64_t>& postings) {
  const int64_t& first = postings[0]; // Alias 1: borrows element 0
  if (first > 0) {
    postings.push_back(500);          // Alias 2: mutates the vector!
```

```
    std::cout << "Triggered by posting: " << first << "\n"; // BUG!
  }
}
```

Walk through what happens in memory when `postings` enters this function with length 4 and capacity 4. On line 6, `first` stores the raw 64-bit machine address of element 0 inside the heap buffer owned by `postings`. Even though `first` is marked `const`, `const` in C++ only means "*I* promise not to mutate through `first`"; it does not stop anyone else from mutating or freeing the memory behind it.

On line 8, `postings.push_back(500)` sees that length equals capacity. To grow, the vector allocates a fresh 8-element heap buffer, copies the four existing `int64_t` values into the new buffer, writes `500` into slot 4, and calls `delete[]` on the *old* heap buffer. Now step onto line 9: `first` still holds the address of the old, freed buffer. Reading `first` is a textbook **use-after-free** in clean, single-threaded C++. Under light load the old bytes might happen to sit untouched in the allocator's free list and print the expected number in every unit test; under production load another request allocates that exact chunk for an HTTP header, and `first` reads garbage bytes or crashes the process.

Now write the analogous loop in Java 6:

```
import java.util.ArrayList;
import java.util.List;

public class LedgerAdjuster {
  public static void appendBonusForPositives(List<Long> postings) {
    for (Long amount : postings) {        // Alias 1: hidden Iterator<Long>
      if (amount.longValue() > 0L) {
        postings.add(Long.valueOf(500L)); // Alias 2: mutates ArrayList!
      }
    }
  }
}
```

Java's garbage collector prevents a raw memory-corruption crash here: even if `ArrayList.add` allocates a larger `Object[]` backing array, the old array stays alive in the GC heap as long as any reference points to it. Yet when you run this method on a non-empty list with a positive entry, it blows up on the next loop step with `java.util.ConcurrentModificationException` — in a program with only one thread. Why? Because the `for`-each loop holds an internal `Iterator` cursor over `postings` while the loop body mutates `postings` behind the iterator's back. Java's `ArrayList` keeps an integer counter `modCount` on every list instance, increments it on every `add` or `remove`, and checks `modCount == expectedModCount` on every `iterator.next()` call, paying a runtime branch on every element so that the bug becomes an exception instead of silently skipped or repeated elements.

Worse, `modCount` only guards standard collection iterators. When two components in a Java 6 service hold references to the same mutable `OrderDraft` object — say a risk-scoring cache and a fee calculator — mutating `draft.setAmountCents(0)` through one reference silently alters what the other component observes mid-calculation, with no exception at all.

**Intuition**

Strip away C++ destructors and Java garbage collection, and ask: what two ingredients had to collide in *every single one* of those failures?

1. **Aliasing:** Two or more live access paths (`postings` and `first`; the loop's `Iterator` and `postings`; two object references `a` and `b`, or two threads) reach overlapping bytes of memory at the same time.
2. **Mutation:** At least one of those paths modifies (or reallocates) the underlying state while another path expects that state to hold still.

Aliasing *without* mutation is completely harmless: fifty readers can inspect the same immutable configuration table or UTF-8 string across threads without interfering with one another. Mutation *without* aliasing is just as harmless: if you hold the *only* pointer to a buffer in the entire program, you can overwrite it, resize it, or free it, and nobody else's pointer can possibly dangle because nobody else has one. Trouble only begins when **aliasing and mutation happen simultaneously**.

Rust turns that observation into a rule the compiler checks.

**Definition 3.1** (The Aliasing XOR Mutability Law)**.** At every point in a safe Rust program's execution, any value in memory may be accessed through:

- **Shared read-only borrows (`&T`):** any number of simultaneous aliases $(1, 2, \ldots, n)$, through none of which the value may be mutated; **or**

- **An exclusive mutable borrow (`&mut T`) or unique owner (`T`):** exactly *one* active access path through which the value may be read and mutated, with zero other live readers or writers.

Never both at the same time. For this reason, Rustaceans often read `&T` aloud as "*shared* reference" and `&mut T` as "*unique* (or *exclusive*) reference."

If you have used `java.util.concurrent.locks.ReentrantReadWriteLock` in Java, you already know this rule: a read-write lock permits either many concurrent `readLock()` holders *or* a single exclusive `writeLock()` holder. The difference is that Java's `ReentrantReadWriteLock` executes atomic CPU instructions at runtime, risks deadlocks, and only protects code that remembers to acquire the lock. Rust runs the read-write lock check *inside the compiler* across every variable, struct field, slice, and thread in your program, and emits zero lock instructions in the compiled binary.

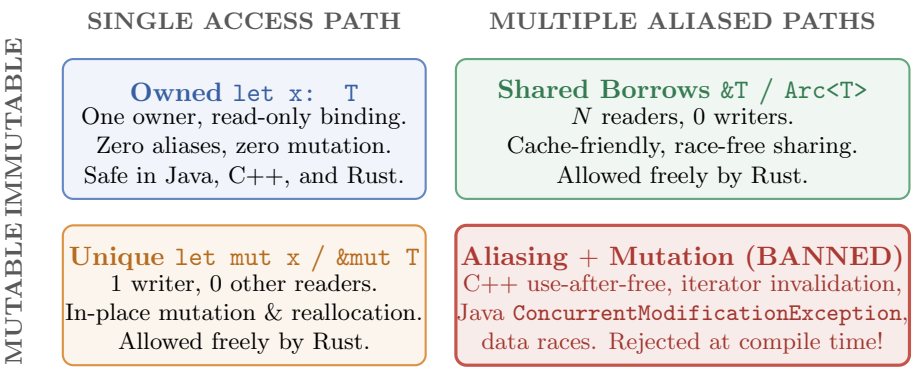



Figure 3.1: The $2 \times 2$ design matrix of aliasing and mutability. Java 6 defaults almost every object variable into the bottom-right quadrant (shared references to mutable objects) and relies on GC plus discipline to survive. C++ lets raw pointers and references enter the bottom-right quadrant unchecked. Safe Rust allows the first three quadrants at full machine speed and forbids the bottom-right quadrant unless gated by explicit synchronization (§3.6).

When the compiler rejects a program with a borrow error, find the two access paths that overlap in Figure 3.1's bottom-right box: *which two paths alias the same memory while one of them mutates it, and what would break if the vector reallocated or a second thread ran at that point?* Answering that question usually tells you how to restructure the code, as the examples later in this chapter show.

## 3.2 Ownership and Exact Byte Layouts on Stack and Heap

To enforce Aliasing XOR Mutability without a garbage collector, Rust starts by giving every allocated resource a single, unambiguous home.

**Definition 3.2** (The Three Rules of Ownership)**.** Every value in Rust obeys three compile-time rules:

1. Every value has a single **owner** (a variable, a struct field, or a collection slot).

2. There can be **only one owner at a time** (unless ownership is explicitly shared through a reference-counted smart pointer such as `Rc<T>` or `Arc<T>`, covered in §3.5).

3. When the owner goes out of scope (and ownership has not been moved elsewhere), the value is **dropped** deterministically right at that point: the compiler inserts a call to `Drop::drop`, freeing heap buffers, closing sockets, or rolling back uncommitted database transactions in reverse order of declaration.

If you read Chapter 2 carefully, Rule 3 looks just like C++ RAII (*Resource Acquisition Is Initialization*), and at runtime it compiles to the exact same machine code: deterministic calls at scope exit, with no background GC threads scanning object graphs. Where Rust parts ways with both Java 6 and C++ is in how values sit in memory and how ownership moves between variables. Let us look directly at the bytes on a 64-bit machine (`usize` = 8 bytes).

### 3.2.1 `String` (24 Bytes Owned) versus `&str` (16-Byte Fat Slice)

In Java 6, a `java.lang.String` involves two levels of indirection. Even the 6-character currency pair `"EURUSD"` requires:

- an 8-byte reference in your local variable or object field, pointing to
- a heap `String` object (16-byte JVM mark+klass header, an 8-byte reference to a `char[]`, plus three 4-byte `int` fields `offset`, `count`, and `hash` — padded to 32 or 40 bytes), pointing to
- a separate heap `char[]` array (another 16-byte array header + length, followed by UTF-16 two-byte characters).

Reading the characters of a Java 6 `String` chases two heap pointers and touches two separate cache lines.

In Rust, an owned `String` is a flat, 24-byte struct sitting directly inside the owning stack frame (or directly inline inside an owning struct — zero object header, zero extra indirection):

- `ptr: *mut u8` (offset 0..8, 8 bytes): non-null machine address of the UTF-8 byte buffer on the heap.
- `len: usize` (offset 8..16, 8 bytes): how many initialized UTF-8 bytes the string currently holds.
- `cap: usize` (offset 16..24, 8 bytes): total byte capacity allocated on the heap before reallocation is needed.

When `let mut pair = String::with_capacity(8); pair.push_str("EURUSD");` runs, the stack frame holds the three 64-bit words (`ptr`, 6, 8), and the heap holds just the raw UTF-8 bytes `E U R U S D` followed by two unused capacity bytes — no object header on the stack, no object header in front of the `u8` array beyond the allocator's internal size class. When `pair` goes out of scope, `Drop` passes `ptr` and `cap` straight to the allocator's `dealloc`.

Now suppose a function only wants to *inspect* text — say, parsing the base currency `"EUR"` out of `"EURUSD"` or checking a string literal `"USD"`. Allocating a new heap `String` just to read three letters would be wasteful, and taking `&String` (an 8-byte pointer to a 24-byte `String` on the stack, which then points to the heap) would force callers with a string literal `"USD"` (baked into the binary's read-only `.rodata` segment, with no heap allocation at all) to heap-allocate a `String` first.

Rust solves this with `&str`, a **16-byte fat pointer** (or *string slice*) consisting of just two 64-bit registers:

- `ptr: *const u8` (offset 0..8, 8 bytes): address of the first UTF-8 byte of the view.
- `len: usize` (offset 8..16, 8 bytes): byte length of the view.

Notice what is *missing* from `&str`: there is no `cap` field. Because `&str` is a borrowed view that neither grows nor frees memory, capacity is none of its business. A single function `fn parse_currency(code: &str)` compiled once into machine code accepts:

1. `&pair[0..3]` — a 3-byte window into our heap-allocated `String`, passed in two CPU registers (`rdi = pair.ptr`, `rsi = 3`);
2. `"USD"` — a string literal of type `&'static str` pointing directly into the executable's read-only

`.rodata` data segment (`rdi = addr_in_rodata`, `rsi = 3`); or

3. a slice of a fixed stack buffer `[u8; 64]` read off a network socket and validated as UTF-8 via `std::str::from_utf8(&buf[..n])` — zero heap allocations from network card to parser.

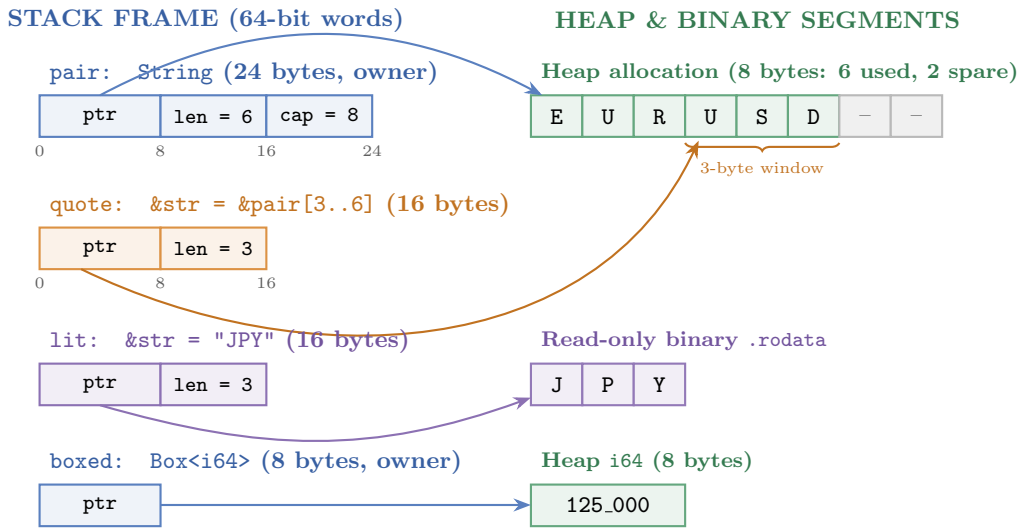



Figure 3.2: Exact 64-bit memory layouts of `String` (24-byte owning triple), `&str` (16-byte non-owning fat slice able to point into a heap `String` or directly into read-only executable `.rodata`), and `Box<i64>` (8-byte unique owning pointer).

### 3.2.2 `Vec<T>` versus `&[T]`, and `Box<T>`

The exact same pairing repeats for general contiguous collections:

- **`Vec<T>` (24 bytes on stack: `ptr`, `len`, `cap`):** owns a contiguous heap array of `cap` slots, the first `len` of which hold initialized `T` values packed back-to-back with *zero* per-element pointers or headers. If `Account` is a 24-byte struct, a `Vec<Account>` of 1,000 accounts is one single 24,000-byte heap allocation through which the CPU hardware prefetcher streams linearly. (Compare Java 6's `ArrayList<Account>`: an array of 1,000 references scattered across 1,000 individually heap-allocated `Account` objects, each carrying a 16-byte JVM header)

- **`&[T]` and `&mut [T]` (16 bytes on stack: `ptr`, `len`):** borrowed slice views into any contiguous sequence of `T` — whether backed by a heap `Vec<T>`, a fixed stack array `[T; N]`, or memory-mapped file bytes.

- **`Box<T>` (8 bytes on stack: `ptr`):** unique ownership of a single `T` placed on the heap. Coming from Java 6, newcomers often wrap structs in `Box<Order>` out of habit because Java puts every object on the heap. In Rust, default to plain stack/inline values (`Order`), and reach for `Box<T>` only for three concrete engineering reasons: (1) recursive types like AST or tree nodes whose size would otherwise be infinite (Chapter 5), (2) moving a multi-kilobyte struct around by copying an 8-byte pointer instead of `memcpy`-ing 4,096 bytes, or (3) holding a dynamically sized trait object `Box<dyn Pricing>` (Chapter 6).

**Common trap**

**Never write `&String` or `&Vec<T>` in function parameters when `&str` or `&[T]` suffices.** If you write `fn total(xs: &Vec<i64>) -> i64`, a caller holding a fixed stack array `[10, 20, 30]` or a sub-slice `&vec[1..5]` cannot call your function without allocating a brand-new heap `Vec<i64>`. Worse, inside `total`, reading an element through `&Vec<i64>` chases two pointers (caller stack → `Vec` header → heap elements) instead of reading `ptr` directly from the 16-byte `&[i64]` register pair. Meanwhile, callers who *do* own a `Vec<i64>` or `String` can still pass `&my_vec` or `&my_string` effortlessly because Rust automatically deref-coerces `&Vec<T>` to `&[T]` and `&String` to `&str`.

You do not have to take any byte count in this book on faith. Because Rust knows exact type layouts at compile time, we can verify every layout with `const` assertions checked by `rustc` before a binary is even emitted:

```
use std::mem::size_of;

pub struct Account {
    pub id: u64,            // 8 bytes
    pub balance_cents: i64, // 8 bytes
    pub currency: [u8; 3],  // 3 bytes + 5 bytes trailing alignment padding
}

const _: () = assert!(size_of::<Account>() == 24);
const _: () = assert!(size_of::<String>() == 24);
const _: () = assert!(size_of::<&str>() == 16);
const _: () = assert!(size_of::<Vec<Account>>() == 24);
const _: () = assert!(size_of::<&[Account]>() == 16);
const _: () = assert!(size_of::<Box<Account>>() == 8);
```

## 3.3 Bitwise Copy, Destructive Move, and Explicit Clone

Here is what the simplest statement in programming, `b = a;`, does in Java 6, C++, and Rust:

- In **Java 6**, `b = a;` copies the bits for the eight primitive types (`int`, `long`, `double`, ...), and for *every class type* copies the 8-byte object reference so `a` and `b` now alias the same mutable heap object.
- In **C++**, `b = a;` on a `std::string` or `std::vector` invokes the *copy assignment operator*, which calls `malloc` and duplicates the entire heap buffer behind the scenes. If you want to transfer ownership cheaply instead, you must write `b = std::move(a);`, invoking C++11's *move constructor/assignment*.

Here is the subtle detail from Chapter 2 that makes C++ moves surprisingly tricky: in C++, **moves are non-destructive**. When you write `std::string b = std::move(a);`, variable `a` is still an in-scope C++ object whose destructor `~basic_string()` *must* still execute when the enclosing block hits }. Therefore, C++'s move constructor has to copy `a`'s three words into `b` *and* overwrite `a` into a valid "empty shell" (setting `a.ptr = nullptr, a.len = 0`) so `a`'s destructor does not double-free `b`'s buffer. Even worse, nothing in the C++ type system stops a programmer five lines later from

accidentally reading `a` in its hollowed-out moved-from state — a classic source of production bugs known as *use-after-move*.

Rust avoids both the hidden copy and the hollowed-out shell with a different rule.

**Focus**

**How `let b = a;` works physically in Rust.** At the machine level, *every* variable-to-variable assignment, function argument pass, and return value in Rust is a shallow, bitwise `memcpy` of `size_of::<T>()` stack bytes (and LLVM routinely optimizes even that `memcpy` away into registers). Rust **never** implicitly invokes user-defined copy constructors or deep heap allocations on `=`.
The *only* difference between a **Move** and a **Copy** happens in the compiler's static tracker right after those stack bytes are copied:

1. **Destructive Move (the default for all types):** After copying the 24 stack bytes of `a: String` into `b`, the compiler marks slot `a` **statically uninitialized**. Touching `a` again is a compile-time error, and **no destructor runs on `a` at scope exit**.
2. **`Copy` (opt-in for plain self-contained bit values):** If type `T` implements the marker trait `Copy` (like `i64`, `bool`, a 3-byte `Currency([u8; 3])`, or a shared reference `&U`), both `a` and `b` remain live and usable after the bitwise copy.

This example shows why a language without a garbage collector needs destructive moves to prevent double frees:

```
1 fn process_memo() {
2     let a = String::from("WIRE-9941"); // Heap alloc #1 (9 bytes)
3     let b = a;                         // 24-byte stack copy; `a` is now dead!
4
5     // println!("{}", a);
6     // ^ COMPILE ERROR: borrow of moved value: `a`
7
8     println!("Processing {}", b);
9 } // Only `b` is live here -> Drop frees Heap alloc #1 exactly ONCE.
```

Look at what would happen if `a` remained usable after `let b = a;` without cloning the heap buffer: both `a.ptr` and `b.ptr` would hold the exact same heap address, creating two mutable aliases to the same buffer during `process_memo` *and* causing `Drop` to call `free()` twice on the same pointer at } (a heap-corrupting **double free**). By killing `a` statically at line 3, Rust transfers unique ownership in three register moves — zero `nullptr` writes into `a`, zero branch inside `a`'s non-existent destructor, and zero possibility of use-after-move.

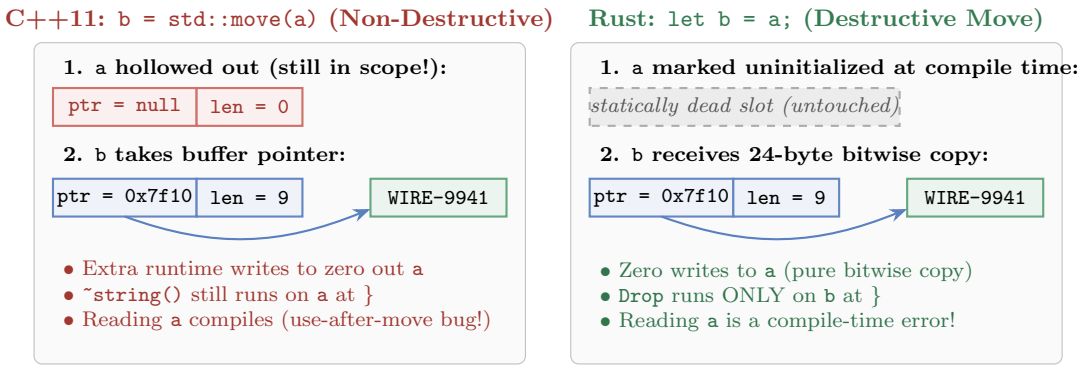



Figure 3.3: Why Rust's destructive move is both safer and faster than C++11's `std::move`. C++ must leave `a` in a valid empty state and run its destructor at scope exit; Rust invalidates `a` inside the type checker.

Formal

**What if ownership is moved inside only one branch of an `if`?** Suppose `if flag { drop(a); }` moves and drops `a` only when `flag` is `true`. At compile time, touching `a` after the `if` is rejected because `a` *might* have been moved. At runtime, how does the closing brace } know whether to drop `a`? Only in such dynamically conditional moves does `rustc` allocate a tiny 1-byte boolean **drop flag** on the stack frame, setting it to `false` inside the branch that moved `a`, and checking that stack byte at scope exit. For all straight-line moves, the cost is zero instructions.

### 3.3.1 When Is a Type Allowed to Be `Copy`? And When Do We `.clone()`?

For a 64-bit integer `let x: i64 = 5000; let y = x;`, invalidating `x` would be silly: an `i64` owns no heap buffer, file descriptor, or database lock — its 8 stack bytes *are* the entire value. The same is true for a compact 3-byte currency code `Currency(*b"USD")` or a 16-byte money value `Money { minor: i64, currency: Currency }`. You opt your own small value types into `Copy` explicitly:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Currency(pub [u8; 3]);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Money {
    pub minor_units: i64,
    pub currency: Currency,
}
```

Can you slap `#[derive(Copy, Clone)]` onto `Account` if `Account` contains an owned `owner_name: String`? Try it, and `rustc` refuses immediately: `the trait 'Copy' cannot be implemented for this type; field 'owner_name' does not implement 'Copy'`. In fact, Rust enforces a strict theorem connecting `Copy` and `Drop`:

**Theorem 3.1** (Mutual Exclusion of `Copy` and `Drop`)**.** *No Rust type `T` can implement both `Copy` and `Drop`. A type may implement `Copy` only if every one of its fields is itself `Copy` and the type requires no custom cleanup logic on scope exit.*

*Proof.* Suppose a type `T` implemented both `Copy` (meaning `let b = a;` duplicates `T`'s stack bytes while leaving `a` live) and `Drop` (meaning `T` owns an external resource cleaned up by `Drop::drop(&mut self)` when each live variable exits scope). Then `let b = a;` would produce two bitwise-identical owners of the same external resource, and at scope exit `Drop::drop` would run on both `b` and `a`, freeing or closing that resource twice. Forbidding `Copy + Drop` ensures bitwise duplication never aliases owned resources. □

Aliasing XOR Mutability also decides which reference types are `Copy`:

- A **shared reference `&T` implements `Copy`** (for *every* `T`, even when `T` itself is a non-`Copy` `String` or `Vec`). Why? Because `&T` only grants read access, and Aliasing XOR Mutability allows any number of simultaneous readers. Copying the 8-byte pointer `&Account` creates one more reader.
- An **exclusive mutable reference `&mut T` is NOT `Copy`**. If `let r2 = r1;` copied `r1: &mut Account` while leaving `r1` usable, you would instantly hold *two* active mutable references to the same `Account`, breaking Aliasing XOR Mutability. Moving `r1` into `r2` invalidates `r1`.

Finally, what if you want a second, independent heap copy of a `String` or a `Vec<Account>`? You call `.clone()` explicitly: `let b = a.clone();`. This makes performance review easier than in C++, because in Rust neither `=` nor passing an argument can hide an $O(n)$ heap allocation. Every deep copy says `.clone()` right on the page.

## 3.4 Borrowing, Reborrowing, and Splitting Slices

If passing a non-`Copy` value into a function always moved it, every helper would have to take `Account` by value and hand it back as `(Account, Result)` so the caller could keep using the account afterward.

Instead, callers lend temporary access via **borrowing** — creating either a shared reference `&T` or an exclusive mutable reference `&mut T`. Watch how the borrow checker handles our opening C++ and Java ledger bug at compile time:

```
fn append_bonus_if_positive(postings: &mut Vec<i64>) {
    let first: &i64 = &postings[0]; // Shared borrow of `postings` starts
    if *first > 0 {
        postings.push(500);         // Needs `&mut postings` while `first` is live!
        println!("Triggered by posting: {}", first);
    }
}
```

Feed those seven lines to `rustc` and read the exact diagnostic it prints:

```
error[E0502]: cannot borrow `*postings` as mutable because it is also borrowed as
    immutable
 --> src/lib.rs:4:9
  |
2 |     let first: &i64 = &postings[0];
  |                       -------- immutable borrow occurs here
3 |     if *first > 0 {
4 |         postings.push(500);
  |         ^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

```
5 |         println!("Triggered by posting: {}", first);
  |                                               ----- immutable borrow later used here
```

The diagnostic names the three lines that form the Aliasing + Mutation triangle: line 2 creates an alias into the heap buffer (`first`), line 4 attempts to mutate (and potentially reallocate) the vector, and line 5 reads through the alias afterward. And notice the instant fix: copy the 8-byte integer out (`let first: i64 = postings[0];`) *or* finish reading `first` before calling `postings.push(500)`:

```rust
fn append_bonus_if_positive_fixed(postings: &mut Vec<i64>) {
    let first_ref: &i64 = &postings[0];
    if *first_ref > 0 {
        let saved: i64 = *first_ref; // Last read of `first_ref` ends its borrow!
        postings.push(500);          // Safe: zero aliases into `postings` remain.
        println!("Triggered by posting: {}", saved);
    }
}
```

Wait — why did `first_ref`'s borrow end right at line 4 instead of lasting all the way down to the closing brace on line 7? Because modern Rust uses **Non-Lexical Lifetimes (NLL)**, tracking borrows up to their *last actual use* along the control-flow graph rather than stubbornly locking variables until }. Chapter 4 opens up the NLL region solver in full detail.

### 3.4.1 Implicit Reborrowing: Why Doesn't Passing `&mut T` Move It?

Pause for a puzzle that trips up observant C++ and Java programmers who pay close attention to types. At the end of §3.3, we proved that `&mut Account` is **not** `Copy`. Now read the following completely valid Rust code:

```rust
pub fn apply_fee(acct: &mut Account, fee_cents: i64) {
    acct.balance_cents -= fee_cents;
}

pub fn snapshot_balance(acct: &Account) -> i64 {
    acct.balance_cents
}

pub fn close_month(r: &mut Account) -> i64 {
    apply_fee(r, 250);       // Wait: why didn't passing `r` move and kill `r`?!
    apply_fee(r, 100);       // `r` still works on the very next line!
    snapshot_balance(r)      // And coerces cleanly from `&mut Account` to `&Account`!
}
```

If `r: &mut Account` is not `Copy`, why didn't passing `r` into the first `apply_fee(r, 250)` move `r` and make line 11 a compile error?

Because whenever a target type is known to be a reference (`&mut Account` or `&Account`), the Rust compiler automatically inserts a **reborrow**: it rewrites `apply_fee(r, 250)` into `apply_fee(&mut *r, 250)`, and `snapshot_balance(r)` into `snapshot_balance(&*r)`.

**Intuition**

Reborrowing works like a stack of temporary permissions:

1. Owner `acct: Account` sits at the bottom of the permission stack.
2. `r: &mut Account` borrows exclusive access from `acct`. While `r` is active, `acct` itself is frozen.
3. Calling `apply_fee(&mut *r, 250)` creates a shorter-lived reborrow `r_sub: &mut Account` on top of `r`. For the duration of `apply_fee`, *both* `acct` and `r` are frozen; `r_sub` is the only usable pointer to the account.
4. As soon as `apply_fee` returns, `r_sub` pops off the stack, and exclusive access returns intact to `r`.

At no microsecond were two `&mut Account` references simultaneously usable. Only the top-most reference on the reborrow stack is active at any instant.

When does implicit reborrowing *not* kick in? When you pass `r` to a generic parameter, as in `fn identity<T>(x: T) -> T`, the signature does not say the argument is a reference. `T` becomes `&mut Account`, and `r` moves. Writing `identity(&mut *r)` reborrows explicitly and keeps `r` usable.

### 3.4.2 Disjoint Struct Fields and Splitting Slices (`split_at_mut`)

As soon as you build real domain structs — say a `LedgerEngine` holding both a fee table and a vector of accounts — you run into a natural question: what if a method needs to *read* `self.fee_bps` while *mutating* `self.accounts`?

```
1  pub struct LedgerEngine {
2      pub fee_bps: i64,
3      pub accounts: Vec<Account>,
4  }
5
6  impl LedgerEngine {
7      // Pitfall: helper borrows ALL of `&self`, including `self.accounts`!
8      fn compute_fee_bad(&self, amount: i64) -> i64 {
9          (amount * self.fee_bps) / 10_000
10     }
11
12     pub fn charge_all(&mut self, amount: i64) {
13         for acct in &mut self.accounts {
14             // Calling `self.compute_fee_bad(amount)` here FAILS to compile:
15             // `self.accounts` is mutably borrowed by the loop, while
16             // `&self` asks to borrow the entire `LedgerEngine` struct!
17             let fee = (amount * self.fee_bps) / 10_000; // OK: disjoint field!
18             acct.balance_cents -= fee;
19         }
20     }
21 }
```

Notice why line 17 compiles while calling `self.compute_fee_bad(amount)` would fail:

- Inside a single function body, Rust's borrow checker understands **disjoint field projection**:

`self.accounts` and `self.fee_bps` are different fields, so they occupy non-overlapping bytes of the struct, so `&mut self.accounts` and `&self.fee_bps` can coexist safely.

- Across a method boundary, however, Rust checks function bodies strictly against their *signatures* (so changing a helper's implementation never silently breaks callers). A signature `fn compute_fee_bad(&self, ...)` claims a shared borrow of *every* field of `LedgerEngine`, colliding with `&mut self.accounts`. The idiomatic Rust fix is clean: make the helper take only the field it actually needs (`fn compute_fee(fee_bps: i64, amount: i64) -> i64`), or factor configuration fields into a sub-struct (`&self.config`).

Now push that same geometric question one step further: what if you have a contiguous slice `accounts: &mut [Account]` and need to move funds from index `from_idx` to index `to_idx`? If you write:

```
// FAILS TO COMPILE: cannot borrow `accounts[_]` as mutable more than once!
// let src = &mut accounts[from_idx];
// let dst = &mut accounts[to_idx];
```

The compiler must reject those two lines because `from_idx` and `to_idx` are runtime integers. What if a caller passes `from_idx == 3` and `to_idx == 3`? Then `src` and `dst` would be **two simultaneous `&mut Account` references pointing to the exact same 24 bytes in memory**, violating Aliasing XOR Mutability (and in banking logic, potentially crediting an account from itself or miscomputing a balance check).

How do we obtain two simultaneous `&mut Account` references into the same slice safely? We check `from_idx != to_idx` and call the standard library's `split_at_mut(mid)` method, which cuts one `&mut [T]` at index `mid` into two non-overlapping mutable sub-slices `(&mut [T], &mut [T])` covering `0..mid` and `mid..len`:

```
pub fn transfer_in_slice(
    accounts: &mut [Account],
    from_idx: usize,
    to_idx: usize,
    amount_cents: i64,
) -> Result<(), &'static str> {
    if from_idx == to_idx {
        return Err("source and destination must be distinct accounts");
    }
    if amount_cents <= 0 {
        return Err("transfer amount must be positive");
    }
    let (from_acct, to_acct) = if from_idx < to_idx {
        let (left, right) = accounts.split_at_mut(to_idx);
        (&mut left[from_idx], &mut right[0])
    } else {
        let (left, right) = accounts.split_at_mut(from_idx);
        (&mut right[0], &mut left[to_idx])
    };
    if from_acct.currency != to_acct.currency {
        return Err("currency mismatch");
    }
    if from_acct.balance_cents < amount_cents {
        return Err("insufficient funds");
    }
```

```
    from_acct.balance_cents -= amount_cents;
    to_acct.balance_cents += amount_cents;
    Ok(())
}
```

To convince `rustc` that `from_acct` and `to_acct` never alias, the function had to handle `from_idx == to_idx` up front. Under the hood (as we will build from scratch in Chapter 4), `split_at_mut` is six lines of pointer arithmetic verified once in `core::slice`, exposing a completely safe interface with zero heap allocations. (In modern Rust since 1.86, slices even provide a convenience wrapper `get_disjoint_mut([from_idx, to_idx])` built on this exact principle.)

## 3.5 Shared Ownership: `Rc<T>` versus `Arc<T>`

Single ownership handles 90% of systems code cleanly: an HTTP request owns its parsed payload, a transaction owns its vector of postings, and helper functions borrow `&T` or `&mut T` views on the way down the call stack.

Occasionally, however, a piece of data genuinely has *multiple co-equal owners* whose lifetimes do not nest neatly inside a single call frame. Imagine an in-memory `FeeSchedule` loaded from PostgreSQL every five minutes: forty in-flight request handlers are currently pricing transfers against version $k$ of that schedule when version $k+1$ arrives. You want version $k$ to stay alive in memory until the last of those forty handlers finishes, and then drop immediately.

In Java 6, you hand all forty handlers a normal object reference and let the garbage collector eventually sweep version $k$. In C++11, you reach for `std::shared_ptr<FeeSchedule>`. Notice two silent costs baked into C++'s `std::shared_ptr<T>`:

1. Every copy and destruction of `std::shared_ptr<T>` executes an **atomic hardware read-modify-write instruction** (`LOCK XADD` on x86-64) on the control block's reference count — even in a purely single-threaded GUI, compiler pass, or event loop that never spawns a thread. Atomic increments stall CPU store buffers and bounce cache lines across cores, costing roughly 5× to 15× more than an ordinary register increment.
2. `std::shared_ptr<FeeSchedule>` hands out mutable `FeeSchedule&` access by default. Two threads sharing a `shared_ptr<FeeSchedule>` have thread-safe *reference counts*, while mutating the `FeeSchedule` fields themselves is an instant, undefined-behavior data race.

Rust splits reference-counted shared ownership into two distinct smart pointers so you never pay for thread synchronization you do not use, and never accidentally mutate aliased data:

- `Rc<T>` **(Reference Counted — Single-Threaded Only):** An 8-byte non-null pointer on the stack pointing to a heap allocation containing `(strong_count: usize, weak_count: usize, value: T)` laid out contiguously in a single heap block. Calling `Rc::clone(&sched)` increments `strong_count` with a plain, non-atomic CPU `add` instruction and returns another 8-byte `Rc<T>` pointer. Better still, `Rc<T>` intentionally opts *out* of Rust's `Send` and `Sync` concurrency marker traits: if you try to pass an `Rc<T>` into `std::thread::spawn`, **your program fails to compile** rather than suffering a reference-count race at runtime.
- `Arc<T>` **(Atomically Reference Counted — Thread-Safe):** Identical 8-byte stack pointer and contiguous heap block `(strong: AtomicUsize, weak: AtomicUsize, value: T)`, ex-

cept `Arc::clone(&sched)` and `Drop` use hardware atomic operations (`Ordering::Relaxed` on increment, `Release`/`Acquire` fence on decrement to zero). Whenever `T` is safe to share across threads, `Arc<T>` can be cloned across worker threads freely.

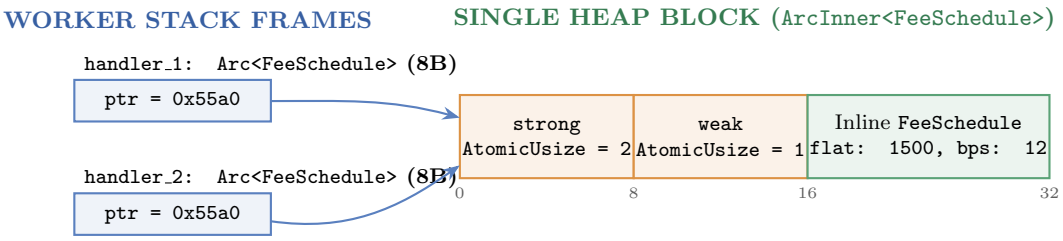



Figure 3.4: Exact memory layout of `Arc<FeeSchedule>` (and `Rc<FeeSchedule>`, with non-atomic `Cell<usize>` counters). Dereferencing either handle yields only `&FeeSchedule` (shared read-only). When `strong` drops to 0, `FeeSchedule` is dropped in place; when `weak` also hits 0, the 32-byte block is freed.

Note the caption of Figure 3.4: **dereferencing `Rc<T>` or `Arc<T>` only ever gives you `&T`, never `&mut T`.** Why? Because the purpose of `Rc` and `Arc` is that *multiple handles can alias the same heap allocation*, and by Aliasing XOR Mutability (§3.1), aliased memory cannot be mutated directly. (The one exception is `Arc::get_mut(&mut arc)` or copy-on-write `Arc::make_mut(&mut arc)`, which checks at runtime whether `strong_count == 1 && weak_count == 0`: if you happen to be the *sole* owner right now, it hands you `&mut T` in place; if others are sharing it, `make_mut` clones `T` into a private allocation before mutating.)

*Remark* 3.1 (Reference Cycles and `Weak<T>`). Unlike a tracing garbage collector, reference counting cannot collect isolated cycles: if node $A$ holds an `Rc` to node $B$ and node $B$ holds an `Rc` back to node $A$, both `strong_counts` stay $\geq 1$ forever and leak memory (safe Rust considers leaking memory undesirable, though memory-safe — no dangling pointer is created). Whenever a parent-child relationship exists, parents own children via `Rc<Node>` (`strong`) while children point upward via `Weak<Node>` (`Rc::downgrade(&parent)`), which increments `weak_count` without keeping the value alive; reading a `Weak<Node>` requires calling `.upgrade() -> Option<Rc<Node>>` to check whether the parent still exists. Even better, as Chapter 8 shows, idiomatic Rust usually represents graphs as a flat `Vec<Node>` indexed by integer IDs (`NodeId(u32)`), avoiding reference-counted pointer webs altogether.

## 3.6 Controlled Interior Mutability: `Cell`, `RefCell`, and `Mutex`

By now a practical question should be nagging at you: *what if two parts of a program genuinely DO need to share ownership of a cache, metrics counter, or account ledger AND mutate it over time?*

Dropping Aliasing XOR Mutability here would bring back the Java 6 and C++ bugs from §3.1. Instead, Rust provides **interior mutability** containers — types that let you mutate an inner value through a shared reference `&Container`, **because the container itself enforces Aliasing XOR Mutability dynamically or structurally at its boundary.** (All of them are built on a single compiler-blessed primitive called `UnsafeCell<T>`, whose mechanics we inspect in Chapter 4.)

Pick the container by asking two simple questions: *is this shared on a single thread or across multiple threads*, and *do callers need references inside the data or just value swaps?*

### 3.6.1 1. `Cell<T>`: Zero-Byte-Overhead Value Replacement (Single-Threaded)

Suppose a single-threaded `QuoteClient` struct needs to increment a `request_count: u64` every time `fetch_quote(&self, symbol: &str)` is called through a shared `&QuoteClient` reference. Wrap the counter in `std::cell::Cell<u64>`:

```rust
use std::cell::Cell;

pub struct QuoteClient {
    endpoint: String,
    request_count: Cell<u64>, // Exact same 8 bytes in memory as a plain `u64`!
}

impl QuoteClient {
    pub fn fetch_quote(&self, symbol: &str) -> String {
        let next = self.request_count.get() + 1; // Copies 8 bytes out
        self.request_count.set(next);            // Overwrites 8 bytes in place
        format!("{}:{}#{}", self.endpoint, symbol, next)
    }
}
```

Why is `self.request_count.set(next)` completely sound through a shared `&self` without any lock or runtime borrow counter? **Because `Cell<T>` never gives out a `&T` or `&mut T` reference to the bytes inside it.** You can only copy values out (`.get()` when `T: Copy`), overwrite the whole value (`.set(new_val)`), or swap a non-`Copy` value in and out atomically on one thread (`.replace(new_val)` / `.take()`). Since no pointer into a `Cell<T>`'s interior can ever exist, overwriting its contents can never invalidate a dangling reference — and because `Cell<T>` is `!Sync` (cannot be shared across threads), no second thread can race with those writes. `size_of::<Cell<u64>>()` is 8 bytes, and `.set()` compiles down to a single `mov` instruction.

### 3.6.2 2. `RefCell<T>`: Single-Threaded Runtime Borrow Checking

What if you are on a single thread and hold a large `Vec<Posting>` or `HashMap` inside an `Rc<...>`, where copying the entire collection in and out of a `Cell` on every read would be absurd? You want real `&Vec<Posting>` and `&mut Vec<Posting>` access, checked dynamically at runtime instead of compile time. That is `std::cell::RefCell<T>`.

In memory on a 64-bit machine, `RefCell<T>` stores `T` alongside one extra 8-byte signed integer `borrow: Cell<isize>`:

- `borrow` $== 0$: currently unborrowed.
- `borrow` $> 0$: currently borrowed immutably by `borrow` active `Ref<'_, T>` smart-pointer guards. Calling `.borrow()` increments `borrow` by 1; when each `Ref` guard drops at scope exit, its `Drop` implementation decrements `borrow` by 1.
- `borrow` $== -1$: currently borrowed mutably by **one** exclusive `RefMut<'_, T>` guard. When that `RefMut` guard drops, `borrow` resets to 0.

If your code calls `.borrow_mut()` while `borrow != 0` (or calls `.borrow()` while `borrow == -1`), `RefCell` catches the Aliasing + Mutation violation immediately on that exact line and **panics**

(unwinding the thread cleanly before a single byte of undefined behavior can occur; if you prefer a recoverable `Result`, call `.try_borrow_mut()`).

```rust
use std::cell::RefCell;

fn audit_and_append(log: &RefCell<Vec<i64>>) {
    {
        let r1 = log.borrow(); // borrow state: 0 -> +1
        let r2 = log.borrow(); // borrow state: +1 -> +2 (multiple readers OK!)
        assert_eq!(r1.len(), r2.len());
    } // `r2` and `r1` drop here -> borrow state returns to 0!

    log.borrow_mut().push(750); // borrow state: 0 -> -1 -> push -> drops back to 0!
}
```

**Common trap**

**Keep `Ref` and `RefMut` guards in tight scopes.** A classic beginner bug with `RefCell` is binding `let entries = log.borrow();` at the top of a 60-line function and then calling a helper method 40 lines later that internally calls `log.borrow_mut()`, triggering a runtime `already borrowed: BorrowMutError` panic. Drop your `Ref` guard explicitly with `drop(entries);` or scope it inside `{ ... }` right before calling anything else that touches the same `RefCell`.

### 3.6.3 3. `Mutex<T>` and `RwLock<T>`: Putting the Data *Inside* the Lock

Now step into multi-threaded server engineering — where an Axum HTTP service handles thousands of concurrent requests across worker threads. Compare how Java 6 (and C++) attach a lock to shared state versus how Rust does it.

In Java 6, a lock and the fields it protects are **two separate objects** connected only by a comment in the author's head:

```java
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class SharedVault {
  private final Lock lock = new ReentrantLock(); // Sits NEXT to the data
  private long balanceCents = 100000L;

  public void withdraw(long cents) {
    lock.lock();
    try {
      balanceCents -= cents;
    } finally {
      lock.unlock();
    }
  }

  // Six months later, a hurried teammate adds a getter WITHOUT `lock.lock()`:
  public long peekBalanceBuggy() {
    return balanceCents; // SILENT DATA RACE! `javac` compiles with 0 warnings!
  }
```

```
}
```

Experienced Java and C++ engineers ship bugs like `peekBalanceBuggy()` regularly: forgetting to acquire the lock in a newly added method, forgetting `try/finally` so an early return leaks a held lock, or returning a reference to a mutable `HashMap` stored inside the class so the caller keeps reading and mutating the map *after* `finally { lock.unlock(); }` has already released the lock.

Look at `std::sync::Mutex<T>` in Rust. In Rust, **a `Mutex` is a generic container that physically wraps and owns the `T` inside itself**:

```rust
use std::sync::{Arc, Mutex};

pub fn concurrent_withdrawal_demo() {
    let vault: Arc<Mutex<Account>> = Arc::new(Mutex::new(Account {
        id: 99,
        balance_cents: 100_000,
        currency: *b"USD",
    }));

    let worker_vault = Arc::clone(&vault);
    let handle = std::thread::spawn(move || {
        // You CANNOT touch `balance_cents` on `worker_vault` directly!
        // `.lock()` acquires the OS mutex and returns an RAII `MutexGuard<Account>`:
        let mut guard = worker_vault.lock().unwrap();
        guard.balance_cents -= 15_000;
    }); // `guard` drops right here -> OS mutex unlocks deterministically!

    handle.join().unwrap();
    assert_eq!(vault.lock().unwrap().balance_cents, 85_000);
}
```

Each of the three properties below is enforced by the type system:

1. **Impossible to forget the lock:** `Mutex<Account>` has no `.balance_cents` field. The *only* API that yields `&mut Account` is `.lock()`, which blocks until the OS mutex is acquired and returns `MutexGuard<'_, Account>`.

2. **Impossible to forget to unlock:** `MutexGuard` implements `Drop`; whether the scope exits normally, hits an early `return` or `?`, or unwinds on a panic, the lock is released at the exact end of `guard`'s scope. (Even better: if a thread panics while holding `MutexGuard`, Rust marks the mutex *poisoned* — returning `Err(PoisonError)` on `.lock()` — warning other threads that an invariant might have been left half-updated.)

3. **Impossible to leak a reference out past unlock:** Suppose someone writes `let leaked: &mut i64 = &mut vault.lock().unwrap().balance_cents;` trying to use `leaked` after the lock releases. The borrow checker sees that `leaked` borrows from the temporary `MutexGuard`, which drops at the semicolon — and **rejects the code at compile time**.

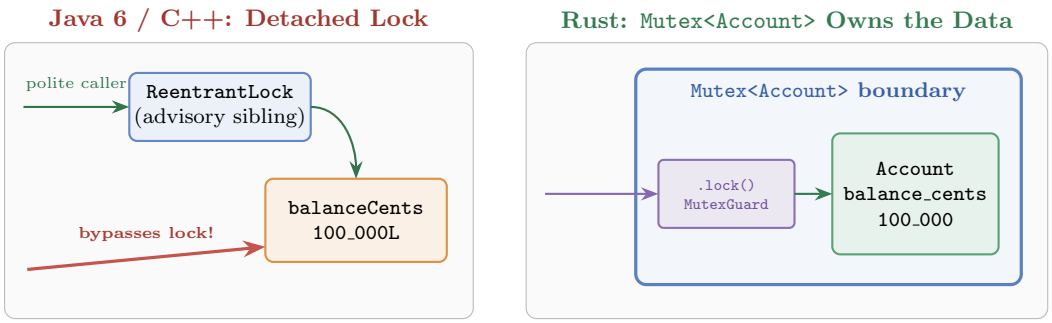



Figure 3.5: Why data races vanish in safe Rust. In Java 6 and C++, locks sit *beside* the fields they protect, so forgetting to lock compiles cleanly. In Rust, `Mutex<T>` (and `RwLock<T>`) wraps `T` inside its type boundary; `MutexGuard` is the sole physical doorway to `&mut T`.

When reads heavily outnumber writes (such as our five-minute `FeeSchedule` refresh read by thousands of HTTP requests per second), swap `Mutex<T>` for `std::sync::RwLock<T>`: `.read()` hands out concurrent shared `RwLockReadGuard<'_, T>` handles across all CPU cores simultaneously, while `.write()` waits for exclusive `RwLockWriteGuard<'_, T>` access — the exact runtime multi-threaded twin of Rust's compile-time `&T` versus `&mut T`.

### 3.6.4 Putting the Entire Pointer & Ownership Toolkit Together

Table 3.1 summarizes every ownership, borrowing, and interior-mutability type we have built in this chapter, along with its exact 64-bit stack size (assuming a sized type `T`, such as our 24-byte `Account`) and when Aliasing XOR Mutability is checked.

| Type | Stack Size | Ownership | Threads | Aliasing XOR Mutability |
|---|---|---|---|---|
| `T` | `size_of::<T>()` | Unique inline | `Send`/`Sync` if `T` | Compile time (0 cost) |
| `&T` / `&[U]` | 8B / 16B | Shared borrow | Cross-thread | Compile time ($N$ readers) |
| `&mut T` / `&mut [U]` | 8B / 16B | Unique borrow | Exclusive | Compile time (1 writer) |
| `Box<T>` | 8B | Unique heap | `Send`/`Sync` if `T` | Compile time (0 header) |
| `Rc<T>` | 8B | Shared heap | Single (`!Send`) | Compile time (`&T` only) |
| `Arc<T>` | 8B | Shared heap | Atomic (`Send+Sync`) | Compile time (`&T` only) |
| `Cell<T>` | `size_of::<T>()` | Inline wrapper | Single (`!Sync`) | Compile time (value swap) |
| `RefCell<T>` | +8B tag | Inline wrapper | Single (`!Sync`) | Runtime `isize` (panics) |
| `Mutex<T>` | Lock + `T` | Inline wrapper | `Send+Sync` | Runtime OS `MutexGuard` |
| `RwLock<T>` | RwLock + `T` | Inline wrapper | `Send+Sync` | Runtime OS read/write guards |

Table 3.1: The complete Rust ownership, borrowing, and interior-mutability map on a 64-bit target.

**In production**

In Chapters 12 through 14, when we wire up our production Axum financial service backed by PostgreSQL, almost every shared service handle in `AppState` will be a cheap-to-clone `Arc<...>` wrapping immutable config and a thread-safe `PgPool`, while per-request ledger mutations happen either on uniquely owned local values or inside PostgreSQL row locks (`SELECT ... FOR UPDATE`) rather than global process-wide mutexes. Knowing every row of Table 3.1 tells you immediately why that architecture scales across 64 cores without contention.

In the next chapter, we look directly at the lifetime tick marks (`'a`) we glimpsed on borrowed references and guards — stripping away every trace of mysticism around how the compiler tracks regions, when lifetimes can be elided, why `String: 'static` is true, and how six lines of carefully audited `unsafe` code power `Vec<T>` and `split_at_mut`.

## 3.7 Exercises

**Exercise 3.1** (Spot the hidden Aliasing + Mutation triangle)**.** Consider the following single-threaded C++ function operating on a `std::unordered_map<std::string, int64_t> balances`:

```
void match_default_balance(std::unordered_map<std::string, int64_t>& balances,
                           const std::string& new_user) {
  const int64_t& template_bal = balances["DEFAULT_TIER"];
  balances[new_user] = template_bal;
}
```

(a) Trace step-by-step how line 4 can trigger undefined behavior (reading freed memory) even when `"DEFAULT_TIER"` is already present in `balances`. Which internal resize event inside `operator[]` invalidates `template_bal`?
(b) Translate those two lines into Rust using `HashMap<String, i64>` from `std::collections`, explain why `rustc` rejects the naive version, and write the one-line safe fix.

**Exercise 3.2** (Exact 64-bit byte accounting)**.** On a 64-bit Linux x86-64 machine, compute the exact stack byte size *and* total heap byte allocation (excluding internal `malloc` bin metadata) right after all three bindings below execute:

```
let mut codes: Vec<String> = Vec::with_capacity(4);
codes.push(String::from("USD"));
codes.push(String::from("CHF"));
let view: &[String] = &codes[0..2];
```

(a) How many bytes sit on the stack frame for `codes` and `view` combined?
(b) How many distinct heap allocations exist, and what are their exact sizes and byte contents?
(c) Compare your answer to storing two three-letter currency codes in `Vec<[u8; 3]>` with capacity 4: how many heap allocations and total heap bytes does `Vec<[u8; 3]>` use, and why?

**Exercise 3.3** (Destructive move versus C++ `std::move`)**.** Suppose `Buffer` owns a 4,096-byte heap array. Trace what happens in machine instructions and destructor calls for:

```
fn relay(b1: Buffer) -> Buffer {
    let b2 = b1;
    b2
}
```

versus the C++11 equivalent `Buffer relay(Buffer b1) { Buffer b2 = std::move(b1); return b2; }` (assuming NRVO does not elide `b1` into `b2`). Specifically, how many times does `~Buffer()` execute in C++ versus `Drop::drop` in Rust, and what prevents Rust from ever needing a "moved-from empty state" inside `Buffer`?

**Exercise 3.4** (Why `&T` is `Copy` while `&mut T` is not)**.** Suppose Rust mistakenly implemented `Copy` for `&mut T`. Construct a 6-line safe Rust program (using zero `unsafe` blocks and zero threads) that exploits `&mut T: Copy` to cause a genuine memory-corruption use-after-free crash on `Vec<String>`.

**Exercise 3.5** (Reborrowing and generic functions)**.** Explain why `call_twice_concrete` below compiles cleanly while `call_twice_generic` fails to compile on the second call, and show the 5-character edit at the call site inside `call_twice_generic` that makes it compile:

```
fn touch_concrete(x: &mut i64) { *x += 1; }
fn touch_generic<T>(x: T) -> T { x }

fn call_twice_concrete(r: &mut i64) {
    touch_concrete(r);
    touch_concrete(r);
}

fn call_twice_generic(r: &mut i64) {
    let _ = touch_generic(r);
    // let _ = touch_generic(r); // Why does this line fail without an explicit reborrow?
}
```

**Exercise 3.6** (Generalizing `split_at_mut` to three accounts)**.** In §3.4 we wrote `transfer_in_slice` to obtain two disjoint `&mut Account` references from `accounts: &mut [Account]`. Now suppose every transfer also credits a fee directly to a house fee account at index `fee_idx` inside the *same* `&mut [Account]` slice, requiring **three** simultaneous mutable references `(&mut Account, &mut Account, &mut Account)` for `(from_idx, to_idx, fee_idx)`.
(a) What precondition on `(from_idx, to_idx, fee_idx)` must hold for three simultaneous mutable borrows to be sound?
(b) Write a safe Rust helper `borrow_three_mut<T>(slice: &mut [T], i: usize, j: usize, k: usize)` returning `Option<(&mut T, &mut T, &mut T)>` using two calls to `split_at_mut` (Hint: sort the three indices along with their original positions first).

**Exercise 3.7** (Tracing `RefCell<T>` borrow states)**.** For the snippet below, write the exact integer value stored in `cell`'s internal `borrow: isize` counter at the end of each numbered comment line, or state which line panics at runtime and why:

```
let cell = std::cell::RefCell::new(vec![10, 20]); // (1)
let a = cell.borrow();                            // (2)
let b = cell.borrow();                            // (3)
drop(a);                                          // (4)
let ok = cell.try_borrow_mut().is_ok();           // (5) What is `ok`, and what is `borrow`?
drop(b);                                          // (6)
let mut w = cell.borrow_mut();                    // (7)
w.push(30);
drop(w);                                          // (8)
```

**Exercise 3.8** (Choosing the sharpest smart-pointer combination)**.** For each engineering requirement below, pick the simplest, lowest-overhead Rust type from Table 3.1 (e.g., `T`, `Box<T>`, `Rc<T>`, `Arc<T>`, `Rc<RefCell<T>>`, `Arc<RwLock<T>>`, `Arc<Mutex<T>>`) and justify in two sentences why both weaker and heavier choices are worse:

(a) A read-only TLS certificate chain loaded at startup and inspected on every incoming HTTPS connection across 32 Tokio worker threads.

(b) A single-threaded compiler pass where AST expression nodes share references to symbol-table entries whose inferred type slot starts `None` and gets filled in during type checking.

(c) An in-memory FX rate map read 50,000 times per second across 16 threads and replaced once every 2 seconds by a background market-data feed thread. (Bonus: compare wrapping the map in `Arc<RwLock<RateMap>>` versus storing `Mutex<Arc<RateMap>>` so readers clone an immutable `Arc<RateMap>`)

**Exercise 3.9** (Why `MutexGuard` cannot leak an unprotected reference)**.** A Java 6 developer accustomed to `synchronized` writes the following Rust function intending to "grab a mutable reference quickly under the lock and mutate it afterward without holding the lock":

```
use std::sync::Mutex;

fn unlocked_ref_attempt(m: &Mutex<Vec<i64>>) {
    // let r: &mut Vec<i64> = &mut *m.lock().unwrap();
    // r.push(42);
}
```

Wait: uncomment those two lines and test them mentally (or in `rustc`). Does `let r: &mut Vec<i64> = &mut *m.lock().unwrap();` compile when `r` is used on the next line? Explain carefully what happens to the temporary `MutexGuard`'s lifetime versus when someone tries `let r = { let mut g = m.lock().unwrap(); &mut *g };`, and why *neither* form allows `r.push(42)` to execute while the mutex is unlocked.

# Chapter 4

# Lifetimes Without Mysticism: Regions, Elision, and the Unsafe Boundary

Chapter 3 established the physical rules of Rust memory: every heap buffer or stack slot has a single owner, moving copies a small stack header to transfer ownership, and dropping runs deterministically at scope exit. Borrowing lets code read (`&T`) or mutate (`&mut T`) memory without taking ownership, under one aliasing rule — at any point in execution, you may hold either any number of shared readers or one exclusive writer, never both at once.

Sooner or later you write a function that accepts two string slices and returns one of them, or you define a parser struct holding slices into a network buffer, and the compiler asks for a *lifetime annotation* written with a tick mark: `'a`. Engineers arriving from Java 6 or C++ often guess at first that `'a` tells the allocator how long to keep an object alive in memory.

At the hardware level, nothing of the sort happens. A lifetime parameter `'a` generates **zero machine instructions** and **never extends or shortens when any value is created, moved, or freed**. Every local variable still drops at the exact point Chapter 3 described. Instead, `'a` is a compile-time region variable in a constraint solver: it states in a function or type signature which owner's memory a reference may point into, proving before code generation that no pointer ever outlives its referent.

Seeing lifetimes as region constraints over stack frames and control-flow graphs connects every topic in this chapter into one picture: why modern Rust ends borrows at their last read rather than at a closing brace, how the three elision rules fill in everyday signatures automatically, when zero-copy structs holding `&'a str` beat heap allocation (and when owned types beat both), why `&'static str` and `T: 'static` mean completely different things, why `&'a mut T` refuses to shrink lifetimes inside `T`, and how six lines of audited `unsafe` code wrap raw pointer arithmetic behind a safe signature.

---

## 4.1 Demystifying `'a` From the Machine Up

Suppose a payment gateway receives raw ASCII tags such as `"USD:12500"` and needs a helper that strips an optional `"USD:"` prefix without copying bytes or touching the heap. Writing the lifetime

parameter out in full gives:

```rust
pub fn strip_usd_prefix<'a>(tag: &'a [u8]) -> &'a [u8] {
    if tag.len() >= 4 && &tag[..4] == b"USD:" {
        &tag[4..]
    } else {
        tag
    }
}
```

What happens to `<'a>` when `rustc` compiles this function with `-C opt-level=3` on 64-bit x86 Linux? Under the System V AMD64 ABI, an input byte slice `&[u8]` arrives in two 64-bit registers (`rdi` = address, `rsi` = length) and returns in `rax` (address) and `rdx` (length). Here is the exact assembly emitted by `rustc`:

```
strip_usd_prefix:
    mov  rdx, rsi                  ; default return len = input len
    mov  rax, rdi                  ; default return ptr = input ptr
    cmp  rsi, 4                    ; if len < 4, return unchanged
    jb   .Ldone
    cmp  dword ptr [rax], 977556309 ; 0x3a445355 = little-endian "USD:"
    jne  .Ldone
    add  rdx, -4                   ; return len = len - 4
    add  rax, 4                    ; return ptr = ptr + 4
.Ldone:
    ret
```

Nine instructions do the entire job. There is no heap allocation, no reference-count update, no garbage-collector write barrier, and no runtime lifetime metadata alongside the pointer. Even the four-byte check `&tag[..4] == b"USD:"` folded into one 32-bit compare against `0x3a445355`. Once the compiler front end verifies region constraints, every `'a` is erased before LLVM emits machine code.

**Focus**

**The physical law of Rust lifetimes.** A lifetime annotation `'a` describes memory validity; it never alters runtime execution. No lifetime annotation can keep a stack frame alive past `ret` or delay a `Drop` destructor by a single cycle. Owners govern when bytes are freed; `'a` proves that every borrow stops being used beforehand.

Compare how C++, Java 6, and Rust treat a classic hazard: formatting an account label inside a helper and returning a view of the text.

```cpp
// C++17: compiles cleanly under default flags; instant Undefined Behavior!
#include <string>
#include <string_view>

std::string_view format_account_bad(uint64_t id) {
    std::string local = "ACCT-" + std::to_string(id);
    return std::string_view(local); // `local` frees its buffer right here!
}
```

In C++, `std::string_view` has the exact same 16-byte `(ptr, len)` register layout as Rust's `&str`. On return, `std::string_view(local)` copies the pointer and length into `rax` and `rdx`, and `local`'s destructor immediately frees the backing characters. The caller receives a dangling pointer into dead memory — often printing `"ACCT-42"` in isolated unit tests and corrupting wire messages under production load.

Java 6 avoids dangling pointers by outlawing stack buffers and interior slices altogether:

```
// Java 6: memory-safe, paid for by mandatory heap allocation and GC sweeps.
public static String formatAccount(long id) {
    return "ACCT-" + id; // allocates StringBuilder, char[], and String on heap
}
```

Java 6 buys safety by extending *runtime* liveness on the heap until objects become unreachable. Rust keeps C++'s exact stack frames, deterministic destructors, and 16-byte register slices, and proves statically for every borrow in the program that

$$\text{region}(\text{borrow}) \subseteq \text{liveness}(\text{referent}).$$

### 4.1.1 Lexical Scopes vs. Non-Lexical Lifetimes (NLL)

Before Rust's 2018 edition, borrow regions were strictly *lexical*: every reference stayed active from its `let` binding down to the closing brace } of its block, forcing programmers to wrap short reads inside artificial { ... } scopes. Modern Rust computes borrow regions on the **control-flow graph (CFG)** via **Non-Lexical Lifetimes (NLL)**: a borrow spans only the CFG points from creation along paths reaching its *last actual use*.

Trace NLL on a ledger helper that reads the first and last posting in a vector to pick a fee, then pushes the fee onto that same vector:

```
pub fn append_settlement_fee(postings: &mut Vec<i64>) {
    if postings.is_empty() { return; }
    let first: &i64 = &postings[0];                  // L1: shared borrow 'r1 starts
    let last: &i64 = &postings[postings.len() - 1];  // L2: shared borrow 'r2 starts
    let fee = if *first > 10_000 || *last < 0 {      // L3: last read of `first` & `last`
        250
    } else {
        50
    };
    // Both 'r1 and 'r2 end right after L3!
    postings.push(-fee);                             // L4: exclusive borrow 'w (&mut Vec)
}                                                    // L5: lexical brace closes
```

If `first` remained live across `L4`, `postings.push(-fee)` could reallocate the vector's heap buffer and leave `first` dangling. Because neither `first` nor `last` is read after `L3`, NLL ends `'r1` and `'r2` at `L3`, leaving `postings` unborrowed when `L4` takes `&mut Vec<i64>`. Figure 4.1 contrasts the owner's full scope with these flow-sensitive spans.

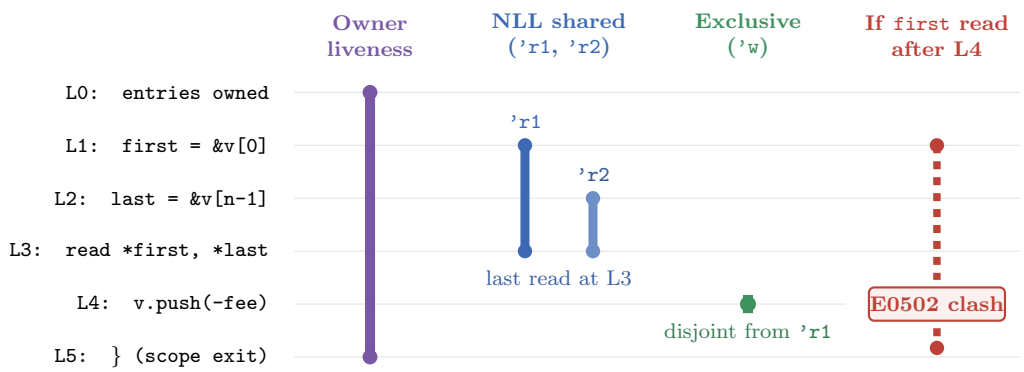



Figure 4.1: Non-Lexical Lifetimes (NLL) on a control-flow graph. Shared borrows `'r1` and `'r2` end at their last read (`L3`) rather than the closing brace (`L5`), allowing `v.push(-fee)` at `L4`. Reading `first` after `L4` stretches `'r1` across `L4` and triggers compile error `E0502`.

Adding `println!("{}", first);` below `L4` stretches `'r1` across the mutation, and `rustc` pinpoints all three lines: where the shared borrow started, where the mutable borrow clashed, and where the shared borrow was later read.

**Intuition**

**Struct field splitting vs. methods.** Borrowing distinct struct fields directly — say `let c = &acct.currency;` followed by `acct.balance_cents -= 500;` — lets `c` remain live across the mutation because `rustc` sees two disjoint byte offsets inside `acct`. Calling a method `acct.debit(500)` takes `&mut self`, borrowing the whole struct at once because callers check methods solely by signature.

## 4.2 Functions, Borrow Signatures, and the Three Elision Rules

Inside one function body, NLL solves every region constraint without annotations. Lifetime parameters appear at **function and type boundaries** because Rust checks functions modularly.

Consider a C++ header declaration in a large trading system:

```
const char* longer_id(const char* a, const char* b);
```

If you pass a long-lived session ID as `a` and a short-lived stack buffer as `b`, then free `b` while keeping the returned pointer, is that safe? From the header alone, nobody can tell whether the return pointer aliases `a`, aliases `b`, points to a static buffer, or came from `malloc`. Worse, changing the `.cpp` body later can silently break callers across the tree without a single compiler warning.

Rust enforces **signature-level modularity**: callers are verified solely against the callee's signature, and the callee's body is verified solely against its own signature. Whenever a function returns a reference, the signature must declare which input borrow(s) the output may alias:

```
pub fn longer_id<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

Formal

**Contract of a shared lifetime variable.** In `fn longer_id<'a>(x: &'a str, y: &'a str) -> &'a str`, for *any* region `'a` where both arguments stay live (liveness($x$) $\supseteq$ `'a` and liveness($y$) $\supseteq$ `'a`), the returned slice stays valid throughout `'a`. At any call site, the largest valid region satisfies

$$\texttt{'a} \subseteq \text{liveness}(x) \cap \text{liveness}(y).$$

When two arguments have different stack scopes, `rustc` solves for their intersection:

```
fn check_ids() {
    let session_id = String::from("SESS-998877"); // lives until end of check_ids
    let chosen: &str;
    {
        let req_id = String::from("REQ-1");       // lives only until inner `}`
        chosen = longer_id(&session_id, &req_id);
        println!("inside inner block: {}", chosen); // OK: both live here
    } // `req_id` drops here
    // println!("after: {}", chosen); // Error E0597: `req_id` does not live long enough
}
```

Even though `"SESS-998877"` is longer than `"REQ-1"` at runtime, `check_ids` inspects only `longer_id`'s signature — never its body — so `chosen` cannot outlive the shorter-lived input `req_id`.

### 4.2.1 Distinct Lifetimes, Outlives Bounds, and Callee Stack Frames

When a function reads a second slice for validation or logging yet always returns data from the first slice, giving each input its own lifetime parameter avoids tying them together:

```
pub fn select_primary<'a, 'b>(primary: &'a str, audit_tag: &'b str) -> &'a str {
    if audit_tag.is_empty() { primary.trim() } else { primary }
}
```

Because `'b` does not appear in `&'a str`, calling `select_primary(&session_id, &req_id)` lets the returned slice live as long as `session_id` after `req_id` drops. When a second borrow *can* be returned as `&'a str` provided it lasts at least as long as `'a`, write the outlives bound `'b: 'a` (`'b` $\supseteq$ `'a`):

```
pub fn pick_bounded<'a, 'b: 'a>(first: &'a str, second: &'b str) -> &'a str {
    if first.len() >= second.len() { first } else { second }
}
```

Common trap

**Why `fn broken_label<'a>(cents: i64) -> &'a str` can never return a local `String`.** Trying to format a local `let s = format!("USD {}", cents);` and return `&s` fails with `E0515` no matter what lifetime parameter you write:

```
pub fn broken_label<'a>(cents: i64) -> &'a str {
    let s = format!("USD {}", cents);
    &s // Error E0515: cannot return reference to local variable `s`
}
```

The caller chooses `'a` in its own stack frame *after* `broken_label` executes `ret`, whereas `s` drops its heap buffer right *before* `ret` pops `rsp`. Return an owned `String` instead (`-> String`): Rust moves the 24-byte `(ptr, len, cap)` header to the caller with zero extra heap copies.

### 4.2.2 Deriving the Three Lifetime Elision Rules

In pre-1.0 Rust, even one-input functions like `fn trim<'a>(s: &'a str) -> &'a str` required explicit ticks. Because nearly 90% of borrow signatures follow three mechanical patterns, Rust expands unannotated (*elided*) references deterministically before borrow checking. Parameter lifetimes are called *input lifetimes*; return-type lifetimes are *output lifetimes*.

**Definition 4.1** (The Three Lifetime Elision Rules)**.** Given a function or method signature containing elided references:

1. **Distinct inputs.** Each elided input reference gets its own lifetime parameter (`'a`, `'b`, ...).
2. **Single input.** If there is *exactly one* input lifetime parameter, assign it to all elided output lifetimes.
3. **Method receiver.** If multiple input lifetimes exist and one is `&self` or `&mut self`, assign `self`'s lifetime to all elided output lifetimes.

If any output lifetime remains unassigned after Rules 1–3, `rustc` stops and asks for an explicit annotation.

| Written signature | Expanded by Rules 1–3 | Rules |
|---|---|---|
| `fn trim(s: &str) -> &str` | `fn trim<'a>(s: &'a str) -> &'a str` | 1, 2 |
| `fn kv(s: &str) -> (&str, &str)` | `fn kv<'a>(s: &'a str) -> (&'a str, &'a str)` | 1, 2 |
| `fn id(&self, fmt: &str) -> &str` | `fn id<'a,'b>(&'a self, fmt: &'b str) -> &'a str` | 1, 3 |
| `fn max(x: &str, y: &str) -> &str` | *Compile error: ambiguous output* | 1 only |

Rule 3 deserves special attention when writing parsers. On `fn strip_from(&self, wire: &str) -> Option<&str>`, Rule 3 ties the returned `&str` to `&self`. When the returned slice actually points into `wire` rather than `self`, spelling out `'s` and `'buf` overrides Rule 3 so callers can drop `PrefixMatcher` while keeping the slice of `wire`:

```
pub struct PrefixMatcher { prefix: String }

impl PrefixMatcher {
    pub fn strip_from<'s, 'buf>(&'s self, wire: &'buf str) -> Option<&'buf str> {
        wire.strip_prefix(self.prefix.as_str())
    }
}
```

## 4.3 Structs Holding References: Zero-Copy Views vs. Owned Data

What happens when a `struct` stores a reference inside its fields? Consider an order gateway ingesting 500,000 pipe-delimited wire lines such as `"ORD-9042|AAPL|BUY|250|18945"` every second off a TCP

socket.

In Java 6, decoding bytes and calling `split` allocates seven to ten heap objects per message:

```
// Java 6: allocates decoded String + char[], ArrayList, String[], and 5 Strings!
String line = new String(rxBuf, 0, bytesRead, "US-ASCII").trim();
String[] parts = line.split("\\|");
Order ord = new Order(parts[0], parts[1], parts[2],
    Long.parseLong(parts[3]), Long.parseLong(parts[4]));
```

History and connection

**Why Java 7 ripped zero-copy `substring` out of Java 6.** Java 6's `String` stored `(char[] value, int offset, int count)`, so `line.substring(0, 8)` shared the backing `char[]` without copying. Oracle removed that sharing in Java 7u6 (2012) and forced `substring` to copy `char[]` eagerly because Java lacked lifetime parameters: caching a 10-byte substring parsed from a 2 MB HTTP body quietly pinned the entire 2 MB `char[]` on the GC heap forever. With `'a`, Rust gives you zero-copy views while statically forbidding accidental long-lived retention.

In Rust, `WireOrder<'a>` slices straight into `raw: &'a [u8]`. Any `struct` or `enum` holding a borrow declares that lifetime on its type header:

```
use std::mem::size_of;

#[derive(Debug, PartialEq, Eq)]
pub struct WireOrder<'a> {
    pub cl_ord_id: &'a str,
    pub symbol: &'a str,
    pub side: &'a str,
    pub qty: u64,
    pub price_ticks: i64,
}

const _: () = assert!(size_of::<WireOrder<'static>>() == 64); // 1 CPU cache line!

pub fn parse_wire_order<'a>(raw: &'a [u8]) -> Option<WireOrder<'a>> {
    let line = std::str::from_utf8(raw).ok()?.trim_ascii();
    let mut parts = line.split('|');
    let (cl_ord_id, symbol, side) = (parts.next()?, parts.next()?, parts.next()?);
    let qty = parts.next()?.parse::<u64>().ok()?;
    let price_ticks = parts.next()?.parse::<i64>().ok()?;
    if parts.next().is_some() || cl_ord_id.is_empty() || symbol.is_empty() {
        return None;
    }
    Some(WireOrder { cl_ord_id, symbol, side, qty, price_ticks })
}
```

Read `WireOrder<'a>` as a proof bound: *no `WireOrder<'a>` instance may outlive the buffer borrow `'a` backing its string slices.* Calling `parse_wire_order` performs **zero heap allocations**, returning a 64-byte stack value whose three `(ptr, len)` pairs point straight into `rx_buf` (Figure 4.2).

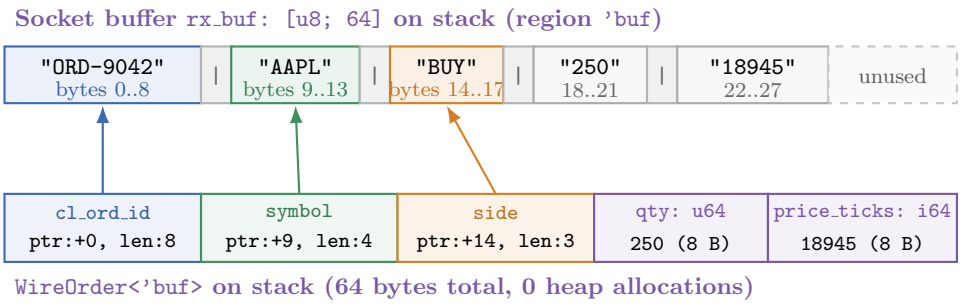



Figure 4.2: Zero-copy layout of `WireOrder<'buf>`. Three 16-byte `(ptr, len)` slices point straight into `rx_buf`, while `qty` and `price_ticks` sit inline as 64-bit integers.

That region parameter catches a notorious C/C++ ring-buffer bug at compile time. If a loop reuses one stack buffer `rx_buf` across `sock.read(&mut rx_buf)` calls while pushing parsed `WireOrder` views into an outer `Vec`, C++ `std::string_view` silently corrupts every earlier order with bytes from the latest packet, whereas Rust rejects the loop immediately:

```
fn broken_batch_reader(sock: &mut impl std::io::Read) {
    let mut rx_buf = [0_u8; 64];
    let mut batch: Vec<WireOrder<'_>> = Vec::new();
    while let Ok(n) = sock.read(&mut rx_buf) { // Error E0502 on next iteration!
        if n == 0 { break; }
        if let Some(ord) = parse_wire_order(&rx_buf[..n]) {
            batch.push(ord); // extends shared borrow of `rx_buf` to `batch`
        }
    }
}
```

### 4.3.1 Architectural Guidance: Avoiding "Lifetime Infection" and Self-References

Putting `&'a str` into long-lived domain models, caches, and order books forces every outer container and service to carry `<'a>`, an anti-pattern known as **lifetime infection**.

**In production**

**When to borrow in structs vs. when to own.**

- **Borrow (`Struct<'a>`) for short-lived synchronous views:** wire parsers, serializers, iterators (`Iter<'a, T>`), and SQL query parameter tuples scoped to one buffer.
- **Own at state and concurrency boundaries:** once validated orders enter an order book, cache, thread channel, or async task, convert text to fixed ASCII arrays (`[u8; 8]` for tickers — `Copy` and heap-free), `Box<str>` (16 bytes, omitting `String`'s 8-byte capacity), `Arc<str>` for shared immutable identifiers, or `String` when mutating.

What if you try to bundle both the owned buffer and the borrowed view inside a single struct `struct Bundle<'a> { raw: String, parsed: WireOrder<'a> }`? Safe Rust forbids naive self-referential structs: borrowing `raw` prevents moving `raw` into `Bundle`, and because Rust can relocate any struct between stack frames via bitwise `memcpy`, any pointer into an inline field would dangle after a move.

Instead, either parse on demand via a method `fn view(&self) -> Option<WireOrder<'_>>`, or store integer byte ranges `(usize, usize)` that stay valid across arbitrary moves:

```
pub struct OwnedPacketOrder {
    raw: Box<[u8]>,
    id_range: (usize, usize),
    sym_range: (usize, usize),
    pub qty: u64,
    pub price_ticks: i64,
}

impl OwnedPacketOrder {
    pub fn cl_ord_id(&self) -> &str {
        let (start, end) = self.id_range;
        std::str::from_utf8(&self.raw[start..end]).expect("validated at construction")
    }
}
```

(Chapter 12 examines the one case where compiler-generated structs genuinely hold internal pointers — `async`/`await` state machines — and how `Pin<P>` prevents moving them once polled.)

## 4.4 Busting the #1 Beginner Trap: `&'static str` vs. `T: 'static`

Spawning threads or registering Axum handlers often produces an error mentioning `'static`, leading newcomers to think Rust wants them to leak memory or stick data into global variables. That confusion disappears once you separate two distinct constructs: the **reference type** `&'static str` versus the **trait bound** `T: 'static`.

A `&'static str` is a concrete 16-byte reference whose target bytes remain at a valid address until process exit — typically string literals like `"NASDAQ"` baked into the binary's read-only `.rodata` segment, or startup configuration intentionally leaked once via `Box::leak`.

The bound `T: 'static` asks something completely different:

**Definition 4.2** (The `T: 'static` Lifetime Bound)**.** A type $T$ satisfies `T: 'static` if and only if $T$ contains **no non-`'static` borrows**. Any value of type $T$ is self-contained: its current owner may drop it immediately or hold it until program exit without depending on any caller stack frame.

Under Definition 4.2, an owned `String` allocated inside a loop and dropped microseconds later satisfies `T: 'static` completely:

```
pub fn assert_static_bound<T: 'static>(_value: T) {}

pub fn test_ephemeral_owned() {
    for i in 0..3 {
        let temp: String = format!("BATCH-{}", i);
        assert_static_bound(temp); // Compiles! `String: 'static` is TRUE!
    }
}
```

Why? Because `T: 'static` is a property of the *type* `String`: `temp` owns its heap buffer outright and borrows zero local stack frames.

| Type `T` | `T: 'static`? | Reason |
|---|---|---|
| `u64`, `[u8; 8]` | Yes | Inline value bytes; zero borrows |
| `String`, `Vec<u8>`, `Arc<str>` | Yes | Owns heap allocation; zero stack borrows |
| `&'static str` | Yes | Targets `.rodata`; valid until exit |
| `&'a str`, `WireOrder<'a>` | **No** | Tied to non-`'static` borrow `'a` |

That explains why `std::thread::spawn` requires `F: FnOnce() -> T + Send + 'static`: an OS thread may keep running after the parent stack frame returns. In C++, detaching a `std::thread` whose lambda captures `[&local]` causes a use-after-return crash; in Rust, you either `move` owned `'static` data into `thread::spawn`, or use `std::thread::scope` to join workers before the stack frame exits:

```rust
use std::thread;

pub fn concurrent_wire_notional() {
    // 1. Background thread (`'static` closure): move owned `String`
    let owned_msg = String::from("ORD-1001|MSFT|BUY|100|41250");
    let bg = thread::spawn(move || {
        let ord = parse_wire_order(owned_msg.as_bytes()).unwrap();
        ord.qty * (ord.price_ticks as u64)
    });
    assert_eq!(bg.join().unwrap(), 4_125_000);

    // 2. Scoped thread: guaranteed join before scope exit permits `&stack_msg`!
    let stack_msg = String::from("ORD-1002|GOOG|SELL|50|17500");
    let notional = thread::scope(|s| {
        s.spawn(|| {
            let ord = parse_wire_order(stack_msg.as_bytes()).unwrap();
            ord.qty * (ord.price_ticks as u64)
        }).join().unwrap()
    });
    assert_eq!(notional, 875_000);
}
```

## 4.5 Subtyping and Variance: Why `&'a mut T` Is Invariant in `T`

Java 6 defines subtyping over class hierarchies (`Integer` <: `Number`). Rust has no struct inheritance; subtyping in Rust is purely about **region containment**. Wherever code expects a reference valid over a short span `'short`, passing a reference valid over an enclosing span `'long` (such as `'static`) is always safe:

$$\texttt{'long: 'short} \iff \texttt{'long} <: \texttt{'short}.$$

How compound types propagate subtyping from their parameters is called **variance**:

**Definition 4.3** (Covariance, Invariance, and Contravariance)**.** For a type constructor $F\langle X\rangle$ and $A <: B$:

- $F$ is **covariant** in $X$ if $A <: B \implies F\langle A\rangle <: F\langle B\rangle$ (lifetimes inside $F$ may shrink).
- $F$ is **invariant** in $X$ if neither $F\langle A\rangle <: F\langle B\rangle$ nor $F\langle B\rangle <: F\langle A\rangle$ holds for $A \neq B$.

- $F$ is **contravariant** in $X$ if $A <: B \implies F\langle B\rangle <: F\langle A\rangle$ (as in `fn(&'short str) <: fn(&'long str)`).

| Reference type | In outer lifetime `'a` | In referent type `T` |
|---|---|---|
| Shared reference `&'a T` | Covariant | Covariant |
| Exclusive reference `&'a mut T` | Covariant | **Invariant** |

Shrinking `'a` on either reference is safe, and shrinking a lifetime inside `T` behind read-only `&'a T` is safe because the holder only reads `T`. Why must `&'a mut T` be strictly **invariant** in `T`? Watch what seven lines of `unsafe`-free code would do if `&mut &'static str` could coerce to `&mut &'short str`:

```
fn overwrite<'s>(slot: &mut &'s str, replacement: &'s str) {
    *slot = replacement;
}

fn exploit_if_covariant() {
    let mut ticker: &'static str = "MSFT"; // promises ''static' target
    {
        let local = String::from("BAD");    // lives only for ''short'
        overwrite(&mut ticker, &local);     // Error E0597: blocked by invariance!
    } // 'local' drops and frees "BAD"
    println!("ticker = {}", ticker);        // WOULD READ FREED HEAP MEMORY!
}
```

Figure 4.3 traces the prevented exploit. Because `slot` can be *written to*, letting `&mut &'static str` pose as `&mut &'short str` would allow `overwrite` to write `&local` into the outer `ticker` slot, leaving `ticker` dangling as soon as `local` drops. Invariance locks `'s = 'static`, so `rustc` reports `type annotation requires that local is borrowed for 'static` (E0597).

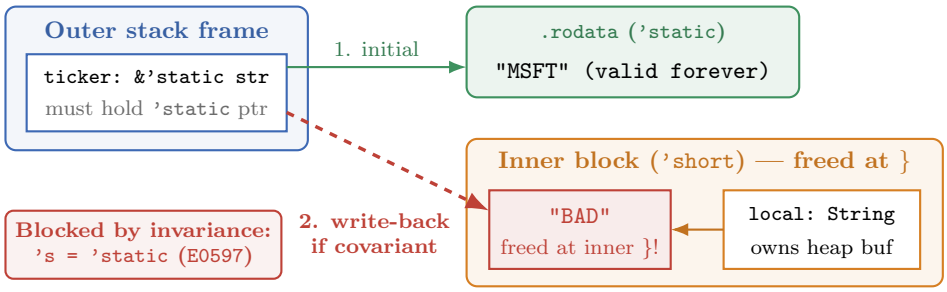



Figure 4.3: Why `&'a mut T` is invariant in `T`. Coercing `&mut &'static str` to `&mut &'short str` would let `overwrite` aim `ticker` at `"BAD"` on the heap, which is freed when the inner block exits.

**Connection**

**Java 6's broken array covariance and `ArrayStoreException`.** Java 1.0 made mutable arrays covariant (`Object[] slots = new String[]{"MSFT"};`), allowing `slots[0] = Integer.valueOf(42);` to compile and crash at runtime with `ArrayStoreException`. Java 6 catches that write only by storing a runtime class tag in every array header and checking types on *every reference array store*. Because Rust erases lifetimes at compile time, no runtime tag exists to throw a `LifetimeStoreException`; invariance of `&'a mut T` in `T` keeps zero-cost

lifetime erasure sound.

## 4.6 The Unsafe Boundary: Raw Pointers, `UnsafeCell`, and Sound Abstractions

Any decidable static borrow checker is conservative: it must reject some memory-safe programs whose safety hinges on array index arithmetic, and at the lowest layer — inside `Vec<T>`, `Arc<T>`, `Mutex<T>`, and OS syscalls — code has to manipulate raw addresses directly. Rust isolates those obligations inside `unsafe` blocks.

Writing `unsafe { ... }` does **not** disable the borrow checker: safe references (`&T`, `&mut T`), moves, lifetimes, and `Drop` remain enforced inside `unsafe` blocks. Instead, `unsafe` permits **five specific operations**:

1. **Dereferencing raw pointers** (`*const T` and `*mut T`). Creating and copying raw pointers is safe (`&x as *const i32`, `slice.as_mut_ptr()`), and so is `wrapping_add`; dereferencing `*p` requires `unsafe`, and so does `ptr.add`, which is an `unsafe fn` (see item 2).
2. **Calling an `unsafe fn`** (including C `extern "C"` functions), asserting you have met its documented preconditions.
3. **Implementing an `unsafe trait`** (such as `Send` or `Sync`), asserting your type upholds its contract.
4. **Accessing or mutating a `static mut`** global variable.
5. **Accessing fields of a C-style `union`**, where the active variant is untagged.

### 4.6.1 Case Study: Implementing `split_two_mut` Behind a Safe Signature

Suppose two worker threads want mutable slices of the morning and afternoon halves of an intraday price array `&mut [T]`. Writing `(&mut values[..mid], &mut values[mid..])` fails with `E0499` because the borrow checker treats both sub-slices as overlapping mutable borrows of `values` rather than proving $[0, \text{mid}) \cap [\text{mid}, \text{len}) = \emptyset$.

Checking `assert!(mid <= len)` in safe code and forming the two non-overlapping slices via raw pointers wraps that arithmetic proof behind a 100% safe signature (Figure 4.4):

```
use std::slice;

pub fn split_two_mut<T>(values: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = values.len();
    assert!(mid <= len, "mid index out of bounds");
    let ptr: *mut T = values.as_mut_ptr();
    // SAFETY:
    // 1. `mid <= len` holds by `assert!`, so `[0, mid)` and `[mid, len)` lie within
    //    the single allocated, aligned, initialized slice `values`.
    // 2. `[0, mid)` and `[mid, len)` are disjoint, so the two `&mut [T]` slices never
    alias.
    // 3. Elision Rule 2 ties both outputs to `values`'s input borrow lifetime.
```

```
    unsafe {
        (
            slice::from_raw_parts_mut(ptr, mid),
            slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}
```

In production Rust, every `unsafe` block carries a `// SAFETY:` comment discharging the exact preconditions of each operation inside it (`ptr.add` staying within the allocated object; `from_raw_parts_mut` receiving a valid, aligned, initialized, unaliased range).

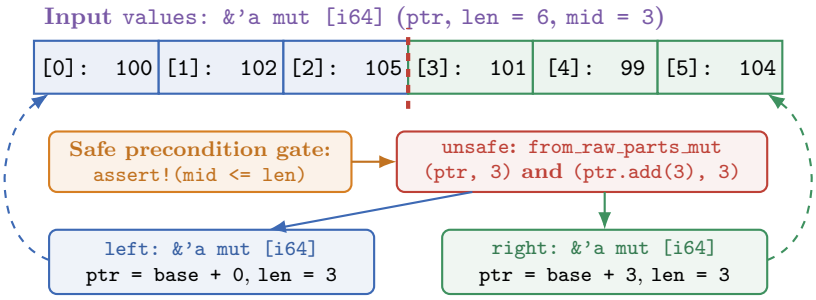



Figure 4.4: Inside `split_two_mut`. Once `assert!(mid <= len)` validates $0 \le 3 \le 6$ in safe code, raw pointer offsets construct two disjoint `&'a mut [i64]` slices tied to input borrow `'a`.

**Definition 4.4** (Soundness and the Module Trust Boundary)**.** A public Rust API is **sound** if no sequence of calls written in *safe* Rust — for any caller inputs — can ever trigger Undefined Behavior.

Definition 4.4 carries an architectural lesson shown in Figure 4.5: **the soundness boundary is the module privacy boundary (`mod`), not merely the `unsafe { ... }` braces.** Deleting the safe line `assert!(mid <= len)` outside `unsafe` makes `split_two_mut` unsound immediately, just as a safe bug modifying private `Vec::len` inside `Vec`'s module breaks `Vec`'s `unsafe` invariants.

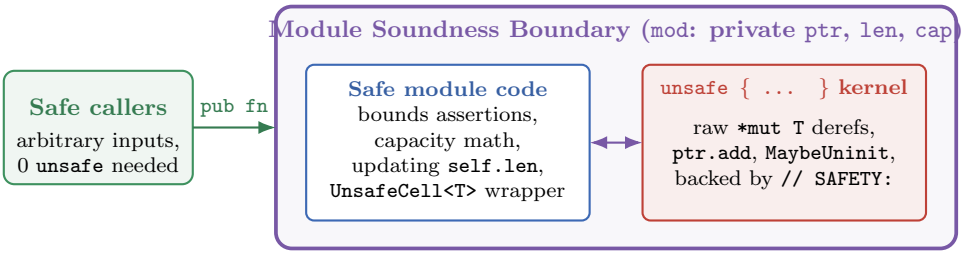



Figure 4.5: The safe abstraction capsule. Module privacy shields representation invariants (`len <= cap`, disjoint ranges) from external callers while safe checks and the small `unsafe` kernel maintain soundness together.

### 4.6.2 `UnsafeCell<T>`, Aliasing UB, `MaybeUninit<T>`, and Miri

Three habits from C and C++ cause **instant Undefined Behavior in Rust**, even inside `unsafe`, because Rust hands LLVM far stronger aliasing guarantees than C or C++:

**1. Mutating behind `&T` without `UnsafeCell<T>` is always UB.** Every `&T` carries an LLVM `readonly`/`noalias` contract unless `T` wraps its mutable bytes inside `std::cell::UnsafeCell<U>` (the sole primitive underlying `Cell`, `RefCell`, `Mutex`, and `AtomicU64`). Casting `&u64` to `*mut u64` and writing to it without `UnsafeCell` miscompiles under optimization:

```
use std::cell::UnsafeCell;

pub struct CounterCell { value: UnsafeCell<u64> } // `!Sync`; opts out of `&T` readonly

impl CounterCell {
    pub const fn new(initial: u64) -> Self { Self { value: UnsafeCell::new(initial) } }
    pub fn increment(&self) -> u64 {
        // SAFETY: `!Sync` guarantees single-threaded access; no `&u64` lives across `*p`.
        unsafe {
            let p: *mut u64 = self.value.get();
            *p += 1;
            *p
        }
    }
}
```

**2. Two aliasing `&mut T` references are instant UB — even without writes.** Every `&mut T` is emitted as an LLVM `noalias` pointer. Forming `let r1 = &mut *p; let r2 = &mut *p;` while `r1` remains live violates Rust's Stacked/Tree Borrows model immediately. Inside doubly linked lists or intrusive queues where pointers alias, keep raw `*mut Node` pointers until the exact instant of an unaliased access.

**3. Every `&T` and `&mut T` must be non-null, aligned, and initialized.** Taking a reference to an unaligned packed network field is UB (use `std::ptr::addr_of!(hdr.field).read_unaligned()` instead), and materializing a `T` or `&mut [u8]` over uninitialized memory is immediate UB. Use `std::mem::MaybeUninit<T>` to populate buffers safely before calling `assume_init()`:

```
use std::mem::MaybeUninit;

pub fn fill_ascii_digits(count: usize) -> Option<[u8; 16]> {
    if count > 16 { return None; }
    let mut buf: MaybeUninit<[u8; 16]> = MaybeUninit::uninit();
    let base: *mut u8 = buf.as_mut_ptr() as *mut u8;
    // SAFETY: `base` points to 16 stack bytes; all 16 are written before `assume_init`.
    unsafe {
        for i in 0..16 {
            base.add(i).write(if i < count { b'0' + (i as u8 % 10) } else { b'_' });
        }
        Some(buf.assume_init())
    }
}
```

**Work it yourself**

**Auditing `unsafe` with Miri.** Passing unit tests on x86-64 never proves `unsafe` code free of UB. Run `cargo +nightly miri test`: **Miri** interprets Rust's Mid-level IR on an instrumented abstract machine that tracks per-byte initialization, alignment, bounds, and pointer

provenance, catching aliasing UB, use-after-free, and data races on the exact offending line.

With ownership (Chapter 3), region lifetimes, and safe abstraction capsules in place, you hold Rust's complete memory model. Chapter 5 moves from *how long bytes live* to *what shapes values take*: algebraic data types, niche layout optimizations that pack `Option<&'a str>` into the exact same 16 bytes as `&'a str`, and exhaustive pattern matching for domain state machines.

## 4.7 Exercises

**Exercise 4.1** (Zero runtime cost of `'a`)**.** Write a helper `pub fn head_four<'a>(buf: &'a [u8]) -> Option<&'a [u8]>` (returning `Some(&buf[..4])` when `buf.len() >= 4`) alongside an elided version omitting `<'a>`. (a) Compile with `rustc -C opt-level=3 --emit=asm` and verify that both emit identical assembly. (b) Explain in terms of stack frames and registers why changing `<'a>` to `<'static>` cannot extend how long a caller's `Vec<u8>` stays allocated at runtime.

**Exercise 4.2** (Tracing Non-Lexical Lifetimes by hand)**.** Given `let mut ledger = vec![100_i64, -40, 250];`, predict whether each snippet below compiles under NLL; if it fails, identify the overlapping regions and fix it in two lines without cloning:

(a) `let r = &ledger[0]; let x = *r; ledger.push(x); println!("{}", x);`

(b) `let r = &ledger[0]; ledger.push(50); println!("{}", *r);`

(c) `let r = &ledger[0]; if *r > 0 { ledger.push(10); } else { println!("{}", *r); }`

**Exercise 4.3** (Applying and overriding the Three Elision Rules)**.** Apply Definition 4.1 to expand each signature, or explain why `rustc` rejects it:

(a) `fn parse_ticker(line: &str) -> Result<&str, u32>`

(b) `fn first_non_empty(primary: &str, backup: &str) -> &str`

(c) `impl HeaderConfig { fn extract_host(&self, raw_http: &str) -> Option<&str> }`

In (c), rewrite the signature with explicit lifetimes when the returned host slice borrows `raw_http` rather than `self`, allowing `HeaderConfig` to drop early.

**Exercise 4.4** (Zero-copy HTTP header line parser)**.** Define a struct `HttpHeader<'a>` with public fields `name: &'a str` and `value: &'a str`, and implement a function `parse_header<'a>` that takes `line: &'a [u8]`, returns `Option<HttpHeader<'a>>`, splits an ASCII header line like `b"Content-Length: 1024"` on the first `':'`, trims ASCII whitespace, rejects empty names, and performs zero heap allocations. Verify at compile time that `size_of::<HttpHeader<'static>>() == 32`.

**Exercise 4.5** (Diagnosing `&'static str` vs. `T: 'static`)**.** State whether each type satisfies `T: 'static` and justify in one sentence: (a) `Vec<String>` created and dropped inside a three-line block; (b) `Result<u64, &'static str>`; (c) `Option<WireOrder<'a>>` for an arbitrary `'a`; (d) `std::sync::Arc<[u8]>`. Then show how to run `parse_wire_order` on a `stack_buf: [u8; 64]` on a worker thread first using `std::thread::scope` and second using `std::thread::spawn`.

**Exercise 4.6** (Constructing a variance counterexample for `Cell<T>`)**.** In the standard library, `std::cell::Cell<T>` mutates its inner `T` through a shared reference `&Cell<T>` via `Cell::set(&self,`

`val: T)`, built atop `UnsafeCell<T>`. Write a 10-line counterexample modeled on `exploit_if_covariant` proving that `&'a Cell<T>` must be invariant in `T` to prevent a dangling `&'static str`.

**Exercise 4.7** (Auditing module-level soundness)**.** Consider this miniature buffer module whose `set_len` contains zero `unsafe` keywords:

```
pub mod mini_buf {
    pub struct MiniBuf { data: [u8; 8], len: usize }
    impl MiniBuf {
        pub fn new() -> Self { Self { data: [0; 8], len: 0 } }
        pub fn set_len(&mut self, new_len: usize) { self.len = new_len; }
        pub fn as_slice(&self) -> &[u8] {
            unsafe { std::slice::from_raw_parts(self.data.as_ptr(), self.len) }
        }
    }
}
```

(a) Write a 3-line *safe* caller outside `mini_buf` that triggers an out-of-bounds memory read. (b) Explain why `MiniBuf` violates Definition 4.4, and give two distinct fixes (one checking bounds in safe `set_len`, and one marking `pub unsafe fn set_len`).

**Exercise 4.8** (Safe abstraction: `first_and_last_mut`)**.** Implement a generic helper `first_and_last_mut<T>` taking `slice: &mut [T]` and returning `Option<(&mut T, &mut T)>` for the first and last elements when `slice.len() >= 2`, or `None` otherwise. (a) Implement it first *without* `unsafe` using `split_first_mut()` and `split_last_mut()`. (b) Implement it with raw pointers and a `// SAFETY:` proof, and explain what UB occurs on a 1-element slice if you guard with `!slice.is_empty()` instead of `slice.len() >= 2`.

**Exercise 4.9** (Eliminating uninitialized-memory UB with `MaybeUninit`)**.** A C programmer writes:

```
pub unsafe fn bad_read_header(fill_bytes: fn(*mut u8, usize)) -> [u8; 32] {
    let mut hdr: [u8; 32] = std::mem::MaybeUninit::uninit().assume_init();
    fill_bytes(hdr.as_mut_ptr(), 32);
    hdr
}
```

Explain why calling `assume_init()` before `fill_bytes` writes all 32 bytes is immediate Undefined Behavior in Rust, and rewrite the function so `assume_init()` runs only after initialization completes.

# Chapter 5

# Algebraic Data Types: Making Illegal States Unrepresentable

Chapters 3 and 4 tracked *who owns* a block of memory and *how long* a borrow may point into it. Ownership and lifetimes rule out dangling pointers, double frees, and iterator invalidation, yet a memory-safe program can still wire money twice, settle a canceled trade, or panic at three in the morning when an unexpected `null` reaches a report generator. Those bugs live one level up: in the gap between the bit patterns a data structure *can* hold in RAM and the states that actually make sense in the problem domain.

If you grew up in Java 6 and C++, your reflexes for modeling domain data were shaped by mutable classes full of nullable references and boolean flags, plus open `extends` hierarchies on the heap. Rust drops both `extends` implementation inheritance and implicit `null`. In their place, it gives you *algebraic data types* (`struct` and data-carrying `enum`) paired with exhaustive pattern matching (`match`). This chapter counts how many nonsensical states each design allows, and then inspects the exact bytes the compiler emits for the safer design.

We will study Rust's data model from two angles: first from above, using simple counting algebra to measure how tightly a type fits its domain, and then down to the metal, counting exact bytes, alignment padding, discriminant tags, and bit-pattern niches on a 64-bit machine.

---

## 5.1 Why Rust Dropped `extends` and `null`: The Algebra of Cardinality

Every type defines the set of values a variable of that type can hold at runtime. Before asking what methods a type offers, a Rust engineer asks a simpler counting question: *how many distinct values inhabit this type?*

**Definition 5.1** (Cardinality of a type)**.** For any type `T`, its *cardinality*, written $|\texttt{T}|$, is the number of distinct valid values of type `T` (ignoring finite RAM limits on heap containers).

Count the inhabitants of Rust's smallest built-in primitives:

- **Booleans and fixed-width integers.** A `bool` has $|\texttt{bool}| = 2$ (`false`, `true`); a `u8` has

$|\texttt{u8}| = 2^8 = 256$; a `u64` has $|\texttt{u64}| = 2^{64}$.

- **The unit type `()`: $|()| = 1$.** Its sole value is also written `()`. Where C and Java use `void` for "returns nothing," creating an awkward split between `void` methods and generic functions returning `T`, Rust functions without a return arrow return `()`. Because only one value exists, carrying a `()` conveys zero runtime bits of information, so `core::mem::size_of::<()>() == 0`.
- **The never type `!`: $|!| = 0$.** Rust also has a type with *zero* inhabitants, written `!`. Expressions that diverge before producing a value — `return`, `break`, `continue`, `panic!(...)`, `std::process::abort()`, and an infinite `loop {}` — all have type `!`.

A zero-inhabitant type is useful every time a `match` mixes value-producing arms with early exits:

```
fn unwrap_or_abort<T>(opt: Option<T>) -> T {
    match opt {
        Some(value) => value,            // produces T
        None => std::process::abort(),   // produces !, which coerces to any T
    }
}
```

Every arm of a `match` must agree on one result type `T`. Because `abort()` has type `!` ($|!| = 0$), control never reaches the end of the second arm. Just as the empty set $\emptyset$ is a subset of every set $S$, the uninhabited type `!` coerces soundly to any type `T`.

### 5.1.1 Product Types: `struct` Multiplies Possibilities

Bundling fields `a: A` and `b: B` into a record lets every value of `A` pair independently with every value of `B`. That set is the Cartesian product $A \times B$, so cardinalities multiply:

$$|\texttt{struct \{ a: A, b: B \}}| = |A \times B| = |A| \cdot |B|.$$

Because their cardinalities multiply, structs and tuples are *product types*. Rust provides three forms:

1. **Named-field structs**, such as `struct Point { x: i32, y: i32 }`, with cardinality $2^{32} \cdot 2^{32} = 2^{64}$.
2. **Tuple structs**, such as `pub struct BasisPoints(pub u16);`, with cardinality 65,536. Wrapping a primitive in a single-field tuple struct — the *newtype* idiom — creates a distinct compile-time type with *zero* heap boxing or object header: in machine code it is an unboxed 2-byte `u16` in a register, yet the compiler rejects passing a bare `u16` where `BasisPoints` are expected.
3. **Unit structs**, such as `pub struct Usd;`, an empty product ($|\texttt{Usd}| = 1$, zero runtime bytes) used as type-level markers in Chapter 6.

Multiplication is how flag-and-nullable classes end up allowing far more states than the domain has. Here is a typical settlement record from a Java 6 back-office service:

```
// Java 6: A four-field product of booleans and nullable references
public final class SettlementRecord {
    private final boolean isSettled;
    private final boolean isFailed;
    private final Long settledAtEpochMs; // null until settled
```

```
    private final Integer failureCode;   // null unless failed

    public SettlementRecord(
            boolean isSettled, boolean isFailed,
            Long settledAtEpochMs, Integer failureCode) {
        this.isSettled = isSettled;
        this.isFailed = isFailed;
        this.settledAtEpochMs = settledAtEpochMs;
        this.failureCode = failureCode;
    }
}
```

Count the domain states first. A transfer is in *one* of three mutually exclusive stages: **Pending** (1 state), **Settled** at a 64-bit timestamp ($2^{64}$ states), or **Failed** with a 32-bit code ($2^{32}$ states) — a total of $1 + 2^{64} + 2^{32}$ meaningful states.

Now count what `SettlementRecord` actually represents. Because every Java object reference includes `null`, $|\texttt{Long}| = 1 + 2^{64}$ and $|\texttt{Integer}| = 1 + 2^{32}$. Multiplying the four fields yields:

$$2 \cdot 2 \cdot (1 + 2^{64}) \cdot (1 + 2^{32}) \;=\; 4 \cdot (1 + 2^{32} + 2^{64} + 2^{96}) \;\approx\; 2^{98}.$$

Fewer than *one in ten billion* representable configurations of `SettlementRecord` are valid. All the rest — setting both flags to `true`, marking `isSettled` while leaving `settledAtEpochMs` as `null` (which throws on unboxing), or leaving a stale `failureCode` on a pending transfer — force defensive `if` checks into every method.

### 5.1.2 Sum Types: `enum` Adds Possibilities

Look back at the true domain count: $1 + 2^{64} + 2^{32}$. Those are *plus* signs, not multiplication signs. What the domain describes is a disjoint union where each alternative carries *only* its own payload. In Rust, an `enum` defines a closed *sum type* whose variants may be unit-like, tuple-like, or struct-like:

```
pub enum SettlementStatus {
    Pending,
    Settled { settled_at_epoch_ms: u64 },
    Failed { failure_code: u32 },
}
```

Its cardinality is the exact sum $|\texttt{()}| + |\texttt{u64}| + |\texttt{u32}| = 1 + 2^{64} + 2^{32}$. None of the illegal states can be represented: a value holds only one tag at a time, `Settled` cannot be built without a `u64` timestamp, and `failure_code` does not even exist on `Settled`. Figure 5.1 compares the Cartesian grid of a flag product against the disjoint union of a sum type.

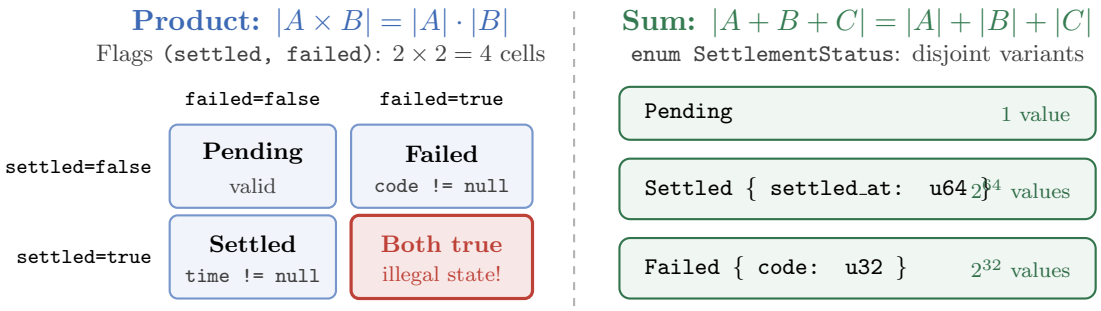



Figure 5.1: Product types multiply fields into a Cartesian grid with illegal cells such as `(settled=true, failed=true)`, whereas sum types add disjoint variants carrying only their own non-null fields.

**Intuition**

Reading $\times$ as `struct`/tuple, $+$ as `enum`, 1 as `()`, and 0 as `!` turns ordinary algebra into type isomorphisms:

- $T \times 1 = T$: `(u64, ())` carries the exact same $2^{64}$ states and 8 bytes as `u64`.
- $T + 0 = T$: `Result<u64, !>` has $2^{64} + 0 = 2^{64}$ inhabitants because `Err(!)` is unconstructible.
- $T \times 0 = 0$: `struct Impossible { id: u64, proof: ! }` has $2^{64} \cdot 0 = 0$ inhabitants; nobody can ever construct it without first producing a value of `!`.
- $1 + T$: `Option<T>` adds *one* explicit absence state (`None`) only to types that ask for it.

### 5.1.3 Why Java 6 Subclasses and C `unions` Fall Short

Java 6 and C each offer half of what Rust's `enum` provides:

- **Java 6 `enum` + `abstract class`.** Java's `enum` constants are fixed singletons allocated at class load time, so `SETTLED` cannot hold a per-transfer timestamp. Modeling per-case data in Java 6 forces an `abstract class SettlementStatus` with heap-allocated subclasses (`PendingStatus`, `SettledStatus`, `FailedStatus`). Every field now chases a pointer to a separate GC object with a 12-to-16-byte header, the reference can still be `null`, and open subclassing across packages defeats compiler-checked exhaustiveness unless you write clunky Visitor boilerplate. (Chapter 10 examines how Kotlin's `sealed` classes modernize the JVM side of this story.)
- **C's unchecked `union`.** A C `struct` pairing an `enum` tag with a `union { uint64_t settled_at; uint32_t failure_code; }` sits flat inline in memory with zero pointer indirection, just like systems code demands:

```c
enum StatusTag { TAG_PENDING, TAG_SETTLED, TAG_FAILED };
struct SettlementC {
    enum StatusTag tag;
    union {
        uint64_t settled_at_epoch_ms;
        uint32_t failure_code;
    } payload;
```

```
};
```

Reading `rec.payload.settled_at_epoch_ms` while `rec.tag` holds `TAG_FAILED` compiles in C and silently reinterprets an error code plus four uninitialized bytes as a timestamp. Rust's `enum` compiles down to that same flat tagged-union layout while making it impossible in safe code to touch a variant's payload without matching its tag first.

History and connection

Closed sum types with exhaustive pattern matching come from the Edinburgh functional-programming tradition. **Robin Milner**'s **ML** (1973) was built as the scripting language of the LCF theorem prover, and the related language **Hope** (Burstall, MacQueen, and Sannella, 1980) introduced user-defined algebraic data types with pattern matching, which Standard ML then adopted. Mainstream systems languages followed a different path. In 1965 **Tony Hoare** added a `null` reference to **ALGOL W**, by his own account "simply because it was so easy to implement," and C, C++, and Java inherited the idea. At QCon London in 2009 he called it his "billion-dollar mistake." Rust joins the two traditions: ML-style algebraic types, compiled down to C-style flat memory and bit-pattern niches.

## 5.2 Exact Byte Layout: Struct Reordering, Tagged Unions, and Niche Optimization

Every sized type `T` in Rust has two compile-time constants, which you can read with the functions `size_of` and `align_of` from `core::mem`:

- **Alignment (`align_of::<T>()`):** a power of two $(1, 2, 4, 8, \ldots)$ dictating that every valid address of `T` must be a multiple of that byte count. On 64-bit targets, `u8`/`bool` have alignment 1, `u16` has 2, `u32` has 4, and `u64`, `usize`, `&T`, and `Box<T>` have 8.
- **Size (`size_of::<T>()`):** the byte stride between elements in `[T; N]`, always rounded up to a multiple of `align_of::<T>()`.

### 5.2.1 Struct Field Reordering: `repr(Rust)` vs. `#[repr(C)]`

C and C++ require struct fields to sit in memory in the exact order written in source code. Alternating small and wide fields forces dead alignment padding between them. Compare `#[repr(C)]` against Rust's default `repr(Rust)` on 64-bit hardware:

```
use core::mem::{align_of, size_of};

#[repr(C)]
pub struct PaddedHeaderC {
    pub version: u8,    // byte 0   (1B) + 7B padding so tx_id lands on byte 8
    pub tx_id: u64,     // bytes 8..=15  (8B)
    pub flags: u8,      // byte 16  (1B) + 7B tail padding so stride % 8 == 0
}

pub struct CompactHeaderRust {
```

```
    pub version: u8,
    pub tx_id: u64,
    pub flags: u8,
}

const _: () = assert!(size_of::<PaddedHeaderC>() == 24);
const _: () = assert!(align_of::<PaddedHeaderC>() == 8);
const _: () = assert!(size_of::<CompactHeaderRust>() == 16);
const _: () = assert!(align_of::<CompactHeaderRust>() == 8);
```

Because `size_of` and `align_of` are `const fn`s, `const _: () = assert!(...);` checks layouts at compile time at zero runtime cost. Under `#[repr(C)]`, 10 bytes of actual payload expand to 24 bytes (14 bytes of padding) so `tx_id` aligns to 8 bytes in every element of `[PaddedHeaderC; N]`. Under default `repr(Rust)`, field offsets are intentionally unspecified so `rustc` can place `tx_id: u64` first at bytes `0..=7`, followed by `version` at byte 8, `flags` at byte 9, and 6 tail-padding bytes (`10..=15`) — saving 33% of cache space without forcing you to reorder source fields for the machine. Add `#[repr(C)]` only at C FFI or kernel ABI boundaries.

### 5.2.2 How a General `enum` Sits in Memory: The Tagged Union

Now examine how `rustc` lays out a multi-variant domain enum:

```
use core::mem::{align_of, size_of};

#[repr(C, u8)]
pub enum LedgerEvent {
    Heartbeat,                                          // tag = 0
    WirePosted { tx_id: u64, fee_bps: u16 },            // tag = 1
    BatchClosed { batch_id: u64, count: u32, flags: u32 }, // tag = 2
}

const _: () = assert!(size_of::<LedgerEvent>() == 24);
const _: () = assert!(align_of::<LedgerEvent>() == 8);
```

Adding `#[repr(C, u8)]` pins a 1-byte discriminant tag at byte 0 followed by a C-compatible union of the variant structs (omitting the attribute produces the same 24-byte size here). Figure 5.2 maps all 24 bytes (`0..=23`): byte `0` holds tag `0`, `1`, or `2`; bytes `1..=7` pad out to an 8-byte boundary for the `u64` fields; and bytes `8..=23` overlay whichever variant is active, sized to the 16-byte `BatchClosed` payload.

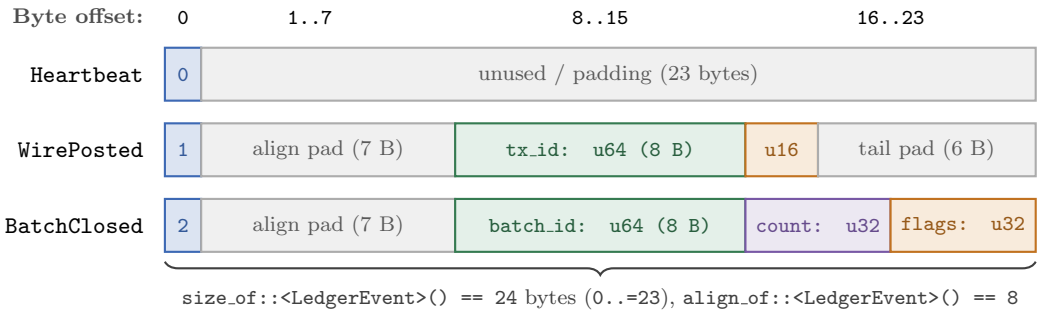



Figure 5.2: Byte layout of `#[repr(C, u8)] enum LedgerEvent`. Byte `0` stores the tag; bytes `1..=7` align the 8-byte payload region; bytes `8..=23` overlay variant fields (`u16` in `WirePosted` is `fee_bps`).

What if `BatchClosed` carried `region: u16` instead of `flags: u32`, shrinking its payload to $8 + 4 + 2 = 14$ bytes? Under `#[repr(C, u8)]`, the enum still takes 24 bytes because the C layout insists on a whole 8-byte-aligned union after the tag. Under default `repr(Rust)`, `rustc` is free to lay out each variant around the tag. It keeps the tag at byte 0, slides `region` into bytes `2..=3` and `count` into bytes `4..=7` (the slots that used to be padding), and puts `batch_id` at bytes `8..=15`. The whole enum fits in **16 bytes**. Dumping the raw bytes of `BatchClosed { batch_id: 0x1111..., count: 0x2222_2222, region: 0x3333 }` on x86-64 shows exactly that: `02 00 33 33 22 22 22 22 11 11 11 11 11 11 11 11`.

### 5.2.3 Niche Optimization: Zero-Overhead `Option<T>`

Why did C and Java put `NULL` / `null` inside every pointer type in the first place? Because systems programmers refused to pay 16 bytes (an 8-byte pointer plus a 1-byte tag plus 7 padding bytes) for every nullable pointer in a tree or hash table. Rust achieves *both* static non-null references *and* an 8-byte `Option<&T>` through *niche optimization*.

**Definition 5.2** (Bit-pattern niche)**.** Let `T` occupy $k =$ `size_of::<T>()` bytes ($8k$ bits). Any $8k$-bit pattern that is never a valid inhabitant of `T` under Rust's validity invariants is a *niche* of `T`.

Compare `Option<u64>` against types that possess at least one niche:

- **`u64` has zero niches.** Every 64-bit pattern from `0` through `u64::MAX` is a valid integer. Distinguishing `None` from `Some(0)` requires an extra tag byte plus 7 alignment padding bytes, making `Option<u64>` 16 bytes wide.
- **`&T`, `&mut T`, `Box<T>`, `NonNull<T>`, and `fn()` forbid `0x0`.** Because safe references, `Box`es, and function pointers are never null, the all-zero bit pattern is a niche. Thus `Option<&u64>` and `Option<Box<u64>>` store `None` as zero and `Some(p)` as the raw address in **8 bytes total** — matching a C pointer at machine level with static null checking on top.
- **`NonZeroU64` and newtypes.** Ruling out zero with `core::num::NonZeroU64` creates the same `0x0` niche for integers (`size_of::<Option<NonZeroU64>>() == 8`), and niches propagate through wrapper structs such as `struct Cents(NonZeroU64)`.
- **`bool` has 254 niches.** Only `0x00` and `0x01` are valid `bool`s, so `Option<bool>` encodes `None` as `0x02` in **1 byte**.

```rust
use core::mem::size_of;
use core::num::NonZeroU64;

const _: () = assert!(size_of::<u64>() == 8);
const _: () = assert!(size_of::<Option<u64>>() == 16);

const _: () = assert!(size_of::<Option<&u64>>() == 8);
const _: () = assert!(size_of::<Option<&mut u64>>() == 8);
const _: () = assert!(size_of::<Option<Box<u64>>>() == 8);
const _: () = assert!(size_of::<Option<NonZeroU64>>() == 8);
const _: () = assert!(size_of::<Option<fn()>>() == 8);
const _: () = assert!(size_of::<Option<bool>>() == 1);
```

Figure 5.3 contrasts the 16-byte layout of `Option<u64>` with the 8-byte niche layout of `Option<&u64>` and `Option<NonZeroU64>`.

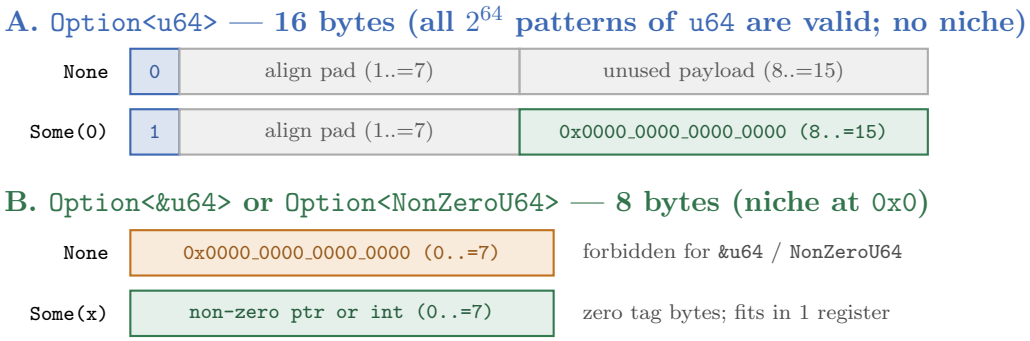



Figure 5.3: Tag-beside-payload layout of `Option<u64>` (16 bytes) versus niche-encoded `Option<&u64>` and `Option<NonZeroU64>` (8 bytes), where `None` occupies the forbidden all-zero bit pattern.

**Common trap**

**The Giant Variant Trap.** Because an `enum` is sized to its *largest* variant inline, one cold diagnostic variant carrying a 512-byte buffer inflates *every* `Result` returning that error:

```rust
pub enum WireError {
    Timeout,
    InvalidRoutingNumber(u32),
    DiagnosticDump([u8; 512]),        // inline 512-byte array!
}
pub enum SlimWireError {
    Timeout,
    InvalidRoutingNumber(u32),
    DiagnosticDump(Box<[u8; 512]>),   // 8-byte heap pointer on cold path
}
const _: () = assert!(size_of::<Result<u64, WireError>>() == 520);
const _: () = assert!(size_of::<Result<u64, SlimWireError>>() == 16);
```

Returning `Result<u64, WireError>` copies a 520-byte stack slot even when 99.999% of calls return `Ok(n)`. Boxing the cold payload shrinks `Result<u64, SlimWireError>` to 16 bytes

— two CPU registers (`rax`, `rdx`) and zero heap allocation on the happy path. Clippy flags oversized error types automatically via `clippy::result_large_err`.

## 5.3 `Option<T>`, `Result<T, E>`, and `?` vs. Java 6 `null` and Checked Exceptions

Neither `Option<T>` nor `Result<T, E>` relies on special compiler privilege for its definition; both are five-line standard-library `enum`s in `core`:

```
pub enum Option<T> {
    None,
    Some(T),
}

#[must_use]
pub enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

### 5.3.1 `Option<T>` and Combinator Pipelines

In Java 6, `Map.get(id)` returns `Account` whether the key exists or not; calling a getter such as `getDailyLimitCents()` on the result compiles without warning, and when the key is absent it fails at runtime with a `NullPointerException`. In Rust, `HashMap::get` returns `Option<&Account>`, which is a different type from `&Account`. You unpack the `Option` once at the lookup boundary, and downstream functions take plain `&Account`.

Rather than nesting four-line `match` blocks for every optional step, idiomatic Rust chains combinator methods:

```
use std::collections::HashMap;

pub struct Account {
    pub daily_limit_cents: Option<u64>,
    pub frozen: bool,
}

pub fn effective_limit_cents(
    accounts: &HashMap<String, Account>,
    owner_id: &str,
    default_limit: u64,
) -> u64 {
    accounts
        .get(owner_id)                          // Option<&Account>
        .filter(|acct| !acct.frozen)            // None if account is frozen
        .and_then(|acct| acct.daily_limit_cents) // flatten Option<Option<u64>>
        .map(|cents| cents.min(1_000_000))      // cap at $10,000.00
```

```
        .unwrap_or(default_limit)                // fallback u64 on None
}
```

Five combinators handle almost every pipeline you will write:

- `.map(f)` transforms `Some(x)` into `Some(f(x))` and leaves `None` untouched.
- `.and_then(f)` flat-maps when `f` *itself* returns `Option<U>`, avoiding nested `Option<Option<U>>`.
- `.filter(pred)` keeps `Some(x)` when `pred(&x)` is true and turns it into `None` otherwise.
- `.unwrap_or(val)` vs. `.unwrap_or_else(|| compute())`: function arguments evaluate eagerly, so `opt.unwrap_or(load_default())` executes `load_default()` *even when `opt` is `Some`*. Use `unwrap_or` for cheap constants and `unwrap_or_else` whenever the fallback allocates or computes.
- `.ok_or(err)` and `.ok_or_else(|| make_err())` promote `Option<T>` into `Result<T, E>`, mapping `None` to `Err(err)` so you can propagate it with `?`.

**Common trap**

**Calling `.unwrap()` in request handlers re-invites `NullPointerException`.** Calling `None.unwrap()` or `Err(e).unwrap()` triggers a `panic!`, crashing the current thread or Tokio task. Using `.unwrap()` on network payloads, query parameters, or database lookups throws away static error handling. Save `.expect("invariant reason")` for invariants proven true at startup or by an immediately preceding check.

### 5.3.2 `Result<T, E>` vs. C `errno` and Java 6 Checked Exceptions

Recoverable error handling in C and Java 6 sits at opposite extremes:

1. **C's `-1` and `errno`** return in a single CPU register at zero allocation cost, yet nothing in C's type system warns when a caller ignores the return code and reads an uninitialized output buffer. Rust's `Result<T, E>` carries `#[must_use]`, so dropping a `Result` unread triggers an immediate compiler warning.
2. **Java 6 checked exceptions (`throws IOException, SQLException`)** put failure in the method signature, yet couple static checking to *stack unwinding* and *stack-trace allocation*. Constructing `new Exception()` walks every active stack frame inside `Throwable.fillInStackTrace()`, turning a burst of bad client requests or downstream timeouts into microsecond-scale heap allocation storms.

Rust decouples static error checking from unwinding: `Result<T, E>` is returned through the exact same register/stack calling convention as any other value, executing a normal `ret` instruction with zero unwinding tables or heap allocation on the error path.

### 5.3.3 Desugaring the `?` Operator Step by Step

To propagate `Result` values without repetitive `if err != nil` blocks, postfix `?` unwraps `Ok(v)` inline or returns `Err` early:

```rust
use std::collections::HashMap;
use std::num::ParseIntError;

#[derive(Debug, PartialEq, Eq)]
pub enum ConfigError {
    MissingKey(&'static str),
    InvalidInt(ParseIntError),
    PortZeroOrReserved(u16),
}

impl From<ParseIntError> for ConfigError {
    fn from(err: ParseIntError) -> Self {
        ConfigError::InvalidInt(err)
    }
}

pub fn parse_service_port(
    env_map: &HashMap<String, String>,
) -> Result<u16, ConfigError> {
    let raw = env_map
        .get("LISTEN_PORT")
        .ok_or(ConfigError::MissingKey("LISTEN_PORT"))?;
    let port: u16 = raw.trim().parse()?;
    if port < 1024 {
        return Err(ConfigError::PortZeroOrReserved(port));
    }
    Ok(port)
}
```

Notice how `raw.trim().parse()?` produces a `ParseIntError`, yet returns cleanly from a function returning `Result<u16, ConfigError>`. Ignoring trait plumbing (`core::ops::Try`), `expr?` on a `Result` expands to:

```rust
match expr {
    Ok(val) => val,
    Err(err) => return Err(core::convert::From::from(err)),
}
```

Because we implemented `From<ParseIntError> for ConfigError`, `From::from(err)` wraps the low-level parse error into `ConfigError::InvalidInt(err)` automatically; when the error already has type `ConfigError`, the standard identity `impl<T> From<T> for T` compiles to a no-op. Figure 5.4 traces both value tracks.

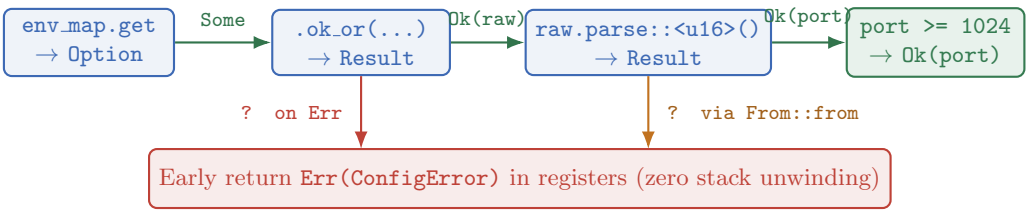



Figure 5.4: Two-track control flow of `?` inside `parse_service_port`. Each `?` unwraps `Ok` rightward or converts `Err` via `From::from` into a normal register return.

The `?` operator also works on `Option<T>` inside functions returning `Option<U>`:

```
pub fn header_fee_bps(lines: &[&str]) -> Option<u16> {
    let first = lines.first()?;                    // None -> early return None
    let digits = first.strip_prefix("FEE_BPS=")?; // None -> early return None
    digits.trim().parse::<u16>().ok()              // Result<u16, _> -> Option<u16>
}
```

Rust intentionally rejects mixing `?` on `Option` directly inside a `Result`-returning function because `None` carries no error value `E`; bridge explicitly with `.ok_or(...)` or `.ok()`.

## 5.4 Exhaustive Pattern Matching as Compiler-Enforced Case Analysis

Pattern matching is the exact dual of data construction: it inspects a composite value's shape and binds variables to its interior pieces in one checked step.

### 5.4.1 `match` Expressions, Destructuring, `@` Bindings, and Guards

Unlike C and Java 6 `switch` statements — limited to primitive integers or enum constants, prone to accidental fallthrough without `break;`, and unable to produce a value — Rust's `match` is an expression with zero fallthrough that destructures arbitrary nested shapes:

```
#[derive(Debug, Clone)]
pub struct WireFrame {
    pub channel: u8,
    pub sender: String,
    pub payload: Vec<u8>,
}

pub fn classify_frame(frame: &WireFrame, fast_limit: usize) -> String {
    match frame {
        WireFrame {
            channel: ch @ 1..=9,
            sender,
            payload,
        } if payload.len() <= fast_limit => match payload.as_slice() {
            [] => format!("priority ch {ch} ({sender}): keepalive"),
            [opcode] => format!("priority ch {ch} ({sender}): ctrl 0x{opcode:02x}"),
            [opcode, body @ ..] => format!(
                "priority ch {ch} ({sender}): op 0x{opcode:02x}, {}B",
                body.len()
            ),
        },
        WireFrame { channel: 0, sender, .. } => format!("sys ({sender})"),
        WireFrame { channel, sender, payload } => {
            format!("std ch {channel} ({sender}): {}B", payload.len())
        }
    }
}
```

Four pattern tools work together in `classify_frame`:

1. **Ranges and `@` bindings (`channel: ch @ 1..=9`):** `1..=9` tests inclusive range membership while `ch @` binds the matched byte so the arm body can print `ch`.
2. **Exhaustive slice patterns (`[]`, `[opcode]`, `[opcode, body @ ..]`):** destructure slices of length 0, 1, and $\geq 2$ without manual index bounds checks.
3. **Rest patterns (`..`):** ignore remaining struct fields (`WireFrame { channel: 0, sender, .. }`) or bind remaining slice elements (`body @ ..` as `&[u8]`).
4. **Match guards (`if payload.len() <= fast_limit`):** attach a runtime boolean filter. Because the compiler cannot prove an arbitrary runtime boolean is always true, a guarded arm never satisfies exhaustiveness by itself; frames failing the guard continue to later arms.

### 5.4.2 Match Ergonomics: Borrowing Without Moving

Trace ownership in `classify_frame` using Chapters 3 and 4: `frame` has type `&WireFrame`, and `sender: String` is not `Copy`. Why didn't `WireFrame { sender, .. }` fail to move `sender` out of a shared reference? Under *match ergonomics*, matching a value pattern against `&T` (or `&mut T`) pushes the reference inward so every variable binds as `&Field` (or `&mut Field`) automatically: `sender` gets type `&String` and `payload` gets `&Vec<u8>`. You only need explicit `ref` or `ref mut` when matching on an *owned* value where you wish to borrow one field without consuming the rest (e.g., `Some(ref s)` on an owned `Option<String>`).

### 5.4.3 Exhaustiveness Across Codebase Evolution and Control-Flow Shortcuts

Suppose `enum OrderStatus { Pending, Settled, Failed }` is matched in forty files across a service. Six months later, a teammate adds `Disputed { dispute_id: u64 }`. In Java 6, existing `switch (status)` blocks with a `default:` branch compile cleanly and silently misroute disputed orders in production. In Rust, `cargo check` immediately flags `non-exhaustive patterns: 'OrderStatus::Disputed { .. }' not covered` at every single unhandled `match` across all forty files, which gives you a list of every place that must decide how to handle the new state.

**Common trap**

**Avoid wildcard `_ => ...` arms on your own domain enums.** A catch-all `_` arm disables future exhaustiveness errors on that `match`. Group shared cases with an explicit or-pattern instead (`OrderStatus::Settled | OrderStatus::Failed => false`), reserving `_` for scalars (`u8`, `&str`) or external `#[non_exhaustive]` enums.

When a full multi-arm `match` is more ceremony than a single check needs, reach for `let-else` guard clauses or `matches!`:

```
pub fn parse_bearer_token(header: Option<&str>) -> Result<&str, &'static str> {
    let Some(raw) = header else {
        return Err("missing Authorization header");
    };
    let Some(token) = raw.strip_prefix("Bearer ") else {
        return Err("Authorization scheme must be Bearer");
```

```rust
    };
    let token = token.trim();
    if token.is_empty() {
        return Err("empty bearer token");
    }
    Ok(token)
}

pub fn is_retryable_http(code: Option<u16>) -> bool {
    matches!(code, Some(429 | 500..=599))
}
```

In `let Some(raw) = header else { return ...; };`, `raw` stays bound at the outer indentation level with zero rightward drift because the compiler verifies that the `else` block diverges to type `!`.

**Focus**

**Choosing the right pattern construct:**

- `match expr { ... }`: multi-variant dispatch and domain enums where future variant additions must trigger compiler errors.
- `let Pat = expr else { return ...; };`: flat guard clauses unpacking the happy path into outer scope.
- `if let Pat = expr { ... }`: optional side effect on a single pattern when all other cases truly do nothing.
- `matches!(expr, Pat)`: concise boolean test supporting ranges, destructuring, and `|` alternatives.

## 5.5 Case Study: Refactoring a Legacy Java 6 `PaymentOrder` into a Rust State Machine

Let us combine cardinality algebra, niche newtypes, `Result`, and exhaustive `match` to refactor a buggy Java 6 payment order class into a compile-time-checked Rust state machine.

### 5.5.1 Four Silent Bugs in the Legacy Java 6 `PaymentOrder`

```java
// Java 6: Mutable product class with nullable lifecycle fields
public class PaymentOrder {
    public enum Status { CREATED, AUTHORIZED, CAPTURED, SETTLED, REFUNDED, FAILED }

    private final long orderId;
    private Status status;
    private long requestedCents, authorizedCents, capturedCents, settledCents;
    private String authCode;        // null in CREATED, FAILED
    private String captureTxId;     // null until CAPTURED
    private Long settledAtEpochMs;  // null until SETTLED
    private String refundReason;    // null unless REFUNDED
    private String failureCode;     // null unless FAILED
```

```
13
14      public PaymentOrder(long orderId, long requestedCents) {
15          this.orderId = orderId;
16          this.status = Status.CREATED;
17          this.requestedCents = requestedCents;
18      }
19
20      public void capture(long amountCents, String txId) {
21          this.capturedCents = amountCents;
22          this.status = Status.CAPTURED;
23          // Bug 1: forgot this.captureTxId = txId -> NPE in nightly settlement!
24          // Bug 2: forgot status == AUTHORIZED and amountCents <= authorizedCents!
25      }
26
27      public void refund(long amountCents, String reason) {
28          if (amountCents > this.settledCents) {
29              throw new IllegalArgumentException("refund exceeds settled");
30          }
31          this.refundReason = reason;
32          this.status = Status.REFUNDED; // Bug 3: refund(0, "x") succeeds on CREATED!
33      }
34  }
```

Java's compiler accepts all four failure modes here without complaint: (1) moving to `CAPTURED` while leaving `captureTxId == null`, which surfaces hours later as a `NullPointerException` in the nightly settlement job; (2) capturing an order that was never authorized, or capturing more than was authorized, because `capture` checks neither `status` nor `authorizedCents`; (3) calling `refund(0, "x")` on an uncharged `CREATED` order, which passes the guard because `settledCents` defaulted to `0L`; and (4) accepting zero or negative amounts, since every amount is a signed `long`. On top of that, a method that throws halfway through leaves a half-mutated object behind.

### 5.5.2 Step-by-Step Refactoring into an Algebraic State Machine

**Step 1: Validate leaf primitives with niche-preserving newtypes.** Wrapping `NonZeroU64` in `Cents` rules out zero and negative amounts at construction while keeping `Option<Cents>` at 8 bytes:

```
use core::num::NonZeroU64;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Cents(NonZeroU64);

impl Cents {
    pub fn new(raw: u64) -> Option<Self> { NonZeroU64::new(raw).map(Cents) }
    pub fn get(self) -> u64 { self.0.get() }
}
const _: () = assert!(core::mem::size_of::<Option<Cents>>() == 8);

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct AuthCode(String);
impl AuthCode {
    pub fn parse(raw: &str) -> Option<Self> {
        let s = raw.trim();
```

```
        (!s.is_empty()).then(|| AuthCode(s.to_owned()))
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CaptureTxId(String);
impl CaptureTxId {
    pub fn parse(raw: &str) -> Option<Self> {
        let s = raw.trim();
        (!s.is_empty()).then(|| CaptureTxId(s.to_owned()))
    }
}
```

**Step 2: Model the lifecycle as a sum of exact per-stage payloads.** Each variant of `PaymentState` carries *only* the data that exists in that stage, matching Figure 5.5:

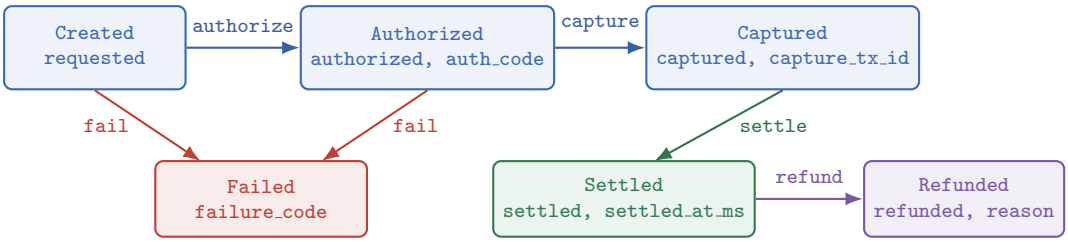



Figure 5.5: Lifecycle graph of `PaymentOrder`. Each variant owns only the fields valid in that state, and `fail` is reachable only before funds are captured.

**Step 3: Implement ownership-consuming transitions with explicit exhaustive arms.** Bringing variants into scope with `use PaymentState::*;` keeps each `match` arm concise while spelling out rejected states via or-patterns (rather than `_`) so adding a new state later triggers compiler errors across every transition:

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum PaymentState {
    Created { requested: Cents },
    Authorized { authorized: Cents, auth_code: AuthCode },
    Captured { authorized: Cents, captured: Cents, capture_tx_id: CaptureTxId },
    Settled { settled: Cents, capture_tx_id: CaptureTxId, settled_at_ms: u64 },
    Refunded { refunded: Cents, capture_tx_id: CaptureTxId, reason: String },
    Failed { failure_code: String },
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct PaymentOrder {
    pub order_id: u64,
    pub state: PaymentState,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum OrderTransitionError {
    InvalidState { attempted: &'static str },
    CaptureExceedsAuth { authorized: u64, requested: u64 },
    RefundExceedsSettled { settled: u64, requested: u64 },
```

```
    EmptyReason,
}

use PaymentState::*;

impl PaymentOrder {
    pub fn new(order_id: u64, requested: Cents) -> Self {
        Self { order_id, state: Created { requested } }
    }

    pub fn authorize(self, auth_code: AuthCode) -> Result<Self, OrderTransitionError> {
        match self.state {
            Created { requested } => Ok(Self {
                order_id: self.order_id,
                state: Authorized { authorized: requested, auth_code },
            }),
            Authorized { .. } | Captured { .. } | Settled { .. }
            | Refunded { .. } | Failed { .. } => {
                Err(OrderTransitionError::InvalidState { attempted: "authorize" })
            }
        }
    }

    pub fn capture(
        self,
        amount: Cents,
        capture_tx_id: CaptureTxId,
    ) -> Result<Self, OrderTransitionError> {
        match self.state {
            Authorized { authorized, .. } => {
                if amount.get() > authorized.get() {
                    return Err(OrderTransitionError::CaptureExceedsAuth {
                        authorized: authorized.get(),
                        requested: amount.get(),
                    });
                }
                Ok(Self {
                    order_id: self.order_id,
                    state: Captured { authorized, captured: amount, capture_tx_id },
                })
            }
            Created { .. } | Captured { .. } | Settled { .. }
            | Refunded { .. } | Failed { .. } => {
                Err(OrderTransitionError::InvalidState { attempted: "capture" })
            }
        }
    }

    pub fn settle(self, settled_at_ms: u64) -> Result<Self, OrderTransitionError> {
        match self.state {
            Captured { captured, capture_tx_id, .. } => Ok(Self {
                order_id: self.order_id,
                state: Settled { settled: captured, capture_tx_id, settled_at_ms },
            }),
```

```
            Created { .. } | Authorized { .. } | Settled { .. }
            | Refunded { .. } | Failed { .. } => {
                Err(OrderTransitionError::InvalidState { attempted: "settle" })
            }
        }
    }

    pub fn refund(self, amount: Cents, reason: String) -> Result<Self,
    OrderTransitionError> {
        if reason.trim().is_empty() {
            return Err(OrderTransitionError::EmptyReason);
        }
        match self.state {
            Settled { settled, capture_tx_id, .. } => {
                if amount.get() > settled.get() {
                    return Err(OrderTransitionError::RefundExceedsSettled {
                        settled: settled.get(),
                        requested: amount.get(),
                    });
                }
                Ok(Self {
                    order_id: self.order_id,
                    state: Refunded { refunded: amount, capture_tx_id, reason },
                })
            }
            Created { .. } | Authorized { .. } | Captured { .. }
            | Refunded { .. } | Failed { .. } => {
                Err(OrderTransitionError::InvalidState { attempted: "refund" })
            }
        }
    }

    pub fn fail(self, failure_code: String) -> Result<Self, OrderTransitionError> {
        match self.state {
            Created { .. } | Authorized { .. } => Ok(Self {
                order_id: self.order_id,
                state: Failed { failure_code },
            }),
            Captured { .. } | Settled { .. } | Refunded { .. } | Failed { .. } => {
                Err(OrderTransitionError::InvalidState { attempted: "fail" })
            }
        }
    }
}
```

Each of the four Java 6 bugs is now either unrepresentable or rejected by a checked transition: `Captured` cannot be constructed without a non-empty `CaptureTxId`; `capture` only accepts an `Authorized` order and returns `CaptureExceedsAuth` instead of over-capturing; `refund` can only read a balance out of `PaymentState::Settled`; and `Cents` cannot hold zero or a negative number. Because each transition consumes `self` and builds a fresh value, there is no half-mutated object to leave behind, and callers cannot keep using a stale pre-transition variable. Moving `capture_tx_id` from `Captured` into `Settled` copies 24 stack bytes without cloning the heap characters.

Connection

**Recovering `self` on error, and compile-time typestate.** If a caller needs the untransitioned `PaymentOrder` back when a transition fails without cloning up front, return `Result<Self, (Self, OrderTransitionError)>`. The standard library uses the same ownership-recovery idea: `String::from_utf8` hands the bytes back inside its `FromUtf8Error`, and `mpsc::Sender::send` hands the unsent value back inside its `SendError`. In Chapter 8 we will also meet the *typestate* pattern (`PaymentOrder<Created>`, `PaymentOrder<Authorized>`), which encodes known states in generic type parameters; runtime sum enums (`PaymentState`, essential when loading rows dynamically from PostgreSQL in Chapter 13) and compile-time typestates work side by side in production Rust.

In Chapter 6 we move from modeling the *shape* of a single type to abstracting *shared behavior* across types: traits, generics, monomorphization, and `dyn Trait` vtables — the machinery behind `From::from` and `?` in this chapter, built without a single `extends` superclass.

## 5.6 Exercises

**Exercise 5.1** (Counting inhabitants vs. domain states)**.** A Java 6 `Subscription` class stores `boolean isTrial; boolean isCanceled; Integer cancelCode;`, whereas in the domain a subscription is either in a free trial, active paid, or canceled with a 16-bit code (`0..=65535`). (a) Compute the exact representable cardinality of the Java 6 class (remembering `null` inside `Integer`). (b) Compute the exact cardinality of valid domain states. (c) Write a Rust `enum SubscriptionState` whose cardinality equals (b) *exactly*.

**Exercise 5.2** (Algebraic identities with `!` and `()`)**.** Compute $|T|$ for (a) `Result<bool, !>`, (b) `(u8, ())`, (c) `(u64, !)`, and (d) `Option<Option<bool>>`, giving a simpler isomorphic Rust type for each. Then explain why `fn witness(x: (u64, !)) -> String { match x.1 {} }` compiles with zero `match` arms and can never be called at runtime.

**Exercise 5.3** (Struct padding by hand and with `rustc`)**.** Given fields `a: u8, b: u32, c: u16, d: u64` on a 64-bit target: (a) Under `#[repr(C)]` in order `(a, b, c, d)`, list every field offset, padding range, `align_of`, and `size_of`. (b) Reorder the fields manually under `#[repr(C)]` to minimize `size_of`, and verify both structs with compile-time `assert!`s in `rustc`.

**Exercise 5.4** (Tagged union byte layout and tag tucking)**.** Define `MarketTick` with three variants: `Halted`, `Quote { symbol_id: u32, bid: u64, ask: u64 }`, and `Trade { symbol_id: u32, price: u64, lots: u16 }`. (a) Find `align_of` and `size_of` under `#[repr(C, u8)]` and give every field's byte offset. (b) Under default `repr(Rust)`, `MarketTick` shrinks from 32 bytes to 24 bytes. Explain where inside those 24 bytes `rustc` tucks the discriminant tag, and verify with `rustc`.

**Exercise 5.5** (Hunting for niches)**.** Predict `size_of` on 64-bit Rust before checking with `rustc`, justifying each answer via niches: (a) `Option<Option<bool>>`; (b) `Option<&[u8]>` (recall from Chapter 3 that `&[u8]` is a 16-byte `(ptr, len)` pair); (c) `Option<Option<NonZeroU64>>`; (d) `Result<&u64, ()>`. Explain why (c) grows to 16 bytes while (a) stays 1 byte and (d) stays 8 bytes.

**Exercise 5.6** (Diagnosing and curing a Giant Variant Trap)**.** A middleware returns `Result<UserId, AuthError>`, where `UserId` wraps a `NonZeroU64`. The `AuthError` variants are `MissingCookie`, `Expired { at_s: u64 }`, and `InvalidJwt { raw: [u8; 1024], pos: usize }`. (a) Find the

size of `Result<UserId, AuthError>` and explain why it slows down the 99.99% `Ok` fast path. (b) Refactor `AuthError` so the `Result` drops to 16 bytes while keeping all diagnostic data, and prove it with a `const` assertion.

**Exercise 5.7** (Flattening nested matches with `?`, `From`, and `let-else`)**.** Write a function `parse_ledger_line` that takes `raw: Option<&str>` and returns a `Result` whose success value is the pair `(NonZeroU64, i64)` and whose error type is your own `LineParseError`. Reject `None`, split once on `':'`, parse the left side as `NonZeroU64` and the right as `i64`, and use `let-else`, `?`, and `From` with *zero* nested `match` blocks. Write out the manual `match` expansion of one `?` in your solution.

**Exercise 5.8** (Match ergonomics and accidental moves)**.** Let `JobOutcome` have variants `Done(u64)` and `Aborted(String)`, and let `out` have type `&mut JobOutcome`. Explain why matching on `*out` with arm `JobOutcome::Aborted(r) => r.len()` fails to compile whereas matching on `out` directly succeeds. Give the exact type of `r` in the working code and two ways to obtain `&String` instead.

**Exercise 5.9** (Extending the `PaymentOrder` state machine)**.** Extend `PaymentState` from Section 5.5 with two new variants: `PartiallyRefunded` (carrying `settled`, `refunded_so_far`, and `capture_tx_id` while $0 < R < S$, moving to `Refunded` once $R == S$) and `Disputed` (carrying `dispute_id: u64` and `capture_tx_id`, entered from `Settled` or `PartiallyRefunded` via `open_dispute` and blocking manual refunds). Compile before editing any methods to watch exhaustiveness flag all five transitions, then implement `refund` and `open_dispute` with unit assertions.

# Chapter 6

# Traits, Generics, Monomorphization, and Fat Pointers

In Chapter 5, we threw out two pillars of Java 6 and C++ object-oriented design at the same time: `null` references and `extends` class inheritance. Replacing `null` with `Option<T>` and state flags with `enum`s immediately paid off—illegal states stopped compiling, and data sat flat in memory without hidden object headers.

Yet if you have spent years building systems in Java 6 or C++, a nagging question should be tugging at your sleeve right now. *Without class inheritance, how do we write code that works across many types?* How do you write a single settlement engine that can price a wire transfer under a flat cents fee, a basis-points percentage fee, or a tiered institutional schedule? How do you swap a live PostgreSQL ledger store for a fast in-memory fake inside unit tests? How does `slice.sort()` sort integers, timestamps, and account identifiers without knowing your structs ahead of time?

Rust's answer is the **trait**. At a casual squint from thirty thousand feet, a `trait` looks a lot like a Java 6 `interface`. Both list method signatures that a concrete type promises to provide. Below the surface, however, Rust traits work on a completely different mechanical model from both Java interfaces and C++ `virtual` base classes:

1. **Data and behavior live apart.** A `struct` or `enum` defines *only* byte layout; `impl Trait for Type` attaches shared behavior afterward, from the outside—even onto primitive integers like `i64` or types from another library.
2. **Generics specialize without losing type checking.** When you write a bounded signature `fn price<P: FeePolicy>(p: &P)`, Rust checks the body of `price` *once* against `FeePolicy` (like Java generics, unlike C++98 templates), and then stamps out specialized, vtable-free machine code for each concrete `P` (like C++ templates, unlike Java's type erasure and heap boxing).
3. **Dynamic dispatch puts the vtable pointer in the reference, not in the object.** A Rust struct never carries a hidden C++ `vptr` or a 16-byte Java object header. You pay for virtual dispatch *only* at the exact boundary where you ask for a **trait object** (`&dyn FeePolicy` or `Box<dyn FeePolicy>`), represented as a 16-byte **fat pointer** pairing `data_ptr` and `vtable_ptr`.

Once you see those three mechanics clearly—down to the stack slots, heap allocations, and read-only

`.rodata` tables—every rule in Rust's trait system stops looking like an arbitrary compiler restriction and starts looking like simple physical necessity.

## 6.1 Decoupling Data from Behavior: `struct` vs `impl Trait`

Think about how Java 6 asks you to model a fee calculation contract for a payment processor. You start with an `interface`, and every concrete pricing rule must declare `implements FeePolicy` on the very line where the class is born:

Listing 6.1: Java 6 fuses memory layout, inherent methods, and interface membership inside a single class declaration.

```
public interface FeePolicy {
    long feeInCents(long amountCents);
}

public final class FlatCentsFee implements FeePolicy {
    private final long cents;

    public FlatCentsFee(long cents) {
        this.cents = cents;
    }

    public long feeInCents(long amountCents) {
        return this.cents;
    }
}
```

That arrangement feels completely natural until the day you meet two everyday engineering problems:

- **Retroactive capability.** Suppose a risk library you imported defines `FixedRateBps`, or suppose you want a raw primitive `long` (representing basis points) to satisfy `FeePolicy` directly in tests. In Java 6, `FixedRateBps` was compiled without `implements FeePolicy`, and `long` is not even an `Object`. Your only option is the Adapter pattern: allocate a brand-new wrapper object on the heap around every single value just to forward one method call.
- **Coupled responsibilities.** A Java `class` simultaneously defines three distinct things: (a) how fields sit in memory, (b) helper methods specific to that type, and (c) every polymorphic contract the type will ever participate in. Large enterprise classes accumulate `implements Serializable, Comparable<Order>, Auditable, Printable, Cloneable` until a single file becomes a magnet for every subsystem in the codebase.

Rust pulls those three jobs apart into separate, independent declarations. First, `struct` (or `enum`) describes *pure memory layout*—nothing else. Second, an **inherent `impl` block** (`impl FlatCentsFee`) attaches constructors and methods that belong directly to that type. Third, a **trait `impl` block** (`impl FeePolicy for Type`) bridges a trait contract onto *any* type—including built-in primitives like `i64`:

Listing 6.2: Rust separates byte layout (struct), inherent methods (impl Type), and polymorphic contracts (impl Trait for Type).

```rust
pub trait FeePolicy {
    // Required method: every implementor must supply this body.
    fn fee_in_cents(&self, amount_cents: i64) -> i64;

    // Default method: calls fee_in_cents(&self, ...) automatically.
    // Implementors get this for free, or may override it.
    fn net_settlement_cents(&self, amount_cents: i64) -> i64 {
        amount_cents - self.fee_in_cents(amount_cents)
    }
}

// 1. Pure data layout: exactly 8 bytes in memory, zero hidden header.
pub struct FlatCentsFee {
    pub cents: i64,
}

// 2. Inherent methods specific to FlatCentsFee.
impl FlatCentsFee {
    pub fn new(cents: i64) -> Self {
        assert!(cents >= 0, "fee cannot be negative");
        Self { cents }
    }
}

// 3. Attaching the FeePolicy contract to FlatCentsFee.
impl FeePolicy for FlatCentsFee {
    fn fee_in_cents(&self, _amount_cents: i64) -> i64 {
        self.cents
    }
}

// 4. Attaching our trait directly to Rust's built-in 64-bit integer!
//    Here an i64 represents basis points (1 bp = 1/100 of 1 percent).
impl FeePolicy for i64 {
    fn fee_in_cents(&self, amount_cents: i64) -> i64 {
        (amount_cents * (*self)) / 10_000
    }
}
```

Pause on block 4 for a moment. We just implemented `FeePolicy` directly on `i64`. No `LongWrapper` class, no heap allocation, no pointer indirection: calling `25_i64.fee_in_cents(100_000)` passes the address of an 8-byte integer on the stack and computes `250` cents directly in registers. Figure 6.1 contrasts the coupled OOP class boundary against Rust's separated layout and trait bridges.

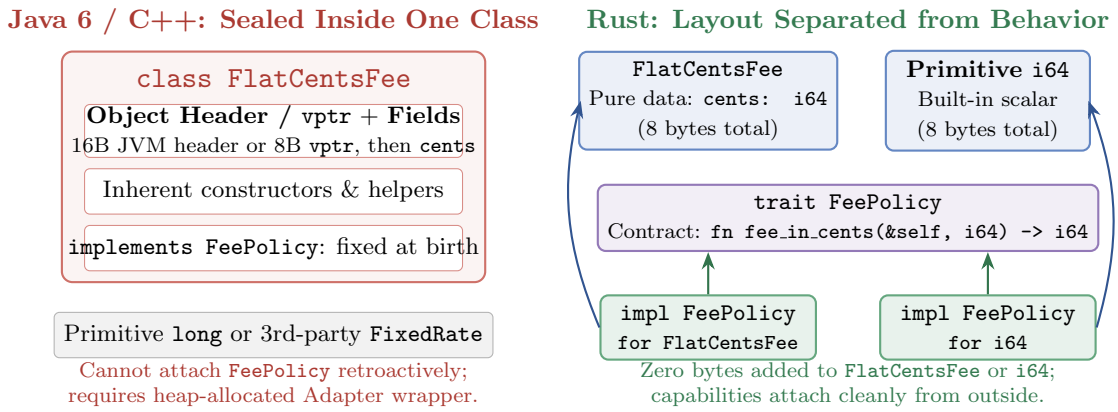



Figure 6.1: In Java 6 and C++, polymorphic interfaces and virtual tables must be baked into the class definition itself, bloating object layout and excluding primitives or external structs. Rust keeps data layout completely separate from `impl Trait for Type` blocks.

### 6.1.1 Reading `self` vs `Self` Through the Ownership Lens

Notice two spellings that appear side by side in almost every Rust trait: lowercase `self` and uppercase `Self`. Keeping them distinct makes every trait signature in the standard library immediately legible:

- **Uppercase `Self` is a type alias for "whichever concrete type is implementing this trait right now."** Inside `impl FeePolicy for FlatCentsFee`, `Self` means `FlatCentsFee`; inside `impl FeePolicy for i64`, `Self` means `i64`.
- **Lowercase `self` is the method's first value parameter—the receiver—with its ownership mode spelled out explicitly.** In Java 6 and C++, every non-static method implicitly receives a mutable pointer `this`. Rust refuses to hide how a method accesses its receiver; instead, trait methods use the exact three ownership modes from Chapter 3:
  1. `&self` (short for `self: &Self`): borrows the receiver immutably, as in `fee_in_cents`. Ten callers can share references and compute fees at the same time.
  2. `&mut self` (short for `self: &mut Self`): borrows the receiver exclusively to mutate state in place, as when `next(&mut self)` on `Iterator` advances a cursor.
  3. `self` (short for `self: Self`): **consumes the receiver by value**, like `into_iter(self)` on `IntoIterator` tearing open a `Vec<T>` to yield owned items, after which the original `Vec` cannot be touched again.
- **Trait functions with no `self` parameter at all** are called **associated functions** (analogous to static interface factories, except polymorphic over `Self`), such as `Default::default()` returning `Self` or `FromStr::from_str(s)` returning `Result<Self, Self::Err>`.

**Common trap**

**Trait methods are only in scope when the trait itself is in scope.** If module `billing` defines `pub trait FeePolicy` and implements it for `FlatCentsFee`, another module that imports *only* `use crate::billing::FlatCentsFee;` cannot call `flat.fee_in_cents(500)` until it *also* writes `use crate::billing::FeePolicy;`. At first this surprises Java developers

(where importing a class imports all its interface methods). Rust requires importing the trait so that two independent crates can each define their own helper trait on `FlatCentsFee` (or `i64`) without colliding in modules that did not ask for them.

## 6.2 Generics Across Three Worlds: Erasure, Templates, and Bounds

Suppose we want to write a batch settlement calculator `total_fees` that sums the fees across a slice of payment amounts using *any* pricing rule that implements `FeePolicy`. Java 6, C++, and Rust all let you write a generic function for this job, yet what their compilers check—and what bytes they feed to the CPU—differ sharply. Let us line all three up on the same problem.

### 6.2.1 World 1: Java 6 Type Erasure (Safe Definition Check, Boxed Runtime)

In Java 6 (which added generics in JDK 5 while preserving backwards bytecode compatibility with JDK 1.4), you write a bounded type parameter `<P extends FeePolicy>`:

Listing 6.3: Java 6 bounded generic method and what the compiler erases it into.

```
// What you write in Java 6:
public static <P extends FeePolicy> long totalFees(
        List<Long> amounts, P policy) {
    long sum = 0L;
    for (Long amt : amounts) {
        sum += policy.feeInCents(amt.longValue());
    }
    return sum;
}

// What javac actually emits in bytecode (type erasure):
public static long totalFees(List amounts, FeePolicy policy) {
    long sum = 0L;
    for (Iterator it = amounts.iterator(); it.hasNext(); ) {
        Long amt = (Long) it.next(); // runtime checkcast!
        sum += policy.feeInCents(amt.longValue()); // invokeinterface!
    }
    return sum;
}
```

Here is the bargain Java 6 struck. On the bright side, **definition-site type checking is rock solid**: `javac` type-checks `totalFees` *once*, right when you compile `totalFees`, verifying that every method called on `policy` is guaranteed by `FeePolicy`. If you accidentally type `policy.discountBps()` inside `totalFees`, `javac` rejects `totalFees` immediately, before anyone ever calls it.

On the hardware side, however, type erasure exacts three steep tolls:

1. **No primitives in type parameters.** Because every generic parameter `T` erases to `Object` (or its upper bound), `List<long>` is illegal in Java 6. A million payment amounts in `List<Long>` become a million 8-byte references pointing to a million separate 24-byte `java.lang.Long`

heap objects—32 MB of pointer-chasing cache misses for 8 MB of numbers, as we measured in Chapter 1.

2. **One unspecialized bytecode body.** Whether you pass `FlatCentsFee` or `TieredFee`, both execute the exact same erased `totalFees` method and dispatch `policy.feeInCents(...)` via an `invokeinterface` lookup. (HotSpot's JIT compiler works hard at runtime with inline caches to devirtualize monomorphic call sites, and falls right back off a cliff into megamorphic vtable dispatch the moment three or four implementations share that call site)

3. **Runtime amnesia.** Inside a generic Java method, `T` is gone at runtime: you cannot write `new T()`, `new T[n]`, or `x instanceof T`.

### 6.2.2 World 2: C++ Templates (Zero-Cost Machine Code, Late Duck-Typed Checks)

C++ went to the opposite extreme. A C++98/17 function template does not erase anything; it treats the template like a typed code generator stamped out separately for every concrete type `P` you pass at call sites:

Listing 6.4: C++98/17 unconstrained template vs C++20 Concept constraint.

```cpp
#include <cstdint>
#include <vector>

// C++98/17 unconstrained template: P is not checked against any
// explicit contract until a caller instantiates total_fees<SomeType>!
template <typename P>
std::int64_t total_fees_cpp17(
        const std::vector<std::int64_t>& amounts, const P& policy) {
    std::int64_t sum = 0;
    for (std::int64_t amt : amounts) {
        sum += policy.fee_in_cents(amt); // duck-typed lookup!
    }
    return sum;
}

// C++20 Concept: constrains call sites structurally, though the
// template body itself is still NOT restricted solely to the concept!
template <typename P>
concept FeePolicyConcept = requires(const P& p, std::int64_t amt) {
    { p.fee_in_cents(amt) } -> std::same_as<std::int64_t>;
};
```

At runtime, C++ wins completely: `std::vector<std::int64_t>` is one contiguous slab of 8-byte integers, `policy` has no `virtual` keyword, and calling `total_fees_cpp17` with `FlatCentsFee{25}` generates a dedicated machine-code function where `policy.fee_in_cents(amt)` inlines straight into `sum += 25`.

Yet notice the modularity flaw in C++ templates: **structural duck typing at instantiation time**. In `total_fees_cpp17`, if the library author accidentally writes `policy.typo_method()` inside an `if constexpr` or helper branch, or if a caller five layers up passes a struct whose `fee_in_cents` returns `double` instead of `int64_t`, the error explodes *inside the library's implementation lines* with a notorious multi-screen template instantiation backtrace. Even with C++20 `concepts` (which cleanly catch

bad arguments at the call boundary), the compiler *still* permits the body of a concept-constrained C++20 template to call extra methods on `policy` that were never listed in `FeePolicyConcept`—you only find out you forgot a requirement in your concept when some future caller passes a type that satisfies the concept minimalistically.

### 6.2.3 World 3: Rust's Synthesis (Definition-Checked Bounds + Specialization)

Rust combines Java's strict, modular definition-time contract checking with C++'s zero-cost per-type code generation. Look at `total_fees` in Rust:

Listing 6.5: Rust checks generic bodies strictly against trait bounds once at definition time, then compiles specialized machine code per concrete type.

```
pub fn total_fees<P: FeePolicy>(amounts: &[i64], policy: &P) -> i64 {
    let mut sum = 0_i64;
    for &amt in amounts {
        sum += policy.fee_in_cents(amt);
    }
    sum
}
```

Try an experiment in your head: suppose `FlatCentsFee` also has an inherent method `pub fn label(&self) -> &'static str`, and inside `total_fees` we try to call `policy.label()`. Even if the *only* call to `total_fees` in the entire program passes a `FlatCentsFee`, **`rustc` rejects `total_fees` immediately.** Why? Because `P: FeePolicy` promises *only* the methods of `FeePolicy`, and `label` is not part of `FeePolicy`. Once `total_fees` compiles, you have a mathematical guarantee that *every* type implementing `FeePolicy`—today or five years from now in a downstream crate—will instantiate `total_fees` without a single error inside its body. Figure 6.2 maps all three compilation models side by side.

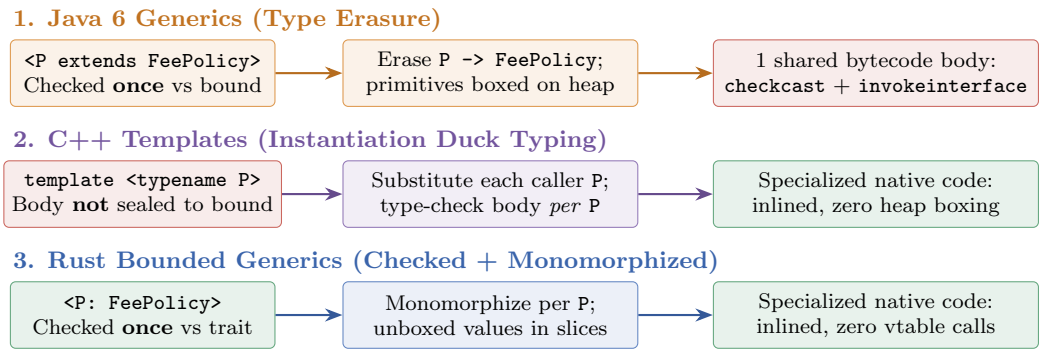



Figure 6.2: Three ways to compile a generic function. Java 6 checks the generic body cleanly at definition time, then erases type arguments to heap pointers at runtime. C++ templates emit specialized native code, at the cost of checking the body during substitution. Rust keeps definition-time checking *and* specialized zero-overhead machine code.

### 6.2.4 Syntax Ergonomics: `where` Clauses and `impl Trait`

As signatures grow beyond one bound, stuffing constraints inside `<...>` clutters the function name and parameter list. Rust gives you three clean ways to express trait bounds, and knowing what `impl Trait` means in argument position versus return position saves hours of confusion:

Listing 6.6: Equivalent trait bound spellings, plus return-position impl Trait.

```
use std::fmt::Debug;

// 1. Inline bound: crisp for 1 or 2 short bounds.
pub fn quote_a<P: FeePolicy + Debug>(amt: i64, policy: &P) -> i64 {
    policy.fee_in_cents(amt)
}

// 2. 'where' clause: cleanest when bounds span multiple types.
pub fn quote_b<P>(amt: i64, policy: &P) -> i64
where
    P: FeePolicy + Debug,
{
    policy.fee_in_cents(amt)
}

// 3. Argument-position 'impl Trait': syntactic sugar for an anonymous
//    generic parameter (note: callers cannot turbofish ::<Type>!).
pub fn quote_c(amt: i64, policy: &(impl FeePolicy + Debug)) -> i64 {
    policy.fee_in_cents(amt)
}

// 4. Return-position '-> impl Trait': "I return ONE specific concrete
//    type known to the compiler, without exposing its struct name."
pub fn standard_retail_fee() -> impl FeePolicy {
    FlatCentsFee::new(35)
}
```

**Common trap**

**Return-position `-> impl FeePolicy` still returns ONE concrete type.** A classic surprise for Java developers is writing:

```
// COMPILE ERROR: 'if' and 'else' have incompatible types!
pub fn choose_policy(vip: bool) -> impl FeePolicy {
    if vip {
        FlatCentsFee::new(0)  // 8-byte FlatCentsFee
    } else {
        25_i64                // 8-byte i64 (different concrete type!)
    }
}
```

Why does `rustc` reject this? Because `-> impl FeePolicy` is *static dispatch* with zero heap boxing: the caller reserves exact stack space for *one* concrete return type decided at compile time (often an unnameable closure or iterator adapter chain, as we will see in Chapter 7). If two branches genuinely return *different* concrete types chosen at runtime, you have two

idiomatic options: either return a closed `enum AnyFee { Flat(FlatCentsFee), Bps(i64) }` (stack-allocated, zero heap), or box a trait object `Box<dyn FeePolicy>` (Section 6.4).

## 6.3 Static Dispatch and Monomorphization Down to Machine Code

Let us trace *monomorphization* (from Greek *mono* "single" + *morphe* "form": turning one polymorphic generic template into single-form concrete functions) all the way down to what LLVM hands your CPU. Suppose our binary calls `total_fees` in two places—once with a `FlatCentsFee` and once with a `TieredBpsFee`:

Listing 6.7: Two calls to a generic function and the two specialized functions rustc stamps out before LLVM optimization.

```
pub struct TieredBpsFee {
    pub threshold_cents: i64,
    pub low_bps: i64,
    pub high_bps: i64,
}

impl FeePolicy for TieredBpsFee {
    fn fee_in_cents(&self, amount_cents: i64) -> i64 {
        let bps = if amount_cents >= self.threshold_cents {
            self.high_bps
        } else {
            self.low_bps
        };
        (amount_cents * bps) / 10_000
    }
}

pub fn run_settlement_demo(batch: &[i64]) -> (i64, i64) {
    let flat = FlatCentsFee::new(30);
    let tiered = TieredBpsFee {
        threshold_cents: 100_000,
        low_bps: 25,
        high_bps: 10,
    };
    // Call 1 instantiates P = FlatCentsFee
    // Call 2 instantiates P = TieredBpsFee
    (total_fees(batch, &flat), total_fees(batch, &tiered))
}
```

During compilation, `rustc` walks the call graph starting from `main` (or exported public symbols), discovers that `total_fees` is instantiated with `P = FlatCentsFee` and `P = TieredBpsFee`, and emits two completely separate functions in LLVM IR—as if you had written them by hand:

Listing 6.8: Conceptual monomorphized output generated by rustc prior to LLVM inlining.

```
fn total_fees_for_FlatCentsFee(
    amounts: &[i64],
    policy: &FlatCentsFee,
```

```
) -> i64 {
    let mut sum = 0_i64;
    for &amt in amounts {
        // Direct static call to FlatCentsFee::fee_in_cents!
        sum += FlatCentsFee::fee_in_cents(policy, amt);
    }
    sum
}

fn total_fees_for_TieredBpsFee(
    amounts: &[i64],
    policy: &TieredBpsFee,
) -> i64 {
    let mut sum = 0_i64;
    for &amt in amounts {
        // Direct static call to TieredBpsFee::fee_in_cents!
        sum += TieredBpsFee::fee_in_cents(policy, amt);
    }
    sum
}
```

Under the hood

**Why skipping the vtable call is only 10% of the speedup; the other 90% is inlining.** Beginners often hear "virtual calls are slower than static calls" and think of the 1–3 CPU cycles spent loading a function pointer from a vtable. On a modern out-of-order CPU with branch target buffers, an indirect call that always hits the same target barely costs two nanoseconds on its own.

The real mechanical advantage of static dispatch is **compiler visibility across the call boundary**. In `total_fees_for_FlatCentsFee`, LLVM sees the body of `FlatCentsFee::fee_in_cents` right inside the loop:

1. LLVM inlines `policy.cents` directly into `sum += policy.cents`.
2. LLVM notices `amt` is never even read. It deletes the entire memory scan over `amounts` and collapses a 1,000,000-iteration loop into a **single integer multiplication**: `(amounts.len() as i64) * policy.cents`.

Across an opaque virtual call (`&dyn FeePolicy`), the compiler cannot prove what `fee_in_cents` does, so it *must* execute all 1,000,000 loop iterations and indirect calls one by one. Static dispatch lets small generic abstractions melt completely into surrounding arithmetic.

### 6.3.1 The Cost of Monomorphization—and the Inner-Function Cure

Zero runtime overhead is not zero *compile-time* or *binary-size* cost. Suppose you write a 150-line function `load_settlement_csv<P: AsRef<Path>>(path: P)` that converts `path.as_ref()` into a `&Path` on line 1, and then spends lines 2–150 opening a file, parsing headers, and validating rows. If callers invoke `load_settlement_csv` with `&str`, `String`, `PathBuf`, and `&Path`, naive monomorphization stamps out **four identical 149-line copies** of the CSV parser in your compiled binary—slowing down LLVM optimization and polluting the CPU's L1 instruction cache.

Look inside Rust's own standard library (`std::fs::read`) and you will see the three-line cure used everywhere in production Rust: **monomorphize only the tiny conversion shim, and delegate immediately to a non-generic inner function**:

Listing 6.9: The std::fs::read pattern: a 3-line generic wrapper calling a single non-generic machine-code body.

```
use std::io;
use std::path::Path;

#[inline]
pub fn load_settlement_csv<P: AsRef<Path>>(path: P) -> io::Result<usize> {
    // Only this 1-line conversion is monomorphized per P (and inlined)!
    load_settlement_csv_inner(path.as_ref())
}

// Compiled ONCE in the binary: zero code duplication!
fn load_settlement_csv_inner(path: &Path) -> io::Result<usize> {
    let contents = std::fs::read_to_string(path)?;
    Ok(contents.lines().count())
}
```

## 6.4 Dynamic Dispatch and the 16-Byte Fat Pointer (`&dyn Trait`)

Static dispatch works whenever the compiler knows the exact concrete type at each call site. Sometimes, however, the whole point of your program is that *you do not know the concrete types until runtime*. Imagine loading active merchant fee rules from PostgreSQL at server startup: Merchant A uses a `FlatCentsFee`, Merchant B uses a `TieredBpsFee`, Merchant C uses a promotional `ZeroFee`, and you want to store all of them side by side in a single `Vec` or `HashMap<MerchantId, ...>`.

Can we write `Vec<FeePolicy>`? Think physically about what `Vec<T>` is from Chapter 3: a contiguous array where slot $k$ starts at byte offset $k \times$ `size_of::<T>()`. `FlatCentsFee` is 8 bytes; `TieredBpsFee` is 24 bytes; a custom `BracketSchedule` might be 64 bytes. A `FeePolicy` without a pointer wrapper has no fixed size (**Dynamically Sized Type**, or **DST**, written `!Sized`), so it cannot sit directly by value inside a `Vec` or stack slot. To hold heterogeneous implementors uniformly, we must refer to them behind a pointer: `&dyn FeePolicy` (borrowed) or `Box<dyn FeePolicy>` (heap-owned). The keyword `dyn` (added in Rust 2018 so dynamic dispatch is never invisible at the type level) marks a **trait object**.

Now comes the central design question of this chapter: *where does the virtual method table pointer live in memory?*

### 6.4.1 Why C++ and Java Hide Vtable Pointers Inside Every Object

In C++, the moment you mark *even one* method `virtual` on a base class `FeePolicy`, the C++ compiler inserts an invisible 8-byte pointer called a `vptr` at byte offset 0 of *every instance* of `FlatCentsFee`:

Listing 6.10: Adding virtual in C++ silently doubles the byte size of every FlatCentsFee instance from 8 bytes to 16 bytes.

```cpp
#include <cstdint>

struct NonVirtualFlatFee {
    std::int64_t cents; // sizeof == 8 bytes
};

struct VirtualFeeBase {
    virtual ~VirtualFeeBase() = default;
    virtual std::int64_t fee_in_cents(std::int64_t amt) const = 0;
};

struct VirtualFlatFee : VirtualFeeBase {
    std::int64_t cents; // sizeof == 16 bytes! ([8B vptr | 8B cents])
};
```

Look at the two consequences of embedding `vptr` inside the object itself (Java 6 does the exact same thing via its 12-to-16-byte `mark + klass` object header on every heap object):

1. **You pay the memory tax even when you do not use dynamic dispatch.** If you hold `std::vector<VirtualFlatFee>` with ten million elements and call methods statically, you still waste $10{,}000{,}000 \times 8 = 80$ MB of RAM storing ten million identical copies of the exact same `vptr`.

2. **You can never implement an interface on a type you did not define.** Where would the `vptr` go on a raw 8-byte `int64_t`, or on a `struct` compiled in a third-party C/C++ library? Those 8 bytes are already spoken for; there is nowhere inside the value to stash a vtable address.

### 6.4.2 Rust's Inversion: Keep Objects Headerless, Make the Pointer 16 Bytes

Rust turns the C++/Java layout inside out. `FlatCentsFee` is **8 bytes, always**—whether zero traits are implemented for it or fifty traits are implemented for it. An `i64` is **8 bytes, always**. When you coerce a regular 8-byte reference `&flat` (or `&25_i64`) into a trait object reference `&dyn FeePolicy`, **the pointer itself widens from 8 bytes to 16 bytes** (two machine words on 64-bit hardware):

- **Word 0 (`bytes 0..8`): `data_ptr`** — points directly to the raw, headerless concrete value in memory (e.g., the 8 bytes of `FlatCentsFee` or `i64` on the stack or heap).

- **Word 1 (`bytes 8..16`): `vtable_ptr`** — points to a static, read-only table in the binary's `.rodata` segment generated specifically for the pair `(ConcreteType, Trait)`.

Figure 6.3 lays out all three memory representations byte by byte. Study where the green vtable arrow originates in each row—that single arrow explains half of Rust's design.

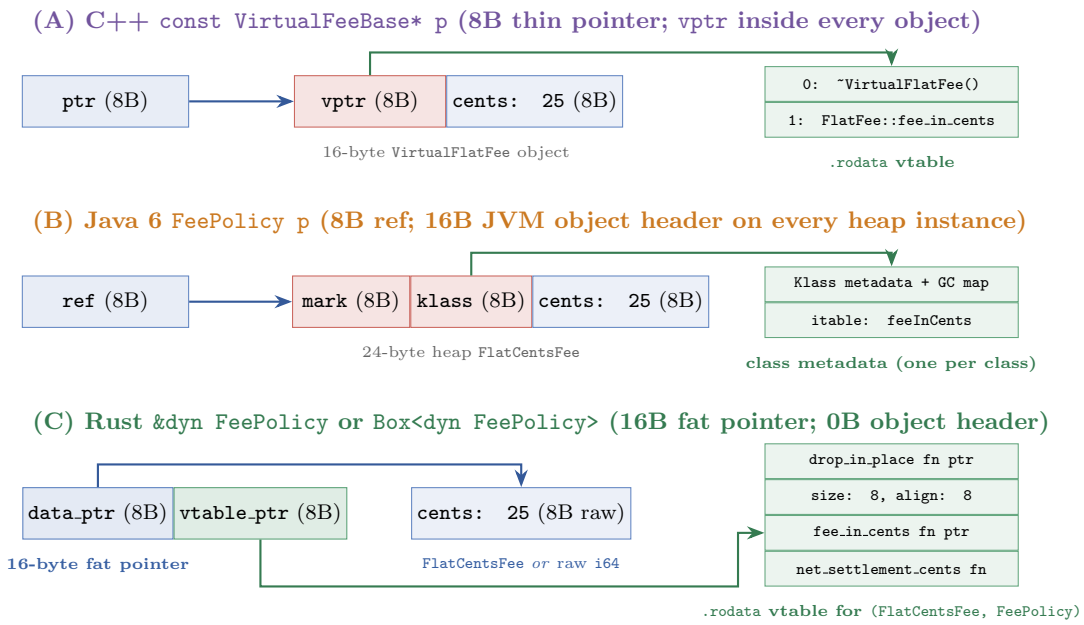



Figure 6.3: Where the vtable pointer lives. C++ and Java embed a vtable/class pointer inside *every* object instance, inflating object size and barring primitive scalars or external structs from implementing new interfaces. Rust keeps values completely bare (8 bytes) and pairs `(data_ptr, vtable_ptr)` into a 16-byte fat pointer *only* when coerced to `dyn Trait`.

### 6.4.3 Tracing a `dyn` Call and a `Box<dyn Trait>` Drop by Hand

We can spell out exactly what `rustc` generates for a borrowed or boxed trait object. The listing below checks the sizes and builds both kinds:

Listing 6.11: What &dyn FeePolicy and Box¡dyn FeePolicy¿ do at the machine-word level.

```
use std::mem::size_of;

pub fn demo_fat_pointers() {
    let flat = FlatCentsFee::new(25);
    let raw_bps: i64 = 30;

    // Regular thin references are 1 machine word (8 bytes on 64-bit):
    assert_eq!(size_of::<&FlatCentsFee>(), 8);
    assert_eq!(size_of::<&i64>(), 8);

    // Trait-object references are 2 machine words (16 bytes):
    assert_eq!(size_of::<&dyn FeePolicy>(), 16);
    assert_eq!(size_of::<Box<dyn FeePolicy>>(), 16);

    // Heterogeneous slice on the stack -- zero heap allocations!
    // Even raw_bps (a plain stack i64!) participates directly:
    let rules: [&dyn FeePolicy; 2] = [&flat, &raw_bps];
    let total: i64 = rules.iter().map(|r| r.fee_in_cents(100_000)).sum();
    assert_eq!(total, 25 + 300);

    // Owned heterogeneous collection on the heap:
```

```
    let owned_rules: Vec<Box<dyn FeePolicy>> = vec![
        Box::new(FlatCentsFee::new(25)),
        Box::new(30_i64),
        Box::new(TieredBpsFee {
            threshold_cents: 50_000,
            low_bps: 20,
            high_bps: 10,
        }),
    ];
    assert_eq!(owned_rules.len(), 3);
} // <-- When owned_rules drops here, how does Box free 3 differently
  //     sized heap allocations? Through the vtable!
```

Now consider the closing brace on the last line. `owned_rules[0]` points to an 8-byte heap block; `owned_rules[2]` points to a 24-byte heap block; another element might own an internal `String` whose destructor must free a second buffer. At the closing brace, the compiler only sees `Box<dyn FeePolicy>`—how does `Box::drop` know which destructor to run and how many bytes to hand back to the global allocator?

Look back at the four rows inside Rust's `.rodata` vtable in Figure 6.3:

1. `drop_in_place: unsafe fn(*mut ())`: a pointer to the concrete type's destructor (a no-op for `i64` or `FlatCentsFee`, or field-by-field cleanup if the struct owns a `String` or socket).

2. `size: usize` and `align: usize`: the exact byte size and alignment of the concrete value (`8, 8` for `FlatCentsFee`; `24, 8` for `TieredBpsFee`).

3. One function pointer slot per trait method (`fee_in_cents`, `net_settlement_cents`).

When `r.fee_in_cents(100_000)` executes on `r: &dyn FeePolicy`, the CPU loads the function pointer from `r.vtable_ptr` and invokes `fn_ptr(r.data_ptr, 100_000)`—passing the thin `data_ptr` in register `rdi` and `100_000` in `rsi`. When `Box<dyn FeePolicy>` goes out of scope, `Drop` invokes `drop_in_place(data_ptr)` from the vtable, reads `vtable.size` and `vtable.align`, and if `size > 0`, passes that exact `Layout` to `dealloc(data_ptr, layout)`.

**Intuition**

Notice the resemblance to Chapter 3's slices. In Rust, there are two classic kinds of **16-byte fat pointers** you meet constantly:

- **Slices (`&[T]`, `&str`, `Box<[T]>`):** `(data_ptr: *const T, len: usize)` — word 1 tells you *how many elements* live at `data_ptr`.
- **Trait objects (`&dyn Trait`, `Box<dyn Trait>`):** `(data_ptr: *const (), vtable_ptr: *const Vtable)` — word 1 tells you *how to operate on and drop* the erased bytes at `data_ptr`.

Both solve the exact same physical problem: taking an unsized (`!Sized`) view and attaching the one extra machine word of metadata needed to use it safely without bloating the underlying data.

## 6.5 dyn-Compatibility (Object Safety) Derived from the Vtable Layout

Sooner or later, every Rust programmer tries to write `Box<dyn Clone>` or `&dyn PartialEq` or adds a generic method to a trait, and meets a compiler diagnostic saying: *"the trait ... is not **dyn** compatible"* (historically called *object safety* prior to Rust 1.83).

You do not need to memorize the rules. Look at our 16-byte fat pointer and static `.rodata` vtable from Figure 6.3 and ask *"could the compiler actually build a vtable slot and call instruction for this method?"*, and each rule follows. Here are the three core rules directly from hardware mechanics:

1. **Methods cannot return bare `Self` by value (nor accept bare `Self` arguments).** Consider `Clone`, whose signature is `fn clone(&self) -> Self`. Suppose you hold `p: &dyn Clone` and write `let copy = p.clone();`. Before the CPU executes the `call` instruction, the caller's stack frame must reserve space for `copy`. *How many stack bytes should the caller reserve?* Eight bytes for `FlatCentsFee`? Twenty-four bytes for `String`? Four kilobytes for an array? At compile time, `p` could point to anything. Worse, for `PartialEq::eq(&self, other: &Self)`, if you hold two `&dyn PartialEq` pointers, nothing guarantees `a.data_ptr` and `b.data_ptr` even point to the same concrete type.
2. **Methods cannot carry generic type parameters, such as `fn encode<W: Write>(&self, w: &mut W)`.** Remember how generics compile in Rust: instantiations for `File`, `Vec<u8>`, and `TcpStream` are three distinct machine-code functions, potentially compiled months later in downstream crates. A `.rodata` vtable is a fixed, finite array of function pointers emitted at compile time. You cannot pack an open-ended family of monomorphized functions into one vtable slot.
3. **Methods dispatched through `dyn` must take a receiver** (such as `&self`, `&mut self`, `Box<Self>`, `Rc<Self>`, `Arc<Self>`, or `Pin<P>`). Consider `Default::default() -> Self`. There is no `self` argument at all—which means the caller has *no fat pointer* from which to read a `vtable_ptr`.

### 6.5.1 Two Practical Patterns When You Hit dyn-Compatibility

What do you do in real production code when you have `Box<dyn FeePolicy>` and genuinely need to clone it, or want a convenience generic method on the trait? Rust gives you clean patterns for both:

Listing 6.12: Making a trait object cloneable via Box, and keeping generic helpers off the vtable with 'where Self: Sized'.

```
use std::io::Write;

pub trait CloneFeePolicy {
    fn clone_box(&self) -> Box<dyn FeePolicy>;
}

// Blanket implementation: EVERY type T that is 'static + FeePolicy + Clone
// automatically knows how to clone itself into a fresh Box<dyn FeePolicy>!
impl<T> CloneFeePolicy for T
where
    T: 'static + FeePolicy + Clone,
```

```
{
    fn clone_box(&self) -> Box<dyn FeePolicy> {
        Box::new(self.clone()) // Here T is known and Sized!
    }
}

pub trait ReportableFee: FeePolicy {
    // 1. Vtable-safe core method takes a trait object (&mut dyn Write):
    fn write_audit_line(&self, out: &mut dyn Write) -> std::io::Result<()>;

    // 2. 'where Self: Sized' opts this generic helper OUT of the vtable!
    //    Static callers get full generic convenience, while
    //    '&dyn ReportableFee' remains fully dyn-compatible!
    fn format_with<F>(&self, amt: i64, f: F) -> String
    where
        Self: Sized,
        F: FnOnce(i64) -> String,
    {
        f(self.fee_in_cents(amt))
    }
}
```

Look at `where Self: Sized` on `format_with`: because `dyn ReportableFee` is *not* `Sized`, adding `where Self: Sized` tells `rustc` "omit this method from the vtable; only allow calling it when the concrete type is statically known." That exact clause is how Rust's own `std::iter::Iterator` trait offers seventy generic adapter methods (`map`, `filter`, `collect`) and *still* lets you pass `&mut dyn Iterator<Item = i64>` dynamically.

## 6.6 Associated Types, Coherence, Supertraits, and Blanket Impls

Once you start designing your own traits for domain services and storage layers, four structural questions come up right away:

1. When should a trait use an **associated type** (`type Output;`) instead of a **generic parameter** (`Trait<T>`)?
2. Why does `rustc` forbid implementing someone else's trait on someone else's struct (**coherence** and the **orphan rule**)?
3. How do traits build on other traits (**supertraits**) and grant methods across thousands of types at once (**blanket implementations**)?
4. When does a closed `enum` beat `Box<dyn Trait>`?

### 6.6.1 Associated Types (`type Item`) vs Generic Type Parameters (`From<T>`)

Compare two standard library traits side by side:

Listing 6.13: Generic type parameter (1-to-many input) vs Associated type (1-to-1 output).

```
// Generic parameter T: ONE type can implement From<T> for MANY T's!
```

```rust
pub trait From<T>: Sized {
    fn from(value: T) -> Self;
}

// Associated type Item: ONE cursor type yields ONE specific Item type!
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

Why wasn't `Iterator` written as `pub trait GenericIter<Item>`? Imagine what happens to a caller if `GenericIter<Item>` uses a type parameter. In theory, a single struct `CsvLines` *could* implement `GenericIter<String>` *and* `GenericIter<Vec<u8>>` *and* `GenericIter<i64>` at the same time. Because multiple implementations could exist, every caller writing `let x = iter.next();` would suffer type-inference ambiguity unless they annotated which `Item` they wanted, and every generic function taking `C: GenericIter<Row>` would have to drag extra type parameters through its `<...>` list.

The design rule is clean and memorable:

- **Use a generic parameter `Trait<In>` for *inputs* (many implementations per `Self`):** An `AccountId` can legitimately be constructed `From<u64>` *and* `From<u32>`. An arithmetic type `Money` might implement `Mul<i64>` (scaling money by an integer count) *and* `Mul<BasisPoints>`.
- **Use an associated type `type Out;` for *outputs* determined uniquely by `Self` (at most one implementation per `Self`):** Given a `PostgresPostingCursor`, the row type `Item = PostingRecord` and error type `Error = SqlError` are uniquely fixed by the cursor itself.

Notice that `std::ops::Add` combines *both* ideas with a default type parameter: `trait Add<Rhs = Self>` declares an associated `type Output;` and `fn add(self, rhs: Rhs) -> Self::Output`. You can add `Timestamp + Duration` (`Rhs = Duration`, input parameter), and once `Self` and `Rhs` are chosen, the result type (`Output = Timestamp`, associated output type) is uniquely locked in.

### 6.6.2 Coherence and the Orphan Rule: Saving Your `HashMap` from Silent Corruption

Try compiling this snippet in your own crate:

```rust
// COMPILE ERROR E0117: only traits defined in the current crate
// can be implemented for arbitrary types!
impl std::fmt::Display for Vec<i64> {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "len={}", self.len())
    }
}
```

Here `Display` lives in `std`, and `Vec<T>` also lives in `std`—neither belongs to your crate. Rust rejects the implementation under its **coherence** law, enforced by the **orphan rule**:

**Definition 6.1** (Coherence and the Orphan Rule)**.** A trait system satisfies **coherence** if, for any trait `Trait` and concrete types $T_0, T_1, \ldots, T_k$, there exists *at most one* implementation of `Trait` for those types across the entire dependency graph of a compiled program. To check coherence modularly

(one crate at a time, without peeking into downstream crates), Rust enforces the **orphan rule**: a crate may write `impl Trait for Type` only if either `Trait` or `Type` (subject to precise parameter coverage rules for generic traits) is defined inside the current crate.

**Why it works**

**Why does coherence matter so deeply?** Imagine Rust *did* allow orphan implementations. Crate `alpha` (your fraud-scoring dependency) implements `Hash` for an external `CurrencyPair` struct using little-endian byte order and builds a `HashMap<CurrencyPair, Rate>`. Crate `beta` (your reporting dependency) independently implements `Hash` for `CurrencyPair` using ASCII uppercase hashing. Now your `main` binary receives the `HashMap` built by `alpha` and passes it into `beta`, which calls `map.get(&eur_usd)`.
Because `beta` hashes `eur_usd` with a *different* `Hash` implementation than the one that inserted the bucket in `alpha`, `map.get` probes the wrong bucket and returns `None` for a key sitting right inside the map. Haskell actually permits "orphan instances" (with a compiler warning) and calls this exact bug the *HashMap coherence hazard*. Rust eliminates it unconditionally at compile time. Whenever you want `Display` or `Serialize` on an external `Vec<i64>`, you wrap it in a zero-cost local **Newtype** `pub struct LedgerAmounts(pub Vec<i64>);`—a flagship pattern we develop thoroughly in Chapter 8.

### 6.6.3 Supertraits and Blanket Implementations

Instead of class inheritance (`class Sub extends Base`), Rust composes contracts cleanly through **supertraits** and **blanket implementations**:

Listing 6.14: Supertraits require prerequisite contracts; blanket impls grant behavior to every type meeting a bound.

```
use std::fmt::{Debug, Display};

// Supertrait: any type implementing AuditableFee MUST also implement
// both FeePolicy and Debug. Inside AuditableFee methods, self can call
// methods from all three traits!
pub trait AuditableFee: FeePolicy + Debug {
    fn audit_entry(&self, amount_cents: i64) -> String {
        format!(
            "policy={:?} amount={} fee={}",
            self,
            amount_cents,
            self.fee_in_cents(amount_cents)
        )
    }
}

// Extension trait + Blanket implementation:
// Define our own trait SummaryExt and implement it in ONE stroke for
// every type P anywhere in the universe that implements FeePolicy!
pub trait FeeBatchExt: FeePolicy {
    fn max_fee_in_batch(&self, batch: &[i64]) -> i64 {
        batch
            .iter()
```

```
            .copied()
            .map(|amt| self.fee_in_cents(amt))
            .max()
            .unwrap_or(0)
    }
}

impl<P: FeePolicy + ?Sized> FeeBatchExt for P {}
```

Look at that single line at the bottom: `impl<P: FeePolicy + ?Sized> FeeBatchExt for P {}`. Two gems sit right inside those forty-five characters:

1. **Blanket reach.** Because `FeeBatchExt` is a local trait in our crate, the orphan rule is completely happy. Immediately, `FlatCentsFee`, `TieredBpsFee`, `i64`, and every future type implementing `FeePolicy` gain `.max_fee_in_batch(slice)`. Rust's standard library uses this exact mechanism everywhere: `impl<T: Display + ?Sized> ToString for T` gives `.to_string()` to every printable type, and `impl<T, U> Into<U> for T where U: From<T>` automatically provides `Into` whenever you implement `From`.

2. **The `?Sized` relaxation.** By default, every generic type parameter `<P>` in Rust carries an implicit bound `P: Sized` (meaning `P`'s byte size is known at compile time), because 95% of generic functions pass or store values on the stack. Writing `+ ?Sized` ("might or might not be `Sized`") lifts that default so `FeeBatchExt` *also* works on unsized trait objects like `dyn FeePolicy`.

### 6.6.4 Choosing Your Polymorphism Tool: Generics vs `dyn Trait` vs `enum`

Before reaching for `Box<dyn FeePolicy>` out of Java habit, always ask one architectural question: *is the set of concrete implementations open to downstream crates, or closed and known inside my system?* When the set of fee schedules in your financial backend has three or four known variants (`Flat`, `Bps`, `Tiered`), a plain Chapter 5 `enum FeeSchedule` beats `Box<dyn FeePolicy>` on every axis: zero heap allocation, contiguous `Vec<FeeSchedule>` cache locality, `Copy`/`Clone`/`Serialize` via one `#[derive(...)]` line, and branch-predicted `match` arms that inline completely. Figure 6.4 summarizes all three tools side by side.

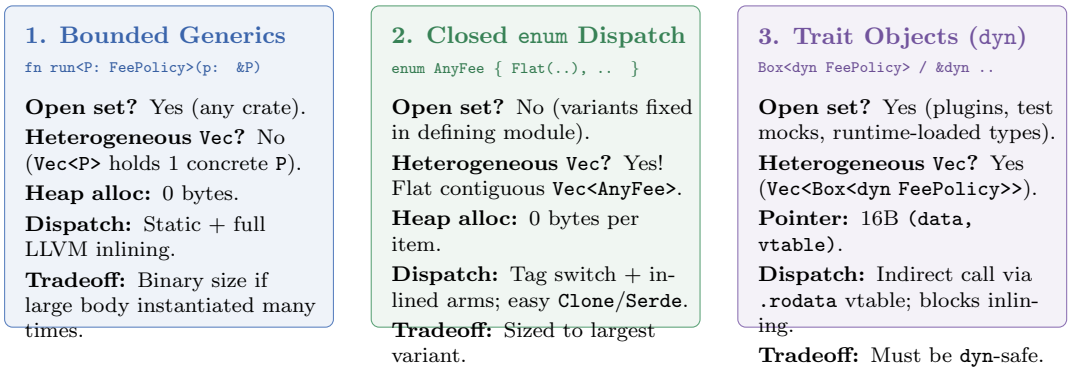



Figure 6.4: Choosing the right polymorphism mechanism in Rust. Default to bounded generics for single-type fast paths, closed `enum`s when a known set of variants must live together in a `Vec`, and `dyn Trait` fat pointers when an open set of types must mix at runtime or compile times demand erasing a boundary.

## 6.7 The Everyday Standard Library Trait Vocabulary

In Java 6, `java.lang.Object` forces five behaviors onto every object whether they make sense or not: `equals`, `hashCode`, `toString`, `clone` (which notoriously throws `CloneNotSupportedException` unless you also implement the empty marker `Cloneable`), and `wait`/`notify` monitor locks. Calling `equals` on two open `Socket`s or using a mutable `ArrayList` as a `HashMap` key compiles without complaint and misbehaves at runtime.

In Rust, a fresh `struct LedgerEntry` implements *none* of those operations until you opt in—often via a one-line `#[derive(Debug, Clone, PartialEq, Eq)]` macro that inspects every field and generates the boilerplate only if every field supports it. Let us walk through the core standard traits you will read and implement every day, highlighting the subtle design wins over Java 6 and C++.

### 6.7.1 `Debug` vs `Display`, and `Default`

- **`Debug` (`{:?}`, derivable):** Developer diagnostic formatting in `std::fmt` for logs, `assert_eq!`, and error traces. Almost every public type in Rust derives `Debug` (except types holding raw keys or passwords, where a custom `Debug` prints `"[REDACTED]"` so secrets never leak into logs).

- **`Display` (`{}`, never auto-derived):** User-facing text output in `std::fmt`. Rust deliberately refuses to provide `#[derive(Display)]` because how an end user should see a `Money` value (`"$12.50"`) or an error message is a domain contract, never an accidental dump of internal field names like Java's `Order@6d06d69c`.

- **`Default` (`Default::default()`, derivable):** Provides a canonical baseline value (`0` for numbers, `""` for `String`, empty `Vec`, `false` for `bool`). Combined with struct update syntax (`PoolConfig { max_conns: 32, ..Default::default() }`), it replaces telescoping constructor overloads cleanly.

### 6.7.2 `PartialEq/Eq` and `PartialOrd/Ord`: Why `f64` Cannot Corrupt Your `BTreeMap`

Why does Rust split equality into *two* traits (`PartialEq` and `Eq`) and ordering into *two* traits (`PartialOrd` and `Ord`)? Because of IEEE-754 floating-point `NaN` ("Not a Number", such as `0.0 / 0.0`).

Recall from discrete mathematics that a true **equivalence relation** requires three laws: symmetry ($a = b \implies b = a$), transitivity ($a = b \wedge b = c \implies a = c$), and **reflexivity** ($a = a$ for *every* $a$). Under IEEE-754 hardware rules, `f64::NAN == f64::NAN` evaluates to `false`—violating reflexivity. Similarly, `1.0 < f64::NAN` and `1.0 >= f64::NAN` are *both* `false`, violating a total ordering.

In Java 6 or C++, nothing stops you from sorting floats containing `NaN` or inserting weird comparators into a tree map, corrupting binary search invariants at runtime. Rust encodes the exact mathematical truth in traits:

- `PartialEq` requires symmetry and transitivity (`f64` implements `PartialEq`, so `x == y` works on floats).
- `pub trait Eq: PartialEq {}` is a zero-method marker subtrait promising **reflexivity** ($x == x$ always). Integers, `String`, and structs of integers implement `Eq`; `f32` and `f64` **do not**.
- `HashMap<K, V>` requires `K: Eq + Hash`, whereas `BTreeMap<K, V>` and `slice::sort` require `Ord` (total ordering returning `Less`, `Equal`, or `Greater`; by contrast, `partial_cmp` returns `Option` because comparing with `NaN` yields `None`). Try writing `HashMap<f64, Order>` or sorting an `f64` slice in Rust and the compiler stops you before a single broken search can ship—nudging you directly toward fixed-point integer cents and basis points, which implement `Eq + Ord + Hash` cleanly.

### 6.7.3 Conversions: `From/Into` and `TryFrom/TryInto`

C and C++ silently convert `int64_t` to `int32_t` or `double` to `int`, truncating upper bits without a murmur. Rust forbids all implicit numeric coercions (even widening `i32` to `i64` must be explicit) and organizes type conversions into two symmetric trait pairs:

Listing 6.15: Lossless infallible From/Into vs fallible range-checked TryFrom/TryInto.

```
use std::convert::{TryFrom, TryInto};

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct BasisPoints(u16);

// 1. Infallible lossless widening: u16 always fits inside i64!
//    Implementing From<BasisPoints> for i64 AUTOMATICALLY gives
//    BasisPoints::into(self) -> i64 via std's blanket impl!
impl From<BasisPoints> for i64 {
    fn from(bp: BasisPoints) -> Self {
        i64::from(bp.0)
    }
}

// 2. Fallible domain conversion: not every i64 is valid [0..=10_000] bps!
#[derive(Debug, PartialEq, Eq)]
pub struct InvalidBps(pub i64);
```

```rust

impl TryFrom<i64> for BasisPoints {
    type Error = InvalidBps;

    fn try_from(raw: i64) -> Result<Self, Self::Error> {
        let small = u16::try_from(raw).map_err(|_| InvalidBps(raw))?;
        if small <= 10_000 {
            Ok(BasisPoints(small))
        } else {
            Err(InvalidBps(raw))
        }
    }
}

pub fn conversion_demo() -> Result<i64, InvalidBps> {
    let bp = BasisPoints::try_from(250_i64)?;
    let widened: i64 = bp.into(); // lossless!
    let _also_works: BasisPoints = 75_i64.try_into()?;
    Ok(widened)
}
```

**Golden rule:** Always implement `From<A> for B` (or `TryFrom<A> for B`), never `Into` directly—because the standard library's blanket implementation automatically grants `Into<B> for A` (and `TryInto<B> for A`) for free, and Chapter 5's `?` operator uses `From::from` to convert inner error types automatically.

### 6.7.4 Pointer-Like Views and Cleanup: `AsRef`, `Borrow`, `Deref`, and `Drop`

Four final traits tie Rust's ownership model (Chapters 3–4) into everyday API ergonomics:

- **`AsRef<U>` (`fn as_ref(&self) -> &U`):** Cheap, zero-cost reference-to-reference view conversion. Both `String` and `&str` implement `AsRef<[u8]>` and `AsRef<Path>`, letting functions accept any path-like or byte-slice-like caller cleanly.
- **`Borrow<Borrowed>`:** Looks identical to `AsRef` in its method signature, with one strict semantic law: **the borrowed view must produce the exact same `Eq`, `Ord`, and `Hash` results as the owned value.** That law is why a `HashMap<String, Account>` lets you call `map.get("ACCT-01")` passing a stack `&str` without allocating a temporary heap `String` just to look up a key.
- **`Deref` (`type Target; fn deref(&self) -> &Self::Target`):** Powers the `*x` dereference operator on smart pointers (`Box<T>`, `Rc<T>`, `Arc<T>`, `MutexGuard<T>`) and owned buffers (`String -> str`, `Vec<T> -> [T]`). Whenever you pass `&my_string` to a function expecting `&str`, the compiler automatically applies `Deref` **coercion** (`&String -> &str`).
- **`Drop` (`fn drop(&mut self)`):** Deterministic RAII cleanup executed automatically when an owned value goes out of scope (closing sockets, unlocking a `MutexGuard`, rolling back an uncommitted `sqlx::Transaction`). Notice how `rustc` forbids implementing both `Copy` and `Drop` on the same type: if copying a value duplicated it via raw `memcpy`, both copies would run `Drop` at scope exit and double-free the resource.

Common trap

**Never abuse `Deref` to fake Java `extends` class inheritance.** Newcomers from OOP occasionally wrap `struct SavingsAccount { base: Account, rate_bps: u16 }` and implement `Deref<Target = Account>` so `savings.balance()` auto-forwards to `base.balance()`. Resist that temptation. `Deref` is exclusively for *smart pointers and buffer wrappers* where the wrapper conceptually *points to* the target (so `*wrapper` makes physical sense). Using `Deref` between distinct domain structs creates baffling method-resolution priorities, breaks generic trait bounds (`SavingsAccount` still does *not* implement traits implemented on `Account`), and confuses readers. Prefer explicit composition (`savings.account().balance()`) or shared domain traits.

Table 6.1: Summary of Rust's core standard traits compared with Java 6 and C++.

| Rust Trait | Derivable? | Contract & Contrast with Java 6 / C++ |
|---|---|---|
| `Debug` / `Display` | Yes / No | Developer {:?} dump vs explicit user-facing {}; replaces universal `Object.toString()`. |
| `Clone` / `Copy` | Yes / Yes | Explicit `.clone()` code vs implicit bitwise `memcpy` marker (`Copy: Clone`, mutually exclusive with `Drop`). |
| `PartialEq` / `Eq` | Yes / Yes | Partial vs reflexive equivalence; keeps `f64::NAN` out of `HashMap` keys at compile time. |
| `PartialOrd` / `Ord` | Yes / Yes | Partial vs total order; prevents broken `NaN` sorting and `BTreeMap` corruption. |
| `From<T>` / `TryFrom<T>` | No / No | Explicit lossless vs fallible conversions; replaces silent C/C++ narrowing casts. |
| `AsRef<T>` / `Borrow<T>` | No / No | Zero-cost borrowed views; `Borrow` additionally guarantees matching `Hash/Eq/Ord`. |
| `Deref` / `Drop` | No / No | Smart-pointer dereferencing (`&String -> &str`) and deterministic scope-exit RAII cleanup. |

## 6.8 Exercises

**Exercise 6.1** (Counting Stack, Heap, and Vtable Bytes)**.** Assume a 64-bit target (`usize` = 8 bytes). Given `struct FlatCentsFee { cents: i64 }` and `struct TieredBpsFee { threshold: i64, low_bps: i64, high_bps: i64 }`, both implementing `FeePolicy`:

1. What is the exact stack size (`size_of`) and total heap byte footprint of `Vec<FlatCentsFee>` holding 1,000 elements with capacity 1,000?

2. What is the exact stack size and minimum heap byte footprint of `Vec<Box<dyn FeePolicy>>` holding 500 `FlatCentsFee` items and 500 `TieredBpsFee` items with capacity 1,000? How many `.rodata` vtables exist in the binary for those two types?

3. Compare both answers against a Java 6 `ArrayList<FeePolicy>` backed by an `Object[1000]` array on a 64-bit JVM without compressed OOPs (16-byte object header, 8-byte references, 8-byte alignment).

**Exercise 6.2** (Diagnosing and Fixing `dyn`-Compatibility)**.** A teammate migrating from Java 6 writes the following trait for pluggable ledger serializers and attempts to store them in a `Vec<Box<dyn LedgerFormat>>`:

```rust
pub trait LedgerFormat {
    fn format_name() -> &'static str;
    fn clone_format(&self) -> Self;
    fn write_entry<W: std::io::Write>(
        &self,
        entry_id: u64,
        cents: i64,
        out: &mut W,
    ) -> std::io::Result<()>;
}
```

1. Explain from first principles (stack frame layout and `.rodata` vtable structure) why *all three* methods prevent `LedgerFormat` from being used as `dyn LedgerFormat`.
2. Refactor `LedgerFormat` so a `Vec` of `Box<dyn LedgerFormat>` compiles cleanly, supports cloning boxed formatters, exposes the format name on a trait object, *and* still offers a generic `write_to` helper when the concrete formatter type is statically known.

**Exercise 6.3** (Associated Types vs Generic Parameters in a Rate Engine)**.** You are designing two traits for a multi-currency settlement service:

- `QuoteSource`: given a base currency code and quote currency code, fetches the latest spot rate, returning an implementation-specific error type on failure (for example, `RedisError` for a cache source or `HttpError` for a live feed).
- `ApplyAdjustment<Adj>`: adjusts a `Money` value by either a `FlatDiscount`, a `BasisPointMarkup`, or a `TaxRule`—all supported on the *same* `Money` struct.

Write the idiomatic Rust trait definitions for `QuoteSource` and `ApplyAdjustment`, justifying where you chose an associated type versus a generic type parameter.

**Exercise 6.4** (De-Bloating Monomorphized APIs)**.** Consider a generic function `parse_swift<P>` bounded by `P: AsRef<Path>` and returning an `io::Result<Vec<i64>>`. Its body contains 180 lines of file reading, line splitting, and SWIFT tag parsing, and it is called across 12 modules with `&str`, `String`, `PathBuf`, `&Path`, and `Cow<'_, str>`.

1. How many copies of the 180-line parser body does `rustc` emit before Link-Time Optimization, and how can this hurt runtime throughput even if RAM is plentiful?
2. Rewrite `parse_swift` using the standard library's non-generic inner-function pattern so the 180-line parser is compiled *once* while preserving identical caller ergonomics.

**Exercise 6.5** (Total Ordering Without `NaN` Bugs)**.** Explain why `BTreeMap<f64, Vec<OrderId>>` fails to compile in Rust, and construct a small concrete sequence of three `BTreeMap` insertions and lookups that would break binary search if `f64::NAN` were permitted as a key. Then implement a validated wrapper struct `FiniteRate(f64)` whose constructor `try_new(x: f64) -> Option<Self>` rejects `NaN` (and normalizes `-0.0` to `+0.0`), manually implementing `PartialEq`, `Eq`, `PartialOrd`, and `Ord` safely.

# Part III

# Thinking Functionally and Writing Idiomatic Rust

# Chapter 7

# Thinking Functionally: From Imperative Loops through OCaml and Haskell to Rust

In Part II we opened the hood on Rust's memory and type machinery: single ownership and borrowing in Chapter 3, region lifetimes in Chapter 4, algebraic `struct` and `enum` layouts in Chapter 5, and Chapter 6's separation of flat data from behavior through traits such as `FeePolicy`, static monomorphization, and 16-byte `&dyn FeePolicy` fat pointers. You can now read a Rust type signature and draw the exact stack and heap bytes it governs.

Yet engineers arriving from Java 6 and C often hit a second wall when writing their first thousand lines of Rust. Their code compiles, yet every function starts with four `let mut` accumulators and boolean flags, every conditional mutates an outer variable across three branches, and every fallible lookup drowns in nested `match` statements. They are writing imperative Java 6 in Rust syntax—and the borrow checker pushes back because mutable state smeared across wide scopes creates overlapping borrows.

Here is the historical connection that explains idiomatic Rust: although Rust took its curly braces and flat memory model from C and C++, its control flow, algebraic types, pattern matching, traits, closures, and error pipelines descend directly from two functional languages—**OCaml** and **Haskell**. Read without the academic jargon, both languages teach a small set of practical engineering habits: compute by reducing expressions to values rather than mutating places step by step; push side effects to explicit boundaries; make absence and failure ordinary values; and compose small functions into pipelines. Studying OCaml and Haskell alongside Rust shows where those ideas came from *and* how ownership lets Rust compile them into zero-allocation, register-tight machine loops without a garbage collector.

---

## 7.1 Rewiring the `for`-Loop Brain: Statements vs. Expressions

Java 6 and C split their grammar into two unequal categories: *statements*, which execute in sequence to mutate memory places or jump elsewhere, and *expressions*, which evaluate to values (`a + b`, `x > 0`). Because `if`/`else`, `switch`, `for`, and `{...}` blocks are statements in Java 6, they cannot hand

back values directly. That grammar trains your fingers to compute by *place-mutation*: declare an empty variable in an outer scope, walk through branches that overwrite it, and hope every path left it consistent.

Consider summarizing settled USD postings in a ledger batch using Chapter 6's `FeePolicy` (`fee_in_cents`) to deduct fees on credits and assign a reserve status string. In Java 6 we reach for outer mutable slots:

```java
// Java 6: computation via step-by-step place-mutation
public static BatchSummary summarizeUsd(
        List<Posting> postings, FeePolicy feePolicy) {
    long total = 0L;
    for (int i = 0; i < postings.size(); i++) {
        Posting p = postings.get(i);
        if (p.isSettled() && "USD".equals(p.getCurrency())) {
            if (p.isCredit()) {
                long gross = p.getAmountCents();
                total += gross - feePolicy.feeInCents(gross);
            } else {
                total -= p.getAmountCents();
            }
        }
    }

    String status;
    if (total >= 10000000L) {
        status = "INSTITUTIONAL_SURPLUS";
    } else if (total >= 0L) {
        status = "STANDARD_POSITIVE";
    } else {
        status = "DEFICIT_WATCH";
    }
    return new BatchSummary(total, status);
}
```

Notice three maintenance hazards hidden inside this everyday method:

1. **Split declaration and initialization.** `String status;` creates a named slot before any value exists. If a teammate initializes `String status = null;` during a six-branch refactor to silence Java's definite-assignment check and forgets one `else if` arm, `null` escapes silently into production. In C or C++, reading an uninitialized `int64_t total;` on a missed branch is immediate undefined behavior.
2. **Temporal coupling and wide mutable scope.** Both `total` and `status` remain mutable until `return`. Reordering two statements below the loop can silently read half-updated state.
3. **Entangled loop mechanics.** One `for` body mixes index stepping, USD settlement filtering, credit/debit sign logic, fee deduction, and summation.

### 7.1.1 Everything is an Expression: Java 6 vs. OCaml vs. Rust

In OCaml and Rust, `if`/`else`, `match`, and `{...}` blocks are *expressions* that evaluate directly to typed values. Compare the same settled-USD calculation in OCaml and in Rust:

```
(* OCaml: match, if/else, and let-in blocks all evaluate to values *)
type direction = Credit | Debit
type posting = {
  settled : bool; usd : bool; dir : direction; cents : int64
}

let signed_net (fee_fn : int64 -> int64) (p : posting) : int64 =
  match p.dir with
  | Credit ->
      let fee = fee_fn p.cents in
      Int64.sub p.cents fee
  | Debit -> Int64.neg p.cents

let summarize_usd (posts : posting list) (fee_fn : int64 -> int64) =
  let total =
    posts
    |> List.filter (fun p -> p.settled && p.usd)
    |> List.map (signed_net fee_fn)
    |> List.fold_left Int64.add 0L
  in
  let status =
    if total >= 10000000L then "INSTITUTIONAL_SURPLUS"
    else if total >= 0L then "STANDARD_POSITIVE"
    else "DEFICIT_WATCH"
  in
  (total, status)
```

```
// Rust: blocks { ... tail_expr }, if, and match bind immutable values
#[derive(Clone, Copy, PartialEq, Eq)]
pub enum Direction { Credit, Debit }

#[derive(Clone, Copy, PartialEq, Eq)]
pub enum Currency { Usd, Eur }

pub struct Posting {
    pub settled: bool,
    pub currency: Currency,
    pub dir: Direction,
    pub amount_cents: i64,
}

pub trait FeePolicy {
    fn fee_in_cents(&self, amount_cents: i64) -> i64;
}

pub fn signed_net<P: FeePolicy>(policy: &P, p: &Posting) -> i64 {
    match p.dir {
        Direction::Credit => {
            let fee = policy.fee_in_cents(p.amount_cents);
            p.amount_cents - fee // tail expression: value of this block!
        }
        Direction::Debit => -p.amount_cents,
    }
```

```rust
}

pub fn summarize_usd<P: FeePolicy>(
    postings: &[Posting],
    policy: &P,
) -> (i64, &'static str) {
    let total: i64 = postings
        .iter()
        .filter(|p| p.settled && p.currency == Currency::Usd)
        .map(|p| signed_net(policy, p))
        .sum();

    let status = if total >= 10_000_000 {
        "INSTITUTIONAL_SURPLUS"
    } else if total >= 0 {
        "STANDARD_POSITIVE"
    } else {
        "DEFICIT_WATCH"
    };
    (total, status)
}
```

Every mutable outer slot disappeared. `total` and `status` are bound once, immutably, at the exact instant their values are ready. Inside the `Direction::Credit` arm, the block `{ let fee = ...; p.amount_cents - fee }` scopes `fee` locally, evaluates its tail expression without a semicolon, and hands the resulting `i64` out as the value of the arm. Delete the final `else` branch of `status`, and the program refuses to compile because every arm of an expression must produce the exact same type.

Focus

**Statements Mutate Places; Expressions Reduce to Values.** Whenever you catch yourself declaring `let mut result;` above an `if` or `match` in Rust, rewrite it as `let result = match ... { ... };`. You eliminate uninitialized gaps, shrink mutable borrow lifetimes to zero, and force the type checker to verify that every branch produces a compatible value.

### 7.1.2 Why ; Yields `()` and Why C/Java `void` Breaks Generics

Why does omitting the semicolon on `p.amount_cents - fee` return the `i64`, while adding `;` fails with "expected `i64`, found `()`"? Because Rust borrowed OCaml's **unit type** `()` instead of C and Java's `void`.

In C and Java 6, `void` is a syntactic keyword meaning "no value at all." You cannot declare `void x;` or plug `void` into a type parameter `<T>`:

- Java 6 had to invent *both* `Runnable` (`void run()`) and `Callable<V>` (`V call()`), plus the uninstantiable class `java.lang.Void` whose sole value is `null`—forcing every `Callable<Void>` implementation to end with `return null;`.
- C++ libraries must specialize `Promise<void>` separately from `Promise<T>` because `T val = f();` fails to parse when `T` is `void`.

**Definition 7.1** (The Zero-Byte Unit Type `()`)**.** The *unit type* (written `unit` with sole value `()` in

OCaml, and `()` for both the type and its sole value in Haskell and Rust) is the 0-element product type inhabited by exactly one value. Because distinguishing among 1 possibility requires $\log_2(1) = 0$ bits, `std::mem::size_of::<()>() == 0`.

Making `()` a genuine zero-sized type (*ZST*) unifies three rules cleanly:

1. Writing `fn record_metric(x: i64)` is exact shorthand for `fn record_metric(x: i64) -> ()`.
2. Placing `;` after an expression evaluates the expression, drops its result, and evaluates the block to `()`. Likewise, `if cond { work(); }` without `else` compiles *only* when its body yields `()`, matching the implicit `else { () }`.
3. Generic types work uniformly with zero overhead: `Result<(), LedgerError>` carries zero payload bytes in its `Ok(())` variant, and Rust's standard `HashSet<K>` is a thin wrapper around `HashMap<K, ()>` generating zero value-storage bytes or instructions.

## 7.2 OCaml's Direct DNA in Rust

History and connection

In the 1970s at Edinburgh, Robin Milner designed **ML** (*Meta Language*) to script proof tactics inside the LCF theorem prover, inventing polymorphic Hindley–Milner type inference and abstract module types so no buggy tactic could ever forge an invalid theorem of type `thm`. At INRIA in France, Xavier Leroy, Damien Doligez, Jérôme Vouillon, and colleagues evolved Milner's ML through Caml into **OCaml** (1996), pairing ML's expressive type system with a fast native-code compiler (`ocamlopt`) and a low-latency generational garbage collector.
When **Graydon Hoare** started Rust in 2006, he wrote the entire original Rust bootstrap compiler (`rustboot`, roughly 38,000 lines of code developed from 2006 to 2010 before `rustc` became self-hosting in 2011) *in OCaml*. That is why Rust's `let`, `fn`, `match`, `Option`/`Result`, `()` unit values, and tick-prefixed lifetimes (`'a`, echoing OCaml's polymorphic type variables `'a`) carry unmistakable ML DNA.

Let us put real, runnable OCaml side by side with Java 6 and Rust to master five ML ideas—and pinpoint exactly where Rust's lack of a garbage collector changes the underlying data structures.

### 7.2.1 Immutability by Default, Shadowing, and `ref` Cells vs. `let mut`

In OCaml, `let` bindings are permanently immutable. Multi-step pipelines use *lexical shadowing* to bind clean intermediate stages, while genuine mutation requires allocating an explicit mutable heap cell (`'a ref`) created with `ref`, read with `!`, and overwritten with `:=`:

```
(* OCaml: immutable shadowing vs explicit heap-allocated ref cell *)
let clamp_after_fee (raw_cents : int) (bps : int) : int =
  let cents = max 0 raw_cents in
  let fee   = (cents * bps) / 10000 in
  let cents = cents - fee in  (* shadows earlier cents; does not mutate! *)
  max 0 cents
```

```
7
8  let count_positive (xs : int list) : int =
9    let count = ref 0 in (* allocates 16B mutable cell on GC heap *)
10   List.iter (fun x -> if x > 0 then count := !count + 1) xs;
11   !count
```

**Under the hood**

**OCaml's 63-Bit Tagged Integers vs. Rust's Unboxed `let mut`.** Because OCaml compiles polymorphic functions once without monomorphization, its moving garbage collector must distinguish integers from pointers at runtime inside any 64-bit register. OCaml reserves bit 0 of every 64-bit word as a tag: heap pointers are 8-byte aligned (ending in `0`), while immediate `int`s, `bool`s, and `()` set bit 0 to `1` (`(n << 1) | 1`), giving 63-bit signed integers with zero boxing. (Full 64-bit `int64` values box on the heap unless locally unboxed by `ocamlopt`.) Rust has no GC and monomorphizes generics, so `let mut count: usize = 0;` mutates a raw 64-bit stack/register integer in place with zero heap allocation, using borrow checking (`&mut count`) rather than a heap `ref` box when a closure mutates it.

### 7.2.2 Algebraic Variants and Structural Pattern Matching

Compare an OCaml algebraic variant representing credits, debits, and inline-record reversals with its direct Rust `enum` mirror:

```
(* OCaml: algebraic variant + structural match with 'when' guards *)
type posting =
  | Credit of int64
  | Debit of int64
  | Reversal of { orig_id : int; reason : string }

let net_delta (frozen : bool) (p : posting) : (int64, string) result =
  match (frozen, p) with
  | (_, Credit c) when c > 0L -> Ok c
  | (true, Debit _) -> Error "account frozen"
  | (false, Debit c) when c > 0L -> Ok (Int64.neg c)
  | (false, Reversal { orig_id; reason }) when orig_id > 0 ->
      if String.length reason >= 3 then Ok 0L else Error "short reason"
  | _ -> Error "invalid posting"
```

```
// Rust: identical variant modeling and 2D tuple + guard matching
pub enum PostingOp {
    Credit(i64),
    Debit(i64),
    Reversal { orig_id: u32, reason: String },
}

pub fn net_delta(
    frozen: bool, p: &PostingOp,
) -> Result<i64, &'static str> {
    match (frozen, p) {
        (_, PostingOp::Credit(c)) if *c > 0 => Ok(*c),
        (true, PostingOp::Debit(_)) => Err("account frozen"),
```

```
        (false, PostingOp::Debit(c)) if *c > 0 => Ok(-*c),
        (false, PostingOp::Reversal { orig_id, reason })
            if *orig_id > 0 =>
        {
            if reason.len() >= 3 { Ok(0) } else { Err("short reason") }
        }
        _ => Err("invalid posting"),
    }
}
```

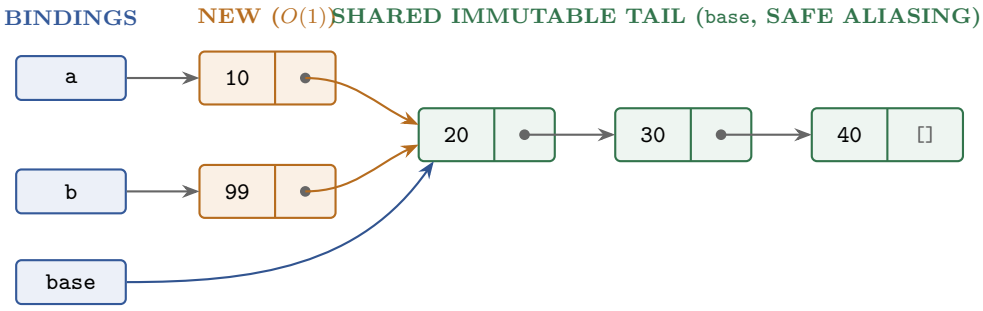
Instead of Java 6 `instanceof` ladders or sprawling Visitor boilerplates, both languages match the pair `(frozen, p)` as a flat 2D truth table and verify exhaustiveness at compile time.

### 7.2.3 Persistent Data Structures and Structural Sharing (`x :: xs`)

If OCaml lists are immutable, does prepending an element onto a 10,000-item list copy all 10,000 items? Never. An OCaml `'a list` is either empty (`[]`) or an immutable heap **cons cell** `x :: xs` storing head `x` and a pointer to tail `xs`:

```
(* OCaml: O(1) persistent updates share the exact same immutable tail *)
let base = [20; 30; 40] (* 20 :: 30 :: 40 :: [] -> 3 cons cells *)
let a    = 10 :: base   (* 1 new cons cell pointing to base *)
let b    = 99 :: base   (* 1 new cons cell ALSO pointing to base *)
```

As Figure 7.1 illustrates, only **five** cons cells exist in RAM. Because every cell is permanently immutable, `a` and `b` safely alias `base` without defensive `new ArrayList<>(base)` copies.




**GC Nursery vs. Rust `Vec<T>`:** Each OCaml cons cell occupies 24B (8B GC header + 8B head + 8B tail pointer). OCaml's bump-pointer minor nursery allocates `10 :: base` in ∼2 instructions and collects dead cells in bulk. Because Rust has **no GC**, sharing tails requires `Rc<Node<T>>` (adding 16B ref-count headers + pointer-chasing cache misses), so idiomatic Rust favors flat contiguous `Vec<T>` buffers and slices (`&[T]`) paired with lazy iterators.

Figure 7.1: Persistent structural sharing in OCaml: `a = 10 :: base` and `b = 99 :: base` allocate one cons cell each while sharing the immutable tail `[20; 30; 40]`.

### 7.2.4 Currying, Partial Application, Pipes (`|>`), and Tail Recursion

Every multi-argument OCaml function is **curried**: `apply_bps` below has type `int -> int -> int`, meaning `int -> (int -> int)`. Supplying just the first argument (`apply_bps 25`) returns a par-

tially applied closure `int -> int` ready to drop straight into a `|>` (`let (|>) x f = f x`) pipeline alongside `List.filter`, `List.map`, and `List.fold_left`:

```
(* OCaml: currying, partial application, |> pipe, and tail recursion *)
let apply_bps (bps : int) (cents : int) : int =
  cents - ((cents * bps) / 10000)

let net_after_25bps : int -> int = apply_bps 25

let settled_batch_total (amounts : int list) : int =
  amounts
  |> List.filter (fun c -> c > 0)
  |> List.map net_after_25bps
  |> List.fold_left ( + ) 0

(* Explicit-accumulator tail recursion compiles to a register jump *)
let sum_tail_recursive (xs : int list) : int =
  let rec loop acc remaining =
    match remaining with
    | [] -> acc
    | x :: rest -> loop (acc + x) rest (* tail call: reuses frame! *)
  in
  loop 0 xs
```

Look at `List.fold_left` and `loop`: instead of mutating an outer variable, the running state rides explicitly in `acc`. Because `loop (acc + x) rest` is the final action of the function (**tail position**), `ocamlopt`'s **Tail-Call Optimization (TCO)** updates the CPU registers holding `acc` and `remaining` and jumps back to the top of `loop`—consuming $O(1)$ stack space across ten million nodes. (Rust does not *guarantee* tail calls, partly because RAII `Drop` destructors must run after a callee returns, so a call that looks like it is in tail position often is not. Idiomatic Rust uses a loop or `Iterator::fold`, which compiles directly to a machine loop.)

### 7.2.5 Hindley–Milner Inference, `fn` Boundaries, and Modules

Write a polymorphic list transformer in OCaml with zero type annotations:

```
let rec map_postings xs f =
  match xs with
  | [] -> []
  | x :: rest -> (f x) :: (map_postings rest f)
(* Inferred: val map_postings : 'a list -> ('a -> 'b) -> 'b list *)
```

Using **Hindley–Milner type inference** (Algorithm W), OCaml assigns fresh type variables to `xs` and `f`, collects equations from `x ::  rest` and `f x`, and solves (*unifies*) them to deduce the most general type `'a list -> ('a -> 'b) -> 'b list`.

**Why it works**

**Why Rust Stops Hindley–Milner at `fn` Boundaries.** Inside a function body, Rust uses Hindley–Milner-style unification (`let mut v = Vec::new();` is inferred from a later `v.push(5_i64)`). Yet Rust mandates explicit parameter and return types on every `fn` header

for three engineering reasons:

1. **Strictly local error messages:** a mistake inside function $B$ always fails inside $B$ against $B$'s signature rather than silently changing $B$'s inferred type and triggering a baffling unification clash three modules away.
2. **SemVer contracts:** editing a function body cannot accidentally break callers' type or lifetime bounds.
3. **Tractable traits + parallel checking:** trait-based method lookup, lifetime subtyping, and deref coercions would make whole-program inference fragile and slow; explicit `fn` headers let `rustc` check each function body on its own.

Even in OCaml, production engineers seal module boundaries behind explicit signatures (`module type` / `sig ... end`), hiding internal representations just as Rust uses `mod` with private struct fields:

```
module type FEE_POLICY = sig
  type t
  val make_bps     : int -> t option
  val fee_in_cents : t -> int64 -> int64
end

module TieredBps : FEE_POLICY = struct
  type t = int (* hidden outside TieredBps! *)
  let make_bps b = if b >= 0 && b <= 10000 then Some b else None
  let fee_in_cents bps cents =
    Int64.div (Int64.mul cents (Int64.of_int bps)) 10000L
end
```

## 7.3 Haskell's Big Ideas Without the Ivory-Tower Fog

Released in 1990 by Simon Peyton Jones, Philip Wadler, Paul Hudak, John Hughes, and colleagues, **Haskell** pushed functional design to a clean extreme: pure functions everywhere, lazy evaluation by default, and ad-hoc overloading through typeclasses. Let us strip the academic mist off Haskell's four biggest ideas and see how each one maps into everyday Rust.

### 7.3.1 Referential Transparency vs. Rust's Controlled Local Mutation

**Definition 7.2** (Referential Transparency)**.** An expression $e$ is *referentially transparent* if replacing $e$ anywhere with its evaluated value leaves program behavior completely unchanged. A function is *pure* if its output depends solely on its explicit arguments and it produces zero observable side effects (no mutation of external state, no I/O).

Pure functions give you algebraic substitution, zero hidden temporal coupling, trivial unit tests, and race-free parallelism. Compare sorting and deduplicating ledger amounts in pure Haskell versus owned-value Rust:

```
-- Haskell: pure transformation allocating new immutable list nodes
```

```haskell
import Data.List (sort)

sortAndDedup :: [Int] -> [Int]
sortAndDedup xs = dedup (sort xs)
  where
    dedup [] = []
    dedup [y] = [y]
    dedup (y1 : y2 : rest)
      | y1 == y2  = dedup (y2 : rest)
      | otherwise = y1 : dedup (y2 : rest)
```

```rust
// Rust: in-place L1-cache mutation inside, 100-percent pure to callers!
pub fn sort_and_dedup(mut items: Vec<i64>) -> Vec<i64> {
    items.sort_unstable(); // mutates flat buffer in place, 0 heap allocs
    items.dedup();         // linear in-place compaction
    items
}
```

Consider `sort_and_dedup(mut items: Vec<i64>) -> Vec<i64>`. Inside its body, it mutates array slots in L1 cache with zero garbage-collector churn. Yet to *every caller in the program*, `sort_and_dedup` is **100 percent referentially transparent**. Because `items` is moved in by value (and even on borrowed helpers, `&mut` forbids any external alias from existing), no outside observer can witness the in-place swaps. This is the style PL engineers call *functional in the large, imperative in the small*.

### 7.3.2 Lazy Evaluation vs. Zero-Allocation Demand-Driven Iterators

Haskell evaluates expressions *lazily* (call-by-need): an unevaluated expression is packaged as a heap **thunk** `[code_ptr | env]` and forced only when downstream pattern matching demands it. That lets Haskell compose pipelines over infinite lists cleanly:

```haskell
-- Haskell: [1..] is infinite, yet 'take 5' pulls only until 5 evens arrive
firstFiveEvenSquares :: [Int]
firstFiveEvenSquares =
  take 5 (filter even (map (\x -> x * x) [1..]))
-- Evaluates to [4, 16, 36, 64, 100] and halts!
```

**Common trap**

**Heap Thunks and Haskell's `foldl` Space Leak.** Pervasive laziness has a sharp hardware cost: every suspended computation is a heap object, and lazy accumulators silently build long chains of thunks. In Haskell, `foldl (+) 0 [1..1000000]` does *not* add integers as it walks the list; it allocates one million nested heap thunks `(((0 + 1) + 2) + ...)`, blowing memory when forced at the end unless you switch to strict `Data.List.foldl'`.

Look at how Rust achieves the exact same pull-based infinite pipeline in a strict language with **zero heap thunks**:

```rust
// Rust: pull-based laziness encoded as flat stack state machines
pub fn first_five_even_squares() -> Vec<i64> {
    (1_i64..)                       // 8-byte RangeFrom<i64> on stack
```

```
        .map(|x| x * x)            // wraps RangeFrom in stack Map struct
        .filter(|x| (x & 1) == 0)  // wraps Map in stack Filter struct
        .take(5)                   // pulls .next() until 5 items match
        .collect()                 // [4, 16, 36, 64, 100]
}
```

Every expression in Rust is strict, so an `i64` is always a raw 8-byte integer, never a thunk. Laziness is opt-in through the `Iterator` trait, where each stage advances on stack registers one element at a time when `.next(&mut self)` is pulled.

### 7.3.3 Typeclasses (`Eq`, `Ord`, `Show`, `FeePolicy`) → Rust Traits

In 1989, Philip Wadler and Stephen Blott introduced Haskell **typeclasses** to support principled ad-hoc polymorphism (`Eq`, `Ord`, `Show`) without OOP class hierarchies. Put Haskell's `class`/`instance` right next to Chapter 6's Rust `trait`/`impl`:

```haskell
-- Haskell: typeclass + separate instance + constrained generic function
class FeePolicy a where
  feeInCents :: a -> Int -> Int

data FlatBps = FlatBps Int deriving (Eq, Ord, Show)

instance FeePolicy FlatBps where
  feeInCents (FlatBps bps) cents = div (cents * bps) 10000

netCredit :: FeePolicy p => p -> Int -> Int
netCredit policy cents = cents - feeInCents policy cents
```

```rust
// Rust: direct descendant -- traits + derive + monomorphized bounds
#[derive(PartialEq, Eq, PartialOrd, Ord, Debug)]
pub struct FlatBps(pub i64);

impl FeePolicy for FlatBps {
    fn fee_in_cents(&self, cents: i64) -> i64 {
        (cents * self.0) / 10_000
    }
}

pub fn net_credit<P: FeePolicy>(policy: &P, cents: i64) -> i64 {
    cents - policy.fee_in_cents(cents)
}
```

Rust traits are Wadler–Blott typeclasses compiled via C++-style monomorphization (or explicit `&dyn FeePolicy` fat pointers). Unlike Java 6 interfaces, which attach to heap objects via `this` headers, typeclasses and traits attach to *types* —allowing static factories like `Default::default() -> Self`, retroactive implementations on existing types, and global *coherence* (orphan rules).

### 7.3.4 Functors (`.map()`) and Monads (`.and_then()` and `?`)

Suppose validating a ledger debit runs four fallible steps in sequence: `parse_id -> find_account -> active_limit -> check_headroom`, followed by a pure cent-to-dollar scaling step. In Java 6 returning `null` on failure, four dependent steps create a deeply indented `if (x != null)` pyramid:

```
// Java 6: nested null-check pyramid
public static Long checkDebitDollars(
        LedgerBook book, String rawId, long debitCents) {
    Long id = parseId(rawId);
    if (id != null) {
        Account acct = book.findAccount(id);
        if (acct != null) {
            Long limit = activeLimit(acct);
            if (limit != null) {
                Long headroom = checkHeadroom(limit, debitCents);
                if (headroom != null) {
                    return headroom / 100L;
                }
            }
        }
    }
    return null;
}
```

Functional languages flatten this staircase with two exact abstractions:

1. **Functor (`fmap` in Haskell, `.map()` in OCaml/Rust):** lifts an *infallible* function $f : A \to B$ into a context $F\langle A\rangle \to F\langle B\rangle$ (`Maybe a`/`Option<A>`, `Either e a`/`Result<A, E>`, or `Iterator`), obeying identity (`x.map(|a| a) == x`) and composition (`x.map(f).map(g) == x.map(|a| g(f(a)))`—the mathematical license for *loop fusion*).
2. **Why `.map()` alone nests wrappers:** `find_account` has type `u64 -> Option<&Account>`. Calling `parse_id(raw).map(|id| find_account(book, id))` wraps the inner `Option` inside a second `Some`, yielding `Option<Option<&Account>>`.
3. **Monad (`>>=` "bind" in Haskell, `Option.bind` in OCaml, `.and_then()` in Rust):** takes $M\langle A\rangle$ and a context-producing step $A \to M\langle B\rangle$, runs the step if a value is present, and *flattens one layer* so the output stays a flat $M\langle B\rangle$.

Compare Haskell's `>>=` and `do`-notation directly beside Rust's `.and_then()` and `?` operator on the exact same four-step pipeline:

```
-- Haskell: monadic bind (>>=) and equivalent do-notation
checkDebitBind :: LedgerBook -> String -> Int -> Maybe Int
checkDebitBind book raw debit =
  parseId raw
    >>= (\acctId   -> findAccount book acctId)
    >>= (\acct     -> activeLimit acct)
    >>= (\limit    -> checkHeadroom limit debit)
    >>= (\headroom -> Just (div headroom 100))

checkDebitDo :: LedgerBook -> String -> Int -> Maybe Int
checkDebitDo book raw debit = do
```

```haskell
  acctId   <- parseId raw
  acct     <- findAccount book acctId
  limit    <- activeLimit acct
  headroom <- checkHeadroom limit debit
  Just (div headroom 100)
```

```rust
// Rust: .and_then() + .map() chain vs identical '?' operator
pub fn check_debit_chain(
    book: &LedgerBook, raw: &str, debit_cents: i64,
) -> Option<i64> {
    parse_id(raw)
        .and_then(|id| book.find_account(id))
        .and_then(|acct| active_limit(acct))
        .and_then(|limit| check_headroom(limit, debit_cents))
        .map(|headroom| headroom / 100)
}

pub fn check_debit(
    book: &LedgerBook, raw: &str, debit_cents: i64,
) -> Option<i64> {
    let id = parse_id(raw)?;
    let acct = book.find_account(id)?;
    let limit = active_limit(acct)?;
    let headroom = check_headroom(limit, debit_cents)?;
    Some(headroom / 100)
}
```

Look at `checkDebitDo` and `check_debit`: a Monad is simply a **programmable semicolon**. Between each line, `<-` in Haskell and `?` in Rust inspect the `Maybe`/`Option` (or `Either`/`Result`) tag, unwrapping `Some`/`Ok` values along the top rail and diverting `None`/`Err` onto a short-circuit bypass rail (Figure 7.2).

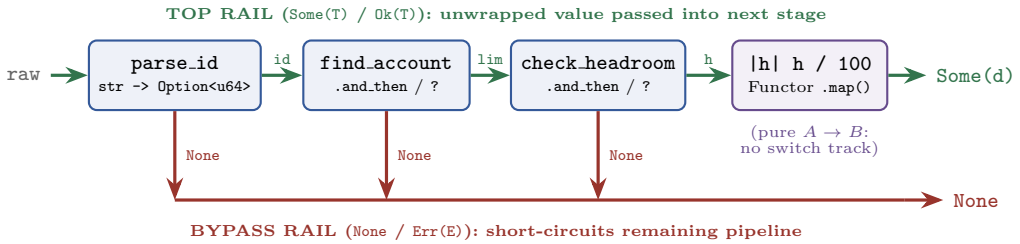



Figure 7.2: Railway-oriented pipeline: monadic bind (`.and_then` / `?`) splices two-track switches without nesting `Option<Option<T>>`, whereas functor `.map` lifts a single-track function $A \to B$ onto the top rail.

## 7.4 Zero-Cost Closures in Rust: Anonymous Structs and `Fn`, `FnMut`, `FnOnce`

Every higher-order pipeline we wrote in OCaml and Haskell passed closures—anonymous functions that capture variables from their enclosing scope (`fun p -> p.cents >= min_cents`). Before we

see how Rust compiles iterator chains into single machine loops, we have to answer a fundamental hardware question: *where do a closure's captured variables live in memory, and how does the CPU jump to its code?*

In C, a plain function pointer `bool (*pred)(const Posting*)` is just an 8-byte instruction address. It has nowhere to store a captured threshold `min_cents`, which is why C APIs like `qsort_r` or `pthread_create` force you to pack state into a separate struct and thread an untyped `void *ctx` pointer by hand.

In Java 6 (anonymous inner classes like `new Predicate<Posting>() { ... }` capturing `final long minCents`) and in OCaml and Haskell, the compiler packs captured variables into a **heap-allocated closure record**: a GC header, a function or vtable pointer, and the captured fields. Passing a closure passes a pointer to that heap block, and invoking it performs an *indirect call* through the code pointer—which allocates on the heap and blocks the compiler from inlining tiny arithmetic bodies unless a JIT compiler later devirtualizes the call site.

Rust borrows C++11's unboxed lambda insight and strengthens it with ownership: in Rust, **every closure expression generates its own unique, anonymous compiler `struct` type** that stores its captured variables inline on the stack with zero heap allocation and dispatches statically by default.

### 7.4.1 Exact Compiler Desugaring of Three Ledger Closures

To remove all mystery from Rust closures, let us trace three closures in our financial ledger—one reading a threshold, one counting rejected items, and one flushing an owned audit batch—and write down the exact `struct` and method signature `rustc` synthesizes for each:

```
// Three Rust closures with three distinct capture & call modes:
#[derive(Clone)]
pub struct AuditEntry { pub post_id: u64, pub note: String }

pub fn demo_three_closures(
    min_cents: i64,
    mut rejected_count: usize,
    staged_audit: Vec<AuditEntry>,
) {
    // 1. Borrows &min_cents immutably -> implements Fn, FnMut, FnOnce
    let above_floor = |cents: i64| cents >= min_cents;

    // 2. Mutates &mut rejected_count -> implements FnMut, FnOnce (NOT Fn)
    let mut track_rejects = |cents: i64| -> bool {
        if cents < min_cents {
            rejected_count += 1;
            false
        } else {
            true
        }
    };

    // 3. Moves staged_audit out on call -> implements FnOnce ONLY
    let flush_audit = || -> usize {
        let consumed: Vec<AuditEntry> = staged_audit; // moves Vec out!
        consumed.len()
```

```
    };

    let _ = above_floor(500);
    let _ = track_rejects(100);
    let _ = flush_audit();
}
```

Under the hood, `rustc` inspects how each closure body uses each outer variable (choosing the least restrictive capture mode: shared borrow `&T` first, mutable borrow `&mut T` second, move `T` third) and synthesizes three ordinary flat structs right in `demo_three_closures`'s stack frame:

```
// Conceptual desugaring generated by rustc (simplified trait signatures):
pub trait MyFnOnce<Arg> {
    type Output;
    fn call_once(self, arg: Arg) -> Self::Output;
}
pub trait MyFnMut<Arg>: MyFnOnce<Arg> {
    fn call_mut(&mut self, arg: Arg) -> Self::Output;
}
pub trait MyFn<Arg>: MyFnMut<Arg> {
    fn call(&self, arg: Arg) -> Self::Output;
}

// 1. Closure 'above_floor': 8 bytes on stack (1 shared reference)
struct __ClosureEnv1<'a> { min_cents: &'a i64 }

impl<'a> MyFn<i64> for __ClosureEnv1<'a> {
    fn call(&self, cents: i64) -> bool { cents >= *self.min_cents }
}
// (Plus MyFnMut and MyFnOnce impls delegating to self.call(cents)!)

// 2. Closure 'track_rejects': 16 bytes on stack (&i64 + &mut usize)
struct __ClosureEnv2<'a, 'b> {
    min_cents: &'a i64,
    rejected_count: &'b mut usize,
}

impl<'a, 'b> MyFnMut<i64> for __ClosureEnv2<'a, 'b> {
    fn call_mut(&mut self, cents: i64) -> bool {
        if cents < *self.min_cents {
            *self.rejected_count += 1;
            false
        } else {
            true
        }
    }
}
// Not MyFn(&self): mutating *self.rejected_count requires &mut self!

// 3. Closure 'flush_audit': 24 bytes on stack (ptr + len + cap of Vec)
struct __ClosureEnv3 { staged_audit: Vec<AuditEntry> }

impl MyFnOnce<()> for __ClosureEnv3 {
    type Output = usize;
```

```
    fn call_once(self, _arg: ()) -> usize {
        let consumed: Vec<AuditEntry> = self.staged_audit; // moves Vec!
        consumed.len()
    }
}
// Not MyFnMut(&mut self): a 2nd call would read a moved-from Vec!
```

Look at what happens to both memory and instruction dispatch:

- **Zero heap allocation.** `__ClosureEnv2` occupies 16 stack bytes. Capture-free closures (`|x: i64| x * 2`) occupy **0 bytes** and coerce to bare C-style function pointers `fn(i64) -> i64`.
- **Static dispatch that LLVM can inline.** Passing `track_rejects` to a generic `retain` bounded by `F: FnMut(i64) -> bool` monomorphizes it for `F = __ClosureEnv2`; LLVM sees the exact call target statically and inlines the body straight into the loop in optimized builds (while `&dyn Fn(i64) -> bool` or `Box<dyn Fn(i64) -> bool>` remains available when runtime heterogeneity is needed).

### 7.4.2 Why the Hierarchy Runs `Fn:` `FnMut:` `FnOnce`, and the `move` Trap

At first glance, the subtrait bound `Fn:` `FnMut:` `FnOnce` looks backward to many engineers: "Wait—doesn't `Fn` allow *more* ways to call a closure than `FnOnce`? Why is `Fn` a subtrait of `FnOnce`?"

Read the hierarchy as **callee requirements vs. closure capabilities** (Figure 7.3):

1. **`FnOnce(self)` is the weakest demand a function can place on a callback:** "Give me something I can call *at most once* by consuming it" (used by `Option::unwrap_or_else` and `std::thread::spawn`). *Every* closure in Rust satisfies `FnOnce`. Why? Because if you hand `unwrap_or_else` a pure `Fn(&self)` closure like `above_floor`, `unwrap_or_else` takes ownership of `self`, borrows `&self` once to run `Fn::call`, and drops it. Thus the set of `Fn` closures is a *subset* of `FnOnce` closures (`Fn` ⊂ `FnMut` ⊂ `FnOnce`).
2. **`FnMut(&mut self)` asks for sequential repeatability:** "I may call your closure zero, one, or many times in sequence, giving it unique `&mut self` access each time" (used by `Iterator::map`, `Iterator::filter`, and `Vec::retain`). Both `FnMut` and `Fn` closures qualify; consuming `FnOnce`-only closures are rejected.
3. **`Fn(&self)` is the strictest demand:** "I may call your closure many times through shared `&self` borrows, possibly from several threads at once when the closure is also `Sync`" (used by `rayon`'s parallel `par_iter().map(...)` across CPU cores). Only closures that never mutate or consume captures qualify.

**Common trap**

**The Classic `move |...|` Myth: Capture Mode ≠ Call Trait.** One of the most persistent misconceptions in Rust is believing that writing `move |x| ...` turns a closure into an `FnOnce`-only closure. **False.**

- The `move` keyword controls *how captured variables enter the closure struct* when the closure is constructed (moving ownership into the struct's fields instead of storing `&` borrows, so the closure can outlive the current stack frame).

- `Fn`, `FnMut`, and `FnOnce` are determined solely by *what the closure body does to its fields when called.*

Look at this three-line proof:

```
let currency = String::from("USD");
// 'currency' is MOVED into the closure struct, yet body only reads &str!
let is_ccy = move |code: &str| code == currency.as_str();
assert!(is_ccy("USD") && !is_ccy("EUR")); // implements Fn -- call 1000x!
```

Here `is_ccy` owns a 24-byte `String` field inside its struct (so it is `'static` and safe to send to another thread), yet calling `is_ccy("USD")` only borrows `&self.currency`. Therefore `is_ccy` implements `Fn`, `FnMut`, *and* `FnOnce`.

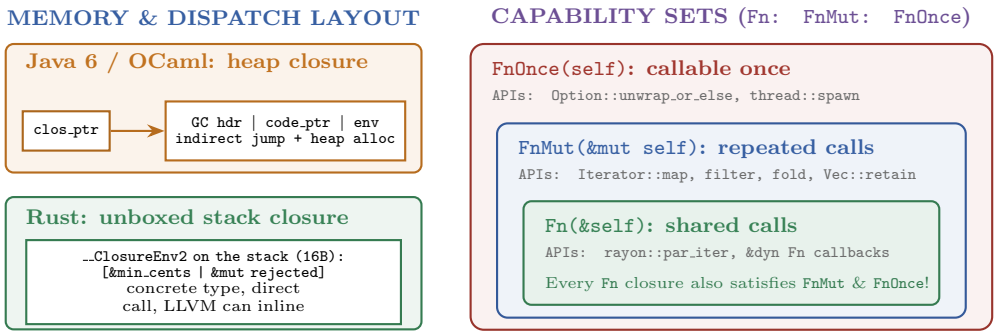



Figure 7.3: Left: Heap-allocated closure indirection in Java 6/OCaml versus Rust's flat stack-allocated closure struct. Right: Nested closure trait sets mapping receiver borrows (`self`, `&mut self`, `&self`) to standard combinators.

## 7.5 Zero-Cost Iterator Pipelines, `Result` Transposition, and Loop Fusion

Now put stack-allocated closures together with pull-based iterators, and look at what happens when Rust compiles a multi-stage functional pipeline.

In strict OCaml (`List.filter` followed by `List.map`) or JavaScript (`arr.filter(...).map(...)`), each stage runs *eagerly* from start to finish, allocating a brand-new heap list or array of intermediate results before the next stage even begins. On a batch of 100,000 ledger postings, chaining two `filter`s and two `map`s allocates four intermediate heap collections and traverses RAM five times.

In Rust, the entire iterator ecosystem rests on one tiny trait with one required state-machine method:

```
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // 70+ default combinator methods (map, filter, fold, collect, ...)
}
```

Trace every type and byte when we run our settled-USD batch fold in Rust:

```rust
pub fn net_settled_usd<P: FeePolicy>(
    postings: &[Posting], policy: &P,
) -> i64 {
    postings
        .iter()
        .filter(|p| p.settled && p.currency == Currency::Usd)
        .map(|p| signed_net(policy, p))
        .fold(0_i64, |acc, delta| acc + delta)
}
```

What sits in memory right before `.fold(...)` runs?

1. `postings.iter()` creates `std::slice::Iter<'_, Posting>`—just two raw pointers (`ptr` and `end`, 16 bytes on the stack).
2. `.filter(...)` wraps that `Iter` plus the 0-byte stateless predicate closure inside a stack struct `Filter<Iter<'_, Posting>, Pred>` (still 16 bytes).
3. `.map(...)` wraps `Filter` plus the 8-byte closure struct holding `&policy` inside `Map<Filter<Iter<'_, Posting>, Pred>, Mapper>` (24 bytes flat on the stack, **zero heap allocations**).
4. Finally, `fold` runs `while let Some(d) = self.next() { acc += d; }`. Because every adapter and closure type is statically known, LLVM inlines `Map::next`, `Filter::next`, and `Iter::next` into **one fused machine loop** (Figure 7.4)—often faster than `for i in 0..postings.len()` because advancing `ptr` up to `end` eliminates per-step array bounds checks.

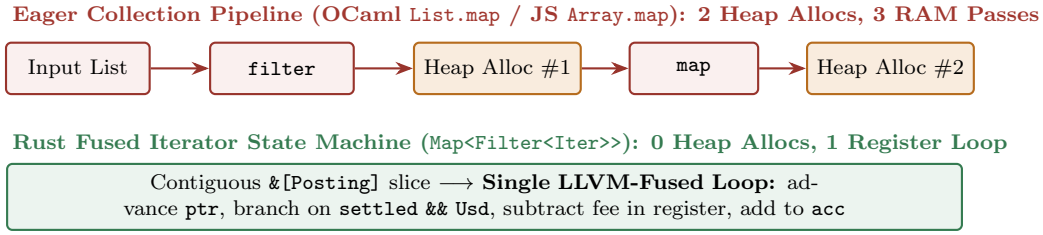



Figure 7.4: Eager pipelines materialize a full heap collection between every stage; Rust's pull-based adapter structs inline and fuse into a single allocation-free pass over flat memory.

### 7.5.1 Ownership Entry Points, Combinators, and `Result` Transposition

To read and write idiomatic Rust pipelines effortlessly, keep three groups of tools at your fingertips:

1. **Three ownership entry points (mirroring `&T`, `&mut T`, `T`):**
   - `xs.iter()` yields shared references `&T`, leaving `xs` intact afterwards.
   - `xs.iter_mut()` yields exclusive references `&mut T` to update elements in place.
   - `xs.into_iter()` consumes `xs` and moves owned `T` values out one by one.
2. **Work-saving structural adapters:**
   - `filter_map(|x| ... -> Option<B>)` fuses filtering and transforming into one pattern match—never write `.filter(|x| x.is_some()).map(|x| x.unwrap())`.

- `flat_map(|entry| entry.postings.iter())` flattens nested slices (one journal entry → its double-entry legs) without allocating intermediate `Vec`s.
- `enumerate()`, `zip(other)`, `chunks(64)`, and `windows(2)` eliminate manual index arithmetic (for instance, `balances.windows(2).map(|w| w[1] - w[0])` computes daily balance deltas with zero risk of `i - 1` underflow).

3. **Short-circuiting fallible iteration (`try_fold` and `collect::<Result<Vec<T>, E>>()`):** What if parsing or arithmetic inside a pipeline can fail with `Result<T, E>`? Rust's `From Iterator` trait implements a clean **type transposition**: an `Iterator<Item = Result<T, E>>` can be collected directly into a `Result<Vec<T>, E>`.

```rust
// Short-circuiting Result transposition + O(1)-space checked try_fold
pub fn parse_and_checked_sum(
    raw_lines: &[&str],
) -> Result<i64, &'static str> {
    // 1. Transposes Iterator<Item = Result<i64, E>> -> Result<Vec<i64>, E>
    //    Stops pulling IMMEDIATELY on the first Err!
    let amounts: Vec<i64> = raw_lines
        .iter()
        .map(|s| s.trim().parse::<i64>().map_err(|_| "bad integer"))
        .collect::<Result<Vec<i64>, &'static str>>()?;

    // 2. try_fold threads a fallible accumulator. On its own (fed by the
    //    parsing map directly) it needs no Vec at all -- see Exercise 7.1.
    amounts.iter().try_fold(0_i64, |acc, &x| {
        acc.checked_add(x).ok_or("i64 balance overflow")
    })
}
```

Look at `.collect::<Result<Vec<i64>, &'static str>>()?`. If line 3 of 10,000 lines contains `"abc"`, `collect` sees `Err("bad integer")`, halts the iterator immediately without parsing lines 4 through 10,000, drops the partially populated `Vec`, and returns `Err`. And when you only need a scalar aggregate rather than a `Vec`, `try_fold` threads a fallible accumulator in $O(1)$ stack space with the exact same instant short-circuiting.

### 7.5.2 Engineering Judgment: When a Plain `for` Loop Wins

Thinking functionally does *not* mean outlawing `for` loops or playing combinator golf. Remember that `for x in xs` in Rust already desugars to `IntoIterator::into_iter(xs)`. Experienced Rust engineers prefer a plain `for` loop over a contorted iterator chain in three clear situations:

1. **Early function returns or multi-level loop exits:** if finding a fraud flag inside a loop should immediately `return Ok(Decision::Freeze)` from the enclosing function while updating a side log, `for` + `return` is far clearer than encoding exits inside `ControlFlow`.
2. **Coupled multi-variable state machines:** when each step conditionally mutates three hash maps and a sliding watermark, threading a 4-tuple `(map1, map2, map3, mark)` through `.fold(...)` hurts readability. Encapsulate that local mutation inside a pure outer function (just like `sort_and_dedup`) and use a clean `for` loop inside.
3. **Asynchronous `.await` points:** synchronous closures passed to `Iterator::map` cannot `.await`

an SQLx query directly, whereas `for cmd in batch { execute(&pool, cmd).await?; }` interacts naturally with Chapter 12's async state machines and the `?` operator.

## 7.6 Exercises

**Exercise 7.1** (Refactoring Imperative Java 6 into OCaml and Zero-Allocation Rust)**.** Consider a Java 6 method scanning raw cent strings: it skips blank strings, parses integers, ignores non-positive numbers, deducts a 20-basis-point fee (`c - (c * 20) / 10000`), and returns `null` if any non-blank string fails to parse or if the running 64-bit sum overflows.

1. Write this pipeline in idiomatic **OCaml** using `int64`, `List.filter`, and a recursive or `fold_left` short-circuiting pass returning `int64 option`.
2. Write the pipeline in **Rust** with *zero* `Vec` allocations by combining `.iter()`, `.filter(...)` on trimmed slices, and `.try_fold(0_i64, ...)` returning `Option<i64>`. Verify that a malformed string at index 0 never parses index 1.

**Exercise 7.2** (Counting Cons Cells and Pointer Overhead: OCaml vs. Rust)**.** Trace the following OCaml definitions on a 64-bit machine where each cons cell occupies 24 bytes (8B GC header + 8B head + 8B tail pointer):

```
let s0 = [100; 200; 300; 400]
let s1 = 50 :: s0
let s2 = 75 :: (List.tl s0)
```

1. Draw the memory graph for `s0`, `s1`, and `s2`. How many cons cells and total heap bytes exist across all three lists combined?
2. Suppose you model this exact shared-tail graph in safe Rust without a GC using `enum List { Nil, Cons(i64, Rc<List>) }`. Given that `Rc<T>`'s heap allocation stores `(strong_count: usize, weak_count: usize, payload: T)`, calculate the exact byte size of one `Rc<List>` node (accounting for `List`'s tag + alignment) and compare cache-line locality against a flat `Vec<i64>`.

**Exercise 7.3** (Manual Rust Closure Desugaring and Trait Classification)**.** For each Rust closure below (assuming 64-bit Linux where `&T` is 8B, `String` and `Vec<T>` are 24B), write down (i) the exact fields of the compiler-generated `struct __ClosureEnv`, (ii) its stack byte size, and (iii) which subset of {`Fn`, `FnMut`, `FnOnce`} it implements:

```
let rate: i64 = 35;
let c1 = |amt: i64| (amt * rate) / 10_000;

let allow: Vec<u64> = vec![1, 2, 9];
let c2 = move |id: u64| allow.contains(&id);

let mut max_seen: i64 = 0;
let c3 = |x: i64| { if x > max_seen { max_seen = x; } };

let logs: Vec<String> = vec![];
let c4 = || logs.into_iter().count();
```

Explain specifically why `c2` has `move` yet differs in trait capability from `c4`.

**Exercise 7.4** (Custom Zero-Allocation Lazy `Iterator`)**.** Implement `Iterator<Item = i64>` for the following lazy adapter over any signed-delta iterator `I: Iterator<Item = i64>`:

```
pub struct RunningBalances<I> {
    pub iter: I, pub balance: i64, pub tripped: bool,
}
```

On each `next` call, add the next delta to `balance`; if `balance < 0` (an overdraft), set `tripped = true` and return `None` on that step and all later calls; otherwise yield `Some(balance)`. Show that chaining this adapter with `.take(3)` allocates zero heap bytes.

**Exercise 7.5** (Proving the Three Monad Laws on `Option::and_then`)**.** For any pure Rust functions `f: fn(i64) -> Option<i64>` and `g: fn(i64) -> Option<i64>`, state and prove by exhaustive case analysis on `m = None` and `m = Some(v)` that `Option` obeys all three monad laws:

1. **Left identity:** `Some(x).and_then(f) == f(x)`.
2. **Right identity:** `m.and_then(Some) == m`.
3. **Associativity:** `m.and_then(f).and_then(g) == m.and_then(|x| f(x).and_then(g))`.

Finally, give a concrete example with two impure closures `h1` and `h2` that push a label into a shared `RefCell<Vec<String>>` log. Run them over a three-element `Vec` first as two eager passes (map with `h1` and `collect`, then map with `h2` and `collect`) and then as one fused lazy chain, and show that the log records the side effects in a different order. This is why rewriting pipelines freely (loop fusion) is only safe for pure stages.

# Chapter 8

# How Idiomatic Rust Is Actually Written: Architecture, Patterns, and Anti-Patterns

Chapters 3 through 7 were about single functions: what each byte on the stack and heap is doing, how borrows and lifetimes are checked, how enums, traits, and iterators compile. This chapter is about the next problem: putting a few thousand lines together.

When you build a five-thousand-line service of your own, a different set of errors appears. You try to wire up a hierarchical chart of accounts where each child account points back to its parent and each parent holds a list of children, and the borrow checker blocks every edge. You reach for `Rc<RefCell<Account>>` to make the compiler stop complaining, only to watch a subtle re-entrant call panic at runtime. You write a `SettlementEngine` struct with ten fields, call `self.apply_posting(&self.fee_schedule)` inside a `&mut self` method, and hit compiler error `E0502`: cannot borrow `self.fee_schedule` as immutable because it is also borrowed as mutable. You pass `&String` and `&Vec<Posting>` across helper functions, sprinkle defensive `if amount <= 0` checks at the top of every method, and end up with more checks than logic.

**Intuition**

Fighting the borrow checker is almost never a sign that you forgot a lifetime annotation. Nine times out of ten, it is an architectural diagnostic: you are sketching a *Java object graph* — cyclic pointers, shared mutable state, runtime boolean flags, and unvalidated primitive arguments — and asking Rust to certify that no two pointers alias mutably. Idiomatic Rust does not patch Java architectures with extra lifetime syntax or `RefCell` wrappers. It reshapes data ownership, state transitions, and module boundaries so that the hardware layout is flat and invalid states cannot even be written down.

This chapter answers the question at the heart of Part III: *how do experienced systems engineers actually see a problem through the Rust lens, and how should production Rust be structured?* Every section contrasts a familiar instinct from Java 6 or C++ against the idiomatic Rust design that replaces it, anchored in the financial ledger, payment routing, and settlement domain we will take all the way to a live Axum, SQLx, and PostgreSQL backend in Part V.

## 8.1 Curing "Java-in-Rust Syndrome": Data-Oriented Arenas and Typed Index Handles

Consider a core piece of any corporate treasury or double-entry accounting system: a *hierarchical chart of accounts*. A multinational corporation holds a root treasury account, regional sub-accounts under that root, and operating accounts under each region. Whenever an operating account settles a wire transfer, two things happen: the sender and receiver operating balances change, and each ancestor account along the tree updates its rolled-up exposure total so risk limits can be checked in $O(1)$ time.

In Java 6, every developer draws and codes this the exact same way without thinking twice:

```
// Java 6: bidirectional heap pointers everywhere
public class AccountNode {
    private final String code;
    private long balanceCents;
    private long subtreeExposureCents;
    private AccountNode parent; // null for root
    private final List<AccountNode> children =
        new ArrayList<AccountNode>();

    public void addChild(AccountNode child) {
        child.parent = this;
        this.children.add(child);
    }
}
```

In Java, object identity and object access are fused into a single concept: a garbage-collected heap pointer. Every `AccountNode` floats independently on the heap, `parent` points upward, `children` points downward, an order-book map holds another reference from the side, and an audit listener holds a fourth. Because the JVM makes no compile-time promises about aliasing, any method can mutate any reachable node at any instant.

Here is what happens when an engineer translates `AccountNode` line for line into Rust.

### 8.1.1 Why Direct Translation Produces `Rc<RefCell<T>>` Soup

The first attempt — storing `parent: Option<&'a mut AccountNode<'a>>` alongside `children: Vec<AccountNode<'a>>` — is rejected immediately by the compiler. If the parent owns the child inside `children`, and the child simultaneously holds a mutable reference `&mut` back to its parent, you have two active paths capable of mutating the parent at the same time: the owner path from above and the `parent` pointer from below. That directly violates Chapter 3's *Aliasing XOR Mutability* law. In addition, moving the parent `Vec` during reallocation would invalidate every child's raw address.

Frustrated, the newcomer discovers Chapter 3's escape hatches for shared ownership and interior mutability — `Rc<T>`, `Weak<T>`, and `RefCell<T>` — and wraps every node in sight:

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

// Anti-pattern: recreating a Java object graph via Rc<RefCell<...>>
```

```
type SharedAccount = Rc<RefCell<BadAccountNode>>;

struct BadAccountNode {
    code: String,
    balance_cents: i64,
    subtree_exposure_cents: i64,
    parent: Option<Weak<RefCell<BadAccountNode>>>,
    children: Vec<SharedAccount>,
}
```

It compiles. But wrapping a domain graph in `Rc<RefCell<T>>` forfeits almost every advantage you came to Rust to get, while introducing failure modes Java 6 never had:

1. **Heap fragmentation and CPU cache misses.** Every `Rc::new(RefCell::new(...))` performs an individual allocator request. Inside each heap block sits a 16-byte reference-count header (`strong: usize`, `weak: usize`) plus an 8-byte `BorrowFlag` (`isize`) before your fields even begin. Walking 10,000 accounts chases arbitrary 64-bit addresses scattered across megabytes of virtual memory, stalling the CPU pipeline for $\sim$ 100–200 cycles on L3/DRAM misses instead of streaming sequential 64-byte cache lines.
2. **Accidental reference-cycle memory leaks.** If a developer later adds a cross-reference field `counterparty: Option<SharedAccount>` using strong `Rc` instead of `Weak`, two accounts pointing at each other keep their strong counts $\geq 1$ forever. Rust has no tracing garbage collector; those nodes and everything they own leak silently for the lifetime of the process.
3. **Thread hostility.** `Rc` and `RefCell` use non-atomic integer increments, so the compiler strips `Send` and `Sync` from the entire graph. You cannot pass this chart of accounts into a multi-threaded Tokio HTTP server in Part V without rewriting every node to `Arc<Mutex<T>>` or `Arc<RwLock<T>>`, turning every field read into an atomic lock acquisition and inviting deadlocks when two transfers lock nodes in opposite orders.
4. **Delayed runtime `BorrowMutError` panics.** `RefCell` does not remove the aliasing rule — it merely moves enforcement from compile time to runtime.

**Common trap**

Here is how `Rc<RefCell<T>>` panics at runtime in an ordinary helper call:

```
fn debit_and_recompute(acct: &SharedAccount, cents: i64) {
    // 1. Acquire exclusive runtime borrow (borrow_flag becomes -1)
    let mut node = acct.borrow_mut();
    node.balance_cents -= cents;

    // 2. Ask parent to recompute exposure across all its children
    if let Some(ref weak_parent) = node.parent {
        if let Some(parent_rc) = weak_parent.upgrade() {
            recompute_parent(&parent_rc); // <-- CRASH HIDING HERE!
        }
    }
}

fn recompute_parent(parent: &SharedAccount) {
    let mut p = parent.borrow_mut();
```

```
    p.subtree_exposure_cents = p.children
        .iter()
        // One of p.children IS 'acct', still mutably borrowed above!
        // child.borrow() panics: "RefCell already mutably borrowed"
        .map(|child| child.borrow().balance_cents.abs())
        .sum();
}
```

The compiler approves both functions without a single warning. Everything passes unit tests on root accounts that have `parent: None`. Then, in production, a transfer hits a child account: `debit_and_recompute` holds `node = acct.borrow_mut()` on the stack while `recompute_parent` iterates over `p.children` and calls `child.borrow()` on that exact same child. The thread immediately panics with `BorrowMutError`.

### 8.1.2 The Rust Architectural Shift: Separate Identity from Ownership

Ask what `parent` and `children` in our chart of accounts actually mean. Does a child account *own the lifetime of* its parent account's memory? No. In a database, the `accounts` table owns all rows, and `parent_account_id` is simply an integer foreign key naming another row in that same table.

Idiomatic Rust applies that exact relational insight directly to in-memory data structures:

**Focus**

**Separate Identity from Ownership.** Whenever entities form graphs, trees with parent pointers, doubly linked queues, or cross-referenced registries, give **one container** (an *arena*, backed by a contiguous `Vec`) exclusive ownership of all nodes, and represent edges between nodes as small, `Copy`, strongly typed **index handles** rather than pointers or reference-counted wrappers.

Every problem from the `Rc<RefCell<T>>` list goes away with this shift:

- **One clear owner, automatic cleanup.** When the `AccountArena` struct goes out of scope, dropping its single `Vec` frees every node at once. Cycles among integer handles are just numbers; they can never leak memory.
- **Hardware sympathy.** Every `AccountNode` sits back to back in one contiguous slab of RAM. Scanning balances streams straight through the CPU hardware prefetcher, and handles shrink from 8-byte aligned pointers (plus 24 bytes of `Rc`/`RefCell` metadata) down to compact 4-byte or 8-byte values.
- **Automatic `Send` and `Sync`.** Plain structs holding integers, `Vec`s, and `String`s are automatically `Send + Sync` with zero atomic refcount overhead.
- **Compile-time borrow checking restored.** Instead of hidden per-node `RefCell` flags, borrowing happens at the arena boundary: `arena.get(key)` takes `&self` (allowing unlimited simultaneous readers), and `arena.get_mut(key)` takes `&mut self` (making mutation explicit in the function signature).

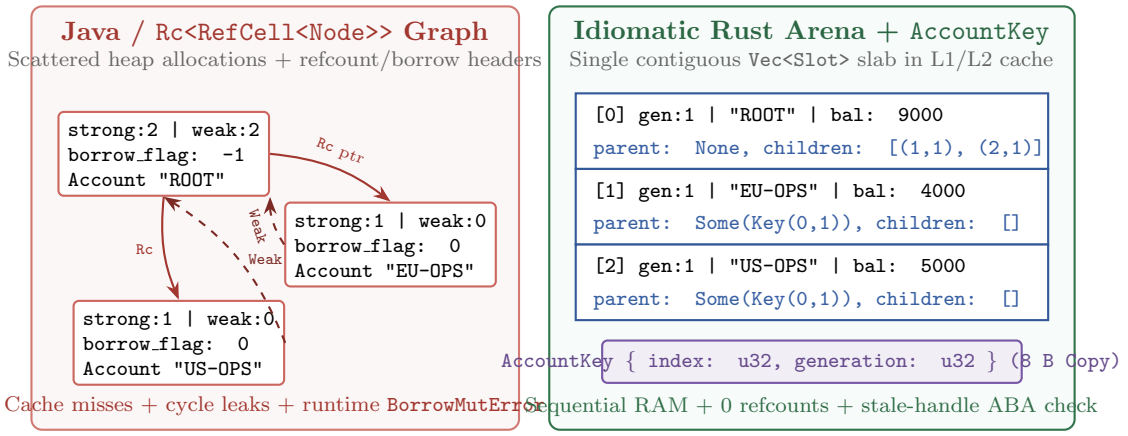



Figure 8.1: Pointer-based object graphs versus a flat generational arena. Left: `Rc<RefCell<T>>` scatters nodes across separate heap allocations with 24 bytes of refcount and borrow-flag overhead per node. Right: one `Vec` owns all nodes contiguously while lightweight `Copy` keys encode relationships as data.

### 8.1.3 Generational Indices: Solving the Stale-Handle (ABA) Problem

A sharp C programmer looking at Figure 8.1 immediately asks an important question: *"Wait — if an index handle is just `struct AccountKey(u32)`, what happens when account `1` is closed, removed from the arena, and its slot is later reused for a brand-new customer?"*

If a queued settlement batch still holds the old `AccountKey(1)`, indexing into slot `1` will not corrupt raw memory the way a C dangling pointer does, yet it will commit an equally nasty *logical* bug: crediting money to the wrong customer. In systems programming, this is the classic **ABA problem**: slot 1 went from Customer $A$, to free, to Customer $B$, and a naive index cannot tell the second occupant apart from the first.

The cure — used for entity handles in Rust game engines, in compilers (`rustc` itself stores AST, HIR, and MIR graphs in arenas), and the widely used `slotmap` crate — is a **generational index**:

**Definition 8.1** (Generational Handle and Slot Map)**.** A *generational arena* pairs each array slot $i \in \{0, \ldots, n-1\}$ with a monotonically increasing counter $\text{gen}_i \in \mathbb{N}$. Every handle handed to callers is an 8-byte pair (index : `u32`, generation : `u32`).

- On **lookup** of $(\text{idx}, g)$, the arena checks both $\text{idx} < n$ and that slot idx is currently `Occupied` with $\text{gen}_{\text{idx}} == g$. If either fails, lookup cleanly returns `None`.
- On **removal** of $(\text{idx}, g)$, slot idx transitions to `Vacant`, increments its generation to $g+1$ (using wrapping or checked arithmetic), and pushes idx onto an embedded singly linked free-list of reusable slots.

Any stale handle still holding $(\text{idx}, g)$ will forever see $g \neq g+1$ and fail closed as `None`, even after the slot is recycled millions of times.

Notice how neatly the free-list fits inside the vacant slots themselves without a second `Vec`: when a slot is empty, its payload space stores `next_free: Option<u32>` pointing to the next available index.

### 8.1.4 Mutating Two Nodes Safely with `split_at_mut`

A second practical question surfaces the moment you write `transfer(sender_key, receiver_key, cents)` on an arena: *how do you obtain `&mut AccountNode` to both the sender and receiver at the same time?*

If you write two consecutive calls — `let s = arena.get_mut(sender)?;` followed by `let r = arena.get_mut(receiver)?;` — the compiler rejects the second line because `get_mut(&mut self, ...)` borrows the entire arena exclusively for the lifetime of `s`. And rightly so. What if a caller passed `sender == receiver`? Returning two `&mut AccountNode` references pointing to the exact same slot would create aliasing mutable references — instant undefined behavior in Rust's memory model.

How, then, do we give callers simultaneous mutable access to two distinct slots without `unsafe` code? The standard library provides `slice::split_at_mut(mid)`, which splits one mutable slice `&mut [T]` at index `mid` into two provably non-overlapping sub-slices `(&mut [T], &mut [T])`: indices `0..mid` on the left, and indices `mid..len` on the right.

Let us put the entire generational `Arena<T>`, including `get_two_mut` and our hierarchical `ChartOfAccounts` ledger with ancestor exposure rollups, together in roughly 120 lines of 100-percent safe, production-grade Rust:

```rust
#[derive(Copy, Clone, Debug, PartialEq, Eq, Hash)]
pub struct AccountKey {
    index: u32,
    generation: u32,
}

enum Slot<T> {
    Occupied { generation: u32, value: T },
    Vacant { generation: u32, next_free: Option<u32> },
}

pub struct Arena<T> {
    slots: Vec<Slot<T>>,
    free_head: Option<u32>,
}

impl<T> Arena<T> {
    pub fn new() -> Self {
        Self { slots: Vec::new(), free_head: None }
    }

    pub fn insert(&mut self, value: T) -> AccountKey {
        if let Some(idx) = self.free_head {
            let slot = &mut self.slots[idx as usize];
            match *slot {
                Slot::Vacant { generation, next_free } => {
                    self.free_head = next_free;
                    *slot = Slot::Occupied { generation, value };
                    AccountKey { index: idx, generation }
                }
                Slot::Occupied { .. } => {
                    unreachable!("invariant: free_head names Vacant")
```

```
                }
            }
        } else {
            let idx = u32::try_from(self.slots.len())
                .expect("arena capacity exceeded u32::MAX slots");
            self.slots.push(Slot::Occupied { generation: 1, value });
            AccountKey { index: idx, generation: 1 }
        }
    }

    pub fn remove(&mut self, key: AccountKey) -> Option<T> {
        let slot = self.slots.get_mut(key.index as usize)?;
        match slot {
            Slot::Occupied { generation, .. }
                if *generation == key.generation =>
            {
                let next_gen = generation.saturating_add(1);
                let old = std::mem::replace(
                    slot,
                    Slot::Vacant {
                        generation: next_gen,
                        next_free: self.free_head,
                    },
                );
                self.free_head = Some(key.index);
                match old {
                    Slot::Occupied { value, .. } => Some(value),
                    Slot::Vacant { .. } => unreachable!(),
                }
            }
            _ => None,
        }
    }

    pub fn get(&self, key: AccountKey) -> Option<&T> {
        match self.slots.get(key.index as usize)? {
            Slot::Occupied { generation, value }
                if *generation == key.generation => Some(value),
            _ => None,
        }
    }

    pub fn get_mut(&mut self, key: AccountKey) -> Option<&mut T> {
        match self.slots.get_mut(key.index as usize)? {
            Slot::Occupied { generation, value }
                if *generation == key.generation => Some(value),
            _ => None,
        }
    }

    /// Safely borrow two distinct elements mutably at the same time.
    pub fn get_two_mut(
        &mut self,
        a: AccountKey,
```

```
        b: AccountKey,
    ) -> Option<(&mut T, &mut T)> {
        if a.index == b.index {
            return None; // Cannot mutably alias the same slot!
        }
        let (ai, bi) = (a.index as usize, b.index as usize);
        if ai >= self.slots.len() || bi >= self.slots.len() {
            return None;
        }
        let (slot_a, slot_b) = if ai < bi {
            let (left, right) = self.slots.split_at_mut(bi);
            (&mut left[ai], &mut right[0])
        } else {
            let (left, right) = self.slots.split_at_mut(ai);
            (&mut right[0], &mut left[bi])
        };
        match (slot_a, slot_b) {
            (
                Slot::Occupied { generation: ga, value: va },
                Slot::Occupied { generation: gb, value: vb },
            ) if *ga == a.generation && *gb == b.generation => {
                Some((va, vb))
            }
            _ => None,
        }
    }
}
```

Look at how clean the hierarchical chart of accounts becomes on top of `Arena<AccountNode>`:

```
pub struct AccountNode {
    pub code: String,
    pub balance_cents: i64,
    pub subtree_exposure_cents: i64,
    pub parent: Option<AccountKey>,
    pub children: Vec<AccountKey>,
}

pub struct ChartOfAccounts {
    nodes: Arena<AccountNode>,
}

impl ChartOfAccounts {
    pub fn transfer(
        &mut self,
        sender: AccountKey,
        receiver: AccountKey,
        cents: i64,
    ) -> Result<(), &'static str> {
        if cents <= 0 {
            return Err("transfer amount must be positive");
        }
        // Step 1: Mutate sender and receiver together via split_at_mut
        {
```

```
            let (s, r) = self
                .nodes
                .get_two_mut(sender, receiver)
                .ok_or("invalid or identical sender/receiver key")?;
            if s.balance_cents < cents {
                return Err("insufficient funds");
            }
            s.balance_cents -= cents;
            r.balance_cents += cents;
        } // <-- both &mut borrows of self.nodes end here!

        // Step 2: Walk upward from each touched leaf. Cannot panic.
        self.recompute_upward(Some(sender));
        self.recompute_upward(Some(receiver));
        Ok(())
    }

    // Recomputes exposure for `cursor` and then every ancestor above it.
    fn recompute_upward(&mut self, mut cursor: Option<AccountKey>) {
        while let Some(key) = cursor {
            let Some(node) = self.nodes.get(key) else { break };
            let next_parent = node.parent;
            // Sum own balance + children exposures via shared &self
            let own = node.balance_cents.abs();
            let child_sum: i64 = node
                .children
                .iter()
                .filter_map(|&ck| self.nodes.get(ck))
                .map(|c| c.subtree_exposure_cents)
                .sum();
            if let Some(node_mut) = self.nodes.get_mut(key) {
                node_mut.subtree_exposure_cents = own + child_sum;
            }
            cursor = next_parent;
        }
    }
}
```

**Why it works**

Trace why the re-entrancy crash from `Rc<RefCell<T>>` cannot happen here. In Step 1 of `transfer`, `s` and `r` borrow two slots of `self.nodes` only inside the inner block `{ ... }`, and the compiler ends both borrows at its closing brace. By Step 2 nothing is borrowed at all, so `recompute_upward` is free to read any child slot through `get` and then write the current node through `get_mut`, one step at a time. The ordering that `RefCell` checked at runtime (and got wrong in the trap above) is now checked by `rustc` before the program ever runs. Starting the walk at the touched leaf itself, rather than at its parent, also matters: the leaf's own `subtree_exposure_cents` must be refreshed before its parent sums it.

## 8.2 Parse, Don't Validate: Zero-Cost Newtypes and Smart Constructors

Look back at the signature of `ChartOfAccounts::transfer` above. Notice how we already replaced raw `u32` account indices with `AccountKey`, yet the final parameter is still `cents: i64`, forcing the method to open with `if cents <= 0 { return Err(...); }`. Now imagine scaling that style to an international wire-submission function written in the "primitive obsession" style common in Java 6 and C codebases:

```
// Anti-pattern: Primitive Obsession & Shotgun Validation
pub fn submit_international_wire(
    customer_id: u64,
    sender_account_id: u64,
    recipient_account_id: u64,
    destination_iban: &str,
    currency_iso: &str,
    amount_cents: i64,
    fee_cents: i64,
) -> Result<u64, String> {
    if sender_account_id == recipient_account_id { /* ... */ }
    if amount_cents <= 0 || fee_cents < 0 { /* ... */ }
    if currency_iso.len() != 3 { /* ... */ }
    if !is_valid_iban_checksum(destination_iban) { /* ... */ }
    // ...
    Ok(1)
}
```

Count the structural hazards hiding inside those seven primitive parameters:

1. **Silent argument-swap bugs.** Three adjacent parameters have type `u64`, two have type `&str`, and two have type `i64`. If a caller invokes `submit_international_wire(acct_id, cust_id, recip_id, currency, iban, fee, amount)`, `rustc` sees `(u64, u64, u64, &str, &str, i64, i64)` — a type-perfect match. In production, you have just debited the wrong account or swapped the transfer principal with the wire fee.

2. **Shotgun validation.** Suppose `submit_international_wire` calls `is_valid_iban_checksum` and passes `destination_iban: &str` down to `route_correspondent_bank` and `format_swift_mt103`. When you read `format_swift_mt103` six months later, how do you know its caller remembered to validate the IBAN? You don't. The type `&str` includes `""`, `"hello\nworld"`, and a 10-megabyte novel. So defensive programmers paste the exact same validation `if` checks at the top of every downstream helper — scattering domain rules across dozens of files while paying repeated runtime CPU checks.

### 8.2.1 Validation Throws Proof Away; Parsing Preserves It in a Type

In a classic 2019 essay originating in the Haskell community and adopted across the Rust ecosystem, Alexis King named the cure in three words: **Parse, don't validate**.

**Definition 8.2** (Validation versus Parsing)**.** Compare two function signatures operating on raw

untrusted input:

Validate: `fn validate_iban(raw: &str) -> Result<(), IbanError>`
Parse: `fn parse_iban(raw: &str) -> Result<Iban, IbanError>`

Both inspect the exact same characters and reject the exact same malformed strings. Yet `validate_iban` returns `()` — the zero-byte unit type from Chapter 7 — *evaporating* the knowledge it just worked to establish and leaving the caller holding the same unrefined `&str`. By contrast, `parse_iban` narrows the output into a distinct type `Iban` whose only constructors enforce the invariant. Downstream functions accepting `iban: &Iban` neither re-check the string nor return `Result` for malformed syntax; possessing an `Iban` value *is* a machine-checked proof that parsing succeeded.

Why didn't Java 6 developers wrap every `long` ID and positive cent amount in its own wrapper class (`new AccountId(42L)`, `new PositiveCents(5000L)`)? Because on the JVM (prior to Project Valhalla value classes), every wrapper object carries a 12-to-16-byte object header, lives as a separate heap allocation behind an 8-byte pointer, can be `null`, stresses garbage-collection allocation rates, and turns arrays into arrays of pointers.

In Rust, wrapping a single field inside a tuple struct — called the **Newtype pattern** — has **zero runtime memory or CPU cost**.

### 8.2.2 Zero-Cost Newtypes, `#[repr(transparent)]`, and Module Privacy

Let us build a real domain module `mod domain` containing distinct ID types, a strictly positive cent type `PositiveCents`, and an ISO 13616 `Iban` newtype that verifies country length, alphanumeric characters, and the international modulo-97 check-digit equation at construction time:

```
pub mod domain {
    use std::fmt;
    use std::num::NonZeroI64;
    use std::str::FromStr;

    // 1. Distinct zero-cost identifier newtypes (inner field is PRIVATE!)
    #[repr(transparent)]
    #[derive(Copy, Clone, Debug, PartialEq, Eq, Hash, PartialOrd, Ord)]
    pub struct CustomerId(u64);

    #[repr(transparent)]
    #[derive(Copy, Clone, Debug, PartialEq, Eq, Hash, PartialOrd, Ord)]
    pub struct AccountId(u64);

    impl CustomerId {
        pub const fn new(raw: u64) -> Self { Self(raw) }
        pub const fn get(self) -> u64 { self.0 }
    }

    impl AccountId {
        pub const fn new(raw: u64) -> Self { Self(raw) }
        pub const fn get(self) -> u64 { self.0 }
    }

    // 2. Strictly positive minor-unit money (> 0 cents guaranteed!)
```

```rust
    #[repr(transparent)]
    #[derive(Copy, Clone, Debug, PartialEq, Eq, PartialOrd, Ord)]
    pub struct PositiveCents(NonZeroI64);

    #[derive(Debug, Clone, PartialEq, Eq)]
    pub enum MoneyError {
        NonPositiveAmount(i64),
        Overflow,
    }

    impl PositiveCents {
        pub fn try_new(cents: i64) -> Result<Self, MoneyError> {
            if cents <= 0 {
                return Err(MoneyError::NonPositiveAmount(cents));
            }
            // Safe: we just proved cents > 0, so NonZeroI64::new is Some
            let nz = NonZeroI64::new(cents)
                .ok_or(MoneyError::NonPositiveAmount(cents))?;
            Ok(Self(nz))
        }

        pub const fn get(self) -> i64 {
            self.0.get()
        }

        #[must_use = "checked_add returns a new Result without mutating"]
        pub fn checked_add(self, rhs: Self) -> Result<Self, MoneyError> {
            let sum = self.get()
                .checked_add(rhs.get())
                .ok_or(MoneyError::Overflow)?;
            Self::try_new(sum)
        }
    }

    // 3. Validated IBAN newtype with ISO 13616 modulo-97 checksum
    #[derive(Clone, Debug, PartialEq, Eq, Hash)]
    pub struct Iban(String); // Private inner String!

    #[derive(Debug, Clone, PartialEq, Eq)]
    pub enum IbanError {
        InvalidLength(usize),
        InvalidCountryHeader,
        InvalidCharacter(char),
        ChecksumFailed,
    }

    impl FromStr for Iban {
        type Err = IbanError;

        fn from_str(raw: &str) -> Result<Self, Self::Err> {
            let canonical: String = raw
                .chars()
                .filter(|c| !c.is_ascii_whitespace())
                .map(|c| c.to_ascii_uppercase())
```

```
                .collect();

            if !(15..=34).contains(&canonical.len()) {
                return Err(IbanError::InvalidLength(canonical.len()));
            }
            let b = canonical.as_bytes();
            if !b[0].is_ascii_uppercase() || !b[1].is_ascii_uppercase()
                || !b[2].is_ascii_digit() || !b[3].is_ascii_digit()
            {
                return Err(IbanError::InvalidCountryHeader);
            }

            // ISO 13616: move first 4 chars to end, map A..Z -> 10..35,
            // and check that the integer has remainder 1 modulo 97.
            let rearranged =
                canonical[4..].chars().chain(canonical[..4].chars());
            let mut rem: u32 = 0;
            for ch in rearranged {
                if ch.is_ascii_digit() {
                    let digit = ch as u32 - b'0' as u32;
                    rem = (rem * 10 + digit).rem_euclid(97);
                } else if ch.is_ascii_uppercase() {
                    let val = (ch as u32 - b'A' as u32) + 10;
                    rem = (rem * 100 + val).rem_euclid(97);
                } else {
                    return Err(IbanError::InvalidCharacter(ch));
                }
            }
            if rem != 1 {
                return Err(IbanError::ChecksumFailed);
            }
            Ok(Self(canonical))
        }
    }

    impl Iban {
        pub fn as_str(&self) -> &str { &self.0 }
        pub fn country_code(&self) -> &str { &self.0[0..2] }
    }

    impl fmt::Display for Iban {
        fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
            f.write_str(&self.0)
        }
    }
}
```

Examine three architectural details working together in `mod domain`:

1. **Field privacy is what gives a Newtype teeth.** Notice we declared `PositiveCents(NonZeroI64)` with a private inner field rather than `pub NonZeroI64`. Because the tuple field `.0` is private to `mod domain`, code outside `mod domain` cannot construct `PositiveCents(-500)` using literal syntax and cannot mutate the inner value directly. Every value of type `PositiveCents`

in the entire program *must* have passed through `PositiveCents::try_new` or `checked_add`. This pattern — private fields paired with fallible factory functions returning `Result<Self, Err>` — is called a **smart constructor**.

2. **`#[repr(transparent)]` and register-level zero cost.** Putting `#[repr(transparent)]` on a single-field struct instructs the compiler that the wrapper must have the *exact same size, alignment, and C/Rust calling-convention ABI* as its inner field. Look at what happens down at the hardware level:

**Under the hood**

On 64-bit x86-64 Linux/macOS (System V AMD64 ABI):

- `size_of::<AccountId>() == 8` and `align_of::<AccountId>() == 8`. Passing `(CustomerId, AccountId, PositiveCents)` passes three plain 64-bit integers directly in CPU registers `rdi`, `rsi`, and `rdx` — zero pointers, zero heap headers, zero indirection.
- Because `PositiveCents` wraps `std::num::NonZeroI64` (which forbids the bit pattern `0x0000_0000_0000_0000`), the compiler uses `0` as the *niche discriminant* for `None`: `size_of::<Option<PositiveCents>>() == 8`. An optional validated positive dollar amount takes the exact same 8 bytes on the stack as a bare C `int64_t`.

Once `rustc` finishes type-checking, `AccountId` and `CustomerId` erase completely from the emitted machine instructions. All the safety lives at compile time; zero bytes remain at runtime.

3. **Solving Chapter 6's Orphan Rule cleanly.** Recall from Chapter 6's coherence law that Rust forbids implementing a foreign trait (say `std::fmt::Display` or `sqlx::Encode`) for a foreign type (say `Vec<LedgerEntry>` or a third-party crate's `Decimal`). A Newtype `pub struct StatementReport(pub Vec<LedgerEntry>);` is a *local* type owned by your crate, so implementing `fmt::Display for StatementReport` is 100-percent legal and keeps your formatting rules cleanly scoped without polluting global method resolution on every `Vec`.

Look at our wire-submission signature now:

```
pub fn submit_international_wire(
    customer: domain::CustomerId,
    sender: domain::AccountId,
    recipient: domain::AccountId,
    destination: &domain::Iban,
    amount: domain::PositiveCents,
) -> Result<WireReceipt, WireError>
```

Swapping `customer` with `sender` is an immediate compiler error. Passing a negative or zero amount is impossible. Re-checking the IBAN checksum inside `submit_international_wire` is unnecessary because `&Iban` already proves ISO 13616 validity. Validation happened once, at the HTTP JSON deserialization edge, and the type system carried the proof everywhere else for free.

## 8.3 Compile-Time State Machines: The Typestate Pattern and Builders

Newtypes enforce invariants on *individual values*. What about invariants across *time* — multi-step protocols where operations are only legal in a strict order?

Think about the lifecycle of a high-value wire order inside a bank:

1. An API handler creates a **Draft** wire order holding the sender ID, recipient IBAN, and `PositiveCents` amount.
2. A sanctions and anti-money-laundering (AML/OFAC) engine screens the counterparties. Only if screening passes does the order become **ComplianceCleared**, acquiring a cryptographic officer/screening stamp.
3. A settlement worker transmits a cleared order to the central clearing rail, producing a **Settled** wire with a rail transaction sequence number.

Two bugs must never occur: (a) settling a `Draft` wire that bypassed AML compliance screening, and (b) settling the *same* `ComplianceCleared` wire twice.

In Java 6, a single mutable `WireOrder` class tries to police all three phases at runtime using nullable fields and boolean flags:

```
// Java 6: temporal protocol encoded via mutable flags + runtime exceptions
public class WireOrder {
    private boolean complianceCleared = false;
    private boolean settled = false;
    private String ofacTicket; // null until complianceCleared == true!
    private Long railTxId;     // null until settled == true!

    public void approveCompliance(String ticket) {
        if (this.complianceCleared || this.settled) {
            throw new IllegalStateException("Invalid state for approval");
        }
        this.complianceCleared = true;
        this.ofacTicket = ticket;
    }

    public void settle(long txId) {
        if (!this.complianceCleared || this.settled) {
            throw new IllegalStateException(
                "Wire not cleared or already settled!"
            );
        }
        this.settled = true;
        this.railTxId = txId;
    }
}
```

Every caller can invoke `order.settle(999L)` on a fresh draft; the Java compiler says "looks fine to me," and you only find out when an `IllegalStateException` is thrown in production (or worse, if someone forgets the `if (!this.complianceCleared)` guard inside a new batch-settlement method,

an unscreened wire hits the SWIFT network). And notice those two nullable fields `ofacTicket` and `railTxId`: two booleans plus two nullable fields create $2 \times 2 \times 2 \times 2 = 16$ representable combinations in memory when only 3 states are legal.

### 8.3.1 Encoding Protocol States in Type Parameters and `PhantomData`

In Rust, we can lift the state of the protocol out of runtime booleans and put it directly into a compile-time generic parameter `S` on `WireOrder<S>`. This is the **Typestate pattern**.

You actually have two complementary ways to represent the state parameter `S`, and you should know both:

1. **Pure Zero-Sized Marker Types + `PhantomData<S>`.** When two states store the exact same runtime fields and differ only in which methods are permitted (for instance, `RawQuery` versus `SanitizedQuery`, or `Locked` versus `Unlocked`), we define empty unit structs `pub struct Draft;` and `pub struct Verified;` that occupy **0 bytes** in RAM. Because Rust requires every generic type parameter `S` declared on `struct Thing<S>` to be referenced in at least one field (so the compiler knows `S`'s variance and drop behavior from Chapter 4), we include a zero-byte marker field `_state: std::marker::PhantomData<S>`.
2. **State-Specific Payload Structs (`state: S`).** Even better, when entering a state *produces new data* that did not exist before — such as `ComplianceCleared` acquiring an `ofac_ticket` and `Settled` acquiring a `rail_tx_id` — we store `state: S` directly as a field. For `Draft`, `pub struct Draft;` is still a 0-byte zero-sized type (ZST), so `WireOrder<Draft>` pays zero bytes for `state`. For `ComplianceCleared`, `state` holds the `ApprovalStamp` directly without any `Option<...>` wrapper.

Formal

**What `PhantomData<S>` Is at the Byte and Type Level.** Inside `core::marker`, `PhantomData<T>` is a compiler-intrinsic zero-sized type:

$$\texttt{std::mem::size_of::<PhantomData<S>>()} = 0,$$
$$\texttt{std::mem::align_of::<PhantomData<S>>()} = 1.$$

At runtime, a `PhantomData<S>` field emits *zero bits* in memory and *zero instructions* during moves or copies. At compile time, it tells `rustc` to treat the enclosing struct "as if" it logically owned a value of type `S` for type discrimination, lifetime variance, and `Send`/`Sync` auto-trait derivation. (If you want invariance without drop/send ownership effects, a common systems idiom is `PhantomData<fn(S) -> S>`, which is always `Copy + Send + Sync` regardless of `S`.)

### 8.3.2 Why Rust's Destructive Move Makes Typestate Airtight

Here is the deeper insight that surprises engineers who have tried to emulate typestate using generics in Java, C++, or TypeScript. Suppose in Java you wrote `ClearedWire approve(DraftWire d)`. Nothing in Java stops the caller from writing:

```
// In Java / TypeScript / Go, the old reference remains usable!
ClearedWire c1 = draft.approve("OFAC-OK-1");
```

```
ClearedWire c2 = draft.approve("OFAC-OK-2"); // Compiles! Double-approved!
```

Even in C++11, `std::move(draft)` leaves `draft` as a live lvalue on the stack in a "valid but unspecified" state whose methods can still be called by mistake.

In Rust, look at what happens when transition methods take **`self` by value** (not `&self`, and not `&mut self`):

```
use std::marker::PhantomData;
use crate::domain::{AccountId, Iban, PositiveCents};

// State types: Draft is 0 bytes; later states carry their proof artifacts!
pub struct Draft;

pub struct ComplianceCleared {
    pub officer_id: u32,
    pub ofac_ticket: [u8; 16],
}

pub struct Settled {
    pub officer_id: u32,
    pub ofac_ticket: [u8; 16],
    pub rail_tx_id: u64,
}

pub struct Rejected {
    pub reason: String,
}

// Generic wire order parameterized by its current protocol state S,
// plus a zero-byte PhantomData marker for the rail protocol R.
pub struct SwiftRail;

pub struct WireOrder<S, R = SwiftRail> {
    sender: AccountId,
    destination: Iban,
    amount: PositiveCents,
    state: S,
    _rail: PhantomData<R>, // 0 bytes at runtime!
}

// Phase 1 methods: ONLY exist on WireOrder<Draft, R>!
impl<R> WireOrder<Draft, R> {
    pub fn new_draft(
        sender: AccountId,
        destination: Iban,
        amount: PositiveCents,
    ) -> Self {
        Self {
            sender,
            destination,
            amount,
            state: Draft,
            _rail: PhantomData,
        }
```

```
    }

    // Takes `self` BY VALUE: consumes WireOrder<Draft, R> permanently!
    pub fn approve_compliance(
        self,
        officer_id: u32,
        ofac_ticket: [u8; 16],
    ) -> WireOrder<ComplianceCleared, R> {
        WireOrder {
            sender: self.sender,
            destination: self.destination,
            amount: self.amount,
            state: ComplianceCleared { officer_id, ofac_ticket },
            _rail: PhantomData,
        }
    }

    pub fn reject(
        self,
        reason: impl Into<String>,
    ) -> WireOrder<Rejected, R> {
        WireOrder {
            sender: self.sender,
            destination: self.destination,
            amount: self.amount,
            state: Rejected { reason: reason.into() },
            _rail: PhantomData,
        }
    }
}

// Phase 2 methods: ONLY exist on WireOrder<ComplianceCleared, R>!
impl<R> WireOrder<ComplianceCleared, R> {
    pub fn ofac_ticket(&self) -> &[u8; 16] {
        &self.state.ofac_ticket
    }

    // Consumes WireOrder<ComplianceCleared, R> -> cannot settle twice!
    pub fn settle(self, rail_tx_id: u64) -> WireOrder<Settled, R> {
        WireOrder {
            sender: self.sender,
            destination: self.destination,
            amount: self.amount,
            state: Settled {
                officer_id: self.state.officer_id,
                ofac_ticket: self.state.ofac_ticket,
                rail_tx_id,
            },
            _rail: PhantomData,
        }
    }
}

// Shared read-only accessors available in EVERY state S:
```

```rust
impl<S, R> WireOrder<S, R> {
    pub fn sender(&self) -> AccountId { self.sender }
    pub fn destination(&self) -> &Iban { &self.destination }
    pub fn amount(&self) -> PositiveCents { self.amount }
}
```

Try breaking this protocol in client code:

- **Try settling an unscreened draft:** `draft.settle(1001)` fails to compile with `no method named ‘settle‘ found for struct ‘WireOrder<Draft>‘`. The method does not exist in `Draft`'s method table.
- **Try settling a cleared wire twice:**

```rust
let cleared = draft.approve_compliance(7, *b"OFAC-2026-PASS01");
let done_1 = cleared.settle(90001); // ‘cleared‘ is MOVED here!
// let done_2 = cleared.settle(90002);
// ^-- COMPILE ERROR: borrow/use of moved value: ‘cleared‘!
```

Because `WireOrder` deliberately does *not* derive `Clone`, and `settle(self, ...)` takes ownership by value, Rust's affine ownership system enforces *linear state progression*: one `Draft` token enters, one `ComplianceCleared` token emerges, and the `Draft` binding ceases to exist in the compiler's live-variable table.

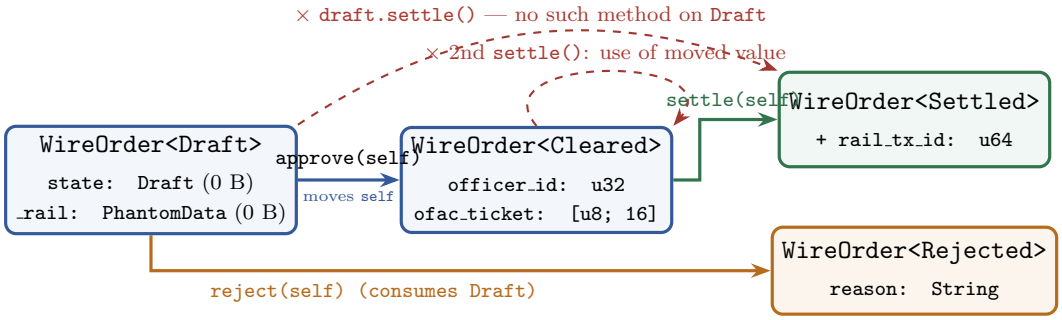



Figure 8.2: Compile-time Typestate automaton for `WireOrder<S>`. Valid transitions consume `self` by value; out-of-order calls (`draft.settle()`) and duplicate settlements fail at compile time with zero runtime flag overhead.

### 8.3.3 Compile-Time Typestate vs Runtime `enum` State Machines

Once engineers discover Typestate, a common temptation is to try to use it for *every* state machine in a codebase — and promptly hit a wall the first time they load rows from PostgreSQL.

Ask yourself: if you run `SELECT id, status, sender_id, iban, cents FROM wire_orders WHERE customer_id = $1`, what is the return type of that function? At compile time, `rustc` cannot possibly know whether row 0 in PostgreSQL currently has `status = 'DRAFT'` or `status = 'SETTLED'`. Even more directly, a `Vec<T>` requires every element to have the *exact same concrete type* `T`, so you cannot put a `WireOrder<Draft>` and a `WireOrder<Settled>` side by side in the same `Vec`.

Here is the crisp architectural decision rule separating Chapter 5's runtime `enum`s from Chapter 8's compile-time Typestate — and how production Rust combines both:

**Intuition**

- **Use Compile-Time Typestate (`Order<S>`)** when the state progression is *statically choreographed by the code path itself*: builders, multi-step cryptographic handshakes, SQL transaction lifecycles, or a handler pipeline that always parses a request into `Draft`, screens it into `ComplianceCleared`, and settles it.
- **Use a Runtime Sum Type (`enum AnyWireOrder`)** when state depends on *external data unknown until runtime* (database rows, network messages, user events) or when you must store a *heterogeneous collection* (`Vec<AnyWireOrder>`) of items currently sitting at different lifecycle stages.

You can combine the two. Wrap your strongly typed typestate structs *inside* a runtime `enum` envelope at storage and network boundaries:

```
// Runtime envelope wrapping compile-time typestate structs!
pub enum AnyWireOrder {
    Draft(WireOrder<Draft>),
    Cleared(WireOrder<ComplianceCleared>),
    Settled(WireOrder<Settled>),
    Rejected(WireOrder<Rejected>),
}

pub fn process_pending_from_db(
    row: AnyWireOrder,
    officer_id: u32,
) -> AnyWireOrder {
    match row {
        // Unpacking `Draft(draft)` yields a typed WireOrder<Draft>!
        AnyWireOrder::Draft(draft) => {
            let cleared =
                draft.approve_compliance(officer_id, *b"OFAC-AUTO-OK-001");
            AnyWireOrder::Cleared(cleared)
        }
        // Cleared, settled, or rejected rows cannot be re-approved:
        other => other,
    }
}
```

Look how cleanly the two layers interlock: PostgreSQL hydration produces `AnyWireOrder`, `match` narrows a dynamic variant into a concrete `WireOrder<Draft>`, and inside that branch all of Typestate's compile-time guarantees govern every transition.

### 8.3.4 Idiomatic Builders: Owned Chaining and Typestate Builders

Unlike Java 6, Rust intentionally has neither method overloading (you cannot write four `new(...)` constructors with 2, 3, 5, and 8 parameters) nor default/optional function arguments. Whenever a configuration or request struct has more than three or four knobs, idiomatic Rust provides a **Builder**.

Two flavors appear across the ecosystem:

**1. Owned-Chaining Builder (for optional knobs with sensible defaults).** When all required arguments fit cleanly in `new(...)` and the remaining settings have safe defaults, each builder method takes `mut self` by value, updates one field, and returns `Self`:

```rust
pub struct RetryPolicy {
    max_attempts: u32,
    initial_backoff_ms: u64,
    max_backoff_ms: u64,
    jitter: bool,
}

impl RetryPolicy {
    #[must_use]
    pub fn new() -> Self {
        Self {
            max_attempts: 3,
            initial_backoff_ms: 50,
            max_backoff_ms: 2_000,
            jitter: true,
        }
    }

    #[must_use]
    pub fn max_attempts(mut self, attempts: u32) -> Self {
        self.max_attempts = attempts.max(1);
        self
    }

    #[must_use]
    pub fn backoff_bounds_ms(mut self, initial: u64, max: u64) -> Self {
        self.initial_backoff_ms = initial;
        self.max_backoff_ms = max.max(initial);
        self
    }

    #[must_use]
    pub fn jitter(mut self, enabled: bool) -> Self {
        self.jitter = enabled;
        self
    }
}
```

Taking `mut self -> Self` supports both fluent chains like `RetryPolicy::new().max_attempts(5).jitter(false)` *and* conditional updates inside loops (`policy = policy.max_attempts(10)`) without dangling temporary `&mut` borrows.

**2. Typestate Builder (for mandatory configuration with zero `.unwrap()`).** What if a `LedgerPoolBuilder` has *mandatory* fields without sensible defaults — say `postgres_url` and `signing_key` — alongside optional timeouts? A Java 6 builder initializes those fields to `null` and throws `IllegalStateException("missing postgresUrl")` inside `.build()`. Using Typestate, `.build()` is only implemented on `LedgerPoolBuilder<HasUrl, HasKey>`. Forgetting to call `.postgres_url(...)` makes `.build()` a compile-time error rather than a startup crash:

```rust
pub struct Missing;
pub struct Provided<T>(T);

pub struct LedgerPoolBuilder<U, K> {
    url: U,
    signing_key: K,
    max_connections: u32,
}

impl LedgerPoolBuilder<Missing, Missing> {
    pub fn new() -> Self {
        Self { url: Missing, signing_key: Missing, max_connections: 16 }
    }
}

impl<U, K> LedgerPoolBuilder<U, K> {
    pub fn max_connections(mut self, n: u32) -> Self {
        self.max_connections = n.max(1);
        self
    }

    pub fn postgres_url(
        self,
        url: impl Into<String>,
    ) -> LedgerPoolBuilder<Provided<String>, K> {
        LedgerPoolBuilder {
            url: Provided(url.into()),
            signing_key: self.signing_key,
            max_connections: self.max_connections,
        }
    }

    pub fn signing_key(
        self,
        key: [u8; 32],
    ) -> LedgerPoolBuilder<U, Provided<[u8; 32]>> {
        LedgerPoolBuilder {
            url: self.url,
            signing_key: Provided(key),
            max_connections: self.max_connections,
        }
    }
}

pub struct LedgerPoolConfig {
    pub url: String,
    pub signing_key: [u8; 32],
    pub max_connections: u32,
}

// .build() ONLY exists once BOTH mandatory fields have been supplied!
impl LedgerPoolBuilder<Provided<String>, Provided<[u8; 32]>> {
    pub fn build(self) -> LedgerPoolConfig {
        LedgerPoolConfig {
```

```
            url: self.url.0,
            signing_key: self.signing_key.0,
            max_connections: self.max_connections,
        }
    }
}
```

Notice that the caller can supply `.signing_key(...)` and `.postgres_url(...)` in *either* order, interleave `.max_connections(32)` anywhere, and `.build()` returns an infallible `LedgerPoolConfig` directly — no `Option`, no `Result`, no runtime `.unwrap()`.

## 8.4 Ergonomic, Zero-Copy API Boundaries and Borrow-Splitting Idioms

Once your data structures and state machines are cleanly modeled, the next place "Java-in-Rust syndrome" leaks CPU cycles and frustrates callers is at *function boundaries* — specifically, how functions accept borrowed views, how constructors accept owned buffers, and how methods borrow fields of `self`.

### 8.4.1 Why `&String`, `&Vec<T>`, and `&Box<T>` Are Anti-Patterns

A developer accustomed to Java's `String` and `List<Posting>` (or C++'s `const std::string&` and `const std::vector<Posting>&`) naturally writes Rust helper signatures like these:

```
// Anti-pattern: borrowing the heap-owning container header!
pub fn is_sepa_country(iban: &String) -> bool {
    matches!(&iban[0..2], "DE" | "FR" | "NL" | "ES" | "IT" | "CH")
}

pub fn net_balance_delta(postings: &Vec<i64>) -> i64 {
    postings.iter().copied().sum()
}

pub fn audit_fee_rule(rule: &Box<FeeSchedule>) -> i64 {
    rule.base_fee_cents
}
```

Every seasoned Rust reviewer flags `&String`, `&Vec<T>`, and `&Box<T>` immediately. Look at what is wrong on *both* the caller side and the hardware side:

1. **You lock out valid callers who do not own a `String` or `Vec`.** Suppose a caller wants to check `is_sepa_country("DE89370400440532013000")` using a string literal in read-only `.rodata` memory (`&'static str`), or slice an IBAN field directly out of a 4-kilobyte network socket buffer (`&packet_str[8..30]`). Neither caller possesses a heap-allocated `String`. Forcing `&String` makes them call `.to_string()` just to allocate a throwaway heap copy so your read-only function can glance at two bytes and immediately drop it. Similarly, `&Vec<i64>` rejects a fixed-size stack array `[100, -50, -50]` or a window slice `&all_postings[10..20]`.

2. **You introduce a pointless double pointer dereference.** Recall the stack layout from Chapter 3: a `String` on the stack is 24 bytes `(ptr, len, cap)`. Passing `iban: &String` passes an 8-byte pointer to that 24-byte stack `String`, which the CPU must first dereference just to load `ptr` and `len` before dereferencing `ptr` a second time to read the UTF-8 characters on the heap.

The idiomatic replacement is universal: **whenever a function only reads data without growing or shrinking its backing allocation, accept the borrowed unsized view**:

- Replace `&String` with `&str` (a 16-byte `(ptr, len)` fat pointer passed in two CPU registers, pointing directly at UTF-8 bytes whether they live on the heap, on the stack, or in static `.rodata`).
- Replace `&Vec<T>` with `&[T]` (a 16-byte `(ptr, len)` slice view over any contiguous `T` sequence).
- Replace `&Box<T>` (or `&Rc<T>`, `&Arc<T>`) with `&T`.
- For filesystem paths, accept `path: impl AsRef<Path>` (from `std::path::Path`) so callers can pass `"config/fees.toml"` (`&str`), `PathBuf`, or `&Path` without conversion ceremony.

And what about callers who *do* already hold a `s: String` or `v: Vec<i64>`? Thanks to Chapter 6's `Deref` **coercion** (`String: Deref<Target = str>` and `Vec<T>: Deref<Target = [T]>`), writing `is_sepa_country(&s)` and `net_balance_delta(&v)` still compiles with zero changes at the call site. Switching from `&String` to `&str` strictly widens who can call your function while eliminating a pointer hop.

### 8.4.2 Caller-Controlled Allocation: `impl Into<String>` and `Cow<'a, str>`

Now consider the opposite case: what if a function or constructor *does* need to store an owned `String` inside a struct?

```
pub struct AuditNote {
    actor: String,
    memo: String,
}

// Anti-pattern: taking &str when you ALWAYS allocate an owned String!
impl AuditNote {
    pub fn new_wasteful(actor: &str, memo: &str) -> Self {
        Self {
            actor: actor.to_owned(), // ALWAYS allocates + copies!
            memo: memo.to_owned(),
        }
    }

    // Idiomatic: let the caller move an existing String in O(1),
    // or allocate from &str only when needed!
    pub fn new(actor: impl Into<String>, memo: impl Into<String>) -> Self {
        Self {
            actor: actor.into(),
            memo: memo.into(),
        }
    }
```

```
}
```

Compare what happens when an HTTP JSON parser has *already* allocated `parsed_memo: String` on the heap:

- Calling `AuditNote::new_wasteful(&parsed_memo)` borrows the heap string as `&str`, calls `malloc` to allocate a *second* identical heap buffer, `memcpy`s the bytes, and frees `parsed_memo` at the end of the caller's scope.
- Calling `AuditNote::new(actor_literal, parsed_memo)` monomorphizes two conversions: for `actor_literal: &str`, `.into()` allocates once; for `parsed_memo: String`, `Into<String>::into(self)` is the identity function — it moves 24 bytes on the stack in $O(1)$ time with **zero heap allocations**. (For tiny non-generic compile speed, accepting `memo: String` directly is equally idiomatic and makes allocation visible right at the call site.)

What about the trickiest middle ground of all: a function that **99 percent of the time can return a slice of its input untouched, and 1 percent of the time needs to allocate a modified string**?

In payment processing, every incoming ACH or SWIFT remittance memo must be normalized before ledger ingestion: leading/trailing ASCII whitespace is trimmed, and if the memo contains ASCII control characters (like \r, \n, or \t), each control character must be replaced by a single space `' '`. In production traffic, 99 percent of bank memos are already clean printable ASCII. Returning `String` would force `malloc + memcpy` on all 100 percent of requests just to handle the dirty 1 percent.

Rust's standard library solves this with `std::borrow::Cow<'a, B>` (*Clone-on-Write*):

```
// Simplified definition from std::borrow:
pub enum Cow<'a, B: ?Sized + ToOwned> {
    Borrowed(&'a B),               // Zero heap allocation: borrows input!
    Owned(<B as ToOwned>::Owned),  // Heap owned: e.g., String for B = str
}
```

Because `Cow<'a, str>` implements `Deref<Target = str>`, callers can read a `Cow<'a, str>` just like any `&str` regardless of which variant came back, or call `.into_owned()` if they later decide to persist an owned `String` (which moves the existing `String` out of `Cow::Owned` without re-allocating, or allocates once if it was `Cow::Borrowed`).

Here is `sanitize_remittance_memo` handling both paths:

```
use std::borrow::Cow;

/// Trims ASCII whitespace and replaces ASCII control chars with ' '.
/// Fast path (clean input): 0 heap allocations!
pub fn sanitize_remittance_memo(raw: &str) -> Cow<'_, str> {
    let trimmed = raw.trim_matches(|c: char| c.is_ascii_whitespace());

    // Scan once for the first offending control character
    let Some(first_bad_byte) = trimmed
        .bytes()
        .position(|b| b.is_ascii_control())
    else {
        // 99-percent fast path: return a 16-byte slice into `raw`!
        return Cow::Borrowed(trimmed);
```

```
    };

    // 1-percent slow path: allocate ONCE, copy clean prefix, sanitize rest
    let mut cleaned = String::with_capacity(trimmed.len());
    cleaned.push_str(&trimmed[..first_bad_byte]);
    for ch in trimmed[first_bad_byte..].chars() {
        if ch.is_ascii_control() {
            cleaned.push(' ');
        } else {
            cleaned.push(ch);
        }
    }
    Cow::Owned(cleaned)
}
```

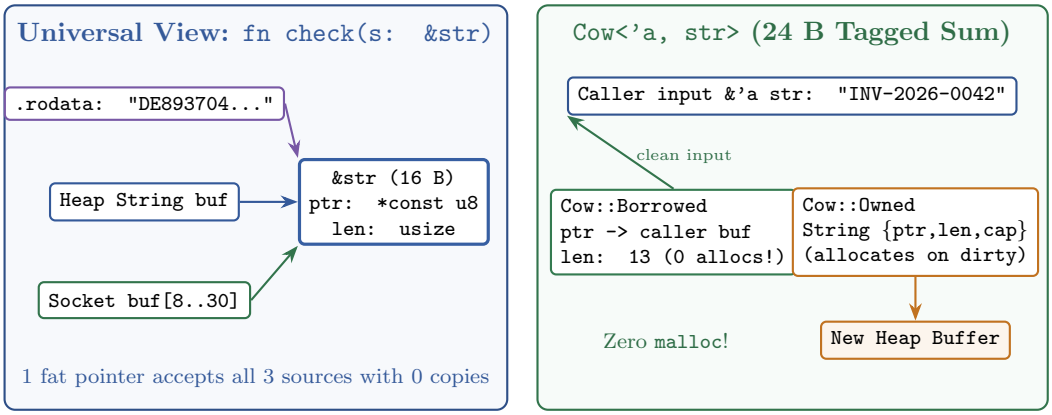



Figure 8.3: Zero-copy boundary idioms. Left: accepting `&str` (or `&[T]`) lets static literals, heap `String`s, and network sub-slices enter through a single 16-byte register pair. Right: `Cow<'a, str>` avoids heap allocation on clean inputs while retaining the ability to allocate when normalization mutates bytes.

### 8.4.3 Defeating the Whole-Struct `&mut self` Trap (Split Borrows)

This is one of the most common borrow-checker errors in application code. Once you understand *why* it happens, the fixes below are mechanical.

Suppose we build a `SettlementEngine` owning three distinct components: an in-memory `LedgerBook` of account balances, an immutable `FeeSchedule` loaded at startup, and an append-only `audit_log`:

```
pub struct FeeSchedule {
    pub flat_wire_fee_cents: i64,
}

pub struct LedgerBook {
    pub balances: std::collections::HashMap<u64, i64>,
}

pub struct SettlementEngine {
    pub ledger: LedgerBook,
```

```
    pub fee_schedule: FeeSchedule,
    pub audit_log: Vec<String>,
}
```

Following OOP habits, you write a helper `apply_fee(&mut self, ...)` on `SettlementEngine` and invoke it inside `settle_account(&mut self)` while passing `&self.fee_schedule`:

```
impl SettlementEngine {
    // Helper mutates ONLY self.ledger and self.audit_log...
    // ...yet its signature demands `&mut self` (the ENTIRE struct)!
    fn apply_fee(&mut self, acct: u64, rules: &FeeSchedule) {
        let bal = self.ledger.balances.entry(acct).or_insert(0);
        *bal -= rules.flat_wire_fee_cents;
        self.audit_log.push(format!("charged fee to {acct}"));
    }

    pub fn settle_account(&mut self, acct: u64) {
        // COMPILE ERROR E0502: cannot borrow `self` as mutable because
        // `self.fee_schedule` is also borrowed as immutable!
        // self.apply_fee(acct, &self.fee_schedule);
    }
}
```

The body of `apply_fee` only mutates the ledger and the audit log; it never touches the fee schedule. Why can't `rustc` just peek inside `apply_fee` and see that the fields are disjoint?

**Why it works**

**Signature Modularity Is Deliberate.** In Rust, borrow checking is strictly *modular across function boundaries*: `rustc` checks a caller against the callee's **signature alone**, never its body. Why? Suppose `apply_fee(&mut self, rules: &FeeSchedule)` were allowed today because its body didn't mutate `self.fee_schedule`. Six months later, a teammate adds `self.fee_schedule.flat_wire_fee_cents += 1;` inside `apply_fee`. That edit would silently create an active mutable alias against `rules: &FeeSchedule` *or* break distant callers across the codebase without the method signature ever changing. Writing `&mut self` is a contract claiming exclusive access to **every field** of `Self`.

Inside a *single* function body, however, `rustc`'s borrow checker *does* track every struct field independently. It knows `&mut self.ledger`, `&self.fee_schedule`, and `&mut self.audit_log` occupy non-overlapping byte offsets inside `SettlementEngine`. That immediately gives us **three clean idioms** to cure whole-struct `&mut self` conflicts:

**Idiom 1: Put the method on the sub-struct it actually mutates (or make it an associated function taking disjoint fields).** Instead of making `SettlementEngine` a "God object" where every helper takes `&mut self`, put `apply_fee` on `LedgerBook` (or pass `&mut self.ledger` and `&mut self.audit_log` explicitly):

```
impl LedgerBook {
    pub fn apply_fee(
        &mut self,
        acct: u64,
```

```
        rules: &FeeSchedule,
        audit_log: &mut Vec<String>,
    ) {
        let bal = self.balances.entry(acct).or_insert(0);
        *bal -= rules.flat_wire_fee_cents;
        audit_log.push(format!("charged fee to {acct}"));
    }
}

impl SettlementEngine {
    pub fn settle_account(&mut self, acct: u64) {
        // Compiles cleanly! rustc sees three disjoint field borrows:
        //   &mut self.ledger, &self.fee_schedule, &mut self.audit_log
        self.ledger.apply_fee(
            acct,
            &self.fee_schedule,
            &mut self.audit_log,
        );
    }
}
```

**Idiom 2: Group fields by mutation lifecycle.** If five fields always change together while three configuration fields stay read-only after startup, group them into `state: EngineState` and `config: EngineConfig`. Then `self.state.step(&self.config)` reads like plain English and never collides.

**Idiom 3: Explicit "View Structs" (Borrow Projections) for complex subsystems.** When several helpers need mutable access to two fields and shared access to two others, passing four separate parameters gets noisy. Define a lightweight **View Struct** holding only field references, plus a one-line `view_mut(&mut self)` projection method:

```
struct PostingView<'a> {
    ledger: &'a mut LedgerBook,
    audit_log: &'a mut Vec<String>,
    fees: &'a FeeSchedule,
}

impl<'a> PostingView<'a> {
    fn apply_fee(&mut self, acct: u64) {
        let bal = self.ledger.balances.entry(acct).or_insert(0);
        *bal -= self.fees.flat_wire_fee_cents;
        self.audit_log.push(format!("charged fee to {acct}"));
    }
}

impl SettlementEngine {
    fn posting_view(&mut self) -> PostingView<'_> {
        PostingView {
            ledger: &mut self.ledger,
            audit_log: &mut self.audit_log,
            fees: &self.fee_schedule,
        }
    }
```

```
23
24      pub fn settle_batch(&mut self, accounts: &[u64]) {
25          let mut view = self.posting_view();
26          for &acct in accounts {
27              view.apply_fee(acct);
28          }
29      }
30  }
```

Look at what `PostingView<'a>` achieves: the signature itself documents *exact field-level permissions* (mutable ledger and audit log, read-only fee schedule), and every helper on `PostingView` enjoys clean `&mut self` method syntax with zero borrow conflicts.

## 8.5 Idiomatic Error Handling: `thiserror` vs `anyhow`, and `Result` vs `panic!`

In Chapter 5 we saw how `Result<T, E>` and the `?` operator replace both C's silent integer error codes and Java 6's stack-unwinding exceptions. Yet knowing how `Result<T, E>` works mechanically leaves open an architectural question that trips up many new Rust teams: *what should the error type `E` actually be across a multi-layer codebase?*

If you make `E = String` everywhere, callers cannot `match` on failure causes without fragile substring searches. Conversely, if you force an ops CLI migration script to define a 25-variant custom `enum` just to print an error message and exit with code 1, you drown in ceremony.

The Rust ecosystem converged on a two-zone rule based on a single question: **will the caller programmatically branch on which failure occurred, or will it report/log the failure up the stack?**

### 8.5.1 Zone 1: Domain and Library Layers Use Structured `enums` (`thiserror`)

Inside your domain logic, database repositories, and reusable library crates, callers *act differently* depending on what failed:

- In our Part V Axum HTTP API, `TransferError::InsufficientFunds` maps to HTTP `422 Unprocessable Entity` with a JSON body telling the client their available balance, and **must not** wake the on-call SRE at 3:00 AM.
- `TransferError::AccountFrozen` maps to HTTP `403 Forbidden` and emits a compliance audit event.
- `TransferError::Storage(sql_err)` means PostgreSQL dropped a connection or timed out; it maps to HTTP `503 Service Unavailable`, hides internal SQL table names from external clients, and increments a critical Prometheus alert counter.

For Zone 1, define a concrete, exhaustive `enum` error per module or subsystem. Let us first write out the pure standard-library implementation so you see every trait involved, and then show the concise `#[derive(thiserror::Error)]` macro used in almost every production Rust workspace:

```
1  use std::fmt;
```

```

#[derive(Debug)]
pub struct SqlError {
    pub code: &'static str,
    pub detail: String,
}

impl fmt::Display for SqlError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "database error ({}): {}", self.code, self.detail)
    }
}
impl std::error::Error for SqlError {}

// Structured domain error enum for the transfer subsystem:
#[derive(Debug)]
pub enum TransferError {
    InsufficientFunds { available_cents: i64, requested_cents: i64 },
    AccountFrozen { account_id: u64 },
    SameSourceAndDestination { account_id: u64 },
    Storage(SqlError),
}

// 1. Human-readable single-level description (no trailing newline!)
impl fmt::Display for TransferError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::InsufficientFunds {
                available_cents,
                requested_cents,
            } => write!(
                f,
                "insufficient funds: have {available_cents} cents, \
                 need {requested_cents} cents"
            ),
            Self::AccountFrozen { account_id } => {
                write!(f, "account {account_id} is frozen by compliance")
            }
            Self::SameSourceAndDestination { account_id } => {
                write!(f, "self-transfer forbidden on {account_id}")
            }
            Self::Storage(_) => write!(f, "ledger storage query failed"),
        }
    }
}

// 2. Causal chain hook so loggers can walk `.source()` down to SqlError!
impl std::error::Error for TransferError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            Self::Storage(err) => Some(err),
            _ => None,
        }
    }
```

```
}

// 3. Automatic `?` conversion from lower-level SqlError!
impl From<SqlError> for TransferError {
    fn from(err: SqlError) -> Self {
        Self::Storage(err)
    }
}
```

Notice how clean those three trait implementations are — zero hidden runtime reflection, zero stack-trace capture overhead on the `InsufficientFunds` path. Because writing `Display`, `Error::source`, and `From` by hand across twenty enums gets repetitive, David Tolnay's `thiserror` crate generates **those exact three trait `impl` blocks at compile time** with zero runtime dependency overhead:

```
// Identical compiled machine code using `thiserror`:
#[derive(Debug, thiserror::Error)]
pub enum TransferError {
    #[error(
        "low funds: have {available_cents}, need {requested_cents}"
    )]
    InsufficientFunds { available_cents: i64, requested_cents: i64 },

    #[error("account {account_id} is frozen by compliance")]
    AccountFrozen { account_id: u64 },

    #[error("self-transfer forbidden on {account_id}")]
    SameSourceAndDestination { account_id: u64 },

    #[error("ledger storage query failed")]
    Storage(#[from] SqlError), // Generates From + Error::source()!
}
```

**Common trap**

Do **not** slap `#[from] std::io::Error` blindly onto a single variant if your function performs three *different* I/O operations (say reading a fee config file, reading a TLS certificate, and writing an audit socket). A bare `io::Error("No such file or directory")` never records *which* path failed. Instead, wrap underlying errors with operational fields:

```
#[error("failed reading fee schedule at {path}")]
FeeConfigRead {
    path: std::path::PathBuf,
    #[source]
    source: std::io::Error,
}
```

and map at the call site with `.map_err(|source| ConfigError::FeeConfigRead { path: p.to_path_buf(), source })?`.

### 8.5.2 Zone 2: Application Entrypoints, Workers, and CLIs Use `anyhow`

At the top of a binary crate — `fn main()`, a nightly settlement batch job, a database migration CLI, or an integration test harness — nobody above you is going to `match` on variants. The job of the top level is to glue together five subsystems (CLI flags, TOML files, TLS certs, PostgreSQL pools, HTTP listeners) and, if startup or batch execution fails, emit a clear causal story for the operator on call.

That is what `anyhow::Error` (a compact single-word boxed trait-object error with causal context chaining and optional backtrace capture) and its `Context` extension trait are built for:

```
use anyhow::{Context, Result};
use std::path::Path;

pub fn bootstrap_fee_engine(config_path: &Path) -> Result<FeeSchedule> {
    let raw_toml = std::fs::read_to_string(config_path).with_context(|| {
        format!("failed reading fee config file {}", config_path.display())
    })?;

    let flat_cents: i64 = raw_toml
        .trim()
        .parse()
        .context("fee config must contain an integer cent amount")?;

    anyhow::ensure!(
        flat_cents >= 0,
        "flat wire fee cannot be negative (got {flat_cents})"
    );
    Ok(FeeSchedule { flat_wire_fee_cents: flat_cents })
}
```

Look at two subtle details in `bootstrap_fee_engine`:

1. **Prefer `.with_context(|| format!(...))` over `.context(format!(...))`.** Remember eager argument evaluation: `.context(format!(...))` formats and heap-allocates the error string *even on the 99.99-percent success path.* Passing a closure to `.with_context(|| ...)` allocates nothing unless an `Err` actually occurred. Use plain `.context("...")` only for zero-allocation `&'static str` literals.

2. **Causal chain output.** When `/etc/ledger/fees.toml` has bad permissions, formatting `err` with {`err:#`} yields:

   ```
   failed reading fee config file /etc/ledger/fees.toml:
   Permission denied (os error 13)
   ```

   capturing both the high-level business intent and the low-level OS kernel cause in a single log line.

### 8.5.3 The Hard Line Between `Result` and `panic!`

Newcomers sometimes treat `.unwrap()` as "quick error handling" during prototyping and accidentally ship those calls to production. In a multi-threaded Tokio/Axum server, a `panic!` inside an HTTP

handler unwinds and aborts that request task (turning a malformed user payload into an instant HTTP `500` crash or dropped socket, and potentially poisoning a `std::sync::Mutex` if a lock guard was held across the panic).

Keep the boundary sharp:

**Focus**

- **Return `Result<T, E>`** whenever failure can be triggered by *anything outside the current function's proven invariants*: network packets, HTTP bodies/headers, CLI args, environment variables, filesystem state, clock drift, database errors, or violated business rules (insufficient balance, expired quote).
- **Use `panic!`, `assert!`, or `.expect("invariant: ...")`** *only* when failure means **a bug already exists in our own program** — a broken internal invariant where continuing could corrupt state or miscalculate money.

Notice why seasoned Rust engineers forbid bare `.unwrap()` in production code (often enforcing `#[deny(clippy::unwrap_used)]`) while permitting `.expect("invariant: ...")` in the rare spots where human reasoning exceeds what the type checker knows:

```
// Compile-time constant IP/header parsing or mathematically proven step:
let loopback: std::net::IpAddr = "127.0.0.1"
    .parse()
    .expect("invariant: literal '127.0.0.1' is valid IPv4 syntax");
```

Writing the word `"invariant: ..."` forces the author to articulate *why* `None`/`Err` is impossible, and gives any future maintainer an immediate explanation if a refactor ever violates that assumption.

## 8.6 Modules, Visibility Capsules, `#[must_use]`, and Rust Testing Culture

All five patterns we have built so far — arenas, newtypes, typestates, view structs, and structured errors — depend on one foundational mechanism to keep their guarantees: **module visibility boundaries**.

### 8.6.1 Privacy Lives at the Module Boundary, Not the Struct Boundary

Here is a rule that catches every Java and C++ programmer off guard the first week they write Rust:

**Common trap**

In Java and C++, `private` means *private to the enclosing `class`*. Two classes in the same file cannot touch each other's `private` fields (unless C++ explicitly grants `friend`).
In Rust, **the unit of encapsulation is the `mod` (module), NOT the `struct`.** Every item and struct field in Rust is private to its enclosing module by default — which means:

1. Any function or sibling struct written inside *the same module* (and any *descendant* child module, such as `mod tests` inside that file) **can read and mutate private fields**

**directly**.

2. Code in *parent* or *cousin* modules **cannot** access private items or construct structs with private fields unless the module explicitly marks them `pub`, `pub(crate)`, or `pub(super)`.

Why did Rust choose module-level privacy? Think back to `PostingView` or `WireOrder<S>`. Often two or three tightly coupled types (`Arena<T>`, `Slot<T>`, and `ArenaIter<'a, T>`) share a single representation invariant. Module-level privacy lets them cooperate without C++ `friend` declarations, *provided you keep invariant-enforcing modules small.* If you dump 3,000 lines of unrelated business logic into one large `ledger.rs` file alongside `PositiveCents`, then all 3,000 lines can bypass `PositiveCents::try_new`, wrap *any* `NonZeroI64` directly (including a negative one), or mutate the private field.

Put core value types (`money.rs`, `iban.rs`, `wire_state.rs`) in dedicated small leaf modules. Those modules act as **invariant capsules**: inside the 60-line module you audit every line that touches `.0`; across the remaining 50,000 lines of your workspace, the compiler enforces the invariant mechanically.

Alongside private (default) and `pub` (exported to the entire world), use `pub(crate)` liberally. Marking internal repository helpers, View Structs, or arena internals `pub(crate)` lets sibling modules inside your crate share them freely while keeping your crate's public API surface minimal and free to evolve without breaking external callers. Likewise, practice *prelude restraint*: export explicit names rather than encouraging `use my_crate::prelude::*;` wildcard imports that obscure where traits and types come from during code review.

### 8.6.2 Compiler Guardrails with `#[must_use]`

Have you ever wondered how `rustc` knows to print `warning: unused 'Result' that must be used` whenever you forget `?` on a fallible call? There is no compiler special-casing for `Result`. In the standard library, `Result` is simply annotated with the attribute `#[must_use]`.

You can — and should — place `#[must_use = "reason"]` on your own pure value transformations, builders, and domain receipts. Consider an immutable money operation or settlement receipt:

```
#[must_use = "SettlementReceipt must be persisted to the outbox"]
pub struct SettlementReceipt {
    pub wire_id: u64,
    pub settled_cents: i64,
}

#[derive(Copy, Clone, Debug, PartialEq, Eq)]
pub struct Cents(pub i64);

impl Cents {
    #[must_use = "with_fee returns a new Cents without modifying self"]
    pub const fn with_fee(self, fee: Cents) -> Cents {
        Cents(self.0 + fee.0)
    }
}
```

If a teammate accustomed to mutable Java objects writes `total.with_fee(wire_fee);` as a standalone statement expecting `total` to mutate in place, the compiler immediately flags the bug with

your exact custom explanation.

### 8.6.3 Crate Layering for Production Services

As a Rust backend grows beyond ten thousand lines (as our financial backend will in Part V), organizing code into a multi-crate **Cargo workspace** reinforces clean architecture at the dependency-graph level because Cargo strictly forbids circular dependencies between crates:

1. `ledger-domain` **(Pure Core Crate):** Holds Newtypes (`AccountId`, `PositiveCents`, `Iban`), Typestate protocols (`WireOrder<S>`), Arena/chart-of-accounts logic, fee-allocation math, and structured `thiserror` domain errors. Most importantly, `ledger-domain` has **zero network, Tokio, HTTP, or SQL dependencies**. It compiles in seconds, and all its tests run deterministically in memory.
2. `ledger-db` **(Storage Adapter Crate):** Depends on `ledger-domain` and `sqlx`. Translates PostgreSQL rows into validated domain types (`AnyWireOrder`, `PositiveCents`) and maps database errors into domain storage errors.
3. `ledger-api` **(Transport & Binary Entrypoint Crate):** Depends on `ledger-domain`, `ledger-db`, `axum`, `tokio`, and `anyhow`. Parses untrusted JSON payloads at the perimeter into `ledger-domain` smart constructors, invokes application workflows, and maps `TransferError` variants to HTTP status codes.

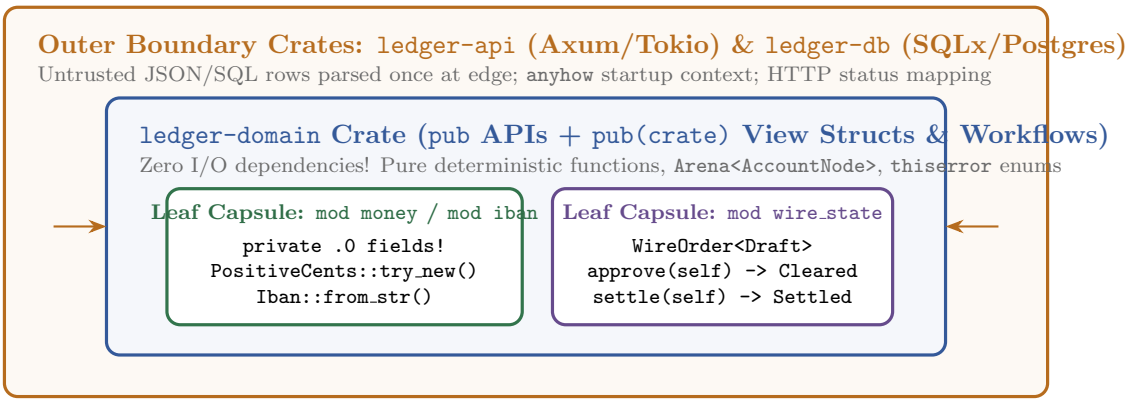



Figure 8.4: Concentric invariant-capsule architecture. Untrusted inputs are parsed at the outer HTTP and database boundary into proof-carrying Newtypes and Typestates owned by small leaf modules inside a zero-I/O `ledger-domain` core.

### 8.6.4 Rust's Four-Tier Testing Culture and Conservation Laws

Rust ships a first-class test runner inside `cargo test` with zero external framework setup, and idiomatic crates combine four complementary layers of verification:

1. **Co-located Unit Tests (`#[cfg(test)] mod tests`):** Placed at the bottom of each source file. The `#[cfg(test)]` conditional-compilation gate completely strips the module from release builds, while its position as a child module allows testing private helper invariants alongside public constructors.

2. **Black-Box Integration Tests (`tests/*.rs`):** Every file placed in a crate's top-level `tests/` directory is compiled by Cargo as a *separate external crate* linking against your library. It cannot touch private or `pub(crate)` items — proving that external callers can actually use your public API cleanly and cannot subvert capsule invariants.

3. **Executable Documentation Tests (`///` doc-comments):** Every fenced Rust snippet inside a `///` doc-comment is compiled and executed during `cargo test --doc`. Documentation examples in Rust never silently drift out of date when signatures evolve.

4. **Property and Conservation-Law Tests:** Financial math has sharp algebraic invariants (such as *conservation of money*: splitting a fee across $N$ sub-accounts must never create or destroy a single cent).

Let us see conservation-law testing in action on a classic ledger problem: splitting a total fee of `total_cents` equally across `n` participating sub-accounts without floating-point rounding drift. Using integer division and remainder (`div_euclid` and `rem_euclid`), the first `rem` accounts receive `q + 1` cents and the remaining `n - rem` accounts receive `q` cents:

```rust
/// Splits `total_cents >= 0` across `buckets > 0` accounts so the shares
/// sum to `total_cents` to the exact cent with at most 1-cent difference.
///
/// ```
/// # use my_ledger::allocate_fee_shares;
/// assert_eq!(allocate_fee_shares(100, 3), vec![34, 33, 33]);
/// ```
pub fn allocate_fee_shares(total_cents: i64, buckets: usize) -> Vec<i64> {
    assert!(total_cents >= 0, "invariant: fee total must be non-negative");
    assert!(buckets > 0, "invariant: bucket count must be positive");

    let n = buckets as i64;
    let base = total_cents.div_euclid(n);
    let remainder = total_cents.rem_euclid(n) as usize;

    (0..buckets)
        .map(|i| if i < remainder { base + 1 } else { base })
        .collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn fee_allocation_conserves_exact_cents_across_sweep() {
        // Verify conservation + fairness across 5,000 inputs
        // (For huge input spaces, pair this with `proptest`!)
        for total in 0..=250_i64 {
            for buckets in 1..=20_usize {
                let shares = allocate_fee_shares(total, buckets);
                assert_eq!(shares.len(), buckets);

                // Law 1: Exact cent conservation (0 cents gained/lost)
                let sum: i64 = shares.iter().copied().sum();
                assert_eq!(sum, total);
```

```
37
38                  // Law 2: Fairness (max share - min share <= 1 cent)
39                  let min = *shares.iter().min().expect("buckets >= 1");
40                  let max = *shares.iter().max().expect("buckets >= 1");
41                  assert!(max - min <= 1);
42              }
43          }
44      }
45  }
```

### 8.6.5 Summary: Translating Instincts into Idiomatic Rust

Table 8.1 condenses every architectural shift from this chapter into a quick desk reference you can consult whenever code starts feeling awkward or borrow errors pile up.

| Java 6 / C++ Instinct | Idiomatic Rust Pattern | Hardware & Type Payoff |
|---|---|---|
| Cyclic `Rc<RefCell<Node>>` object graphs | Contiguous `Vec` Arena + `Copy` generational keys | Sequential cache lines, zero cycle leaks, no `BorrowMutError` |
| Raw `u64`/`i64`/`&str` params + repeated `if` checks | `#[repr(transparent)]` Newtypes + smart constructors | Register-identical ABI, 0-cost compile-time proof tokens |
| Mutable `boolean` flags + `IllegalStateException` | `Order<S>` Typestate + consuming `self` moves | Illegal order & double-execution caught before binary runs |
| Heterogeneous DB rows fighting Typestate | `enum AnyOrder` wrapping `Order<S>` variants | Runtime DB hydration + static safety inside `match` arms |
| `&String`, `&Vec<T>`, `&Box<T>` function params | `&str`, `&[T]`, `&T`, `impl AsRef<Path>` | Removes double pointer hop; accepts literals & sub-slices |
| Always `.to_string()` on mostly clean strings | `Cow<'a, str>` (`Borrowed` vs `Owned`) | Zero `malloc` on 99-percent clean fast path |
| God-struct `&mut self` locking disjoint fields | Sub-struct methods or explicit View Structs | Modular field-level borrowing without `RefCell` |
| Stringly errors or `.unwrap()` on user input | `thiserror` in domain, `anyhow` at app top | Exhaustive HTTP error mapping + rich operator causal logs |

Table 8.1: Summary of anti-patterns and their idiomatic Rust architectural replacements.

Part V uses each of these patterns directly: generational handles for in-memory account graphs, validated newtypes for amounts and IBANs at the HTTP boundary, typestate for transfer settlement, `thiserror` and `anyhow` for the two error zones, and conservation-law tests for the ledger.

## 8.7 Exercises

**Exercise 8.1** (Refactoring `Rc<RefCell<OrderNode>>` to a Generational Arena)**.** A teammate ported a Java 6 parent-child order-slicing graph (where cancelling a parent institutional order cancels all active child slice orders, and filling a child slice updates the parent's `remaining_qty`) using

`Rc<RefCell<OrderNode>>` and `Weak<RefCell<OrderNode>>`. Calling `fill_slice` mutably borrows a child, upgrades `parent`, and calls `parent.borrow_mut().reconcile_children()`, triggering a runtime `BorrowMutError` panic.

(a) Replace the pointer graph with an `OrderArena` owning `Vec<Slot<OrderNode>>` and an 8-byte `Copy` handle `OrderKey { index: u32, generation: u32 }`.

(b) Implement `insert`, `remove` (with embedded free-list and saturating generation increment), `get`, `get_mut`, and `get_two_mut` using `split_at_mut`.

(c) Implement `fill_slice(&mut self, slice: OrderKey, qty: u64)` and write two unit tests: one verifying parent `remaining_qty` rollup without panicking, and a second proving that removing a slice and inserting a new order into the recycled slot causes the old `OrderKey` to return `None` (ABA protection).

**Exercise 8.2** (Zero-Cost ABA `RoutingNumber` Newtype)**.** US Automated Clearing House (ACH) transfers identify banks via a 9-digit ABA routing transit number $d_1d_2d_3d_4d_5d_6d_7d_8d_9$. A string is a valid ABA routing number if and only if it consists of exactly 9 ASCII digits (`'0'..='9'`) and satisfies the weighted checksum identity

$$\big(3(d_1 + d_4 + d_7) + 7(d_2 + d_5 + d_8) + (d_3 + d_6 + d_9)\big) \bmod 10 \;=\; 0.$$

(a) Inside a dedicated `pub mod routing`, define a transparent wrapper `RoutingNumber([u8; 9])` marked `#[repr(transparent)]` with a *private* inner array and a structured `RoutingError` enum with three variants: `WrongLength(usize)`, a `NonDigit` variant recording the offending position and character, and `ChecksumMismatch`.

(b) Implement `FromStr`, `TryFrom<&str>`, `Display`, and `as_str(&self) -> &str`. (Hint: once parse-time checks prove all 9 bytes lie in `b'0'..=b'9'`, calling `from_utf8(&self.0)` followed by `.expect("invariant: 9 ASCII digits")` cannot fail) Use `.rem_euclid(10)` for the checksum remainder.

(c) Write unit tests confirming `size_of::<RoutingNumber>() == 9`, accepting a valid routing number (`"111000025"`), and rejecting a single-digit typo and an adjacent-digit transposition. (Weights 3 and 7 catch every single-digit error but miss swaps of two digits that differ by exactly 5; explain why.)

**Exercise 8.3** (Compile-Time `BatchSettlement<S>` Typestate + Runtime Envelope)**.** Model an end-of-day clearing batch that progresses strictly through three states: `Open` (accepts `push_posting(&mut self, AccountId, PositiveCents)`), `Sealed` (carries a 32-byte SHA-256 batch digest and frozen total cents; forbids adding more postings), and `Submitted` (carries a clearinghouse acknowledgement ID).

(a) Implement `BatchSettlement<S>` with consuming `self` transitions: `seal(self)` returning `Result<BatchSettlement<Sealed>, EmptyBatchError>`, and `submit(self, ack_id: u64)` returning a pair `(BatchSettlement<Submitted>, SettlementReceipt)`. Mark `SettlementReceipt` with `#[must_use]`.

(b) Explain in two sentences why taking `self` by value in `submit` prevents double-submission in a way a Java 6 `SealedBatch.submit()` method cannot.

(c) Define a runtime `pub enum AnyBatch` wrapping all three typestates so an SQL loader can return `Vec<AnyBatch>` from mixed database rows and dispatch only `Open` batches to `seal()`.

**Exercise 8.4** (Zero-Allocation Fast Path with `Cow<'_, str>`)**.** Ledger account codes follow canonical format `"US-OPERATING-01"`: non-empty ASCII uppercase letters, digits, and single `'-'` separators, with no leading/trailing spaces and no underscores `'_'`. Upstream CSV files usually arrive already canonical, yet occasionally contain lowercase letters (`"us-operating-01"`) or underscores (`"US_OPERATING_01"`).

(a) Write `normalize_account_code(raw: &str)` returning `Result<Cow<'_, str>, CodeError>` so that it trims ASCII whitespace, validates syntax, and returns `Ok(Cow::Borrowed(trimmed))` with **zero heap allocations** whenever `trimmed` is already canonical.

(b) Only when lowercase ASCII letters or `'_'` appear, allocate a single `String` of exact capacity, uppercasing letters and replacing `'_'` with `'-'`, returning `Ok(Cow::Owned(...))`.

(c) Write a unit test checking `matches!(res, Cow::Borrowed(_))` and comparing raw pointers via `std::ptr::eq(res.as_bytes().as_ptr(), input.as_bytes().as_ptr())` to prove that clean inputs share the exact same memory address with zero copying.

**Exercise 8.5** (Split Borrows and Two-Zone Error Architecture)**.** Consider a `ReconciliationRunner` struct owning three fields: two maps from `AccountId` to `i64` named `ledger_balances` and `bank_tolerances`, and a `Vec<Discrepancy>` named `discrepancies`.

(a) Show why `check_row(&mut self, acct: AccountId, cents: i64, max_drift: &i64)` fails to compile when called as `self.check_row(id, cents, tol)` while iterating over `&self.bank_tolerances`. Refactor `ReconciliationRunner` using an explicit `ReconcileView<'a>` borrow-projection struct so the batch loop compiles cleanly with zero cloning and zero `RefCell`.

(b) Design a structured Zone 1 `ReconcileError` enum (implementing `Display`, `Error`, and `From<CsvParseError>`) for the library crate, and write a Zone 2 CLI entrypoint `run_cli(csv_path: &Path) -> anyhow::Result<()>` using `.with_context(|| ...)` so file-path context is attached lazily only on failure.

# Part IV

# Modern Type Systems and Concurrency: TypeScript, Kotlin, and Go

# Chapter 9

# TypeScript From the Ground Up: Structural Subtyping, Set Theory, and Erase-to-JS

In Parts II and III we lived inside compilers where types govern physical memory. When you write `struct Posting { account_id: u64, cents: i64 }` in Rust or C++, that declaration decides exact byte offsets, register assignments, stack-frame sizes, and machine instructions. Even in Java 6, where generics lose their type arguments at compile time, every object on the heap still carries a 16-byte header whose `klass` pointer names the exact class that allocated it; try an invalid cast at runtime and the JVM halts the instruction with a `ClassCastException`.

Welcome to Part IV, where we step outside Rust to study three modern languages—TypeScript, Kotlin, and Go—that made different engineering bets about types, memory, and concurrency. We begin with TypeScript because no mainstream language looks more familiar on the surface to a Java programmer while operating on more different mathematical foundations underneath.

Start with the keywords in a TypeScript file: `interface`, `class`, `implements`, `extends`, `private`, `readonly`, `<T>`. Every single one looks like Java. If you sit down to write TypeScript carrying your Java 6 mental model, everything feels fine for about twenty minutes—right until two unrelated types become interchangeable without an `implements` clause, `instanceof` refuses to compile against an `interface`, `keyof (A | B)` evaluates to an intersection instead of a union, a mutable `Dog[]` slips silently into an `Animal[]`, and a malformed JSON payload passes straight through `as Account` into production without throwing a cast exception.

None of these behaviors are quirks to memorize one by one. Every one of them follows from two foundational design choices:

1. **Structural subtyping (types as mathematical sets of shapes):** TypeScript checks whether a value has the required *shape*, not which named declaration created it. A type is simply a mathematical set of runtime JavaScript values, and subtyping ($S <: T$) is ordinary set inclusion ($S \subseteq T$).

2. **Total type erasure to JavaScript:** the TypeScript compiler (`tsc`) emits plain JavaScript with *zero* runtime type descriptors. Every `interface`, `type` alias, union, generic parameter, and `private` modifier vanishes before V8 or Node.js sees a single byte of code.

With both principles in hand, TypeScript is best understood as a set-theoretic static analyzer layered over a dynamic runtime. The rest of the chapter builds up each half.

## 9.1 Two Mental Shifts from Java 6: Shape Typing and Total Erasure

Suppose we are building the browser dashboard and Node.js API gateway that sit in front of a double-entry financial ledger. Consider two class declarations in Java 6:

```
// Java 6: nominal typing -- names and explicit hierarchy decide compatibility
public final class InvoiceTotal {
    private final long cents;
    public InvoiceTotal(long cents) { this.cents = cents; }
    public long getCents() { return cents; }
}

public final class LedgerBalance {
    private final long cents;
    public LedgerBalance(long cents) { this.cents = cents; }
    public long getCents() { return cents; }
}
```

In Java 6, `InvoiceTotal` and `LedgerBalance` have identical memory layouts (a 16-byte JVM object header plus an 8-byte `long`) and identical method signatures. Yet you cannot pass an `InvoiceTotal` to a method expecting a `LedgerBalance`. Even if you later introduce an interface `HasCents { long getCents(); }`, existing classes in third-party JARs cannot satisfy `HasCents` unless their original authors wrote `implements HasCents` on the class header. Compatibility is *nominal*: it lives in declared names and explicit lineage. Rust `structs` and `traits` are nominal in the same sense—two structs with identical fields are distinct types, and a type satisfies a trait only when an explicit `impl Trait for Type` block says so (though Rust, unlike Java 6, lets you write that `impl` block outside the struct declaration).

Now write the analogous declarations in TypeScript:

```
// TypeScript: structural typing -- the set of properties decides compatibility
interface InvoiceTotal {
  readonly cents: bigint;
}

interface LedgerBalance {
  readonly cents: bigint;
}

function formatUsd(amount: InvoiceTotal): string {
  const sign = amount.cents < 0n ? '-' : '';
  const abs = amount.cents < 0n ? -amount.cents : amount.cents;
  const dollars = abs / 100n;
  const rem = (abs - dollars * 100n).toString().padStart(2, '0');
  return `${sign}$${dollars}.${rem}`;
}
```

```
17
18 const bal: LedgerBalance = { cents: 12500n };
19 console.log(formatUsd(bal));              // Compiles! "$125.00"
20 console.log(formatUsd({ cents: -450n })); // Plain object literal compiles! "-$4.50"
```

Consider `formatUsd(bal)`. Nobody declared that `LedgerBalance` extends `InvoiceTotal`, and the anonymous literal `{ cents: -450n }` has no class name at all. TypeScript accepts both calls because `formatUsd` asks only one question: *is the argument guaranteed to be an object possessing a `cents` property whose value is a `bigint`?* Both `LedgerBalance` and `{ cents: -450n }` answer yes, so both belong to the set of values accepted by `InvoiceTotal`.

**Definition 9.1** (Nominal vs. Structural Subtyping)**.** In a *nominally typed* language (Java 6 classes and interfaces, C++ class inheritance, Rust structs and traits, Kotlin classes), type $S$ is a subtype of $T$ if and only if the program explicitly declares that named relationship ($S$ `extends` or `implements` $T$).

In a *structurally typed* language (TypeScript types and interfaces, OCaml object types, Go interfaces on method sets), type $S$ is a subtype of $T$ ($S <: T$) whenever every member required by the structure of $T$ is present with a compatible type in $S$, regardless of what $S$ and $T$ are named or where they were declared.

**Intuition**

**Why did TypeScript choose structural typing?** Because TypeScript did not invent its own runtime; it was designed by Anders Hejlsberg and his team at Microsoft (first released in 2012) to type-check idiomatic JavaScript as JavaScript programmers actually write it. In JavaScript, functions routinely build and return ad-hoc object literals `{ ok: true, txId: 'tx_91' }`, compose configuration bags across libraries that have never heard of each other, and pass callbacks without ceremony. Forcing every object literal to instantiate a nominal class hierarchy would have turned JavaScript into Java. Structural subtyping lets TypeScript assign exact static types to plain data shapes without changing a single runtime allocation.

### 9.1.1 Mental Shift 2: Total Erasure and What V8 Actually Sees

Now let us follow `formatUsd` across the compiler boundary. When you run `tsc --target es2022`, here is the exact JavaScript that emerges:

```
1  // Emitted JavaScript: interfaces, type annotations, and readonly are gone
2  function formatUsd(amount) {
3    const sign = amount.cents < 0n ? '-' : '';
4    const abs = amount.cents < 0n ? -amount.cents : amount.cents;
5    const dollars = abs / 100n;
6    const rem = (abs - dollars * 100n).toString().padStart(2, '0');
7    return `${sign}$${dollars}.${rem}`;
8  }
9
10 const bal = { cents: 12500n };
11 console.log(formatUsd(bal));
12 console.log(formatUsd({ cents: -450n }));
```

Compare the two listings line by line. Both `interface InvoiceTotal` and `interface LedgerBalance` evaporated into zero bytes. The parameter annotation `: InvoiceTotal`, the return type `: string`, and the variable annotation `: LedgerBalance` vanished too. Even `readonly` disappeared: at runtime, nothing stops untyped JavaScript code from mutating `bal.cents = 0n` unless you explicitly call `Object.freeze(bal)` at the value level.

Figure 9.1 compares what survives compilation in Java 6, Rust, and TypeScript.

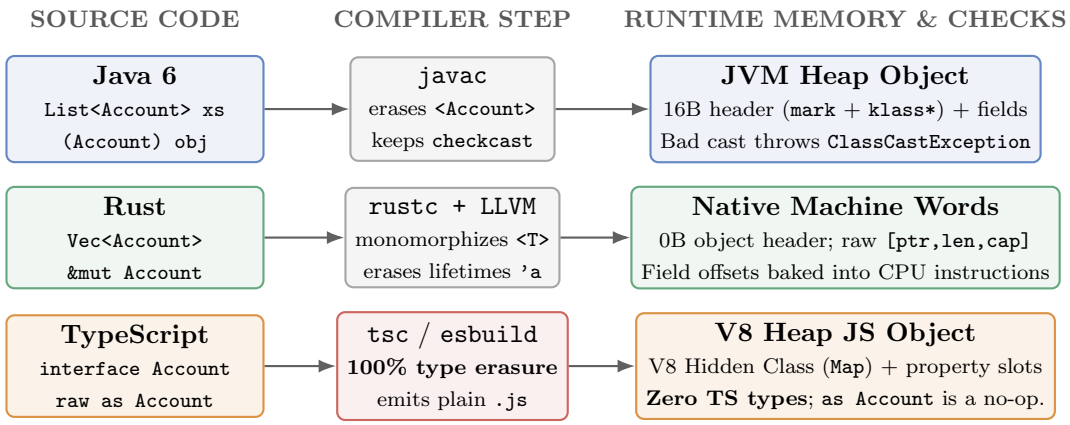



Figure 9.1: What survives compilation in Java 6, Rust, and TypeScript. Java 6 erases generic type arguments but keeps every object's runtime `klass` pointer and emits `checkcast` bytecode. Rust bakes exact field offsets into machine code and needs no object header. TypeScript erases *all* static type constructs, leaving plain JavaScript values managed by V8 hidden classes.

Java 6 developers often hear "TypeScript uses type erasure" and assume it means the same thing as Java 6 generics erasure. It goes much deeper than that:

- **In Java 6, casts are runtime-checked operations.** Writing `(LedgerBalance) obj` compiles to the JVM bytecode instruction `checkcast #LedgerBalance`, which inspects `obj`'s runtime `klass` word and throws `ClassCastException` immediately if `obj` is actually a `String` or `HashMap`.
- **In TypeScript, `x as LedgerBalance` emits zero instructions.** It is not a conversion and it is not a checked cast. It is a compile-time note telling `tsc`: "stop checking `x` and pretend it belongs to `LedgerBalance`." If `x` at runtime is `{ cents: "not-a-bigint" }`, `x as LedgerBalance` succeeds silently and your program fails later when arithmetic hits a string.
- **You cannot use `instanceof` against an `interface` or `type`.** Writing `if (x instanceof LedgerBalance)` fails at compile time because JavaScript's `instanceof` walks an object's prototype chain (`Object.getPrototypeOf(x)`) looking for a runtime constructor's `.prototype` object. Since `interface LedgerBalance` was erased before runtime, no `LedgerBalance` value exists in JavaScript for `instanceof` to inspect.

Under the hood

**What does V8 actually store in memory for `{ cents: 12500n }`?** Even though TypeScript types vanish, V8 does not represent every JavaScript object as a slow `HashMap<String, Object>`. Internally, V8 attaches a pointer to a hidden descriptor called a *Map* (often called

a *shape* or *hidden class*) at word 0 of every JS heap object. When V8 executes { `cents: 12500n` }, it assigns the Hidden Class "object with property `cents` at in-object slot 0." When `formatUsd` reads `amount.cents`, V8 uses an *inline cache*: if the Hidden Class pointer at `amount` matches the Hidden Class seen on previous calls, V8 loads slot 0 at a fixed byte offset in a couple of instructions—almost like a Java field load. However, if callers pass objects whose properties were added in six different orders ({ `a, b` } vs. { `b, a` }), V8 sees distinct Hidden Classes, the inline cache turns *megamorphic*, and property access falls back to a slower dictionary lookup. Writing consistent TypeScript interfaces naturally nudges authors to construct objects with consistent property shapes, accidentally helping V8's JIT optimizer even though V8 never reads a single line of TypeScript.

Common trap

**Three TypeScript constructs that look erased, and aren't.** Almost everything in TypeScript's type syntax erases cleanly, with three legacy exceptions inherited from TypeScript's pre-ES6 days: `enum`, parameter properties (`constructor(public readonly id: string)`), and legacy `namespace` blocks. A non-`const` TypeScript `enum Status { Open, Closed }` compiles into an IIFE that allocates a real JavaScript lookup object at runtime—and numeric enums even create reverse number-to-string mappings so `Status[0] === 'Open'`. Modern TypeScript codebases (and Node.js's built-in `--experimental-strip-types` loader, which strips types directly without code generation) strongly prefer union string literals `type Status = 'open' | 'closed'` over TypeScript `enum`. Every union, interface, and type alias erases cleanly by simple whitespace stripping.

## 9.2 Types as Mathematical Sets: From `never` to `unknown`

In Java 6 or C++, we usually picture types as blueprints for constructing instances, arranged in a family tree of superclasses and subclasses. In TypeScript, a cleaner picture makes every rule predictable:

Focus

**The Set-Theory Mental Model.** Every static type $T$ in TypeScript denotes a **set $[\![T]\!]$ of runtime JavaScript values**. Saying "expression $e$ has type $T$" ($e : T$) means that whatever value $v$ expression $e$ evaluates to at runtime (assuming termination), $v \in [\![T]\!]$. Saying "$S$ is a subtype of $T$"—written `S extends T` in TypeScript and $S <: T$ in type theory—means nothing more and nothing less than **set inclusion**:

$$S <: T \quad \Longleftrightarrow \quad [\![S]\!] \subseteq [\![T]\!].$$

Whenever $S <: T$, any value in $[\![S]\!]$ is already an element of $[\![T]\!]$, so assigning a variable of type $S$ to a variable of type $T$ requires zero runtime conversion.

Let us climb the entire TypeScript type lattice from the smallest possible set to the largest, checking every rung against set algebra.

### 9.2.1 Cardinality 0: `never` Is the Empty Set $\emptyset$

In set theory, there is a unique set with zero elements: the empty set $\emptyset$. What type denotes $\emptyset$—the set of runtime values that has *no inhabitants at all*? TypeScript calls it `never` (the exact counterpart of Rust's `!` never type and Kotlin's `Nothing`).

Two consequences follow from $[\![\texttt{never}]\!] = \emptyset$:

1. **Is `never` assignable to every type $T$?** Yes. In set theory, $\emptyset \subseteq A$ holds vacuously for *every* set $A$ (every element of $\emptyset$ is in $A$, because there are no elements in $\emptyset$ to violate the condition). Thus `never` $<: T$ for all $T$: `never` sits at the very bottom of the subtype lattice, which is why type theorists call it the *bottom type* ($\bot$).
2. **Can any non-empty type $T$ be assigned to `never`?** Never. For any set $A$, $A \subseteq \emptyset$ holds if and only if $A = \emptyset$. If a variable $x$ still has even one possible runtime value (say $x$ : `'chargeback'`), assigning `const check: never = x` is a compile-time type error because $\{\texttt{'chargeback'}\} \not\subseteq \emptyset$.

When does a real expression have type `never`? In two everyday situations: first, as the return type of a function that never returns normally (because it unconditionally throws an exception or runs an infinite event loop); second, as the narrowed type of a variable after you have eliminated every possible case in a `switch` or `if/else` chain.

```
function failInvariant(msg: string): never {
  throw new Error(`Ledger invariant violated: ${msg}`);
}

// Because never <: bigint, failInvariant() can stand inside a bigint expression!
function requirePositive(cents: bigint): bigint {
  return cents > 0n ? cents : failInvariant(`expected > 0, got ${cents}`);
}
```

Consider why `cents > 0n ? cents : failInvariant(...)` has type `bigint` rather than `bigint | void`. The true branch produces a value in $[\![\texttt{bigint}]\!]$; the false branch produces a value in $[\![\texttt{never}]\!] = \emptyset$. The conditional expression's type is the union of both branches:

$$[\![\texttt{bigint}]\!] \cup [\![\texttt{never}]\!] \;=\; [\![\texttt{bigint}]\!] \cup \emptyset \;=\; [\![\texttt{bigint}]\!].$$

In Java 6, `failInvariant` would have to return `void` or take a dummy generic `<T> T` return type, and you could never place a `void` call inside a ternary expression. Set algebra makes `bigint | never = bigint` automatic.

### 9.2.2 Cardinality 1 and Finite Sets: Literal Types and Unions ($A \cup B$)

Above $\emptyset$ sit the *singleton sets*—types containing exactly one runtime value:

$$[\![\texttt{undefined}]\!] = \{\texttt{undefined}\}, \qquad [\![\texttt{null}]\!] = \{\texttt{null}\},$$
$$[\![\texttt{'credit'}]\!] = \{\texttt{'credit'}\}, \qquad [\![\texttt{200}]\!] = \{200\}, \qquad [\![\texttt{true}]\!] = \{\texttt{true}\}.$$

In Java 6 or C++, `"credit"` and `200` are strictly *values*, never *types*. The smallest type containing `"credit"` in Java 6 is `java.lang.String`, an infinite set containing every possible Unicode string

(plus `null`). Because TypeScript treats types as sets, nothing stops a type from being the 1-element set {`'credit'`}. Since {`'credit'`} $\subsetneq$ ⟦`string`⟧, `'credit'` is automatically a subtype of `string`: every function expecting a `string` accepts a `'credit'`, while a function expecting `'credit' | 'debit'` rejects `'refund'` at compile time.

Combining sets with the union operator `|` corresponds directly to mathematical set union:

$$[\![A \mid B]\!] \;=\; [\![A]\!] \cup [\![B]\!].$$

In fact, TypeScript's primitive `boolean` type is not an indivisible atom at all—inside the compiler, `boolean` is the 2-element union `true | false`. Similarly:

```
type EntrySide = 'debit' | 'credit';                // Cardinality 2
type HttpSuccess = 200 | 201 | 204;                 // Cardinality 3
type NullableOwner = string | null;                 // Infinite set union { null }

let mutableSide = 'credit';    // Widened to `string` because `let` can be reassigned!
const fixedSide = 'credit';    // Narrowed to singleton literal type `'credit'`!

const routeConfig = {
  method: 'POST',
  retries: 3,
} as const;
// Type of routeConfig is `{ readonly method: 'POST'; readonly retries: 3 }`
```

Notice the difference between `let` and `const`. When you write `let mutableSide = 'credit'`, TypeScript knows you will likely reassign `mutableSide = 'debit'` two lines later, so it widens the inferred type from {`'credit'`} to ⟦`string`⟧. When you write `const fixedSide = 'credit'`, the binding can never point to a different string, so TypeScript infers the exact 1-element set `'credit'`. Appending `as const` to an object or array literal applies that same non-widening rule recursively to every nested property and marks every slot `readonly`.

**Common trap**

**Never compile TypeScript without `"strict": true` (especially `strictNullChecks`).** In early TypeScript (before TypeScript 2.0 in 2016), `null` and `undefined` behaved like Java 6's `null`: they were silent inhabitants of *every* type, so ⟦`string`⟧ implicitly included `null` and `undefined`—reproducing Tony Hoare's "billion-dollar mistake." Turning on `"strict": true` in `tsconfig.json` enables `strictNullChecks`, which evicts `null` and `undefined` into their own disjoint singleton sets. Under `strictNullChecks`, `string` contains *only* strings; if a value might be absent, you must write the union `string | null` or `string | undefined` explicitly.

### 9.2.3 Open Object Sets: Why More Properties Mean a *Smaller* Set

Here is the first place where Java 6 intuition conflicts with set theory. Consider two TypeScript object types:

```
interface Identifiable {
  id: string;
}

```

```ts
interface AccountSummary {
  id: string;
  owner: string;
  balanceCents: bigint;
}
```

Ask a room of programmers: "Which type is *larger*—`Identifiable` or `AccountSummary`?" A C or Java programmer instinctively pictures memory size on the heap and answers "`AccountSummary` is larger; it has three fields instead of one."

Now ask the set-theory question: **which set of runtime values has more elements?**

- $[\![\texttt{Identifiable}]\!]$ is the set of *all* non-null objects that possess at least a property `id` whose value is a `string`. That set includes `{ id: 'acc_1' }`, `{ id: 'acc_1', active: true }`, and every `AccountSummary` ever created.
- $[\![\texttt{AccountSummary}]\!]$ is the set of objects that possess `id: string` *and* `owner: string` *and* `balanceCents: bigint`. Every new required property adds another filter condition, *shrinking* the set of qualifying values:

$$[\![\texttt{AccountSummary}]\!] \subsetneq [\![\texttt{Identifiable}]\!], \quad \text{so} \quad \texttt{AccountSummary} <: \texttt{Identifiable}.$$

Type theorists call this **width subtyping**: a wider record (more properties) is a *subtype* (smaller set) of a narrower record (fewer properties). One consequence follows directly: **TypeScript object types are open, not sealed.** Knowing `x : Identifiable` tells you that `x.id` is a `string`; it does *not* promise that `id` is the *only* property on `x` at runtime.

```ts
function logId(entity: Identifiable): void {
  // Why does Object.keys(entity) return `string[]` instead of `('id')[]`?
  for (const key of Object.keys(entity)) {
    console.log(key);
  }
}

const acc: AccountSummary = { id: 'acc_7', owner: 'Linus', balanceCents: 9000n };
logId(acc); // Legal because AccountSummary <: Identifiable!
// Prints: "id", "owner", "balanceCents" -- NOT just "id"!
```

This explains why `Object.keys(entity)` in TypeScript's standard library returns `string[]` rather than `(keyof Identifiable)[]`. Because `AccountSummary <: Identifiable`, `acc` is a completely valid `Identifiable`, and at runtime `Object.keys(acc)` returns `['id', 'owner', 'balanceCents']`. If TypeScript claimed `Object.keys(entity)` contained only `'id'`, any code indexing a lookup table typed only for `'id'` would crash on `'owner'`.

**Common trap**

**Excess Property Checking Is a Lint Rule on Fresh Literals, Not Sealed Types.** Wait—if `AccountSummary` has extra properties and assigns cleanly to `Identifiable`, why does TypeScript reject `const e: Identifiable = { id: 'acc_7', owenr: 'Linus' }` when you write an object literal directly? Because when you type a *fresh object literal* straight into a typed slot, nobody else holds a reference to that literal; any extra property written right there (like a misspelled optional field `owenr` instead of `owner`) would be dead, unreachable

data and is almost certainly a typo. So `tsc` runs a special *excess property check* on fresh literals only. Assign that exact same literal to an intermediate variable first (`const tmp = { id: 'acc_7', owenr: 'Linus' }; const e: Identifiable = tmp;`), and it compiles without complaint because set inclusion $[\![\texttt{typeof tmp}]\!] \subseteq [\![\texttt{Identifiable}]\!]$ holds. Never confuse excess property checking with sealed objects: extra properties can *always* enter through subtype variables.

### 9.2.4 Intersections ($A \cap B$) and the Dual Algebra of `keyof`

Just as `A | B` forms the set union $[\![A]\!] \cup [\![B]\!]$, the intersection operator `A & B` forms the mathematical set intersection:

$$[\![A \,\&\, B]\!] \;=\; [\![A]\!] \cap [\![B]\!].$$

Let us test `&` on both primitives and object shapes to make sure our intuition never slips:

- **Intersecting disjoint primitives:** What is `string & number`? Which JavaScript runtime value is simultaneously a primitive `string` and a primitive `number`? None. $[\![\texttt{string}]\!] \cap [\![\texttt{number}]\!] = \emptyset = [\![\texttt{never}]\!]$. Indeed, `tsc` simplifies `string & number` directly to `never`.
- **Intersecting overlapping unions:** What is `('a' | 'b') & ('b' | 'c')`? $\{\texttt{'a'}, \texttt{'b'}\} \cap \{\texttt{'b'}, \texttt{'c'}\} = \{\texttt{'b'}\}$, so `tsc` simplifies it to the literal type `'b'`.
- **Intersecting two object types:** Let `HasId` be `{ id: string }` and let `Timestamped` be `{ createdAt: Date }`. What is `HasId & Timestamped`? Beginners often trip here and say: "Wait—`HasId & Timestamped` has *both* `id` and `createdAt`, so didn't it take the *union* of their properties?" Compare the sets of values. $[\![\texttt{HasId}]\!]$ is all objects with `id: string`; $[\![\texttt{Timestamped}]\!]$ is all objects with `createdAt: Date`. Which runtime objects belong to the intersection of those two sets? Precisely those objects that satisfy *both* membership requirements—meaning they carry *both* `id: string` and `createdAt: Date`. Intersecting the *value sets* unions the *required property sets*.

That exact inversion between value sets and property key sets gives a short theorem about `keyof`. Recall that for any type $T$, `keyof T` denotes the union of property keys known to exist on *every* runtime value $v \in [\![T]\!]$.

**Theorem 9.1** (Contravariant Key Duality of Unions and Intersections)**.** *For any two object types $A$ and $B$ in TypeScript (assuming compatible property types so $A \,\&\, B \neq$ `never`), the* `keyof` *operator swaps unions and intersections, just like De Morgan's laws:*

$$\texttt{keyof}\ (A \mid B) \;=\; (\texttt{keyof}\ A) \,\&\, (\texttt{keyof}\ B), \tag{9.1}$$

$$\texttt{keyof}\ (A \,\&\, B) \;=\; (\texttt{keyof}\ A) \mid (\texttt{keyof}\ B). \tag{9.2}$$

*Proof.* A key $k$ belongs to `keyof` $T$ if and only if accessing $v[k]$ is guaranteed to find a declared property on *every* value $v \in [\![T]\!]$ ($\forall v \in [\![T]\!],\ k \in \text{props}(v)$).

For (9.1), take $T = A \mid B$, so $[\![A \mid B]\!] = [\![A]\!] \cup [\![B]\!]$. Requiring $\forall v \in [\![A]\!] \cup [\![B]\!],\ k \in \text{props}(v)$ holds if and only if ($\forall v \in [\![A]\!],\ k \in \text{props}(v)$) **and** ($\forall v \in [\![B]\!],\ k \in \text{props}(v)$)—that is, $k \in$ `keyof` $A$ and $k \in$ `keyof` $B$, which is (`keyof` $A$) & (`keyof` $B$).

For (9.2), every value $v \in [\![A \& B]\!] = [\![A]\!] \cap [\![B]\!]$ satisfies both $A$ and $B$, so $v$ possesses every guaranteed key of $A$ together with every guaranteed key of $B$: every key in (`keyof` $A$)$\cup$(`keyof` $B$) is guaranteed on $v$. TypeScript *defines* `keyof` $(A \,\&\, B)$ syntactically as the union (`keyof` $A$) | (`keyof` $B$); the argument above shows that this definition is sound, because every key it lists is present on every value of the intersection. □

Let us see Theorem 9.1 in code on two payment rails:

```
interface CardPayment {
  amountCents: bigint;
  currency: 'USD' | 'EUR';
  cardLast4: string;
}

interface WirePayment {
  amountCents: bigint;
  currency: 'USD' | 'EUR';
  swiftBic: string;
}

// Keys guaranteed on EITHER payment rail = intersection of key sets:
// ('amountCents' | 'currency' | 'cardLast4') & ('amountCents' | 'currency' | 'swiftBic')
type SharedPaymentKeys = keyof (CardPayment | WirePayment);
// Evaluates to: 'amountCents' | 'currency'

// Keys present on an object satisfying BOTH interfaces = union of key sets:
type MergedPaymentKeys = keyof (CardPayment & WirePayment);
// Evaluates to: 'amountCents' | 'currency' | 'cardLast4' | 'swiftBic'
```

Concretely: if a function accepts `p: CardPayment | WirePayment` and has not yet checked which rail `p` came from, the only fields safe to read on `p` are `p.amountCents` and `p.currency`—precisely `keyof (CardPayment | WirePayment)`.

### 9.2.5 Top of the Lattice: `unknown` ($\mathcal{U}$) vs. the Escape Hatch `any`

At the very bottom of the lattice sits `never` ($\emptyset$). What sits at the very top—the universal set $\mathcal{U}$ containing *every* possible JavaScript runtime value (`42`, `'hi'`, `null`, `undefined`, {}, functions, symbols)?

TypeScript has *two* types that accept any value, and confusing them is the single biggest source of preventable production crashes in TypeScript services:

- **`unknown` is the true mathematical top type $\mathcal{U}$ (added in TypeScript 3.0).** Because $[\![T]\!] \subseteq \mathcal{U}$ for every type $T$, *every* value is assignable to `unknown`. Conversely, because $\mathcal{U} \not\subseteq [\![T]\!]$ for any proper subset $T \subsetneq \mathcal{U}$, **you cannot assign an `unknown` to anything else, call it, or read properties off it** until you narrow it down with a runtime check. In Java 6 terms, `unknown` behaves like `Object` (plus primitives and `null`), except you don't even get `.toString()` until you check for `null`/`undefined`.
- **`any` is NOT a set—it disables the type checker.** Assigning `const x: any = ...` tells `tsc` to allow *both* directions: any type $T$ assigns into `any`, *and* `any` silently assigns into any

type $T$ (except `never`), lets you access `x.nonExistent.deep.field()`, and infects downstream return types with `any` without a single warning.

```
function handleWebhookUnsafe(payload: any): bigint {
  // Compiles with ZERO warnings -- crashes at runtime if payload or data is undefined!
  return BigInt(payload.data.amountCents);
}

function handleWebhookSafe(payload: unknown): bigint {
  // payload.data; // COMPILER ERROR: 'payload' is of type 'unknown'.
  if (
    typeof payload === 'object' &&
    payload !== null &&
    'amountCents' in payload &&
    typeof payload.amountCents === 'string'
  ) {
    return BigInt(payload.amountCents); // Narrowed cleanly!
  }
  throw new Error('Malformed webhook payload');
}
```

Figure 9.2 puts the entire set-inclusion hierarchy together in one picture, contrasting the sound lattice from `never` ($\emptyset$) to `unknown` ($\mathcal{U}$) against the side-channel short-circuit of `any`.

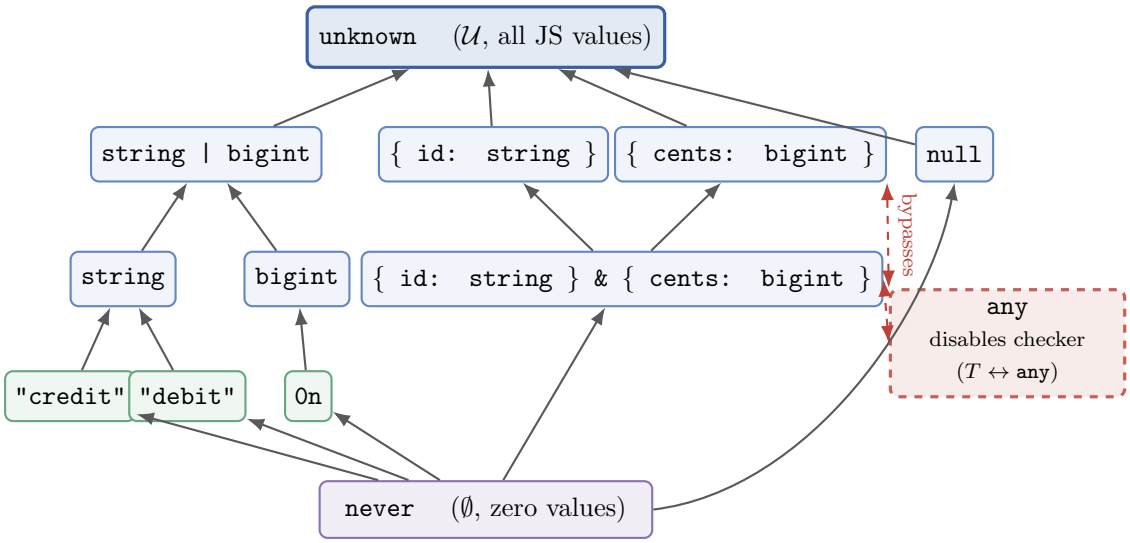



Figure 9.2: The TypeScript subtype lattice ordered by set inclusion ($S \to T$ means $[\![S]\!] \subseteq [\![T]\!]$). Every type sits between `never` ($\emptyset$) at the bottom and `unknown` ($\mathcal{U}$) at the top; object intersections sit *below* their constituent object sets because requiring more properties shrinks the set of inhabitants. Off to the right, `any` steps outside the lattice by permitting unchecked two-way assignment.

## 9.3 Discriminated Unions and Control-Flow Narrowing

In Chapter 5 we saw why Rust's algebraic `enum` sum types eliminate entire classes of invalid-state bugs that plague Java 6 classes full of nullable fields and boolean flags. Let us recall that lesson

briefly. In Java 6, a newcomer modeling a ledger event often writes one "god object" with nullable columns for every possible event type:

```
// Java 6 anti-pattern: 3^3 combinations of nulls, most of them nonsensical!
public final class LedgerEvent {
    public enum Kind { DEPOSIT, REVERSAL, FEE_WAIVER }
    public final Kind kind;
    public final Long amountCents;     // non-null only for DEPOSIT and FEE_WAIVER?
    public final String originalTxId;  // non-null only for REVERSAL!
    public final String reasonCode;    // non-null for REVERSAL and FEE_WAIVER?
    // ... constructor and getters ...
}
```

To make illegal states unrepresentable in Java 6 without Java 17+ `sealed` classes, you had to write a 60-line *Visitor pattern*: an abstract base class `LedgerEvent`, three final subclasses, an interface `LedgerEventVisitor<R>` with `visitDeposit`, `visitReversal`, and `visitFeeWaiver`, and an `accept(LedgerEventVisitor<R>)` method on each subclass.

In Rust, you wrote a 6-line `enum`, and the compiler laid it out on the stack as a 1-byte integer tag followed by a union of the largest variant's payload. How do we get that exact same algebraic precision in TypeScript when everything compiles down to untagged V8 objects?

### 9.3.1 Tagged Shapes: TypeScript's Counterpart to Rust's `enum`

Recall how a Rust `enum` is laid out in memory: memory bytes alone cannot tell a `Deposit` apart from a `Reversal`, so `rustc` prefixes every instance with a discriminant tag byte $(0, 1, 2)$. In TypeScript, we put that discriminant tag right into the JavaScript object as a **singleton literal property**—traditionally named `kind`, `type`, or `status`—and form the union of the variant interfaces:

```
// Direct TypeScript counterpart to Rust's `enum LedgerEvent`
type LedgerEvent =
  | { readonly kind: 'deposit'; readonly amountCents: bigint; readonly sourceIban: string }
  | { readonly kind: 'reversal'; readonly originalTxId: string; readonly reason: string }
  | { readonly kind: 'fee_waiver'; readonly waivedCents: bigint; readonly approvedBy: string };

function netBalanceDelta(event: LedgerEvent): bigint {
  switch (event.kind) {
    case 'deposit':
      // Here tsc narrows `event` to the 'deposit' variant ONLY!
      return event.amountCents;
    case 'reversal':
      // Reading `event.amountCents` here would be a compile-time error!
      return 0n; // Actual delta looked up by event.originalTxId
    case 'fee_waiver':
      return event.waivedCents;
    default: {
      // Exhaustiveness proof: if all 3 variants were handled, `event` is `never`!
      const _exhaustive: never = event;
      return _exhaustive;
    }
  }
```

```
}
```

Trace the set algebra inside `netBalanceDelta` step by step. At the top of the function, `event` belongs to $D \cup R \cup W$, where $D$, $R$, and $W$ are the three disjoint object sets whose `kind` property equals `'deposit'`, `'reversal'`, and `'fee_waiver'` respectively. Notice why those three sets are guaranteed *pairwise disjoint* ($D \cap R = \emptyset$): no JavaScript object can have `event.kind` simultaneously equal to `'deposit'` and `'reversal'`.

1. Inside `case 'deposit':`, the runtime test `event.kind === 'deposit'` succeeded. TypeScript intersects the current type $D \cup R \cup W$ with the constraint $\{\, v \mid v.\texttt{kind} = \texttt{'deposit'} \,\}$:

$$(D \cup R \cup W) \cap \{\, v \mid v.\texttt{kind} = \texttt{'deposit'} \,\} \;=\; D \cup \emptyset \cup \emptyset \;=\; D.$$

   Inside that branch, `event.amountCents` and `event.sourceIban` are known to exist without any cast. And because `return` exits the branch, $D$ is subtracted from the flow state that continues past `case 'deposit'`.

2. After `case 'reversal':` returns, $R$ is subtracted, leaving only $W$.

3. After `case 'fee_waiver':` returns, $W$ is subtracted, leaving $W \setminus W = \emptyset =$ `never` reaching `default:`.

4. Now imagine six months later a colleague adds a fourth variant to `LedgerEvent`:

```
| { readonly kind: 'chargeback'; readonly disputeId: string; readonly
    clawbackCents: bigint }.
```

   Every `switch` in the entire codebase that forgot `case 'chargeback':` now arrives at `default:` with residual type `{ readonly kind: 'chargeback'; ... }`. The line `const _exhaustive: never = event` immediately fails to compile with:

```
Type '{ readonly kind: "chargeback"; ... }' is not assignable to type
    'never'.
```

   Just like an unhandled `match` arm in Rust or OCaml, the compiler refuses to build until every call site handles chargebacks.

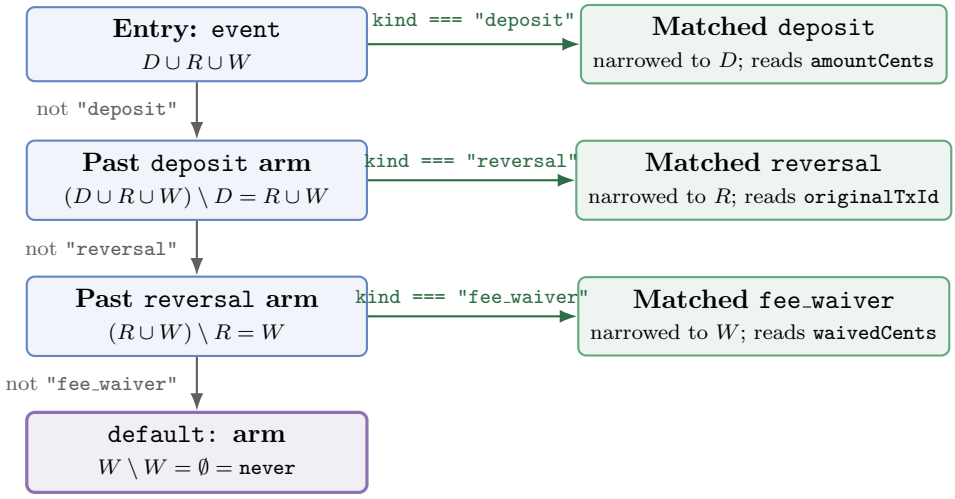



Figure 9.3: Control-flow type narrowing across a discriminated union `switch`. Each returning `case` arm intersects the union with one literal discriminant on the rightward branch and subtracts that variant along the downward fallthrough spine, reducing the residual set at `default:` to $\emptyset$ = `never`.

### 9.3.2 How Control-Flow Narrowing Works—and Three Classic JavaScript Traps

TypeScript's control-flow analyzer recognizes all of JavaScript's built-in runtime inspection idioms:

- `typeof x === '...'` for primitives (`'string'`, `'number'`, `'bigint'`, `'boolean'`, `'symbol'`, `'undefined'`, `'function'`, `'object'`);
- `x instanceof SomeClass` when `SomeClass` is an actual runtime `class` or constructor (`Date`, `Error`, `Uint8Array`);
- `'propName' in x` to check property presence on objects;
- `Array.isArray(x)` to narrow `unknown` or unions down to array types;
- **Strict equality** (`===`, `!==`) and **nullish coalescing** (`x ?? fallback`, which checks `x !== null && x !== undefined`).

Because TypeScript faithfully models JavaScript's actual runtime semantics, three JavaScript quirks show up directly during narrowing:

Common trap

**Three Narrowing Traps Every Java/C++ Engineer Hits Once:**

1. `typeof null === 'object'`. Due to a type-tag bug in the original 1995 Mocha implementation of JavaScript, `typeof null` returns the string `'object'`. TypeScript knows this. If `x : Account | null`, checking `if (typeof x === 'object')` narrows `x` to `Account | null`—it does *not* eliminate `null`. Always write `if (typeof x === 'object' && x !== null)`.
2. **Truthiness narrowing drops `0`, `0n`, and `""`.** In Java 6, `if (amount)` does not even compile unless `amount` is a `boolean`. In JavaScript, `if (amount)` coerces `amount` to boolean, where `0`, `0n`, `""` (empty string), `NaN`, `null`, and `undefined` are *all* falsy. If

`feeCents: bigint | undefined` happens to be `0n` (a zero-fee transfer), writing `if (feeCents)` takes the `else` branch and treats `0n` as missing. Write `if (feeCents !== undefined)` or use `feeCents ?? defaultFee` instead of `feeCents || defaultFee`.

3. **Narrowing does not cross non-inlined callbacks.** Suppose `let acct: Account | null = findAccount(id); if (acct !== null) { items.forEach(item => use(acct.id)); }`. Inside the callback, `tsc` flags `acct` as possibly `null`. Why? Because `acct` is a mutable `let` variable, and `tsc` cannot prove when `forEach` (or an async `setTimeout`) invokes the closure—outer code could reassign `acct = null` before the callback runs. Fix: bind a `const verifiedAcct = acct;` right inside the `if` block; a `const` binding can never be reassigned, so its narrowed type stays narrowed inside every nested closure.

### 9.3.3 User-Defined Type Guards (`x is T`) and Assertion Functions (`asserts x is T`)

Built-in checks work inside a single function, but what happens when you extract a validation helper into its own function? If a helper returns plain `boolean`, TypeScript sees only "this function returned `true` or `false`"—it forgets *why* it returned `true`. To carry narrowing across function boundaries, TypeScript provides **type predicates** (`param is Type`) and **assertion signatures** (`asserts param is Type`):

```
type DepositEvent = Extract<LedgerEvent, { kind: 'deposit' }>;

// 1. User-defined type guard: returns boolean AND tells tsc what `true` proves
function isDeposit(event: LedgerEvent): event is DepositEvent {
  return event.kind === 'deposit';
}

// 2. Assertion function: returns normally ONLY if condition holds (throws otherwise)
function assertNonEmptyString(val: unknown, name: string): asserts val is string {
  if (typeof val !== 'string' || val.trim().length === 0) {
    throw new Error(`Expected non-empty string for ${name}`);
  }
}

function summarizeDeposits(events: readonly LedgerEvent[]): bigint {
  // Without `event is DepositEvent`, .filter() would return LedgerEvent[], NOT
    DepositEvent[]!
  const deposits: DepositEvent[] = events.filter(isDeposit);
  return deposits.reduce((sum, d) => sum + d.amountCents, 0n);
}
```

Look at `events.filter(isDeposit)`. Because `Array.prototype.filter` has a TypeScript overload accepting `(value: T) => value is S` and returning `S[]`, passing `isDeposit` narrows the entire filtered array from `LedgerEvent[]` down to `DepositEvent[]` in one step. Just remember the contract you sign when writing `x is T`: `tsc` trusts the boolean your predicate returns. If you accidentally write `return event.kind !== 'deposit'` inside `isDeposit`, `tsc` will narrow `true` branches to `DepositEvent` anyway, creating a soundness bug that you authored yourself. Keep custom type guards tiny and thoroughly unit-tested.

## 9.4 Type-Level Programming: Transforming Types Like Data

So far we have this picture: types are sets, unions and intersections are set operations, and object shapes map keys to sets. Now take the next step. In Java 6, generics let you parameterize a container by a type (`List<T>`, `Map<K, V>`), and nothing more: you cannot loop over the fields of `T` at compile time to generate a read-only version of `T`, or inspect a function type `F` to extract its return type, or compute a union of valid event topic strings from currency codes. To do any of that in Java 6, teams turned to runtime reflection (`java.lang.reflect`), bytecode weaving, or external code generators.

TypeScript takes a completely different route: **every TypeScript file actually contains two languages**.

- At the *value level*, you have JavaScript: variables, loops, objects, and functions that run inside V8 at runtime.
- At the *type level*, you have a **purely functional compile-time language over sets**: generic type aliases are functions from types to types, `keyof` and `T[K]` inspect record structures, mapped types iterate over finite unions of keys, conditional types (`T extends U ? X : Y`) act as `if`/`else` expressions, and `infer` performs pattern matching.

Every computation in this type-level language finishes inside `tsc` and erases to zero runtime bytes. Let us build each operator from simple, concrete ledger examples.

### 9.4.1 `typeof`, `keyof`, and Indexed Access `T[K]`

Often a constant configuration object or table schema already exists at the value level, and writing a separate `interface` by hand would force you to keep two copies in sync. TypeScript's type-level `typeof` operator lifts any value expression's inferred type straight into the type language, while `keyof` extracts its union of property keys, and **indexed access** `T[K]` looks up the value set stored at key (or key union) `K`:

```
const CURRENCY_DECIMALS = {
  USD: 2,
  EUR: 2,
  JPY: 0,
  BHD: 3, // Bahraini Dinar uses fils (1/1000)
} as const;

// 1. Lift the runtime constant's shape into the type language:
type CurrencyMap = typeof CURRENCY_DECIMALS;
// { readonly USD: 2; readonly EUR: 2; readonly JPY: 0; readonly BHD: 3 }

// 2. Extract the union of keys:
type SupportedCurrency = keyof CurrencyMap;
// 'USD' | 'EUR' | 'JPY' | 'BHD'

// 3. Indexed access with a UNION of keys distributes across every key!
type AllowedScale = CurrencyMap[SupportedCurrency];
// CurrencyMap['USD'] | CurrencyMap['EUR'] | CurrencyMap['JPY'] | CurrencyMap['BHD']
// Simplifies by set union to: 2 | 0 | 3
```

Look at step 3: `CurrencyMap[SupportedCurrency]` is not an array lookup at runtime—it is a *type-level* lookup. Passing a union of keys `'USD' | 'EUR' | 'JPY' | 'BHD'` into `T[K]` evaluates `T[k]` for each key $k$ in the union and takes the union of all resulting types, simplifying $\{2\} \cup \{2\} \cup \{0\} \cup \{3\} = \{0, 2, 3\}$. Add `BTC: 8` to `CURRENCY_DECIMALS` tomorrow, and both `SupportedCurrency` and `AllowedScale` update across your entire project automatically.

Indexed access also works on arrays and tuples using the index type `number`: if `const ROLES = ['viewer', 'auditor', 'approver'] as const`, then `(typeof ROLES)[number]` evaluates directly to `'viewer' | 'auditor' | 'approver'`. Better still, combining `K extends keyof T` with `T[K]` gives us compile-time type-safe property lookup—something Java 6 reflection (`Field.get(obj)` returning raw `Object`) could never check statically:

```
interface LedgerAccount {
  id: string;
  currency: SupportedCurrency;
  balanceCents: bigint;
  frozen: boolean;
}

function getField<T, K extends keyof T>(record: T, key: K): T[K] {
  return record[key];
}

declare const acc: LedgerAccount;
const bal = getField(acc, 'balanceCents'); // Inferred type: bigint!
const cur = getField(acc, 'currency');     // Inferred type: 'USD' | 'EUR' | 'JPY' | 'BHD'!
// getField(acc, 'balance');               // COMPILER ERROR: '"balance"' is not in keyof LedgerAccount
```

### 9.4.2 Mapped Types: Looping Over Keys at Compile Time

Suppose we are building an HTTP `PATCH /accounts/:id` endpoint where a client can update a subset of fields on a `LedgerAccount`, or an audit snapshot where every field of `LedgerAccount` must be frozen `readonly`. Writing `LedgerAccountPatch` and `ReadonlyLedgerAccount` by hand duplicates every field name and drifts out of date whenever a column is added.

Instead, TypeScript's **mapped types** let you iterate `[K in Union]` right inside an object type, transforming each property's value type and toggling two orthogonal property modifiers—`readonly` (vs. mutable `-readonly`) and optional `?` (vs. required `-?`):

```
// Built right into TypeScript's standard lib.es5.d.ts -- zero compiler special-casing!
type MyReadonly<T> = {
  readonly [K in keyof T]: T[K];
};

type MyPartial<T> = {
  [K in keyof T]?: T[K];
};

type MyRequired<T> = {
  [K in keyof T]-?: T[K]; // `-?` strips optionality from every property!
```

```
};

type MyPick<T, Keys extends keyof T> = {
  [K in Keys]: T[K];
};

// Domain example: only allow patching `frozen` and `currency`, never `id` or `balanceCents`!
type AccountPatchRequest = Partial<Pick<LedgerAccount, 'frozen' | 'currency'>>;
// Evaluates to: { frozen?: boolean | undefined; currency?: SupportedCurrency | undefined }
```

Look at `MyReadonly`, `MyPartial`, `MyRequired`, and `MyPick`. When programmers coming from Java 6 first see `Partial<LedgerAccount>` or `Pick<LedgerAccount, 'id' | 'balanceCents'>`, they often assume `Partial` and `Pick` are special keywords hard-wired inside the TypeScript compiler. They aren't. Open `lib.es5.d.ts` in any TypeScript installation and you will find the exact three-line mapped types shown above. You can even remap property names on the fly using an `as` clause inside the bracket:

```
// Generate getter method signatures for every property of T!
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

type AccountGetters = Getters<Pick<LedgerAccount, 'id' | 'balanceCents'>>;
// { getId: () => string; getBalanceCents: () => bigint }
```

### 9.4.3 Conditional Types, Union Distributivity, and Pattern Matching with `infer`

Mapped types give us compile-time `for` loops. To make the type language genuinely expressive, we also need compile-time `if/else` branching and pattern matching. TypeScript provides both via **conditional types**:

$$\texttt{T extends U ? TrueBranch : FalseBranch}.$$

Remember our set-theory translation: `T extends U` asks "is $[\![T]\!] \subseteq [\![U]\!]$?" If yes, the type expression evaluates to `TrueBranch`; otherwise, to `FalseBranch`.

The following rule explains most of TypeScript's standard utility types, and is easiest to follow in terms of sets:

**Definition 9.2** (Distributive Conditional Types)**.** When `T` is a *naked type parameter* (meaning `T` appears bare on the left side of `extends`, not wrapped inside `[T]` or `Box<T>`), applying a conditional type $\Phi(\texttt{T}) = (\texttt{T extends U ? X : Y})$ to a union $A \mid B \mid C$ **distributes over the union element by element**:

$$\Phi(A \mid B \mid C) \;=\; \Phi(A) \;\mid\; \Phi(B) \;\mid\; \Phi(C).$$

In particular, $\Phi(\texttt{never}) = \texttt{never}$ because `never` is the 0-element union.

Union distributivity, together with $\emptyset = \texttt{never}$, turns `Exclude` and `Extract` into one-liners:

```
// Built-in Exclude<T, U>: drop every union member assignable to U!
```

```
type MyExclude<T, U> = T extends U ? never : T;

// Trace MyExclude<'deposit' | 'reversal' | 'fee_waiver', 'fee_waiver'>:
//   ('deposit'    extends 'fee_waiver' ? never : 'deposit')
// | ('reversal'   extends 'fee_waiver' ? never : 'reversal')
// | ('fee_waiver' extends 'fee_waiver' ? never : 'fee_waiver')
// = 'deposit' | 'reversal' | never
// = 'deposit' | 'reversal'   (since A U empty_set = A!)

// Built-in NonNullable<T>: strip null and undefined from any union
type MyNonNullable<T> = T extends null | undefined ? never : T;
```

**Common trap**

**When distributivity surprises you—and how `[T] extends [U]` turns it off.** Suppose you write `type ToArray<T> = T extends unknown ? T[] : never` and evaluate `ToArray<string | number>`. Because `T` is a naked type parameter, it distributes: `ToArray<string> | ToArray<number>` = `string[] | number[]` (an array of all strings *or* an array of all numbers—NOT a mixed `(string | number)[]`). Even wilder: if someone writes `type IsNever<T> = T extends never ? true : false`, what is `IsNever<never>`? Because `never` is the empty union, distributing over zero elements produces the empty union `never` instead of `true`. Whenever you want to test a type $T$ *as a single whole set* without distributing across unions, wrap both sides in a 1-tuple:

`type IsNever<T> = [T] extends [never] ? true : false;` (`IsNever<never>` is `true`.)

Now pair conditional types with the `infer` keyword. Inside the `extends` clause of a conditional type, writing `infer R` tells TypeScript: "if `T` matches this structural pattern, bind the unknown piece at this slot to a fresh type variable `R` so I can use `R` in the true branch." If you wrote OCaml or Rust pattern matching in Parts II and III, `infer` is structural pattern matching at the type level:

```
// 1. Extract the return type of any function type F
type MyReturnType<F> = F extends (...args: never[]) => infer R ? R : never;

// 2. Recursively unwrap nested Promises (just like TypeScript's built-in Awaited<T>)
type UnwrapPromise<T> =
  T extends Promise<infer Inner> ? UnwrapPromise<Inner> : T;

type Resolved = UnwrapPromise<Promise<Promise<{ txId: string }>>>;
// Evaluates to: { txId: string }

// 3. Extract the payload shape for a specific discriminated-union kind!
type EventPayload<E, K extends string> =
  E extends { kind: K } ? Omit<E, 'kind'> : never;

type ReversalFields = EventPayload<LedgerEvent, 'reversal'>;
// Evaluates to: { readonly originalTxId: string; readonly reason: string }
```

### 9.4.4 Template Literal Types: Compile-Time String Algebra

TypeScript 4.1 extended literal sets with **template literal types**, allowing backtick string interpolation *inside type expressions*. When you interpolate unions into a template literal type, `tsc` computes the Cartesian product across all combinations, and when you combine template literals with `infer`, you can parse route paths and event names at compile time.

```
type EventStage = 'initiated' | 'settled' | 'failed';
// Cartesian product: 4 currencies * 3 stages = 12 exact string literal types!
type LedgerTopic = `ledger.${Lowercase<SupportedCurrency>}.${EventStage}`;
// 'ledger.usd.initiated' | 'ledger.usd.settled' | ... | 'ledger.bhd.failed'

// Parse `:param` segments out of an Express / Fastify HTTP route string at compile time!
type RouteParams<Path extends string> =
  Path extends `${string}:${infer Param}/${infer Rest}`
    ? { [K in Param | keyof RouteParams<`/${Rest}`>]: string }
    : Path extends `${string}:${infer Param}`
      ? { [K in Param]: string }
      : Record<never, never>;

type TransferRouteParams = RouteParams<'/ledgers/:ledgerId/accounts/:accountId'>;
// Evaluates to: { ledgerId: string; accountId: string }
```

Compare this with Java 6. In a Java 6 servlet or Spring 2.5 controller, calling `getParameter` with the misspelled name `"acountId"` compiled cleanly and failed with a `NullPointerException` only when an HTTP request hit that route. Here, `tsc` parses the route string literal during compilation, synthesizes the object type `{ ledgerId: string; accountId: string }`, gives you instant IDE autocomplete on `req.params.accountId`, and flags `req.params.acountId` before a single server process boots—and emits no runtime parsing code.

**Work it yourself**

**Don't confuse `as T` with `satisfies T` (TypeScript 4.9+).** Suppose you have a dictionary of route handlers or ledger accounts and want *both* compile-time checking against a schema *and* narrow literal inference on the keys/values:

- Writing `const cfg: Record<string, number | string> = { port: 8080, host: '0.0.0.0' }` checks the object, *but widens* `cfg.port` to `string | number` (forgetting it is a `number`).
- Writing `const cfg = { ... } as Record<...>` suppresses real type errors.
- Writing `const cfg = { port: 8080, host: '0.0.0.0' } satisfies Record<string, number | string>` validates that `cfg` belongs to ⟦`Record<...>`⟧ **without widening** `cfg`'s inferred type: `cfg.port` stays `number` and `cfg.host` stays `string`. Reach for `satisfies` whenever you want schema validation at compile time without throwing away exact literal/field information.

## 9.5 Variance, Unsoundness Holes, Branding, and Runtime Boundaries

Every industrial type system makes deliberate trade-offs between strict mathematical soundness and practical ergonomics. TypeScript's design goals explicitly state that it does *not* aim for 100% provable soundness at every corner if doing so would make everyday JavaScript patterns unusable; instead, it aims for high productivity while catching bugs early. As an engineer building financial software, however, *you* need to know exactly where the ice is thin. Let us walk through variance, the mutable-array soundness hole, nominal branding, and the untrusted JSON boundary.

### 9.5.1 Variance From Set Inclusion—and the Mutable Array Trap

Suppose `CorporateAccount` <: `Account` ($[\![\texttt{CorporateAccount}]\!] \subseteq [\![\texttt{Account}]\!]$). How do compound types built out of $T$—arrays, promises, callbacks, comparators—preserve or reverse that subtype order?

1. **Covariant (same direction, producers/readers):** `ReadonlyArray<CorporateAccount>` <: `ReadonlyArray<Account>`, and `() => CorporateAccount` <: `() => Account`. If a caller expects a function that produces an `Account`, handing them a function that always produces a `CorporateAccount` is 100% safe, because every `CorporateAccount` *is* an `Account`.
2. **Contravariant (reversed direction, consumers and writers):** Consider callback types of the form `Handler<T> = (item: T) => void`. Which way does subtyping go between `Handler<Account>` and `Handler<CorporateAccount>`? Think about who calls the callback. A function expecting a `Handler<CorporateAccount>` will pass a `CorporateAccount` into it. If you supply a `Handler<Account>` (which knows how to handle *any* `Account`, using only base `Account` properties), passing a `CorporateAccount` into it works every time. Conversely, supplying a `Handler<CorporateAccount>` (which reads `item.taxEin`, a field present *only* on corporate accounts) where a general `Handler<Account>` is expected would crash the moment someone passed a personal account. Thus function parameter types invert the inclusion arrow:

$$\texttt{CorporateAccount} <: \texttt{Account}$$
$$\implies \quad \texttt{(x: Account) => void} <: \texttt{(x: CorporateAccount) => void}.$$

   TypeScript enforces function-parameter contravariance whenever `strictFunctionTypes` (included in `"strict": true`) is enabled, for arrow-type signatures `prop: (x: T) => void`.
3. **Invariant (neither direction, mutable read-write cells):** What if a container *both* produces $T$ (covariant read) and consumes $T$ (contravariant write), like a mutable array `T[]` or `{ value: T }`? Mathematically, both $S <: T$ and $T <: S$ must hold simultaneously, forcing $S = T$ (*invariance*)—which is why Rust's `&'a mut T` and Java 6's `ArrayList<T>` are strictly invariant.

Here is what happens in TypeScript (and compare it directly to Java 6 arrays):

```
interface Account {
  id: string;
}
interface CorporateAccount extends Account {
  taxEin: string; // Required string on every CorporateAccount!
```

```
}

const corpAccounts: CorporateAccount[] = [
  { id: 'corp_1', taxEin: '12-3456789' },
];

// UNSOUND COVARIANCE: TypeScript allows CorporateAccount[] -> Account[]!
const generalAccounts: Account[] = corpAccounts;

// Mutating through the wider alias inserts an Account with NO `taxEin`:
generalAccounts.push({ id: 'personal_9' });

// Compiles cleanly -- crashes at runtime with TypeError: Cannot read properties of
    undefined!
console.log(corpAccounts[1].taxEin.toUpperCase());
```

Compare this line for line against Java 6's notorious covariant `CorporateAccount[]` → `Account[]` bug.

- In **Java 6**, `Account[] general = corpAccounts;` also compiles, *but* the JVM heap array object stores its reified element class `[LCorporateAccount;` in its header. On the line `general[1] = new PersonalAccount("personal_9")`, the JVM executes `aastore`, sees the runtime element mismatch, and immediately throws `java.lang.ArrayStoreException` right at the bad write.

- In **TypeScript**, because of Mental Shift 2 (total type erasure), the V8 array at runtime is just a plain JS `Array` with zero knowledge that it was born as a `CorporateAccount[]`. So `generalAccounts.push({ id: 'personal_9' })` succeeds silently, corrupting the invariant of `corpAccounts`, and the crash happens much later (possibly in a completely different module) when a reader dereferences `.taxEin`.

**Common trap**

**Two Defensive Habits That Close TypeScript's Array Holes:**

1. **Default to `readonly T[]` (or `ReadonlyArray<T>`) on function parameters.** Because `readonly T[]` strips `.push()`, `.pop()`, `.splice()`, and index assignment, it is a pure producer—so its covariance is 100% mathematically sound. Better still, mutable `T[]` is a subtype of `readonly T[]`, so callers can pass mutable arrays in without copying.

2. **Enable `"noUncheckedIndexedAccess": true` in `tsconfig.json`.** Even `"strict": true` leaves `arr[i]` and `dict[key]` typed as `T` rather than `T | undefined`, pretending out-of-bounds reads never happen. Turning on `noUncheckedIndexedAccess` changes `arr[i]` to `T | undefined`, forcing you to handle empty/out-of-bounds indices explicitly just like Rust's `slice.get(i) -> Option<&T>`. (Also prefer arrow property syntax `validate: (x: T) => boolean` over method syntax `validate(x: T): boolean` in interfaces; TypeScript leaves method syntax bivariant so `Array<T>` stays covariant.)

### 9.5.2 Nominal Branding: Teaching a Structural Type Checker About Units and IDs

Structural typing works well for composing shapes, and is dangerous when two domain concepts share the same primitive representation. In our financial ledger:

- `UsdCents` and `EurCents` are both `bigint` at runtime.
- `AccountId` and `TransactionId` are both `string` at runtime.

In Rust (Chapter 5), we wrote zero-cost tuple structs such as `UsdCents(i64)` and `EurCents(i64)`; because Rust is nominal, mixing dollars and euros is a hard compile error with zero heap wrapper overhead. In TypeScript, writing `type UsdCents = bigint` and `type EurCents = bigint` creates two aliases for the *exact same set* ⟦`bigint`⟧—passing `EurCents` into `settleUsdWire(amount: UsdCents)` compiles without a peep.

How do we carve disjoint nominal subsets out of `bigint` *without* wrapping every number in a heap-allocated box object { `kind: 'USD', cents: 500n` } (which would mirror Java 6's costly `Long` boxing)? We use **branded types** (also called *opaque* or *phantom* types):

```
// Declare phantom unique symbols -- erased completely at runtime!
declare const UsdBrand: unique symbol;
declare const EurBrand: unique symbol;

export type UsdCents = bigint & { readonly [UsdBrand]: never };
export type EurCents = bigint & { readonly [EurBrand]: never };

// Zero-allocation smart constructors: validate invariants once, return bare bigint!
export function usdCents(raw: bigint): UsdCents {
  return raw as UsdCents;
}

export function eurCents(raw: bigint): EurCents {
  return raw as EurCents;
}

export function addUsd(a: UsdCents, b: UsdCents): UsdCents {
  return (a + b) as UsdCents; // UsdCents <: bigint, so `+` works directly!
}

const price = usdCents(2500n);
const taxEu = eurCents(475n);

const totalUsd = addUsd(price, usdCents(500n)); // Compiles! Runtime value is bare `3000n`
// addUsd(price, taxEu);
// COMPILER ERROR: Argument of type 'EurCents' is not assignable to parameter of type '
    UsdCents'.
```

Here is what happens at compile time and at runtime:

1. At **compile time**, `UsdCents` is the intersection `bigint` & { `readonly [UsdBrand]: never` }. Because `UsdCents` <: `bigint`, you can still pass a `UsdCents` to `>`, `===`, or string formatting expecting a `bigint`. Yet a plain `bigint` is *not* assignable to `UsdCents`, and `EurCents` is *not* assignable to `UsdCents`, because neither carries the unique phantom symbol `UsdBrand`.

2. At **runtime**, `declare const UsdBrand`, `type UsdCents`, and `as UsdCents` all erase to zero bytes. Calling `usdCents(2500n)` executes `return raw;`, keeping raw unboxed V8 `BigInt` primitives in memory—giving us the exact zero-wrapper discipline of Rust's newtypes and Kotlin's `@JvmInline value class` (which we will meet in Chapter 10).

### 9.5.3 The Runtime Trust Boundary: Parse, Don't Cast

We close Chapter 9 with the single architectural lesson that ties structural set theory and total erasure together in real production services.

Inside your compiled TypeScript modules, `tsc` verifies every function call, union branch, and branded unit. Now ask: **where do values enter your process from the outside world?** HTTP request bodies (`req.body`), PostgreSQL JSONB columns, Kafka messages, environment variables, `fetch()` responses, and browser `localStorage`.

In TypeScript's standard library, `JSON.parse(text: string): any` historically returned `any`. Here is what happens when a developer writes the tempting one-liner at an API boundary:

```
interface WireTransferPayload {
  fromAccount: string;
  toAccount: string;
  amountCents: number;
}

// BROKEN IN PRODUCTION: `as WireTransferPayload` performs ZERO runtime checks!
function handleTransferBroken(rawJson: string): number {
  const req = JSON.parse(rawJson) as WireTransferPayload;
  const flatFeeCents = 150;
  // Suppose attacker sends: {"fromAccount":"a","toAccount":"b","amountCents":"900000"}
  // In JavaScript, `"900000" + 150` performs STRING CONCATENATION -> `"900000150"`!
  return req.amountCents + flatFeeCents;
}
```

Trace what goes wrong in `handleTransferBroken`. In Java 6 with Jackson or Gson, deserializing `"amountCents": "900000"` into a `long` field either parses the integer into a real 64-bit `long` or throws a deserialization exception, because the target Java class exists at runtime and its field types are inspected by reflection. In TypeScript, `interface WireTransferPayload` vanished at compile time, and `as WireTransferPayload` vanished at compile time. At runtime, `req.amountCents` is the string `"900000"`, and JavaScript's overloaded `+` operator concatenates `"900000" + 150` into `"900000150"`.

The architectural cure is the principle **Parse, Don't Validate** (and never `as`-cast untrusted data): treat every external byte stream as `unknown` ($\mathcal{U}$), pass it through a **runtime schema parser** (such as `Zod`, `Valibot`, or `ArkType`) right at the edge of your process, and derive your static TypeScript type *from the runtime schema itself* so the runtime check and compile-time type can never drift apart.

```
import { z } from 'zod';

// 1. Define the RUNTIME parser value once at the boundary:
const LedgerEventSchema = z.discriminatedUnion('kind', [
  z.object({
```

```
    kind: z.literal('deposit'),
    amountCents: z.string().regex(/^[1-9]\d*$/).transform((s) => usdCents(BigInt(s))),
    sourceIban: z.string().min(15).max(34),
  }),
  z.object({
    kind: z.literal('reversal'),
    originalTxId: z.string().uuid(),
    reason: z.string().min(1),
  }),
]);

// 2. Derive the STATIC TypeScript type automatically via type-level typeof + infer!
type VerifiedLedgerEvent = z.infer<typeof LedgerEventSchema>;
// { kind: 'deposit'; amountCents: UsdCents; sourceIban: string }
// | { kind: 'reversal'; originalTxId: string; reason: string }

export function ingestHttpPayload(rawJson: string): VerifiedLedgerEvent {
  const untrusted: unknown = JSON.parse(rawJson); // Immediately lock down `any` to `
    unknown`!
  const parsed = LedgerEventSchema.safeParse(untrusted);
  if (!parsed.success) {
    throw new Error(`Invalid ledger payload: ${parsed.error.message}`);
  }
  // Inside our domain core, `parsed.data` is guaranteed at BOTH runtime and compile time!
  return parsed.data;
}
```

Each idea from this chapter appears in those twenty lines (Figure 9.4):

1. Assigning `JSON.parse(rawJson)` directly to `untrusted: unknown` immediately closes the `any` escape hatch and places the untrusted payload at the top of our mathematical lattice ($\mathcal{U}$).

2. `LedgerEventSchema` is a real JavaScript object that executes actual V8 property, regex, and discriminant checks at runtime, transforming decimal integer strings safely into branded `UsdCents` `bigints` (avoiding IEEE-754 `number` precision loss above $2^{53} - 1 = 9{,}007{,}199{,}254{,}740{,}991$).

3. `z.infer<typeof LedgerEventSchema>` uses the exact type-level programming tools from Section 9.4 (`typeof`, conditional types, and `infer`) to compute the static discriminated union `VerifiedLedgerEvent` directly from the runtime validator.

4. Once `parsed.data` crosses the boundary into your core domain logic, zero redundant runtime `typeof` checks are needed: static set inclusion carries the guarantee from there on.

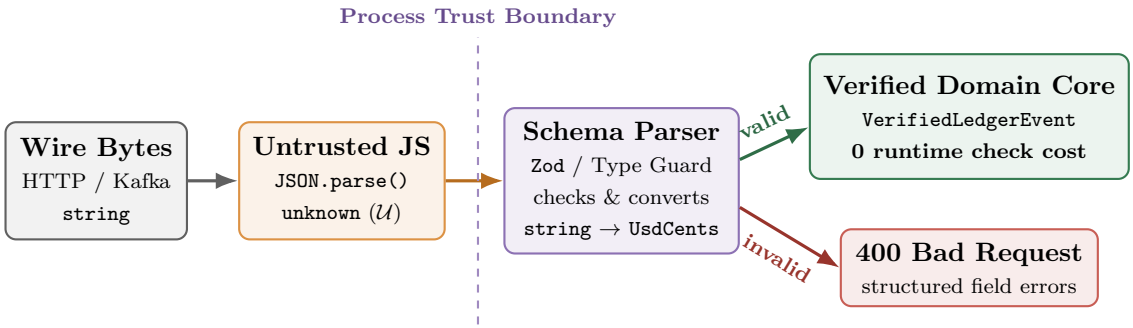



Figure 9.4: The "Parse, Don't Validate" architecture required by total type erasure. External bytes enter as `unknown` ($\mathcal{U}$) and cross a runtime schema parser (`Zod`) once at the trust boundary; invalid payloads are rejected immediately, while valid payloads enter the statically verified core with branded types and discriminated unions.

**Connection**

**Looking Ahead to Chapter 10 (Kotlin on the JVM).** TypeScript showed us what happens when you layer a rich, structural, set-theoretic type checker over a completely dynamic runtime where all static types erase. What if instead you want modern null-safety, algebraic sum types, zero-overhead domain wrappers, and lightweight concurrency *without* giving up JVM bytecode verification, nominal runtime class descriptors, and direct calls to twenty years of existing Java 6 libraries? That is the exact design space of **Kotlin**, where we head next.

## 9.6 Exercises

**Exercise 9.1** (Set Algebra of Unions, Intersections, and `keyof`)**.** Consider two object types: `A` with fields `id: string`, `status: 'open' | 'frozen'`, and `limit: bigint`; and `B` with fields `id: string`, `status: 'frozen' | 'closed'`, and `closedAt: Date`.

1. Compute the exact simplified TypeScript types for `keyof (A | B)` and `keyof (A & B)`.
2. What is the type of `(A & B)['status']`? Prove your answer using $[\![A \,\&\, B]\!] = [\![A]\!] \cap [\![B]\!]$.
3. What would happen to $[\![A \,\&\, B]\!]$ if `A` had `status: 'open'` and `B` had `status: 'closed'`? Verify what `tsc` infers for `A & B` in that disjoint case.

**Exercise 9.2** (Exhaustiveness Checking and Refactoring Safety)**.** Model a payment settlement state machine as a discriminated union `SettlementState` with four variants: `Pending` (`submittedAt: Date`), `Authorized` (`authCode: string` and `holdCents: bigint`), `Captured` (`settledCents: bigint` and `receiptId: string`), and `Declined` (a `declineReason` that is one of `'insufficient_funds'`, `'fraud_suspected'`, or `'network_timeout'`).

1. Explain why storing these four states in a single Java 6 class with nullable fields allows at least $2^6 - 4$ nonsensical combinations of null and non-null fields, whereas the TypeScript union allows only valid states.
2. Write `canRetry(state: SettlementState): boolean` using a `switch` and a `never` exhaustiveness check so that only `Declined` states with reason `'network_timeout'` return `true`, and

adding any fifth variant forces a compile error.

**Exercise 9.3** (Mutable Covariance vs. `readonly`)**.** Consider `interface Box<T> { value: T }` alongside `interface ReadonlyBox<T> { readonly value: T }` and `interface Sink<T> { put: (item: T) => void }`.

1. Given `CorporateAccount <: Account`, determine whether `ReadonlyBox`, `Sink`, and `Box` should be covariant, contravariant, or invariant in `T` to preserve runtime type soundness.
2. In TypeScript, even in strict mode, test whether a box of corporate accounts is assignable to a box of plain accounts. Construct a 6-line snippet showing how writing through the wider box's `value` field causes a runtime `TypeError` without any `any` or `as` casts. Then explain why Java 6's generic `AtomicReference` rejects the analogous assignment at compile time.

**Exercise 9.4** (Writing `DeepReadonly<T>` at the Type Level)**.** TypeScript's built-in `Readonly<T>` is *shallow*: on an `Order` interface with `id: string` and nested `customer: { name: string }`, `Readonly<Order>` blocks reassigning the `customer` field, yet still lets you overwrite the customer's `name` with `'Mallory'`. Write a recursive conditional and mapped type `DeepReadonly<T>` that leaves function types and primitives untouched, converts `Array<infer U>` into `ReadonlyArray<DeepReadonly<U>>`, and recursively marks every property of object types `readonly`. Test it on nested arrays and objects.

**Exercise 9.5** (Compile-Time SQL Column Projection with Template Literals and `infer`)**.** Using template literal types, conditional types, and `infer`, write a generic type `ParseColumns<Query extends string>` that extracts a union of trimmed column names from simple strings such as `'SELECT id, owner, balanceCents FROM accounts'`, so that applying `Pick<LedgerAccount, ParseColumns<...>>` to `'SELECT id, balanceCents FROM accounts'` evaluates at compile time to `{ id: string; balanceCents: bigint }` and flags a misspelled column name `'SELECT id, balence FROM accounts'` as a compile error.

**Exercise 9.6** (Closing an IEEE-754 Precision and Brand Bug at an HTTP Boundary)**.** A teammate wrote an HTTP handler accepting a JSON body whose `amountCents` field is the bare number `9007199254740993`, parsed via `JSON.parse(body)` and then cast with `as` to an object type with `accountId: string` and `amountCents: number`.

1. Explain two independent bugs in this handler: first, what `JSON.parse` actually produces in V8 for the integer 9,007,199,254,740,993 $= 2^{53} + 1$ when parsed into an IEEE-754 double-precision `number`; second, what happens if a client sends `"amountCents": null`.
2. Redesign the wire format and write a boundary parser (either using `Zod` or a zero-dependency type-guard/parser function taking `unknown`) that accepts exact base-10 integer strings, rejects negative or malformed strings, and returns an object carrying branded `AccountId` and `UsdCents` types.

# Chapter 10

# Kotlin From Java 6: Null-Safety, Algebraic Hierarchies, and Coroutines on the JVM

In Chapter 9 we saw one path away from Java 6: TypeScript abandons the JVM altogether, treats types as mathematical sets of shapes, and erases every static annotation down to dynamic JavaScript objects on V8. That design fits browser interfaces and Node.js gateways well, yet it comes with real trade-offs: numbers default to 64-bit IEEE-754 floats unless you reach for `bigint`, there is only one user-space thread per V8 isolate, and no nominal class descriptors survive at runtime to guard external boundaries automatically.

Now ask a different engineering question—the exact question JetBrains faced in 2010 while maintaining millions of lines of Java 6 IDE and server code: *what if we keep the JVM itself—HotSpot's C2 JIT compiler, mature concurrent garbage collectors, profiling tools, and twenty years of existing Java libraries—and only want to fix the static language sitting on top of it?*

Look back at our list of Java 6 pain points from Chapter 1:

1. **Billion-dollar nullability:** every reference type `Account` implicitly includes `null`, turning every dot (`acct.getOwner().trim()`) into a runtime `NullPointerException` gamble.
2. **Ceremony and mutability by default:** an immutable three-field value record requires 65 lines of constructors, getters, `equals`, `hashCode`, and `toString`.
3. **Wrapper boxing tax:** giving a primitive `long` account ID its own type `class AccountId` costs a 16-byte JVM object header plus pointer indirection on every instance.
4. **Closed sum types require the Visitor pattern:** modeling a fixed set of payment outcomes demands four classes and a double-dispatch visitor interface.
5. **One-thread-per-request concurrency:** every blocking JDBC or HTTP call ties up a 1 MB operating-system kernel thread, hitting an out-of-memory wall long before CPU or network bandwidth saturates.

Kotlin solves all five problems while compiling down to standard JVM `.class` files that call Java 6 JARs—and can be called back *from* Java 6 code—within the exact same process and heap. The cleanest way for a Java 6 and C/Rust programmer to master Kotlin is never to treat its features

as black-box syntax sugar. Instead, for every construct we meet in this chapter, we will look at **both** the clean Kotlin surface code **and** the exact JVM bytecode (decompiled back into equivalent Java 6) that `kotlinc` emits underneath. The bytecode shows what each feature costs, from nullable primitives and `@JvmInline` value classes to `reified` generics and `suspend` coroutines.

## 10.1 Bridging Java 6 to Kotlin: Same Bytecode Machine, Sharper Lens

When `kotlinc` compiles a Kotlin file, it does not invoke a custom runtime VM. It emits standard JVM bytecode instructions (`aload`, `invokestatic`, `invokevirtual`, `checkcast`, `tableswitch`) plus a tiny runtime JAR (`kotlin-stdlib`, roughly 1.7 MB) and a `@kotlin.Metadata` annotation attached to each generated `.class` header so other Kotlin files can read Kotlin-specific type information (such as nullability, `sealed` subclasses, and `inline` bodies) that raw Java 6 bytecode descriptors could not express on their own.

Let us warm up by translating four everyday Kotlin idioms straight down to the Java 6 bytecode `kotlinc` generates:

1. **`val` vs. `var` and Properties (Not Raw Fields):** In Kotlin, `val` declares a read-only binding (like Java 6 `final`), while `var` declares a reassignable binding. When declared on a class, `val balanceCents: Long` is *not* a raw public Java field; `kotlinc` emits a `private final long balanceCents` backing field together with a public `getBalanceCents()` method. If Java 6 code calls `acct.getBalanceCents()` and Kotlin code writes `acct.balanceCents`, both execute the exact same `invokevirtual getBalanceCents()J` instruction—and you can later replace the stored backing field with a computed custom getter `val isOverdrawn: Boolean get() = balanceCents < 0L` without breaking binary compatibility.

2. **Default Arguments via Synthetic Bitmask Bridges:** Java 6 requires writing three telescopic method overloads just to give two parameters default values. In Kotlin, you write one declaration:

```
fun openAccount(owner: String, currency: String = "USD", overdraftLimit: Long = 0L)
```

   and `kotlinc` emits two methods in bytecode: the real 3-argument method, plus a synthetic bridge `openAccount$default` that takes the same three arguments followed by an `int mask` and an unused `Object` marker. The bridge inspects bits of `mask` (`(mask & 2) != 0`, `(mask & 4) != 0`) to fill in whichever arguments the call site omitted. Adding a `@JvmOverloads` annotation tells `kotlinc` to emit the classic telescopic Java 6 overloads as well, so legacy Java callers get the defaults too.

3. **Extension Functions Compile to Static Receiver-First Methods:** In Java 6 codebases, every project grows a drawer full of `StringUtils`, `CollectionUtils`, and `MoneyUtils` classes because you cannot add methods to `java.lang.String`. Kotlin lets you write a *receiver type* right before the function name:

```
// File: IbanStrings.kt
package ledger.util

```

```kotlin
fun String.maskedIban(): String {
    if (this.length <= 8) return "****"
    return this.take(4) + "..." + this.takeLast(4)
}

// Caller reads like a natural method call on String:
val safeLog = "DE89370400440532013000".maskedIban() // "DE89...3000"
```

How can Kotlin add `maskedIban()` to `java.lang.String` when `java.lang.String` is `final` inside `rt.jar`? It doesn't touch `java.lang.String` at all. Run `javap -c ledger.util.IbanStringsKt` and look at the decompiled Java 6 equivalent:

```kotlin
// Exact JVM equivalent emitted by kotlinc for IbanStrings.kt:
public final class IbanStringsKt {
    public static final String maskedIban(String $this$maskedIban) {
        Intrinsics.checkNotNullParameter($this$maskedIban, "<this>");
        if ($this$maskedIban.length() <= 8) return "****";
        return StringsKt.take($this$maskedIban, 4) + "..."
             + StringsKt.takeLast($this$maskedIban, 4);
    }
}
```

**Common trap**

**Extension Functions Dispatch Statically, Never Virtually.** Because `a.describe()` on an extension function compiles to `invokestatic ExtensionsKt.describe(DeclaredType a)` rather than `invokevirtual`, the JVM chooses which extension function to call based on the **compile-time declared type** of the variable, *not* the runtime class of the heap object. If you define both `fun Account.tag() = "base"` and `fun CorporateAccount.tag() = "corp"`, and then write `val x: Account = CorporateAccount(...); println(x.tag())`, it prints `"base"`. Use member methods inside an interface or sealed hierarchy whenever you want polymorphic runtime dispatch; use extension functions for stateless helpers attached to a type from the outside.

## 10.2 Nullability in the Type System: Splitting Every Reference Set in Two

In Java 6, the reference type `String` actually represents the mathematical union String $\cup$ {`null`}, and the compiler refuses to let you name the subset String *without* `null`. In Rust (Chapter 5), references are never null and optionality is an explicit sum type `Option<String>`. In TypeScript with `strictNullChecks` (Chapter 9), optionality is the explicit set union `string | null`.

Kotlin brings that exact set-theoretic split onto the JVM. Every type `T` in Kotlin has two distinct forms:

- `T` **(non-null):** guaranteed at compile time never to hold `null`.
- `T?` **(nullable):** the mathematical union $[\![$`T`$]\!] \cup$ {`null`}.

Because $[\![\texttt{T}]\!] \subsetneq [\![\texttt{T?}]\!]$, **every non-null type `T` is a subtype of its nullable counterpart `T?`** (`String` <: `String?`). Passing a `String` to a parameter expecting `String?` always succeeds; passing a `String?` to a parameter expecting `String` is a hard compile error.

At the bottom of Kotlin's lattice sits `Nothing` ($\emptyset$, the exact twin of TypeScript's `never` and Rust's `!`), which is a subtype of every type `T`—while `Nothing?` ($\emptyset \cup \{\texttt{null}\} = \{\texttt{null}\}$) is the type of the literal `null` itself. At the top sits `Any` (the root of all non-null types, unifying both reference classes and primitives like `Int`) below `Any?` (the universal top type).

### 10.2.1 Zero-Wrapper Reference Nullability vs. Boxed Nullable Primitives

Now ask the systems-programmer question: **what does `String?` vs. `String`—and `Long?` vs. `Long`—cost in JVM memory?**

In Java 8, the library team introduced `java.util.Optional<String>`. Unfortunately, `Optional` is an ordinary heap-allocated class: wrapping a `String` reference inside `Optional.of(s)` allocates an *extra* 16-byte JVM object header plus 8-byte reference field (24 bytes with alignment) around the `String` pointer.

Kotlin's `String?` pays **zero wrapper overhead**. The reason: at the hardware level, a 64-bit JVM reference pointer already has the all-zero bit pattern `0x0000000000000000` reserved for `null` (the exact niche optimization we saw for Rust's `Option<&T>` in Chapter 5). Therefore, in JVM bytecode:

- Both `owner: String` and `memo: String?` compile to the **exact same bare JVM reference descriptor** `Ljava/lang/String;`. No `Optional` wrapper object is ever allocated.
- To protect non-null Kotlin parameters against untyped or Java 6 callers passing `null` across a JAR boundary, `kotlinc` injects a cheap guard call (`Intrinsics.checkNotNullParameter`, passing the argument and its name) at the entry of public methods accepting non-null references, failing fast at the boundary instead of letting `null` corrupt an internal data structure ten calls deeper.

Now contrast reference types with 64-bit integers (`Long`). Every one of the $2^{64}$ bit patterns of a raw JVM primitive `long` (`J`) represents a valid signed integer from $-2^{63}$ to $2^{63} - 1$; there is *no spare bit pattern* left over to represent `null`. Here is what `kotlinc` must emit for `Long` vs. `Long?` (Figure 10.1):

- `val cents: Long` compiles to a raw 8-byte JVM primitive `long` (`J`) directly in a CPU register or stack slot—zero heap allocation, zero object header.
- `val optionalFee: Long?` *must* compile to a boxed heap reference `Ljava/lang/Long;`. Whenever `optionalFee` is non-null (outside `Long.valueOf`'s tiny −128..127 cache), it points to a 24-byte heap object (16-byte JVM header + 8-byte `long`).

**Common trap**

**Beware `Long?` and `Double?` Inside Million-Element Loops.** In Rust, `Option<i64>` occupies 16 bytes *inline on the stack or inside a flat `Vec<Option<i64>>`* (an 8-byte tag+padding word next to the 8-byte `i64`, zero pointers, zero heap headers). On today's pre-Valhalla JVM, `Array<Long?>` or `List<Long>` allocates boxed `java.lang.Long` heap objects, scattering 24-byte nodes across memory. In hot financial aggregation loops on Kotlin/JVM, use Kotlin's

specialized primitive array `LongArray` (which compiles to JVM `long[]`, a contiguous flat slab of 8-byte integers) and domain sentinels or parallel bitsets rather than `List<Long?>`.

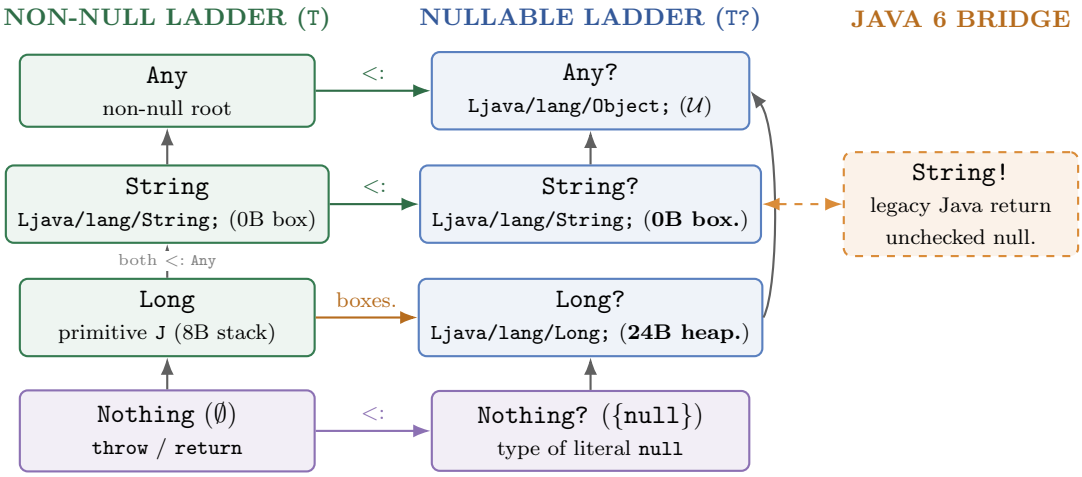



Figure 10.1: Kotlin's split subtype hierarchy and JVM descriptors. Every non-null type `T` embeds into `T?`. Reference types (`String?`) reuse the null pointer niche with zero wrapper allocation, whereas nullable primitives (`Long?`) must box onto the JVM heap as `java.lang.Long`. Unannotated Java 6 methods surface as platform types (`String!`).

### 10.2.2 Safe Calls, Elvis with `Nothing`, Smart Casts, and Platform Types

Once `null` is isolated inside `T?`, Kotlin gives you concise, composable operators that compile directly into tight null-branching bytecode with zero allocations:

```
class Customer(val name: String, val taxCountry: String?)
class Account(val id: Long, val owner: Customer?)

fun resolveWithholdingCountry(account: Account?): String {
    // 1. Safe-call chain `?.` short-circuits to null on the first null link;
    // 2. Elvis `?:` supplies the right-hand fallback when the left side is null:
    return account?.owner?.taxCountry ?: "US"
}

fun requireOwnerName(account: Account?): String {
    // Because `throw ...` has type `Nothing` (empty set!), `Customer? ?: Nothing`
    // narrows `owner` directly to non-null `Customer` for the rest of the function!
    val owner: Customer = account?.owner
        ?: throw IllegalArgumentException("Account has no owner")
    return owner.name.trim() // `owner` is guaranteed non-null here!
}
```

Consider `val owner: Customer = account?.owner ?: throw ...` in `requireOwnerName`. Why does Kotlin allow a `throw` (or an early `return`) on the right-hand side of the Elvis operator `?:`? Apply the exact same set algebra we learned in Chapter 9. The left side of `?:` has type `Customer?`, which is $[\![\texttt{Customer}]\!] \cup \{\texttt{null}\}$. When `?:` handles the `null` case, the right side evaluates an expression of

type `Nothing` ($\emptyset$). Thus the combined expression has type:

$$(\llbracket \texttt{Customer} \rrbracket \setminus \{\texttt{null}\}) \cup \llbracket \texttt{Nothing} \rrbracket = \llbracket \texttt{Customer} \rrbracket \cup \emptyset = \llbracket \texttt{Customer} \rrbracket.$$

Compare this directly to Rust's `let Some(owner) = ... else { return ... }` from Chapter 5: early-exit guards flatten nested `if (x != null)` pyramids into straight-line code.

Just like TypeScript's control-flow narrowing, `kotlinc` also performs **smart casts**: after `if (x != null)` or `if (x is CorporateAccount)`, `x` automatically narrows to non-null `T` or `CorporateAccount`. Notice one soundness rule where Kotlin is *stricter* than TypeScript: `kotlinc` refuses to smart-cast a mutable `var` property (or a `val` with a custom getter) on an object, because another JVM thread could mutate that field between your `if (acct.owner != null)` check and your `acct.owner.name` read. Bind it to a local immutable variable first (`val owner = acct.owner; if (owner != null) ...`)—local stack-frame slots are thread-private on the JVM, so the smart cast is 100% thread-safe.

Common trap

**Guarding the Java 6 Boundary: Platform Types (`T!`).** What happens when Kotlin calls a legacy Java 6 method without `@Nullable` or `@NotNull` annotations, such as `java.sql.ResultSet.getString("memo")`? If Kotlin forced every unannotated Java return value to be `String?`, calling Java libraries would require thousands of redundant null checks; if Kotlin assumed `String`, the first SQL `NULL` would silently violate non-null guarantees. So `kotlinc` assigns unannotated Java values a **platform type**, printed in compiler diagnostics as `String!` ("`String` of unknown nullability"). If you assign a `String!` straight into `val s: String = rs.getString("memo")`, `kotlinc` emits an immediate runtime `checkNotNullExpressionValue` check that throws `NullPointerException` right on the assignment line if SQL returned `NULL`. Rule of thumb when calling Java 6 APIs: *always write an explicit type annotation* (`val memo: String? = rs.getString("memo")`) at the exact line where a Java value enters Kotlin, locking down `String!` immediately.

## 10.3 Data, Value, and Sealed Types: Algebraic Modeling on the JVM

With nullability tamed, let us bring the algebraic data types we loved in Rust and OCaml (Parts II and III) onto the JVM—and measure their exact object-header and bytecode costs.

### 10.3.1 `data class`: Value Equality and Copy Without Boilerplate

In Java 6, `Object.equals` defaults to pointer identity (`==`), so every domain value object (`Posting`, `Money`, `DateRange`) needs manual `equals`, `hashCode`, and `toString` implementations. Prefixing a Kotlin class with `data` instructs `kotlinc` to generate `equals`, `hashCode`, `toString`, `copy`, and `component1()..componentN()` destructuring methods directly from the primary constructor properties:

```
data class LedgerPosting(
    val accountId: AccountId,
    val amountCents: Long,
    val currency: String,
```

```
)

val p1 = LedgerPosting(AccountId(1001L), 25_00L, "USD")
val p2 = p1.copy(amountCents = 30_00L) // Immutable update; p1 is untouched!
val (acct, cents, curr) = p2           // Calls component1(), component2(), component3()
```

Notice Kotlin's equality operators (the reverse of Java 6): in Kotlin, `a == b` **compiles to null-safe structural** `equals()` (`Intrinsics.areEqual(a, b)`), while `a === b` **tests raw JVM heap pointer identity**. You will never again lose an afternoon in production because someone wrote `currency == "USD"` in Java 6 on a freshly allocated `String` and got `false`.

**Common trap**

**Two Subtle `data class` Traps in Financial Code:**

1. **Positional destructuring swaps same-typed fields silently.** Writing `val (from, to) = transfer` compiles to `val from = transfer.component1(); val to = transfer.component2();` purely by *parameter position*, ignoring variable names. If a teammate later reorders `data class Transfer(val toAccount: AccountId, val fromAccount: AccountId)`, every destructuring site silently debits the recipient and credits the sender with zero compiler warnings. Prefer explicit property access (`transfer.fromAccount`) on records with multiple fields of the same type.

2. **Never put raw `ByteArray` or `LongArray` in a `data class` without overriding `equals/hashCode`.** On the JVM, array `.equals()` checks pointer identity, not element contents (`Arrays.equals`), so two `data class HmacDigest(val bytes: ByteArray)` instances containing identical SHA-256 bytes will evaluate `d1 == d2` to `false`. Wrap byte buffers in an immutable `ByteString` or override `equals/hashCode` with `contentEquals/contentHashCode`.

### 10.3.2 `@JvmInline value class`: Zero-Header Domain Types on the JVM

Recall from Chapter 1 why disciplined Java 6 engineers often gave up on strongly typed wrappers like `AccountId` and `UsdCents` in high-throughput services: on a 64-bit JVM (even with compressed OOPs), wrapping an 8-byte `long` inside `final class AccountId { private final long raw; }` adds a 12-to-16-byte JVM object header plus alignment padding (24 bytes total on the heap) *plus* an 8-byte pointer in the referring object, tripling memory footprint and causing L1 cache misses.

In Rust, `struct AccountId(i64)` has zero runtime header: at machine level it is just an `i64` in a CPU register. In TypeScript (Chapter 9), branded intersections `bigint & { [Brand]: never }` achieved the same zero-wrapper result by erasing the brand. How does Kotlin achieve zero-allocation nominal wrappers on the JVM? Via `@JvmInline value class`:

```
@JvmInline
value class AccountId(val raw: Long) {
    init {
        require(raw > 0L) { "AccountId must be positive, got $raw" }
    }
}

@JvmInline
```

```
value class UsdCents(val raw: Long) {
    operator fun plus(other: UsdCents): UsdCents = UsdCents(this.raw + other.raw)
    operator fun minus(other: UsdCents): UsdCents = UsdCents(this.raw - other.raw)
}

fun bookDebit(account: AccountId, amount: UsdCents): UsdCents {
    return amount + UsdCents(150L) // + $1.50 wire fee
}
```

Now inspect the decompiled JVM bytecode emitted by `kotlinc` for `bookDebit`:

```
// Exact JVM bytecode signature emitted by kotlinc (decompiled to Java):
// Notice the mangled suffix `-4k8pQ2A` and raw `long` (J) parameters & return!
public static final long bookDebit_4k8pQ2A/* $FF was: bookDebit-4k8pQ2A */(
    long account,
    long amount
) {
    return UsdCents.constructor_impl(amount + UsdCents.constructor_impl(150L));
}
```

Three details in that bytecode matter:

1. **Zero heap allocation on the fast path:** Both `AccountId` and `UsdCents` were completely unboxed into raw 64-bit JVM `long` values passed in CPU registers. Even the call `UsdCents.constructor_impl(150L)` is a static method returning primitive `long` (running the `init` validation check without allocating an object).

2. **Why is the method name mangled to `bookDebit-4k8pQ2A`?** Think about two things: first, what if you overload `fun settle(x: UsdCents)` and `fun settle(x: EurCents)` in the same class? After unboxing, *both* would have the JVM signature `settle(J)V`, which the JVM classfile verifier rejects as a duplicate method. Hashing the Kotlin value-class types into a suffix (`-4k8pQ2A` vs. `-9m2zR1B`) keeps their JVM descriptors distinct. Second, a hyphen `-` is legal in JVM bytecode method names and *illegal* in Java source identifiers—which prevents legacy Java 6 code from accidentally calling `bookDebit(rawLong1, rawLong2)` with swapped raw `long`s.

3. **When does a `value class` still box onto the heap?** Exactly like Java 6 `int` → `Integer`, `kotlinc` generates a hidden wrapper class used *only* when polymorphism or nullability forces an object pointer: when an `AccountId` is typed as nullable `AccountId?`, upcast to an interface `Any` or `Comparable<AccountId>`, or stored inside an erased generic collection `List<AccountId>` (because on today's JVM, `java.util.List<E>` stores `Object[]` pointers).

### 10.3.3 `sealed interface` and Exhaustive `when`: Sum Types on the JVM

Now let us compare how Java 6, Rust, TypeScript, and Kotlin represent a closed sum type of payment authorization outcomes. In Kotlin, marking an `interface` or `class` as `sealed` restricts all direct implementations to live in the same package and compilation module, allowing `kotlinc` (and the JVM's `PermittedSubclasses` classfile attribute on JDK 17+) to know the complete, closed universe of variants at compile time:

```
sealed interface AuthOutcome {
```

```
    data class Approved(
        val authCode: String,
        val authorizedAmount: UsdCents,
    ) : AuthOutcome

    data class Declined(
        val reason: String,
        val retryAfterSeconds: Int?,
    ) : AuthOutcome

    // `data object` = single JVM static singleton (zero allocation per return!)
    data object LedgerTemporarilyOffline : AuthOutcome
}

fun httpStatusFor(outcome: AuthOutcome): Int = when (outcome) {
    is AuthOutcome.Approved -> {
        // Smart-cast automatically to Approved: `outcome.authCode` is in scope!
        200
    }
    is AuthOutcome.Declined -> if (outcome.retryAfterSeconds != null) 429 else 422
    AuthOutcome.LedgerTemporarilyOffline -> 503
    // NO `else` branch needed -- and adding a 4th subtype breaks compilation here!
}
```

`when (outcome)` works as a value-returning *expression* (not merely a C/Java 6 statement that can accidentally fall through if you forget `break`). Because `AuthOutcome` is `sealed`, `kotlinc` verifies that the three arms (`Approved`, `Declined`, and the offline singleton) cover every subtype of `AuthOutcome`—and emits a chain of JVM `instanceof` + `checkcast` instructions (or a `tableswitch`/`invokedynamic` type switch on modern targets).

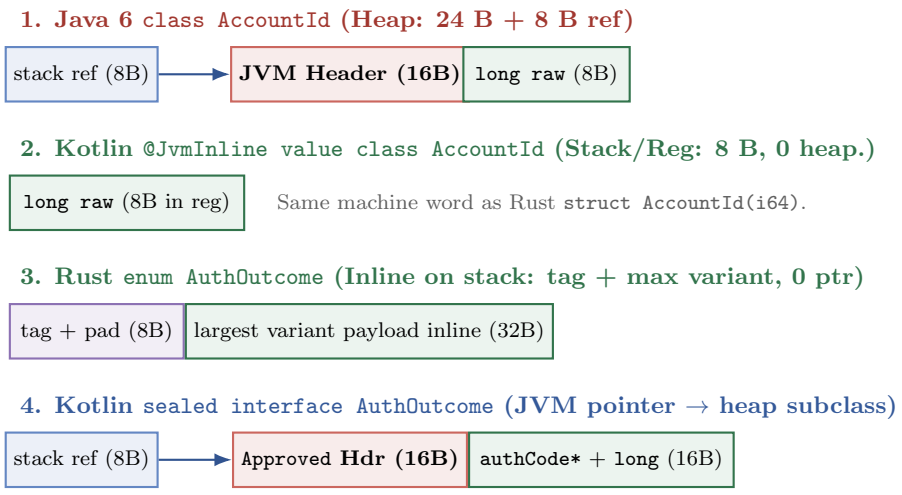



Figure 10.2: Physical memory layouts compared. Kotlin's `@JvmInline value class` eliminates the 16-byte JVM object header on single-field wrappers, matching Rust's newtypes. For multi-field sum types, Rust packs the discriminant tag and union payload flat on the stack, whereas Kotlin's `sealed interface` uses a standard JVM reference to a heap-allocated variant subclass (except `data object` variants, which reuse one global singleton).

Figure 10.2 makes the exact trade-off between Rust's `enum` and Kotlin's `sealed interface` transparent. At the type-system level, both give you closed algebraic sum types and compiler-enforced exhaustiveness. At the hardware level, Rust's `enum` lives inline on the stack (or flat inside a `Vec<AuthOutcome>`), whereas Kotlin's `Approved` variant is still a heap-allocated JVM object (though HotSpot's C2 JIT escape analysis can scalar-replace short-lived `Approved` objects that do not escape an inlined call chain, and zero-field variants declared `data object` allocate once at class initialization and never again).

## 10.4 Functions, Receivers, Inline Reification, and Sequences

In Java 6, passing a two-line predicate into a method meant writing an eight-line anonymous inner class (`new Predicate<LedgerPosting>() { public boolean apply(LedgerPosting p) { ... } }`), generating a synthetic `MyService$1.class` file on disk and allocating a heap object every time local variables were captured. Kotlin treats functions as first-class types (`(LedgerPosting) -> Boolean`), yet if you stopped there, every higher-order helper on the JVM would still pay a closure-allocation and virtual-dispatch tax. Let us see how `kotlinc` eliminates both taxes—and defeats Java 6 generics erasure along the way.

### 10.4.1 Lambdas with Receivers (`T.() -> R`): How Type-Safe Builders Work

First, let us demystify one of Kotlin's signature features: **function types with a receiver**, written `Receiver.() -> ReturnType`. Beginners who encounter Kotlin Gradle scripts, Ktor routes, or SQL/HTML builders often wonder how a block like `journalEntry { debit(...); credit(...) }` brings `debit` and `credit` into scope without an import or object prefix.

There is zero compiler special-casing behind Kotlin DSLs. Recall from Section 10.1 that an extension function `fun String.maskedIban()` is simply a static function whose first parameter is named `this`. A **lambda with receiver** `JournalBuilder.() -> Unit` is the exact same idea applied to a function value: at the JVM bytecode level, it is a one-argument function `Function1<JournalBuilder, Unit>` where inside the braces `{ ... }`, that first parameter is bound to the implicit receiver `this`.

```
@DslMarker
annotation class LedgerDsl

@LedgerDsl
class JournalBuilder(private val description: String) {
    private val legs = mutableListOf<LedgerPosting>()

    fun debit(account: AccountId, cents: Long, currency: String = "USD") {
        require(cents > 0L)
        legs.add(LedgerPosting(account, -cents, currency))
    }

    fun credit(account: AccountId, cents: Long, currency: String = "USD") {
        require(cents > 0L)
        legs.add(LedgerPosting(account, +cents, currency))
    }

    fun build(): List<LedgerPosting> {
```

```
        val net = legs.sumOf { it.amountCents }
        check(net == 0L) { "Unbalanced journal '$description': net = $net cents" }
        return legs.toList()
    }
}

// Trailing lambda with receiver `JournalBuilder.() -> Unit`:
inline fun journalEntry(
    description: String,
    setup: JournalBuilder.() -> Unit,
): List<LedgerPosting> {
    val builder = JournalBuilder(description)
    builder.setup() // Equivalent to `setup.invoke(builder)`!
    return builder.build()
}

// Call site: reads like a custom language, fully type-checked by kotlinc!
val postings = journalEntry("Invoice #4092 settlement") {
    debit(AccountId(1010L), 250_00L)   // Implicitly `this.debit(...)` on JournalBuilder
    credit(AccountId(4000L), 250_00L)  // Implicitly `this.credit(...)` on JournalBuilder
}
```

Notice two details in those thirty lines:

1. In Kotlin, whenever the *last* parameter of a function is itself a function type, callers may move the lambda braces *outside* the parentheses (`journalEntry("...") { ... }`). Combine trailing lambdas with a receiver type `JournalBuilder.() -> Unit`, and you get type-safe builders in twenty lines of ordinary Kotlin.

2. What does `@DslMarker` do? If builders nest (`ledger { journalEntry { ... } }`), both the outer `LedgerBuilder` and the inner `JournalBuilder` would normally be implicit `this` receivers in scope, letting someone accidentally call an outer `journalEntry` method inside an inner block. Annotating builder classes with the same `@DslMarker` meta-annotation hides outer implicit receivers unless explicitly qualified (`this@ledger`).

### 10.4.2 `inline` Functions and `reified T`: Defeating Closure Allocation and JVM Erasure

Look back at the keyword `inline` on `inline fun journalEntry(...)` above. Why did we write `inline`, and what happens in JVM bytecode without it?

Suppose we write a helper `withDatabaseTx(conn) { ... }` *without* `inline`, and call it inside a hot settlement loop where the lambda captures two local variables `accountId` and `amountCents`. Without `inline`, `kotlinc` must compile that lambda into an object implementing the interface `Function0` (from `kotlin.jvm.functions`) whose constructor stores `accountId` and `amountCents` in fields—allocating a fresh closure object on the JVM heap on every single iteration, plus an `invokeinterface` call.

Marking the higher-order function `inline` tells `kotlinc` to perform C++/Rust-style call-site expansion on *both* the higher-order function *and* every lambda passed to it:

```
inline fun <T> timedOperation(metricName: String, block: () -> T): T {
```

```
    val startNs = System.nanoTime()
    try {
        return block()
    } finally {
        val elapsedNs = System.nanoTime() - startNs
        Metrics.recordLatency(metricName, elapsedNs)
    }
}
```

When a caller writes `val ok = timedOperation("post_tx") { repo.save(tx) }`, `kotlinc` splices the bytecode of `timedOperation` and `repo.save(tx)` straight into the caller's method body: **zero `Function0` class, zero heap closure allocation, and zero virtual call**. Even better, because the lambda's bytecode now sits directly inside the caller's method frame, writing a bare `return` inside `timedOperation { if (invalid) return ... }` performs a **non-local return** straight out of the enclosing function—making custom control-flow helpers (`synchronized`, `use`, `forEach`) behave identically to built-in `for` and `if` statements. (If an `inline` function passes a lambda on to another execution context such as a thread pool where non-local `return` cannot jump across stack frames, Kotlin requires marking that parameter `crossinline`, or `noinline` if it must remain a real heap function object.)

Now watch the second payoff of `inline`—solving one of Java 6's most annoying generics limitations. In Java 6, because generic type arguments erase to `Object` in bytecode, you can *never* write `if (x instanceof T)` or `T.class`:

```
// Java 6: forced to pass an explicit 'Class<T> clazz' token everywhere!
public static <T> T decodeJson(String raw, Class<T> clazz) {
    return OBJECT_MAPPER.readValue(raw, clazz);
}
// Call site repeats the type name:
TransferReq req = decodeJson(body, TransferReq.class);
```

Wait: why did Java 6 lose `T`? Because a normal generic method is compiled *once* into a single bytecode body shared by all callers, so inside that single shared body `T` is unknown. What about a Kotlin `inline` function? Its bytecode is *not* shared—it is pasted straight into each call site, and **at the call site the compiler knows the exact concrete type argument** `TransferReq`. Adding the modifier `reified` on an `inline` type parameter instructs `kotlinc` to embed the actual runtime `Class` token at every inlined call site:

```
inline fun <reified T : Any> decodeJson(raw: String): T {
    // Legal ONLY because 'inline' + 'reified' substitutes the concrete Class<T> at call
    sites!
    return OBJECT_MAPPER.readValue(raw, T::class.java)
}

inline fun <reified E : LedgerEvent> List<LedgerEvent>.filterVariant(): List<E> {
    val out = ArrayList<E>()
    for (item in this) {
        if (item is E) out.add(item) // Real JVM 'instanceof' emitted for concrete E!
    }
    return out
}

// Call sites: zero Class<T> boilerplate, checked at runtime by the JVM!
```

```
val req: TransferReq = decodeJson(rawBody) // Inlines `ldc TransferReq.class`!
```

**Intuition**

**Four Generics Strategies Compared.** The four generics designs in this book line up on one spectrum:

- **TypeScript:** 100% compile-time erasure; neither `List` nor `T` leaves a nominal runtime descriptor.
- **Java 6 (and non-inline Kotlin):** type-argument erasure; the container `ArrayList` exists at runtime, while `<T>` erases to `Object` plus `checkcast` at callers.
- **Kotlin `inline <reified T>`:** call-site bytecode inlining substitutes the concrete `T::class.java` and `instanceof` instructions at each caller without needing VM changes.
- **C++ Templates & Rust Generics:** full compile-time monomorphization generates distinct native machine-code functions *and* specialized flat memory layouts (`Vec<i64>` vs. `Vec<Posting>`). Note what Kotlin `reified` *cannot* do: you still cannot declare a class `class Cache<reified T>` because class fields live on the heap across methods and cannot be "inlined".

### 10.4.3 Eager `Iterable` Extensions vs. Lazy `Sequence<T>`

Here is a performance trap that catches almost every developer arriving in Kotlin from Java 8 `Streams` or Rust `Iterators`. Compare these two pipelines over a list of one million ledger postings:

```
// Pipeline A: EAGER `Iterable` extensions -- allocates TWO intermediate ArrayLists!
val firstThreeLargeIdsEager: List<AccountId> = postings
    .filter { it.amountCents >= 100_000_00L } // Allocates full List #1 of all matches!
    .map { it.accountId }                     // Allocates full List #2 of all AccountIds!
    .take(3)                                  // Copies first 3, throws the rest to GC!

// Pipeline B: LAZY `Sequence<T>` -- zero intermediate lists, stops on match #3!
val firstThreeLargeIdsLazy: List<AccountId> = postings
    .asSequence()
    .filter { it.amountCents >= 100_000_00L } // Builds lightweight Sequence stage
    .map { it.accountId }                     // Builds lightweight Sequence stage
    .take(3)                                  // Pulls elements ONE BY ONE until 3 match!
    .toList()                                 // Terminal op materializes 3-element List
```

Why did Kotlin make standard `List.filter` and `List.map` *eager* instead of lazy? The answer is in how `Iterable.filter` is implemented in `kotlin-stdlib`: it is an `inline` function containing a plain `for` loop. When you transform a small 10-element list with a single `.map { ... }`, an eager `inline` loop allocates *zero* closure or iterator-stage objects and runs as fast as a hand-written Java 6 `for` loop. However, the moment you chain *multiple* stages (`filter` → `map` → `take` or `firstOrNull`) over thousands of elements, horizontal eager evaluation materializes an entire intermediate `ArrayList` after every dot. Calling `.asSequence()` switches to vertical pull-based lazy evaluation—just like Rust's `iter().filter(...).map(...).take(3).collect()`—advancing one element through all stages before touching the next element.

## 10.5 Kotlin Coroutines Under the Hood: CPS State Machines and Structured Concurrency

We now reach the centerpiece of server-side and Android Kotlin engineering: **coroutines**.

Let us start with the hardware arithmetic that broke Java 6's one-thread-per-request server model. Suppose our payment gateway receives 100,000 concurrent checkout requests during a flash sale. Each request spends 0.2 ms of actual CPU time validating signatures, and 150 ms waiting on network I/O (a PostgreSQL query plus an external card-network HTTPS call).

In Java 6 (classic Tomcat or blocking thread pools), every `java.lang.Thread` wraps a 1:1 operating-system kernel thread (`pthread` on Linux):

- Each JVM thread reserves a contiguous **1 MB native call stack** by default (`-Xss1m`) so deep recursive Java/C JNI calls cannot overflow into neighboring memory, plus kernel `task_struct` bookkeeping. Multiplied by 100,000 concurrent requests, thread stacks alone need 100 **GB of reserved stack address space**. Long before that, the process hits the operating system's per-process thread limit or `ulimit -u`, and the JVM fails, at under 10% CPU load, with `java.lang.OutOfMemoryError: unable to create new native thread`.
- Even at 10,000 threads, parking and waking kernel threads across `futex`/`epoll` system calls incurs $\sim$ 1.5–3 $\mu$s per context switch plus L1/L2 instruction- and data-cache thrashing across 10,000 disjoint 1 MB stacks.

To escape the 1 MB-per-connection wall in Java 6/7/8, engineers turned to asynchronous callbacks (Netty `ChannelFutureListener`, `CompletableFuture.thenCompose`). Callbacks solved the memory problem—100,000 small heap state objects fit in under 20 MB—but at a high cost in readability: simple `for` loops, `try/catch/finally` resource cleanup, and sequential step-by-step control flow turned into deeply nested callback pyramids whose exception stack traces started in an anonymous Netty event-loop thread with all caller context erased.

Kotlin asks a clean compiler question: **what if you write ordinary sequential code with normal loops and `try/finally`, mark suspending functions with one keyword `suspend`, and let `kotlinc` automatically rewrite that sequential function into a tiny, zero-thread-blocking heap state machine?**

### 10.5.1 Inside `kotlinc`: Continuation-Passing Style (CPS) and the `label` Switch

Let us write a realistic twelve-line `suspend` function with two network suspension points, and then inspect the *exact* Java 6 equivalent state machine that `kotlinc` generates.

```
suspend fun settleWire(txId: Long, feeCents: Long): Receipt {
    // State 0: runs up to Suspension Point #1
    val account: Account = fetchAccountAsync(txId)
    // State 1: resumes here once `account` arrives over the socket!
    val debitedCents: Long = account.balanceCents - feeCents
    val receipt: Receipt = postClearingHouseAsync(account.id, debitedCents)
    // State 2: resumes here once `receipt` arrives!
    auditLog(txId, receipt.confirmationCode)
    return receipt
}
```

The function `settleWire` makes two calls to other `suspend` functions: `fetchAccountAsync` and `postClearingHouseAsync`. Those two points divide the function body into **three sequential basic blocks (States 0, 1, and 2)**. Notice which local variables must stay alive *across* suspension points:

- `txId` and `feeCents` are alive across Suspension Point #1 (and `txId` is still needed after Suspension Point #2 for `auditLog`).
- `account` is computed at the end of State 0 and consumed in State 1; it is never used in State 2.

If `settleWire` surrenders its underlying JVM OS thread while waiting 80 ms for `fetchAccountAsync` to hear back from PostgreSQL, it *cannot* leave `txId` and `feeCents` sitting on the JVM thread stack—another coroutine is about to use that exact OS thread stack two microseconds from now.

So `kotlinc` performs two mechanical transformations:

1. **Continuation-Passing Style (CPS) signature rewrite:** Every `suspend fun foo(args): R` receives one extra hidden trailing parameter `Continuation<? super R> $completion`, and its JVM return type changes from `R` to `Object`. Why `Object`? Because a suspending call can return *either* an immediate `Receipt` (fast path: data was already in cache, zero suspension needed) *or* the special singleton sentinel `CoroutineSingletons.COROUTINE_SUSPENDED`, signaling to its caller: "I have registered a callback on the socket; unwind the JVM stack immediately and free this OS worker thread."
2. **State-machine `ContinuationImpl` class:** `kotlinc` generates a compact state-machine class holding an `int label` field $(0, 1, 2)$ plus fields (`J$0`, `J$1`, `L$0`) for only those local variables whose lifetimes cross a suspension boundary.

Here is the decompiled Java 6 code corresponding directly to `kotlinc`'s bytecode for `settleWire`:

```
// Simplified Java 6 decompilation of kotlinc's CPS state machine for `settleWire`:
public static final Object settleWire(
    long txId,
    long feeCents,
    Continuation<? super Receipt> $completion
) {
    // Re-entering from resumeWith? Reuse existing state machine; else allocate once!
    SettleWireCont sm;
    if ($completion instanceof SettleWireCont
            && (((SettleWireCont) $completion).label & Integer.MIN_VALUE) != 0) {
        sm = (SettleWireCont) $completion;
        sm.label -= Integer.MIN_VALUE;
    } else {
        sm = new SettleWireCont($completion);
    }

    Object $result = sm.result;
    Object SUSPENDED = CoroutineSingletons.COROUTINE_SUSPENDED;

    switch (sm.label) {
        case 0: {
            ResultKt.throwOnFailure($result); // Rethrow if resumed with an exception
            sm.J$0 = txId;                    // Spill `txId` to heap frame
            sm.J$1 = feeCents;                // Spill `feeCents` to heap frame
            sm.label = 1;                     // Next resume enters `case 1:`!
            $result = fetchAccountAsync(txId, sm);
```

```
            if ($result == SUSPENDED) {
                return SUSPENDED;              // Unwind JVM stack -> OS thread is FREE!
            }
            // Fast path: if already complete, fall straight through to case 1!
        }
        case 1: {
            feeCents = sm.J$1;                 // Restore `feeCents`
            txId = sm.J$0;                     // Restore `txId`
            ResultKt.throwOnFailure($result);
            Account account = (Account) $result;
            long debitedCents = account.getBalanceCents() - feeCents;

            sm.J$0 = txId;                     // Keep `txId` for State 2
            sm.label = 2;                      // Next resume enters `case 2:`!
            $result = postClearingHouseAsync(account.getId(), debitedCents, sm);
            if ($result == SUSPENDED) {
                return SUSPENDED;              // Unwind JVM stack -> OS thread is FREE!
            }
        }
        case 2: {
            txId = sm.J$0;                     // Restore `txId`
            ResultKt.throwOnFailure($result);
            Receipt receipt = (Receipt) $result;
            auditLog(txId, receipt.getConfirmationCode());
            return receipt;                    // Normal completion!
        }
        default:
            throw new IllegalStateException("Invalid coroutine state");
    }
}
```

Reading that `switch (sm.label)` block from top to bottom answers two questions.

- **How much memory does a suspended coroutine take?** `SettleWireCont` holds a JVM object header, a reference to the caller's continuation, the coroutine context and intercepted-continuation references inherited from `ContinuationImpl`, an `int label`, `Object result`, and two `long` slots `J$0`, `J$1`—**under 64 bytes on the JVM heap** with compressed pointers, compared to 1,048,576 bytes for a parked OS thread. Spawning 100,000 concurrent coroutines waiting on sockets consumes a few dozen megabytes of young-generation heap instead of 100 GB of kernel stacks.

- **What happens when the PostgreSQL socket finishes reading?** Netty/NIO's selector wakes up on a worker thread and invokes `sm.resumeWith(Result.success(account))`. Inside `BaseContinuationImpl.resumeWith`, a non-recursive loop sets `sm.result = account`, flips the sign bit on `sm.label`, and calls `settleWire(0L, 0L, sm)`, which jumps straight to `case 1:` in a single `tableswitch` instruction. And if the socket failed with an `IOException`, the continuation is resumed with `Result.failure(ex)`; it enters `case 1:`, where the `throwOnFailure` call rethrows the exception right inside any surrounding `try/catch/finally` block.

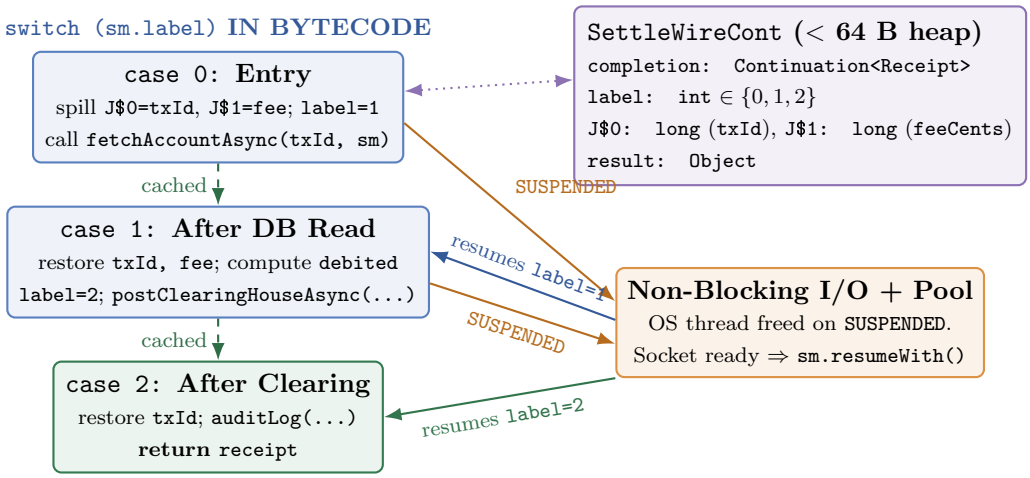



Figure 10.3: How `kotlinc` compiles a `suspend` function into a Continuation-Passing Style (CPS) state machine. Each suspension point splits the function into numbered `switch (sm.label)` cases; local variables live across suspension boundaries spill into a small (under 64-byte) `ContinuationImpl` heap object while the underlying OS worker thread is completely released.

**Connection**

**Kotlin CPS Coroutines vs. Rust `Future` State Machines (Preview of Chapter 12).** Both Kotlin and Rust choose *stackless* coroutines (compiling `suspend`/`async` functions into `label`-switched state machines, unlike Go's 2 KB *stackful* goroutines in Chapter 11), yet they differ in one key memory detail:

- **Kotlin (push-resumed CPS, per-frame heap object):** when `A()` calls suspending `B()` which calls suspending `C()`, suspending allocates three small linked `ContinuationImpl` objects on the JVM heap ($A \leftarrow B \leftarrow C$). Because the JVM already has a fast bump-pointer generational allocator, a few dozen bytes per suspended frame is cheap and permits arbitrary recursive `suspend` calls naturally.
- **Rust (pull-polled `Future::poll`, single flat inline struct):** in Rust, an `async fn A()` awaiting `B()` awaiting `C()` compiles all three nested state machines into **one contiguous inline `enum` value** whose exact byte size is computed at compile time—requiring only *one* heap allocation for the entire top-level task on `tokio::spawn`, and *zero* allocations on nested `.await` calls. The price of that flat inline layout is that self-referential borrows across `.await` points cannot move in memory, requiring Rust's `Pin<&mut Self>` type (Chapter 12).

### 10.5.2 Dispatchers and Structured Concurrency: Taming Background Work

Knowing how a single coroutine suspends is half the story; the other half is deciding *which thread pool executes each resumption* and *who owns the lifecycle of spawned coroutines.*

In `kotlinx.coroutines`, a `CoroutineDispatcher` governs thread scheduling:

- `Dispatchers.Default`: a work-stealing pool sized to the number of CPU cores ($\max(2, N_{\text{cores}})$), designed strictly for CPU-bound calculation (hashing, JSON parsing, sorting).

- `Dispatchers.IO`: an elastic thread pool (defaulting up to $\max(64, N_{\text{cores}})$ threads, sharing underlying worker threads with `Default` to avoid extra context switches.) designed strictly to wrap **legacy blocking Java 6 calls** (JDBC `Connection` queries, blocking file I/O). You hop onto it with `withContext(Dispatchers.IO) { ... }` around the blocking call.
- `Dispatchers.Main`: single UI event-loop thread (on Android/Desktop), keeping UI mutations race-free just like browser JavaScript.

Even more important than dispatchers is **Structured Concurrency**, pioneered in production by Roman Elizarov and the Kotlin team. Think about why `new Thread(runnable).start()` in Java 6 (or `go fn()` without cancellation discipline) causes resource leaks in large servers: once launched, a detached thread has no lexical parent. If a user cancels their HTTP request after 10 ms, or if one of two parallel fraud-check RPCs fails after 5 ms, detached background tasks keep burning database connections in the dark, and unhandled exceptions vanish into `UncaughtExceptionHandler`.

Kotlin enforces a strict lexical invariant: **you cannot call `launch` or `async` out of thin air—every coroutine must be spawned inside a `CoroutineScope` whose `Job` forms a strict parent–child tree** (Figure 10.4):

```
data class EnrichedAccount(val account: Account, val riskScore: Int, val fxRate:
    BigDecimal)

// `coroutineScope` creates a lexical child scope; it NEVER returns until ALL children
    finish!
suspend fun loadEnrichedAccount(accountId: Long): EnrichedAccount = coroutineScope {
    // Spawn two concurrent child coroutines in parallel:
    val riskDeferred = async { fetchFraudRiskScoreAsync(accountId) }
    val fxDeferred   = async { fetchSpotFxRateAsync("USD", "EUR") }
    val account      = fetchAccountAsync(accountId)

    // If `fetchFraudRiskScoreAsync` throws an IOException after 5ms:
    // 1. `fxDeferred` is AUTOMATICALLY cancelled immediately!
    // 2. `coroutineScope` waits for `fxDeferred` cleanup (`finally` blocks), then
    rethrows!
    EnrichedAccount(
        account = account,
        riskScore = riskDeferred.await(),
        fxRate = fxDeferred.await(),
    )
}
```

Memorize the three guarantees of Kotlin's parent–child `Job` tree:

1. **Lifetime containment (no leaked background work):** a `coroutineScope { ... }` block cannot return to its caller until every child coroutine launched inside it has reached a terminal state (completed or finished cancelling).
2. **Downward cancellation:** cancelling the parent scope (for instance, when an HTTP client disconnects or `withTimeout(250.milliseconds)` expires) recursively cancels every descendant coroutine in the subtree.
3. **Upward failure propagation (`coroutineScope` vs. `supervisorScope`):** in a standard `coroutineScope`, if *any* child throws an uncaught exception (other than a cancellation exception), it immediately notifies the parent, which cancels all sibling children before propagating

the exception upward—exactly right when computing one unified answer like `EnrichedAccount`. Conversely, when a server loop or batch processor launches *independent* tasks where failure of task #1 must *not* abort task #2, wrap them in `supervisorScope { ... }`, which keeps downward cancellation intact while turning off upward sibling-killing.

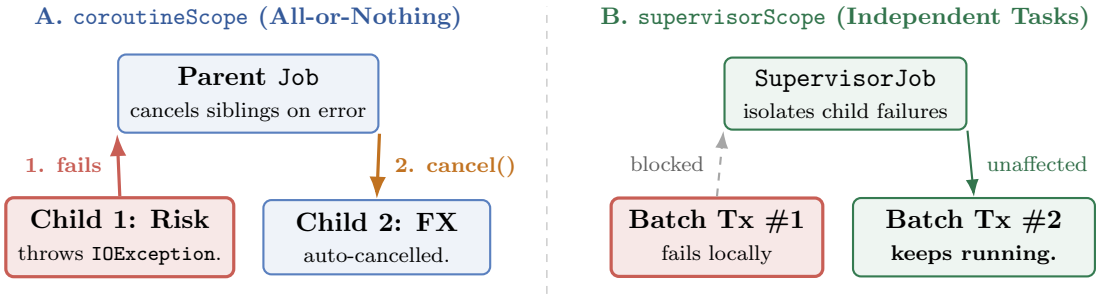



Figure 10.4: Failure propagation in structured concurrency. Left: in `coroutineScope`, an uncaught failure in Child 1 propagates up to the parent `Job`, which immediately cancels Child 2 before rethrowing. Right: in `supervisorScope`, Child 1's failure is handled locally without cancelling sibling Batch Tx #2, though cancelling the `SupervisorJob` from above still cancels all children.

**Common trap**

**Two Coroutine Cancellation Traps That Freeze Production Servers:**

1. **Swallowing `CancellationException` inside `catch (e: Exception)`.** How does Kotlin cancel a suspended coroutine? By resuming its continuation with a `kotlinx.coroutines.CancellationException`. If you write a retry loop `while (true) { try { doRpc() } catch (e: Exception) { log.warn(e) } }`, your broad `catch (e: Exception)` catches `CancellationException`, logs it, and keeps looping forever—turning the coroutine into an unkillable zombie. Always rethrow cancellation: `catch (e: CancellationException) { throw e }` (or use `runCatching` carefully, rethrowing if `!coroutineContext.isActive`).

2. **Cooperative CPU loops must check `ensureActive()`.** Unlike Java 6's deprecated `Thread.stop()`, coroutine cancellation never asynchronously corrupts memory mid-instruction; it checks for cancellation *at suspension points* (`delay`, `withContext`, non-blocking socket calls). A tight CPU loop `while (i < 1_000_000_000) { hash() }` has zero suspension points and will ignore cancellation unless you call `ensureActive()` or `yield()` periodically inside the loop.

### 10.5.3 Cold `Flow<T>` vs. Hot `Channel<T>` and `StateFlow<T>`

What if a suspending operation produces not one value (`suspend fun`: 0 or 1 async result), and not a synchronous pull stream (`Sequence<T>`: blocks caller thread), but an *asynchronous stream of values over time*—such as streaming ten million PostgreSQL ledger rows over a cursor or subscribing to live FX ticks? Kotlin separates asynchronous streams cleanly into **cold** and **hot** primitives:

- **Cold `Flow<T>` (lazy suspending recipe, with automatic backpressure):** Building a flow such as `val rows = flow { ... }`, whose body loops over a database cursor and calls

`emit(row)` for each row, starts *zero* I/O and allocates no background coroutine. Nothing runs until a consumer invokes the terminal suspending call `rows.collect { writeCsv(it) }`. Even better: `emit(value)` inside `flow { ... }` is itself a `suspend` **function** that directly calls the collector's lambda. If `writeCsv` takes 5 ms on slow disk I/O, `emit` suspends the database producer for 5 ms automatically—giving you lossless, zero-configuration **backpressure** without reactive-streams ceremony.

- **Hot `Channel<T>` and `StateFlow<T>` (active concurrent communication):** Whereas each `.collect()` on a cold `Flow` starts a fresh execution, a `Channel<T>` is a hot, thread-safe, suspending bounded queue between independent coroutines (`send` suspends when the buffer is full; `receive` suspends when empty—the exact Communicating Sequential Processes model we will study at the core of Go in Chapter 11.), and a `StateFlow<T>` holds a single current state value (such as live circuit-breaker health or FX snapshot) broadcast to all current subscribers.

Connection

**Looking Ahead to Chapter 11 (Go and the Grand Synthesis).** Chapters 9 and 10 covered two designs. TypeScript gave us structural set types with 100% compile-time erasure on a single-threaded event loop; Kotlin kept the nominal JVM heap and bytecode ecosystem while adding null-safety, `@JvmInline` value types, `sealed` sum hierarchies, and compiler-generated CPS coroutine state machines. Notice that in *both* TypeScript (`async`/`await`) and Kotlin (`suspend`), functions split into two "colors": synchronous functions vs. asynchronous/suspending functions. In Chapter 11, we meet **Go**—a language that threw out classes, exceptions, and function coloring altogether in favor of flat value structs, 16-byte implicit structural interfaces, and a runtime M:N scheduler running millions of 2 KB *stackful* goroutines—and then bring all eight languages of this book together in one unified engineering synthesis.

## 10.6 Exercises

**Exercise 10.1** (Reference Niche Optimization vs. Nullable Primitive Boxing)**.** Write down the exact JVM field descriptors emitted by `kotlinc` for all four properties of `class Snapshot(val id: Long, val parentId: Long?, val currency: String, val note: String?)`.

1. On a 64-bit JVM with 16-byte object headers and 8-byte references, compute the total heap memory (including any referenced `java.lang.Long` wrapper object when `parentId = 50000L`) consumed by one `Snapshot` instance, and explain why `note: String?` costs zero extra wrapper bytes over `currency: String` while `parentId: Long?` costs an extra 24-byte heap allocation.

2. Compare this layout byte for byte against Rust's `struct Snapshot { id: i64, parent_id: Option<i64>, currency: String, note: Option<String> }`.

**Exercise 10.2** (`@JvmInline value class` Mangling and Boxing Boundaries)**.** Define two inline value classes `UsdCents(val raw: Long)` and `EurCents(val raw: Long)`, each annotated with `@JvmInline`.

1. Identify every line that allocates a heap wrapper object in the following snippet: (a) `val a = UsdCents(500L)`; (b) `val b: UsdCents? = a`; (c) `val xs: List<UsdCents> = listOf(a)`;

(d) `val arr = LongArray(1000)`.

2. Design a zero-boxing `@JvmInline` value class `UsdCentsArray` wrapping a private `LongArray`, with `operator fun get(index: Int): UsdCents` and `operator fun set(index: Int, value: UsdCents)` so a million-element array of `UsdCents` occupies one flat 8 MB slab on the JVM heap with full compile-time currency type safety.

**Exercise 10.3** (Extension Dispatch vs. `sealed interface` Dispatch)**.** Suppose `sealed interface FeeRule` has two implementations: `data object ZeroFee` and `data class FlatFee(val cents: Long)`. A colleague defines two overloaded extension functions, `FeeRule.label()` returning `"generic"` and `FlatFee.label()` returning `"flat"`.

1. What does `val r: FeeRule = FlatFee(150L); println(r.label())` print, and what exact JVM instruction (`invokestatic` vs. `invokeinterface`) does `kotlinc` emit?

2. Rewrite `label()` cleanly in two idiomatic ways—once as a polymorphic member property on `FeeRule`, and once as a single extension function using an exhaustive `when (this)` expression—and explain the trade-off when adding new variants vs. adding new operations (the classic *Expression Problem* from Part III).

**Exercise 10.4** (Tracing `inline`, `noinline`, `crossinline`, and `reified`)**.** Explain why each of the following three Kotlin snippets fails to compile without a modifier change, and fix each one:

1. `fun <T : Any> checkType(x: Any): Boolean = x is T` (without `inline`/`reified`).

2. An `inline fun runOnPool` taking an `Executor` and a `block: () -> Unit`, whose body runs `exec.execute { block() }` (why can't a non-`crossinline` lambda be invoked inside another object or lambda?).

3. A 4-stage `Iterable` pipeline chaining `.filter`, `.map`, `.filter`, and `.firstOrNull()` over a 2,000,000-element `List<Tx>` where the very first element matches all predicates: count the intermediate list allocations before and after adding `.asSequence()`.

**Exercise 10.5** (Hand-Compiling a `suspend` Loop into a CPS State Machine)**.** Consider the suspending retry helper:

```
suspend fun fetchWithRetry(id: Long, maxAttempts: Int): Account {
    var attempt = 1
    while (attempt < maxAttempts) {
        try {
            return fetchAccountAsync(id)
        } catch (e: IOException) {
            delay(50L * attempt)
            attempt += 1
        }
    }
    return fetchAccountAsync(id)
}
```

1. Count the suspension points in `fetchWithRetry` and list the exact `label` states $(0, 1, \dots)$ and spilled fields on the generated `ContinuationImpl` subclass.

2. Trace why catching only `IOException` (rather than `Exception` or `Throwable`) lets a 100 ms `withTimeout` on an outer scope cancel the retry loop immediately during `delay`, whereas catching `Exception` would delay or defeat cancellation.

**Exercise 10.6** (`coroutineScope` vs. `supervisorScope` in a Ledger Settlement Batch)**.** You are implementing two endpoints: (A) `transferWithAudit(cmd)`, which concurrently checks fraud limits and AML sanctions before booking a single transfer (if either check fails, the transfer must abort immediately and cancel the other check); and (B) `settleEndOfDayBatch(accountIds)`, which settles 500 independent merchant accounts with bounded concurrency of 16 at a time (if merchant #42 fails, the remaining 499 merchants must still settle, while server shutdown must still cancel all 500). Write idiomatic Kotlin coroutine skeletons for both (A) and (B), specifying whether each uses `coroutineScope` or `supervisorScope` (plus `Semaphore(16)`) and verifying zero leaked coroutines in every error scenario.

# Chapter 11

# Go and the Grand Language Synthesis: Memory, Types, and Concurrency Compared

Chapters 9 and 10 showed two very different ways to move past Java 6. TypeScript keeps JavaScript's runtime and adds a rich, set-theoretic type checker that disappears at compile time. Kotlin keeps the JVM and fixes the language on top of it: nullable types, sealed hierarchies, inline value classes, and compiler-generated coroutine state machines.

Go answers a third question. In 2007, Robert Griesemer, Rob Pike, and Ken Thompson were writing large C++ and Java servers at Google, and three problems kept slowing them down: 45-minute C++ builds, deep class hierarchies that nobody could change safely, and thread-per-request servers that ran out of memory long before they ran out of CPU. They asked what a language would look like if it were designed for that environment from scratch: fast to compile, easy to read in a code review by someone who did not write it, and able to run a hundred thousand concurrent network connections in one process.

The answer they shipped is deliberately small. Go has no classes, no inheritance, no exceptions, no operator overloading, and (until 2022) no generics. What it does have is a short list of carefully chosen mechanisms: flat value structs, slices, implicit interfaces, explicit error values, `defer`, a garbage collector tuned for low pause times, and a runtime scheduler that multiplexes millions of cheap *goroutines* onto a handful of operating-system threads.

This chapter has two halves. In the first half we take Go apart the same way we took Kotlin apart: for every construct, we look at what it costs in memory and what the runtime actually does. In the second half we step back and line up all eight languages in this book—Java 6, modern C++, Rust, TypeScript, Kotlin, Go, OCaml, and Haskell—across seven axes, so that you leave with one coherent map instead of eight separate mental models.

---

## 11.1 Go's Design Trade-off: Values, Slices, Errors, and Fast Builds

History and connection

Go was sketched on a whiteboard at Google in September 2007, open-sourced in November 2009, and reached version 1.0 in March 2012 with a written compatibility promise: code that compiles under Go 1.0 should keep compiling under every later Go 1.x release. That promise has held for over a decade, which is part of why so much cloud infrastructure (Docker, Kubernetes, etcd, Prometheus, Terraform) is written in Go. The concurrency model comes from Tony Hoare's 1978 paper *Communicating Sequential Processes* (CSP), by way of Pike's earlier languages Newsqueak and Limbo.

### 11.1.1 Why Go Compiles So Fast

A Java 6 developer is used to `javac` being fast, because it only produces bytecode and leaves the heavy optimization to the JIT at runtime. A C++ developer is used to builds being slow, because every `#include` pastes thousands of header lines into every translation unit, and templates are re-instantiated and re-optimized in each one.

Go produces native machine code like C++, yet compiles large programs in seconds. Three design rules make that possible:

1. **No header files and no textual inclusion.** When package `ledger` imports package `money`, the compiler reads a compact export-data summary of `money` that was written when `money` itself was compiled. It never re-parses `money`'s source.
2. **Unused imports are a compile error, and import cycles are forbidden.** The dependency graph is a DAG that the build tool can compile in parallel, bottom-up, with no wasted work.
3. **A simple grammar and a modest optimizer.** Go's compiler does inlining, escape analysis, and bounds-check elimination, but nothing like the whole-program template expansion and link-time optimization that dominate C++ and Rust release builds. Go knowingly gives up some peak performance in exchange for a tight edit-compile-test loop.

### 11.1.2 Flat Value Structs and Escape Analysis

Here is the first place where Go feels more like C than like Java 6. In Java 6, every `new Posting(...)` lives on the garbage-collected heap behind a 16-byte object header, and an `ArrayList<Posting>` is an array of pointers to scattered objects. In Go, a `struct` is a plain value: its fields are laid out contiguously, it has no header, and a slice of structs is one contiguous block of memory, exactly like a C array or a Rust `Vec<Posting>`.

```
package ledger

type Posting struct {
    AccountID int64 // offset 0
    Cents     int64 // offset 8
}                  // sizeof(Posting) == 16 bytes, no header
```

```
7
8  // []Posting with 1,000,000 elements = one 16 MB contiguous block.
9  // Java 6 ArrayList<Posting> = 8 MB of pointers + 1,000,000 separate
10 // 32-byte heap objects scattered across the heap.
11 func TotalCents(ps []Posting) int64 {
12     var total int64
13     for i := range ps {
14         total += ps[i].Cents // sequential, cache-friendly loads
15     }
16     return total
17 }
```

Go still has a garbage collector, though. So which values live on the stack, and which on the heap? You never write `new` versus stack allocation explicitly. Instead, the compiler runs **escape analysis**: for every variable, it asks whether any pointer to that variable can outlive the function call that created it. If not, the variable stays in the function's stack frame and costs nothing to free. If a pointer might escape (it is returned, stored in a heap object, captured by a goroutine, or converted to an interface whose lifetime the compiler cannot bound), the variable is moved to the heap.

```
1  func sumFees(n int) int64 {
2      var total int64
3      for i := 0; i < n; i++ {
4          p := Posting{AccountID: int64(i), Cents: 150} // stays on the stack
5          total += p.Cents
6      }
7      return total
8  }
9
10 func newPosting(id int64) *Posting {
11     p := Posting{AccountID: id, Cents: 150}
12     return &p // pointer outlives the call: p is moved to the heap
13 }
14
15 // $ go build -gcflags=-m ./ledger
16 // ./ledger.go:14:2: moved to heap: p
```

Look at `newPosting`. In C, returning `&p` for a local `p` is the classic dangling-pointer bug. In Rust, the borrow checker rejects it at compile time (Chapter 4). In Go, it is perfectly safe: the compiler sees the escape and quietly allocates `p` on the heap instead. You get C's value layout when the compiler can prove it is safe and Java's heap safety when it cannot, with `-gcflags=-m` telling you which one you got.

**Intuition**

Rust and Go both ask "does this reference outlive its referent?" at compile time. They do opposite things with the answer. Rust treats an escaping reference as a *bug* and refuses to compile, which forces you to choose an owner (`Box`, `Arc`, a returned value). Go treats an escaping reference as a *fact* and silently promotes the value to the garbage-collected heap. Rust's approach gives you full control and zero GC; Go's approach gives you simpler code and a GC that has to clean up afterwards.

### 11.1.3 Slices: A 24-Byte Header Over a Shared Backing Array

A Go slice `[]T` is not an object. It is a three-word value, 24 bytes on a 64-bit machine, that describes a window into some backing array:

```
// What the runtime actually stores for a []T (runtime/slice.go):
type slice struct {
    array unsafe.Pointer // pointer to element 0 of the window
    len   int            // number of elements you may index
    cap   int            // elements available before reallocation
}
```

That is the same three words as Rust's `Vec<T>` (pointer, length, capacity), and the same idea as Rust's `&[T]` fat pointer with a capacity field added. Passing a slice to a function copies those 24 bytes, never the elements. Slicing an existing slice (`ps[2:5]`) creates a new header pointing into the same backing array, again without copying.

The built-in `append(s, x)` writes `x` into slot `len` if `len < cap`, and returns a header with `len+1`. If the backing array is full, it allocates a larger array (roughly doubling for small slices, growing about 1.25× for large ones), copies the elements, and returns a header pointing at the new array. That rule creates the most common Go aliasing bug, shown in Figure 11.1:

```
base := make([]int64, 3, 4) // len 3, cap 4: one free slot
a := append(base, 100)      // fits in cap: writes slot 3 of SHARED array
b := append(base, 200)      // also fits: overwrites the same slot 3!
fmt.Println(a[3], b[3])     // prints "200 200", not "100 200"
```

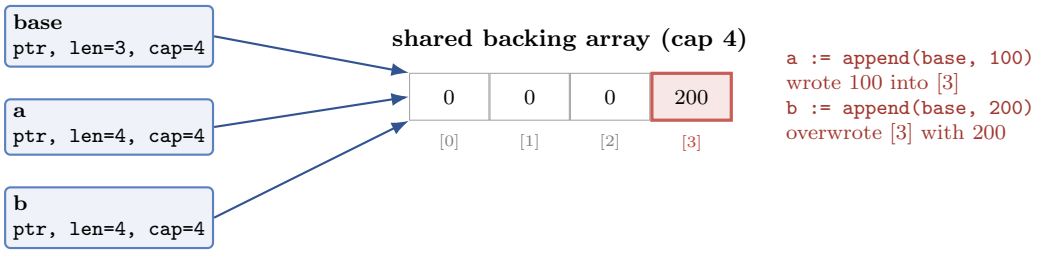



Figure 11.1: Three slice headers, one backing array. Because `base` had spare capacity, both `append` calls wrote into the same slot 3, so `a[3]` and `b[3]` both read 200. Rust's borrow checker rejects the equivalent code, because two live mutable views of the same buffer violate aliasing-XOR-mutability.

**Common trap**

**Rule of thumb for slices you did not create:** if a function receives a slice and wants to append to it and keep the result separately, copy first (`out := append([]int64(nil), in...)`) or use a full slice expression `in[:len(in):len(in)]`, which caps the capacity so the next `append` is forced to reallocate. The Go race detector (`go test -race`) catches concurrent versions of this bug at runtime; nothing catches the single-goroutine version except careful reading.

Strings get the same treatment: a Go `string` is a 16-byte (pointer, length) header over immutable bytes, exactly like Rust's `&str`. Substring slicing `s[4:10]` is free and shares memory.

### 11.1.4 Errors Are Values, and `defer` Runs Cleanup

Go has no `try`/`catch`. A function that can fail returns an extra result of the built-in interface type `error`, and the caller checks it explicitly:

```
var ErrInsufficientFunds = errors.New("insufficient funds")

func PostBatch(db *sql.DB, batch []Posting) (err error) {
    tx, err := db.Begin()
    if err != nil {
        return fmt.Errorf("begin tx: %w", err) // %w wraps, keeping the cause
    }
    // Deferred calls run when PostBatch returns, in LIFO order,
    // whether it returns normally or with an error.
    defer func() {
        if err != nil {
            _ = tx.Rollback()
        }
    }()

    for _, p := range batch {
        _, err = tx.Exec(
            "INSERT INTO postings (account_id, cents) VALUES ($1, $2)",
            p.AccountID, p.Cents)
        if err != nil {
            return fmt.Errorf("insert posting for account %d: %w", p.AccountID, err)
        }
    }
    return tx.Commit()
}

// Caller inspects the chain of wrapped errors:
if err := PostBatch(db, batch); errors.Is(err, ErrInsufficientFunds) {
    // map to HTTP 422
}
```

Compare this with the other error models we have seen. Java 6 checked exceptions force callers to acknowledge failure but make the failing path invisible at the call site (any line inside the `try` could have thrown). Rust's `Result<T, E>` plus `?` makes every failing call visible with one character and forces you to handle the error type. Go sits between them: every failing call is visible (`if err != nil`), but the compiler does not force you to look at `err`, and `error` is a single open interface type rather than a precise enum of failure cases. The `%w` verb, `errors.Is`, and `errors.As` give you a linked chain of causes that you can query, which plays the role of Java's `getCause()`.

**Common trap**

**The compiler errors you will see in your first hour of Go.** Go turns several things that are warnings elsewhere into hard errors, so that unused code never accumulates:

```
./ledger.go:4:2: "strings" imported and not used
./ledger.go:9:2: declared and not used: total
./ledger.go:15:1: missing return
```

What the compiler does *not* reject is ignoring an error result: `tx.Exec(...)` on a line by itself compiles, and the error vanishes. Linters such as `errcheck` (included in `golangci-lint`) catch that; Rust's equivalent is the built-in `#[must_use]` warning on `Result`.

Go does have `panic` and `recover`, which unwind the stack like an exception. By convention they are reserved for programmer bugs (index out of range, nil map write) and for a top-level safety net in HTTP servers, the same role Rust gives to `panic!`.

`defer` deserves its own note. It is Go's answer to C++ RAII destructors and Java's `finally`: `defer f.Close()` schedules the call to run at function exit. Since Go 1.14, most `defer` statements are compiled "open-coded" straight into the function's exit paths, so they cost about as much as a direct call. Two details trip people up: the deferred call's *arguments* are evaluated when the `defer` statement runs, not when the function exits; and `defer` is scoped to the function, not the block, so a `defer` inside a loop over ten thousand files keeps ten thousand files open until the function returns.

## 11.2 Implicit Interfaces: 16-Byte Fat Pointers and the Typed-Nil Trap

In Chapter 9 we met structural typing in TypeScript, where it exists only at compile time. Go interfaces are also structural—a type satisfies an interface simply by having the right methods, with no `implements` clause—but unlike TypeScript, Go interfaces have a concrete runtime representation, and you need to understand it.

```
type Notifier interface {
    Notify(accountID int64, msg string) error
}

type EmailNotifier struct{ From string }

// No "implements Notifier" anywhere. Having this method is enough.
func (e *EmailNotifier) Notify(accountID int64, msg string) error {
    return sendMail(e.From, accountID, msg)
}

var n Notifier = &EmailNotifier{From: "ledger@bank.example"}
```

This is how Go lets you define an interface in the package that *consumes* it, after the fact, and have types from third-party packages satisfy it automatically—something Java 6 could never do, because Java needs the `implements` clause on the class declaration.

### 11.2.1 What an Interface Value Looks Like in Memory

A variable of a non-empty interface type like `Notifier` is a two-word value, 16 bytes on a 64-bit machine:

```
// runtime/runtime2.go (simplified)
type iface struct {
    tab  *itab          // which concrete type, plus its method table
```

```
    data unsafe.Pointer // pointer to the concrete value
}

type itab struct {
    inter *interfacetype // the interface (Notifier)
    _type *_type         // the concrete type (*EmailNotifier)
    hash  uint32         // copy of _type.hash, used by type switches
    _     [4]byte
    fun   [1]uintptr     // method pointers; variable length, starts at offset 24
}
```

Calling `n.Notify(42, "hi")` loads `n.tab`, loads the function pointer at `tab.fun[0]`, and calls it with `n.data` as the receiver. That is two dependent loads and an indirect call—the same cost as a C++ virtual call or a Rust `dyn Trait` call. In fact the layout is the same as Rust's `&dyn Notifier`, which is also a (data pointer, vtable pointer) pair; Go just puts the type pointer first. When both the concrete type and the interface are known at compile time (as in the assignment to `n` above), the compiler emits the `itab` as a static read-only symbol. Only dynamic conversions—a type assertion to an interface type, or converting one interface to another—have to look up or build the `itab` at runtime, using a global cache keyed by the (interface, concrete type) pair.

The empty interface `any` (spelled `interface{}` before Go 1.18) uses a slightly smaller header, `eface`, with a bare `_type` pointer instead of an `itab`, since there are no methods to look up. Storing a non-pointer value larger than a machine word (such as a 16-byte `Posting`) in an interface usually copies it to the heap so that `data` has something to point at. That boxing is invisible in the source, and it is the main reason hot Go loops avoid `any`.

### 11.2.2 The Typed-Nil Interface Trap

Now we can explain a well-known Go bug. Read this function carefully:

```
type LedgerError struct{ Code string }

func (e *LedgerError) Error() string { return e.Code }

func validate(cents int64) error {
    var lerr *LedgerError // a nil *LedgerError pointer
    if cents < 0 {
        lerr = &LedgerError{Code: "NEGATIVE_AMOUNT"}
    }
    return lerr // BUG: converts a typed nil pointer to a non-nil error
}

func main() {
    if err := validate(500); err != nil {
        fmt.Println("rejected:", err) // runs, and prints "rejected: <nil>"
    }
}
```

The amount is positive, `lerr` is `nil`, and yet `err != nil` is true. Figure 11.2 shows why. An interface value is `nil` only when *both* words are zero. When `return lerr` converts the `*LedgerError` into an `error`, Go fills in `tab` with the `itab` for `(error, *LedgerError)`, because the static type of

`lerr` is known, and copies the nil pointer into `data`. The result is an interface that says "I hold a `*LedgerError`, and that pointer happens to be nil." The comparison `err != nil` compares against an interface with both words zero, so it reports *not equal*.

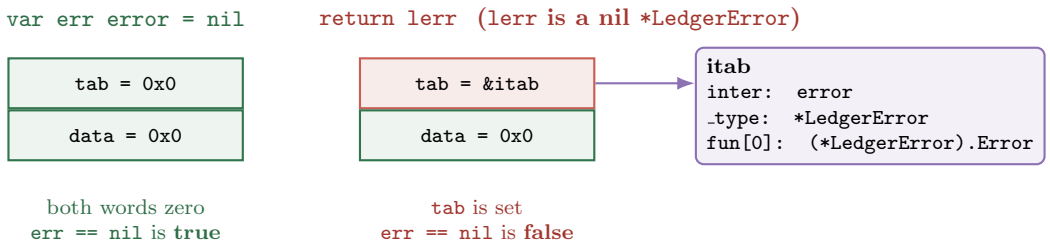



Figure 11.2: The typed-nil trap. A Go interface value is a (`tab`, `data`) pair, and it equals `nil` only when both words are zero. Converting a nil `*LedgerError` to `error` fills in `tab`, so the result is a non-nil interface holding a nil pointer.

The compiler accepts this code. The `staticcheck` linter flags some forms of it (check SA4023, "impossible comparison of interface value with untyped nil"), but the reliable fix is a habit: functions that return `error` should return the literal `nil` on success, never a typed pointer variable that might be nil.

```
func validate(cents int64) error {
    if cents < 0 {
        return &LedgerError{Code: "NEGATIVE_AMOUNT"}
    }
    return nil // untyped nil: both interface words are zero
}
```

**Connection**

Compare the same situation in the other languages. In Rust, a function returning `Result<(), LedgerError>` cannot accidentally return "an error that is not there": `Ok(())` and `Err(e)` are different enum variants. In Kotlin, `LedgerError?` is one nullable reference with no hidden type word. In TypeScript, interfaces are erased, so there is nothing to fill in. Go's trap exists because its interfaces are structural *and* carry a runtime type word, and because `nil` is allowed for both pointers and interfaces.

### 11.2.3 No Sum Types: Interfaces Plus Type Switches

Go has no `enum` with payloads, no `sealed` keyword, and no exhaustive `match`. The idiomatic substitute is an interface with an unexported marker method (so only types in the same package can satisfy it) plus a `type switch`:

```
type AuthOutcome interface{ isAuthOutcome() }

type Approved struct{ AuthCode string; Cents int64 }
type Declined struct{ Reason string }

func (Approved) isAuthOutcome() {}
```

```
func (Declined) isAuthOutcome() {}

func HTTPStatus(o AuthOutcome) int {
    switch v := o.(type) { // compares o's type word against each case
    case Approved:
        _ = v.AuthCode
        return 200
    case Declined:
        return 422
    default:
        panic("unhandled AuthOutcome") // compiler cannot prove exhaustiveness
    }
}
```

The `default: panic(...)` line is the giveaway. Kotlin's `sealed interface` and Rust's `enum` let the compiler reject a `when` or `match` that forgets a variant. Go cannot, so teams that rely on this pattern add a linter (such as `exhaustive` or `go-sumtype`) to their CI.

### 11.2.4 Generics Since Go 1.18: GC-Shape Stenciling

Go added type parameters in Go 1.18 (2022). The syntax is familiar:

```
type Number interface{ ~int64 | ~float64 }

type Cents int64 // ~int64 means "any type whose underlying type is int64"

func Sum[T Number](xs []T) T {
    var total T
    for _, x := range xs {
        total += x
    }
    return total
}

var fees = []Cents{150, 250, 99}
var total = Sum(fees) // T = Cents
```

The implementation is a deliberate middle ground between Java's erasure and Rust's monomorphization. The Go compiler generates one copy of `Sum` per *GC shape*: all type arguments with the same size, alignment, and pointer layout share one compiled body. So `Sum[int64]` and `Sum[Cents]` share code, and every pointer type (`*Account`, `*Posting`) shares one body too. Where the shared body needs type-specific behaviour (calling a method, or comparing values), it receives a hidden *dictionary* argument with the necessary function pointers, much like Haskell's type-class dictionaries. The result: compile times and binary sizes stay close to non-generic Go, at the cost of an extra indirection in some generic calls that Rust or C++ would have specialized away.

## 11.3 Go's M:N Runtime Scheduler: Goroutines, Channels, and Context

### 11.3.1 Goroutines: 2 KB Stacks That Grow by Copying

Writing `go settle(id)` starts a new goroutine. It looks like Java 6's `new Thread(r).start()`, but the cost is completely different. Let us redo the arithmetic from Chapter 10 for one hundred thousand concurrent requests:

- **Java 6 threads:** each `java.lang.Thread` is a kernel thread with a fixed stack reservation of about 1 MB. $100{,}000 \times 1$ MB $\approx 100$ GB of reserved stack address space. Long before that, the process hits the operating system's per-process thread limit and thread creation fails.
- **Go goroutines:** each goroutine starts with a **2 KB** stack plus a few hundred bytes of runtime bookkeeping (the `g` struct). $100{,}000 \times 2$ KB $\approx 200$ MB, and most real goroutines that just wait on a socket never grow beyond that.

How can a stack start at 2 KB without overflowing on the first deep call chain? Every Go function begins with a short prologue that compares the stack pointer against a guard value stored in the goroutine's `g` struct. If the next frame would not fit, the prologue calls `runtime.morestack`, which allocates a new stack twice the size, copies the old stack into it, fixes up every pointer that points into the old stack, and resumes. Stacks can grow to 1 GB on 64-bit systems.

That copy step is only possible because of a guarantee Go's design maintains everywhere: the runtime knows the exact location of every pointer into a goroutine's stack, and escape analysis guarantees that no heap object ever points into a stack. Rust cannot do this. Rust references can point anywhere on the stack, raw pointers can be cast to integers, and there is no runtime to fix them up—which is a big part of why Rust removed its green-thread runtime before 1.0 (in 2014) and later chose stackless `async` state machines instead (Chapter 12).

### 11.3.2 The G-M-P Model

The Go runtime schedules goroutines with three kinds of objects:

- **G (goroutine):** a stack, an instruction pointer, and status. Millions can exist.
- **M (machine):** an operating-system thread. The runtime creates them as needed.
- **P (processor):** a scheduling context with a local run queue of up to 256 runnable Gs. There are exactly `GOMAXPROCS` Ps, which defaults to the number of CPU cores. An M must hold a P to run Go code.

Figure 11.3 shows how they fit together. Each M takes its P's local queue and runs Gs one after another. When a P's queue runs dry, its M first checks the global run queue, then the network poller, and then *steals half* of another P's queue. Every 61st scheduling decision, a P also checks the global queue first so that goroutines parked there cannot starve.

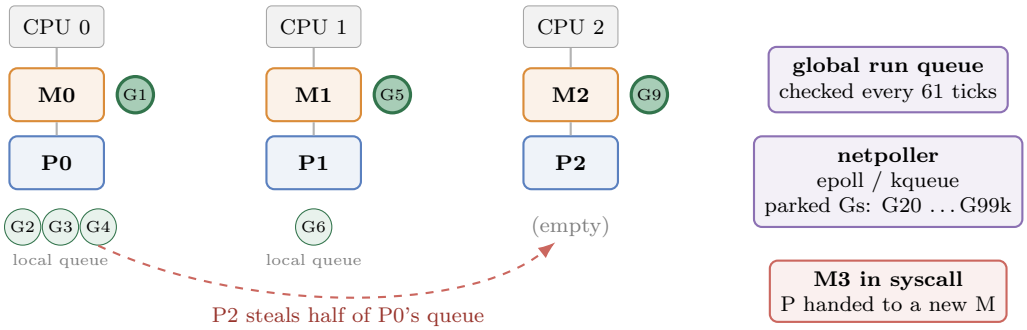



Figure 11.3: The G-M-P scheduler with `GOMAXPROCS=3`. Each OS thread (M) holds one P and runs goroutines (G) from that P's local queue. Idle Ps steal work from busy ones. Goroutines blocked on network I/O wait in the netpoller and cost no thread; a thread blocked in a system call gives up its P so the other goroutines keep running.

Three more mechanisms complete the picture:

1. **Network I/O never blocks a thread.** When a goroutine calls `conn.Read` and no data is ready, the runtime registers the socket with the operating system's `epoll` (Linux) or `kqueue` (macOS/BSD), parks the goroutine, and the M picks another G. When data arrives, the netpoller marks the goroutine runnable again. You write blocking-style code; the runtime does the event loop for you.
2. **Blocking system calls hand off the P.** File I/O and cgo calls really do block the OS thread. When that happens, a background monitor thread (`sysmon`) notices after about 20 microseconds and hands the P to another M, so the remaining goroutines keep running on all cores.
3. **Preemption.** Since Go 1.14, `sysmon` sends a signal to any goroutine that has run for more than about 10 ms without yielding, forcing it to stop at the next safe point. A tight CPU loop can no longer freeze a P, which is a real difference from Kotlin coroutines, where a loop without suspension points ignores cancellation and holds its thread.

Focus

**No function coloring.** In TypeScript, Kotlin, and Rust, a function that waits on I/O must be marked (`async`, `suspend`, `async fn`), and only marked functions can call other marked functions directly. This splits every codebase into two "colors." Go has one color. Every function can block, because blocking just parks a goroutine with its whole stack. The price is a runtime that owns your threads and a stack per goroutine; the payoff is that a library written in plain synchronous style works unchanged inside a server handling a hundred thousand connections.

### 11.3.3 Channels and `select`: Communicating Sequential Processes

Go's motto is "Do not communicate by sharing memory; instead, share memory by communicating." A channel `chan T` is a typed, thread-safe queue. Internally it is a heap object (`runtime.hchan`) containing a circular buffer, send and receive indices, a lock, and two wait queues of parked goroutines.

- **Unbuffered channel** (`make(chan T)`): a send blocks until a receiver takes the value. The

two goroutines meet at that point, so the send also acts as a synchronization point. If every goroutine in the program ends up blocked on a channel, the runtime detects it and exits with `fatal error: all goroutines are asleep - deadlock!`. (It only detects the case where *all* goroutines are stuck; a partial deadlock in a server just looks like a hang.)

- **Buffered channel** (`make(chan T, 64)`): a send blocks only when the buffer is full; a receive blocks only when it is empty. The buffer size is a built-in backpressure limit.
- **Closing:** `close(ch)` tells receivers that no more values are coming. Receiving from a closed, drained channel returns the zero value immediately, and `v, ok := <-ch` reports `ok == false`. Sending on a closed channel panics, so by convention only the sender closes.
- **nil channels block forever**, which sounds useless until you use it in `select` to switch off a case dynamically.

The `select` statement waits on several channel operations at once and runs whichever becomes ready first. If several are ready, it picks one *uniformly at random*, so no case can be starved:

```
func bestQuote(ctx context.Context, primary, backup <-chan Quote) (Quote, error) {
    timeout := time.NewTimer(150 * time.Millisecond)
    defer timeout.Stop()
    select {
    case q := <-primary:
        return q, nil
    case q := <-backup:
        return q, nil
    case <-timeout.C:
        return Quote{}, errors.New("fx quote timed out")
    case <-ctx.Done(): // caller gave up (client disconnected, deadline hit)
        return Quote{}, ctx.Err()
    }
}
```

Channels are not always the right tool. For protecting a small piece of shared state (a balance map, a counter), a mutex is simpler and faster:

```
type Ledger struct {
    mu       sync.Mutex
    balances map[int64]int64 // guarded by mu (by convention only!)
}

func (l *Ledger) Transfer(from, to, cents int64) error {
    l.mu.Lock()
    defer l.mu.Unlock()
    if l.balances[from] < cents {
        return ErrInsufficientFunds
    }
    l.balances[from] -= cents
    l.balances[to] += cents
    return nil
}
```

Note the comment on `balances`: "guarded by `mu`, by convention only." Nothing stops another method from reading `l.balances` without taking the lock. This is the same detached-lock design as Java 6's

`synchronized`, and it is exactly what Rust's `Mutex<HashMap<i64, i64>>` prevents (Chapter 3): in Rust the map lives *inside* the mutex, and the only way to reach it is through the guard returned by `lock()`.

**Common trap**

**Data races in Go are not just logic bugs.** Go is memory-safe in single-goroutine code, but a data race on a multi-word value can corrupt memory. If one goroutine writes an interface variable (two words: `tab` and `data`) while another reads it, the reader can see the new `tab` with the old `data`, and then call a method of one type on the bytes of another. The same applies to slice headers (three words) and maps (concurrent map writes crash the process with a fatal error). Rust rules out these races at compile time through the `Send` and `Sync` traits; Go relies on the race detector (`go test -race`) and code review. Run your tests with `-race` in CI.

### 11.3.4 `context.Context`: Cancellation Down a Tree of Goroutines

In Chapter 10 we saw Kotlin's structured concurrency: every coroutine has a parent `Job`, and cancelling the parent cancels the children. Go has no built-in parent-child relation between goroutines. A goroutine started with `go f()` runs until `f` returns, and nothing else can stop it. Go achieves the same discipline by convention, with `context.Context` passed as the first argument to every function that does I/O or might take a while:

```
func SettleAll(ctx context.Context, ids []int64, workers int) error {
    ctx, cancel := context.WithCancel(ctx) // child context
    defer cancel()                         // always release it

    jobs := make(chan int64)
    errc := make(chan error, workers) // each worker sends at most one error
    var wg sync.WaitGroup

    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for id := range jobs { // loop ends when jobs is closed
                if err := settleOne(ctx, id); err != nil {
                    errc <- err
                    cancel() // tell the producer and other workers to stop
                    return
                }
            }
        }()
    }

    go func() { // producer
        defer close(jobs)
        for _, id := range ids {
            select {
            case jobs <- id:
            case <-ctx.Done():
                return
```

```
            }
        }
    }()

    wg.Wait()
    close(errc)
    return <-errc // first error, or nil if the channel is empty
}
```

A `Context` carries a `Done()` channel that is closed when the context is cancelled or its deadline passes. Contexts form a tree: `WithCancel`, `WithTimeout`, and `WithDeadline` create children, and cancelling a parent closes every descendant's `Done()` channel. Every well-behaved blocking call (`http.Client`, `database/sql`, gRPC) takes a context and returns early with `ctx.Err()` when it fires.

The difference from Kotlin is enforcement. In Kotlin, `coroutineScope` cannot return while a child is still running, so leaks are structurally impossible. In Go, if you forget to pass `ctx` down, or start a goroutine that never checks `ctx.Done()`, it keeps running after its caller has returned: a *goroutine leak*. The `golang.org/x/sync/errgroup` package packages the pattern above (start goroutines, cancel all on first error, wait for all) into a few lines, and it is the closest Go equivalent of Kotlin's `coroutineScope`.

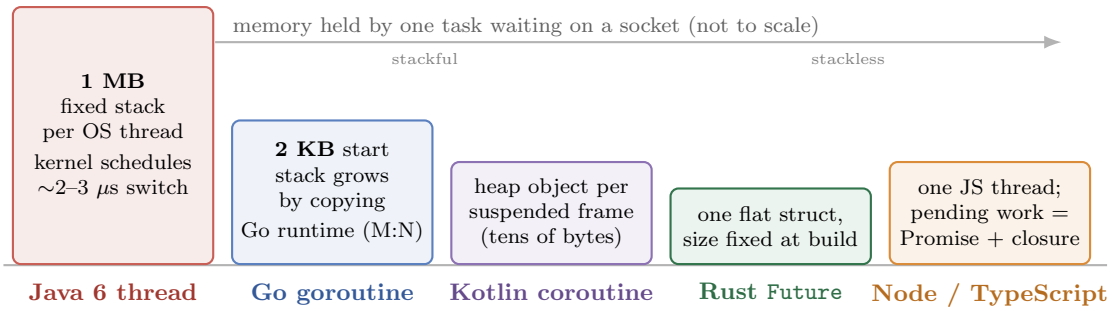



Figure 11.4: What one waiting task costs in each concurrency model. Java 6 pays for a full kernel thread stack. Go keeps a small growable stack per goroutine, so any function can block without being marked. Kotlin and Rust compile waiting functions into state machines that keep only live variables; Node runs all JavaScript on one thread and stores pending work as promise callbacks.

## 11.4 The Grand Synthesis: Eight Languages on Seven Axes

We have now looked inside eight languages. This section lines them up so you can compare them directly. The goal is not to pick a winner; each language is a set of trade-offs that made sense for the problems its designers had. The goal is that when you meet a new codebase in any of them, you can predict where its bugs and its costs will come from.

### 11.4.1 One Task, Five Languages

Start with one concrete task that every backend does: load an account, and *in parallel* fetch its fraud-risk score and a currency rate. If either fetch fails, cancel the other and return the error. Here is the same task in five of our languages, from Java 6 to Rust.

```java
// Java 6: thread pool + Future. No structured cancellation.
ExecutorService pool = Executors.newFixedThreadPool(16);
Future<Integer> risk = pool.submit(new Callable<Integer>() {
    public Integer call() throws Exception { return fetchRisk(accountId); }
});
Future<BigDecimal> fx = pool.submit(new Callable<BigDecimal>() {
    public BigDecimal call() throws Exception { return fetchFx("USD", "EUR"); }
});
try {
    return new Enriched(fetchAccount(accountId), risk.get(), fx.get());
} catch (ExecutionException e) {
    risk.cancel(true); // you must remember to cancel the sibling yourself
    fx.cancel(true);
    throw e;
}
```

```typescript
// TypeScript: Promise.all fails fast but does NOT cancel the other request.
const ctrl = new AbortController();
try {
  const [account, risk, fx] = await Promise.all([
    fetchAccount(accountId, ctrl.signal),
    fetchRisk(accountId, ctrl.signal),
    fetchFx('USD', 'EUR', ctrl.signal),
  ]);
  return { account, risk, fx };
} catch (e: unknown) {
  ctrl.abort(); // explicit: tell the losers to stop
  throw e;
}
```

```kotlin
// Kotlin: coroutineScope cancels the siblings and waits for them automatically.
suspend fun enrich(id: Long): Enriched = coroutineScope {
    val risk = async { fetchRisk(id) }
    val fx = async { fetchFx("USD", "EUR") }
    Enriched(fetchAccount(id), risk.await(), fx.await())
}
```

```go
// Go: errgroup cancels ctx on the first error; each call must honor ctx.
func enrich(ctx context.Context, id int64) (Enriched, error) {
    g, ctx := errgroup.WithContext(ctx)
    var risk int
    var fx float64
    g.Go(func() (err error) { risk, err = fetchRisk(ctx, id); return })
    g.Go(func() (err error) { fx, err = fetchFx(ctx, "USD", "EUR"); return })
    acct, err := fetchAccount(ctx, id)
    if werr := g.Wait(); werr != nil {
        return Enriched{}, werr
    }
    if err != nil {
        return Enriched{}, err
    }
    return Enriched{acct, risk, fx}, nil
}
```

```rust
// Rust: try_join! polls all three futures; on the first Err, it drops the
// others, and dropping a future cancels it (Chapter 12).
async fn enrich(id: i64) -> Result<Enriched, Error> {
    let (account, risk, fx) =
        tokio::try_join!(fetch_account(id), fetch_risk(id), fetch_fx("USD", "EUR"))?;
    Ok(Enriched { account, risk, fx })
}
```

Read the five versions for *where cancellation lives*. In Java 6 and TypeScript, it is your job, and forgetting it leaks work. In Go, it is a convention (`ctx`) that every callee must follow. In Kotlin, the scope enforces it. In Rust, it falls out of ownership: a future that is dropped simply never gets polled again, and its destructor runs. The tables below make the same kind of comparison along each axis.

### 11.4.2 Axes 1 and 2: Memory and Ownership, Dispatch and Polymorphism

| Language | Who frees memory, and where values live | What goes wrong in practice |
|---|---|---|
| Java 6 | Tracing GC. Every object on the heap with a 12–16 B header; only primitives are inline. | GC pauses, pointer-chasing cache misses, `Integer` boxing in collections. |
| C++ | You do, via RAII: destructors run at scope exit; `unique_ptr`, `shared_ptr`. | Use-after-free and dangling references after a move or a `vector` reallocation. |
| Rust | Ownership: exactly one owner; the compiler inserts `drop`. Borrows checked at compile time. | Fighting the borrow checker on graphs and callbacks; reaching for `Rc<RefCell<T>>` or arenas. |
| TypeScript | V8's generational GC. All objects on the heap, shaped by hidden classes. | Megamorphic property access; retained closures leaking memory in long-lived Node processes. |
| Kotlin | JVM tracing GC. `@JvmInline value class` removes the header for one-field wrappers. | Hidden boxing of `Long?`, value classes in generic collections. |
| Go | Concurrent, non-moving tracing GC; escape analysis keeps non-escaping structs on the stack. | Hidden heap escapes (`-gcflags=-m`); GC CPU cost at high allocation rates; slice aliasing. |
| OCaml | Generational GC; 63-bit integers unboxed via a tag bit; immutable by default. | Floats in polymorphic code are boxed; 63-bit `int` surprises when porting 64-bit arithmetic. |
| Haskell | Generational GC; lazy evaluation allocates thunks for unevaluated expressions. | Space leaks from thunk chains (`foldl` vs `foldl'`). |

Table 11.1: Axis 1: memory and ownership.

| Language | How a call on an abstract type finds its code | Consequence |
|---|---|---|
| Java 6 | Nominal classes and interfaces; `invokevirtual` through a vtable in the class, found via the object's `klass` pointer. | Must declare `implements` up front; JIT inlines monomorphic call sites. |
| C++ | `virtual` functions through a vptr stored in each object; templates for static dispatch. | Two separate worlds (OOP vs templates) with different error messages and costs. |
| Rust | Traits. Generics are monomorphized (static); `dyn Trait` is a 16 B (data, vtable) fat pointer. | No vptr inside objects; you choose static or dynamic per call site. |
| TypeScript | Structural at compile time; at runtime, JS property lookup via prototype chain and inline caches. | Any object with the right shape fits; nothing checks it at runtime. |
| Kotlin | JVM virtual dispatch for members; extension functions are *static* calls. | Extensions do not override: the declared type picks the function. |
| Go | Implicit structural interfaces; 16 B (`itab`, data) value; no inheritance, struct embedding instead. | Interfaces defined by the consumer; typed-nil trap; no exhaustiveness. |
| OCaml | Modules and functors (static); structural object types and polymorphic variants. | Very fast static code; functors replace most uses of interfaces. |
| Haskell | Type classes, compiled to dictionary arguments (often specialized away). | Ad-hoc polymorphism resolved by types, including on the return type. |

Table 11.2: Axis 2: dispatch and polymorphism.

### 11.4.3 Axes 3 and 4: Absence, Alternatives, and Failure

| Language | Nullability and sum types | Error handling |
|---|---|---|
| Java 6 | Every reference may be `null`. No sum types; Visitor pattern or `instanceof` chains. | Checked and unchecked exceptions; `finally` for cleanup. |
| C++ | `nullptr`; `std::optional`, `std::variant` (C++17) with `std::visit`. | Exceptions or error codes; `std::expected` (C++23). |
| Rust | No null. `Option<T>`; `enum` with exhaustive `match`; niche optimization. | `Result<T, E>` and `?`; `panic!` for bugs. |
| TypeScript | `T \| null` under `strictNullChecks`; discriminated unions with `never` checks. | `throw` any value; `catch (e: unknown)`; Result-style libraries. |
| Kotlin | `T` vs `T?`; `sealed` hierarchies with exhaustive `when`. | Unchecked exceptions only; `Result` and `runCatching`. |
| Go | `nil` pointers, maps, slices, channels, interfaces. No sum types; interface plus type switch. | `error` values returned alongside results; `panic`/`recover` for bugs. |
| OCaml | No null; `option`; variants with exhaustiveness warnings (the model Rust copied). | Exceptions (cheap) and `result`; both idiomatic. |
| Haskell | No null; `Maybe`; algebraic data types with pattern matching. | `Either`, `Maybe`, monad transformers; IO exceptions. |

Table 11.3: Axes 3 and 4: absence and alternatives, and how failure is reported.

### 11.4.4 Axes 5 and 6: Generics Implementation and Concurrency Runtime

Generics and memory management each fall on a spectrum. Figure 11.5 places the languages on both.

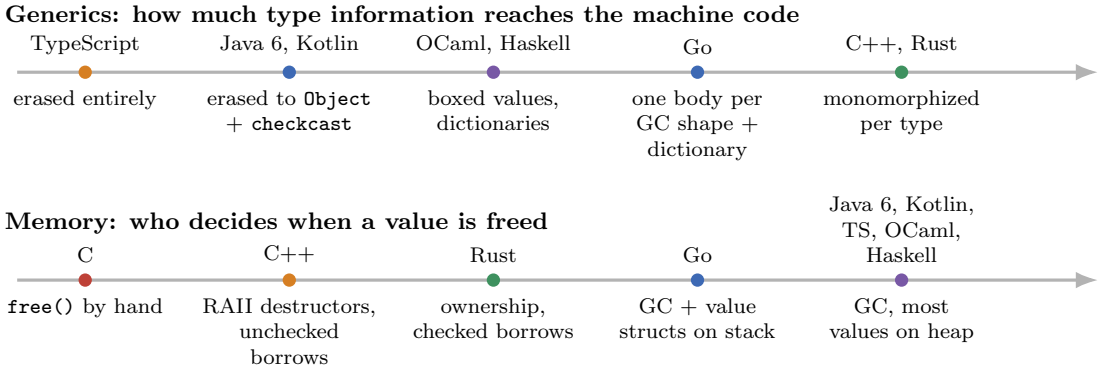



Figure 11.5: Two spectra. Top: how much of a generic type parameter survives into compiled code, from TypeScript (nothing) to C++ and Rust (a separate copy of the code per type). Bottom: who decides when memory is freed, from C (the programmer, unchecked) through Rust (the compiler, checked) to fully garbage-collected languages (the runtime).

Moving right on the generics line buys speed (each copy is specialized and inlinable) at the cost of compile time and binary size. Moving right on the memory line gives up control over when memory

is freed in exchange for not having to think about it. Rust is unusual in sitting near the "control" end of the memory line while still being checked; Go is unusual in being garbage-collected while still laying out structs and slices flat, like C.

| Language | Concurrency runtime | Shared-state safety |
|---|---|---|
| Java 6 | 1:1 OS threads (∼1 MB stack each); `synchronized`, `java.util.concurrent`. (Java 21 later added virtual threads.) | Java Memory Model defines races; nothing prevents them. |
| C++ | 1:1 `std::thread`; C++20 stackless coroutines without a standard scheduler. | Data races are undefined behaviour. |
| Rust | 1:1 threads plus stackless `async` futures run by a library executor (Tokio). | `Send`/`Sync` reject data races at compile time. |
| TypeScript | One JS thread per isolate; event loop; libuv thread pool for file and DNS work. | No shared memory between JS callbacks; Workers communicate by message. |
| Kotlin | Stackless CPS coroutines on JVM thread pools; structured concurrency. | Same JVM memory model; `Mutex`, `Channel`, confinement to one dispatcher. |
| Go | M:N stackful goroutines (2 KB start); channels and `select`; netpoller. | Races detectable at runtime with `-race`; multi-word races can corrupt memory. |
| OCaml | OCaml 5: multicore domains plus effect handlers for lightweight fibers. | Data races are bounded (no crashes), but still logic bugs. |
| Haskell | GHC M:N green threads; software transactional memory (STM). | Immutability by default; `TVar` transactions compose. |

Table 11.4: Axes 5 and 6: generics implementation is summarized in Figure 11.5; this table covers the concurrency runtime and what protects shared state.

### 11.4.5 Axis 7: Where Each Language Is Used, and Why

Languages win in production when their trade-offs match a problem's constraints. The examples below are real and well documented.

- **Java (and the JVM generally):** large enterprise systems and data infrastructure (Kafka, Hadoop, Spark, Elasticsearch). The JIT and mature GCs give good throughput on long-running servers, and the tooling for profiling and monitoring is the deepest of any platform.
- **C++:** anything where latency or memory control is the product: game engines, browsers (Chrome, Firefox's older parts), database engines, high-frequency trading. Decades of libraries, and it runs everywhere.
- **Rust:** infrastructure that must be both fast and memory-safe: AWS Firecracker (the microVM behind Lambda), Cloudflare's proxies, parts of Firefox and Android, and increasingly the Linux kernel. Discord moved a latency-sensitive Go service to Rust in 2020 because GC pauses caused regular latency spikes.
- **TypeScript:** browser front ends and Node.js back-end-for-frontend services, where sharing types between client and server removes a whole class of API mismatches.
- **Kotlin:** Android (Google's preferred language since 2019) and JVM back ends that want null safety and coroutines while keeping every Java library.

- **Go:** cloud infrastructure and network services: Docker, Kubernetes, etcd, Prometheus, Terraform, CockroachDB. Fast builds, single static binaries, and goroutines fit services that mostly wait on the network.
- **OCaml:** Jane Street runs its trading systems on it; it is also a favourite for compilers and proof tools (the first Rust compiler was written in OCaml, as are the Rocq/Coq proof assistant and Meta's Flow and Hack type checkers).
- **Haskell:** domains where correctness arguments matter more than hiring pool: Meta's Sigma anti-abuse rule engine (via the Haxl library), Standard Chartered's internal financial modeling, the Cardano blockchain, and the pandoc document converter.

### 11.4.6 Putting It Together: Choosing Languages for the Financial Backend

Part V builds a financial ledger service in Rust with Axum, SQLx, and PostgreSQL. A real company running that ledger would rarely use one language for everything. Here is how the trade-offs in this chapter map onto a typical architecture, and the reason behind each choice:

1. **Ledger core (posting, balance checks, double-entry invariants): Rust.** This is where a bug costs money. Ownership rules out data races on balances, `enum` plus exhaustive `match` rules out unhandled transaction states, newtypes like `UsdCents(i64)` rule out unit mix-ups at zero cost, and there are no GC pauses in the hot path. Chapters 12–14 build this.
2. **Internal platform services (rate limiting, webhook fan-out, job schedulers, Kubernetes operators): Go.** These services mostly wait on the network, need to be written and changed quickly by many teams, and benefit from fast builds and simple deployment. Use `context` everywhere, run tests with `-race`, and keep money arithmetic out of them.
3. **Web dashboard and API gateway: TypeScript.** Share request and response types between browser and Node, but validate every untrusted payload at the boundary with a runtime schema (Chapter 9), and never move money amounts through JavaScript `number`.
4. **Mobile app: Kotlin** on Android. Sealed classes model screen states; coroutines and `Flow` handle network calls without blocking the UI thread.
5. **Batch analytics and reconciliation: the JVM** (Java or Kotlin with Spark), where throughput on large datasets matters more than latency.

The boundaries between these components are network APIs, so the type system of one language does not protect another. That is why every boundary needs an explicit schema (OpenAPI, Protocol Buffers, or a shared JSON schema) and why amounts should cross every boundary as integer minor units or decimal strings, never as floating-point numbers.

## 11.5 Exercises

**Exercise 11.1** (Escape Analysis by Hand)**.** For each function below, predict whether `p` stays on the stack or moves to the heap, then check with `go build -gcflags=-m`: (a) a function that creates a `Posting` and returns `p.Cents`; (b) one that returns `&p`; (c) one that appends `p` (by value) to a slice parameter; (d) one that stores `&p` in a package-level variable; (e) one that passes `p` to `fmt.Println`. Explain case (e) using what you know about converting a value to an interface.

**Exercise 11.2** (Slice Aliasing in a Batch Splitter)**.** A function `split(batch []Posting) (debits, credits []Posting)` builds both results by appending to `batch[:0]`. Show with a concrete 4-element input why the returned slices can overwrite each other and the input. Rewrite it so the outputs never alias the input, and state the extra allocation cost in bytes for a batch of $n$ postings.

**Exercise 11.3** (The Typed-Nil Trap and Its Detector)**.** Write a function `lookup(id int64) error` that returns a nil `*NotFoundError` variable on success. (a) Draw the two words of the returned interface. (b) Show that `errors.Is(err, nil)` and `err == nil` both report false. (c) Run `staticcheck` on your code and record the diagnostic it gives (check SA4023). (d) Explain why the equivalent Rust function returning `Result<(), NotFoundError>` cannot have this bug.

**Exercise 11.4** (Deadlock, Leak, and Race)**.** Write three tiny Go programs that produce, respectively: (a) `fatal error: all goroutines are asleep - deadlock!` using an unbuffered channel; (b) a goroutine leak where a sender blocks forever after its receiver returned early (detect it by printing `runtime.NumGoroutine()` in a loop); (c) `fatal error: concurrent map writes`. For each, write the fix, and for (c) also the output of `go run -race` before the fix.

**Exercise 11.5** (Cancellation Semantics Compared)**.** In the "one task, five languages" listings, suppose `fetchRisk` fails after 5 ms while `fetchFx` would take 300 ms. For each of the five languages, state (a) when the caller sees the error, (b) whether the FX request keeps running, and (c) what code change, if any, is needed to stop it. Then explain in two sentences why Rust and Kotlin get this right without extra code, and what each relies on (dropping futures, and the `Job` tree).

**Exercise 11.6** (Choosing a Language for a New Service)**.** Your team must build a service that receives card-network authorization messages (about 20,000 per second at peak, p99 latency budget 15 ms), checks a balance held in memory, and writes an append-only log. Using the seven axes of this chapter, write a one-page design note comparing Rust, Go, and Kotlin for this service. Address GC pauses against the latency budget, how each language would represent the authorization outcome, how each protects the in-memory balance from data races, and how a newly hired Java 6 developer would ramp up on each.

# Part V

# Capstone: A Financial Backend in Rust with Axum, SQLx, and PostgreSQL

# Chapter 12

# Async Rust From Silicon to Tokio: State Machines, Pinning, and Cancellation

Throughout the first four parts of this book, almost every function we traced ran from top to bottom on a contiguous call stack. When a function called a helper, the CPU decremented the stack pointer, wrote a new frame, ran the helper to completion, popped the frame, and carried on. Even when we stepped outside synchronous code in Part IV — examining Kotlin's continuation-passing coroutines and Go's dynamically resized goroutine stacks — we looked at concurrency models from the outside in, comparing trade-offs across languages.

Now we open Part V, where every concept we have built — stack versus heap layout, destructive moves, lifetimes, algebraic data types, traits, monomorphization, and functional error pipelines — converges on a single engineering goal: building a production-grade financial ledger and payment API in Rust using Axum, SQLx, and PostgreSQL.

Why begin a financial backend capstone with the low-level mechanics of asynchronous I/O rather than jumping straight to HTTP routes and SQL tables? Because a payment service spends well over 99% of its wall-clock execution time waiting on network wires. Consider what physically happens during one transfer request:

1. Bytes trickle across a client's TLS connection into a kernel socket buffer as mobile packets arrive.

2. Your handler parses the JSON request and sends a parameterized query over a TCP socket to PostgreSQL.

3. Your process waits hundreds of microseconds to several milliseconds while PostgreSQL acquires row locks, appends redo records to its Write-Ahead Log, and flushes nonvolatile storage via `fsync`.

4. Your handler serializes the receipt and waits for the outbound socket buffer to drain back to the caller.

At 3 GHz, one millisecond of network and disk latency is three million clock cycles. How your programming language represents a paused request during those three million cycles determines

two critical properties of a financial system. First, it governs capacity and tail latency: whether a single server handles 800 concurrent requests before thrashing memory or 80,000 requests with sub-millisecond scheduling overhead. Second, and far more subtly, it governs *what happens when a wait is interrupted*: if a mobile client loses cellular signal halfway through a transfer, or a load-balancer timeout fires between two database calls, where does execution stop, which destructors run, and can half a financial mutation leak into permanent state?

In Rust, asynchronous functions are not scheduled by a virtual machine, nor do they allocate miniature call stacks. Instead, the compiler lowers each `async fn` into a plain, exact-sized `enum` state machine driven by a single three-line trait called `Future`. Once you see that transformation down to bytes and addresses, the signatures that confuse most newcomers — `Pin<&mut Self>`, `Unpin`, `Context<'_>`, `Waker`, and `Send + 'static` — each follow from one fact: a paused function's local variables are stored inside an ordinary Rust value that can be moved, shared, or dropped.

## 12.1 Why Thread-per-Request Hits a Hardware Wall

If you built backend services in Java 6 — with `ServerSocket`, classic Servlet containers such as Tomcat 6, and JDBC `Connection` pools — you grew up with the *thread-per-request* execution model. An incoming HTTP connection is handed to a worker thread from an `ExecutorService` (`java.util.concurrent`). That OS thread owns the request from start to finish. Whenever the handler reads from the client socket or executes a SQL query over JDBC, it makes a synchronous, *blocking* system call into the operating system kernel.

Let us write down the classic Java 6 shape so we have a concrete baseline to measure against.

Listing 12.1: A classic Java 6 thread-per-request transfer server using blocking socket and JDBC calls.

```
public final class BlockingTransferServer {
    private final ExecutorService workers =
        Executors.newFixedThreadPool(512);
    private final DataSource dataSource;

    public BlockingTransferServer(DataSource ds) {
        this.dataSource = ds;
    }

    public void serve(ServerSocket serverSocket) throws IOException {
        while (!Thread.currentThread().isInterrupted()) {
            final Socket client = serverSocket.accept(); // blocks accept thread
            workers.submit(new Runnable() {
                public void run() {
                    handleClient(client);
                }
            });
        }
    }

    private void handleClient(Socket client) {
        try {
            InputStream in = client.getInputStream();
```

```
            OutputStream out = client.getOutputStream();
            TransferCmd cmd = WireCodec.readCommand(in); // OS thread blocks here

            Connection conn = dataSource.getConnection(); // may block on pool
            try {
                conn.setAutoCommit(false);
                LedgerDao.postTransfer(conn, cmd);        // OS thread blocks on TCP
                conn.commit();                            // OS thread blocks on WAL
            } catch (SQLException e) {
                conn.rollback();
                throw e;
            } finally {
                conn.close();
            }
            WireCodec.writeReceipt(out, cmd.getId());     // OS thread blocks here
        } catch ( Exception e ) {
            // log failure
        } finally {
            try { client.close(); } catch (IOException ignored) {}
        }
    }
}
```

For moderate concurrency — say fifty internal corporate users — this model is wonderfully straightforward. The sequential order of lines in `handleClient` matches the chronological order of events, local variables sit cleanly in stack frames, and `try`/`finally` scopes clean up resources.

Now put this server at the edge of a retail payment gateway handling 25,000 concurrent mobile connections over high-latency cellular networks, plus occasional downstream slowdowns when a fraud-scoring service or contested database row takes 200 ms instead of 2 ms. By Little's Law, the average number of in-flight requests $L$ in a stationary system equals the arrival rate $\lambda$ times the mean residence time $W$:

$$L = \lambda \cdot W.$$

At $\lambda = 10{,}000$ requests/s and $W = 2$ ms (0.002 s), only $L = 20$ requests are simultaneously in flight. If network jitter and a brief downstream stall push $W$ to 1.5 s, in-flight concurrency jumps immediately to $L = 15{,}000$ requests.

What happens when you ask an operating system and a Java 6 JVM to run 15,000 threads? Two hardware walls appear at once: one in memory capacity, and one in CPU cache residency.

### 12.1.1 The Memory Cost of Preallocated Call Stacks

An OS thread (a Linux `pthread`, which is a kernel task created via `clone`) can call arbitrary C functions, dynamic libraries, and recursive routines whose maximum stack depth the compiler cannot predict ahead of time. Because C and JVM native frames hold raw stack pointers into themselves, an OS thread stack *cannot be moved or reallocated* once created; doing so would invalidate every frame pointer (`rbp`) and stack-local address inside it.

Consequently, Linux and the JVM reserve a large, contiguous virtual address range up front for every thread — typically 1 MiB on 64-bit HotSpot (`-Xss1m`) and 2 MiB to 8 MiB for default glibc `pthread` stacks, bounded at the bottom by an unmapped 4 KiB guard page to trap stack overflow

with `SIGSEGV`. Even if physical RAM pages are faulted in lazily on first touch, any thread that has traversed deep servlet filters, TLS buffers, JSON parsers, and JDBC drivers dirties 64 KiB to 256 KiB of physical pages — pages the kernel can never reclaim while that thread remains alive in a pool, plus kernel-space stack memory (16 KiB pinned per task on x86-64 Linux) and `task_struct` scheduler metadata.

Multiply 15,000 threads by 1 MiB of virtual reservation and 128 KiB of resident stack + kernel pages: you burn 15 GiB of virtual address space and roughly 2 GiB of physical RAM *merely to remember where paused functions were parked* while waiting on idle sockets. Worse, if you cap `newFixedThreadPool(512)` to avoid exhausting RAM, the moment 512 slow mobile clients occupy those threads waiting to upload their request bodies, request 513 — even from a fast internal health check — sits starving in an unbounded queue.

### 12.1.2 The Hidden Tax of Kernel Context Switches

Even when RAM is plentiful, switching between thousands of runnable OS threads punishes modern superscalar silicon. When a blocking `recvfrom()` system call finds no unread bytes in the socket receive buffer, the CPU must:

1. Trap from user mode (Ring 3) into kernel mode (Ring 0) via `syscall`, flushing parts of the instruction pipeline and applying Spectre/Meltdown page-table or branch-target mitigations where configured.
2. Append the current task onto the socket's wait queue, mark the task `TASK_INTERRUPTIBLE`, and invoke the Linux CFS/EEVDF scheduler.
3. Save general-purpose and SIMD/vector registers, swap stack pointers and thread-local storage (`fs` segment base), and — if switching processes — reload `CR3` page-table roots.

The direct register-and-kernel switch takes roughly 1.5 to 4 $\mu$s. The *indirect* cost is often five to ten times larger: when Thread A yields and 400 other threads run before Thread A wakes up again, Thread A's 32 KiB L1 data cache, L1 instruction cache, L2 cache lines, and Translation Lookaside Buffer (TLB) entries have been evicted by those intervening stacks. Thread A resumes on a cold CPU and stalls for hundreds of nanoseconds per cache miss while refetching its stack frame from DRAM.

### 12.1.3 Non-Blocking Sockets and Kernel Readiness (`epoll` and `io_uring`)

How do we wait on 50,000 sockets using only 8 OS threads (one per physical CPU core)? By separating *waiting on I/O* from *holding an OS thread*.

If you mark a Linux socket file descriptor with `O_NONBLOCK` and call `read(fd, buf, len)` when no bytes have arrived yet, the kernel does *not* put your thread to sleep. It returns immediately in a fraction of a microsecond with $-1$ and sets `errno = EAGAIN` (`EWOULDBLOCK` — "nothing ready right now; ask me later").

Of course, looping over 50,000 non-blocking sockets in user space calling `read()` over and over would burn 100% CPU for nothing. Instead, modern operating systems expose kernel event multiplexers: `epoll` on Linux, `kqueue` on BSD/macOS, IOCP on Windows, and more recently Linux `io_uring`. With `epoll`, a single worker thread registers tens of thousands of non-blocking file descriptors inside

an in-kernel red-black tree via `epoll_ctl`, attaching a 64-bit user-data token (`epoll_data.u64`) to each socket. When a network interface card (NIC) DMAs an incoming TCP packet into RAM and raises a hardware interrupt, the kernel TCP stack appends just the newly ready socket's token onto an internal readiness list in $O(1)$ time. One call to `epoll_wait` then hands the worker thread the exact batch of sockets that actually have bytes ready right now, in time proportional to *active events* rather than total open connections. Linux `io_uring` goes a step further, placing two lock-free ring buffers (a Submission Queue and a Completion Queue) in shared memory mapped into both user space and kernel space so batches of network and storage operations can be submitted and harvested with minimal or zero system calls.

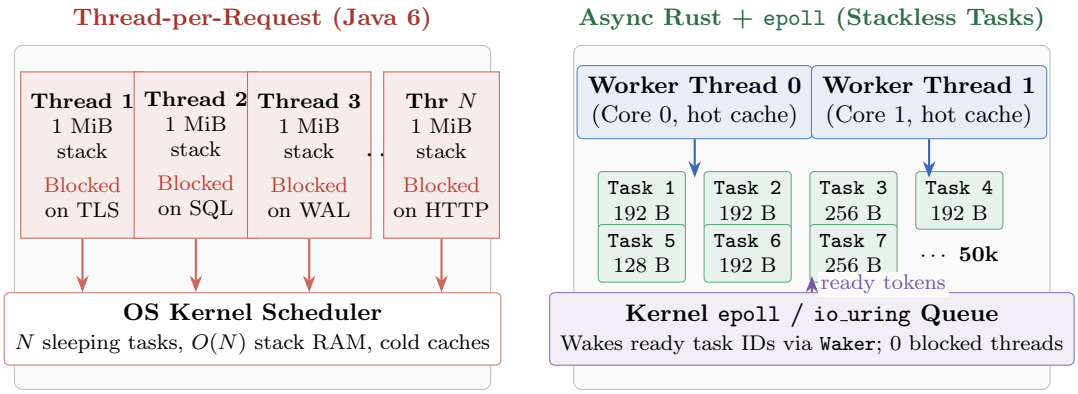



Figure 12.1: Blocking thread-per-request versus stackless event-driven execution. On the left, every waiting HTTP or SQL request pins an entire OS thread and its megabyte-scale call stack. On the right, a small pool of worker threads (matched to physical CPU cores) polls compact, exact-sized `Future` state machines whenever the kernel's `epoll` readiness queue signals that a socket can make progress.

Figure 12.1 shows the architectural shift. What the operating system gives us with `epoll` is low-cost multiplexing, *provided* our user-space code can pause in the middle of handling request $k$ when a socket returns `EAGAIN`, save just enough state to remember where it left off, go work on request $m$, and resume request $k$ later when `epoll_wait` reports that socket $k$ is ready.

In Java 6 NIO (using `Selector` from `java.nio`), doing that meant tearing your `handleClient` method apart by hand into callback objects and manual byte-buffer state fields — losing local variables, structured `try` and `catch` error handling, and readable control flow. Rust's `async` and `.await` give you back ordinary sequential code while compiling down to the exact same lean state machine you would have written by hand in C.

## 12.2 Rust's Stackless Model: The `Future` Trait

Once language designers decide not to park an OS thread on every waiting socket, they face a fundamental fork in the road: *stackful* coroutines versus *stackless* state machines.

Go (Chapter 11) and Java 21 Virtual Threads choose the *stackful* route. Each goroutine allocates a small heap stack (starting at 2 KiB in modern Go) and runs ordinary function calls on it. If the call depth exceeds 2 KiB, the runtime allocates a larger buffer, copies the stack, and adjusts internal pointers (requiring a language runtime that tightly controls pointer representations and garbage

collection). Calling out to C via FFI cannot run directly on a tiny movable 2 KiB stack, so the runtime must switch onto a full-sized OS stack across C boundaries.

Rust is a systems language with zero mandatory runtime, deterministic memory layout, and zero-overhead C FFI. Rust cannot assume a garbage collector or stack-rewriting runtime exists; an `async fn` must work just as well on a bare-metal microcontroller with 32 KiB of SRAM as on a 64-core Linux payment server. Rust therefore chooses the *stackless* route:

- A running task uses the worker thread's ordinary native CPU stack *only during the brief microseconds between two `.await` points.*
- The instant the task hits `.await` and cannot proceed yet, it unwinds cleanly back to the executor loop, leaving the native CPU stack completely empty for the next task.
- Only the specific local variables that must survive across that `.await` point are stored inside a compiler-generated `struct`/`enum` value representing the suspended task.

**Intuition**

**Lazy pull versus eager push.** If you have written JavaScript `Promise`s, C# `Task`s, or Java 8 `CompletableFuture`s, watch out for a major behavioral difference. In JavaScript, calling `const p = sendPayment(req);` immediately starts executing `sendPayment` up to its first `await`, allocates a heap promise object, and registers callback closures to be *pushed* results later.
In Rust, calling `let fut = send_payment(req);` executes **zero lines** of `send_payment`'s body and performs **zero I/O** and **zero heap allocations**. Just as calling `v.iter().map(f)` in Chapter 7 merely constructs a cold iterator struct waiting for `.next()` to pull values out of it, calling an `async fn` in Rust merely constructs a cold state-machine struct implementing `std::future::Future`. Nothing happens until an executor (or an enclosing `.await`) actively *polls* it.

### 12.2.1 Dissecting `std::future::Future`

The entire contract between the Rust compiler and every async runtime in the ecosystem (Tokio, `smol`, Embassy on embedded ARM) fits in nine lines of standard library code inside `core::future` and `core::task`:

Listing 12.2: The exact definition of Future and Poll in Rust's core library.

```
pub enum Poll<T> {
    Ready(T),
    Pending,
}

pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
```

Here is what each piece of this interface does:

1. `type Output;` — an associated type (Chapter 6), not a generic type parameter `Future<T>`, because a given concrete future state machine produces exactly one output type when it finishes (for instance, `Result<Receipt, LedgerError>`).
2. `Poll<Self::Output>` — a two-variant sum type (Chapter 5). When an executor calls `poll`, the future advances its state machine synchronously as far as it can without blocking the OS thread.
   - If the computation reaches its final `return` value, `poll` returns `Poll::Ready(val)`. The future is now exhausted and must never be polled again.
   - If the computation reaches a non-blocking socket or lock or timer that is not ready yet (e.g., `EAGAIN`), `poll` saves its current position inside `self`, arranges for `cx.waker()` to be notified when readiness changes, and returns `Poll::Pending` immediately.
3. `cx: &mut Context<'_>` — a lightweight wrapper whose sole method today is `cx.waker() -> &Waker`. This is how a leaf I/O resource tells the top-level runtime scheduler, "Task 4182 is now ready to be polled again," without the executor ever spinning in a busy loop.
4. `self: Pin<&mut Self>` — instead of taking an ordinary mutable reference `&mut self`, `poll` demands a *pinned* mutable reference. For the next five pages, treat `Pin<&mut Self>` mentally as "a mutable reference `&mut Self` accompanied by a compile-time promise that the caller will never move the bytes of `Self` to a different memory address." In Section 12.4 we will prove with raw hex addresses why omitting `Pin` would let safe Rust corrupt memory.

### 12.2.2 Writing a Leaf `Future` by Hand

Before seeing what the compiler generates for `async`/`await`, let us write a concrete `Future` implementation by hand. Suppose our financial client needs an optimistic retry delay that yields once on its first poll (registering a wakeup on a background timer thread) and resolves on its second poll. Seeing the raw trait implementation removes any sense that `Future` is special compiler machinery.

Listing 12.3: Implementing Future manually for a leaf timer state machine.

```
use std::future::Future;
use std::pin::Pin;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::task::{Context, Poll};
use std::time::Duration;

pub struct BackoffDelay {
    duration: Duration,
    started: bool,
    completed: Arc<AtomicBool>,
}

impl BackoffDelay {
    pub fn new(duration: Duration) -> Self {
        Self {
            duration,
            started: false,
            completed: Arc::new(AtomicBool::new(false)),
        }
```

```
    }
}

impl Future for BackoffDelay {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        // Fast path: has the timer already fired?
        if self.completed.load(Ordering::Acquire) {
            return Poll::Ready(());
        }

        if !self.started {
            self.started = true;
            let done_flag = Arc::clone(&self.completed);
            // Clone the Waker handle so the timer can notify the executor.
            let waker = cx.waker().clone();
            let wait_for = self.duration;

            // For illustration only: real runtimes (Tokio) register all timers
            // in one shared O(1) hashed timer wheel rather than spawning threads.
            std::thread::spawn(move || {
                std::thread::sleep(wait_for);
                done_flag.store(true, Ordering::Release);
                waker.wake(); // tells the executor: poll this task again!
            });
        }

        Poll::Pending
    }
}
```

Notice why `mut self: Pin<&mut Self>` let us write `self.started = true` directly on line 34: every field of `BackoffDelay` (`Duration`, `bool`, `Arc<AtomicBool>`) is an ordinary value with no internal self-pointers, so `BackoffDelay` automatically implements `Unpin`, allowing `Pin<&mut BackoffDelay>` to dereference mutably just like `&mut BackoffDelay`.

## 12.3 How `async`/`await` Compiles Into an Enum State Machine

Writing `Future` by hand for one leaf timer took forty lines; writing it by hand for an HTTP handler with five sequential SQL queries, error propagation with `?`, and conditional branches would be just as painful as Java 6 NIO callbacks. That mechanical translation is what `async` and `.await` automate.

Let us take a realistic ledger helper with two `.await` points and trace the exact state machine that `rustc` synthesizes for it. Suppose that before authorizing a card capture, we load the account's settled balance and then sum any active authorization holds that have not yet settled:

Listing 12.4: An async function with two suspension points (.await) and early error return (?).

```
pub async fn compute_spendable_cents(
    pool: &PgPool,
    account_id: Uuid,
```

```
) -> Result<i64, DbError> {
    let settled: i64 = fetch_settled_balance(pool, account_id).await?;
    let active_holds: i64 = fetch_active_holds(pool, account_id).await?;
    Ok(settled - active_holds)
}
```

Now track which variables are alive at each point in the body of `compute_spendable_cents`:

- **Before the body starts** (right after the caller invokes `compute_spendable_cents(pool, id)`), neither sub-query has been created yet. We only need to hold the input arguments: `pool: &'a PgPool` and `account_id: Uuid`.
- **At suspension point 1** (`fetch_settled_balance(...).await`), we are waiting on the child future `FetchSettledFut<'a>`. Which locals must survive until after `.await` #1 finishes? Both `pool` and `account_id`, because line 6 still needs them to launch `fetch_active_holds`. Notice that `settled` does not exist yet.
- **At suspension point 2** (`fetch_active_holds(...).await`), `settled: i64` has now been computed, and we are waiting on `FetchHoldsFut<'a>`. Do we still need `pool` or `account_id` after line 6? *No.* Neither variable is ever read again on line 7. So the compiler drops them from the second suspension state and reuses their bytes for `settled` and `FetchHoldsFut<'a>`.
- **After `Ok(settled - active_holds)` returns**, the future has completed. If buggy code ever polls it again, it transitions to an empty terminal state and panics deterministically.

Here is the concrete Rust `enum` and `Future` implementation corresponding to what `rustc` lowers `compute_spendable_cents` into:

Listing 12.5: Conceptual desugaring of `compute_spendable_cents` into an exact-sized enum state machine.

```
pub enum SpendableStateMachine<'a> {
    // State 0: created, never polled yet.
    Start {
        pool: &'a PgPool,
        account_id: Uuid,
    },
    // State 1: suspended at the first .await point.
    WaitingSettled {
        pool: &'a PgPool,
        account_id: Uuid,
        fut1: FetchSettledFut<'a>,
    },
    // State 2: suspended at the second .await point.
    // Notice pool and account_id are gone -- only settled and fut2 remain!
    WaitingHolds {
        settled: i64,
        fut2: FetchHoldsFut<'a>,
    },
    // State 3: completed (or exited early via Err).
    Done,
}

impl<'a> Future for SpendableStateMachine<'a> {
    type Output = Result<i64, DbError>;
```

```
25
26      fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
27          // Safety sketch: inside compiler-generated code, rustc projects Pin
28          // solely onto the currently active child future (fut1 or fut2) while
29          // updating plain Unpin state fields in place.
30          let this = unsafe { self.get_unchecked_mut() };
31
32          loop {
33              match this {
34                  SpendableStateMachine::Start { pool, account_id } => {
35                      let pool_ref = *pool;
36                      let id = *account_id;
37                      let fut1 = fetch_settled_balance(pool_ref, id);
38                      *this = SpendableStateMachine::WaitingSettled {
39                          pool: pool_ref,
40                          account_id: id,
41                          fut1,
42                      };
43                      // Do NOT return Pending here! Loop immediately to poll fut1.
44                  }
45                  SpendableStateMachine::WaitingSettled {
46                      pool,
47                      account_id,
48                      fut1,
49                  } => {
50                      let pinned_fut1 = unsafe { Pin::new_unchecked(fut1) };
51                      match pinned_fut1.poll(cx) {
52                          Poll::Pending => return Poll::Pending,
53                          Poll::Ready(Err(e)) => {
54                              *this = SpendableStateMachine::Done;
55                              return Poll::Ready(Err(e));
56                          }
57                          Poll::Ready(Ok(settled)) => {
58                              let fut2 = fetch_active_holds(*pool, *account_id);
59                              *this = SpendableStateMachine::WaitingHolds {
60                                  settled,
61                                  fut2,
62                              };
63                              // Loop around immediately to poll fut2!
64                          }
65                      }
66                  }
67                  SpendableStateMachine::WaitingHolds { settled, fut2 } => {
68                      let pinned_fut2 = unsafe { Pin::new_unchecked(fut2) };
69                      match pinned_fut2.poll(cx) {
70                          Poll::Pending => return Poll::Pending,
71                          Poll::Ready(Err(e)) => {
72                              *this = SpendableStateMachine::Done;
73                              return Poll::Ready(Err(e));
74                          }
75                          Poll::Ready(Ok(active_holds)) => {
76                              let diff = *settled - active_holds;
77                              *this = SpendableStateMachine::Done;
78                              return Poll::Ready(Ok(diff));
```

```
                        }
                    }
                }
                SpendableStateMachine::Done => {
                    panic!("async fn polled after completion");
                }
            }
        }
    }
}
```

The `loop` around a `match` shows three properties of async Rust:

1. **Zero intermediate heap allocations across nested `async` calls.** Notice that `fut1: FetchSettledFut<'a>` and `fut2: FetchHoldsFut<'a>` are stored *by value* directly inside the enum variants of `SpendableStateMachine`, not behind `Box<dyn Future>`. If your Axum HTTP handler awaits a domain service method, which awaits a repository method, which awaits a SQLx socket read, all four nested state machines flatten into *one single contiguous struct* whose exact byte size is known at compile time. When `tokio::spawn` allocates that top-level task on the heap, it performs **one** allocation for the entire call tree, compared to allocating a fresh `Promise`/`CompletableFuture` node on every single `.await` step in JavaScript or Java.

2. **Immediate fast-path fallthrough via `loop`.** Suppose `fetch_settled_balance` checks an in-memory cache or finds that PostgreSQL's response packet has already arrived in the kernel socket buffer before `poll` returns. Then `pinned_fut1.poll(cx)` yields `Poll::Ready` on its first try. Because the state machine sits inside a `loop`, it transitions straight into `WaitingHolds` and polls `fut2` in the exact same CPU time slice, without bouncing through the Tokio scheduler or touching an atomic queue.

3. **Union memory reuse across sequential `.await`s.** Recall from Chapter 5 that a Rust `enum`'s size is its discriminant tag plus the size of its *largest* variant (not the sum of all variants). `fut1` and `fut2` never exist simultaneously, so they share the exact same byte offsets inside `SpendableStateMachine`.

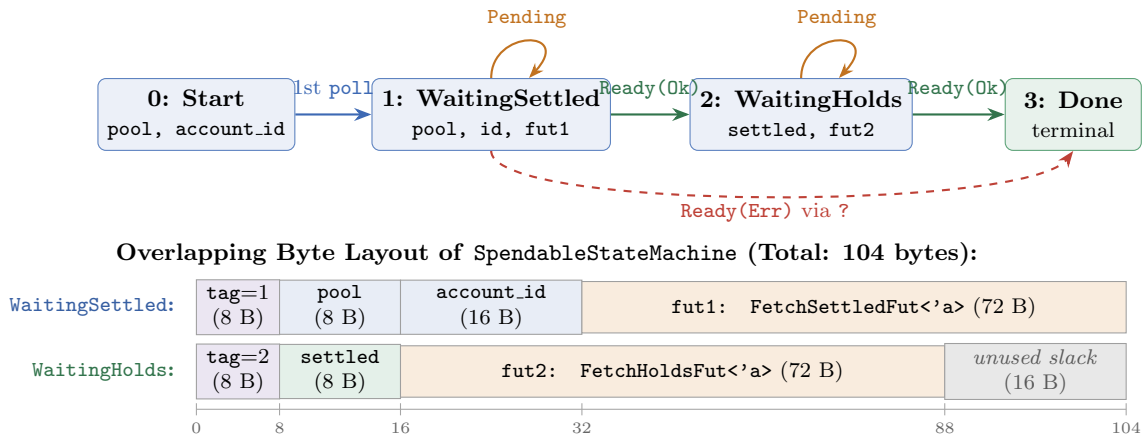



Figure 12.2: Top: state transitions of `SpendableStateMachine` across `poll` calls. Bottom: contiguous union byte layout. Because `fut1` and `fut2` are active at disjoint suspension points, they share the same memory region instead of adding their sizes together.

**Common trap**

**Large futures from stack buffers held across `.await`.** Because every variable that remains in scope across an `.await` point becomes a stored field inside the compiler-generated `enum`, declaring a large scratch array on the stack and leaving it in scope across `.await` inflates the byte size of the `Future` itself:

```
async fn bad_audit_export(pool: &PgPool) -> Result<(), DbError> {
    let mut csv_scratch = [0u8; 65_536]; // 64 KiB array!
    let n = format_header(&mut csv_scratch);
    // csv_scratch is still in scope across this .await:
    save_export_chunk(pool, &csv_scratch[..n]).await?;
    Ok(())
}
```

Here `size_of_val(&bad_audit_export(&pool))` exceeds 64 KiB. Moving that future by value or copying it into a parent state machine copies 64 KiB on the CPU stack. Fix it either by restricting the buffer to an inner synchronous block { `let mut buf = ...; ...` } that ends *before* the `.await`, or by heap-allocating large buffers via `Vec<u8>` (whose stack handle inside the future is just 24 bytes: `ptr`, `len`, `cap`).

## 12.4 Self-Referential Futures and Why `Pin` Exists

We are now ready to keep the promise we made in Section 12.2 and explain `Pin<&mut Self>` and `Unpin` from raw memory addresses. If you have ever bounced off Rust's pinning documentation, set aside all type-level jargon for three minutes and look purely at what happens to bytes in RAM when an `async fn` borrows one of its own local variables across `.await`.

In synchronous Rust (Chapters 3 and 4), every value is freely *relocatable* by `memcpy` whenever you own it and no active borrow exists. If a `String` lives at stack address `0x7ffd_1000`, its 24-byte header contains a pointer to a heap buffer at `0x55a0_8000`. Moving that `String` to `0x7ffd_2000`

copies the 24-byte header verbatim; the heap pointer inside still points to `0x55a0_8000`, which has not moved. Nothing points to `0x7ffd_1000`, so the move is completely valid. And if you had created a borrow `let r = &s;` pointing to `0x7ffd_1000`, the borrow checker flatly refuses to let you move `s` while `r` is alive.

Now look at five completely innocent lines of asynchronous Rust that read a 4-byte big-endian message length prefix from a TCP socket:

Listing 12.6: An ordinary async function that borrows a local buffer across .await.

```
async fn read_length_prefix(stream: &mut TcpStream) -> std::io::Result<u32> {
    let mut buf: [u8; 4] = [0; 4];
    let slice_ref: &mut [u8] = &mut buf; // points to local variable buf!
    stream.read_exact(slice_ref).await?; // suspends WHILE slice_ref is live!
    Ok(u32::from_be_bytes(buf))
}
```

Look at what must survive across `stream.read_exact(slice_ref).await`:

1. The 4-byte array `buf: [u8; 4]` must survive, because line 5 reads `u32::from_be_bytes(buf)` after `.await` completes.
2. The child future `ReadExact<'_, TcpStream>` must also survive across `.await` (say only 2 of the 4 bytes arrived in the first TCP segment, so `ReadExact::poll` returned `Poll::Pending` with 2 bytes left to read). Inside `ReadExact` sits the slice `slice_ref: &mut [u8]`, whose raw data pointer points directly at `buf`.

Where does `buf` live in our stackless model? *Inside the* ***`ReadLengthFuture`*** *enum struct itself.* And where does `ReadExact` (containing `slice_ref`) live? *Inside that exact same* ***`ReadLengthFuture`*** *enum struct.*

Let us trace concrete hexadecimal addresses to see what happens if we move the state machine `ReadLengthFuture` *after* its first `poll`:

- **Step 1 (Constructed, unpolled):** Calling `read_length_prefix` returns `fut` in its `Start` variant at stack address `0x7ffd_1000`. Neither `buf` nor `slice_ref` exists yet. Moving `fut` into a `Box` on the heap at address `0x55a0_0040` copies zero internal pointers, so moving it is 100% safe.
- **Step 2 (First `poll` creates a self-pointer):** The executor calls `poll` on `fut` at `0x55a0_0040`. `fut` transitions to its `WaitingRead` variant:
  - `fut.buf` sits at offset `+16`, which is machine address `0x55a0_0050`.
  - `fut.read_exact.slice_ptr` stores the address of `fut.buf`: `0x55a0_0050`.

  Suppose only 2 bytes have arrived on the wire; `poll` writes them into bytes `0..2` at `0x55a0_0050` and returns `Poll::Pending`. `fut` **is now a self-referential struct.**
- **Step 3 (What would happen without `Pin`):** Suppose `Future::poll` took an ordinary `&mut Self` instead of `Pin<&mut Self>`. In safe Rust, if you hold `f1: &mut F` and `f2: &mut F` (or own `fut` in a `Vec<F>` that reallocates), you can legally move `fut` to a brand-new address `0x55a0_9000` via `std::mem::swap` or `Vec::push`. A Rust move is always a blind bitwise `memcpy`:
  - `fut.buf` now lives at `0x55a0_9010`.

– *Nothing updated the* ***slice_ptr*** *inside* ***fut.read_exact.*** Bitwise `memcpy` copied the old pointer value `0x55a0_0050` verbatim into the new struct.

- **Step 4 (Silent memory corruption):** Meanwhile, the heap allocator reused the freed block at `0x55a0_0040` for an unrelated customer's `Transfer` struct. Two milliseconds later, the remaining 2 TCP bytes arrive and `poll` runs on the moved future at `0x55a0_9000`. `ReadExact` writes those 2 incoming network bytes directly to `slice_ptr` (`0x55a0_0050`), overwriting two bytes inside that other customer's `Transfer` while `fut.buf` at `0x55a0_9010` still holds zeroes.

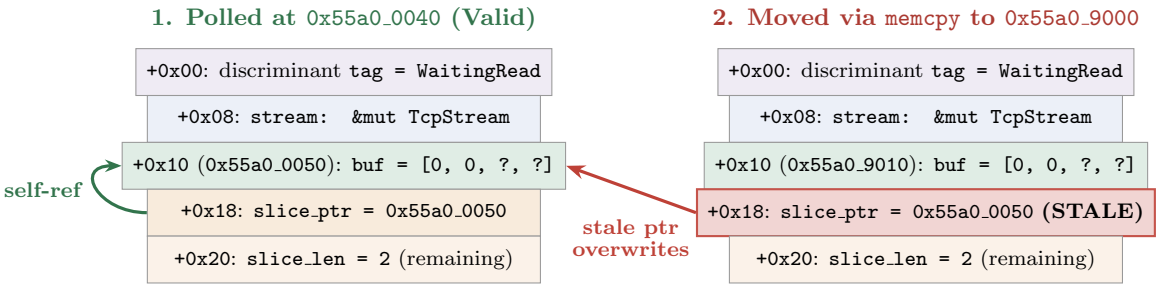



Figure 12.3: Why moving a polled self-referential future breaks memory safety. After the first `poll`, `slice_ptr` stores the physical address `0x55a0_0050` of `buf` inside the same struct. A naive `memcpy` to `0x55a0_9000` relocates `buf` to `0x55a0_9010` while leaving `slice_ptr` pointing back at `0x55a0_0050`.

Figure 12.3 is the entire reason `Pin` exists in Rust. Look at the timeline in those four steps and ask: *what is the exact minimal rule that prevents Step 3 without adding a garbage collector, pointer-patching runtime, or extra heap indirection?*

Formal

**The Pinning Guarantee.** A pointer `Pin<P>` (such as `Pin<&mut T>` or `Pin<Box<T>>`) wraps a pointer `P` whose target `*P` has type `T`. If `T` does *not* implement the marker trait `Unpin`, then once a `Pin<P>` pointing to an instance of `T` at memory address $A$ has been constructed, **the bytes of that `T` instance must remain at address $A$ — never moved, swapped, or deallocated without running `T::drop` in place at address $A$.**

Notice how surgical that rule is:

1. **Before the first `poll`, a future can move freely.** When `read_length_prefix` returns `fut`, nobody has called `poll` yet, so `fut` is still in its `Start` variant with zero internal self-pointers. You can return `fut` from helper functions, store it in structs, or pass it by value into `tokio::spawn(fut)` without pinning it first.

2. **Calling `poll` requires `Pin<&mut Self>`.** The very act of polling is what can create internal self-references, so the type signature `fn poll(self: Pin<&mut Self>, ...)` refuses to run Step 2 until you have already placed the future somewhere its address is locked down.

3. **How `Pin<&mut T>` prevents moving in safe Rust at zero runtime cost.** At the machine level, `Pin<&mut T>` is `#[repr(transparent)]` around a single 8-byte pointer `&mut T`. How does a zero-cost wrapper stop safe code from calling `std::mem::swap(a, b)`? Because `mem::swap` requires `&mut T`, and **`Pin<&mut T>` refuses to hand out `&mut T` in safe code unless `T: Unpin`.** Look at the standard library's conditional implementation:

```rust
impl<'a, T: Unpin> Pin<&'a mut T> {
    pub fn get_mut(self) -> &'a mut T { ... } // ONLY exists when T: Unpin!
}

impl<'a, T> Pin<&'a mut T> {
    // For !Unpin types (like async fn futures), getting &mut T is unsafe:
    pub unsafe fn get_unchecked_mut(self) -> &'a mut T { ... }
}
```

Without `&mut T`, safe Rust cannot call `mem::swap`, `mem::replace`, or `Option::take` on the underlying future. The self-pointer inside `fut` stays valid until `Drop::drop` runs at that exact address.

### 12.4.1 Decoding the Double Negative: What `Unpin` Means

Beginners often trip over the trait name `Unpin` because it sounds like an action. Read `T: Unpin` as an adjective: **"`T` is address-insensitive (safe to unpin and move at will)."**

Almost every ordinary Rust type — `i64`, `Uuid`, `String`, `Vec<u8>`, `TcpStream`, `PgPool`, and our hand-written `BackoffDelay` from Section 12.2 — implements the auto-trait `Unpin` automatically. Even if you wrap `&mut String` inside `Pin<&mut String>`, `Pin` relaxes completely and lets you call `Pin::new(&mut s)` and `get_mut()` in safe code, because moving a `String`'s 24-byte stack header never invalidates any internal self-pointer.

What does *not* implement `Unpin`?

- Compiler-generated `async fn` and `async { ... }` state machines (because they may hold borrows across `.await` like `read_length_prefix`).
- Any custom self-referential struct containing the zero-sized marker field `PhantomPinned` from `std::marker`.

### 12.4.2 The Two Ways You Pin a Future in Real Code

In 99% of application code, you simply write `foo().await`, and the enclosing `async fn` state machine handles pinning automatically. When you *do* manually drive a `!Unpin` future — for example, polling a stream inside a custom Tower middleware or storing a dynamic future inside a struct — you have two clean, safe idioms:

1. **Heap pinning via `Box::pin(fut)`:** Allocates `fut` on the heap once and returns a `Pin<Box<F>>`. Because the future's bytes now live at a fixed heap address (say `0x55a0_0040`), *you can freely move the 8-byte `Pin<Box<F>>` handle around by value.* In fact, `Pin<Box<F>>` itself implements `Unpin` even when `F` is `!Unpin`, because moving the pointer on the stack does not move the heap allocation it points to.
2. **Zero-allocation stack pinning via `std::pin::pin!`:** If the future does not need to outlive the current scope, calling `std::pin::pin!(fut)` shadows `fut` with a `Pin<&mut F>` tied to a hidden local variable that goes out of scope (running `Drop` in place) before its stack slot can ever be reused or moved. That gives 100% compile-time safety with zero heap allocation.

Common trap

**The error you will see.** The safe constructor `Pin::new` only accepts `Unpin` types, so trying it on an `async fn` future fails:

```
let mut fut = read_length_prefix(&mut sock);
let pinned = Pin::new(&mut fut); // try to poll it by hand
```

```
error[E0277]: `impl Future<Output = Result<u32, std::io::Error>>`
              cannot be unpinned
  = note: consider using the `pin!` macro
          consider using `Box::pin` if you need to access the pinned
          value outside of the current scope
```

The compiler is pointing you to exactly the two idioms above. Reach for `unsafe { Pin::new_unchecked(...) }` only when you are writing a combinator and can prove the value will never move again.

## 12.5 Wakers, Readiness, and Tokio's Work-Stealing Scheduler

We now have half of the async engine: a compiler that turns sequential `async fn` code into a compact, pinned `Future` state machine. Now we need the other half: an *executor* and *I/O reactor* that multiplexes 100,000 of those state machines onto $N$ OS worker threads without ever spinning the CPU or missing a wakeup. In production Rust backends — and throughout Axum and SQLx — that runtime is **Tokio**.

### 12.5.1 The `Waker` Contract and Lost-Wakeup Freedom

Suppose Task 42 awaits `socket.read(&mut buf)`. Under the hood, Tokio's `poll_read` method on `TcpStream` attempts a non-blocking `read()` syscall and gets `EAGAIN`. Before returning `Poll::Pending`, it stores `cx.waker().clone()` inside Tokio's I/O driver slab slot associated with that socket's `epoll` token.

What physically *is* a `std::task::Waker`? Because `core::future::Future` lives in Rust's runtime-agnostic `core` library (which cannot depend on Tokio or even heap memory), `Waker` is a custom two-word fat value `(data: *const (), vtable: &'static RawWakerVTable)` containing four function pointers: `clone`, `wake`, `wake_by_ref`, and `drop`. In Tokio, `data` points directly to the reference-counted header of Task 42's heap allocation. Calling `waker.wake()` transitions Task 42's atomic lifecycle state from `IDLE` to `SCHEDULED` (via a compare-and-swap) and pushes Task 42's pointer onto a worker's run queue.

Common trap

**What if the TCP packet arrives *while* `poll` is returning `Pending`?** In multithreaded systems programming, the classic race condition is the *lost wakeup*:

1. Worker Thread 0 sees `EAGAIN` on Socket 7.
2. Microseconds before Worker 0 finishes returning `Poll::Pending`, a TCP packet arrives

on Socket 7 and the I/O thread fires `waker.wake()`.

3. Worker 0 finishes parking Task 42 as "asleep" — and Task 42 sleeps forever even though bytes are sitting in the socket.

Tokio eliminates this race with a small atomic state machine inside every task header (`RUNNING`, `IDLE`, `NOTIFIED`). If `waker.wake()` fires *while* Worker 0 is still inside `poll()` (`RUNNING`), the atomic transition flips the header to `RUNNING | NOTIFIED`. When Worker 0 finishes `poll()` and sees `Poll::Pending`, its compare-and-swap from `RUNNING` to `IDLE` *fails* because the `NOTIFIED` bit is set — so Worker 0 immediately re-schedules Task 42 instead of putting it to sleep.

### 12.5.2 Inside Tokio's Multi-Threaded Work-Stealing Runtime

When you annotate `#[tokio::main]` above `async fn main()`, the macro expands into building a `tokio::runtime::Runtime` configured by default with one worker OS thread per logical CPU core. Let us trace how Tokio schedules tasks across those cores with minimal lock contention (Figure 12.4):

- **Per-worker local queue + LIFO slot:** Every worker thread owns a fixed-capacity, single-producer multi-consumer ring buffer of 256 task pointers, plus a single *LIFO slot* holding the most recently woken task. Why a LIFO slot? Suppose an Axum handler sends a message over an in-process channel or wakes a parent task: running the newly woken task *immediately next* on the exact same CPU core means the message payload is still sitting hot in that core's 32 KiB L1 data cache. To prevent two ping-ponging tasks from starving older tasks in the ring buffer, Tokio limits consecutive LIFO-slot runs per tick (the cooperative budget, 128 polls by default) before draining the FIFO ring buffer.

- **Global injection queue:** Tasks spawned from outside worker threads (or overflow when a worker's 256-slot local queue fills up) land in a shared MPSC `Inject` queue checked periodically (every 61 ticks by default, a prime interval chosen to avoid resonant starvation cycles).

- **Work stealing:** When Worker 1 exhausts its LIFO slot and its local 256-slot queue, it checks the global queue; if empty, it picks a random sibling (say Worker 0) and atomically steals *half* of Worker 0's queued tasks in a single batch operation. Stealing half amortizes cross-core atomic cache-line traffic while keeping all CPU cores utilized when a few requests take longer than others.

- **Shared I/O reactor and timer wheel:** When a worker finds no runnable tasks anywhere, one idle worker becomes the designated driver thread and parks inside `epoll_wait` (with timeout set to the next expiring deadline in Tokio's $O(1)$ hierarchical hashed timer wheel), waking siblings only when fresh kernel events arrive.

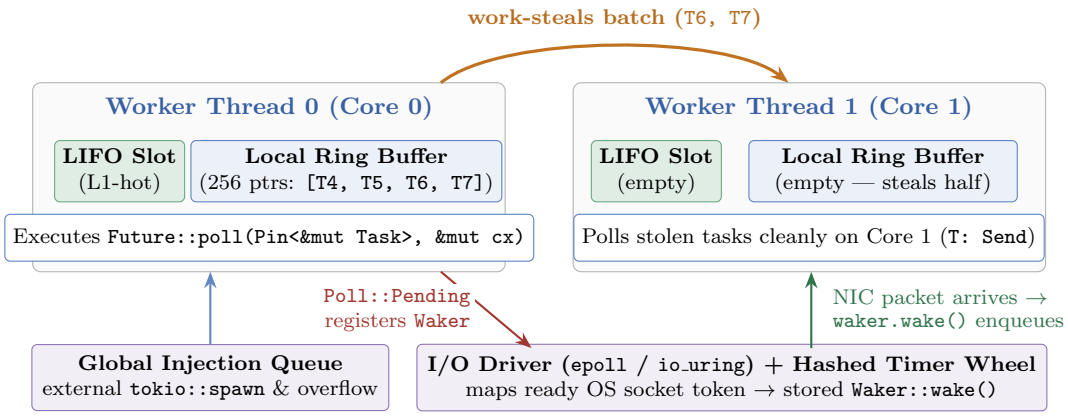



Figure 12.4: Architecture of Tokio's multi-threaded scheduler. Each OS worker thread runs tasks from its cache-hot LIFO slot and 256-slot local ring buffer. Idle siblings steal half a busy worker's queue, which is why spawned futures must implement `Send`. Tasks returning `Poll::Pending` sleep off-CPU until the `epoll` driver or timer wheel invokes their `Waker`.

### 12.5.3 Why `tokio::spawn` Demands `Future + Send + 'static`

Look at the signature of `tokio::spawn`:

```
pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
where
    F: Future + Send + 'static,
    F::Output: Send + 'static,
```

Every single bound in that `where` clause comes straight out of Figure 12.4 and Section 12.3:

1. **Why `F: 'static`?** Recall Chapter 4's golden rule: `T: 'static` does *not* mean `T` lives until the process exits; it means `T` contains no non-`'static` borrowed references `&'a X` pointing into a caller's temporary stack frame. Because a spawned task runs independently in the background, the function calling `tokio::spawn` may return immediately and pop its stack frame while the spawned task is still running. Therefore, a spawned future must own its data (`Uuid`, `String`, `PgPool` — which internally wraps an `Arc`) or share read-only state via `Arc<AppState>`.

2. **Why `F: Send`?** Look at the orange work-stealing arrow at the top of Figure 12.4. Suppose Task T6 starts running on Worker Thread 0 (Core 0), hits `.await`, and suspends inside its `WaitingHolds` enum variant. When PostgreSQL replies 2 ms later, Worker 0 happens to be busy, so Worker Thread 1 (Core 1) steals Task T6 and calls `poll()` on Core 1. Task T6 just migrated between two OS threads midway through its function body. For that migration to be free of data races and undefined behavior, **every local variable stored inside any `.await` state variant of `F` must itself be `Send`**.

### 12.5.4 How the Compiler Saves You From a `MutexGuard` Worker Deadlock

Watch how `F: Send` plus enum state-machine lowering catches a production deadlock at compile time. Suppose a developer coming from Java 6 `synchronized` blocks stores an in-memory exchange-rate cache inside a standard `std::sync::Mutex<RateTable>` and writes an Axum route handler like

this:

Listing 12.7: Holding a synchronous `std::sync::MutexGuard` across `.await` fails to compile.

```
use std::sync::{Arc, Mutex};

// REJECTED BY RUSTC when passed to tokio::spawn or an Axum router:
async fn quote_and_record_broken(
    rates: Arc<Mutex<RateTable>>,
    pool: PgPool,
    amount_eur: i64,
) -> Result<i64, AppError> {
    let guard = rates.lock().unwrap(); // std::sync::MutexGuard<'_, RateTable>
    let rate = guard.eur_to_usd_bps();
    let usd_cents = amount_eur * rate / 10_000;

    // BUG: `guard` is STILL IN SCOPE across this .await!
    record_quote(&pool, usd_cents).await?;

    drop(guard); // guard used after .await, so it lives inside the enum variant!
    Ok(usd_cents)
}
```

When you wire `quote_and_record_broken` into Axum (which requires handlers to produce `Send` futures so Tokio can schedule them on its multi-threaded pool), `rustc` rejects the program with a clear diagnostic:

```
error: future cannot be sent between threads safely
note: future is not `Send` as this value is used across an await
  --> src/handlers.rs:14:35
   | `guard` has type `MutexGuard<'_, RateTable>`, which is not `Send`
```

Why is `std::sync::MutexGuard` marked `!Send`? First, on POSIX systems (`pthread_mutex_t`), locking a mutex on OS Thread 0 and unlocking it after work-stealing onto OS Thread 1 is undefined behavior. Second — and even more dangerous — think about what would happen on Tokio *even without* POSIX UB if you held a synchronous OS mutex across `.await`:

1. Task A locks `rates` on Worker 0, hits `.await` on `record_quote`, and yields `Poll::Pending` *while still holding the locked OS mutex.*
2. Worker 0 is now free to run the next task in its queue: Task B.
3. Task B calls `rates.lock()`. Because `std::sync::Mutex::lock` blocks the **entire OS thread** waiting for Task A to unlock `rates` — and Task A is sitting suspended in memory waiting for a worker thread to poll it after `record_quote` finishes — Worker 0 is now blocked inside `lock()` and polls nothing else. If all $N$ worker threads hit the same lock, your entire payment gateway deadlocks permanently with 0% CPU usage.

The fix is simple, explicit, and makes your critical section ten thousand times shorter: scope the synchronous `MutexGuard` inside an inner block so it drops *before* the `.await` point.

Listing 12.8: Dropping the synchronous MutexGuard before .await makes the Future Send and lock-contention-free.

```rust
async fn quote_and_record_clean(
    rates: Arc<Mutex<RateTable>>,
    pool: PgPool,
    amount_eur: i64,
) -> Result<i64, AppError> {
    let usd_cents = {
        let guard = rates.lock().unwrap();
        let rate = guard.eur_to_usd_bps();
        amount_eur * rate / 10_000
    }; // <-- guard drops RIGHT HERE! Critical section lasts ~15 nanoseconds.

    // Zero locks held across network I/O; enum variant stores only `usd_cents: i64`.
    record_quote(&pool, usd_cents).await?;
    Ok(usd_cents)
}
```

What if you genuinely *must* hold an exclusive lock across asynchronous I/O (for example, serializing sequence numbers on a single multiplexed upstream banking TCP connection)? Then use `tokio::sync::Mutex`, whose `lock().await` yields `Poll::Pending` cleanly via a `Waker` queue instead of blocking the OS worker thread, and whose guard *is* `Send`. For pure in-memory data structures (`HashMap`, rate caches, metrics), always prefer a short synchronous `std::sync::Mutex` (or `parking_lot::Mutex`) dropped before `.await`: locking for 15 ns in memory is dramatically faster than holding an async mutex across a 2 ms database round trip.

### 12.5.5 Cooperative Scheduling and `tokio::task::spawn_blocking`

Because stackless futures yield control *only* at `.await` points, Tokio uses **cooperative multitasking** between suspension points. That creates an operational rule every backend engineer must memorize:

Focus

**Never block or burn ¿100 microseconds of uninterrupted CPU on a Tokio async worker thread.** Calling synchronous `std::fs::read`, legacy blocking C FFI, or CPU-heavy cryptography such as Argon2id password verification (80 ms of tight memory-hard hashing with zero `.await`s) directly inside an `async fn` hijacks that core's worker thread for 80 ms. Every task sitting in that worker's LIFO slot — which sibling workers *cannot* steal — freezes for 80 ms, causing immediate p99 tail-latency spikes across unrelated payment endpoints. Offload CPU-heavy or blocking OS calls onto Tokio's dedicated elastic blocking pool via `tokio::task::spawn_blocking`:

```rust
let password_ok: bool = tokio::task::spawn_blocking(move || {
    verify_argon2id_hash(&candidate_pw, &stored_phc_hash)
})
.await
.map_err(|join_err| AppError::Internal(join_err.to_string()))??;
```

The async worker thread registers its `Waker`, returns `Poll::Pending` in under a microsecond to keep serving HTTP sockets, and resumes cleanly once the blocking pool thread finishes hashing.

## 12.6 Backpressure, Bounded Channels, and Cancellation Safety

Making individual tasks cheap is a double-edged sword. In Java 6, spawning 200,000 threads crashes immediately, forcing engineers to bound thread pools early. In async Rust, spawning 200,000 compact tasks or pushing 5,000,000 audit events into an *unbounded* `mpsc` channel (`unbounded_channel` in `tokio::sync::mpsc`) succeeds right up until a database failover slows consumers while producers keep accepting traffic at line rate, exhausting 32 GiB of RAM and triggering the Linux OOM killer.

### 12.6.1 Explicit Backpressure with Bounded `mpsc` and `Semaphore`

Production financial services enforce **bounded queues everywhere**:

- **Bounded `mpsc::channel(capacity)`:** When all `capacity` slots are full, `tx.send(msg).await` suspends the producer (`Poll::Pending`) until the consumer drains a slot, or `tx.try_send(msg)` fails immediately in $O(1)$ time with `TrySendError::Full` so an HTTP ingress layer can shed excess non-critical load cleanly with `429 Too Many Requests` or `503 Service Unavailable`.
- **Concurrency caps via `Semaphore`:** In `tokio::sync`, a `Semaphore` caps concurrent in-flight calls against a scarce downstream resource (for instance, allowing at most 64 concurrent calls to an external card-network gateway even when 10,000 inbound client connections are open). Calling `sem.acquire().await` hands back a `SemaphorePermit` guard whose `Drop` destructor automatically releases the permit back to the next waiting `Waker` — even on early `?` return or panic.

### 12.6.2 Async Cancellation in Rust: Dropping a `Future` Mid-Flight

This section covers the difference between Rust's async model and Java, Go, or Kotlin that most affects how we structure financial mutations in Chapters 13 and 14.

Ask yourself: *How do you cancel an asynchronous operation in Rust?*

In Java, you call `future.cancel(true)`, which sets an interrupt flag; the worker thread keeps running until it hits an interruptible call or checks `Thread.interrupted()`. In Go, cancelling a `context.Context` closes its `Done()` channel; the goroutine keeps running in the background until its own code checks `ctx.Err()` and returns.

In Rust, a running async operation *is just a passive `Future` state-machine struct owned by its caller or executor.* **To cancel a future in Rust, the owner simply drops it (`drop(fut)`) and never calls `poll()` again.**

Think carefully about what happens at the machine level when Hyper/Axum detects that a mobile client's TCP socket closed (or a three-second `tokio::time::timeout` wrapper around your handler expires) while your `async fn` handler is parked at `.await` #1:

1. The timeout or HTTP connection loop drops `handler_fut` right where it sits, inside its `WaitingStep1` enum variant.
2. **Zero lines of Rust code after `.await` #1 will ever execute.** No `InterruptedException` is thrown into the function body for you to catch; the paused state machine simply ceases to be polled.

3. **Every value currently alive inside the active `WaitingStep1` variant has its deterministic `Drop::drop` destructor run immediately in reverse declaration order** — just like C++ RAII scope exit.

Look at what happens if an engineer writes a naive, non-transactional two-step transfer that commits two separate SQL statements on a connection pool:

Listing 12.9: BROKEN: a non-transactional two-query transfer that cancellation can split in half.

```
// BROKEN: Never split a financial invariant across uncommitted/separate pool calls!
pub async fn broken_transfer(
    pool: &PgPool,
    from: Uuid,
    to: Uuid,
    cents: i64,
) -> Result<(), DbError> {
    // Step 1 checks out a connection, debits `from`, and auto-commits!
    sqlx::query("UPDATE accounts SET balance = balance - $1 WHERE id = $2")
        .bind(cents)
        .bind(from)
        .execute(pool)
        .await?;

    // <--- DANGER ZONE! Suppose right as Step 1 returns or while Step 2 waits
    //      for a pool connection, the caller's 2-second HTTP timeout fires,
    //      or the client disconnects! Hyper/timeout DROPS this Future here.

    // Step 2 NEVER EXECUTES! $100 vanished from `from` and never reached `to`!
    sqlx::query("UPDATE accounts SET balance = balance + $1 WHERE id = $2")
        .bind(cents)
        .bind(to)
        .execute(pool)
        .await?;

    Ok(())
}
```

Now contrast `broken_transfer` with what happens when we bind *both* mutations inside a single RAII `sqlx::Transaction<'_, Postgres>` owned inside the handler's `Future` state machine:

Listing 12.10: Cancellation-safe transfer via an RAII database transaction owned by the Future.

```
pub async fn atomic_transfer(
    pool: &PgPool,
    from: Uuid,
    to: Uuid,
    cents: i64,
) -> Result<(), DbError> {
    // Opens `BEGIN` on one dedicated connection; `tx` lives inside our Future!
    let mut tx = pool.begin().await?;

    debit_within_tx(&mut tx, from, cents).await?;
    // <--- If dropped HERE (timeout, `?` error, client disconnect), `tx` is
    //      dropped WITHOUT `tx.commit()` having run! `Transaction::drop`
    //      automatically schedules an asynchronous `ROLLBACK` on the connection
```

```
    //      before returning it to `pool`. Zero money vanishes!

    credit_within_tx(&mut tx, to, cents).await?;

    // Explicit `COMMIT`: consumes `tx` by value, writing the single atomic
    // commit record to PostgreSQL's WAL.
    tx.commit().await?;
    Ok(())
}
```

Here is how ownership, destructive moves, RAII `Drop`, and stackless futures combine in the function above:

- `tx` is stored as a field inside `atomic_transfer`'s compiler-generated `enum` variants across both `.await` points.
- If *anything* aborts the operation early — an early `?` error on `credit_within_tx`, a worker panic, a `tokio::time::timeout`, or a disconnected client dropping the future — `tx.commit()` was never reached, so `Drop::drop` runs on `tx` and rolls back the PostgreSQL transaction automatically.
- Conversely, `tx.commit().await` takes `self` **by value** (a destructive move, Chapter 3), disarming the rollback destructor once `COMMIT` succeeds and preventing your code from ever accidentally running another SQL query on a committed transaction.

### 12.6.3 The `tokio::select!` Cancellation-Safety Trap

Even outside HTTP disconnects, your own code drops futures routinely whenever you multiplex events with `tokio::select!`. The `select!` macro polls two or more futures concurrently inside the same task; the instant *one* branch returns `Poll::Ready`, `select!` executes that branch's handler block and **immediately drops all the other branches' futures**.

If `select!` sits inside a `loop` — say an event worker listening simultaneously to a bounded `Receiver` and a shutdown signal — each iteration builds fresh futures for the branches that lost the previous race:

Listing 12.11: Cancellation-safe channel receive inside a `tokio::select!` loop.

```
// SAFE inside a select! loop: `rx.recv()` either dequeues a whole message
// atomically or leaves the message untouched in the channel queue when dropped!
loop {
    tokio::select! {
        maybe_job = job_rx.recv() => {
            match maybe_job {
                Some(job) => process_job(&pool, job).await?,
                None => break, // all senders dropped; exit cleanly
            }
        }
        _ = shutdown_rx.changed() => {
            tracing::info!("shutdown signal received; stopping worker loop");
            break;
        }
    }
```

```
}
```

Why was `job_rx.recv()` safe to drop when the `shutdown_rx` branch won the race? Because `Receiver::recv` in `tokio::sync::mpsc` holds zero half-consumed state inside its temporary `Recv` future: if dropped while `Pending`, no item was popped from the channel.

By contrast, `read_exact(&mut buf)` on `AsyncReadExt` is **not cancellation-safe** inside a `select!` loop. As we saw in Figure 12.3, if 2 bytes of a 4-byte frame arrive and then another `select!` branch wins and drops the `ReadExact` future, those 2 bytes have already been drained from the kernel TCP socket into the dropped future's local state and are lost forever when the next loop iteration starts a fresh `read_exact`. Whenever you multiplex byte streams inside `select!`, accumulate partial frames in a persistent buffer (such as `BytesMut` paired with `Framed` in `tokio_util::codec`) owned *outside* the `select!` macro so partial progress lives across iterations.

| Operation / Future | Cancel-Safe? | State When `Pending` | Safe Pattern if Unsafe |
|---|---|---|---|
| `mpsc::Receiver::recv` | **Yes** | Pops item only on `Poll::Ready` | — |
| `tokio::time::sleep` | **Yes** (pinned) | Stateless timer deadline | Pin outside loop (`pin!`) |
| `AsyncReadExt::read_buf` | **Yes** | Appends to caller's `BytesMut` | — |
| `AsyncReadExt::read_exact` | **No** | Holds partial prefix bytes | `Framed` or `read_buf` |
| `mpsc::Sender::send` | **Yes** | Message untouched until ready | Or `reserve().await` |
| Two non-tx SQL queries | **No** | Query 1 already committed | Wrap in `sqlx::Transaction` |

Table 12.1: Cancellation-safety reference for common Tokio and SQLx futures when dropped mid-`.await`.

## 12.7 Exercises

**Exercise 12.1** (Sizing an Async State Machine by Hand)**.** Consider the following 64-bit Rust `async fn`, assuming `&PgPool` is 8 bytes, `Uuid` is 16 bytes, `i64` is 8 bytes, `FetchFeeFut` is 48 bytes (8-byte aligned), and `PostLedgerFut` is 80 bytes (8-byte aligned):

```
async fn apply_fee(pool: &PgPool, acct: Uuid, gross: i64) -> Result<i64, DbError> {
    let mut header_scratch = [0u8; 4096];
    let code = parse_region_tag(&mut header_scratch, acct);
    let fee = fetch_fee_schedule(pool, code).await?;
    let net = gross - fee;
    post_ledger_fee(pool, acct, fee).await?;
    Ok(net)
}
```

1. List the exact fields stored in each variant (`Start`, `WaitingFee`, `WaitingPost`, and `Done`) if `header_scratch` stays in scope until the end of `apply_fee`. What is the total byte size of the resulting future?
2. Rewrite lines 2–3 using a lexical block scope so `header_scratch` is provably dead before the first `.await`. Recalculate the exact fields of each variant and the new total byte size of the future (assuming an 8-byte aligned discriminant tag).

**Exercise 12.2** (Tracing Self-Referential Addresses Without `Pin`)**.** Write a struct `SelfRefPair` {

`val: u64, ptr: *const u64 }` with `new(v: u64) -> Self` (setting `ptr` to null) and `init(&mut self)` (setting `self.ptr = &self.val as *const u64`).

1. Trace step by step with concrete stack addresses (e.g., `a` at `0x1000` with `val = 10`, `b` at `0x1010` with `val = 99`) what `unsafe { *a.ptr }` evaluates to after calling `a.init()`, `b.init()`, and `std::mem::swap(&mut a, &mut b)`.

2. Add a `PhantomPinned` marker field to `SelfRefPair`, change `init` to take `self: Pin<&mut Self>`, and show why stack-pinning `a` via `std::pin::pin!` allows `a.as_mut().init()` while preventing `std::mem::swap` from swapping the underlying `SelfRefPair` values.

**Exercise 12.3** (Hand-Building a `YieldOnce` Cooperative Future)**.** Sometimes a CPU-bound loop voluntarily yields back to the Tokio scheduler once every 10,000 iterations (the behavior of `yield_now` in `tokio::task`). Define a struct `YieldOnce { yielded: bool }` and implement `Future` for `YieldOnce` from scratch using only `Future`, `Pin`, `Context`, and `Poll`:

1. On the first `poll`, what method must you call on `cx.waker()` *before* returning `Poll::Pending`, and what happens at runtime if you forget that single line?

2. Why does `self.yielded = true` compile inside `poll` (with `mut self: Pin<&mut Self>`) without a single line of `unsafe`?

**Exercise 12.4** (Eliminating a `MutexGuard` Deadlock Across `.await`)**.** A teammate ports a Java 6 rate limiter to Rust using a `HashMap` wrapped in `Arc<std::sync::Mutex<...>>`:

```
async fn charge_merchant(
    buckets: Arc<std::sync::Mutex<HashMap<MerchantId, TokenBucket>>>,
    http: reqwest::Client,
    merchant: MerchantId,
) -> Result<Receipt, ApiError> {
    let mut map = buckets.lock().unwrap();
    let bucket = map.entry(merchant).or_default();
    if !bucket.try_take_one() {
        return Err(ApiError::RateLimited);
    }
    let resp = http.post("https://bank.example/settle").send().await?;
    if !resp.status().is_success() {
        bucket.refund_one(); // refunds token if upstream bank fails!
    }
    Ok(resp.json().await?)
}
```

1. Explain both (a) why `tokio::spawn(charge_merchant(...))` fails to compile, and (b) why switching blindly to `tokio::sync::Mutex` on the entire `HashMap` would compile cleanly yet collapse throughput to one single global HTTP call at a time across all merchants.

2. Refactor `charge_merchant` using two short synchronous `std::sync::Mutex` lock scopes (one to reserve a token before `http.post`, and one to refund the token on error after `http.post`), and analyze whether any concurrency anomaly can occur during the unlocked window.

**Exercise 12.5** (Auditing `tokio::select!` for Partial-State Loss)**.** Suppose an ingestion daemon runs a `loop` with `tokio::select!` over two branches:

- Branch 1 awaits a helper `read_and_debit` that first calls `rx.recv().await` on a channel receiver and then awaits `execute_sql(pool, cmd)`.
- Branch 2 ticks a metrics timer every second via `interval.tick()` and records counters via `publish_metrics()`.

Trace what happens when a `Cmd` arrives at $t = 999$ ms, `rx.recv().await` resolves immediately inside `read_and_debit`, `execute_sql` suspends waiting for PostgreSQL (3 ms), and 1 ms later at $t = 1000$ ms Branch 2's timer tick fires. How do you restructure the `select!` loop so `rx.recv()` is the select branch and `execute_sql` is never aborted by a metrics tick?

**Exercise 12.6** (Little's Law, Pool Sizing, and Backpressure)**.** Your payment service runs on an 8-core machine with a PostgreSQL `PgPool` configured to `max_connections = 32`. Each transfer transaction holds a database connection for $W_{\text{db}} = 4$ ms (0.004 s) on average.

1. Using Little's Law ($L = \lambda W$), what is the maximum sustained throughput $\lambda_{\max}$ (in transfers per second) that those 32 database connections can process?
2. Suppose a flash sale sends $\lambda = 12{,}000$ transfers/s for 30 seconds. Compare what happens to memory usage and client latency under (a) unbounded task spawning waiting indefinitely on `pool.begin().await`, versus (b) wrapping ingress with a `tokio::sync::Semaphore` of 256 permits paired with a 250 ms `acquire_timeout` returning HTTP `503 Service Unavailable` with a `Retry-After` header.

# Chapter 13

# Modeling Money and Ledgers in Types: Fixed-Point Arithmetic, Double-Entry Accounting, and Typestate

In Chapter 12 we solved the *temporal* engineering problem of a financial backend: how a single Rust process multiplexes tens of thousands of waiting HTTP and PostgreSQL sockets onto a handful of CPU cores via stackless `Future` state machines, and why async cancellation demands atomic database transactions. Now we turn from how requests wait to *what those requests compute and store*.

In most web applications, minor data anomalies are tolerable irritations. If a video platform miscounts view totals by two during a cache flush, nobody summons an external auditor. Financial systems cannot tolerate that. A single cent created out of rounding noise, lost during a three-way fee split, added across mismatched currencies, or subtracted from one account without an equal counter-entry breaks the conservation laws of the ledger. Worse, silent arithmetic bugs compound: a tenth-of-a-cent rounding bias across forty million card settlements quietly drifts corporate cash accounts by tens of thousands of dollars before month-end reconciliation flags a discrepancy.

This chapter builds the pure domain core of our ledger service before we touch HTTP headers or SQL wire protocols in Chapter 14. You will see how every Rust mechanism from Parts II and III — fixed-width integers, stack-allocated value structs, `#[repr(transparent)]` newtypes, `PhantomData`, exhaustive enums, private module boundaries, and typestate — works together to enforce a single architectural principle: *make illegal financial states unrepresentable at compile time.*

---

## 13.1 Rule 1 of Money: Why IEEE-754 Floats Fail and Fixed-Point Works

Every engineer who has written a toy e-commerce checkout in C, C++, Java, or JavaScript has been tempted at least once to write `double price = 19.99;` or `balance: f64`. Rule 1 of financial engineering is absolute: **never represent monetary balances or ledger postings with binary**

**floating-point types (`f32` or `f64`).**

To understand why this rule admits zero exceptions, we should not stop at memorizing the famous snippet `0.1 + 0.2 == 0.30000000000000004`. Let us look directly at the hardware representation of an IEEE-754 binary64 (`f64` in Rust, `double` in Java 6 and C++) and see *three distinct failure modes* that strike real payment systems.

### 13.1.1 The Three Financial Failures of Binary Floating-Point

An IEEE-754 `f64` packs three fields into 64 bits: 1 sign bit $s$, an 11-bit biased binary exponent $e$, and 52 explicit significand bits (yielding 53 bits of binary precision including the implicit leading 1 bit for normal numbers). Every normal finite value represented by `f64` has the exact mathematical form

$$x = (-1)^s \cdot M \cdot 2^E, \qquad \text{where } M \in \mathbb{Z} \text{ and } 2^{52} \le M < 2^{53}.$$

Notice the denominator: every `f64` is an integer divided by a *power of two*.

Now ask a basic number-theory question: when does a reduced fraction $p/q$ terminate in base $b$? Only when every prime factor of $q$ divides $b$. In human decimal currency (base $10 = 2 \times 5$), one cent is $1/100 = 2^{-2} \cdot 5^{-2}$, and ten cents is $1/10 = 2^{-1} \cdot 5^{-1}$. Because the prime factor 5 never divides any power of 2, **one cent (0.01) and ten cents (0.10) have infinite repeating expansions in binary**:

$$\frac{1}{10} = (0.0001100110011001100110\overline{0011})_2.$$

Hardware must chop that infinite bit string at 53 significant bits. The nearest 64-bit binary float to 0.1 is actually the exact rational number

$$\frac{3{,}602{,}879{,}701{,}896{,}397}{2^{55}} = 0.1000000000000000055511151231257827021181583404541015625.$$

That single mismatch triggers three production hazards:

1. **Inexact cent representation:** Adding ten copies of `0.1_f64` does not equal `1.0_f64`; `0.1_f64 + 0.2_f64 != 0.3_f64`. Any equality check `sum_debits == sum_credits` fails unpredictably on valid transfers unless replaced by ad-hoc epsilon tolerances, and epsilon tolerances let genuine one-cent bugs slip through.

2. **Silent penny absorption at large totals:** The spacing between adjacent `f64` values (one *unit in the last place*, or ulp) doubles at every power of two. Store dollars in an `f64` and, once a clearing account passes $2^{47} = \$140{,}737{,}488{,}355{,}328$, the spacing is $2^{-5} = 0.03125$; adding 0.01 rounds straight back to the original value, so `total + 0.01 == total`. With 32-bit `f32` the same thing happens at just $2^{18} = \$262{,}144$. Individual cents added to the account *disappear without a warning or error.*

3. **Non-associativity breaks SQL and parallel aggregation:** In real arithmetic, $(a + b) + c = a + (b + c)$. In IEEE-754 floating-point, rounding after each step makes addition **non-associative**:

```
let a: f64 = 10_000_000_000.00;
let b: f64 = -10_000_000_000.00;
let c: f64 = 0.01;
assert_ne!((a + b) + c, a + (b + c)); // 0.01 vs 0.010000228881835938
```

If PostgreSQL executes a parallel query plan across four workers tomorrow and sums the exact same million ledger rows in a different order than yesterday's single-threaded scan, an `f64` column produces two different end-of-day balances from identical immutable data.

### 13.1.2 Minor-Unit Signed Integers (`i64` and `i128`) Versus Scaled Decimals

How do we make every currency amount an exact integer multiple of a base-10 unit so addition is exact, associative, and hardware-fast? Production financial systems use two complementary fixed-point representations, each suited to a specific job:

1. **Atomic minor-unit integers (`i64` or `i128`) for ledger postings and balances.** Instead of storing dollars as fractional numbers, store integer counts of the currency's smallest settlement unit (or a fixed micro-unit scale):

   - US Dollars (`USD`) and Euros (`EUR`) have exponent 2: \$19.99 is stored as `1999_i64` cents ($1999 \times 10^{-2}$).
   - Japanese Yen (`JPY`) has exponent 0: 2,500 JPY is stored as `2500_i64` yen ($2500 \times 10^{0}$).
   - Bahraini Dinars (`BHD`) and Kuwaiti Dinars (`KWD`) have exponent 3: 1.250 KWD is stored as `1250_i64` fils ($1250 \times 10^{-3}$).

   How large is a signed 64-bit integer? `i64::MAX` = 9,223,372,036,854,775,807 $\approx 9.22 \times 10^{18}$. In US cents, that is \$92,233,720,368,547,758.07 — over **ninety-two quadrillion dollars**, roughly eight hundred times total global GDP, fitting inside a single 8-byte CPU general-purpose register and mapping one-to-one onto PostgreSQL's 8-byte `BIGINT` column type. Systems that track sub-cent micropayments (such as cloud-compute billing at \$0.000001 per call or crypto-asset satoshis/wei) either scale `i64` to micro-units ($10^{-6}$) or step up to Rust's native 128-bit integer `i128` ($\approx 1.7 \times 10^{38}$, the choice made by the TigerBeetle financial database).

2. **Base-10 scaled decimals (`rust_decimal::Decimal`) for rates, prices, and percentages.** While settled balances live in integer minor units, fee schedules (2.9%), foreign-exchange rates (EUR/USD = 1.08475), and bond yields have varying decimal places. If you come from Java 6, you reached for `java.math.BigDecimal`. Recall from Chapter 1 what a `BigDecimal` costs on the JVM: a heap-allocated wrapper object pointing to a `BigInteger` object pointing to a heap-allocated `int[]` magnitude array — two pointer dereferences and garbage-collector pressure on every arithmetic operation.

   In Rust, the widely used `rust_decimal::Decimal` type is a **16-byte stack-inline `Copy` value** with zero heap pointers. It packs a 96-bit unsigned integer mantissa ($0 \le M < 2^{96} \approx 7.92 \times 10^{28}$), 1 sign bit, and a base-10 scale $s \in [0, 28]$ into four 32-bit words representing $(-1)^{\text{sign}} \cdot M \cdot 10^{-s}$. SQLx maps it to PostgreSQL `NUMERIC`: `NUMERIC(20, 4)` for amounts that need sub-cent precision, and a wider scale such as `NUMERIC(20, 8)` for exchange rates.

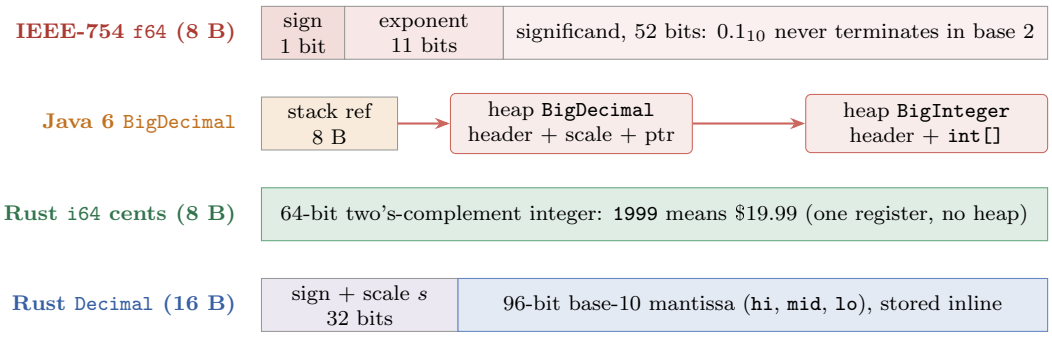



Figure 13.1: Memory layout comparison across numeric representations. Binary `f64` cannot represent 0.01 exactly; Java 6 `BigDecimal` is exact in base 10 but chases heap pointers on every operation; Rust's `i64` minor units (8 B) and `rust_decimal::Decimal` (16 B) keep exact base-10 values flat in CPU registers and stack frames.

### 13.1.3 Checked Arithmetic Versus Silent Release-Mode Wraparound

Switching from `f64` to `i64` closes the binary rounding hole, but immediately opens a classic systems-programming trap if you write careless operators like `a + b` or `a - b`.

In Java 6, `Long.MAX_VALUE + 1L` never throws an exception; it wraps silently via two's-complement arithmetic to `Long.MIN_VALUE`, turning the largest possible positive balance into the most negative one. In C and C++, signed integer overflow is *undefined behavior*, allowing optimizing compilers to delete bounds checks downstream. What does Rust do?

- In **debug builds** (`cargo test`, `cargo build`), `a + b` panics on overflow.
- In **release builds** (`cargo build --release`), unless you explicitly enable `overflow-checks` in `Cargo.toml`, **ordinary `+`, `-`, and `*` wrap two's-complement just like Java**. Even if you turn overflow checks on (which you should in every financial binary's release profile), panicking an HTTP worker thread on untrusted input — say a JSON body whose `amount_cents` field is `9223372036854775807` — turns a malformed payload into a denial-of-service vector.

The idiomatic Rust rule for financial code is straightforward: **ban raw `+`, `-`, `*`, and `/` on domain money types, and use `checked_add`, `checked_sub`, and widened `i128` intermediate products** returning explicit `Option` or `Result` values.

### 13.1.4 Banker's Rounding and Penny-Exact Fee Allocation

Even when balances live in exact integer cents, division re-enters the picture the moment you compute a percentage fee (2.9% of \$10.05 = 29.145 cents) or split an invoice across multiple parties. You must decide two things explicitly: *how to round a single fractional cent*, and *where leftover pennies go when splitting a total*.

**1. Unbiased midpoint rounding (Banker's Rounding).** In elementary school we learned "round half up" ($2.5 \to 3$, $3.5 \to 4$). Notice the asymmetry among the ten tenths $.0, .1, \ldots, .9$: `.0` needs no rounding, four digits (`.1, .2, .3, .4`) round down, four digits (`.6, .7, .8, .9`) round up, and `.5` **always rounds up**. Across tens of millions of two-decimal fee calculations where exact

half-cent midpoints arise routinely (e.g., 2.5% fee on any odd multiple of 20 cents), always rounding `.5` upward introduces a systematic one-way drift.

Many ledgers (and IEEE 754 itself) therefore default to **round half to even** (*Banker's Rounding*, `MidpointNearestEven` in `rust_decimal`): exact midpoints round to whichever neighbor has an *even* last digit ($2.5 \to 2$, $3.5 \to 4$, $4.5 \to 4$, $5.5 \to 6$), canceling midpoint bias across even and odd quanta. Rounding is still a policy choice: many tax and invoicing rules require half-up, so production code takes the rounding mode as an explicit parameter set per jurisdiction or contract rather than hard-coding one. Let us write an exact basis-point fee calculator (1 bp = 0.01% = 1/10,000) using pure integer arithmetic with zero floating-point instructions:

Listing 13.1: Exact integer basis-point fee calculation using widened i128 and round-half-to-even.

```rust
/// Computes `round_half_to_even(amount_minor * rate_bps / 10_000)` for non-negative
/// inputs without floating-point or i64 intermediate overflow.
pub fn compute_fee_bps_even(amount_minor: i64, rate_bps: u32) -> Result<i64, &'static str> {
    if amount_minor < 0 {
        return Err("fee base amount must be non-negative");
    }
    // Widen to i128 BEFORE multiplying: i64::MAX * u32::MAX fits easily in i128!
    let numerator: i128 = (amount_minor as i128) * (rate_bps as i128);
    let quotient: i128 = numerator / 10_000;
    let remainder: i128 = numerator % 10_000;

    let rounded = if remainder > 5_000 {
        quotient + 1
    } else if remainder < 5_000 {
        quotient
    } else {
        // Exact midpoint (.5): round to nearest EVEN integer!
        if quotient % 2 != 0 { quotient + 1 } else { quotient }
    };

    i64::try_from(rounded).map_err(|_| "rounded fee overflowed i64")
}
```

**2. Conservation across splits (The Missing-Penny Problem).** Now suppose three marketplace sellers co-own a \$100.00 (`10_000`-cent) order in equal weights $[1, 1, 1]$. If you compute each seller's share independently via round(10,000/3) = 3,333 cents, the three payouts sum to

$$3{,}333 + 3{,}333 + 3{,}333 = 9{,}999 \text{ cents} \neq 10{,}000 \text{ cents}.$$

One cent vanished. Conversely, splitting \$100.00 across six equal shares of $16.666\ldots\%$ rounded to 1,667 cents yields $6 \times 1{,}667 = 10{,}002$ cents — creating two phantom cents out of thin air.

Whenever a single monetary quantity $T \geq 0$ is partitioned across $n$ legs with positive integer weights $w_1, \ldots, w_n$ ($W = \sum_{i=1}^{n} w_i$), you must allocate shares *jointly* so conservation holds identically. Write $T \cdot w_i = q_i W + r_i$ with $0 \leq r_i < W$ by Euclidean division. Summing over all $i$ gives

$$T \cdot W = W \sum_{i=1}^{n} q_i + \sum_{i=1}^{n} r_i \quad \Longrightarrow \quad R = T - \sum_{i=1}^{n} q_i = \frac{1}{W} \sum_{i=1}^{n} r_i.$$

Because each $0 \le r_i < W$, the leftover penny count $R$ is an exact integer satisfying $0 \le R < n$. Awarding $+1$ minor unit to the $R$ legs with the largest remainders $r_i$ (breaking ties deterministically by index $i$) guarantees $\sum_{i=1}^{n} s_i = T$ with zero rounding drift:

Listing 13.2: Penny-exact proportional allocation guaranteeing `sum(shares) == total_minor`.

```
pub fn allocate_proportionally(
    total_minor: i64,
    weights: &[u64],
) -> Result<Vec<i64>, &'static str> {
    if total_minor < 0 || weights.is_empty() {
        return Err("requires non-negative total and non-empty weights");
    }
    let weight_sum: u128 = weights.iter().map(|&w| w as u128).sum();
    if weight_sum == 0 {
        return Err("at least one weight must be positive");
    }

    let total = total_minor as u128;
    let mut shares = Vec::with_capacity(weights.len());
    let mut remainders = Vec::with_capacity(weights.len());
    let mut allocated: u128 = 0;

    for (idx, &w) in weights.iter().enumerate() {
        let prod = total * (w as u128);
        let q = prod / weight_sum;
        let rem = prod % weight_sum;
        shares.push(q as i64);
        remainders.push((rem, idx));
        allocated += q;
    }

    let leftover = (total - allocated) as usize; // mathematically 0 <= leftover < len!
    // Sort descending by remainder, breaking ties by ascending original index:
    remainders.sort_by(|a, b| b.0.cmp(&a.0).then_with(|| a.1.cmp(&b.1)));
    for &(_, idx) in remainders.iter().take(leftover) {
        shares[idx] += 1;
    }
    Ok(shares)
}
```

Test `allocate_proportionally(10_000, &[1, 1, 1])` by hand: $q = [3333, 3333, 3333]$, $R = 1$, remainders are all equal (1), tie-breaker picks index 0, and the returned vector is `[3334, 3333, 3333]`, summing to `10_000` down to the last cent.

## 13.2 Type-Safe Identifiers and Multi-Currency Money

Look at a typical Java 6 DAO method signature in a legacy banking codebase:

```
public UUID recordTransfer(
    UUID tenantId, UUID debitAccountId, UUID creditAccountId,
    String idempotencyKey, long amountMinor, String currencyCode);
```

Count how many bugs the Java 6 compiler waves right through:

- Swapping `tenantId` and `debitAccountId` (or reversing `debitAccountId` and `creditAccountId` at one call site out of twelve) compiles without a single warning because all three are bare `UUID`s.
- Passing an empty or whitespace-only string `""` for `idempotencyKey` or lowercase `"usd"` for `currencyCode` compiles cleanly.
- Adding an `amountMinor` denominated in `"JPY"` (2500 = 2,500 JPY) to a balance denominated in `"USD"` (2500 = $25.00) compiles cleanly because both are bare `long`s.

In Java 6, wrapping every `UUID` and `long` in its own domain wrapper class meant allocating millions of short-lived 24-byte heap objects per second and paying pointer-indirection taxes everywhere. Recall from Chapter 8 that Rust erases this trade-off completely through **zero-cost `#[repr(transparent)]` newtypes**.

### 13.2.1 Zero-Cost Domain Identifiers and Time-Ordered UUIDv7

Let us wrap each domain identifier in its own distinct single-field tuple struct:

Listing 13.3: Zero-cost domain identifier newtypes with validated construction.

```rust
use uuid::Uuid;

#[repr(transparent)]
#[derive(Debug, Copy, Clone, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub struct AccountId(Uuid);

#[repr(transparent)]
#[derive(Debug, Copy, Clone, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub struct JournalEntryId(Uuid);

impl AccountId {
    pub fn new_v7() -> Self {
        Self(Uuid::now_v7())
    }
    pub const fn from_uuid(id: Uuid) -> Self {
        Self(id)
    }
    pub const fn as_uuid(self) -> Uuid {
        self.0
    }
}

#[derive(Debug, Clone, Eq, PartialEq, Hash)]
pub struct IdempotencyKey(String);

impl IdempotencyKey {
    pub fn parse(raw: impl AsRef<str>) -> Result<Self, &'static str> {
        let trimmed = raw.as_ref().trim();
        if trimmed.is_empty() || trimmed.len() > 128 {
            return Err("idempotency key must be 1..=128 non-whitespace chars");
        }
        if !trimmed.chars().all(|c| c.is_ascii_graphic()) {
```

```
            return Err("idempotency key must contain printable ASCII graphics");
        }
        Ok(Self(trimmed.to_owned()))
    }
    pub fn as_str(&self) -> &str {
        &self.0
    }
}
```

Notice two subtle details here. First, both `AccountId` and `JournalEntryId` derive `Ord`: in Chapter 14 we will sort `AccountId`s in canonical ascending order before acquiring PostgreSQL row locks so concurrent transfers `A -> B` and `B -> A` can never deadlock. Second, `AccountId::new_v7()` generates an RFC 9562 **UUIDv7** (whose top 48 bits store a big-endian Unix millisecond timestamp followed by 74 bits of cryptographic randomness) rather than a purely random UUIDv4. Random UUIDv4 inserts scatter uniformly across an entire multi-gigabyte PostgreSQL B-tree index, dirtying cold 8 KiB pages all over disk; time-ordered UUIDv7 inserts append monotonically at the right edge of the B-tree, keeping active leaf pages hot in shared RAM while still being globally unique without central coordination.

### 13.2.2 Phantom Compile-Time Currencies Versus Runtime Multi-Currency `Money`

How should Rust attach a currency to our `i64` minor-unit amount so `USD` and `JPY` can never be added together? Rust codebases use *two* patterns depending on whether the currency of a variable is known when compiling the binary or discovered dynamically from an incoming HTTP request. Let us build both so you know exactly how they relate.

**Pattern A: Compile-time phantom currencies (`StaticMoney<C>`).** When building a single-currency settlement pipeline or a typed foreign-exchange leg where the source and quote currencies are fixed by business logic, we attach a zero-sized marker type `C` via `PhantomData<C>` (Chapter 8):

Listing 13.4: Compile-time currency safety at zero byte overhead using PhantomData.

```
use std::marker::PhantomData;

pub trait CurrencyTag: Copy + Eq {
    const CODE: &'static str;
    const EXPONENT: u8;
}

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub struct Usd;
impl CurrencyTag for Usd { const CODE: &'static str = "USD"; const EXPONENT: u8 = 2; }

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub struct Jpy;
impl CurrencyTag for Jpy { const CODE: &'static str = "JPY"; const EXPONENT: u8 = 0; }

#[repr(transparent)]
#[derive(Debug, Copy, Clone, Eq, PartialEq, Ord, PartialOrd)]
pub struct StaticMoney<C: CurrencyTag> {
    minor: i64,
```

```
    _currency: PhantomData<C>,
}

impl<C: CurrencyTag> StaticMoney<C> {
    pub const fn from_minor(minor: i64) -> Self {
        Self { minor, _currency: PhantomData }
    }
    pub const fn minor(self) -> i64 { self.minor }

    pub fn checked_add(self, rhs: Self) -> Option<Self> {
        self.minor.checked_add(rhs.minor).map(Self::from_minor)
    }
}
```

Look at the size of `StaticMoney<Usd>`: the `PhantomData` field occupies 0 **bytes**, so the whole struct compiles down to a bare 8-byte `i64` register value. Yet writing `usd_price.checked_add(jpy_fee)` is a **compile-time type error**: the compiler expected `StaticMoney<Usd>` and found `StaticMoney<Jpy>`.

**Pattern B: Runtime-checked multi-currency `Money` for HTTP and SQL.** In our Axum and PostgreSQL ledger service, a single `POST /v1/transfers` endpoint accepts transfers in `USD`, `EUR`, `GBP`, or `JPY` selected dynamically by the JSON payload and account row. A runtime value cannot determine a compile-time generic parameter without branching over every combination. So at the general ledger layer we pair `minor: i64` with a compact 1-byte `Currency` enum, keep both struct fields *private*, and check currency equality inside `checked_add` and `checked_sub`:

Listing 13.5: Dynamic multi-currency Money value enforcing currency matching and overflow checks.

```
#[derive(Debug, Copy, Clone, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub enum Currency {
    Usd,
    Eur,
    Gbp,
    Jpy,
}

impl Currency {
    pub const fn as_str(self) -> &'static str {
        match self {
            Self::Usd => "USD",
            Self::Eur => "EUR",
            Self::Gbp => "GBP",
            Self::Jpy => "JPY",
        }
    }
    pub const fn minor_exponent(self) -> u8 {
        match self {
            Self::Usd | Self::Eur | Self::Gbp => 2,
            Self::Jpy => 0,
        }
    }
}

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
```

```rust
pub enum MoneyError {
    CurrencyMismatch { left: Currency, right: Currency },
    ArithmeticOverflow,
    NonPositiveAmount(i64),
}

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub struct Money {
    minor: i64,         // PRIVATE: external modules cannot mutate raw cents!
    currency: Currency, // PRIVATE: always paired with its ISO currency!
}

impl Money {
    pub const fn from_minor(minor: i64, currency: Currency) -> Self {
        Self { minor, currency }
    }
    pub const fn zero(currency: Currency) -> Self {
        Self { minor: 0, currency }
    }
    pub const fn minor(self) -> i64 { self.minor }
    pub const fn currency(self) -> Currency { self.currency }

    pub fn checked_add(self, rhs: Self) -> Result<Self, MoneyError> {
        if self.currency != rhs.currency {
            return Err(MoneyError::CurrencyMismatch {
                left: self.currency,
                right: rhs.currency,
            });
        }
        let sum = self.minor
            .checked_add(rhs.minor)
            .ok_or(MoneyError::ArithmeticOverflow)?;
        Ok(Self { minor: sum, currency: self.currency })
    }

    pub fn checked_sub(self, rhs: Self) -> Result<Self, MoneyError> {
        if self.currency != rhs.currency {
            return Err(MoneyError::CurrencyMismatch {
                left: self.currency,
                right: rhs.currency,
            });
        }
        let diff = self.minor
            .checked_sub(rhs.minor)
            .ok_or(MoneyError::ArithmeticOverflow)?;
        Ok(Self { minor: diff, currency: self.currency })
    }
}
```

Notice that we deliberately did *not* implement `std::ops::Add` (`+`) for `Money`. Why? Because `Add::add` in Rust must return `Self` directly rather than `Result<Self, MoneyError>`, which would force currency mismatches and integer overflows to `panic!` at runtime. Returning `Result<Money, MoneyError>` composes cleanly with the `?` operator so every caller handles mismatches and overflows

as ordinary typed errors.

## 13.3 Double-Entry Accounting From First Principles

Suppose your product manager asks for a peer-to-peer transfer feature: "Move $100.00 from Alice to Bob, charging a $1.50 platform fee." An engineer who has never built a financial system almost always designs a single database table `users(id, balance_cents)` and runs two mutable updates:

```
UPDATE users SET balance_cents = balance_cents - 10000 WHERE id = $1;
UPDATE users SET balance_cents = balance_cents + 9850  WHERE id = $2;
-- Wait: where did the 150-cent fee go?! Nobody recorded it anywhere!
```

Even if you add a third `UPDATE` to increment a corporate fee row, mutating balances in place without an immutable journal fails every requirement of a real financial backend:

1. **No self-checking conservation law:** In that two-line snippet above, $1.50 simply evaporated from the sum of all balances. If a bug in a promotion code credits Bob `10000` without debiting a marketing-expense account, total money in the database silently inflates with zero structural invariant to catch it.

2. **Zero auditability or point-in-time reconstruction:** Six weeks later, an auditor or a customer disputes why Alice's balance on October 14 at `23:59:59 UTC` was $412.10. An overwritten `balance_cents` cell remembers only its current value; its causal history was destroyed by `UPDATE`.

3. **No place for holds, in-flight settlement, or partner bank reconciliation:** Real money rarely teleports instantaneously. An ACH pull or card capture moves through pending clearing states, custodial bank accounts, reserve holds, and chargeback liabilities.

History and connection

In 1494 in Venice, the Franciscan friar and mathematician Luca Pacioli — a close collaborator of Leonardo da Vinci — published *Summa de arithmetica, geometria, proportioni et proportionalita*. Its treatise *Particularis de computis et scripturis* codified the double-entry bookkeeping method Venetian maritime merchants had refined over two centuries to track multi-currency cargo ventures across distant ports without losing gold to arithmetic errors or embezzlement. Five hundred and thirty years later, Pacioli's exact data structure — an immutable chronological *Journal* of balanced *Debit/Credit* postings mapped onto *Ledger* accounts — remains the foundational schema inside Stripe, Adyen, Square, TigerBeetle, and central-bank settlement engines.

### 13.3.1 Deriving Debits, Credits, and the Five Account Kinds Without Rote Memorization

Engineers often find accounting terminology (*debit* and *credit*) confusing because everyday consumer slang uses them backwards from how your bank sees them. When your bank sends an SMS saying "We *credited* your checking account $50," why does *credit* mean "increased" there, whereas on a company's cash asset account *debit* means "increased"?

Set aside memorization tables and derive the entire sign rule in three lines of algebra. Every financial entity obeys one foundational balance-sheet equation: everything the business *holds or owns* (**Assets**, such as cash held at a partner bank) equals what it *owes to external parties* (**Liabilities**, such as customer wallet balances payable on demand) plus the residual net worth belonging to its owners (**Equity**, which grows via **Revenue** and shrinks via **Expenses**):

$$\text{Assets} = \text{Liabilities} + \text{Equity} + \text{Revenue} - \text{Expenses}.$$

In software and bookkeeping, we hate subtracting signed numbers because negative signs invite sign-flip bugs. Move $-$Expenses across the equals sign so **every single term is non-negative and added**:

$$\underbrace{\text{Assets} + \text{Expenses}}_{\text{Left side: } \textbf{Debit-Normal} \text{ (Dr increases)}} = \underbrace{\text{Liabilities} + \text{Equity} + \text{Revenue}}_{\text{Right side: } \textbf{Credit-Normal} \text{ (Cr increases)}}. \tag{13.1}$$

Equation (13.1) explains *everything* in two words: **Debit (`Dr`) means "left side of Equation (13.1)," and Credit (`Cr`) means "right side of Equation (13.1)."**

- Every ledger posting carries a strictly positive amount $a > 0$ and a side: either `Debit` or `Credit`.
- For an account on the **left side** (`Asset` or `Expense`), a `Debit` *increases* its balance and a `Credit` *decreases* its balance:

$$\text{balance}_{\text{debit-normal}} = \sum \text{Dr} - \sum \text{Cr}.$$

- For an account on the **right side** (`Liability`, `Equity`, or `Revenue`), a `Credit` *increases* its balance and a `Debit` *decreases* its balance:

$$\text{balance}_{\text{credit-normal}} = \sum \text{Cr} - \sum \text{Dr}.$$

- **The Conservation Law of Double-Entry:** Every `JournalEntry` consists of two or more postings whose total Debits equal total Credits for each currency:

$$\sum_{p \in \text{Debits}(E)} p.\text{amount} = \sum_{p \in \text{Credits}(E)} p.\text{amount}. \tag{13.2}$$

Check why Equation (13.2) automatically preserves Equation (13.1) on every single write. Whether a journal entry moves value *across* the equals sign (say +$100 Debit to an Asset and +$100 Credit to a Liability: both sides of (13.1) rise by $100) or *within* the right-hand side (say $100 Debit to Alice's Liability wallet, $97 Credit to Bob's Liability wallet, and $3 Credit to Platform Fee Revenue: the right-hand side changes by $-100 + 97 + 3 = 0$), Equation (13.1) can **never** fall out of balance.

Intuition

**Why a customer's wallet is a Liability (and why "crediting Alice" makes total sense now).** When Alice wires $100.00 into your payment platform, that cash sits physically inside your company's custodial omnibus bank account at a partner bank (an FBO — *For Benefit Of* — account).

- Your `FBO_Bank_Cash` account is an **Asset** (left side): it goes up by $100.00 via a **Debit** of `10_000` cents.

- `Alice_Wallet` is a **Liability** (right side) — money your platform owes back to Alice. Because Liabilities live on the right side, increasing Alice's wallet balance requires a **Credit** of `10_000` cents, and spending from Alice's wallet requires a **Debit**.

That is why banks say "we *credited* your checking account" when you deposit money and call your spending card a "*debit* card": the bank is speaking from the perspective of *its own* ledger, on which your checking account is the bank's Liability to you.

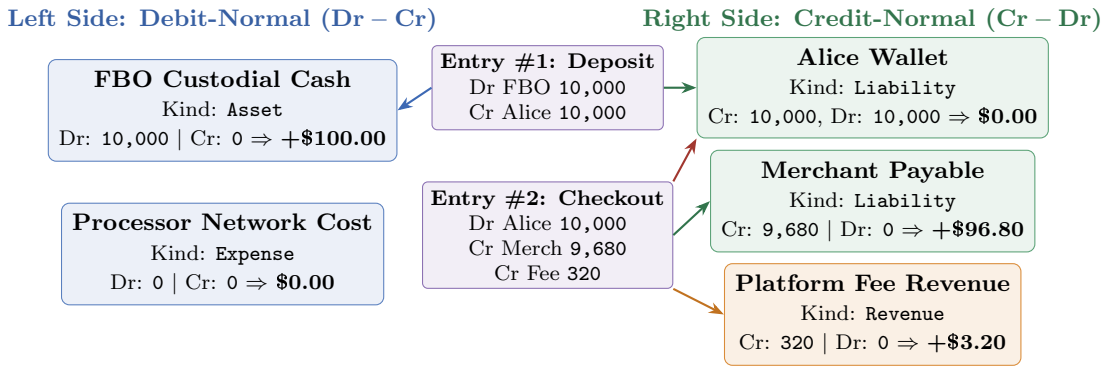



Figure 13.2: Two immutable double-entry journal entries flowing across account categories. Entry #1 records Alice's $100.00 deposit (`Dr` Asset `10,000` = `Cr` Liability `10,000`). Entry #2 settles a $100.00 purchase with a $3.20 fee (`Dr` Alice `10,000` = `Cr` Merchant `9,680` + `Cr` Revenue `320`). Total Assets ($100.00) equal total Liabilities + Revenue ($0 + $96.80 + $3.20 = $100.00) after every entry.

Let us encode these exact accounting concepts into crisp, exhaustive Rust enums so account polarity is computed in one place and never scattered across ad-hoc `if/else` branches:

Listing 13.6: Encoding double-entry account categories, normal polarity, and posting sides in Rust.

```rust
#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub enum NormalBalance {
    DebitNormal,  // Assets, Expenses:  balance = sum(Dr) - sum(Cr)
    CreditNormal, // Liabilities, Equity, Revenue: balance = sum(Cr) - sum(Dr)
}

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub enum AccountKind {
    Asset,
    Liability,
    Equity,
    Revenue,
    Expense,
}

impl AccountKind {
    pub const fn normal_balance(self) -> NormalBalance {
        match self {
            Self::Asset | Self::Expense => NormalBalance::DebitNormal,
            Self::Liability | Self::Equity | Self::Revenue => NormalBalance::CreditNormal,
        }
    }
```

```rust
}

#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub enum EntrySide {
    Debit,
    Credit,
}

impl EntrySide {
    pub const fn opposite(self) -> Self {
        match self {
            Self::Debit => Self::Credit,
            Self::Credit => Self::Debit,
        }
    }

    /// Signed delta (+amount or -amount) on an account's natural balance!
    pub const fn signed_delta(self, kind: AccountKind, positive_minor: i64) -> i64 {
        match (kind.normal_balance(), self) {
            (NormalBalance::DebitNormal, Self::Debit) => positive_minor,
            (NormalBalance::DebitNormal, Self::Credit) => -positive_minor,
            (NormalBalance::CreditNormal, Self::Credit) => positive_minor,
            (NormalBalance::CreditNormal, Self::Debit) => -positive_minor,
        }
    }
}
```

Look at `EntrySide::signed_delta` on lines 40–47: four match arms capture the entire 530-year-old polarity table without a single boolean flag. And notice `EntrySide::opposite`: how do you reverse or refund a committed `JournalEntry` in an immutable append-only ledger? You *never* run `UPDATE` or `DELETE` on historical postings; you construct a new compensating `JournalEntry` pointing to `reverses_entry_id: Some(original.id())` whose postings copy every leg of `original` with `side.opposite()`.

## 13.4 Parse, Don't Validate: Typestate and Proof-Carrying Entries

Now let us bring together everything in this chapter to design the boundary between untrusted outside input (HTTP JSON DTOs) and our database persistence layer.

Recall the distinction between *validation* and *parsing* from Chapter 8:

- A **validation** function such as `check_entry(&RawEntry)` inspects a bag of mutable primitives, returns `Ok(())`, and *throws away the proof*. Downstream functions still accept `&RawEntry`, forcing every layer either to trust that every caller remembered to call `check_entry` (and did not mutate a field afterward) or to re-check invariants defensively everywhere.
- A **parsing** or **typestate** pipeline turns a `JournalDraft<Unsealed>` into a `Result` holding a `BalancedEntry`. It consumes raw data by value and returns a **new, sealed type whose private fields cannot be constructed any other way**. Every downstream function (such as `commit_entry` in the repository layer) demands `&BalancedEntry` in its signature. Passing an unbalanced or zero-amount posting to PostgreSQL becomes a *compile-time type error*.

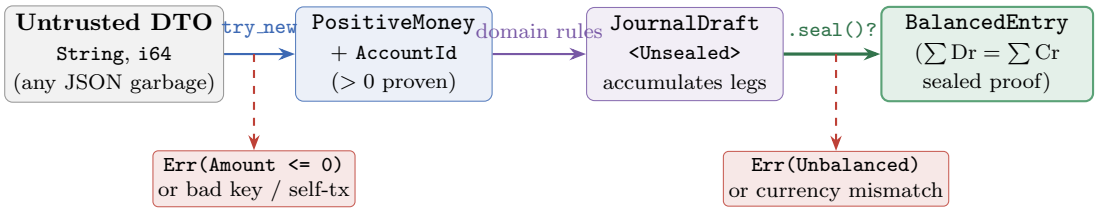



Figure 13.3: The "Parse, Don't Validate" typestate funnel. All invariant checks happen once at type-transition boundaries (`try_new` and `.seal()`). Any code holding a `BalancedEntry` possesses a compile-time proof that every posting is strictly positive, shares a valid currency, and satisfies $\sum \text{Dr} = \sum \text{Cr}$.

### 13.4.1 Step 1: Strictly Positive Amounts (`PositiveMoney`)

In double-entry accounting, a ledger posting never carries a negative number (direction is expressed by `EntrySide::Debit` versus `EntrySide::Credit`) and never carries zero cents (zero-amount rows are meaningless noise). Let us lock that invariant inside `PositiveMoney`:

Listing 13.7: PositiveMoney guarantees minor ¿ 0 by construction via a private field.

```rust
#[derive(Debug, Copy, Clone, Eq, PartialEq)]
pub struct PositiveMoney(Money); // inner field is private to this module!

impl PositiveMoney {
    pub fn try_new(minor: i64, currency: Currency) -> Result<Self, MoneyError> {
        if minor <= 0 {
            return Err(MoneyError::NonPositiveAmount(minor));
        }
        Ok(Self(Money::from_minor(minor, currency)))
    }

    pub const fn as_money(self) -> Money {
        self.0
    }
    pub const fn minor(self) -> i64 {
        self.0.minor()
    }
    pub const fn currency(self) -> Currency {
        self.0.currency()
    }
}
```

Think about an exploit class that `PositiveMoney` kills immediately: in dozens of historical web-wallet vulnerabilities, an attacker submitted {`"to": "victim", "amount": -50000`} to a "send money" endpoint; naive code checked `if sender_balance >= amount` ($0 \geq -50{,}000$ is `true`), subtracted `-50000` from the attacker (adding $500), and added `-50000` to the victim. With `PositiveMoney`, negative and zero transfer amounts cannot even cross into domain functions.

### 13.4.2 Step 2: Typestate Sealing of `JournalDraft` Into `BalancedEntry`

Now let us build the journal entry builder using typestate markers `Unsealed` and `Sealed` (and exporting a clean alias `pub type BalancedEntry = JournalEntry<Sealed>;`). Read how ownership, destructive moves (`self` by value on `seal`), and private struct fields work together:

Listing 13.8: Typestate journal construction proving conservation of money before persistence.

```
use std::marker::PhantomData;

#[derive(Debug, Clone, Eq, PartialEq)]
pub struct Posting {
    account_id: AccountId,
    side: EntrySide,
    amount: PositiveMoney,
}

impl Posting {
    pub const fn new(
        account_id: AccountId,
        side: EntrySide,
        amount: PositiveMoney,
    ) -> Self {
        Self { account_id, side, amount }
    }
    pub const fn account_id(&self) -> AccountId { self.account_id }
    pub const fn side(&self) -> EntrySide { self.side }
    pub const fn amount(&self) -> PositiveMoney { self.amount }
}

// Zero-sized typestate markers:
pub struct Unsealed;
pub struct Sealed;

#[derive(Debug, Clone, Eq, PartialEq)]
pub enum LedgerDomainError {
    Money(MoneyError),
    TooFewPostings(usize),
    SameAccountSimpleTransfer(AccountId),
    UnbalancedEntry {
        currency: Currency,
        total_debits: i64,
        total_credits: i64,
    },
    CaptureExceedsHold { held: i64, requested: i64 },
}

impl From<MoneyError> for LedgerDomainError {
    fn from(e: MoneyError) -> Self { Self::Money(e) }
}

pub struct JournalEntry<State> {
    id: JournalEntryId,
    idempotency_key: IdempotencyKey,
    description: String,
```

```
    currency: Currency,
    postings: Vec<Posting>,
    _state: PhantomData<State>,
}

pub type JournalDraft = JournalEntry<Unsealed>;
pub type BalancedEntry = JournalEntry<Sealed>;

// Methods ONLY available while Unsealed:
impl JournalEntry<Unsealed> {
    pub fn new_draft(
        idempotency_key: IdempotencyKey,
        currency: Currency,
        description: impl Into<String>,
    ) -> Self {
        Self {
            id: JournalEntryId(Uuid::now_v7()),
            idempotency_key,
            description: description.into(),
            currency,
            postings: Vec::with_capacity(4),
            _state: PhantomData,
        }
    }

    pub fn push_debit(
        &mut self,
        account_id: AccountId,
        amount: PositiveMoney,
    ) -> Result<&mut Self, LedgerDomainError> {
        if amount.currency() != self.currency {
            return Err(MoneyError::CurrencyMismatch {
                left: self.currency,
                right: amount.currency(),
            }.into());
        }
        self.postings.push(Posting::new(account_id, EntrySide::Debit, amount));
        Ok(self)
    }

    pub fn push_credit(
        &mut self,
        account_id: AccountId,
        amount: PositiveMoney,
    ) -> Result<&mut Self, LedgerDomainError> {
        if amount.currency() != self.currency {
            return Err(MoneyError::CurrencyMismatch {
                left: self.currency,
                right: amount.currency(),
            }.into());
        }
        self.postings.push(Posting::new(account_id, EntrySide::Credit, amount));
        Ok(self)
    }
```

```rust

    /// Consumes the Unsealed draft by value and returns a Sealed BalancedEntry
    /// ONLY if >= 2 postings exist and sum(Debits) == sum(Credits) without overflow!
    pub fn seal(self) -> Result<BalancedEntry, LedgerDomainError> {
        if self.postings.len() < 2 {
            return Err(LedgerDomainError::TooFewPostings(self.postings.len()));
        }

        let mut debits = Money::zero(self.currency);
        let mut credits = Money::zero(self.currency);

        for p in &self.postings {
            match p.side {
                EntrySide::Debit => debits = debits.checked_add(p.amount.as_money())?,
                EntrySide::Credit => credits = credits.checked_add(p.amount.as_money())?,
            }
        }

        if debits.minor() != credits.minor() {
            return Err(LedgerDomainError::UnbalancedEntry {
                currency: self.currency,
                total_debits: debits.minor(),
                total_credits: credits.minor(),
            });
        }

        Ok(JournalEntry {
            id: self.id,
            idempotency_key: self.idempotency_key,
            description: self.description,
            currency: self.currency,
            postings: self.postings,
            _state: PhantomData,
        })
    }
}

// Read-only accessors ONLY on Sealed (BalancedEntry) -- NO push_debit/push_credit!
impl JournalEntry<Sealed> {
    pub const fn id(&self) -> JournalEntryId { self.id }
    pub const fn idempotency_key(&self) -> &IdempotencyKey { &self.idempotency_key }
    pub fn description(&self) -> &str { &self.description }
    pub const fn currency(&self) -> Currency { self.currency }
    pub fn postings(&self) -> &[Posting] { &self.postings }
}
```

Here is what the compiler now enforces across the whole workspace:

1. Can a caller mutate `BalancedEntry` after calling `.seal()`? *No.* A sealed entry exposes its postings only as an immutable slice (`&[Posting]`) and does **not** implement `push_debit` or `push_credit`.

2. Can a caller pass an unsealed `JournalDraft` directly to `repo::commit_entry(tx, entry: &BalancedEntry)`? *No.* `JournalEntry<Unsealed>` and `JournalEntry<Sealed>` are incom-

patible types at compile time, even though their runtime memory layout is byte-for-byte identical (`_state: PhantomData` is 0 bytes, so `seal()` compiles down to checking the sum and returning the exact same `Vec<Posting>` pointer in registers with zero reallocation).

3. Can a caller construct a sealed entry by struct literal outside the `domain::ledger` module? *No.* Because the fields `id`, `postings`, and `_state` are module-private, the *only* path in the program that produces a `BalancedEntry` runs through the debit-equals-credit check inside `seal()`.

Here is what those rules look like when someone breaks them:

```
let entry: BalancedEntry = draft.seal()?;
entry.push_debit(alice, fee);          // (1) mutate after sealing
repo::commit_entry(&mut tx, &draft);   // (2) persist an unsealed draft
let (captured, e1) = hold.capture(k1, merchant, amt)?;
let (voided, e2) = hold.void_hold(k2)?; // (3) void after capture
```

```
error[E0599]: no method named ‘push_debit‘ found for struct
              ‘JournalEntry<Sealed>‘ in the current scope
error[E0308]: mismatched types
   expected reference ‘&JournalEntry<Sealed>‘
      found reference ‘&JournalEntry<Unsealed>‘
error[E0382]: use of moved value: ‘hold‘
   value moved here: ‘hold.capture(k1, merchant, amt)‘
```

(Error (2) also fires a “use of moved value” for `draft`, since `seal` consumed it.) Each mistake that would be a production incident in a Java 6 ledger becomes a red line in the editor.

### 13.4.3 Putting It Together: Domain Transfer Constructor With Optional Fee Split

Look how clean our high-level domain business rule becomes when built atop `PositiveMoney` and `BalancedEntry`. Suppose a customer transfers `gross` from `sender` to `recipient` with an optional platform fee going to `fee_revenue_account`:

Listing 13.9: Building a guaranteed-balanced customer transfer with optional platform fee leg.

```
pub fn build_peer_transfer_entry(
    key: IdempotencyKey,
    sender_wallet: AccountId,
    recipient_wallet: AccountId,
    fee_revenue_acct: AccountId,
    gross: PositiveMoney,
    fee_bps: u32,
) -> Result<BalancedEntry, LedgerDomainError> {
    if sender_wallet == recipient_wallet {
        return Err(LedgerDomainError::SameAccountSimpleTransfer(sender_wallet));
    }
    let currency = gross.currency();
    let fee_minor = compute_fee_bps_even(gross.minor(), fee_bps)
        .map_err(|_| MoneyError::ArithmeticOverflow)?;
    let net_minor = gross.minor()
        .checked_sub(fee_minor)
        .ok_or(MoneyError::ArithmeticOverflow)?;
```

```
18
19      let mut draft = JournalDraft::new_draft(
20          key,
21          currency,
22          "peer transfer",
23      );
24
25      // Sender wallet is a Credit-normal customer liability: Debit decreases it!
26      draft.push_debit(sender_wallet, gross)?;
27
28      // Credit net proceeds to recipient liability (if net > 0):
29      if let Ok(net_pos) = PositiveMoney::try_new(net_minor, currency) {
30          draft.push_credit(recipient_wallet, net_pos)?;
31      }
32      // Credit fee to platform revenue account (if fee > 0):
33      if let Ok(fee_pos) = PositiveMoney::try_new(fee_minor, currency) {
34          draft.push_credit(fee_revenue_acct, fee_pos)?;
35      }
36
37      // Proves gross == net_minor + fee_minor and seals the entry!
38      draft.seal()
39  }
```

### 13.4.4 Lifecycle Typestate: Authorization Holds, Captures, and Voids

Beyond single-step transfers, card networks and ACH rails separate **authorization** (placing a hold on funds when you swipe your card at a hotel check-in) from **settlement/capture** (charging the final room bill three days later) or **void** (releasing the hold if you cancel).

In Java 6 or C++, an `AuthHold` class typically carries a `Status` enum field (`AUTHORIZED`, `CAPTURED`, `VOIDED`), and every method begins with a guard that throws `IllegalStateException` unless the status is `AUTHORIZED`. In Rust, we encode the lifecycle state as a type parameter: `AuthHold<Authorized>` has `capture` and `void_hold` methods that **consume `self` by value** (a destructive move), so calling `.capture()` twice, or calling `.void_hold()` on an already-captured hold, is a compile-time error:

Listing 13.10: Destructive-move typestate preventing double-capture or void-after-capture at compile time.

```
1   pub struct Authorized;
2   pub struct Captured;
3   pub struct Voided;
4
5   pub struct AuthHold<State> {
6       hold_id: Uuid,
7       customer_wallet: AccountId,
8       customer_hold_subaccount: AccountId,
9       held_amount: PositiveMoney,
10      _state: PhantomData<State>,
11  }
12
13  impl AuthHold<Authorized> {
14      /// Consumes `self` (Authorized) by value! Calling `.capture()` twice or
```

```
    /// `.void_hold()` afterward fails to compile with "use of moved value".
    pub fn capture(
        self,
        key: IdempotencyKey,
        merchant_wallet: AccountId,
        capture_amount: PositiveMoney,
    ) -> Result<(AuthHold<Captured>, BalancedEntry), LedgerDomainError> {
        if capture_amount.currency() != self.held_amount.currency() {
            return Err(MoneyError::CurrencyMismatch {
                left: self.held_amount.currency(),
                right: capture_amount.currency(),
            }.into());
        }
        let unused_minor = self.held_amount.minor()
            .checked_sub(capture_amount.minor())
            .ok_or(MoneyError::ArithmeticOverflow)?;
        if unused_minor < 0 {
            return Err(LedgerDomainError::CaptureExceedsHold {
                held: self.held_amount.minor(),
                requested: capture_amount.minor(),
            });
        }

        let mut draft = JournalDraft::new_draft(
            key,
            self.held_amount.currency(),
            "auth hold capture",
        );
        // Drain the entire held liability subaccount:
        draft.push_debit(self.customer_hold_subaccount, self.held_amount)?;
        // Credit captured portion to merchant:
        draft.push_credit(merchant_wallet, capture_amount)?;
        // Return any uncaptured remainder back to customer's available wallet:
        if let Ok(remainder) = PositiveMoney::try_new(
            unused_minor,
            self.held_amount.currency(),
        ) {
            draft.push_credit(self.customer_wallet, remainder)?;
        }

        let entry = draft.seal()?;
        let captured = AuthHold {
            hold_id: self.hold_id,
            customer_wallet: self.customer_wallet,
            customer_hold_subaccount: self.customer_hold_subaccount,
            held_amount: self.held_amount,
            _state: PhantomData,
        };
        Ok((captured, entry))
    }

    pub fn void_hold(
        self,
        key: IdempotencyKey,
```

```
    ) -> Result<(AuthHold<Voided>, BalancedEntry), LedgerDomainError> {
        let mut draft = JournalDraft::new_draft(
            key,
            self.held_amount.currency(),
            "auth hold void",
        );
        draft.push_debit(self.customer_hold_subaccount, self.held_amount)?;
        draft.push_credit(self.customer_wallet, self.held_amount)?;
        let entry = draft.seal()?;
        let voided = AuthHold {
            hold_id: self.hold_id,
            customer_wallet: self.customer_wallet,
            customer_hold_subaccount: self.customer_hold_subaccount,
            held_amount: self.held_amount,
            _state: PhantomData,
        };
        Ok((voided, entry))
    }
}
```

Notice how subaccounts handle holds in `AuthHold<Authorized>::capture`. Instead of inventing an error-prone "available balance minus pending holds" query across disconnected tables, placing an authorization hold is *itself* a balanced journal entry moving funds from `customer_wallet` to `customer_hold_subaccount` (both Liabilities belonging to Alice). And partial capture (say holding $150.00 at a gas pump and capturing $42.18) drains the $150.00 hold subaccount, credits $42.18 to the merchant, and credits the unused $107.82 right back to Alice's wallet in one balanced 3-leg entry ($15{,}000 = 4{,}218 + 10{,}782$).

With our domain invariants locked tight inside `Money`, `PositiveMoney`, `AccountId`, `IdempotencyKey`, and `BalancedEntry`, we are ready in Chapter 14 to wire this exact domain core into PostgreSQL, SQLx, Tower, and Axum — solving concurrent overdraft races, deadlocks, idempotency retries, and transactional event delivery end to end.

## 13.5 Exercises

**Exercise 13.1** (Quantifying IEEE-754 Drift and Penny Loss)**.** Write a short Rust test or calculate directly from IEEE-754 binary64 layout:

1. The text claimed that a dollar balance stored as `f64` silently swallows added cents once it reaches $2^{47}$. Derive this from the binary64 layout: find the smallest power of two $D$ at which the ulp exceeds 0.02, so that `d + 0.01 == d`. Then explain why balances just below $2^{47}$ still change when you add a cent, but by the wrong amount.
2. A ledger charges a 2.5% fee (250 basis points) on 10,000,000 transactions that alternate between $1.40 and $1.80. Both produce exact half-cent fees (3.5 and 4.5 cents). Compute the total fees collected under schoolbook round-half-up and under `compute_fee_bps_even`, and the dollar difference between them.

**Exercise 13.2** (Proving Remainder Conservation in `allocate_proportionally`)**.** Prove rigorously for any integer $T \geq 0$ and any non-empty slice of weights $w_1, \ldots, w_n \geq 0$ with $W = \sum_{i=1}^{n} w_i > 0$ that:

1. In `allocate_proportionally`, the leftover quantity $R = T - \sum_{i=1}^{n} \lfloor Tw_i/W \rfloor$ is an integer strictly less than the number of strictly positive weights, so the loop that awards leftover cents never gives +1 to a zero-weight participant.

2. Extend `allocate_proportionally` to handle *signed* totals $T < 0$ (for example, allocating a negative fee rebate across co-owners) while preserving $\sum_{i=1}^{n} s_i = T$.

**Exercise 13.3** (Multi-Currency Foreign Exchange via Clearing Accounts)**.** Suppose Alice converts 100.00 EUR (`10_000 Eur` minor units) from her EUR wallet into her USD wallet at an exchange rate of 1.0847 USD/EUR, yielding $108.47 (`10_847 Usd` minor units).

1. Why can a single two-leg entry (`Debit Alice_EUR 10_000`, `Credit Alice_USD 10_847`) never satisfy single-currency conservation, and what goes wrong on your company balance sheet if FX rates move tomorrow?

2. Introduce two internal liquidity/FX position accounts (`FxClearing_EUR` and `FxClearing_USD`) and generalize `JournalDraft` and `BalancedEntry` so a multi-currency FX journal entry contains four postings — two in `Eur` and two in `Usd` — and `seal()` verifies $\sum_{p \in \mathrm{Dr}(c)} p.\mathrm{amount} = \sum_{p \in \mathrm{Cr}(c)} p.\mathrm{amount}$ independently for *every* `Currency` $c$ present in the entry.

**Exercise 13.4** (Tracing T-Accounts Through a Dispute Chargeback Lifecycle)**.** Using the five account categories (`Asset`, `Liability`, `Equity`, `Revenue`, `Expense`), write out the exact sequence of balanced `JournalEntry` postings (account name, `AccountKind`, `EntrySide`, minor units in cents) and verify Equation (13.1) after every step for:

1. Customer Alice funds her wallet with $200.00 via card settlement into `FBO_Cash`.

2. Alice buys a $120.00 jacket from Merchant M; your platform takes a 2.5% fee ($3.00) and credits $117.00 to `Merchant_M_Payable`.

3. Merchant M withdraws $100.00 from `Merchant_M_Payable` out of `FBO_Cash` via ACH payout (leaving $17.00 in `Merchant_M_Payable`).

4. Alice files a $120.00 fraud chargeback that she wins, plus the card network debits a $15.00 dispute penalty from `FBO_Cash`. Show what happens to `Merchant_M_Payable` (or a dedicated `Merchant_Receivable` Asset / negative Liability balance) and why payment processors hold rolling reserves (`Merchant_Reserve` Liability subaccount) to cover Step 4.

**Exercise 13.5** (Property-Based Testing of Domain Invariants)**.** Write three property tests (using logic suitable for the `proptest` crate) against `compute_fee_bps_even`, `allocate_proportionally`, and `build_peer_transfer_entry`:

1. For all `gross` in `1..=10_000_000_000_000_000_i64` and `fee_bps` in `0..=10_000_u32` with distinct `AccountId`s, `build_peer_transfer_entry` always returns `Ok(entry)` whose `postings()` sum to equal debits and credits.

2. What happens in `build_peer_transfer_entry` if the fee rate exceeds 10,000 basis points (say a 120% fee)? Trace which check rejects it, then add an explicit `InvalidFeeRateBps(u32)` variant to `LedgerDomainError` so the rejection documents itself.

**Exercise 13.6** (Designing a Typestate Settlement Batch)**.** At `23:00 UTC` each banking day, your settlement engine groups thousands of merchant payouts into an NACHA ACH batch file:

1. Design a three-state typestate struct `PayoutBatch<State>` that moves from `Open` to `Sealed` to `Transmitted`. The method `add_payout` exists only on `Open`; `compute_control_hash` and

`render_nacha` exist only on `Sealed`; and `mark_transmitted`, which takes the bank's acknowledgement ID, consumes a `Sealed` batch by value and returns a `Transmitted` one.

2. Explain how storing `PhantomData<fn() -> State>` instead of `PhantomData<State>` makes the batch type automatically `Send` and `Sync` even if a marker struct such as `Open` ever held non-`Sync` metadata. Refer to variance and auto-traits from Chapter 4.

# Chapter 14

# Building the Financial Service End-to-End: Axum, Tower, SQLx, and PostgreSQL

We now have both halves of the system in hand. Chapter 12 showed how a Rust process waits: each request becomes a compact `Future` state machine, Tokio polls thousands of them on a few threads, and dropping a future cancels it at its current `.await`. Chapter 13 showed what the process computes: money as checked integers tagged with a currency, double-entry journal entries, and a `BalancedEntry` type that can only exist if debits equal credits.

This chapter wires those halves into a running service. A client sends

```
POST /v1/transfers
Idempotency-Key: 7f3c9a2e-checkout-4411
Content-Type: application/json

{"from": "0190f1c2-...", "to": "0190f1c3-...", "amount_minor": 8000,
 "currency": "USD", "fee_bps": 250}
```

and we follow those bytes all the way down: through Tower middleware, Axum extractors, our domain types, a SQLx transaction, PostgreSQL row locks and constraints, a transactional outbox, and back out as an HTTP response. Along the way we answer the questions that separate a demo from a ledger you can trust with other people's money:

- What does the database itself refuse to store, even if application code has a bug?
- How can a SQL query be type-checked by `cargo build`?
- What happens when two withdrawals of $80 hit a $100 balance in the same millisecond?
- What happens when the client times out, retries, and the first attempt actually succeeded?
- What happens to in-flight transfers when Kubernetes sends `SIGTERM`?

## 14.1 The PostgreSQL Schema: Constraints as a Second Line of Defense

Chapter 13 made illegal states unrepresentable inside Rust. But a database outlives any single binary. Over the years, other services, migration scripts, and an engineer in a `psql` session at 2 a.m. will all write to these tables. So we repeat the most important invariants in the schema itself, where no caller can skip them. The type system is the first line of defense; constraints are the second.

Figure 14.1 shows the five tables and how they reference each other. Read it before the DDL: `accounts` holds one row per ledger account; each `journal_entries` row is one atomic business event; `postings` holds the individual debit and credit legs of each entry; `idempotency_keys` remembers which client requests we already answered; and `outbox_events` queues notifications for other systems.

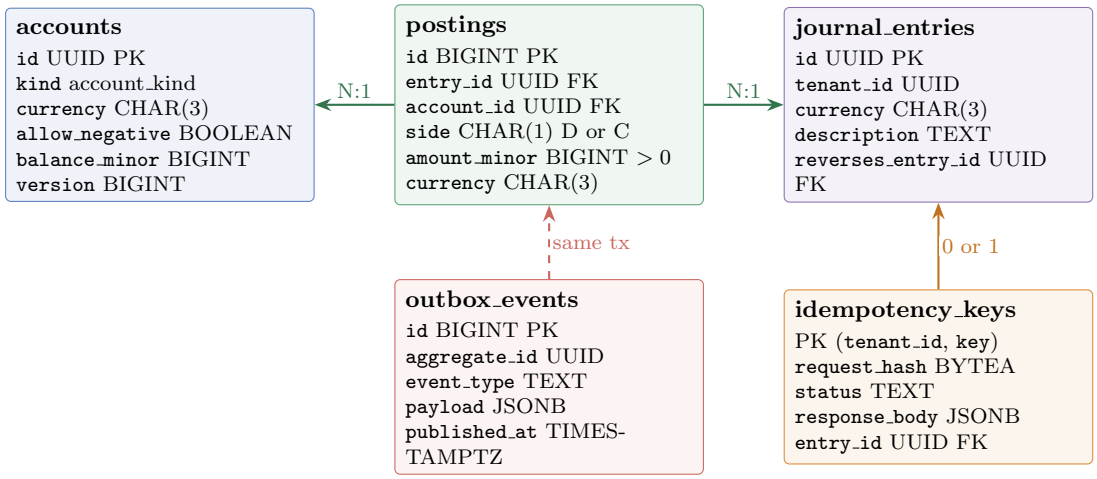



Figure 14.1: The ledger schema. Every posting belongs to exactly one journal entry and one account. An idempotency record points at the entry it produced. Outbox rows are not linked by a foreign key, but they are always inserted in the same transaction as the postings they describe.

Here is the migration, `migrations/0001_ledger.sql`. Read each constraint as a sentence the database will enforce for every writer, forever.

Listing 14.1: The core ledger schema with constraints that mirror the Rust domain invariants.

```
CREATE TYPE account_kind AS ENUM
    ('asset', 'liability', 'equity', 'revenue', 'expense');

CREATE TABLE accounts (
    id              UUID PRIMARY KEY,           -- UUIDv7, generated in Rust
    tenant_id       UUID NOT NULL,
    kind            account_kind NOT NULL,
    currency        CHAR(3) NOT NULL CHECK (currency ~ '^[A-Z]{3}$'),
    allow_negative BOOLEAN NOT NULL DEFAULT FALSE,
    balance_minor   BIGINT NOT NULL DEFAULT 0,  -- projection of postings
    version         BIGINT NOT NULL DEFAULT 0,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT balance_floor CHECK (allow_negative OR balance_minor >= 0)
);
```

```
15
16 CREATE TABLE journal_entries (
17     id                UUID PRIMARY KEY,
18     tenant_id         UUID NOT NULL,
19     currency          CHAR(3) NOT NULL,
20     description       TEXT NOT NULL,
21     reverses_entry_id UUID REFERENCES journal_entries (id),
22     created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
23 );
24
25 CREATE TABLE postings (
26     id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
27     entry_id     UUID NOT NULL REFERENCES journal_entries (id),
28     account_id   UUID NOT NULL REFERENCES accounts (id),
29     side         CHAR(1) NOT NULL CHECK (side IN ('D', 'C')),
30     amount_minor BIGINT NOT NULL CHECK (amount_minor > 0),
31     currency     CHAR(3) NOT NULL,
32     created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
33 );
34 CREATE INDEX postings_by_account ON postings (account_id, created_at, id);
35 CREATE INDEX postings_by_entry   ON postings (entry_id);
36
37 CREATE TABLE idempotency_keys (
38     tenant_id       UUID NOT NULL,
39     key             TEXT NOT NULL CHECK (length(key) BETWEEN 1 AND 128),
40     request_hash    BYTEA NOT NULL,            -- SHA-256 of canonical request
41     status          TEXT NOT NULL CHECK (status IN ('in_progress', 'completed')),
42     response_status SMALLINT,                  -- NULL until completed
43     response_body   JSONB,                     -- NULL until completed
44     entry_id        UUID REFERENCES journal_entries (id),
45     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
46     PRIMARY KEY (tenant_id, key)
47 );
48
49 CREATE TABLE outbox_events (
50     id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
51     aggregate_id UUID NOT NULL,
52     event_type   TEXT NOT NULL,
53     payload      JSONB NOT NULL,
54     created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
55     published_at TIMESTAMPTZ
56 );
57 CREATE INDEX outbox_pending ON outbox_events (id) WHERE published_at IS NULL;
58
59 CREATE TABLE fx_rates (
60     base_currency  CHAR(3) NOT NULL,
61     quote_currency CHAR(3) NOT NULL,
62     rate           NUMERIC(20, 8) NOT NULL CHECK (rate > 0),
63     valid_from     TIMESTAMPTZ NOT NULL,
64     PRIMARY KEY (base_currency, quote_currency, valid_from)
65 );
```

A few of these lines deserve a closer look, because each one corresponds to a decision from earlier chapters.

- `BIGINT` **for every amount.** PostgreSQL's `BIGINT` is an 8-byte signed integer, the exact twin of Rust's `i64`. Arithmetic on it is exact, and `SUM` never depends on row order. `NUMERIC` appears only in `fx_rates`, where the value is a ratio with many decimal places; it maps to `rust_decimal::Decimal`.
- `CHECK (amount_minor > 0).` This is `PositiveMoney` from Chapter 13, restated in SQL. Direction lives in `side`, never in the sign of the amount.
- `balance_floor.` Customer wallets may not go negative; some internal accounts (a settlement clearing account, for instance) legitimately can. Rather than trusting application code to check, the constraint makes an overdraft impossible to commit.
- `balance_minor` **is a projection.** The source of truth is the immutable `postings` table. The balance column is a cached sum we update in the same transaction, so we can check and lock a balance with one indexed row read instead of summing millions of postings. A nightly job recomputes every balance from postings and pages someone if any projection disagrees.
- **Indexes follow the queries.** `postings_by_account` serves statement pages ("all postings on account $X$ in October, in order"). The partial index `outbox_pending` contains only unpublished rows, so it stays tiny no matter how many billions of events have been delivered.

### 14.1.1 A Database-Level Balance Check

Our Rust `BalancedEntry` guarantees that every entry *we* write balances. To protect against every other writer, we add a constraint trigger that re-checks the conservation law inside PostgreSQL. It is declared `DEFERRABLE INITIALLY DEFERRED`, which means it does not fire after each individual row insert (when the first posting of an entry is necessarily unbalanced) but once at `COMMIT`, when all legs are present.

Listing 14.2: A deferred constraint trigger that rejects unbalanced journal entries at commit time.

```
CREATE FUNCTION assert_entry_balanced() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    net BIGINT;
BEGIN
    SELECT COALESCE(SUM(CASE side WHEN 'D' THEN amount_minor
                                  ELSE -amount_minor END), 0)
      INTO net
      FROM postings
     WHERE entry_id = NEW.entry_id;
    IF net <> 0 THEN
        RAISE EXCEPTION 'journal entry % is unbalanced by %',
            NEW.entry_id, net USING ERRCODE = 'check_violation';
    END IF;
    RETURN NULL;
END $$;

CREATE CONSTRAINT TRIGGER postings_must_balance
    AFTER INSERT ON postings
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION assert_entry_balanced();
```

The trigger costs one indexed aggregate per posting at commit. For most ledgers that is a fair price for a guarantee that holds even when someone bypasses the Rust service entirely. Systems pushing tens of thousands of entries per second sometimes drop it and rely on the application type plus the nightly reconciliation; that is an explicit trade, and it should be written down as one.

## 14.2 SQLx: SQL Checked at Compile Time

If you wrote JDBC in Java 6, you remember the routine: build a SQL string, call `prepareStatement`, bind parameters by position with `setLong(2, ...)`, then pull columns out of a `ResultSet` with `getLong("balance_minor")`. Every one of those steps is checked only at runtime. A typo in a column name, a parameter bound at the wrong index, or a nullable column read into a primitive `long` (which silently becomes `0`) all compile cleanly and fail in production.

SQLx takes a different approach. Its `query!` and `query_as!` macros run *inside the compiler* and ask a real PostgreSQL server to check your SQL while `cargo build` runs.

### 14.2.1 What `query!` Does During `cargo build`

Take this query, which loads one account:

Listing 14.3: A compile-time checked query with SQLx.

```
let row = sqlx::query!(
    r#"SELECT id, kind AS "kind: AccountKind", currency,
              allow_negative, balance_minor
         FROM accounts
        WHERE id = $1"#,
    account_id.as_uuid()
)
.fetch_one(&pool)
.await?;

let balance: i64 = row.balance_minor; // a plain i64, not Option<i64>
```

When `rustc` expands the `query!` macro, the macro code does the following:

1. Reads `DATABASE_URL` from the environment (or from `.env`) and opens a connection to a development database that has your migrations applied.
2. Sends the SQL text in a PostgreSQL `Parse` message followed by `Describe`. This is the same extended-protocol handshake used for prepared statements. PostgreSQL parses and plans the statement without running it. If the SQL has a syntax error or names a column that does not exist, the server replies with an error and the macro turns it into a *compile error* pointing at your source line.
3. Reads the reply: a `ParameterDescription` listing the type OID of each `$n` placeholder, and a `RowDescription` listing each output column's name and type OID.
4. Works out nullability. For a column read directly from a table, it looks up `attnotnull` in `pg_attribute`. For expressions and joins, it inspects the query plan (via `EXPLAIN`) to see whether an outer join could produce `NULL`s.

5. Generates an anonymous struct with one field per column, typed by a fixed mapping: `BIGINT` becomes `i64`, `UUID` becomes `uuid::Uuid`, `BOOLEAN` becomes `bool`, `TIMESTAMPTZ` becomes a `time` or `chrono` timestamp, `NUMERIC` becomes `Decimal`, `JSONB` becomes `serde_json::Value`. A `NOT NULL` column becomes `T`; a nullable one becomes `Option<T>`.
6. Checks that each Rust argument you passed can be encoded as the type PostgreSQL expects for its placeholder. Passing a `String` where `$1` is a `UUID` is a compile error.

The nullability rule is where SQLx earns the most trust in a ledger. The `response_body` column of `idempotency_keys` is nullable (it stays `NULL` until the request completes), so SQLx types it as an `Option` wrapping a JSON value. You cannot forget the `NULL` case, because the compiler will not let you treat an `Option` as a value. Compare JDBC, where `getLong` on a `NULL` column quietly returns `0` and you must remember to call `wasNull()` afterwards.

**Common trap**

**`SUM` of a `BIGINT` column is not a `BIGINT`.** PostgreSQL widens `SUM(bigint)` to `NUMERIC` so the total cannot overflow, and it returns `NULL` when there are no rows. So

```
SELECT SUM(amount_minor) AS total FROM postings WHERE account_id = $1
```

gives you `Option<Decimal>`, not `i64`. SQLx is reporting the truth here, and the fix is to say what you mean: `COALESCE(SUM(amount_minor), 0)::BIGINT AS "total!"`. The cast restores `BIGINT` (and will raise an error rather than wrap if the sum ever exceeds the `i64` range), and the `!` suffix in the alias tells SQLx to treat the column as non-null. The opposite override, `"name?"`, forces a column to `Option<T>`; and `"kind: AccountKind"` tells SQLx to decode into your own type, here a Rust enum declared with `#[derive(sqlx::Type)]` that matches the PostgreSQL `account_kind` enum.

### 14.2.2 Offline Mode: Building Without a Database

Requiring a live database for every build would make CI painful. SQLx solves this with offline metadata. Running `cargo sqlx prepare` against your development database writes one JSON file per query into a `.sqlx/` directory, keyed by a hash of the query text. Each file records the parameter types, column types, and nullability the server reported. You commit `.sqlx/` to version control. In CI, setting `SQLX_OFFLINE=true` makes the macros read those files instead of connecting anywhere. If someone edits a query without re-running `prepare`, the hash no longer matches, and the build fails with a message telling them exactly that. A CI step running `cargo sqlx prepare --check` catches a stale `.sqlx/` directory before merge.

### 14.2.3 Connection Pools and the Settings That Protect You

A PostgreSQL connection is not a lightweight handle. Each one is a separate operating-system process on the database server with its own memory (several megabytes once it has run a few queries), and opening one costs a TCP handshake, TLS negotiation, and authentication. So we open a fixed set once and share them through a `PgPool`.

Listing 14.4: Configuring a PgPool with explicit limits and per-session safety timeouts.

```
use sqlx::postgres::{PgConnectOptions, PgPoolOptions};
```

```rust
use sqlx::Executor;
use std::time::Duration;

pub async fn build_pool(opts: PgConnectOptions) -> Result<PgPool, sqlx::Error> {
    PgPoolOptions::new()
        .max_connections(32)
        .min_connections(4)
        .acquire_timeout(Duration::from_millis(500))
        .idle_timeout(Duration::from_secs(300))
        .max_lifetime(Duration::from_secs(30 * 60))
        .after_connect(|conn, _meta| {
            Box::pin(async move {
                conn.execute(
                    "SET statement_timeout = '5s';
                     SET lock_timeout = '2s';
                     SET idle_in_transaction_session_timeout = '10s';",
                )
                .await?;
                Ok(())
            })
        })
        .connect_with(opts)
        .await
}
```

Each setting answers a specific failure:

- `max_connections(32).` More connections do not mean more throughput. PostgreSQL does real work on CPU cores and disks; past a few connections per core, extra connections just contend for locks and caches. Recall Little's Law from Chapter 12: with transactions holding a connection for 4 ms, 32 connections sustain about 8,000 transactions per second. If you run 10 service replicas, the database sees 320 connections, which is why large deployments put PgBouncer in front.
- `acquire_timeout(500 ms).` When all 32 connections are busy, a new request waits in the pool's queue. Without a timeout, a slow database turns into an unbounded pile of waiting futures. With one, the request fails fast and the handler can return `503` with `Retry-After`.
- `max_lifetime.` Recycling connections every 30 minutes lets the pool follow DNS changes after a failover and releases memory that long-lived server processes accumulate.
- `statement_timeout, lock_timeout.` The database will cancel any statement running longer than 5 seconds and any lock wait longer than 2 seconds. A bad query plan or a stuck lock becomes an error you can see instead of a hung request.
- `idle_in_transaction_session_timeout.` If a transaction is open but idle for 10 seconds (for instance, because application code is waiting on something it should not be), PostgreSQL terminates the session and rolls the transaction back, releasing every row lock it held.

### 14.2.4 `Transaction` as an RAII Guard

In Chapter 12 we saw that a dropped future runs the destructors of everything it holds. SQLx's `Transaction` type is built for exactly that situation.

Listing 14.5: The lifecycle of an SQLx transaction: commit consumes it, every other exit rolls it back.

```
let mut tx: sqlx::Transaction<'_, Postgres> = pool.begin().await?; // BEGIN

sqlx::query!("UPDATE accounts SET version = version + 1 WHERE id = $1", id)
    .execute(&mut *tx)     // runs on the transaction's own connection
    .await?;               // an Err here returns early: tx is dropped

tx.commit().await?;        // COMMIT; takes `tx` by value
```

Three paths lead out of this code, and only one of them commits:

1. `tx.commit()`. It takes `self` by value, so after a successful commit the variable `tx` no longer exists. You cannot accidentally run another statement on a finished transaction; the compiler reports "use of moved value".
2. **An early return through `?`, or a panic.** `tx` goes out of scope without being committed, and its `Drop` implementation runs.
3. **The handler future is dropped** because the client disconnected or a timeout fired. Every live local, including `tx`, is dropped where it stands.

In cases 2 and 3, `Drop` cannot `.await` — destructors in Rust are synchronous — so it cannot send `ROLLBACK` and wait for the reply. Instead SQLx marks the connection as having an open transaction and returns it to the pool; the pool sends `ROLLBACK` before handing that connection to anyone else. If the process dies outright, the TCP connection closes and PostgreSQL rolls back the transaction itself. In every case the outcome is the same: none of the transaction's writes become visible. That is the property we needed at the end of Chapter 12.

## 14.3 Two Withdrawals, One Balance: Concurrency Done Right

Alice's wallet holds $100.00 (`10_000` cents). She taps "Pay $80" on her phone and, because the screen seemed frozen, taps it again on her laptop. Two requests with different idempotency keys arrive at two different service replicas within the same millisecond. Exactly one should succeed.

### 14.3.1 The Naive Version and Its Race

Here is the code most people write first. It reads the balance, checks it in Rust, and writes the result back.

Listing 14.6: BROKEN: read, check in application code, then write.

```
let mut tx = pool.begin().await?;              // READ COMMITTED by default

let bal = sqlx::query_scalar!(
    "SELECT balance_minor FROM accounts WHERE id = $1", alice
).fetch_one(&mut *tx).await?;

if bal < amount {
    return Err(LedgerError::InsufficientFunds);
```

```
}

sqlx::query!(
    "UPDATE accounts SET balance_minor = $1 WHERE id = $2",
    bal - amount, alice                          // writes a value computed in Rust
).execute(&mut *tx).await?;
// ... insert postings ...
tx.commit().await?;
```

Follow the two transactions step by step (the left half of Figure 14.2):

1. $T_1$ reads `balance_minor = 10000`.
2. $T_2$ reads `balance_minor = 10000`. Plain `SELECT` takes no row lock, and under `READ COMMITTED` each statement sees all data committed before it began. $T_1$ has not committed, so $T_2$ sees the old value.
3. $T_1$ checks $10{,}000 \geq 8{,}000$ in Rust: OK. It writes `balance_minor = 2000` and inserts its postings.
4. $T_2$ checks $10{,}000 \geq 8{,}000$ in Rust: OK, based on a value that is about to become stale. It tries to write `balance_minor = 2000` and blocks, because $T_1$ holds the row lock taken by its `UPDATE`.
5. $T_1$ commits. $T_2$'s `UPDATE` proceeds and writes `2000` again. $T_2$ commits.

The postings table now shows \$160 leaving a \$100 wallet, but the balance column says \$20. This is a *lost update*: $T_2$ overwrote $T_1$'s write with a value computed from a stale read. The projection and the postings disagree, and Alice has spent money she did not have.

What if we write `SET balance_minor = balance_minor - $1` instead of a precomputed value? Then step 5 changes. When $T_2$'s `UPDATE` unblocks, PostgreSQL notices that the row it was waiting for has been updated, and re-evaluates the statement against the newest committed version of the row (this is called EvalPlanQual re-checking). $T_2$ computes $2{,}000 - 8{,}000 = -6{,}000$. Our `balance_floor` constraint rejects it, so $T_2$ fails with a `check_violation`. The constraint saved us, but the Rust-side check `bal < amount` was useless: it looked at a number that was already out of date by the time it mattered. This is what $T_2$ receives:

```
ERROR:  new row for relation "accounts" violates check constraint "balance_floor"
DETAIL: Failing row contains (0190f1c2-..., ..., liability, USD, f, -6000, ...).
SQLSTATE: 23514
```

In SQLx it arrives as `sqlx::Error::Database`, and calling `.code()` on the inner error returns `"23514"`. In our finished service this path should never fire, because `commit_entry` (below) checks the floor itself while holding the row lock. If it ever does fire, the error falls through to a `500` and an alert, because it means the application and the schema disagree about the rules.

### 14.3.2 Fix 1: A Single Conditional Statement

For a single-account debit, the cleanest fix moves the check into the same statement as the write:

```
UPDATE accounts
```

```
   SET balance_minor = balance_minor - $1, version = version + 1
 WHERE id = $2
   AND (allow_negative OR balance_minor >= $1)
RETURNING balance_minor;
```

If $T_2$ blocks behind $T_1$, PostgreSQL re-checks the whole `WHERE` clause against the row $T_1$ committed. The condition $2{,}000 \geq 8{,}000$ is false, so the statement updates zero rows and returns nothing. The Rust code sees an empty result and returns `InsufficientFunds`. No stale read exists anywhere, because there is no separate read.

### 14.3.3 Fix 2: Lock Every Touched Account, in Sorted Order

A real journal entry touches several accounts: Alice's wallet, the merchant's wallet, and the platform fee account. We need to read all their kinds, currencies, and balances, apply every posting with checked arithmetic, and write the results back, without any other transaction changing those rows underneath us. `SELECT ... FOR UPDATE` does this: it reads the rows and takes an exclusive row lock on each one, held until commit or rollback. Any other transaction that tries to lock or update those rows waits.

That raises a new hazard. Suppose $T_1$ transfers from account $A$ to account $B$, and $T_2$ transfers from $B$ to $A$, and each locks the accounts in the order it mentions them:

| **Time** | $T_1$ $(A \to B)$ | $T_2$ $(B \to A)$ |
|---|---|---|
| $t_1$ | locks $A$ | locks $B$ |
| $t_2$ | waits for $B$ (held by $T_2$) | waits for $A$ (held by $T_1$) |

Each waits for the other forever. This is a deadlock. PostgreSQL detects it after `deadlock_timeout` (one second by default), picks one transaction, and aborts it with error code `40P01`. Nothing is corrupted, but one customer waited a full second for an error that looks like this:

```
ERROR:  deadlock detected
DETAIL: Process 41207 waits for ShareLock on transaction 918344; blocked by process 41211.
        Process 41211 waits for ShareLock on transaction 918343; blocked by process 41207.
HINT:   See server log for query details.
SQLSTATE: 40P01
```

The standard fix is simple: **every transaction acquires locks in the same global order.** If both $T_1$ and $T_2$ lock the lower account ID first, a cycle is impossible. Whichever transaction gets the first lock also gets the second, and the other waits in line. Our `AccountId` derives `Ord` (Chapter 13) for this reason, and the SQL makes the order explicit:

```
SELECT id, kind, currency, allow_negative, balance_minor
  FROM accounts
 WHERE id = ANY($1)
 ORDER BY id
   FOR UPDATE;
```

In the query plan, the row-locking step sits above the sort, so PostgreSQL locks rows in the order they come out of `ORDER BY id`. We also sort and de-duplicate the IDs in Rust before sending them, so the code states the rule as well.

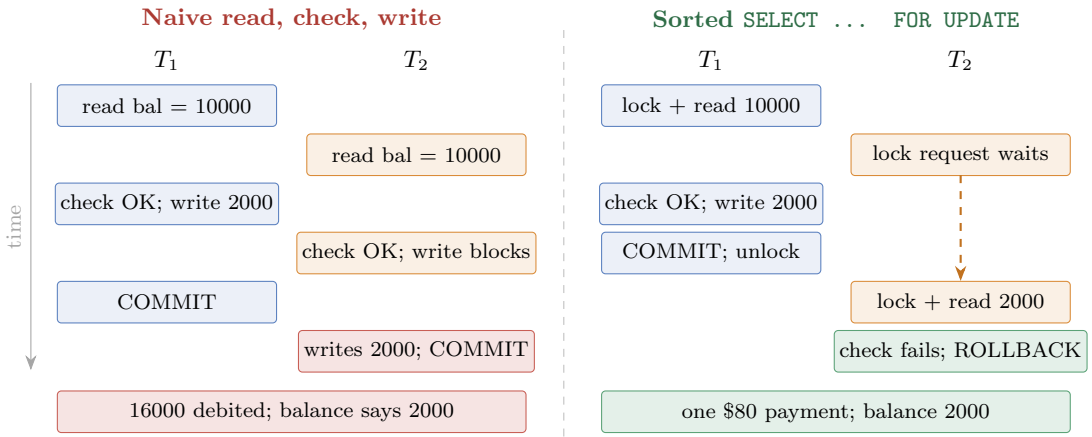



Figure 14.2: Two concurrent $80 debits against a $100 balance. On the left, both transactions read the same balance before either writes, so both checks pass and the second write erases the first. On the right, $T_2$'s lock request waits until $T_1$ commits, then reads the true balance of $20 and is rejected.

### 14.3.4 The Repository Function: `commit_entry`

Now we can write the function that turns a `BalancedEntry` into rows. Its signature is the payoff of Chapter 13: it accepts only a sealed entry, so it cannot be handed an unbalanced one.

Listing 14.7: Persisting a BalancedEntry: canonical locking, checked balance math, batch inserts, and an outbox event in one transaction.

```
#[derive(Debug, thiserror::Error)]
pub enum LedgerError {
    #[error("unknown account {0}")]
    UnknownAccount(Uuid),
    #[error("account {0} currency does not match entry")]
    CurrencyMismatch(Uuid),
    #[error("insufficient funds in account {0}")]
    InsufficientFunds(Uuid),
    #[error("balance arithmetic overflow")]
    Overflow,
    #[error(transparent)]
    Db(#[from] sqlx::Error),
}

struct LockedAccount {
    id: Uuid,
    kind: AccountKind,
    currency: String,
    allow_negative: bool,
    balance_minor: i64,
}

pub async fn commit_entry(
    tx: &mut sqlx::Transaction<'_, Postgres>,
    tenant: TenantId,
    entry: &BalancedEntry,
```

```
) -> Result<(), LedgerError> {
    // 1. Canonical lock order: sorted, de-duplicated account ids.
    let mut ids: Vec<Uuid> = entry
        .postings()
        .iter()
        .map(|p| p.account_id().as_uuid())
        .collect();
    ids.sort();
    ids.dedup();

    let rows = sqlx::query_as!(
        LockedAccount,
        r#"SELECT id, kind AS "kind: AccountKind", currency,
                  allow_negative, balance_minor
             FROM accounts
            WHERE id = ANY($1) AND tenant_id = $2
            ORDER BY id
              FOR UPDATE"#,
        &ids[..],
        tenant.as_uuid()
    )
    .fetch_all(&mut **tx)
    .await?;

    let mut accounts: HashMap<Uuid, LockedAccount> =
        rows.into_iter().map(|r| (r.id, r)).collect();

    // 2. Apply every posting in memory with checked arithmetic.
    for p in entry.postings() {
        let id = p.account_id().as_uuid();
        let acct = accounts
            .get_mut(&id)
            .ok_or(LedgerError::UnknownAccount(id))?;
        if acct.currency != entry.currency().as_str() {
            return Err(LedgerError::CurrencyMismatch(id));
        }
        let delta = p.side().signed_delta(acct.kind, p.amount().minor());
        acct.balance_minor = acct
            .balance_minor
            .checked_add(delta)
            .ok_or(LedgerError::Overflow)?;
    }
    for acct in accounts.values() {
        if !acct.allow_negative && acct.balance_minor < 0 {
            return Err(LedgerError::InsufficientFunds(acct.id));
        }
    }

    // 3. Append the immutable journal rows.
    sqlx::query!(
        "INSERT INTO journal_entries (id, tenant_id, currency, description)
         VALUES ($1, $2, $3, $4)",
        entry.id().as_uuid(),
        tenant.as_uuid(),
```

```
        entry.currency().as_str(),
        entry.description()
    )
    .execute(&mut **tx)
    .await?;

    let acct_ids: Vec<Uuid> =
        entry.postings().iter().map(|p| p.account_id().as_uuid()).collect();
    let sides: Vec<String> = entry
        .postings()
        .iter()
        .map(|p| match p.side() {
            EntrySide::Debit => "D".to_owned(),
            EntrySide::Credit => "C".to_owned(),
        })
        .collect();
    let amounts: Vec<i64> =
        entry.postings().iter().map(|p| p.amount().minor()).collect();

    sqlx::query!(
        "INSERT INTO postings (entry_id, account_id, side, amount_minor, currency)
         SELECT $1, t.account_id, t.side, t.amount, $5
           FROM UNNEST($2::uuid[], $3::text[], $4::int8[])
                AS t(account_id, side, amount)",
        entry.id().as_uuid(),
        &acct_ids[..],
        &sides[..],
        &amounts[..],
        entry.currency().as_str()
    )
    .execute(&mut **tx)
    .await?;

    // 4. Write back the balance projections in one statement.
    let (upd_ids, upd_bals): (Vec<Uuid>, Vec<i64>) =
        accounts.values().map(|a| (a.id, a.balance_minor)).unzip();
    sqlx::query!(
        "UPDATE accounts AS a
            SET balance_minor = v.bal, version = a.version + 1
           FROM UNNEST($1::uuid[], $2::int8[]) AS v(id, bal)
          WHERE a.id = v.id",
        &upd_ids[..],
        &upd_bals[..]
    )
    .execute(&mut **tx)
    .await?;

    // 5. Record the event for other systems, in the same transaction.
    sqlx::query!(
        "INSERT INTO outbox_events (aggregate_id, event_type, payload)
         VALUES ($1, 'ledger.entry_committed', $2)",
        entry.id().as_uuid(),
        serde_json::json!({ "entry_id": entry.id().as_uuid() })
    )
```

```
    .execute(&mut **tx)
    .await?;
    sqlx::query("NOTIFY outbox").execute(&mut **tx).await?;

    Ok(())
}
```

A few details are worth slowing down for.

- **`&mut **tx`.** The parameter `tx` is a `&mut Transaction`. One `*` reaches the `Transaction`; the second goes through its `DerefMut` to the underlying `PgConnection`, which is what SQLx accepts as an executor. Every statement runs on the same connection, inside the same transaction.
- **Locks are held, not just taken.** Between step 1 and `COMMIT`, no other transaction can change any of these balances. Steps 2 through 5 therefore work with numbers that stay true until commit.
- `UNNEST` turns parallel arrays into a set of rows, so inserting a three-leg entry costs one round trip rather than three.
- `NOTIFY` is transactional: PostgreSQL holds the notification and delivers it only if the transaction commits. A rolled-back transfer never wakes the outbox worker.
- **The fee account is a hot row.** Every transfer that charges a fee locks the platform's single fee-revenue account. At high volume, that one row serializes all transfers. A common remedy is to split it into $N$ shard accounts (`fee_revenue_00` through `fee_revenue_15`) and pick one by hashing the entry ID; reports sum the shards.

### 14.3.5 Fix 3: `SERIALIZABLE` and the Retry Loop

PostgreSQL also offers the strongest isolation level, `SERIALIZABLE`. Under it, you write plain reads and writes with no `FOR UPDATE`, and PostgreSQL guarantees that the result equals *some* one-at-a-time ordering of the committed transactions. It does this with Serializable Snapshot Isolation: it tracks which transactions read data that others later wrote, and when it finds a pattern that could not occur in any serial order, it aborts one of the transactions with error code `40001` (`serialization_failure`).

That shifts work onto the application: **every serializable transaction must be retryable**. You re-run the whole transaction from `BEGIN`, because its earlier reads are no longer valid. In Rust, the cleanest shape is a loop around a function that performs one complete attempt:

Listing 14.8: Retrying a whole transaction on serialization failures and deadlocks with jittered backoff.

```
pub trait Retryable {
    fn is_retryable(&self) -> bool;
}

/// 40001 = serialization_failure, 40P01 = deadlock_detected.
pub fn sqlstate_is_retryable(err: &sqlx::Error) -> bool {
    match err {
        sqlx::Error::Database(db) => {
            matches!(db.code().as_deref(), Some("40001") | Some("40P01"))
```

```
        }
        _ => false,
    }
}

impl Retryable for LedgerError {
    fn is_retryable(&self) -> bool {
        matches!(self, LedgerError::Db(e) if sqlstate_is_retryable(e))
    }
}

pub async fn with_retries<T, E, F, Fut>(mut attempt: F) -> Result<T, E>
where
    E: Retryable,
    F: FnMut() -> Fut,
    Fut: std::future::Future<Output = Result<T, E>>,
{
    let mut delay_ms = 5u64;
    for n in 1..=5 {
        match attempt().await {
            Err(e) if e.is_retryable() && n < 5 => {
                let jitter = rand::random::<u64>() % delay_ms;
                tokio::time::sleep(Duration::from_millis(delay_ms + jitter)).await;
                delay_ms *= 2;
            }
            other => return other,
        }
    }
    unreachable!("the final attempt always returns")
}
```

Each call to `attempt()` opens a fresh transaction (for example with `pool.begin()` followed by `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`), so a retry never reuses a doomed one. The helper is generic over the error type: any error that implements the small `Retryable` trait can use it, which is how the HTTP handler later reuses it with its own error type. Note that `40P01` is in the list too: even with sorted locking, a deadlock can arise from other code paths, and retrying is the correct response.

Which approach should a ledger use? Both are correct; they fail differently under load.

| | **Sorted `FOR UPDATE`** | **`SERIALIZABLE` + retry** |
|---|---|---|
| Under contention | Transactions queue and wait; latency rises smoothly | Transactions abort and retry; wasted work rises |
| Code burden | Must lock every row you rely on, in order | Must make every transaction safely rerunnable |
| Protects against | Exactly the rows you locked | Any anomaly, including ones you did not foresee |
| Typical use | Hot-path money movement | Complex reads and rules across many tables |

Our transfer path uses sorted `FOR UPDATE` under `READ COMMITTED`, because the set of rows it depends on is known exactly, and we still wrap it in `with_retries` to absorb the occasional `40P01`.

## 14.4 Idempotency Keys: Safe Retries Over an Unreliable Network

Locks stop two *different* payments from overdrawing an account. They do nothing about the *same* payment arriving twice. And over a real network, the same payment arrives twice all the time. Picture this sequence:

1. The client sends "transfer $80."
2. Our service commits the entry and starts writing the `201 Created` response.
3. A mobile network handover drops the TCP connection. The client never sees the response.
4. From the client's side, the outcome is unknown. It does the reasonable thing and retries.

Without protection, step 4 moves another $80. The client cannot avoid this: it simply has no way to tell "the request never arrived" apart from "the response never arrived."

The fix, popularized by Stripe's API, is the *idempotency key*. The client generates a unique key per logical operation (a UUID is fine) and sends it in a header on the first attempt and on every retry. The server promises: **for a given key, the operation happens at most once, and every retry receives the same response as the first successful attempt.**

### 14.4.1 The Protocol, Step by Step

We store each key in `idempotency_keys` together with a SHA-256 hash of the request, the response we sent, and the entry we created. All of it happens *inside the same database transaction* as the ledger write. That single decision is what makes the scheme correct: the stored response and the money movement commit together or not at all. There is no window in which the money moved but the key was not recorded, or the key was recorded but the money did not move.

Listing 14.9: Executing a transfer exactly once per idempotency key, inside one transaction.

```rust
pub enum TransferOutcome {
    Fresh(StoredResponse),    // we executed it just now
    Replayed(StoredResponse), // we executed it earlier; same answer
}

pub async fn execute_once(
    pool: &PgPool,
    tenant: TenantId,
    key: &IdempotencyKey,
    request_hash: [u8; 32],
    entry: &BalancedEntry,
) -> Result<TransferOutcome, ApiError> {
    let mut tx = pool.begin().await?;

    // 1. Claim the key. If another transaction holds an uncommitted claim
    //    on the same key, this INSERT waits until that transaction ends.
    let claimed = sqlx::query_scalar!(
        "INSERT INTO idempotency_keys (tenant_id, key, request_hash, status)
         VALUES ($1, $2, $3, 'in_progress')
         ON CONFLICT (tenant_id, key) DO NOTHING
         RETURNING true AS \"claimed!\"",
```

```
        tenant.as_uuid(),
        key.as_str(),
        &request_hash[..]
    )
    .fetch_optional(&mut *tx)
    .await?
    .is_some();

    if !claimed {
        // 2. The key already exists and is committed. Replay or reject.
        let prior = sqlx::query!(
            "SELECT request_hash, status, response_status, response_body
               FROM idempotency_keys
              WHERE tenant_id = $1 AND key = $2",
            tenant.as_uuid(),
            key.as_str()
        )
        .fetch_one(&mut *tx)
        .await?;

        if prior.request_hash != request_hash {
            return Err(ApiError::IdempotencyKeyReused);
        }
        return match (prior.status.as_str(), prior.response_status, prior.response_body) {
            ("completed", Some(status), Some(body)) => {
                Ok(TransferOutcome::Replayed(StoredResponse { status, body }))
            }
            _ => Err(ApiError::RequestInProgress),
        };
    }

    // 3. First time we see this key: move the money.
    ledger::commit_entry(&mut tx, tenant, entry).await?;

    let response = StoredResponse::created(entry);
    sqlx::query!(
        "UPDATE idempotency_keys
            SET status = 'completed', response_status = $3,
                response_body = $4, entry_id = $5
          WHERE tenant_id = $1 AND key = $2",
        tenant.as_uuid(),
        key.as_str(),
        response.status,
        &response.body,
        entry.id().as_uuid()
    )
    .execute(&mut *tx)
    .await?;

    // 4. One COMMIT makes the postings, balances, outbox row,
    //    and stored response durable together.
    tx.commit().await?;
    Ok(TransferOutcome::Fresh(response))
}
```

Walk through the cases this handles.

- **First request.** The `INSERT` claims the key, the transfer runs, the response is stored, and everything commits together.
- **Retry after a lost response.** The key exists with status `completed`. The hash matches, so we return the stored body and status. The client sees exactly what the first attempt would have shown it, and no money moves.
- **Two copies racing.** Both try to insert the same primary key. PostgreSQL makes the second `INSERT` wait on the first transaction's uncommitted row. If the first commits, the second sees a conflict and replays the stored response. If the first rolls back (for example, insufficient funds), the second's insert succeeds and it runs the transfer itself, which is correct because the first one left no trace. This relies on `READ COMMITTED`: the follow-up `SELECT` is a new statement with a fresh snapshot, so it sees the row the other transaction just committed.
- **Same key, different request.** A client bug reuses a key for a different amount. The hashes differ, and we return `422 Unprocessable Entity` rather than replaying an answer to a question the client did not ask.
- **Failure mid-transfer.** If `commit_entry` returns an error, or the handler future is dropped, the transaction rolls back and the key claim disappears with it. A later retry starts clean.

Intuition

**Why have an `in_progress` status at all?** In the code above, an `in_progress` row is never visible to anyone else, because it only exists inside an uncommitted transaction. The status earns its place when a request includes a side effect *outside* the database, such as calling a card network. You cannot hold a database transaction open across a two-second HTTP call to a third party (it would hold row locks and trip `idle_in_transaction_session_timeout`). So you commit the `in_progress` claim first, make the external call, and then complete the record in a second transaction. A retry that arrives in between sees `in_progress` and gets `409 Conflict`, telling the client to wait and try again. Stripe describes this as splitting a request into atomic phases separated by recovery points.

The request hash must be computed over a *canonical* form of the request: the method, the path, and the parsed DTO serialized with a fixed field order. Hashing the raw body bytes would make two semantically identical requests (differing only in whitespace or key order) look different. Keys are scoped by tenant, so two customers can never collide, and a periodic job deletes keys older than 24 hours, which is the replay window we promise in the API documentation.

## 14.5 Axum and Tower: From Bytes to a Typed Handler

We now have a function that moves money exactly once. The last layer turns HTTP requests into calls to it and turns its results back into HTTP responses. Axum does this with ordinary Rust traits, and once you see those traits, its behavior stops being surprising.

### 14.5.1 Tower Middleware: Services Wrapping Services

Tower is a small library built around one trait:

```rust
pub trait Service<Request> {
    type Response;
    type Error;
    type Future: Future<Output = Result<Self::Response, Self::Error>>;

    fn poll_ready(&mut self, cx: &mut Context<'_>) -> Poll<Result<(), Self::Error>>;
    fn call(&mut self, req: Request) -> Self::Future;
}
```

A `Service` is an asynchronous function from a request to a response. `poll_ready` lets a service say "not now" (for example, because a concurrency limit is reached), which is how Tower expresses backpressure. A *layer* is a function that takes one service and returns another service wrapping it. Timeouts, request IDs, tracing, body-size limits, and load shedding are all layers. The innermost service is your Axum router. Figure 14.3 shows a request passing inward through the layers to the handler, and the response passing back out.

If you know Java servlet filters, this is the same idea with one difference: each layer's type is known at compile time, so the whole stack is monomorphized into direct calls (Chapter 6) with no virtual dispatch between layers.

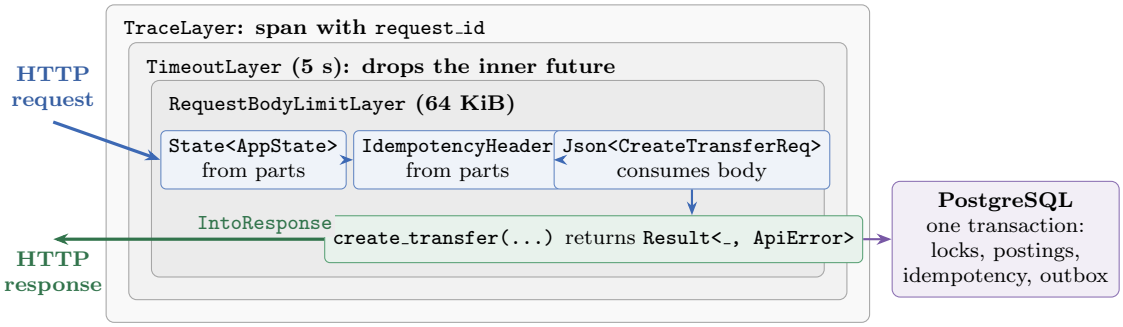



Figure 14.3: The path of one request. Tower layers wrap the router from the outside in. Inside, Axum runs each extractor in argument order; only the last may consume the body. The handler's `Result` becomes an HTTP response through `IntoResponse`. If the timeout layer fires, it drops the inner future, and with it any open transaction.

### 14.5.2 How Axum Calls a Plain `async fn`

Here is our handler. It is an ordinary `async fn`: no annotations, no base class, no reflection.

Listing 14.10: The transfer handler. Each argument is an extractor; the return type implements IntoResponse.

```rust
#[derive(Clone)]
pub struct AppState {
    pub pool: PgPool,
    pub fee_accounts: Arc<FeeAccountShards>,
}

#[derive(Debug, serde::Deserialize, serde::Serialize)]
#[serde(deny_unknown_fields)]
```

```rust
pub struct CreateTransferReq {
    pub from: Uuid,
    pub to: Uuid,
    pub amount_minor: i64,
    pub currency: String,
    pub fee_bps: u32,
}

#[tracing::instrument(
    skip_all,
    fields(tenant = %auth.tenant, idempotency_key = %key.0.as_str())
)]
pub async fn create_transfer(
    State(state): State<AppState>,
    auth: Authenticated,
    key: IdempotencyHeader,
    Json(req): Json<CreateTransferReq>,
) -> Result<(StatusCode, Json<serde_json::Value>), ApiError> {
    // Parse, don't validate: untrusted DTO -> domain types (Chapter 13).
    let currency = Currency::parse(&req.currency) // Option<Currency>
        .ok_or(ApiError::UnsupportedCurrency)?;
    let gross = PositiveMoney::try_new(req.amount_minor, currency)
        .map_err(LedgerDomainError::from)?;
    let entry = build_peer_transfer_entry(
        key.0.clone(),
        AccountId::from_uuid(req.from),
        AccountId::from_uuid(req.to),
        state.fee_accounts.pick(currency),
        gross,
        req.fee_bps,
    )?;

    let hash = canonical_request_hash("POST", "/v1/transfers", &req);
    let outcome = with_retries(|| {
        execute_once(&state.pool, auth.tenant, &key.0, hash, &entry)
    })
    .await?;

    let stored = match outcome {
        TransferOutcome::Fresh(s) | TransferOutcome::Replayed(s) => s,
    };
    let status = StatusCode::from_u16(stored.status as u16)
        .unwrap_or(StatusCode::OK);
    Ok((status, Json(stored.body)))
}
```

How can a router accept functions with any number of differently typed arguments? Through a trait that Axum implements, by macro, for every function shape from zero to sixteen arguments. The three-argument version looks roughly like this:

Listing 14.11: A simplified view of Axum's Handler implementation for three-argument functions.

```rust
impl<F, Fut, S, Res, M, T1, T2, T3> Handler<(M, T1, T2, T3), S> for F
where
    F: FnOnce(T1, T2, T3) -> Fut + Clone + Send + 'static,
```

```
    Fut: Future<Output = Res> + Send,
    S: Send + Sync + 'static,
    Res: IntoResponse,
    T1: FromRequestParts<S> + Send,   // may read headers, URI, state
    T2: FromRequestParts<S> + Send,   // may read headers, URI, state
    T3: FromRequest<S, M> + Send,     // may consume the body
{
    // call(): split the request into parts and body, run T1 and T2
    // against the parts, run T3 against parts + body, await F, then
    // convert the output with IntoResponse.
}
```

Read the bounds as the rules of the game:

- **Every argument except the last implements `FromRequestParts`.** These extractors see only the request line, headers, and extensions. They cannot touch the body.
- **Only the last argument may implement `FromRequest`**, which receives the body. A request body is a stream that can be read once, so only one extractor may consume it, and it must come last. If you put `Json<T>` first, the bounds are not satisfied and you get a compile error, usually phrased as "the trait `Handler<_, _>` is not implemented." Adding `#[axum::debug_handler]` to the function makes the error point at the offending argument.
- **`Fut: Send`.** Tokio may move the handler's future between worker threads at any `.await`, so the future must be `Send`. This is where the `MutexGuard` error from Chapter 12 shows up in practice.
- **`Res: IntoResponse`.** Anything that can turn itself into an HTTP response can be returned. Tuples like `(StatusCode, Json<T>)` implement it, and so does `Result<T, E>` when both `T` and `E` do.

If any extractor fails, Axum does not call your function at all; it converts the extractor's *rejection* into a response. A malformed JSON body produces `400` or `422` before a single line of your handler runs.

### 14.5.3 Writing Our Own Extractor

The idempotency key arrives in a header. We want handlers to receive the validated key type from Chapter 13, never a raw string. A custom extractor does that parsing once, for every handler that asks for it:

Listing 14.12: A custom FromRequestParts extractor that parses and validates the Idempotency-Key header.

```
pub struct IdempotencyHeader(pub IdempotencyKey);

impl<S: Send + Sync> FromRequestParts<S> for IdempotencyHeader {
    type Rejection = ApiError;

    async fn from_request_parts(
        parts: &mut axum::http::request::Parts,
        _state: &S,
    ) -> Result<Self, Self::Rejection> {
```

```
        let raw = parts
            .headers
            .get("idempotency-key")
            .ok_or(ApiError::MissingIdempotencyKey)?
            .to_str()
            .map_err(|_| ApiError::InvalidIdempotencyKey)?;
        IdempotencyKey::parse(raw)
            .map(IdempotencyHeader)
            .map_err(|_| ApiError::InvalidIdempotencyKey)
    }
}
```

Now "this endpoint requires a valid idempotency key" is expressed in the handler's signature. A handler that lists `IdempotencyHeader` as an argument cannot run without one. `Authenticated`, which verifies the caller's token and yields a `TenantId`, follows the same pattern.

### 14.5.4 One Error Type, Mapped Once

Every failure in the request path converts into `ApiError`, and `ApiError` converts into an HTTP response in exactly one place.

Listing 14.13: Centralized error mapping. Clients see stable codes; internal details go only to logs.

```
#[derive(Debug, thiserror::Error)]
pub enum ApiError {
    #[error("missing Idempotency-Key header")]
    MissingIdempotencyKey,
    #[error("invalid Idempotency-Key header")]
    InvalidIdempotencyKey,
    #[error("idempotency key was used with a different request")]
    IdempotencyKeyReused,
    #[error("a request with this key is still in progress")]
    RequestInProgress,
    #[error("unsupported currency")]
    UnsupportedCurrency,
    #[error(transparent)]
    Domain(#[from] LedgerDomainError),
    #[error(transparent)]
    Ledger(#[from] LedgerError),
    #[error(transparent)]
    Db(#[from] sqlx::Error),
}

impl Retryable for ApiError {
    fn is_retryable(&self) -> bool {
        match self {
            Self::Db(e) => sqlstate_is_retryable(e),
            Self::Ledger(e) => e.is_retryable(),
            _ => false,
        }
    }
}

```

```rust
impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (status, code) = match &self {
            Self::MissingIdempotencyKey | Self::InvalidIdempotencyKey => {
                (StatusCode::BAD_REQUEST, "invalid_idempotency_key")
            }
            Self::IdempotencyKeyReused => {
                (StatusCode::UNPROCESSABLE_ENTITY, "idempotency_key_reused")
            }
            Self::RequestInProgress => (StatusCode::CONFLICT, "request_in_progress"),
            Self::Domain(_) | Self::UnsupportedCurrency => {
                (StatusCode::UNPROCESSABLE_ENTITY, "invalid_transfer")
            }
            Self::Ledger(LedgerError::InsufficientFunds(_)) => {
                (StatusCode::UNPROCESSABLE_ENTITY, "insufficient_funds")
            }
            Self::Ledger(LedgerError::UnknownAccount(_)) => {
                (StatusCode::NOT_FOUND, "account_not_found")
            }
            Self::Db(sqlx::Error::PoolTimedOut) => {
                (StatusCode::SERVICE_UNAVAILABLE, "overloaded")
            }
            _ => (StatusCode::INTERNAL_SERVER_ERROR, "internal_error"),
        };
        if status.is_server_error() {
            tracing::error!(error = ?self, "request failed");
        }
        (status, Json(serde_json::json!({ "error": code }))).into_response()
    }
}
```

The `#[from]` attributes generate `From` implementations, which is what lets the handler use `?` on calls returning `sqlx::Error`, `LedgerError`, or `LedgerDomainError`. The client sees a stable machine-readable code. The full error, which may contain SQL text or account IDs, goes only to the logs.

### 14.5.5 Tracing Across Threads

In a Java 6 servlet, you might put the request ID in log4j's `MDC`, which is backed by a `ThreadLocal`. That works because one thread runs one request from start to finish. In Tokio it breaks: a task can start on worker 0, suspend at an `.await`, and resume on worker 3, and meanwhile worker 0 runs other requests. A thread-local would attach the wrong request ID to half your log lines.

The `tracing` crate attaches context to the *future* instead of the thread. `#[tracing::instrument]` wraps the handler's future in an `Instrumented` adapter whose `poll` method enters the span, polls the inner future, and exits the span. Whichever thread polls the task, the span is active for exactly as long as the task runs. Every log line emitted inside `create_transfer` or anything it calls carries `request_id`, `tenant`, and `idempotency_key`, and a JSON log formatter makes them searchable fields.

### 14.5.6 Assembling the Application and Shutting Down Gracefully

Listing 14.14: main: router, middleware, outbox worker, and graceful shutdown on SIGTERM.

```rust
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    tracing_subscriber::fmt().json().init();

    let pool = build_pool(PgConnectOptions::from_str(&env::var("DATABASE_URL")?)?)
        .await?;
    sqlx::migrate!("./migrations").run(&pool).await?;

    let state = AppState {
        pool: pool.clone(),
        fee_accounts: Arc::new(FeeAccountShards::load(&pool).await?),
    };

    let (shutdown_tx, shutdown_rx) = tokio::sync::watch::channel(false);
    let outbox = tokio::spawn(run_outbox_worker(pool.clone(), shutdown_rx));

    let app = Router::new()
        .route("/v1/transfers", post(create_transfer))
        .route("/healthz", get(|| async { "ok" }))
        .with_state(state)
        .layer(
            ServiceBuilder::new()
                .layer(SetRequestIdLayer::x_request_id(MakeRequestUuid))
                .layer(TraceLayer::new_for_http().make_span_with(request_span))
                .layer(TimeoutLayer::new(Duration::from_secs(5)))
                .layer(RequestBodyLimitLayer::new(64 * 1024))
                .layer(PropagateRequestIdLayer::x_request_id()),
        );

    let listener = tokio::net::TcpListener::bind("0.0.0.0:8080").await?;
    axum::serve(listener, app)
        .with_graceful_shutdown(shutdown_signal())
        .await?;

    // HTTP side has drained. Stop the outbox worker, then close the pool.
    let _ = shutdown_tx.send(true);
    outbox.await??;
    pool.close().await;
    Ok(())
}

fn request_span(req: &axum::http::Request<axum::body::Body>) -> tracing::Span {
    let id = req
        .headers()
        .get("x-request-id")
        .and_then(|v| v.to_str().ok())
        .unwrap_or("-");
    tracing::info_span!("http", method = %req.method(),
                        path = %req.uri().path(), request_id = %id)
}

async fn shutdown_signal() {
    let ctrl_c = async { tokio::signal::ctrl_c().await.expect("install Ctrl-C") };
```

```
    let term = async {
        tokio::signal::unix::signal(tokio::signal::unix::SignalKind::terminate())
            .expect("install SIGTERM")
            .recv()
            .await;
    };
    tokio::select! { _ = ctrl_c => {}, _ = term => {} }
    tracing::info!("shutdown signal received; draining in-flight requests");
}
```

When Kubernetes wants to replace a pod, it sends `SIGTERM` and waits for a grace period (30 seconds by default) before sending `SIGKILL`. Here is what happens in our service during that window:

1. `shutdown_signal()` completes. `axum::serve` stops accepting new connections.

2. Requests already in progress keep running. Their futures are still polled to completion: a transfer halfway through `commit_entry` finishes its statements, commits, stores its idempotent response, and writes its HTTP reply.

3. Once the last in-flight request finishes, `serve` returns. We tell the outbox worker to stop, wait for it, and close the pool, which waits for connections to be returned and closes them cleanly.

What if a request is still running when `SIGKILL` arrives? The process dies, its TCP connections to PostgreSQL close, and PostgreSQL rolls back the open transaction. The client sees a dropped connection, retries with the same idempotency key against another pod, and either gets the replayed response (if the commit happened) or a fresh execution (if it did not). Every path ends in a consistent state. That is the result of three separate designs fitting together: atomic transactions, RAII rollback, and idempotency keys.

## 14.6 The Transactional Outbox: Telling Other Systems, Reliably

After a transfer commits, other systems need to hear about it: a notification service sends Alice a receipt, a fraud model scores the payment, a data warehouse records it. The obvious approach is to publish a message to Kafka or a webhook right after `tx.commit()` returns. It fails in two directions:

- **Commit succeeds, publish fails.** The broker is briefly unreachable, or the process is killed between the two calls. The money moved and nobody downstream ever hears about it.

- **Publish first, commit fails.** Downstream systems react to a payment that was rolled back.

The underlying problem is that the database and the broker are two separate systems, and no single operation can commit to both.

The *transactional outbox* pattern removes the second system from the critical moment. In step 5 of `commit_entry`, we inserted a row into `outbox_events` *in the same transaction* as the postings. Either both exist or neither does. A separate background worker then reads unpublished outbox rows and delivers them to the broker, as shown in Figure 14.4.

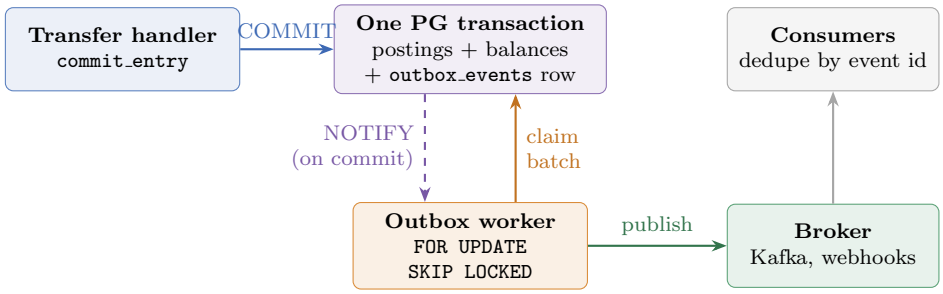



Figure 14.4: The transactional outbox. The ledger write and the outbox row commit atomically. `NOTIFY` wakes the worker promptly, but correctness does not depend on it: the worker also polls. Workers claim rows with `SKIP LOCKED`, publish, and mark them sent. Delivery is at least once, so consumers deduplicate by event ID.

Listing 14.15: An outbox worker using SKIP LOCKED for safe parallelism and LISTEN/NOTIFY for low latency.

```rust
pub async fn run_outbox_worker(
    pool: PgPool,
    mut shutdown: tokio::sync::watch::Receiver<bool>,
) -> anyhow::Result<()> {
    let publisher = Publisher::connect().await?;
    let mut listener = sqlx::postgres::PgListener::connect_with(&pool).await?;
    listener.listen("outbox").await?;

    loop {
        // Drain everything currently pending, one batch at a time.
        while publish_batch(&pool, &publisher).await? > 0 {}

        // Sleep until a NOTIFY, a 5-second safety poll, or shutdown.
        tokio::select! {
            _ = listener.recv() => {}
            _ = tokio::time::sleep(Duration::from_secs(5)) => {}
            _ = shutdown.changed() => break,
        }
    }
    Ok(())
}

async fn publish_batch(pool: &PgPool, publisher: &Publisher) -> anyhow::Result<usize> {
    let mut tx = pool.begin().await?;
    let rows = sqlx::query!(
        "SELECT id, event_type, payload
           FROM outbox_events
          WHERE published_at IS NULL
          ORDER BY id
          LIMIT 100
            FOR UPDATE SKIP LOCKED"
    )
    .fetch_all(&mut *tx)
    .await?;

    for row in &rows {
```

```
        publisher.send(row.id, &row.event_type, &row.payload).await?;
    }

    let ids: Vec<i64> = rows.iter().map(|r| r.id).collect();
    sqlx::query!(
        "UPDATE outbox_events SET published_at = now() WHERE id = ANY($1)",
        &ids[..]
    )
    .execute(&mut *tx)
    .await?;
    tx.commit().await?;
    Ok(rows.len())
}
```

Three PostgreSQL features carry this design.

- `FOR UPDATE SKIP LOCKED`. Each worker locks the rows it selects. If you run four worker replicas, a second worker running the same query *skips* rows locked by the first and takes the next unlocked batch, instead of waiting. The table works as a concurrent job queue without any extra infrastructure, and the partial index `outbox_pending` keeps the scan small.
- `LISTEN` **and** `NOTIFY`. Without notifications, the worker would have to poll frequently to keep latency low. With them, `commit_entry`'s `pg_notify('outbox', '')` wakes the worker within milliseconds of the commit. The 5-second sleep branch is a safety net: if a notification is ever missed (the listener reconnected, say), the next poll finds the rows anyway. Correctness depends only on the table; notifications only affect latency.
- **Transactions around the batch.** If the worker crashes after publishing but before `COMMIT`, the rows are still unpublished in the database and will be published again. Delivery is therefore *at least once*, never *at most once*. Consumers must deduplicate, typically by recording each processed event ID in their own database in the same transaction as their own side effects — the mirror image of our idempotency keys.

Notice also how the worker uses `tokio::select!` with Chapter 12's cancellation rules in mind. The `select!` sits *between* batches, never around `publish_batch`. A shutdown signal can never drop a batch halfway through; it is noticed only after the current batch has committed.

## 14.7 Testing Against a Real Database With `#[sqlx::test]`

Most of the guarantees in this chapter live in PostgreSQL: constraints, row locks, isolation, deferred triggers. Mocking the database would test none of them. SQLx provides a test attribute that gives each test its own real, empty database.

Listing 14.16: Integration tests: concurrent overdraft attempts and idempotent replays against a fresh database per test.

```
#[sqlx::test(migrations = "./migrations")]
async fn concurrent_debits_cannot_overdraw(pool: PgPool) -> anyhow::Result<()> {
    let f = Fixture::seed(&pool, /* alice balance */ 10_000).await?;

    // `transfer` takes `self` by value so each spawned future is 'static.
    let a = tokio::spawn(f.clone().transfer(pool.clone(), 8_000, "key-a"));
```

```
    let b = tokio::spawn(f.clone().transfer(pool.clone(), 8_000, "key-b"));
    let (ra, rb) = (a.await?, b.await?);

    // Exactly one succeeds; the other is rejected for insufficient funds.
    assert_eq!([ra.is_ok(), rb.is_ok()].iter().filter(|ok| **ok).count(), 1);

    let alice = f.balance(&pool, f.alice).await?;
    assert_eq!(alice, 2_000);

    // The conservation law holds across the whole ledger.
    let net = sqlx::query_scalar!(
        r#"SELECT COALESCE(SUM(CASE side WHEN 'D' THEN amount_minor
                                         ELSE -amount_minor END), 0)::BIGINT
           AS "net!" FROM postings"#
    )
    .fetch_one(&pool)
    .await?;
    assert_eq!(net, 0);
    Ok(())
}

#[sqlx::test(migrations = "./migrations")]
async fn retry_with_same_key_replays_response(pool: PgPool) -> anyhow::Result<()> {
    let f = Fixture::seed(&pool, 10_000).await?;

    let first = f.clone().transfer(pool.clone(), 2_500, "same-key").await?;
    let second = f.clone().transfer(pool.clone(), 2_500, "same-key").await?;

    assert_eq!(first.body, second.body);
    let entries = sqlx::query_scalar!(r#"SELECT count(*) AS "n!" FROM journal_entries"#)
        .fetch_one(&pool)
        .await?;
    assert_eq!(entries, 1);
    assert_eq!(f.balance(&pool, f.alice).await?, 7_500);
    Ok(())
}
```

For each test, the attribute connects to the server named in `DATABASE_URL`, creates a new database with a generated name, runs every migration in `./migrations`, passes your test a `PgPool` connected to it, and drops the database when the test passes. Tests therefore run in parallel without seeing each other's rows, and each one exercises the real schema, the real constraints, and the real lock manager.

The first test is worth reading as a specification. It does not check *how* we prevent overdrafts; it checks the outcome a customer and an auditor care about: one payment succeeds, the balance is exactly $20, and debits equal credits across the entire ledger. You could replace sorted `FOR UPDATE` with `SERIALIZABLE` retries and the test would not change. Run it in a loop a few thousand times in CI with `tokio::spawn`ed pairs, and a regression in the locking code shows up as a red build rather than a reconciliation discrepancy weeks later.

Focus

**The whole request, in one paragraph.** Bytes arrive; Tower assigns a request ID, opens a tracing span, starts a 5-second timer, and caps the body size. Axum's extractors parse the tenant, the idempotency key, and the JSON body, each into a validated type, or reject the request before our code runs. The handler converts the DTO into `PositiveMoney` and a sealed `BalancedEntry`. `execute_once` opens one transaction, claims the idempotency key, locks every touched account in ID order, applies checked arithmetic, appends postings, updates balances, writes an outbox row, stores the response, and commits once. A deferred trigger re-checks the balance at commit. Any failure or cancellation along the way rolls everything back. After commit, `NOTIFY` wakes the outbox worker, which publishes at least once. A client retry with the same key gets the same answer and moves no money.

## 14.8 Exercises

**Exercise 14.1** (Reading the Schema as a Specification)**.** For each of the following bugs in application code, name the exact constraint, trigger, or key in this chapter's schema that stops it from being committed, and the PostgreSQL error code the service would see:

1. A refactor accidentally negates `amount_minor` before inserting a posting.
2. A fee calculation bug credits the merchant $97.00 and the fee account $3.20 on a $100.00 debit.
3. A race in a new code path lets a customer wallet reach `-500`.
4. A batch job inserts a posting for an account ID that was never created.

Which of these would *also* be caught at compile time by the Chapter 13 types, and which only by the database?

**Exercise 14.2** (SQLx Types and Nullability)**.** For each query below, state the Rust type SQLx generates for every output column, and fix any query whose result type is not what a ledger developer would want:

```
-- (1)
SELECT balance_minor, allow_negative FROM accounts WHERE id = $1;
-- (2)
SELECT response_body FROM idempotency_keys
 WHERE tenant_id = $1 AND key = $2;
-- (3)
SELECT a.id, SUM(p.amount_minor) AS total
  FROM accounts a LEFT JOIN postings p ON p.account_id = a.id
 GROUP BY a.id;
```

For the third query, explain why `total` is nullable even though `amount_minor` is `NOT NULL`, and why its type is not `i64`.

**Exercise 14.3** (Tracing a Deadlock)**.** Two transactions run concurrently with *unsorted* locking. $T_1$ transfers from $A$ to $B$ with a fee to $F$; $T_2$ transfers from $B$ to $C$ with a fee to $F$. Each locks accounts in the order debit, credit, fee.

1. Find an interleaving that deadlocks and draw the wait-for graph.

2. Show that with `ORDER BY id` locking, no interleaving can deadlock, for any assignment of UUIDs to $A$, $B$, $C$, $F$. (Hint: argue that a cycle in the wait-for graph would require some transaction to wait for a lower-ordered lock while holding a higher-ordered one.)

3. Under sorted locking, $F$ is still locked by every transfer. Estimate the maximum transfer throughput if each transaction holds $F$'s lock for 1.5 ms, and redesign with 16 fee shards. What changes in the nightly reconciliation?

**Exercise 14.4** (Breaking and Fixing Idempotency)**.** A teammate proposes a "simpler" idempotency scheme: check for the key with a `SELECT` at the start of the handler; if absent, run the transfer in its own transaction; after it commits, insert the key and response in a second transaction.

1. Describe a sequence of two concurrent requests with the same key that moves money twice.

2. Describe a crash at a single point that moves money once but causes the client's retry to move it again.

3. Explain how the single-transaction design in `execute_once` rules out both.

**Exercise 14.5** (Cancellation Audit of the Request Path)**.** The `TimeoutLayer` fires at 5 seconds and drops the handler future. For each of the following `.await` points, say what state the database is in if the future is dropped while suspended there, and what a retry with the same key will see:

1. Waiting in `pool.begin()` for a free connection.

2. Waiting for the `INSERT INTO idempotency_keys` to return, because another transaction with the same key is still open.

3. Waiting on `SELECT ... FOR UPDATE` for Alice's row lock.

4. Waiting for `tx.commit()` to return, after PostgreSQL has written the commit record to the WAL but before the reply reaches the service.

Case 4 is the subtle one. Explain why it is the reason idempotency keys exist at all.

**Exercise 14.6** (Outbox Ordering and Delivery)**.** 1. With two outbox workers using `SKIP LOCKED`, can events for the same account be published out of order? Construct an example.

2. Propose a change that preserves per-account ordering while keeping parallelism across accounts. (Consider partitioning work by a hash of `aggregate_id`, or using an advisory lock per partition.)

3. A consumer records processed event IDs in a `processed_events` table. Write the SQL it should run so that handling an event and recording its ID are atomic.

**Exercise 14.7** (A Reversal Endpoint)**.** Design `POST /v1/entries/{id}/reverse`. Your design should include:

1. The domain function that turns a committed entry into a `BalancedEntry` with every side flipped and `reverses_entry_id` set.

2. A database constraint that prevents an entry from being reversed twice, even under concurrent requests with different idempotency keys.

3. What should happen if reversing would push a customer wallet below zero (the recipient has already spent the money), and how a real payment platform handles this case.